

Programación Java

DAM / DAW

Sergi Garcia Barea — CC BY-SA 4.0

Contents

- Inicio
  - 📚 Unidades
  - 📝 Boletines
  - 📄 Licencia
- U01 — Introducción al Pensamiento Computacional
  - 🎯 Objetivo de la unidad
  - 🗺️ Mapa de la unidad
  - 📝 Boletines de la unidad
  - ✅ Criterios de evaluación cubiertos (RA1)
  - 🚪 ¿Por dónde empiezo?
- 01 — Qué es un ordenador y un lenguaje de programación
  - 🧠 1. Un ordenador es extraordinariamente rápido, pero absolutamente tonto
  - 🗣️ 2. ¿Qué es un lenguaje de programación?
  - 🔄 3. El traductor: Compiladores vs. Intérpretes
  - 🔤 4. Conceptos clave que escucharás cada día
  - 💡 Idea clave para recordar
- 02 — Programar no es memorizar código
  - 🦹 La trampa del “Espejo de la Comprensión”
  - ⛰️ La pantalla en blanco es tu hábitat natural
  - 🕳️ Cómo evitar la farsa de “Copiar y Pegar” (Tutorial Hell)
  - 💡 Idea clave para recordar
- 03 — Primero pensamos, después escribimos
  - 🔮 La ilusión de la literalidad (Tu ordenador no es adivino)
  - 🛤️ El camino sagrado del programador
  - 🤖 🧪 Ejercicio práctico: El test del Robot Literal
  - 😈 Los 5 demonios de la lógica
  - 💡 Idea clave para recordar
- 04 — La descomposición
  - 🐘 ¿Cómo comerse un elefante? ¡A bocados!
  - 🏛️ El arte de la descomposición en sub-bloques
  - ⚠️ Las 3 reglas de oro para no morir en la división
  - 💡 Idea clave para recordar

- 05 — Debugging y casos límite
  - 🔴 1. El cambio de chip: El texto rojo es tu amigo
  - 💉 2. Pensar como un programador: Probar los casos límite
  - 💡 Idea clave para recordar
- 06 — El protocolo de paz
  - 🛡️ Los 4 pasos del protocolo
  - 💡 Idea clave para recordar
- 07 — Practicar (menos mirar, más intentar)
  - 🎭 La gran farsa de “Ya lo he entendido”
  - ⏳ 🎯 La Regla del 80/20 para la FP a Distancia
  - 🏋️ Cómo entrenar la “musculatura de programador”
  - 💡 Idea clave para recordar
- 08 — Cómo utilizaremos la IA
  - 🚨 El gran peligro: Pagar a otro para que haga tus flexiones
  - 🏛️ 💬 El Método Pro: La IA como Tutor Socrático
  - 📊 Comparativa de mentalidad
  - 🦹 La Regla del “No te fíes ni de tu sombra” (Alucinaciones)
  - 💡 Idea clave para recordar
- 09 — Mi primer problema (puente a Java)
  - 🥊 El Reto: El Comparador de Números
  - 🧩 Paso 1: Descomposición y Entrenamiento de Datos (Inputs y Outputs)
  - 👣 Paso 2: El Algoritmo en Pseudocódigo (Lógica pura)
  - 💉 Paso 3: El testing de casos límite (Mentalidad de detective)
  - 🌉 Y ahora… ¿Dónde está el puente a Java? ☕
  - 💡 Idea clave para cerrar la Unidad 1
  - 💻 Términos técnicos de Infraestructura
  - 🧠 Términos de lógica y proceso
  - 🦎 Términos de gestión de errores y control
  - 🤖 Términos de trabajo y metodología
- U02 — Introducción a Java y el entorno de trabajo
  - 🎯 Objetivo de la unidad
  - 🗺️ Mapa de la unidad
  - 📝 Boletines de la unidad

    - ✅ Criterios de evaluación cubiertos (RA1)
    - 🚪 ¿Por dónde empiezo?
- 01 — ¿Qué es Java?
    - 🔮 ¿De dónde viene este lenguaje?
    - 📦 ¿Qué es eso del bytecode?
    - ☕ La trilogía del Café: JVM, JRE y JDK
    - 🌍 ¿Por qué sigue vivo Java tantos años después?
    - 🎯 Mini-comprobación
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 02 — Instalación del JDK
    - 🛒 ¿Qué JDK instalo?
    - 🚀 Los primeros pasos
    - 🛠️ El IDE: tu navaja suiza
    - 🏠 Ejemplo guiado: tu primer proyecto en VS Code
    - 🎯 Mini-comprobación
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 03 — Tu primer programa
    - 👋 Hola Mundo, el ritual de iniciación
    - 🔬 Disección de Hola Mundo
    - 🔑 ¿Por qué public static void main(String[] args)?
    - 🚪 El método que no se llama
    - ⭐ Intenta pensar como el código: tú eres la JVM
    - 🎯 Mini-comprobación
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 04 — El depurador
    - 🔮 ¿Qué es depurar?
    - 🛠️ Las cuatro herramientas del detective en VS Code
    - 🏠 Ejemplo guiado: el caso del sospechoso en VS Code
    - 🧠 La regla de oro del depurador
    - 🎯 Mini-comprobación

    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 05 — Comentarios y documentación
    - 📝 Los tres tipos de comentarios
    - 🔑 El consejo que te ahorrará horas: comenta el “por qué”, no el “qué”
    - 📖 Javadoc: documentación que se genera sola
    - 🏫 Ejemplo guiado: documentar un programa completo
    - 🎯 Mini-comprobación
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 06 — Argumentos de línea de órdenes
    - 🎒 ¿Qué es “args”?
    - 🔢 Cómo se indexan las palabras
    - 🧠 ¿Para qué sirve pasar argumentos?
    - 🏫 Ejemplo guiado: el programa que te saluda
    - 🎯 Mini-comprobación
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 07 — El compilador y sus errores
    - 🔍 Compilar ≠ ejecutar
    - 😤 El error del aprendiz: 4 errores en un programa
    - 🔎 Cómo leer un mensaje de error de javac
    - 🚨 Errores típicos del aprendiz (y su remedio)
    - 🧩 EL LÍO: el código revuelto
    - 🎯 Mini-comprobación
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 08 — El IDE y tu flujo de trabajo
    - 🗂️ Anatomía de un proyecto en VS Code
    - 🔁 El ciclo de trabajo (tu nuevo bucle de vida)
    - ⌨️ Los atajos de VS Code que te harán parecer un pro
    - 🧩 Autocompletado (IntelliSense): tu compañero silencioso
    - 🏫 Ejemplo guiado: de cero a ejecutar en 60 segundos en VS Code

    - 🎯 Mini-comprobación
    - ✅ Resumen en 3 frases
    - 🦎 Vocabulario rápido
- 09 — Repaso
    - ⭐ Intenta pensar como el código, my friend…
    - 🔥 Fireside Chat: JDK vs JRE
    - 🦹 ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “El compilador me odia”
    - ⚡ Laboratorio de Tortura: el programa que no arranca
    - 🏆 Logros de esta unidad
    - 🧠 Atréverte a pensar
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U03 — Variables, tipos y operadores
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA2 + RA1)
    - 🚪 ¿Por dónde empiezo?
- 01 — Variables y tipos primitivos
    - 📬 La idea en una frase
    - 📦 La declaración: la receta de una caja
    - 🏷️ Las reglas de nomenclatura (o cómo no meter la pata)
    - 📐 Los 8 primitivos: el catálogo de cajas
    - 🎒 ¿Qué caja uso para cada dato?
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 — String, constantes y final
    - 📬 La idea en una frase
    - 🔤 String: la caja mágica que no es una caja
    - 🧊 La inmutabilidad: no toques, que se rompe

    - 🧲 == vs .equals(): la trampa clásica
    - 🔒 Constantes con final: cajas con superglue
    - 🏫 Ejemplo guiado: la factura que no se puede tocar
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 — Operadores aritméticos
    - 📬 La idea en una frase
    - 💪 El día en el gimnasio: los 5 ejercicios básicos
    - ⚠️ La división entera mata
    - 🎭 Precedencia: la ley del comedor
    - 🌀 Asignación compuesta: el atajo perezoso
    - 💪 ++ y --: flexiones para variables
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 — Relacionales, lógicos y ternario
    - 📬 La idea en una frase
    - ⚖️ Relacionales: el juez de la discusión
    - 🚪 Lógicos: el portero del club
    - ⚡ Cortocircuito: el portero que no mira
    - 🚦 El ternario: el bouncer del club
    - 🏫 Ejemplo guiado: el portero del club
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 — Casting y conversiones
    - 📬 La idea en una frase
    - 🪜 Conversión implícita (widening): mudarse a una caja más grande
    - 📉 Conversión explícita (narrowing): la maleta XXL en un Smart
    - 🪓 El truncamiento: el céntimo olvidado
    - 💥 El desbordamiento: cuando aprietas de más
    - 🏫 Ejemplo guiado: el guardia del almacén

    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 · Scanner: leer por teclado
    - 📬 La idea en una frase
    - 📖 El paquete: importar la librería
    - 🏗️ Instanciar: el constructor
    - 🔢 Pedir datos: los métodos next
    - ⚠️ El lío de nextLine() tras nextInt()
    - 🏫 Ejemplo guiado: la calculadora de la propina
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 · Consola: salida formateada y errores de entrada
    - 📬 La idea en una frase
    - 🖨️ Salida con formato: System.out.printf
    - 📝 String.format: lo mismo, pero sin imprimir
    - 💶 Números grandes: NumberFormat
    - 🚨 Errores clásicos del Scanner (y sus remedios)
    - 🏫 Ejemplo guiado: el validador que no se rompe
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 — Math.random() y números aleatorios
    - 📬 La idea en una frase
    - 🎲 El casino: qué devuelve Math.random()
    - 🎯 La fórmula: de 0.0-1.0 a lo que tú quieras
    - 🧰 Otras herramientas del casino: la clase Math
    - 🏫 Ejemplo guiado: el juego de los dados
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 — Métodos útiles de String

    - 📬 La idea en una frase
    - 🪄 El gimnasio del texto: los métodos que medirás
    - 🔍 Los métodos que buscan
    - ✂️ Los métodos que recortan
    - 🏠 Ejemplo guiado: el nombre del usuario
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 10 · Repaso interactivo: la caja que no cabía
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: int vs double
    - 🕵️ ¿Quién soy?
    - 👺 CONRAD VS EL MUNDO: “El compilador odia tu memoria”
    - ⚡ Laboratorio de tortura: el programa que cobra mal
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U04 — Estructuras de control y excepciones básicas
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA3)
    - 🚪 ¿Por dónde empiezo?
- 01 — if, else if y else: el semáforo del código
    - 📬 La idea en una frase
    - 🚦 El semáforo: if y else
    - 🔀 else if: cuando hay más de dos caminos
    - 🪆 If anidados: semáforos dentro de semáforos
    - 🎚️ El ternario: el if de bolsillo

    - 🏫 Ejemplo guiado: la discoteca municipal
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 — switch: el menú del restaurante
    - 📬 La idea en una frase
    - 🍽️ La carta del restaurante
    - 🧱 Las piezas del puzzle
    - 🌀 El fall-through: ¿error o superpoder?
    - 🆚 switch vs else if: el duelo
    - 🏫 Ejemplo guiado: el menú del día
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 — Bucles: while y do-while
    - 📬 La idea en una frase
    - 🏃 while: comprueba y luego corre
    - ♾️ El bucle infinito (y cómo salir)
    - 🏃 do-while: corre y luego comprueba
    - 🆚 while vs do-while: el duelo rápido
    - 🏫 Ejemplo guiado: la cuenta atrás
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 — Bucle for y bucles anidados
    - 📬 La idea en una frase
    - 🔢 La anatomía del for
    - 🔁 Los tres bucles son el mismo chiste
    - 🧩 Bucles anidados: la parrilla de ejercicios
    - 🏫 Ejemplo guiado: la tabla de multiplicar
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases

- 05 — break, continue y etiquetas
  - 📬 La idea en una frase
  - 🚪 break: el botón de parada
  - ⏭️ continue: el botón de saltar
  - 🏷️ Etiquetas: el GPS de los bucles anidados
  - 🏫 Ejemplo guiado: el detector de primer número
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 06 — Excepciones básicas
  - 📬 La idea en una frase
  - 💥 El crash: tu primera excepción
  - 🌳 La familia Throwable: el árbol genealógico
  - 🗺️ Las excepciones más comunes: la guía de campo
  - 🏫 Ejemplo guiado: el lector de edades
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 07 — try, catch y finally
  - 📬 La idea en una frase
  - 🛡️ La estructura completa
  - 🎯 Captura múltiple: varios catch en fila
  - 🧀 La variable e: el botín del error
  - 🏫 Ejemplo guiado: el menú a prueba de bombas
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 08 — throw y excepciones propias
  - 📬 La idea en una frase
  - 🎳 throw: lanza la piedra
  - 📐 Excepciones propias: tu defecto a medida
  - ⚖️ checked vs unchecked: la burocracia de las excepciones
  - 🏫 Ejemplo guiado: la máquina expendedora

    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 — Repaso interactivo: controla las estructuras
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: if-else vs switch
    - 🕵️‍♀️ ¿Quién soy?
    - 🤬 CONRAD VS EL MUNDO: “El bucle que no acaba”
    - 🎮 El juego de las decisiones
    - ⚡ Laboratorio de tortura: la máquina que llora
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U05 — Funciones y métodos
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA2)
    - 🚪 ¿Por dónde empiezo?
- 01 · ¿Qué es una función?
    - 📬 La idea en una frase
    - 🍳 El problema: el párrafo infinito
    - 📖 La receta con nombre
    - 🔧 ¿Función o método? (no, no es lo mismo… bueno, sí)
    - 🧰 Qué gana tu programa al trocearlo
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 · Tu primer método

    - 📬 La idea en una frase
    - ✍️ La primera receta
    - 🔍 La firma, palabra por palabra
    - 🎬 main también es un método
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 · Parámetros: la entrada
    - 📬 La idea en una frase
    - 🔁 El problema: el método de siempre
    - 📥 Cómo se declaran
    - 🎯 Reglas de las entradas
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 · return: la salida
    - 📬 La idea en una frase
    - 📤 Dos oficios, dos firmas
    - 🏞️ return corta la ejecución
    - 🎭 El pecado de imprimir cuando debes devolver
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 · Ámbito de variables
    - 📬 La idea en una frase
    - 🏠 Cada método es una casa
    - 📏 Las reglas del ámbito
    - 🚪 Lo que entra por la puerta también es de la casa
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 · Errores frecuentes
    - 📬 La idea en una frase

    - 🧯 La galería de los 7 tropiezos
    - 🩺 Cómo diagnosticar sin perderte
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 · Divide el problema
    - 📬 La idea en una frase
    - 🎼 El director no toca los instrumentos
    - 🔪 Cómo trocear: el método de los verbos
    - 🧪 Antes y después: el informe de notas
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 · Be the Code
    - 📬 La idea en una frase
    - 🛠️ Tu punto de partida
    - 🪜 Paso a paso (pistas que no regalan el final)
    - 🧪 El Lío: el refactor malogrado
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 · Repaso interactivo
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: void vs return
    - 🕵️‍♀️ ¿Quién soy?
    - 🚨 CONRAD VS EL MUNDO: “El main que se hizo el listo”
    - 🎮 El juego de las decisiones
    - ⚡ Laboratorio de tortura: el programa sin nombre
    - 🏆 Logros de esta unidad
    - 🧐 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas

    - 🎬 Poscréditos
- U06 — Arrays
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA6)
    - 🚪 ¿Por dónde empiezo?
- 01 · Arrays: el aparcamiento de datos
    - 📬 La idea en una frase
    - 😾 El problema: tienes 100 gatos y un solo nombre
    - 🅿️ El array: tu primer aparcamiento
    - 🚗 Cómo meter cosas en el parking
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 · Recorrer arrays: for y for-each
    - 📬 La idea en una frase
    - 🔗 El dúo inseparable: for + array
    - 🛋️ for-each: la variante perezosa
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 · Arrays multidimensionales
    - 📬 La idea en una frase
    - 🏢 El parking de varias plantas
    - 🎲 Arrays de arrays irregulares
    - 🔁 Recorrer un array 2D: los bucles anidados
    - 🧮 Para qué sirve de verdad
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 · La clase Arrays: tu navaja suiza
    - 📬 La idea en una frase

    - 🧰 La clase Arrays
    - 🧭 El mapa de la navaja
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 · Arrays y métodos
    - 📬 La idea en una frase
    - 🏃 Pasando el testigo
    - 🆚 Primitivos vs arrays: la diferencia clave
    - 🪄 Métodos que usan arrays
    - 🏁 El famoso main(String[] args)
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 · Aplicaciones de los arrays
    - 📬 La idea en una frase
    - 📝 Arrays de String: la lista de la clase
    - 🔤 char[] vs String
    - 🧍 Arrays de objetos: el parking de personas
    - 🗂️ Arrays de arrays: los datos en tabla
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 · Be the Code: el parking se gestiona
    - 📬 La idea en una frase
    - 🕶️ Don Tip: la receta mental del inverso en el sitio
    - 🧩 RETO 1: Invertir el array en el sitio
    - 🧩 RETO 2: Buscar todas las posiciones
    - 🧩 EL LÍO: el parking que se rebeló
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 · Array-revelde: errores comunes y depuración
    - 📬 La idea en una frase

    - 👹 La galería de monstruos
    - 🤬 CONRAD VS EL MUNDO: “El array que no se calla”
    - 🦎 Depurar arrays sin perder la cabeza
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 · Repaso interactivo: el parking a examen
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: for clásico vs for-each
    - 🦹 ¿Quién soy?
    - 🤬 CONRAD VS EL MUNDO: “El array que no para”
    - 🎮 El juego de las decisiones
    - ⚡ Laboratorio de tortura: la media que se resiste
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U07 — Algorítmica I: Fundamentos
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA2 + RA6)
    - 🚪 ¿Por dónde empiezo?
- 01 — Qué es un algoritmo
    - 📬 La idea en una frase
    - 🏛️ Las propiedades de un algoritmo
    - 🧠 La idea vs. el código
    - 📦 Las dos grandes familias
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo

    - ✅ Resumen en 3 frases
- 02 — Búsqueda lineal
    - 📬 La idea en una frase
    - 🔪 El algoritmo
    - ⏱️ ¿Cómo de rápida es?
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 — Búsqueda binaria
    - 📬 La idea en una frase
    - ⚠️ El requisito imprescindible
    - 🕯️ El algoritmo
    - 🧮 ¿Por qué izquierda + (derecha - izquierda) / 2 y no (izquierda + derecha) / 2?
    - ⏱️ El análisis: O(log n)
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 — Ordenación burbuja
    - 📬 La idea en una frase
    - 🫧 El algoritmo
    - 🐌 ¿Por qué es tan lenta?
    - 🚩 La optimización del flag
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 — Ordenación por inserción
    - 📬 La idea en una frase
    - 🃏 El algoritmo
    - 👣 Paso a paso
    - 📊 El análisis: ¿cuándo es buena?
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases

- 06 — Complejidad algorítmica: Big O
  - 📬 La idea en una frase
  - 📈 Las complejidades más comunes
  - 📏 Big O en código
  - 🔪 Las reglas prácticas de Big O
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 07 — Elegir el algoritmo adecuado
  - 📬 La idea en una frase
  - 📋 La tabla de resumen
  - 🧮 La pregunta clave: ¿ordenar antes de buscar?
  - 🏪 Ejemplo guiado: el catálogo de la tienda
  - 📊 Reglas de oro para decidir
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 08 — Be the Code: búsqueda binaria desde cero
  - 📬 La idea en una frase
  - 🕶️ Don Tip: la receta mental de la binaria
  - 🧩 RETO 1: Búsqueda binaria desde cero
  - 🧩 RETO 2: Burbuja a mano
  - 🧩 EL LÍO: la burbuja que apesta
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 09 — Repaso interactivo: ordena y busca como un profesional
  - 📬 La idea en una frase
  - ⭐ Sé el Código, my friend…
  - 🔥 Fireside Chat: búsqueda lineal vs búsqueda binaria
  - 🕵️ ¿Quién soy?
  - 🤬 CONRAD VS EL MUNDO: “El algoritmo que no termina”
  - 🎮 El juego de las decisiones
  - ⚡ Laboratorio de tortura: la máquina que no ordena

    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U08 — Algorítmica II: técnicas avanzadas y modularidad
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA2 + RA6)
    - 🚪 ¿Por dónde empiezo?
- 01 — Recursividad: una función que se llama a sí misma
    - 📬 La idea en una frase
    - 🔁 El ciclo de la vida recursiva
    - 📚 La pila de llamadas: el origen de toda la diversión
    - 🪞 Recursividad vs iteración: el duelo
    - 🌰 Un primer ejemplo completo
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 — Recursividad: ejemplos clásicos
    - 📬 La idea en una frase
    - 🧮 Factorial: el “Hola Mundo” de la recursividad
    - 🐰 Fibonacci ingenua: no hagas esto en casa
    - ⚡ Fibonacci optimizado: con memoización, ahora sí
    - 🔍 Palíndromos: la palabra espejo
    - 🗼 Torres de Hanói: el rey de los clásicos
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 — Divide y vencerás
    - 📬 La idea en una frase
    - 🧩 Los tres pasos del patrón
    - 📊 ¿Cuándo merece la pena?

    - 💡 Un ejemplo sencillo: el máximo de un array
    - 🌳 Aplicaciones famosas
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 — Quicksort: el rápido
    - 📬 La idea en una frase
    - 🎯 Cómo funciona
    - 🛠️ Implementación
    - 🎚️ Elección del pivote
    - 📊 Complejidad
    - 🚫 ¿Es estable?
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 — Mergesort: el fiable
    - 📬 La idea en una frase
    - 🎯 Cómo funciona
    - 🛠️ Implementación
    - 📊 Complejidad
    - ✅ ¿Es estable?
    - 🏆 Cuándo brilla Mergesort
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 — Comparación: cuándo usar cada ordenación
    - 📬 La idea en una frase
    - 🧭 La tabla de decisión
    - 🥊 El duelo de los titanes
    - 🕵️ ¿Qué hace Java por dentro?
    - 📊 Rendimiento en la práctica
    - 🧠 La lógica que hay detrás
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 — Be the Code: quicksort desde cero
    - 📬 La idea en una frase

    - 🚦 Ejercicio guiado 1: Quicksort
    - 🚦 Ejercicio guiado 2: Mergesort
    - ⭐ Niveles de logro
    - 🧩 El lío
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 — Otras técnicas algorítmicas
    - 📬 La idea en una frase
    - 🍖 Algoritmos voraces (greedy)
    - 🔁 Backtracking (vuelta atrás)
    - 🧠 Y con esto, el panorama
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 · Modularidad
    - 📬 La idea en una frase
    - 🧩 ¿Por qué importa tanto?
    - 📦 La pieza oficial en Java: el paquete
    - 🎯 Cohesión alta, acoplamiento bajo
    - ✂️ Cómo decides los cortes
    - 🔮 Lo que viene después
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 10 · Repaso interactivo
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🎯 Verdadero o falso
    - 🔥 Fireside Chat: Quicksort y Mergesort discuten
    - 🕵️ ¿Quién soy?
    - 🧩 Completa el código
    - 🤬 CONRAD VS EL MUNDO: “Tu recursión explotó”
    - ⚡ Laboratorio de tortura: el programa que no ordena
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar

    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U09 — POO: clases y objetos
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA2 + RA4)
    - 🚪 ¿Por dónde empiezo?
- 01 — Qué es la POO
    - 📬 La idea en una frase
    - 📜 De la receta al mundo real
    - 🧩 Los dos ingredientes: atributos y métodos
    - 🏛️ Los cuatro pilares (una promo, no el capítulo)
    - ⭐ Sé el Código, my friend…
    - ❓ ¡No hay preguntas tontas!
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 — Clases y objetos: el cortapastas y las galletas
    - 📬 La idea en una frase
    - 🍪 El cortapastas y la galleta
    - 🔨 Tu primera clase
    - 🏭 new: la máquina de galletas
    - 🥊 Clase vs objeto: el ring
    - ⭐ Sé el Código, my friend…
    - ❓ ¡No hay preguntas tontas!
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 — Atributos y métodos
    - 📬 La idea en una frase
    - 🏷️ Los atributos: las propiedades de la cosa
    - 🎮 Los métodos: los comportamientos de la cosa

    - 📞 Llamar a los métodos
    - 🧬 Cada objeto con su propia copia
    - ⭐ Sé el Código, my friend…
    - ❓ ¡No hay preguntas tontas!
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 — Constructores
    - 📬 La idea en una frase
    - 🎉 ¿Qué es un constructor?
    - 🆓 El constructor por defecto (el que regala Java)
    - 🎛️ El constructor con parámetros (el horno con programa)
    - 🏭 new y el constructor, un tándem
    - ⭐ Sé el Código, my friend…
    - ❓ ¡No hay preguntas tontas!
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 — La palabra clave this
    - 📬 La idea en una frase
    - 🔍 Desambiguar: atributo vs parámetro
    - 🔗 Llamar a otro constructor: this(…)
    - 📦 Pasarte a ti mismo como argumento
    - ⭐ Sé el Código, my friend…
    - ❓ ¡No hay preguntas tontas!
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 — Referencias, null y memoria
    - 📬 La idea en una frase
    - 📬 La carta frente a la casa
    - 👯 El alias: dos variables, un objeto
    - 🪶 null: la referencia al vacío
    - 🏠 Primitivos vs referencias: el resumen
    - ⭐ Sé el Código, my friend…
    - ❓ ¡No hay preguntas tontas!

    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 — Sobrecarga y paso por valor
    - 📬 La idea en una frase
    - 🎭 Sobrecarga: el mismo nombre, otras manos
    - 📦 Paso por valor: Java solo sabe copiar
    - 📊 El resumen que te salva en el examen
    - ⭐ Sé el Código, my friend…
    - ❓ ¡No hay preguntas tontas!
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 — Be the Code: diseña tu clase
    - 📬 La idea en una frase
    - 🚦 Ejercicio guiado: la cuenta bancaria
    - ⭐ Niveles de logro
    - 🧩 El lío
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 — Repaso interactivo: domina las clases
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: Clase vs objeto
    - 🕵️ ¿Quién soy?
    - 😡 CONRAD VS EL MUNDO: “Constructor, ¿dónde estás?”
    - 🎮 El juego de las decisiones
    - ⚡ Laboratorio de tortura: la fábrica que llora
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💭 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U10 — Visibilidad, encapsulación y static

    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA4)
    - 🚪 ¿Por dónde empiezo?
- 01 — Visibilidad: el arte de no enseñarlo todo
    - 📬 La idea en una frase
    - 🏠 El problema de la casa de cristal
    - 🚪 Por qué importa la visibilidad
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 — public, private y protected
    - 📬 La idea en una frase
    - 🗂️ Los 4 niveles: de la valla al diario
    - 📣 public: la valla publicitaria
    - 🔒 private: el diario con candado
    - 👨‍👩‍👧 protected: los secretos de familia
    - 🏘️ package-private: el cotilleo del barrio
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 — Encapsulación: el pilar de la POO
    - 📬 La idea en una frase
    - 📦 La fórmula de la encapsulación
    - 🎁 Las ventajas (o por qué no dormirás peor)
    - 🧬 La convención JavaBeans: el protocolo
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 — Getters y setters
    - 📬 La idea en una frase
    - 🏷️ La convención de nombres (memorízala)

    - 🚪 El setter como portero: validar
    - 🔒 Getter sin setter: atributos inmutables
    - 🤷 No hay preguntas tontas
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 — Atributos static
    - 📬 La idea en una frase
    - 📱 El grupo de WhatsApp vs los mensajes privados
    - 🧠 Cómo se ve en memoria
    - 📞 El uso clásico: contadores e ids
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 — Métodos static y el main
    - 📬 La idea en una frase
    - 📞 El teléfono de la clase
    - 🚫 Lo que un método estático NO puede hacer
    - 🧮 La clase Math: la fábrica de métodos estáticos
    - 🎉 Por qué main es static
    - 🤷 No hay preguntas tontas
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 — Constantes de clase: static final
    - 📬 La idea en una frase
    - 🏷️ La constante de clase
    - 🔒 La regla de las MAYÚSCULAS
    - 🏛️ Las clases utilitarias: el “no necesito pareja” de Java
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 — Be the Code: encapsula tu clase

    - 📬 La idea en una frase
    - 🏠 El punto de partida: la clase del caos
    - 🧰 Paso 1: atributos privados
    - 🧰 Paso 2: getters para leer
    - 🧰 Paso 3: setter para lo que se puede cambiar
    - 🧰 Paso 4: validar también en los métodos
    - 🧰 Paso 5: el toque final, constantes
    - 🏆 La clase final (compárala con la del caos)
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 — Repaso interactivo: blindalo todo
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: la clase y el objeto
    - 🦹 ¿Quién soy?
    - 🎮 El juego de las decisiones
    - 😡 CONRAD VS EL MUNDO: “¿Por qué todo público?”
    - ⚡ Laboratorio de tortura: la caja fuerte sin candado
    - 🧩 EL LÍO: el setter que no validaba
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💭 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U11 — Herencia, polimorfismo e interfaces
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA4 y RA7)
    - 🚪 ¿Por dónde empiezo?
- 01 — Herencia: cuando tus hijos siguen tus pasos
    - 📬 La idea en una frase

    - 🧬 ¿Qué es la herencia?
    - 🔑 extends: “Soy como tú, pero con mejoras”
    - ⚖️ La regla de oro: IS-A vs HAS-A
    - 🌳 Jerarquía de clases: el árbol genealógico
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 — super y @Override
    - 📬 La idea en una frase
    - 📞 super: llamando a mamá/papá para que te ayuden
    - ✍️ @Override: “Papá, yo lo hago mejor”
    - 🔒 final: cuando quieres bloquear la herencia
    - 🏗️ Constructores en la herencia: la cadena de montaje
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 — protected y jerarquías de clases
    - 📬 La idea en una frase
    - 🔐 protected: el secreto que solo la familia ve
    - 🌳 Diseñando jerarquías sensatas
    - 💥 El problema de la clase base frágil
    - 🧩 Composición: “tener” en vez de “ser”
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 — Polimorfismo: el camaleón de la POO
    - 📬 La idea en una frase
    - 🎭 El mismo método, diferentes comportamientos
    - 🔮 Dynamic binding: la decisión en runtime
    - 🔗 Referencias polimórficas
    - 📚 Polimorfismo con colecciones
    - 👷 Polimorfismo con parámetros
    - 🚫 Los atributos NO son polimórficos

    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 — instanceof y downcasting
    - 📬 La idea en una frase
    - 🕵️‍♀️ instanceof: “¿Quién eres realmente?”
    - 📉 Downcasting: bajar la referencia
    - 🛡️ El combo seguro: instanceof + downcasting
    - ✨ Bonus: pattern matching (Java 16+)
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 — La clase Object: el tatarabuelo
    - 📬 La idea en una frase
    - 👴 Object: la raíz de todo
    - 📇 toString(): la tarjeta de presentación
    - ⚖️ equals(): ¿mismo objeto o iguales?
    - 🏷️ hashCode(): el código de barras
    - 🕵️‍♀️ getClass(): el nombre del objeto real
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 — Clases abstractas
    - 📬 La idea en una frase
    - 🎨 ¿Qué es una clase abstracta?
    - 📋 You MUST: implementar los métodos abstractos
    - 📐 Ejemplo: figuras geométricas
    - 🏗️ Constructores en clases abstractas
    - 🧩 Template Method: las abstractas en acción
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 — Interfaces: el contrato que firmas

    - 📬 La idea en una frase
    - ✍️ Declarando e implementando
    - 🎭 Polimorfismo con interfaces
    - 🤹 Múltiples interfaces
    - 🩹 default Methods: parches sin romper nada
    - ⚖️ La gran decisión: abstract class vs interface
    - 🥊 EL RING: extends vs implements
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 — Repaso interactivo: domina la herencia
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: Clase abstracta vs Interfaz
    - 🦹 ¿Quién soy?
    - ⚖️ Verdadero o falso
    - ⚡ Laboratorio de tortura: la herencia que llora
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U12 — Colecciones, genéricos y mapas
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA6)
    - 🚪 ¿Por dónde empiezo?
- 01 · ArrayList y el Java Collections Framework
    - 📬 La idea en una frase
    - 🅿️ El problema: tu array se ha quedado sin plazas
    - 🎓 El Java Collections Framework (JCF)

    - 🅿️📈 ArrayList: el parking que crece solito
    - 🚫 ArrayList NO guarda primitivos
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 · LinkedList: la conga line
    - 📬 La idea en una frase
    - 🎉 LinkedList: la conga line
    - 🛒 Colas y pilas: los dos disfraces de LinkedList
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 · HashSet, TreeSet y LinkedHashSet: los conjuntos
    - 📬 La idea en una frase
    - 🚫 HashSet: el portero que no deja duplicados
    - 🧭 TreeSet: el organizado
    - 🧠 LinkedHashSet: el que recuerda el orden de llegada
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 · Iterator y Collections: recorrer y manipular
    - 📬 La idea en una frase
    - 🍽️ Iterator: el camarero que toma nota
    - 🧰 Collections: el amigo utilitario
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 · Clases genéricas: tu propia `Caja`
    - 📬 La idea en una frase
    - 😱 El infierno antes de los genéricos
    - 🤝 Llegan los genéricos
    - 🔤 Las convenciones: T, E, K, V, N
    - 📦 La clase Caja

    - ⭐ Sé el Código, my friend...
    - 🤷 No hay preguntas tontas
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 · El diamante, métodos genéricos y wildcards
    - 📬 La idea en una frase
    - 💎 El operador diamante <>
    - 🎩 Type erasure: el mago se lleva los genéricos
    - 🧰 Métodos genéricos
    - 🃏 Wildcards: el problema de la invarianza
    - ⭐ Sé el Código, my friend...
    - 🤷 No hay preguntas tontas
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 · HashMap: la guía telefónica
    - 📬 La idea en una frase
    - ☎️ HashMap: clave → valor
    - 🛟 getOrDefault: el salvavidas de los null
    - 🚶 Recorrer un HashMap: tres caminos
    - ⭐ Sé el Código, my friend...
    - 🤷 No hay preguntas tontas
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 · TreeMap y cómo elegir estructura
    - 📬 La idea en una frase
    - 🌳 TreeMap: el ordenado de serie
    - 🔗 LinkedHashMap: el que recuerda el orden de llegada
    - ⚖️ La familia al completo
    - 🥊 EL RING: HashMap vs TreeMap
    - ⭐ Sé el Código, my friend...
    - 🎯 Mini-chequeo
    - ⚖️ Map vs List vs Set: la comparativa definitiva
    - 🧭 El árbol de decisiones

    - 🏅 La regla práctica
    - ✅ Resumen en 3 frases (ampliado)
- 09 · Repaso interactivo: colecciones, genéricos y mapas
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: ArrayList vs LinkedList
    - 🔥 Fireside Chat: el Genérico vs el Tipo Crudo
    - 🕵️‍♀️ ¿Quién soy?
    - 🤬 CONRAD VS EL MUNDO: “El parking que no aparca”
    - 🎮 El juego de las decisiones
    - ⚡ Laboratorio de tortura: la lista que llora
    - ⚡ Laboratorio de tortura: la media que llora
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U13 — Programación funcional (Streams y Lambdas)
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA6)
    - ✅ Criterios de evaluación cubiertos (RA7)
    - 🚪 ¿Por dónde empiezo?
- 01 · Lambdas: funciones que viajan solas
    - 📬 La idea en una frase
    - 😱 La forma antigua: clases anónimas
    - 🏹 Llega la flecha: la lambda
    - 📦 Funciones que viajan en variables
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases

- 02 · Interfaces funcionales: Predicate, Function, Consumer, Supplier
  - 📬 La idea en una frase
  - 🎭 ¿Qué es una interfaz funcional?
  - 🎭 Los cuatro trajes oficiales
  - 📋 La chuleta de las cuatro
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 03 · Referencias a métodos: el atajo `::`
  - 📬 La idea en una frase
  - 🩹 Cuando la lambda es solo una llamada
  - 🗂️ Los cuatro tipos de referencia
  - 📋 La tabla del ::
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 04 · Streams: el flujo de datos
  - 📬 La idea en una frase
  - 📦 ¿Qué es un Stream (y qué NO es)?
  - 🚚 Crear un Stream: las cuatro fuentes
  - 🏭 El pipeline: fuente → intermedias → terminal
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 05 · Operaciones intermedias: filter, map, distinct, sorted, limit
  - 📬 La idea en una frase
  - 🌀 Las cinco estaciones
  - 🏭 Las estaciones en cadena
  - ⭐ Sé el Código, my friend…
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 06 · Operaciones terminales: collect, reduce, count, forEach
  - 📬 La idea en una frase

    - 🚰 Las cuatro terminales
    - 🏭 El pipeline completo
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 · Del stream a la colección: toList, Collectors, groupingBy
    - 📬 La idea en una frase
    - 📥 toList(): la salida rápida (Java 16+)
    - 🗂️ Los Collectors de toda la vida
    - 🗺️ groupingBy: agrupar como en una tabla
    - 🗺️ toMap: de elementos a entradas
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 · Optional: el que acepta la ausencia
    - 📬 La idea en una frase
    - 🎁 La caja en acción
    - 🎁 Abrir la caja con elegancia
    - 🎁 Optional y los streams: pareja de baile
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 · Repaso interactivo: domina la programación funcional
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: el Bucle For vs el Stream
    - 🕵️ ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “El que olvida el grifo”
    - 🎮 El juego de las decisiones
    - ⚡ Laboratorio de tortura: la estadística que no arranca
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits

    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U14 — Ficheros y expresiones regulares (Regex)
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad
    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA5)
    - ✅ Criterios de evaluación cubiertos (RA6)
    - 🚪 ¿Por dónde empiezo?
- 01 · La clase File: tu explorador de archivos
    - 📬 La idea en una frase
    - 🗂️ File: el GPS de tus archivos
    - 📁 Recorrer una carpeta con listFiles
    - 📝 La ruta no es el contenido
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 · Escribir y leer texto: FileWriter, FileReader y BufferedReader
    - 📬 La idea en una frase
    - ✍️ Escribir con FileWriter
    - 📖 Leer con FileReader + BufferedReader
    - 🚨 ¿Y si el archivo no existe al leer?
    - 🧱 ¿Y por qué BufferedReader y no FileReader a secas?
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 · try-with-resources: cerrar sin llorar
    - 📬 La idea en una frase
    - 🔒 El try-with-resources en acción
    - 🧹 Varios recursos a la vez
    - 🧐 ¿Qué recursos valen?
    - ⭐ Sé el Código, my friend…

    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 · PrintWriter y Scanner + File
    - 📬 La idea en una frase
    - 🖨️ PrintWriter: System.out, pero en un archivo
    - 🐙 Scanner + File: leer archivos a pedazos
    - 🪓 useDelimiter: cortar donde tú digas
    - 🆚 Scanner vs BufferedReader: ¿cuál uso?
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 · NIO: Files y Paths (la forma moderna)
    - 📬 La idea en una frase
    - 🗂️ Path y Paths: el nuevo File
    - ⚡ Files: la navaja suiza
    - 📋 Otras joyas de Files
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 · Serialización: guardar objetos con ObjectOutputStream
    - 📬 La idea en una frase
    - 🧳 El trato: implementar Serializable
    - 📦 Guardar: ObjectOutputStream
    - 📭 Recuperar: ObjectInputStream
    - 🧨 ¿Y si la clase cambia entre guardar y leer?
    - 📚 Guardar colecciones enteras
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 · Expresiones regulares: Pattern y Matcher
    - 📬 La idea en una frase
    - ⚠️ El infierno de las contrabarras (léelo dos veces)
    - 🔍 Pattern y Matcher: el molde y el texto

    - 🧩 La tabla de símbolos (tu chuleta de examen)
    - 🦸 Grupos de captura: los paréntesis que recuerdan
    - 🚩 Las banderas (flags)
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 · Regex en acción: matches, replaceAll, split y validaciones
    - 📬 La idea en una frase
    - 🎯 String.matches(): ¿cumple TODO el string?
    - 🧹 replaceAll(): limpiar texto en una línea
    - 🔪 split(): trocear por patrón
    - 🎫 Validaciones de la vida real
    - 🦸 El procesador de logs (grupos de captura + ficheros)
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 · Repaso interactivo: domina ficheros y regex
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: File vs Path
    - 🦸 ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “No cerré el archivo”
    - 🎮 El juego de las decisiones
    - ⚡ Laboratorio de tortura: el validador que no valida
    - 🏆 Logros de esta unidad
    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- U15 — Persistencia de datos: JDBC e introducción a ORM
    - 🎯 Objetivo de la unidad
    - 🗺️ Mapa de la unidad

    - 📝 Boletines de la unidad
    - ✅ Criterios de evaluación cubiertos (RA9)
    - 🚪 ¿Por dónde empiezo?
- 01 · Qué es JDBC
    - 📬 La idea en una frase
    - 🔌 JDBC: el USB de las bases de datos
    - 🗄️ SQLite: la base de datos de entrenamiento
    - 📦 La dependencia Maven
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 · Conexión a SQLite: los 5 pasos y Connection
    - 📬 La idea en una frase
    - 🪜 Los 5 pasos (y el bonus no opcional)
    - 🔗 Connection: la tubería
    - 💊 La conexión completa con try-with-resources
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 · Statement y ResultSet: consultar datos
    - 📬 La idea en una frase
    - 📤 Statement: el mensajero
    - 📊 ResultSet: la tabla virtual
    - ⚖️ executeQuery vs executeUpdate
    - 💊 El programa completo
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 · CRUD completo: insertar, leer, actualizar y borrar
    - 📬 La idea en una frase
    - 🧱 La tabla
    - 🧍 El POJO
    - ✍️ Create (INSERT)
    - 📖 Read (SELECT)
    - 🖍️ Update (UPDATE)
    - 🗑️ Delete (DELETE)

    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 · PreparedStatement y SQL injection
    - 📬 La idea en una frase
    - 💣 Concatenar SQL: la bomba
    - 💉 PreparedStatement al rescate
    - 🍼 Guía de placeholders
    - ⭐ Sé el Código, my friend…
    - 🥊 El ring: Statement vs PreparedStatement
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 06 · El patrón DAO
    - 📬 La idea en una frase
    - 🏛️ La idea: separar el SQL del negocio
    - 📜 La interfaz: el contrato
    - 🏭 La implementación: quien pone las manos en la masa
    - 🤔 DAO vs Repository
    - ⭐ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 · Transacciones: commit, rollback y savepoints
    - 📬 La idea en una frase
    - 🎭 El problema: el limbo del dinero
    - 🔄 setAutoCommit(false): abrir el paréntesis
    - 📍 Savepoints: no deshacerlo todo
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 · Buenas prácticas: el decálogo del JDBC
    - 📬 La idea en una frase
    - 📜 El decálogo del JDBC
    - 🔮 El acertijo
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases

- 09 · Introducción a ORM
  - 📬 La idea en una frase
  - 🔎 ¿Qué es un ORM, en concreto?
  - ⚖️ JDBC vs ORM: la misma tarea, dos estilos
  - 🧰 El arsenal: JPA, Hibernate y compañía
  - 🐘 El lado oscuro: lo que el ORM no te arregla
  - 🧭 ¿Y por qué lo vemos ahora?
  - 🎯 Mini-chequeo
  - ✅ Resumen en 3 frases
- 10 · Repaso interactivo: domina el JDBC
  - 📬 La idea en una frase
  - ⭐ Sé el Código, my friend…
  - 🔥 Fireside Chat: Statement vs PreparedStatement
  - 🦹 ¿Quién soy?
  - 👺 CONRAD VS EL MUNDO: “El JDBC que no se cierra”
  - 🎮 El juego de las decisiones
  - ⚡ Laboratorio de tortura: el CRUD que sangra
  - 🏆 Logros de esta unidad
  - 🧠 Atrévete a pensar
  - 🧩 Crucigrama de bits
  - 💬 Preguntas de entrevista de trabajo
  - 🤷 No hay preguntas tontas
  - 🎬 Poscréditos
- U16 — Servir y consumir APIs con Web
  - 🎯 Objetivo de la unidad
  - 🗺️ Mapa de la unidad
  - 📝 Boletines de la unidad
  - ✅ Criterios de evaluación cubiertos (RA5)
  - 🚪 ¿Por dónde empiezo?
- 01 · El protocolo HTTP en 30 segundos
  - 📬 La idea en una frase
  - 💬 La conversación de la web
  - 🎤 Los verbos de HTTP

    - 🚦 Los códigos de estado más comunes
    - 🧩 Anatomía de una URL
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 02 · Servidor web mínimo con HttpServer
    - 📬 La idea en una frase
    - 🖥️ El servidor mínimo
    - 🕶️ Sé el Código, my friend…
    - 🚨 “Address already in use”
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 03 · Sirviendo HTML
    - 📬 La idea en una frase
    - 📄 HTML desde una cadena
    - 📂 HTML desde un fichero
    - 🕶️ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 04 · Parámetros GET y formularios POST
    - 📬 La idea en una frase
    - 📥 GET: la query string
    - 📤 POST: el cuerpo de la petición
    - 🧰 El parser casero
    - 🕶️ Sé el Código, my friend…
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 05 · Devolviendo JSON: como una API de verdad
    - 📬 La idea en una frase
    - 🧩 Qué es JSON (en 10 segundos)
    - 🚀 Servir JSON desde tu servidor
    - 📡 Consumirlo con fetch
    - 🕶️ Sé el Código, my friend…
    - 🎯 Mini-chequeo

    - ✅ Resumen en 3 frases
- 06 · Mini proyecto: gestor de tareas (API REST)
    - 📬 La idea en una frase
    - 🗂️ Qué es REST (en 20 segundos)
    - 🧱 El modelo y el almacén
    - 🔧 Los cuatro endpoints
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 07 · Consumir APIs externas con HttpClient
    - 📬 La idea en una frase
    - 📡 El trío sagrado del cliente
    - 🧩 Parsear JSON con Gson
    - 🌟 El ejemplo completo: los repos de GitHub
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 08 · HttpClient avanzado
    - 📬 La idea en una frase
    - 📮 POST: enviar datos a una API
    - 🛡️ Cabeceras, timeouts y manejo de errores
    - ⚡ Llamadas asíncronas con sendAsync
    - 🗑️ PUT, DELETE y otros métodos
    - 🏗️ Frameworks modernos (una ojeada)
    - 🎯 Mini-chequeo
    - ✅ Resumen en 3 frases
- 09 · Repaso interactivo: domina la web
    - 📬 La idea en una frase
    - ⭐ Sé el Código, my friend…
    - 🔥 Fireside Chat: GET vs POST
    - 🕵️ ¿Quién soy?
    - 👹 CONRAD VS EL MUNDO: “El puerto que no arranca”
    - 🎮 El juego de las decisiones
    - ⚡ Laboratorio de tortura: el servidor que no era
    - 🏆 Logros de esta unidad

    - 🧠 Atrévete a pensar
    - 🧩 Crucigrama de bits
    - 💬 Preguntas de entrevista de trabajo
    - 🤷 No hay preguntas tontas
    - 🎬 Poscréditos
- Boletín U01 — Inicial
    - Ejercicio 1: “Engaña a tu robot literal” 🤖🥪
    - Ejercicio 2: “Cazadores de bugs humanos” 🐛🎯
    - Ejercicio 3: “Tu protocolo de paz” 🛡️💬
    - Ejercicio 4: El filtro del hospital vs. el gimnasio (Abstracción)
    - Ejercicio 5: El desglosador de promociones (Descomposición)
    - Ejercicio 6: El test de la receta tapada (Metacognición)
    - Ejercicio 7: El Patrón del Cajero y el parking (Patrones)
    - Ejercicio 8: Mi manual para aprender a programar
- Boletín U02 — Inicial
    - Ejercicio 1: Desordena esto
    - Ejercicio 2: ¿Qué imprime?
    - Ejercicio 3: Cazador de errores
    - Ejercicio 4: Tu ficha personal
    - Ejercicio 5: Completa el programa
    - Ejercicio 6: Empareja conceptos
    - Ejercicio 7: El detective de errores
    - Ejercicio 8: Tu biografía
- Boletín U02 — Inicial Resuelto
    - Ejercicio 1: Desordena esto (Solución)
    - Ejercicio 2: ¿Qué imprime? (Solución)
    - Ejercicio 3: Cazador de errores (Solución)
    - Ejercicio 4: Tu ficha personal (Solución)
    - Ejercicio 5: Completa el programa (Solución)
    - Ejercicio 6: Empareja conceptos (Solución)
    - Ejercicio 7: El detective de errores (Solución)
    - Ejercicio 8: Tu biografía (Solución)
- Boletín U02 — Avanzado

    - Ejercicio 1: Calculadora de propinas
    - Ejercicio 2: Conversor dólar-euro
    - Ejercicio 3: ¿Qué imprime? — el casting traidor
    - Ejercicio 4: Interés compuesto (sin bucle)
    - Ejercicio 5: El enigma del post-incremento
    - 💡 Herramientas de ampliación (Operador ternario, .equals(), Scanner y Math.random())
    - Ejercicio 6: El duelo de dados (Extra)
    - Ejercicio 7: Supercalculadora con lógica (Extra)
    - Ejercicio 8: Javadoc de campeonato
- Boletín U02 — Avanzado Resuelto
    - Ejercicio 1: Calculadora de propinas (Solución)
    - Ejercicio 2: Conversor dólar-euro (Solución)
    - Ejercicio 3: ¿Qué imprime? — el casting traidor (Solución)
    - Ejercicio 4: Interés compuesto (sin bucle) (Solución)
    - Ejercicio 5: El enigma del post-incremento (Solución)
    - 💡 Herramientas de ampliación (Operador ternario, .equals(), Scanner y Math.random())
    - Ejercicio 6: El duelo de dados (Extra) (Solución)
    - Ejercicio 7: Supercalculadora con lógica (Extra) (Solución)
    - Ejercicio 8: Javadoc de campeonato (Solución)
- Boletín U02 — Extras
    - Ejercicio 1: CodeWars — Square(n) Sum (Extra)
    - Ejercicio 2: CodeWars — Grasshopper - Summation (Extra)
    - Ejercicio 3: AceptaElReto — 119 Escudos del ejército romano (Extra)
- Boletín U03 — Inicial
    - Ejercicio 1: conversor de temperaturas
    - Ejercicio 2: ¿Qué imprime? — incrementos
    - Ejercicio 3: calculadora de descuentos
    - Ejercicio 4: El tipo perfecto
    - Ejercicio 5: El casting asesino
    - Ejercicio 6: ¿Par o impar?
    - Ejercicio 7: String en acción

    - Ejercicio 8: saludo con Scanner
    - Ejercicio 9: CodeWars — Will you make it?
    - Ejercicio 10: ¿Qué imprime? — printf con conversiones
    - Ejercicio 11: la nota con dos decimales
    - Ejercicio 12: la propina con dos decimales
- Boletín U03 — Inicial Resuelto
    - Ejercicio 1: conversor de temperaturas
    - Ejercicio 2: ¿Qué imprime? — incrementos
    - Ejercicio 3: calculadora de descuentos
    - Ejercicio 4: El tipo perfecto
    - Ejercicio 5: El casting asesino
    - Ejercicio 6: ¿Par o impar?
    - Ejercicio 7: String en acción
    - Ejercicio 8: saludo con Scanner
    - Ejercicio 9: CodeWars — Will you make it?
    - Ejercicio 10: ¿Qué imprime? — printf con conversiones
    - Ejercicio 11: la nota con dos decimales
    - Ejercicio 12: la propina con dos decimales
- Boletín U03 — Avanzado
    - ⭐ Ejercicio 1: calculadora de propinas
    - ⭐ Ejercicio 2: conversor dólar-euro
    - ⭐⭐ Ejercicio 3: ¿Qué imprime? — el casting traidor
    - ⭐⭐ Ejercicio 4: interés compuesto (sin bucle)
    - ⭐⭐⭐ Ejercicio 5: El enigma del post-incremento
    - ⭐⭐ Ejercicio 6: El duelo de dados
    - ⭐⭐ Ejercicio 7: supercalculadora con lógica
    - ⭐⭐⭐ Ejercicio 8: CodeWars — Convert boolean values to strings ‘Yes’ or ‘No’
    - ⭐⭐⭐ Ejercicio 9: AceptaElReto — 114 Último dígito del factorial
    - ⭐⭐ Ejercicio 10: el ticket de compra con NumberFormat
    - ⭐⭐⭐ Ejercicio 11: la edad a prueba de bombas
    - 📚 Referencias
- Boletín U03 — Avanzado Resuelto
    - ⭐ Ejercicio 1: calculadora de propinas

    - ⭐ Ejercicio 2: conversor dólar-euro
    - ⭐⭐ Ejercicio 3: ¿Qué imprime? — el casting traidor
    - ⭐⭐ Ejercicio 4: interés compuesto (sin bucle)
    - ⭐⭐⭐ Ejercicio 5: El enigma del post-incremento
    - ⭐⭐ Ejercicio 6: El duelo de dados
    - ⭐⭐ Ejercicio 7: supercalculadora con lógica
    - ⭐⭐⭐ Ejercicio 8: CodeWars — Convert boolean values to strings 'Yes' or 'No'
    - ⭐⭐⭐ Ejercicio 9: AceptaElReto — 114 Último dígito del factorial
    - ⭐⭐ Ejercicio 10: el ticket de compra con NumberFormat
    - ⭐⭐⭐ Ejercicio 11: la edad a prueba de bombas
    - 📚 Referencias
- Boletín U03 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U04 — Inicial
    - Ejercicio 1: El portero del club
    - Ejercicio 2: ¿Qué imprime? — el semáforo invertido
    - Ejercicio 3: El menú del día
    - Ejercicio 4: La cuenta atrás del cohete
    - Ejercicio 5: La tabla del 7
    - Ejercicio 6: Solo los pares
    - Ejercicio 7: El detective de divisores
    - Ejercicio 8: La edad blindada
    - Ejercicio 9: CodeWars — Even or Odd
- Boletín U04 — Inicial Resuelto
    - Ejercicio 1: El portero del club
    - Ejercicio 2: ¿Qué imprime? — el semáforo invertido
    - Ejercicio 3: El menú del día
    - Ejercicio 4: La cuenta atrás del cohete
    - Ejercicio 5: La tabla del 7
    - Ejercicio 6: Solo los pares
    - Ejercicio 7: El detective de divisores
    - Ejercicio 8: La edad blindada

    - Ejercicio 9: CodeWars — Even or Odd
- Boletín U04 — Avanzado
    - ⭐ Ejercicio 1: La calculadora de notas
    - ⭐ Ejercicio 2: El menú que no se rinde
    - ⭐⭐ Ejercicio 3: ¿Qué imprime? — la pirámide
    - ⭐⭐ Ejercicio 4: La escalera de números
    - ⭐⭐⭐ Ejercicio 5: ¿Qué imprime? — break, continue y la etiqueta
    - ⭐⭐ Ejercicio 6: cazador de primos
    - ⭐⭐ Ejercicio 7: La suma centinela
    - ⭐⭐⭐ Ejercicio 8: CodeWars — Categorize New Member
    - ⭐⭐⭐ Ejercicio 9: AceptaElReto — 156 Ascensor
    - 📚 Referencias
- Boletín U04 — Avanzado Resuelto
    - ⭐ Ejercicio 1: La calculadora de notas
    - ⭐ Ejercicio 2: El menú que no se rinde
    - ⭐⭐ Ejercicio 3: ¿Qué imprime? — la pirámide
    - ⭐⭐ Ejercicio 4: La escalera de números
    - ⭐⭐⭐ Ejercicio 5: ¿Qué imprime? — break, continue y la etiqueta
    - ⭐⭐ Ejercicio 6: cazador de primos
    - ⭐⭐ Ejercicio 7: La suma centinela
    - ⭐⭐⭐ Ejercicio 8: CodeWars — Categorize New Member
    - ⭐⭐⭐ Ejercicio 9: AceptaElReto — 156 Ascensor
    - 📚 Referencias
- Boletín U04 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U05 — Inicial
    - Ejercicio 1: El saludo oficial
    - Ejercicio 2: ¿Qué pasa? — la variable fantasma
    - Ejercicio 3: La ficha de presentación
    - Ejercicio 4: El primer return
    - Ejercicio 5: Doble, por favor
    - Ejercicio 6: ¿Qué imprime? — el viaje de ida y vuelta

    - Ejercicio 7: ¿Par o impar?
    - Ejercicio 8: La puerta de la edad
    - Ejercicio 9: CodeWars — Century From Year
- Boletín U05 — Inicial Resuelto
    - Ejercicio 1: El saludo oficial
    - Ejercicio 2: ¿Qué pasa? — la variable fantasma
    - Ejercicio 3: La ficha de presentación
    - Ejercicio 4: El primer return
    - Ejercicio 5: Doble, por favor
    - Ejercicio 6: ¿Qué imprime? — el viaje de ida y vuelta
    - Ejercicio 7: ¿Par o impar?
    - Ejercicio 8: La puerta de la edad
    - Ejercicio 9: CodeWars — Century From Year
- Boletín U05 — Avanzado
    - ⭐ Ejercicio 1: El clasificador de notas
    - ⭐ Ejercicio 2: Tablas a demanda
    - ⭐⭐ Ejercicio 3: ¿Qué imprime? — la sombra del parámetro
    - ⭐⭐ Ejercicio 4: La suma de dígitos
    - ⭐⭐⭐ Ejercicio 5: ¿Qué imprime? — el return que corta
    - ⭐⭐⭐ Ejercicio 6: Divide el informe
    - ⭐⭐ Ejercicio 7: El contador de vocales
    - ⭐⭐⭐ Ejercicio 8: CodeWars — ‘Disemvowel’ Trolls
    - ⭐⭐⭐ Ejercicio 9: AceptaElReto — 165 Número hyperpar
    - 📚 Referencias
- Boletín U05 — Avanzado Resuelto
    - ⭐ Ejercicio 1: El clasificador de notas
    - ⭐ Ejercicio 2: Tablas a demanda
    - ⭐⭐ Ejercicio 3: ¿Qué imprime? — la sombra del parámetro
    - ⭐⭐ Ejercicio 4: La suma de dígitos
    - ⭐⭐⭐ Ejercicio 5: ¿Qué imprime? — el return que corta
    - ⭐⭐⭐ Ejercicio 6: Divide el informe
    - ⭐⭐ Ejercicio 7: El contador de vocales
    - ⭐⭐⭐ Ejercicio 8: CodeWars — ‘Disemvowel’ Trolls

    - ⭐ ⭐ ⭐ Ejercicio 9: AceptaElReto — 165 Número hyperpar
    - 📚 Referencias
- Boletín U05 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U06 — Inicial
    - Ejercicio 1: ¿Qué imprime? — Array de booleanos
    - Ejercicio 2: Encuentra el error — NullPointerException
    - Ejercicio 3: Completa el código — for básico para buscar el mayor
    - Ejercicio 4: Escribe este programa — contar números pares
    - Ejercicio 5: Encuentra el error — length vs length()
    - Ejercicio 6: ¿Qué imprime? — la suma de los impares
    - Ejercicio 7: Escribe este programa — búsqueda lineal
    - Ejercicio 8: Escribe este programa — el inverso
    - Ejercicio 9: Escribe este programa — la clase Arrays en acción
- Boletín U06 — Inicial Resuelto
    - Ejercicio 1: ¿Qué imprime? — Array de booleanos
    - Ejercicio 2: Encuentra el error — NullPointerException
    - Ejercicio 3: Completa el código — for básico para buscar el mayor
    - Ejercicio 4: Escribe este programa — contar números pares
    - Ejercicio 5: Encuentra el error — length vs length()
    - Ejercicio 6: ¿Qué imprime? — la suma de los impares
    - Ejercicio 7: Escribe este programa — búsqueda lineal
    - Ejercicio 8: Escribe este programa — el inverso
    - Ejercicio 9: Escribe este programa — la clase Arrays en acción
- Boletín U06 — Avanzado
    - ⭐ Ejercicio 1: La fusión de arrays ordenados
    - ⭐ Ejercicio 2: Rotación circular a la derecha
    - ⭐ Ejercicio 3: Suma de diagonales (matriz cuadrada)
    - ⭐ ⭐ Ejercicio 4: ¿Está ordenado?
    - ⭐ ⭐ Ejercicio 5: El inverso en el sitio
    - ⭐ ⭐ Ejercicio 6: ¿Qué imprime? — el doble bucle que cuenta parejas
    - ⭐ ⭐ ⭐ Ejercicio 7: Estadísticas de clase

    - ⭐ ⭐ ⭐ Ejercicio 8: El gran reto — buscaminas simplificado
    - ⭐ ⭐ ⭐ Ejercicio 9: Compactar — los ceros al final
- Boletín U06 — Avanzado Resuelto
    - ⭐ Ejercicio 1: La fusión de arrays ordenados
    - ⭐ Ejercicio 2: Rotación circular a la derecha
    - ⭐ Ejercicio 3: Suma de diagonales (matriz cuadrada)
    - ⭐ ⭐ Ejercicio 4: ¿Está ordenado?
    - ⭐ ⭐ Ejercicio 5: El inverso en el sitio
    - ⭐ ⭐ Ejercicio 6: ¿Qué imprime? — el doble bucle que cuenta parejas
    - ⭐ ⭐ ⭐ Ejercicio 7: Estadísticas de clase
    - ⭐ ⭐ ⭐ Ejercicio 8: El gran reto — buscaminas simplificado
    - ⭐ ⭐ ⭐ Ejercicio 9: Compactar — los ceros al final
- Boletín U06 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U07 — Inicial
    - Ejercicio 1: ¿Qué imprime? — la búsqueda de la gema
    - Ejercicio 2: El buscador de tesoros
    - Ejercicio 3: ¿Qué imprime? — la bombolla mínima
    - Ejercicio 4: La caja de zapatos ordenada
    - Ejercicio 5: El detective de la búsqueda binaria
    - Ejercicio 6: ¿Qué imprime? — la binaria que falla
    - Ejercicio 7: El contador de intercambios
    - Ejercicio 8: La nota más alta de la clase
    - Ejercicio 9: CodeWars — Find the smallest integer in the array
- Boletín U07 — Inicial Resuelto
    - Ejercicio 1: ¿Qué imprime? — la búsqueda de la gema
    - Ejercicio 2: El buscador de tesoros
    - Ejercicio 3: ¿Qué imprime? — la bombolla mínima
    - Ejercicio 4: La caja de zapatos ordenada
    - Ejercicio 5: El detective de la búsqueda binaria
    - Ejercicio 6: ¿Qué imprime? — la binaria que falla
    - Ejercicio 7: El contador de intercambios

    - Ejercicio 8: La nota más alta de la clase
    - Ejercicio 9: CodeWars — Find the smallest integer in the array
- Boletín U07 — Avanzado
    - ⭐ Ejercicio 1: ¿Qué imprime? — la binaria con historial
    - ⭐ Ejercicio 2: El buscador binario con historial
    - ⭐⭐ Ejercicio 3: La bombolla con recuento (y flag)
    - ⭐⭐ Ejercicio 4: La inserción descendente
    - ⭐⭐ Ejercicio 5: El analista de complejidad
    - ⭐⭐ Ejercicio 6: El cazador de parejas
    - ⭐⭐ Ejercicio 7: El detective de inversiones
    - ⭐⭐⭐ Ejercicio 8: CodeWars — Ones and Zeros
    - ⭐⭐⭐ Ejercicio 9: AceptaElReto — 100 Constante de Kaprekar
    - 📚 Referencias
- Boletín U07 — Avanzado Resuelto
    - Ejercicio 1: ¿Qué imprime? — la binaria con historial
    - Ejercicio 2: El buscador binario con historial
    - Ejercicio 3: La bombolla con recuento (y flag)
    - Ejercicio 4: La inserción descendente
    - Ejercicio 5: El analista de complejidad
    - Ejercicio 6: El cazador de parejas
    - Ejercicio 7: El detective de inversiones
    - Ejercicio 8: CodeWars — Ones and Zeros
    - Ejercicio 9: AceptaElReto — 100 Constante de Kaprekar
- Boletín U07 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U08 — Inicial
    - Ejercicio 1: El factorial del valiente
    - Ejercicio 2: La abuela suma el array
    - Ejercicio 3: ¿Qué imprime? — la cuenta atrás recursiva
    - Ejercicio 4: La potencia del aprendiz
    - Ejercicio 5: El detective de dígitos
    - Ejercicio 6: El espejo de letras

    - Ejercicio 7: ¿Qué imprime? — el enigma del stack
    - Ejercicio 8: El guardián de los palíndromos
    - Ejercicio 9: CodeWars — Factorial
- Boletín U08 — Inicial (Resuelto)
    - Ejercicio 1: El factorial del valiente
    - Ejercicio 2: La abuela suma el array
    - Ejercicio 3: ¿Qué imprime? — la cuenta atrás recursiva
    - Ejercicio 4: La potencia del aprendiz
    - Ejercicio 5: El detective de dígitos
    - Ejercicio 6: El espejo de letras
    - Ejercicio 7: ¿Qué imprime? — el enigma del stack
    - Ejercicio 8: El guardián de los palíndromos
    - Ejercicio 9: CodeWars — Factorial
- Boletín U08 — Avanzado
    - Ejercicio 1: ⭐ Fibonacci con el contador chivato
    - Ejercicio 2: ⭐⭐ ¿Qué imprime? — el árbol de llamadas
    - Ejercicio 3: ⭐⭐ El palíndromo rebelde
    - Ejercicio 4: ⭐⭐ La potencia exprés (divide y vencerás)
    - Ejercicio 5: ⭐⭐⭐ Quicksort con mediana de tres
    - Ejercicio 6: ⭐⭐⭐ Mergesort con el contador de comparaciones
    - Ejercicio 7: ⭐⭐⭐ las torres de Hanói con contador
    - Ejercicio 8: ⭐⭐⭐ CodeWars — Sort Numbers
    - Ejercicio 9: ⭐⭐⭐ AceptaElReto — 104 Móviles
- Boletín U08 — Avanzado (Resuelto)
    - Ejercicio 1: ⭐ Fibonacci con el contador chivato
    - Ejercicio 2: ⭐⭐ ¿Qué imprime? — el árbol de llamadas
    - Ejercicio 3: ⭐⭐ El palíndromo rebelde
    - Ejercicio 4: ⭐⭐ La potencia exprés (divide y vencerás)
    - Ejercicio 5: ⭐⭐⭐ Quicksort con mediana de tres
    - Ejercicio 6: ⭐⭐⭐ Mergesort con el contador de comparaciones
    - Ejercicio 7: ⭐⭐⭐ las torres de Hanói con contador
    - Ejercicio 8: ⭐⭐⭐ CodeWars — Sort Numbers
    - Ejercicio 9: ⭐⭐⭐ AceptaElReto — 104 Móviles

- Boletín U08 — Extras
  - CodeWars
  - AceptaElReto
- Boletín U09 — Inicial
  - Ejercicio 1: El perro que habla
  - Ejercicio 2: El teléfono con batería
  - Ejercicio 3: El estudiante con nota
  - Ejercicio 4: La película en cartelera
  - Ejercicio 5: La persona educada
  - Ejercicio 6: El círculo calculador
  - Ejercicio 7: El videojuego jugable
  - Ejercicio 8: ¿Qué imprime? — el taller de coches
  - Ejercicio 9: CodeWars — Object Oriented Piracy
- Boletín U09 — Inicial Resuelto
  - Ejercicio 1: El perro que habla
  - Ejercicio 2: El teléfono con batería
  - Ejercicio 3: El estudiante con nota
  - Ejercicio 4: La película en cartelera
  - Ejercicio 5: La persona educada
  - Ejercicio 6: El círculo calculador
  - Ejercicio 7: El videojuego jugable
  - Ejercicio 8: ¿Qué imprime? — el taller de coches
  - Ejercicio 9: CodeWars — Object Oriented Piracy
- Boletín U09 — Avanzado
  - ⭐ Ejercicio 1: La biblioteca
  - ⭐ Ejercicio 2: El rectángulo razonador
  - ⭐ Ejercicio 3: La cuenta bancaria blindada
  - ⭐⭐ Ejercicio 4: La hora que se corrige sola
  - ⭐⭐ Ejercicio 5: ¿Qué imprime? — el baile de referencias
  - ⭐⭐ Ejercicio 6: El correo que se encadena
  - ⭐⭐ Ejercicio 7: La fracción que se simplifica
  - ⭐⭐⭐ Ejercicio 8: CodeWars — Building blocks
  - ⭐⭐⭐ Ejercicio 9: CodeWars — Object Oriented Piracy

    - 📚 Referencias
- Boletín U09 — Avanzado Resuelto
    - ⭐ Ejercicio 1: La biblioteca
    - ⭐ Ejercicio 2: El rectángulo razonador
    - ⭐ Ejercicio 3: La cuenta bancaria blindada
    - ⭐⭐ Ejercicio 4: La hora que se corrige sola
    - ⭐⭐ Ejercicio 5: ¿Qué imprime? — el baile de referencias
    - ⭐⭐ Ejercicio 6: El correo que se encadena
    - ⭐⭐ Ejercicio 7: La fracción que se simplifica
    - ⭐⭐⭐ Ejercicio 8: CodeWars — Building blocks
    - ⭐⭐⭐ Ejercicio 9: CodeWars — Object Oriented Piracy
- Boletín U09 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U10 — Inicial
    - Ejercicio 1: La casa de cristal
    - Ejercicio 2: El coche del vecindario
    - Ejercicio 3: El termómetro con cerebro
    - Ejercicio 4: getter sin setter
    - Ejercicio 5: El contador de la clase
    - Ejercicio 6: La calculadora sin pilas
    - Ejercicio 7: las constantes del barrio
    - Ejercicio 8: ¿Qué imprime? — el puzle estático
    - Ejercicio 9: CodeWars — Square(n) Sum
- Boletín U10 — Inicial Resuelto
    - Ejercicio 1: La casa de cristal
    - Ejercicio 2: El coche del vecindario
    - Ejercicio 3: El termómetro con cerebro
    - Ejercicio 4: getter sin setter
    - Ejercicio 5: El contador de la clase
    - Ejercicio 6: La calculadora sin pilas
    - Ejercicio 7: las constantes del barrio
    - Ejercicio 8: ¿Qué imprime? — el puzle estático

    - Ejercicio 9: CodeWars — Square(n) Sum
- Boletín U10 — Avanzado
    - ⭐ Ejercicio 1: empleado con validación
    - ⭐ Ejercicio 2: Círculo encapsulado
    - ⭐⭐ Ejercicio 3: JavaBean Alumno
    - ⭐⭐ Ejercicio 4: hora inmutable
    - ⭐⭐ Ejercicio 5: ¿Qué imprime? — el puzle de los gatos
    - ⭐⭐ Ejercicio 6: contador de usuarios
    - ⭐⭐⭐ Ejercicio 7: La clase utilitaria OperacionesArray
    - ⭐⭐⭐ Ejercicio 8: validador de datos
    - ⭐⭐⭐ Ejercicio 9: El gran reto — refactoriza el banco
- Boletín U10 — Avanzado Resuelto
    - ⭐ Ejercicio 1: empleado con validación
    - ⭐ Ejercicio 2: Círculo encapsulado
    - ⭐⭐ Ejercicio 3: JavaBean Alumno
    - ⭐⭐ Ejercicio 4: hora inmutable
    - ⭐⭐ Ejercicio 5: ¿Qué imprime? — el puzle de los gatos
    - ⭐⭐ Ejercicio 6: contador de usuarios
    - ⭐⭐⭐ Ejercicio 7: La clase utilitaria OperacionesArray
    - ⭐⭐⭐ Ejercicio 8: validador de datos
    - ⭐⭐⭐ Ejercicio 9: El gran reto — refactoriza el banco
- Boletín U10 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U11 — Inicial
    - Ejercicio 1: ¿Qué imprime? — La familia musical
    - Ejercicio 2: Encuentra el error — extends mal usado
    - Ejercicio 3: Completa el código — el gato que llama a su padre
    - Ejercicio 4: Escribe este programa — la herencia de vehículos
    - Ejercicio 5: ¿Qué imprime? — polimorfismo con referencias
    - Ejercicio 6: Escribe este programa — la granja polimórfica
    - Ejercicio 7: Encuentra el error — @Override que no lo es
    - Ejercicio 8: Escribe este programa — el perro bien heredado

    - Ejercicio 9: ¿Qué imprime? — la cadena de constructores
- Boletín U11 — Inicial Resuelto
    - Ejercicio 1: ¿Qué imprime? — La familia musical
    - Ejercicio 2: Encuentra el error — extends mal usado
    - Ejercicio 3: Completa el código — el gato que llama a su padre
    - Ejercicio 4: Escribe este programa — la herencia de vehículos
    - Ejercicio 5: ¿Qué imprime? — polimorfismo con referencias
    - Ejercicio 6: Escribe este programa — la granja polimórfica
    - Ejercicio 7: Encuentra el error — @Override que no lo es
    - Ejercicio 8: Escribe este programa — el perro bien heredado
    - Ejercicio 9: ¿Qué imprime? — la cadena de constructores
- Boletín U11 — Avanzado
    - ⭐ Ejercicio 1: Interfaz FiguraGeometrica
    - ⭐ Ejercicio 2: Jerarquía de Empleados
    - ⭐⭐ Ejercicio 3: Sistema de pagos con interfaz
    - ⭐⭐ Ejercicio 4: Interfaces múltiples: Volador y Nadador
    - ⭐⭐ Ejercicio 5: Downcasting seguro
    - ⭐⭐ Ejercicio 6: Calculadora de figuras con clase abstracta
    - ⭐⭐⭐ Ejercicio 7: Sistema de notificaciones polimórfico
    - ⭐⭐⭐ Ejercicio 8: Template method — las bebidas
    - ⭐⭐⭐ Ejercicio 9: El gran reto — vehículos con combustible
- Boletín U11 — Avanzado Resuelto
    - ⭐ Ejercicio 1: Interfaz FiguraGeometrica
    - ⭐ Ejercicio 2: Jerarquía de Empleados
    - ⭐⭐ Ejercicio 3: Sistema de pagos con interfaz
    - ⭐⭐ Ejercicio 4: Interfaces múltiples: Volador y Nadador
    - ⭐⭐ Ejercicio 5: Downcasting seguro
    - ⭐⭐ Ejercicio 6: Calculadora de figuras con clase abstracta
    - ⭐⭐⭐ Ejercicio 7: Sistema de notificaciones polimórfico
    - ⭐⭐⭐ Ejercicio 8: Template method — las bebidas
    - ⭐⭐⭐ Ejercicio 9: El gran reto — vehículos con combustible
- Boletín U11 — Extras
    - CodeWars

    - AceptaElReto
- Boletín U12 — Inicial
    - Ejercicio 1: ¿Qué imprime? — ArrayList remove por índice vs valor
    - Ejercicio 2: Encuentra el error — size() vs length vs length()
    - Ejercicio 3: Completa el código — for-each que suma una lista
    - Ejercicio 4: Escribe este programa — la lista de la compra
    - Ejercicio 5: ¿Qué imprime? — el HashSet que no deja repetir
    - Ejercicio 6: Escribe este programa — sin duplicados
    - Ejercicio 7: Completa el código — tu primera clase genérica
    - Ejercicio 8: ¿Qué imprime? — HashMap con put repetido
    - Ejercicio 9: Escribe este programa — contador de palabras con HashMap
- Boletín U12 — Inicial Resuelto
    - Ejercicio 1: ¿Qué imprime? — ArrayList remove por índice vs valor
    - Ejercicio 2: Encuentra el error — size() vs length vs length()
    - Ejercicio 3: Completa el código — for-each que suma una lista
    - Ejercicio 4: Escribe este programa — la lista de la compra
    - Ejercicio 5: ¿Qué imprime? — el HashSet que no deja repetir
    - Ejercicio 6: Escribe este programa — sin duplicados
    - Ejercicio 7: Completa el código — tu primera clase genérica
    - Ejercicio 8: ¿Qué imprime? — HashMap con put repetido
    - Ejercicio 9: Escribe este programa — contador de palabras con HashMap
- Boletín U12 — Avanzado
    - ⭐ Ejercicio 1: La cola del supermercado con LinkedList
    - ⭐⭐ Ejercicio 2: Intersección y unión de conjuntos
    - ⭐⭐ Ejercicio 3: Eliminar duplicados manteniendo el orden
    - ⭐⭐⭐ Ejercicio 4: ¿Qué imprime? — el remove que rompe el baile
    - ⭐⭐ Ejercicio 5: Filtrar con Iterator
    - ⭐⭐ Ejercicio 6: El TreeSet ordenado
    - ⭐ Pila genérica : Ejercicio 7
    - ⭐⭐ Ejercicio 8: HashMap inverso
    - ⭐⭐ Ejercicio 9: TreeMap — frecuencia de letras
    - ⭐⭐⭐ Ejercicio 10: Wildcards — suma y mezcla de números
    - ⭐⭐⭐ Ejercicio 11: Sistema de votaciones con método genérico

    - ⭐ ⭐ ⭐ Ejercicio 12: el type erasure al descubierto
- Boletín U12 — Avanzado Resuelto
    - ⭐ Ejercicio 1: La cola del supermercado con LinkedList
    - ⭐ ⭐ Ejercicio 2: Intersección y unión de conjuntos
    - ⭐ ⭐ Ejercicio 3: Eliminar duplicados manteniendo el orden
    - ⭐ ⭐ ⭐ Ejercicio 4: ¿Qué imprime? — el remove que rompe el baile
    - ⭐ ⭐ Ejercicio 5: Filtrar con Iterator
    - ⭐ ⭐ Ejercicio 6: El TreeSet ordenado
    - ⭐ Pila genérica : Ejercicio 7
    - ⭐ ⭐ Ejercicio 8: HashMap inverso
    - ⭐ ⭐ Ejercicio 9: TreeMap — frecuencia de letras
    - ⭐ ⭐ ⭐ Ejercicio 10: Wildcards — suma y mezcla de números
    - ⭐ ⭐ ⭐ Ejercicio 11: Sistema de votaciones con método genérico
    - ⭐ ⭐ ⭐ Ejercicio 12: el type erasure al descubierto
- Boletín U12 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U13 — Inicial
    - Ejercicio 1: Completa el código — tu primera lambda
    - Ejercicio 2: ¿Qué imprime? — el orden de la flecha
    - Ejercicio 3: Encuentra el error — la lambda mal vestida
    - Ejercicio 4: Escribe este programa — filtrar pares con streams
    - Ejercicio 5: ¿Qué imprime? — el pipeline básico
    - Ejercicio 6: Completa el código — mayúsculas con map
    - Ejercicio 7: Escribe este programa — longitud de cada palabra
    - Ejercicio 8: Encuentra el error — la cinta que nunca arranca
    - Ejercicio 9: Completa el código — un Consumer para imprimir
- Boletín U13 — Inicial Resuelto
    - Ejercicio 1: Completa el código — tu primera lambda
    - Ejercicio 2: ¿Qué imprime? — el orden de la flecha
    - Ejercicio 3: Encuentra el error — la lambda mal vestida
    - Ejercicio 4: Escribe este programa — filtrar pares con streams
    - Ejercicio 5: ¿Qué imprime? — el pipeline básico

    - Ejercicio 6: Completa el código — mayúsculas con map
    - Ejercicio 7: Escribe este programa — longitud de cada palabra
    - Ejercicio 8: Encuentra el error — la cinta que nunca arranca
    - Ejercicio 9: Completa el código — un Consumer para imprimir
- Boletín U13 — Avanzado
    - ⭐ ⭐ Ejercicio 1: Ordenar con referencias a método
    - ⭐ ⭐ Ejercicio 2: Agrupar palabras por su primera letra
    - ⭐ ⭐ Ejercicio 3: Optional — el que no se deja engañar
    - ⭐ ⭐ ⭐ Ejercicio 4: El pipeline completo
    - ⭐ ⭐ Ejercicio 5: De lista a mapa con toMap
    - ⭐ ⭐ ⭐ Ejercicio 6: El máximo con reduce y comparador
    - ⭐ ⭐ Ejercicio 7: Frecuencias con groupingBy
    - ⭐ ⭐ ⭐ Ejercicio 8: Optional y streams, la pareja
    - ⭐ ⭐ ⭐ Ejercicio 9: el stream que se niega a morir
- Boletín U13 — Avanzado Resuelto
    - ⭐ ⭐ Ejercicio 1: Ordenar con referencias a método
    - ⭐ ⭐ Ejercicio 2: Agrupar palabras por su primera letra
    - ⭐ ⭐ Ejercicio 3: Optional — el que no se deja engañar
    - ⭐ ⭐ ⭐ Ejercicio 4: El pipeline completo
    - ⭐ ⭐ Ejercicio 5: De lista a mapa con toMap
    - ⭐ ⭐ ⭐ Ejercicio 6: El máximo con reduce y comparador
    - ⭐ ⭐ Ejercicio 7: Frecuencias con groupingBy
    - ⭐ ⭐ ⭐ Ejercicio 8: Optional y streams, la pareja
    - ⭐ ⭐ ⭐ Ejercicio 9: el stream que se niega a morir
- Boletín U13 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U14 — Inicial
    - Ejercicio 1: Encuentra el error — IOException sin capturar
    - Ejercicio 2: Completa el código — try-with-resources
    - Ejercicio 3: Escribe este programa — guardar ciudades en un archivo
    - Ejercicio 4: Encuentra el error — File.createNewFile sin comprobar
    - Ejercicio 5: ¿Qué imprime? — el contador de líneas

    - Ejercicio 6: Escribe este programa — el diario personal
    - Ejercicio 7: ¿Qué imprime? — matches() vs find()
    - Ejercicio 8: Escribe este programa — contar palabras con split
    - Ejercicio 9: Escribe este programa — contar dígitos con Matcher
- Boletín U14 — Inicial Resuelto
    - Ejercicio 1: Encuentra el error — IOException sin capturar
    - Ejercicio 2: Completa el código — try-with-resources
    - Ejercicio 3: Escribe este programa — guardar ciudades en un archivo
    - Ejercicio 4: Encuentra el error — File.createNewFile sin comprobar
    - Ejercicio 5: ¿Qué imprime? — el contador de líneas
    - Ejercicio 6: Escribe este programa — el diario personal
    - Ejercicio 7: ¿Qué imprime? — matches() vs find()
    - Ejercicio 8: Escribe este programa — contar palabras con split
    - Ejercicio 9: Escribe este programa — contar dígitos con Matcher
- Boletín U14 — Avanzado
    - ⭐ Ejercicio 1: Buscador de archivos por extensión
    - ⭐ Ejercicio 2: Lector de CSV con Scanner
    - ⭐⭐ Ejercicio 3: Filtro de líneas por palabra clave
    - ⭐⭐ Ejercicio 4: Separador de líneas pares e impares
    - ⭐⭐ Ejercicio 5: Split con regex — analizador de frases
    - ⭐⭐⭐ Ejercicio 6: Validador de datos con regex
    - ⭐⭐⭐ Ejercicio 7: Cifrado César con archivos
    - ⭐⭐⭐ Ejercicio 8: Serialización de estudiantes
    - ⭐⭐ Ejercicio 9: El contador de líneas, palabras y caracteres
- Boletín U14 — Avanzado Resuelto
    - ⭐ Ejercicio 1: Buscador de archivos por extensión
    - ⭐ Ejercicio 2: Lector de CSV con Scanner
    - ⭐⭐ Ejercicio 3: Filtro de líneas por palabra clave
    - ⭐⭐ Ejercicio 4: Separador de líneas pares e impares
    - ⭐⭐ Ejercicio 5: Split con regex — analizador de frases
    - ⭐⭐⭐ Ejercicio 6: Validador de datos con regex
    - ⭐⭐⭐ Ejercicio 7: Cifrado César con archivos
    - ⭐⭐⭐ Ejercicio 8: Serialización de estudiantes

    - ⭐⭐ Ejercicio 9: El contador de líneas, palabras y caracteres
- Boletín U14 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U15 — Inicial
    - Ejercicio 1: ¿Qué necesitas para usar JDBC?
    - Ejercicio 2: Completa el código — la conexión
    - Ejercicio 3: ¿Qué imprime? — ResultSet vacío
    - Ejercicio 4: Encuentra el error — SQLException sin manejo
    - Ejercicio 5: Escribe este programa — la primera conexión
    - Ejercicio 6: ¿Qué imprime? — executeQuery en UPDATE
    - Ejercicio 7: Completa el código — INSERT con PreparedStatement
    - Ejercicio 8: Escribe este programa — listar con try-with-resources
    - Ejercicio 9: Encuentra el error — índices del PreparedStatement
- Boletín U15 — Inicial Resuelto
    - Ejercicio 1: ¿Qué necesitas para usar JDBC?
    - Ejercicio 2: Completa el código — la conexión
    - Ejercicio 3: ¿Qué imprime? — ResultSet vacío
    - Ejercicio 4: Encuentra el error — SQLException sin manejo
    - Ejercicio 5: Escribe este programa — la primera conexión
    - Ejercicio 6: ¿Qué imprime? — executeQuery en UPDATE
    - Ejercicio 7: Completa el código — INSERT con PreparedStatement
    - Ejercicio 8: Escribe este programa — listar con try-with-resources
    - Ejercicio 9: Encuentra el error — índices del PreparedStatement
- Boletín U15 — Avanzado
    - ⭐ Ejercicio 1: Conexión desde archivo de propiedades
    - ⭐ Ejercicio 2: INSERT con clave autogenerada
    - ⭐ Ejercicio 3: UPDATE condicional
    - ⭐⭐ Ejercicio 4: INNER JOIN con PreparedStatement
    - ⭐⭐ Ejercicio 5: Búsqueda con LIKE
    - ⭐⭐ Ejercicio 6: Fechas en JDBC
    - ⭐⭐ Ejercicio 7: Batch INSERT — 100 alumnos de prueba
    - ⭐⭐⭐ Ejercicio 8: El patrón DAO

    - ⭐⭐⭐ Ejercicio 9: Transacción bancaria atómica
- Boletín U15 — Avanzado Resuelto
    - ⭐ Ejercicio 1: Conexión desde archivo de propiedades
    - ⭐ Ejercicio 2: INSERT con clave autogenerada
    - ⭐ Ejercicio 3: UPDATE condicional
    - ⭐⭐ Ejercicio 4: INNER JOIN con PreparedStatement
    - ⭐⭐ Ejercicio 5: Búsqueda con LIKE
    - ⭐⭐ Ejercicio 6: Fechas en JDBC
    - ⭐⭐ Ejercicio 7: Batch INSERT — 100 alumnos de prueba
    - ⭐⭐⭐ Ejercicio 8: El patrón DAO
    - ⭐⭐⭐ Ejercicio 9: Transacción bancaria atómica
- Boletín U15 — Extras
    - CodeWars
    - AceptaElReto
- Boletín U15 — Inicial
    - Ejercicio 1: Escribe este programa — Hola, mundo web
    - Ejercicio 2: Escribe este programa — servidor de la hora
    - Ejercicio 3: Escribe este programa — página que dice tu nombre
    - Ejercicio 4: Escribe este programa — contador de visitas global
    - Ejercicio 5: Escribe este programa — generador de excusas para entregas tarde
    - Ejercicio 6: Escribe este programa — tabla de multiplicar personalizada
    - Ejercicio 7: Escribe este programa — conversor de euros a pesetas (sí, pesetas)
    - Ejercicio 8: Escribe este programa — página de estado del servidor
    - Ejercicio 9: Encuentra el error — el Content-Type olvidado
- Boletín U15 — Inicial Resuelto
    - Ejercicio 1: Escribe este programa — Hola, mundo web
    - Ejercicio 2: Escribe este programa — servidor de la hora
    - Ejercicio 3: Escribe este programa — página que dice tu nombre
    - Ejercicio 4: Escribe este programa — contador de visitas global
    - Ejercicio 5: Escribe este programa — generador de excusas para entregas tarde
    - Ejercicio 6: Escribe este programa — tabla de multiplicar personalizada
    - Ejercicio 7: Escribe este programa — conversor de euros a pesetas
    - Ejercicio 8: Escribe este programa — página de estado del servidor

    - Ejercicio 9: Encuentra el error — el Content-Type olvidado
- Boletín U15 — Avanzado
    - ⭐ Ejercicio 1: API de frases motivacionales
    - ⭐ Ejercicio 2: Formulario de contacto con POST
    - ⭐ Ejercicio 3: Piedra, papel, tijera online
    - ⭐ ⭐ Ejercicio 4: El tiempo que NO hace
    - ⭐ ⭐ Ejercicio 5: Traductor chungo (pero funcional)
    - ⭐ ⭐ Ejercicio 6: API REST de tareas con prioridad
    - ⭐ ⭐ Ejercicio 7: Cliente GET — los repos de GitHub
    - ⭐ ⭐ Ejercicio 8: Cliente POST — crear una publicación en jsonplaceholder
    - ⭐ ⭐ ⭐ Ejercicio 9: Middleware de logging
    - ⭐ ⭐ ⭐ Ejercicio 10: la petición asíncrona con sendAsync
- Boletín U15 — Avanzado Resuelto
    - ⭐ Ejercicio 1: API de frases motivacionales
    - ⭐ Ejercicio 2: Formulario de contacto con POST
    - ⭐ Ejercicio 3: Piedra, papel, tijera online
    - ⭐ ⭐ Ejercicio 4: El tiempo que NO hace
    - ⭐ ⭐ Ejercicio 5: Traductor chungo (pero funcional)
    - ⭐ ⭐ Ejercicio 6: API REST de tareas con prioridad
    - ⭐ ⭐ Ejercicio 7: Cliente GET — los repos de GitHub
    - ⭐ ⭐ Ejercicio 8: Cliente POST — crear una publicación en jsonplaceholder
    - ⭐ ⭐ ⭐ Ejercicio 9: Middleware de logging
    - ⭐ ⭐ ⭐ Ejercicio 10: la petición asíncrona con sendAsync
- Boletín U15 — Extras
    - CodeWars
    - AceptaElReto

Inicio

APUNTES

Programación Java

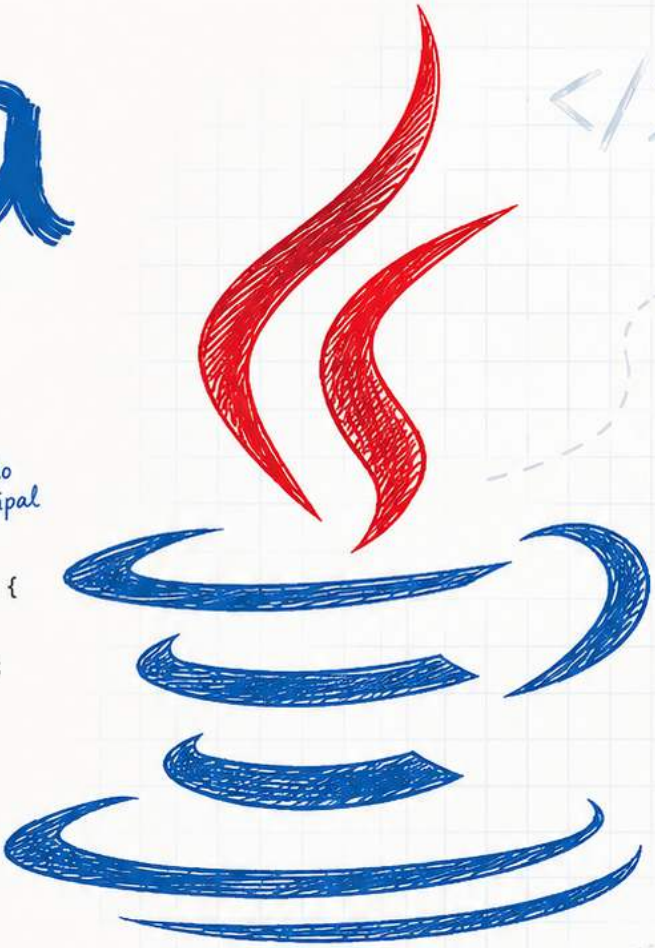
— DAM/DAW —

```java
public class HolaMundo {
    public static void main(String[] args) {
        // Paso de entrada del programa
        System.out.println("¡Hola, Java!");
    }
}
```

clase

método principal

imprime en consola



Por Sergi García Barea

licencia CC BY SA

🚀 Empezar por la Unidad 2

🧠 Primero: Introducción al Pensamiento Computacional

Ver en GitHub



Sergi Garcia Barea — CC BY-SA 4.0

⬇️ ¿Prefieres estudiar sin conexión? Descarga el curso completo

PDF (Castellano) PDF (Valencià) EPUB (Castellano) EPUB (Valencià)

Unidades

UNIDAD 1 RA1

Introducción al Pensamiento Computacional

Antes de código, pensemos. Descomposición, debugging, protocolo de paz y tu primer problema de programación sin tocar Java.

👉 Ver unidad 👈

UNIDAD 2 RA1

Introducción a Java y el entorno de trabajo

Primer contacto con Java: instala el JDK, escribe tu primer programa, conoce el método `main`, los comentarios y los argumentos de línea de comandos.

👉 Ver unidad 👈

UNIDAD 3 RA2

Variables, tipos y operadores

Declara variables, usa tipos primitivos, operadores aritméticos y lógicos, conversiones de tipo y lee datos por teclado con `Scanner`.

👉 Ver unidad 👈

UNIDAD 4 RA3

Estructuras de control y excepciones básicas

Domina `if` / `else`, `switch`, bucles `while` y `for`, y maneja excepciones con `try` / `catch` para que tu programa nunca se rompa.

👉 Ver unidad 👈

UNIDAD 5 RA2

UNIDAD 6 RA6

Funciones y métodos

Pasa de escribir código suelto a diseñarlo: métodos con parámetros, `return` con sentido y variables con su ámbito.

👉 Ver unidad 👈

Arrays

Los arrays son el aparcamiento de datos: declara, recorre, ordena y manipula colecciones de datos con arrays unidimensionales, multidimensionales y la clase `Arrays`.

👉 Ver unidad 👈

UNIDAD 7 RA2, RA6

Algorítmica I: Fundamentos

Aprende a pensar como un programador: divide problemas en partes, usa pseudocódigo, diagramas de flujo y crea funciones reutilizables.

👉 Ver unidad 👈

UNIDAD 8 RA2, RA6

Algorítmica II: técnicas avanzadas y modularidad

Algoritmos de ordenación, búsqueda binaria, recursividad y técnicas divide y vencerás para resolver problemas más complejos.

👉 Ver unidad 👈

UNIDAD 9 RA2, RA4

POO: clases y objetos

Programación Orientada a Objetos: crea clases, instancia objetos, define atributos y métodos, y entiende la magia de los constructores.

👉 Ver unidad 👈

UNIDAD 10 RA4

Visibilidad, encapsulación y static

Controla quién ve qué: modificadores de acceso (`public` , `private` , `protected`), encapsulación con getters/setters, miembros `static` y constantes.

👉 Ver unidad 👈

UNIDAD 11 RA4, RA7

Herencia, polimorfismo e interfaces

Herencia, polimorfismo, clases abstractas e interfaces: la base del diseño flexible y reutilizable en Java. Aprende a sobrescribir métodos y a usar `super` .

👉 Ver unidad 👈

UNIDAD 12 RA6

Colecciones, genéricos y mapas

El Java Collections Framework completo: `ArrayList` , `LinkedList` , `HashSet` , `TreeSet` , `Iterator` , `Collections` , genéricos (`Caja<T>` , wildcards) y mapas (`HashMap` , `TreeMap`).

👉 Ver unidad 👈

UNIDAD 13 RA6, RA7

Programación funcional (Streams y Lambdas)

Lambdas, interfaces funcionales, referencias a métodos, la Stream API (`filter` , `map` , `reduce` ,

UNIDAD 14 RA5

Ficheros y expresiones regulares (Regex)

Lectura y escritura de ficheros de texto y binarios, serialización de objetos y expresiones regulares

`collect`) y `Optional` para escribir Java moderno y expresivo.

👉 Ver unidad 👈

para buscar, validar y reemplazar patrones en texto.

👉 Ver unidad 👈

UNIDAD 15 RA9

Persistencia de datos: JDBC e introducción a ORM

Conecta Java con bases de datos relacionales usando JDBC: `Connection`, `Statement`, consultas, inserciones, actualizaciones y transacciones seguras.

👉 Ver unidad 👈

UNIDAD 16 (EXTRA) RA5

Servir y consumir APIs con Web

Crea un servidor HTTP con Java `HttpServer`, sirve páginas HTML/JS, atiende peticiones JSON, implementa una API REST completa desde cero.

👉 Ver unidad 👈

Boletines

UNIDAD 1

Boletines de la unidad

- Inicial por resolver

UNIDAD 2

Boletines de la unidad

- Inicial por resolver
- Inicial resuelto
- Avanzado por resolver
- Avanzado resuelto
- Extras

UNIDAD 3

Boletines de la unidad

- Inicial por resolver
- Inicial resuelto
- Avanzado por resolver
- Avanzado resuelto
- Extras

UNIDAD 4

Boletines de la unidad

- Inicial por resolver
- Inicial resuelto
- Avanzado por resolver
- Avanzado resuelto
- Extras

UNIDAD 5

Boletines de la unidad

- Inicial por resolver
- Inicial resuelto

UNIDAD 6

Boletines de la unidad

- Inicial por resolver
- Inicial resuelto

⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 7

Boletines de la unidad

🟢 Inicial por resolver
✅ Inicial resuelto
⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 8

Boletines de la unidad

🟢 Inicial por resolver
✅ Inicial resuelto
⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 9

Boletines de la unidad

🟢 Inicial por resolver
✅ Inicial resuelto
⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 10

Boletines de la unidad

🟢 Inicial por resolver
✅ Inicial resuelto
⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 11

Boletines de la unidad

🟢 Inicial por resolver
✅ Inicial resuelto
⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 12

Boletines de la unidad

🟢 Inicial por resolver
✅ Inicial resuelto
⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 13

Boletines de la unidad

🟢 Inicial por resolver
✅ Inicial resuelto
⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 14

Boletines de la unidad

🟢 Inicial por resolver
✅ Inicial resuelto
⭐ Avanzado por resolver
💪 Avanzado resuelto
🔥 Extras

UNIDAD 15

Boletines de la unidad

- Inicial por resolver
- Inicial resuelto
- Avanzado por resolver
- Avanzado resuelto
- Extras

UNIDAD 16 (EXTRA)

Boletines de la unidad

- Inicial por resolver
- Inicial resuelto
- Avanzado por resolver
- Avanzado resuelto
- Extras

Licencia



CC BY-SA 4.0 — *Sergi Garcia Barea*

Licencia Creative Commons Atribución-CompartirIgual 4.0 Internacional

Puedes compartir y adaptar el material para cualquier propósito, incluso comercial, siempre que reconozcas la autoría y compartas las modificaciones bajo la misma licencia.

U01 — Introducción al Pensamiento Computacional

Antes de código, pensemos. La mente del programador antes de Java 🧠

Bienvenido/a a la primera unidad del curso. Aquí no escribiremos ni una sola línea de Java. Todavía no.

Antes de aprender a programar, hay que aprender a **pensar como un programador**: descomponer problemas, entender cómo "ve" un ordenador el código, no tener miedo al texto rojo y saber usar la IA sin que nos sustituya. Esta unidad es el mapa mental que necesitas antes de abrir el editor de código.

Esta unidad se lee como un **libro de 9 capítulos**: cada punto desarrolla una idea completa y enlaza con el siguiente.

🎯 Objetivo de la unidad

Al terminar, serás capaz de explicar:

- Qué es un ordenador y cómo "entiende" el código (binario, compiladores, intérpretes).
- Por qué programar no es memorizar código sino resolver problemas.
- El flujo sagrado del programador: pensar → papel → código.
- La técnica de la **descomposición** para no colapsar ante problemas grandes.
- A mirar los errores como un mapa, no como un fracaso.
- El **protocolo de paz** antes de pedir ayuda.
- La regla del 80/20: menos leer, más intentar.
- A usar la IA como tutor socrático, no como copia.
- Resolver tu primer problema de programación **sin tocar Java**.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Qué es un ordenador y un lenguaje de programación	Binario, compiladores vs intérpretes, JVM, terminología clave	Todos
02 · Programar no es memorizar código	Espejo de la comprensión, pantalla en blanco, tutorial hell	Todos
03 · Primero pensamos, después escribimos	Literalidad, los 4 pasos sagrados, ejercicio del robot	Todos
04 · La descomposición	Divide y vencerás, el arte de partir problemas	Todos
05 · Debugging y casos límite	El texto rojo es tu amigo, edge cases	Todos
06 · El protocolo de paz	4 pasos antes de pedir ayuda en el foro	Todos
07 · Practicar (menos mirar, más intentar)	Regla 80/20, hábitos diarios del programador	Todos
08 · Cómo utilizaremos la IA	Tutor socrático, alucinaciones, reglas de oro	Todos
09 · Mi primer problema (puente a Java)	Comparador de números, pseudocódigo, glosario	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Entrena la mente de programador/a sin una sola línea de Java. Son ejercicios no evaluables, pero son la mejor manera de romper el hielo y compartir ideas en el foro.

🟢 Inicial por resolver

✅ Criterios de evaluación cubiertos (RA1)

RA1: Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

CE	Criterio	Dónde se cubre
a)	Bloques que componen la estructura de un programa	✅ Puntos 3 y 4
b)	Proyectos de desarrollo de aplicaciones	✅ Punto 9

> 📌 El criterio a) se practica descomponiendo y escribiendo el plan antes de programar; el b), cerrando el primer problema completo del punto 9. Los criterios restantes del RA1 (c: entornos integrados de desarrollo, i: comentarios en el código) se ven en la **U02**.

🚪 ¿Por dónde empiezo?

- 🆕 ¿Cero conocimientos? → Empieza en el punto 1, no hay ningún requisito previo.
- 💻 ¿Ya sabes qué es un ordenador? → Ve directo al punto 3 y vuelve atrás solo si te pierdes.
- 🎯 ¿Vienes a repasar? → Haz las actividades.

📍 **Primer punto:** 01 · Qué es un ordenador y un lenguaje de programación ⏭️ **Al acabar la unidad, continúa en U02 · Introducción a Java y el entorno de trabajo.**

01 — Qué es un ordenador y un lenguaje de programación

El primer paso: entender con quién vamos a hablar 🖥️

> 🗺️ **Estás en:** 🧠 **U01 · Introducción al Pensamiento Computacional** → 01 · Qué es un ordenador y un lenguaje de programación

---

Antes de aprender a pensar la lógica de nuestros programas, hay que dar un paso atrás y entender **dónde demonios estamos y con quién vamos a hablar**.

Cuando la gente dice *"estoy aprendiendo a programar"*, en realidad está aprendiendo a traduir ideas humanas a un formato que un conjunto de circuitos electrónicos pueda ejecutar. ¿Pero cómo se produce esa magia? Aquí tienes los conceptos clave que necesitas tener claros antes de empezar. 📌

🧠 1. Un ordenador es extraordinariamente rápido, pero absolutamente tonto

Un ordenador no piensa, no tiene intuición y no sabe qué quieres hacer. Lo único que entiende la máquina a nivel de hardware es el código máquina: una secuencia infinita de ceros y unos (0 y 1), conocida como sistema binario.

- Los 0 y 1 representan, literalmente, la ausencia o el paso de corriente eléctrica por los transistores del procesador.
- Escribir un programa directamente en ceros y unos es una tarea humana prácticamente imposible y dolorosa.

🗣️ 2. ¿Qué es un lenguaje de programación?

Un **lenguaje de programación** es el puente o la interfaz entre tu cerebro y el procesador. Es un conjunto de reglas, palabras clave y estructuras creadas por humanos que nos permiten escribir instrucciones de manera entendible para nosotros, pero que se pueden traducir fácilmente a ceros y unos.

Hay dos grandes tipos de lenguajes según su proximidad a la máquina:

- **Lenguajes de bajo nivel (ej: Ensamblador):** Están muy cerca del hardware. Son ultra-rápidos, pero extremadamente difíciles de escribir y mantener para un humano.
- **Lenguajes de alto nivel (ej: Java, Python, C#):** Utilizan palabras en inglés (if, while, class), estructuras lógicas complejas y símbolos matemáticos. Son fáciles de leer y escribir para nosotros, pero la máquina no los entiende directamente.

🔄 3. El traductor: Compiladores vs. Intérpretes

Como el ordenador solo entiende binario y nosotros escribimos en un lenguaje de alto nivel, **necesitamos un traductor**. Aquí es donde aparece la gran diferencia en cómo se ejecuta el código:

[📝 Código Fuente (Java)] ──► [⚙️ TRADUCTOR / Compilador] ──► [🤖 Código Máquina (0s y 1s)]

1. **Compilador (El traductor de libros):** Coge TU archivo de código fuente, lo revisa entero, comprueba que no haya errores de sintaxis y genera un archivo ejecutable independiente. Si hay un solo error, no traduce nada hasta que lo corrijas. *(Ejemplo: C, C++).*
2. **Intérprete (El traductor en directo):** Lee tu código línea por línea y lo ejecuta al acto. Si encuentra un error en la línea 50, ejecutará las primeras 49 líneas y se parará exactamente en la 50. *(Ejemplo: Python, JavaScript).*

> ☕ **¿Y Java?** Java utiliza un sistema híbrido muy especial. Primero se **compila** a un código intermedio llamado *Bytecode*, y después una "máquina virtual" (**JVM - Java Virtual Machine**) lo **interpreta** en cualquier ordenador (Windows, Mac, Linux). Por eso se dice que Java es *"Escríbelo una vez, ejecútalo donde quieras"*.

🔤 4. Conceptos clave que escucharás cada día

Para no sonar como un extranjero cuando hablamos del módulo, guarda estos 5 términos en tu diccionario personal:

- **Código Fuente (Source Code):** El texto plano que tú escribes en el lenguaje de programación (tus archivos .java).
- **Sintaxis:** Las reglas gramaticales del lenguaje. Si en español no puedes decir *"casa la roja es"*, en Java no puedes olvidar un punto y coma ; o una llave {}. Si te saltas la sintaxis, el compilador se quejará en rojo.

- **IDE (Entorno de Desarrollo Integrado):** El programa o la "fábrica" que utilizamos para escribir código (como VS Code, Eclipse o NetBeans). Es como un procesador de textos (estilo Word), pero diseñado para programar: remarca colores, avisa de errores y tiene el botón de ejecutar integrado.
- **Algoritmo:** La secuencia lógica y finita de pasos que resuelve un problema (independientemente del lenguaje que uses).
- **Ejecución / Run:** El momento en que le dices al ordenador *"Coge mi programa y haz lo que pone aquí"*.

💡 Idea clave para recordar

> **El lenguaje de programación es solo el idioma de la conversación.**

> Aprender la sintaxis de Java es fácil (es como aprender vocabulario). El reto real de este curso no es memorizar el diccionario, sino saber **qué demonios quieres decirle a la máquina** para resolver un problema. ¡Y de eso trata el resto de esta Unidad 1! 🚀

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · Programar no es memorizar código

02 — Programar no es memorizar código

El espejo de la comprensión y la trampa del copy-paste 🧠💻

> 🗺️ **Estás en:** 🧠 **U01 · Introducción al Pensamiento Computacional** → 02 · Programar no es memorizar código

---

Muy a menudo, cuando alguien se enfrenta por primera vez a una asignatura de programación, tiene la idea preconcebida de que aprender a programar es como aprender un idioma extranjero a la antigua: memorizar una lista infinita de palabras raras, reglas de sintaxis y fórmulas mágicas en inglés. 🧙‍♂️✨

Nada más lejos de la realidad. **Aprender a programar no es un ejercicio de memoria; es un entrenamiento para resolver problemas.** El lenguaje de programación (en nuestro caso, Java) es simplemente la herramienta final que utilizamos para comunicar nuestra solución a la máquina, de la misma manera que un carpintero utiliza un martillo o una sierra para construir una mesa. 🔨🪑

🧟‍♂️ La trampa del “Espejo de la Comprensión”

Uno de los fenómenos más habituales en el estudio a distancia es lo que en psicología del aprendizaje se llama **la ilusión de competencia** o el **espejo de la comprensión**.

Seguramente te sonará esta película:

1. Abres la plataforma virtual, lees un ejemplo de código explicado paso a pas o ves un vídeo donde el profesor resuelve un ejercicio. 🎬
2. Mientras lo ves, lo encuentras todo lógico, fluido y clarísimo. Dices para ti mismo/a: *«Es claro, qué genio! ¡Tiene todo el sentido del mundo! Lo he entendido a la primera»*. 😎
3. Cierras el vídeo, abres un archivo vacío para hacer un ejercicio similar y… **te quedas mirando el parpadeo del cursor en blanco durante 20 minutos.** 😳⏱️

¿Por qué pasa esto? ¿Es un problema de capacidad? ¡Qué va, de ninguna manera! Pasa porque tu cerebro utiliza dos circuitos completamente diferentes según la tarea:

- **El circuito del Reconocimiento (Modo Pasivo / “Sofá y Manta” 🛋️):** Cuando lees o ves un ejercicio resuelto, tu cerebro solo tiene que verificar que la lógica que otro ha creado tiene sentido. Es una tarea muy ligera que requiere casi cero esfuerzos mentales.

- **El circuito de la Generación (Modo Activo / "Gimnasio Mental" 🏋️):** Cuando te enfrentas a la pantalla en blanco, tu cerebro tiene que crear la estructura desde cero, tomar decisiones, recordar la lógica y traducirla. Esto requiere un esfuerzo cognitivo intenso.

> 🎹 **La analogía:** Confundir "entender un ejemplo de código" con "saber programar" es como creer que sabes tocar el piano solo porque te gusta mucho la música clásica y entiendes cuando alguien desafina. ¡Escuchar no te hace pianista!

🏔️ La pantalla en blanco es tu hábitat natural

Cuando te sientas a estudiar y te encuentras bloqueado/a frente a la pantalla en blanco, tu primera reacción puede ser de pánico: *«¡Madre mía, esto no es para mí, soy inútil, no se me ocurre nada!»* 😱

Hay que cambiar el chip desde el primer día: **el bloqueo frente a la pantalla en blanco no es una señal de fracaso, es el estado natural de cualquier programador.**

Programar consiste, precisamente, en transformar esa incertidumbre inicial en un conjunto de instrucciones ordenadas. La diferencia entre una persona que está aprendiendo y un profesional de Google no es que el profesional sepa la solución inmediatamente, sino que tiene un método de trabajo para salir de ese bloqueo sin tirar el portátil por la ventana. 🪟💻❌

🕳️ Cómo evitar la farsa de "Copiar y Pegar" (Tutorial Hell)

A distancia, cuando nos quedamos bloqueados, la tentación de la "vía rápida" es enorme: buscar la solución exacta en internet, copiarla, pegarla en nuestro proyecto, ver que se ejecuta y decir: *«¡Ya lo tengo hecho, soy un crack!»* 🤡

Esto solo genera una falsa sensación de progreso. Si no has pasado por el proceso mental de sufrir y pensar la solución, **no has aprendido nada**. La próxima vez que el problema cambie un poquito, te volverás a quedar igual de bloqueado/a.

Para evitar esta trampa, aplicaremos una norma muy simple durante todo el curso:

1. **La Regla de la Pantalla Tapada 🙈:** Si miras la solución de un ejercicio o un ejemplo del temario, cierra la ventana o tapa el código antes de intentar escribir el tuyo.

2. **Nada de milagrosas inspecciones visuales:** Si piensas que te has quedado atascado, deja un poco de margen a tu mente para buscar la salida, revisa solo la línea concreta que te hace dudar y vuelve a tapar la solución inmediatamente.

💡 Idea clave para recordar

> **Ver o leer cómo se resuelve un problema no significa saber resolverlo.**

> El aprendizaje real solo ocurre cuando tu cerebro hace el esfuerzo de construir la solución por sí mismo. ¡Menos mirar, más intentar! 💪🎯

📚 Volver al índice de la unidad · **Anterior:** 01 · Qué es un ordenador y un lenguaje de programación · **Siguiente:** 03 · Primero pensamos, después escribimos

03 — Primero pensamos, después escribimos

El camino sagrado del programador y el test del Robot Literal 📝➡️💻

> 🗺️ **Estás en:** 🧠 **U01 · Introducción al Pensamiento Computacional** → 03 · Primero pensamos, después escribimos

---

Hay un error clásico de principiante que se repite desde la invención de los ordenadores: abrir el editor de código, poner las manos sobre el teclado y empezar a escribir líneas sin tener la más remota idea de cómo se resuelve el problema. 🤦‍♂️💥

El resultado de esto suele ser una mezcla de frustración, errores de sintaxis en rojo por toda la pantalla y la sensación de que la informática es una especie de magia negra reservada para unos pocos elegidos. 🧙‍♂️❌

Hoy queremos romper este mito con una regla de oro que te ahorrará cientos de horas de dolor durante el curso: **El código es simplemente la traducción de una solución que YA has pensado previamente en papel.** 📄✍️

🔮 La ilusión de la literalidad (Tu ordenador no es adivino)

Los humanos somos unas máquinas de "leer entre líneas". Si le dices a un amigo *"Voy a la cocina, ¿quieres agua?"* y te responde *"Sí, por favor"*, entiendes perfectamente que tienes que coger un vaso, abrir el grifo o la botella, llenar el vaso y llevárselo. No se te pasa por la cabeza ir a la cocina, coger un chorro de agua con las manos y tirárselo por encima a la cara, ¿verdad? 🌊😳

Si le pides eso mismo a un ordenador:

- No sabe qué es un vaso 🥃.
- No sabe cuándo parar de verter agua (inundación en la cocina 🌊).
- Se queda bloqueado para siempre esperando que le digas si el agua debe ser del tiempo o fría 🧊.

> 🤖 **La cruda realidad:** Un ordenador no tiene intuición, no tiene sentido común, no tiene empatía y **no interpreta lo que quieres decir: ejecuta exactamente las instrucciones que le**

> **das**. Si tu programa hace cosas raras, no es que la máquina te tenga manía; es que tú has asumido que el ordenador “ya te entendía”. Y no, ¡no te entiende!

🛣️ El camino sagrado del programador

Para no colapsar tu cerebro, el flujo de trabajo para resolver cualquier ejercicio debe seguir SIEMPRE esta secuencia de 4 pasos:

[1. Problema 🧩]──► [2. Paso a paso (Algoritmo) 👣]──► [3. Solución en papel 📄✍️]──► [4. Código 💻]

1. **Entender el problema 🧩:** Leer el enunciado hasta que seas capaz de explicárselo a tu abuela o a tu gato. Si no sabes qué te piden, ¡es imposible resolverlo!
2. **Pensar los pasos (Algoritmo) 👣:** Diseñar la secuencia lógica de pasos que solucionan el problema. ¿Qué dato va primero? ¿Qué tengo que calcular? ¿Qué decisión tengo que tomar?
3. **Escribir la solución en papel 📄:** Usar tu propio idioma (español/valenciano) o un dibujo para comprobar que la lógica tiene sentido.
4. **Traducir a Java 💻:** Abres el ordenador. Ahora solo tienes que preocuparte por la sintaxis (puntos y comas ;, llaves {}, etc.), porque la lógica ya la tienes resuelta en el papel.

Si intentas hacer el Paso 2 y el Paso 4 al mismo tiempo directamente en el teclado, tu “RAM mental” se colapsará y aparecerá el bloqueo. 🤯❌

🤖🧪 Ejercicio práctico: El test del Robot Literal

Para experimentar esto en tu propia piel sin tocar ningún ordenador, vamos a hacer un experimento mental.

Imagina que tienes que dar instrucciones a un **Robot Literal** (un ser que no tiene sentido común y lo hace TODO al pie de la letra) para una de estas tareas cotidianas:

- ☕ **Hacer un café en cafetera de goteo.**
- 🎒 **Preparar la mochila para ir a clase.**
- 🍳 **Hacer una tortilla de patatas.**
- 🚗 **Cambiar una rueda pinchada de un coche.**

- 🛒 **Hacer una compra en una tienda online.**

¿Qué pasa cuando intentamos escribir las instrucciones? ⚠️

Si escribes: *"Coge el café y ponlo en la cafetera"*, el Robot Literal cogerá el paquete de café CERRADO y lo tirará encima de la cafetera sin abrirlo. 📦💥

Para que el Robot lo haga bien, tienes que desglosarlo así:

1. Coge el paquete de café con la mano derecha.
2. Con la mano izquierda, abre la pestaña superior del paquete.
3. Coge la cuchara... *(y así 20 pasos más)*.

😈 Los 5 demonios de la lógica

Cuando hacemos este ejercicio, aparecen inmediatamente los 5 problemas que después te encontrarás programando en Java:

1. **Pasos que faltan** 🚫**:** Abrir la botella de agua antes de verterla.
2. **Orden incorrecto** 🔀**:** Ponerse los zapatos antes que los calcetines.
3. **Ambigüedades** ❓**:** En vez de decir *"Gira a la derecha"*, decir *"Gira un poco"*. ¿Cuánto es "un poco"? ¿45 grados? ¿180 grados?
4. **Información implícita** 🤫**:** Dar por hecho que el Robot sabe que el agua quema.
5. **Decisiones no contempladas** 🔀**:** ¿Qué hace el Robot si cuando coge el café... se ha acabado el café? *(Si no le has dicho nada, el Robot se queda colgado para siempre).*

💡 Idea clave para recordar

> **El código es solo el último paso.**

> Si no sabes resolver un problema con un papel y un bolígrafo en tu propio idioma, es absolutamente imposible que puedas resolverlo en Java frente a la pantalla. ¡Piensa primero, teclea después! 🧠📄➡️⌨️

📚 Volver al índice de la unidad · **Anterior:** 02 · Programar no es memorizar código · **Siguiente:** 04 · La descomposición

04 — La descomposición

Divide y vencerás: el arte de partir un elefante 🧱⛏️

> 🗺️ **Estás en:** 🧠 **U01 · Introducción al Pensamiento Computacional** → 04 · La descomposición

Cuando te encuentras frente a un ejercicio de programación que ocupa media página de enunciado, es completamente normal que tu primera reacción sea una mezcla de sudor frío, ganas de cerrar el portátil y ganas de pedir un cambio de ciclo a uno de Arte Floral. 🌿🙈

El problema no es que el ejercicio sea imposible. El problema es que tu cerebro está intentando procesar **TODA la complejidad de golpe**. Quiere resolver el principio, el medio, el final, la interfaz, los errores y la base de datos en el mismo segundo.

Veremos la técnica secreta (y milenaria) que usan los programadores de todo el planeta para no colapsar: **Divide y vencerás.** ⚔️👑

🐘 ¿Cómo comerse un elefante? ¡A bocados!

Hay una pregunta clásica de reflexión: *«¿Cómo te comes un elefante de 4 toneladas?»*

La respuesta es evidente: *«Bocado a bocado, despacio y sin prisa».* 🥩

En programación es exactamente lo mismo: **un problema grande solo es una colección de problemas pequeños disfrazados.** Si intentas resolverlo todo de golpe, te bloquearás. Si lo desmontas en piezas microscópicas, verás que cada pieza individual es tan sencilla que podrías resolverla casi sin pensar.

🏛️ El arte de la descomposición en sub-bloques

Imagina que tu primer día de prácticas te dicen:

💬 *«Necesitamos que hagas una aplicación entera para gestionar una Biblioteca Municipal».*

Si intentas empezar a programar esto directamente, la pantalla en blanco se reirá de ti a la cara. ¿Cómo aplicamos la descomposición? Dividiendo el “monstruo” en bloques independientes:

[1. Gestión de Usuarios 👤] 📖 ──► [2. Catálogo de Libros 📚] 📍 ──► [3. Sistema de Préstamos 🔄] ⏱️ ──► [4. Devoluciones y Penalizaciones ⏰]──► [5. Búsqueda y filtros 🔍]

¡Falta menos! Ahora, en vez de pensar en "TODA la biblioteca", nos centramos **únicamente** en el sub-bloque número 3: **El sistema de préstamos**.

Pero espera, ¡que aún lo podemos dividir más! ¿Cómo es el proceso de un préstamo paso a paso?

[Préstamo de un Libro] 📖

```
│
├── 1. Identificar al usuario (DNI) 🆔
├── 2. Identificar el libro (Código de barras) 🏷️
├── 3. Comprobar si el usuario tiene multas pendientes 🚫
├── 4. Comprobar si el libro está disponible (no prestado) ❓
├── 5. Registrar la fecha de hoy y la fecha límite 🗓️
└── 6. Cambiar el estado del libro a "Prestado" ✅
```

¡MIRA ESTO! 🎉 La tarea gigante y aterradora de "Hacer una Biblioteca" se ha convertido en una lista de acciones tan simples como: *"Comprobar si un número es mayor que cero"* o *"Guardar un nombre en un texto"*.

¡Felicitaciones! Ya estás diseñando la arquitectura de un programa sin haber tocado una sola línea de Java. 👏🥳

⚠️ Las 3 reglas de oro para no morir en la división

1. **Nunca resuelvas dos sub-problemas a la vez 🚫:** Si estás fabricando la parte donde el usuario pone su DNI, NO pienses todavía en cómo se imprimirá el tiquete en PDF. ¡Una cosa tras otra!
2. **Si un sub-paso todavía te parece difícil, vuelve a dividirlo ✂️:** Si el paso 3 (*Comprobar multas*) te parece embrollado, divídelo en: 3a) Leer multas del usuario, 3b) Sumar el importe total, 3c) Si es mayor que 0€, bloquear el préstamo.
3. **Celebra las pequeñas victorias 🥳:** Ver que el paso 1 (*Leer el DNI*) funciona correctamente te da un chute de dopamina y seguridad para afrontar el paso 2.

💡 Idea clave para recordar

> **Ante un problema grande, no te bloquees: afila las tijeras.**

> Si una tarea te da miedo, es simplemente porque no la has dividido en piezas bastante pequeñas. Haz los pasos más y más pequeños hasta que sean ridículamente fáciles de resolver. ✂️ 🧩 🚀

📚 Volver al índice de la unidad · **Anterior:** 03 · Primero pensamos, después escribimos · **Siguiente:** 05 · Debugging y casos límite

05 — Debugging y casos límite

El texto rojo es tu amigo y el detective nunca prueba solo lo bonito 🚨🕵️‍♀️

> 🗺️ **Estás en:** 🧠 **U01 · Introducción al Pensamiento Computacional** → 05 · Debugging y casos límite

---

Llega el momento inevitable de la vida de cualquier persona que programa: haces clic en el botón verde de ejecutar el programa y, en vez de funcionar como la seda, la pantalla se ilumina con un texto rojo oscuro que parece gritarte: *«¡Has fracasado en la vida! ¡Cierra el portátil y huye al campo a cuidar ovejas!»* 🦙🏃‍♂️💨

¡Tranquilidad! Hay que desmontar el drama desde el minuto 1: **un error en tu código no es una vergüenza, ni un ataque personal, ni una señal de que no vales para la informática.**

En programación, los errores tienen un nombre táctico —**bugs** 🦎— y el arte de encontrarlos y corregirlos se llama **debugging** (o desinsectación mental). Aprenderemos a mirar a los errores a los ojos sin entrar en pánico. 👁️🛑

🔴 1. El cambio de chip: El texto rojo es tu amigo

Para un humano normal, una luz roja gigante significa: *«¡PELIGRO! ¡LO HAS HECHO MAL!»*. Pero para un programador o programadora, **el texto rojo es un asistente ultra-eficiente que habla con tu gratis.** 🤖💬

Piénsalo así: la máquina podría simplemente quedarse en silencio, petar o decirte un "No funciona" y dejarte a oscuras. ¡Pero no! Te da una **stack trace** (la famosa montaña de letras rojas) que te dice dos cosas de oro puro:

1. **¿Qué ha pasado?** 🧐 *(Ej. Te has olvidado de cerrar un paréntesis, o has intentado dividir por cero).*
2. **¿En qué línea exacta ha pasado?** 📍 (Ej. En el archivo Main.java, en la línea 24).

> ❌ **Lo que ve tu cerebro en pánico:** *"FATAL ERROR SYSTEM CRASH DIE DIE DIE BLA BLA BLA"* 😱

> 🟢 **Lo que dice el mensaje en realidad:** "¡Hola! En la línea 24 has escrito num2 en vez de num1. ¡De nada, campeón!" 👍

🧪 2. Pensar como un programador: Probar los casos límite

Otro error de principiante es probar el programa SOLO con los datos “bonitos” y decir: *«¡Ya funciona!»*.

Imagina este programa súper sencillo:

> *“Un programa que indica si una persona es mayor de edad (edad mínima: 18 años).”*

Si lo pruebas poniendo un **20**, el programa dice “Mayor de edad” y tú te vas a hacer un café tan contento/a ☕. Pero un buen programador tiene una **mente de detective malicioso** y se pregunta: *«¿Y cómo podría petar yo esto?»* 😈

Hay que probar SIEMPRE los **casos límite (edge cases)**:

- 🎯 **El límite exacto (18 años):** ¿Tu programa dice que tiene 18 es mayor de edad o se ha quedado fuera por un error de > en vez de >=?
- 🎯 **El límite inferior (17 años):** ¿Dice correctamente que es menor?
- 🎯 **El límite cero (0 años):** ¿Qué hace el programa?
- 💣 **El cas “Gamberro” (-5 años o 200 años):** ¿Tiene sentido que alguien tenga -5 años? ¿O deberías comprobar que la edad sea un número válido antes de hacer nada?

Probar los casos que nadie se espera es la diferencia entre un programa de aficionado y un programa profesional. 🛡️ ✨

💡 Idea clave para recordar

> **Los errores no son el final del camino, son el mapa.**

> Un programa que falla no es un fracaso; es simplemente un rompecabezas que te está dando las pistas exactas para ser resuelto. ¡Abraza el texto rojo! 🔴 🧩 🔎

📚 Volver al índice de la unidad · **Anterior:** 04 · La descomposición · **Siguiente:** 06 · El protocolo de paz

06 — El protocolo de paz

4 pasos antes de pedir ayuda (y no parecer un vago) 🛡️💬

> 🗺️ **Estás en:** 🧠 **U01 · Introducción al Pensamiento Computacional** → 06 · El protocolo de paz

Cuando estudias a distancia y un ejercicio no sale, la tentación es ir directamente al foro o a la IA y escribir el clásico: *«¡Profesor, no me funciona, ayuda!»* 🆘❌

Para no quedarte bloqueado/a y aprender de verdad, antes de enviar cualquier mensaje, **debes seguir religiosamente este protocolo de 4 pasos**:

🛡️ Los 4 pasos del protocolo

[1. Leer el error] 📖 ⟶ [2. Localizar la línea] 📍 ⟶ [3. Hipótesis de los 15 min] ⏱️ ⟶ [4. Preguntar con datos] 💬

1. Lee el error en voz alta 📖

No tan solo los ojos. Lee la línea roja y busca las palabras clave.

2. Ve a la línea exacta 📍

Abres tu código, vas a esa línea y miras qué demonios has escrito ayer.

3. La Regla de los 15 Minutos (La hipótesis) ⏱️

Intenta cambiar UNA SOLA COSA para probar si tu idea de dónde está el fallo es correcta. Si llevas 15 minutos atascado/a en la misma línea… **¡LEVÁNTATE!** 🚶 Ve por un vaso de agua, habla con el gato o mira por la ventana. La solución casi siempre llega cuando la mente se relaja.

4. Pide ayuda como un Pro 💬

Si después de esto no lo resuelves, escribes en el foro, pero **no digas "no funciona"**. Di:

- *"Estoy intentando hacer X…"*
- *"Me salta el error Y en la línea Z…"*

- *“Ya he probado a cambiar A y B, pero sigue fallando.”*

💡 Idea clave para recordar

> **Los errores no son el final del camino, son el mapa.**

> Un programa que falla no es un fracaso; es simplemente un rompecabezas que te está dando las pistas exactas para ser resuelto. ¡Abraza el texto rojo! 🔴🧩🔎

📚 Volver al índice de la unidad · **Anterior:** 05 · Debugging y casos límite · **Siguiente:** 07 · Practicar (menos mirar, más intentar)

07 — Practicar (menos mirar, más intentar)

La regla 80/20 y los hábitos del programador 🏊‍♂️🏋️

> 🗺️ **Estás en:** 🧠 **U01 · Introducción al Pensamiento Computacional** → 07 · Practicar (menos mirar, más intentar)

Imagina que quieres aprender a nadar. Te compras el mejor libro de natación del mundo, lees 500 páginas sobre la física de la flotabilidad, ves 40 horas de vídeos de la Selección Olímpica en 4K y te estudias de memoria el ángulo exacto en que el brazo debe entrar en el agua. 📖🏊‍♂️

Llega el día de la verdad: te vas a la piscina, te tiras a la parte profunda sin haber tocado el agua nunca… ¿y qué pasa? **¡Te vas directamente al fondo como un saco de patatas!** 🌊⚓

Con la programación pasa exactamente lo mismo. Puedes tener los apuntes más bonitos de la clase, haber subrayado todo el temario con rotulador amarillo y tener la colección de videotutorials más grande de Internet, pero **si no tecleas y te equivocas tú solo/a, no estás aprendiendo a programar.** 🤷‍♂️💻

🎭 La gran farsa de “Ya lo he entendido”

El estudio a distancia tiene un peligro silencioso que hay que destapar desde el primer día: **confundir la lectura pasiva con el aprendizaje real.**

Seguramente vivirás esta experiencia en dos pasos muy claros durante las próximas semanas:

🎬 Fase A: El modo espectador (*«¡Qué fada soy!»*)

Lees un algoritmo resuelto en los apuntes o ves cómo el profesor programa la solución de un ejercicio en un vídeo. Todo parece claro, estructuradísimo y lógicísimo. Tu cerebro se relaja, hace un chute de dopamina y dices: *«Vaya, qué ganga de módulo, ¡lo entiendo todo a la primera!»* 😎✨

🛑 Fase B: La prueba maestra (*«Ufff… ¿qué ha pasado?»*)

Cierras la ventana de la solución, te vas a un archivo completamente en blanco e intentas hacer un ejercicio idéntico pero cambiando los datos. De pronto, el vacío absoluto. El cursor parpadea (▌), no sabes si empezar por arriba o por abajo, y la mente se queda en blanco. 😳⚡

> 💡 **La lección de vida:** Entender la solución de otro y saber construir una solución tú mismo/a **son dos habilidades cerebrales completamente diferentes.** La primera es facilísima; la segunda es la que se paga en las empresas de informática.

⏳ 🎯 La Regla del 80/20 para la FP a Distancia

Para no caer en la trampa de estar horas y horas "leyendo apuntes" sin avanzar de verdad, durante este curso aplicaremos una regla de distribución de tu tiempo de estudio muy clara:

- 📖 20% del tiempo: Leer la teoría / Ver el concepto.
- 💻 80% del tiempo: Hacer ejercicios, errar, probar y picar.

Si dedicas 10 horas a la semana a este módulo, solo 2 horas deberían ser para leer o ver explicaciones. Las otras **8 horas deben ser de picar código, pelear con la pantalla en blanco y probar cosas en tu ordenador.** ⌨️💥

🏋️ Cómo entrenar la "musculatura de programador"

La programación no funciona por memorización de datos (como aprenderse las capitales de Europa), sino por **memoria muscular y esquemas mentales** (como ir en bicicleta o jugar a un videojuego).

Para entrenar esta musculatura sin frustrarte, aplica estos 3 hábitos diarios:

1. **La norma del "Hazlo antes de mirar" 🙈 :** Cuando el temario proponga un ejercicio de ejemplo, NO leas la solución directamente. Intenta hacerlo tú en papel durante 5 minutos. ¡Aunque sea malo! El esfuerzo de pensar antes de ver la respuesta multiplica por diez la retención.
2. **El cambio de un 5% 🔀 :** Cuando un ejemplo de la plataforma te funcione, no lo cierres y pases a otra cosa. Pregúntate: *«¿Y si ahora hago que en vez de sumar, multiplique?»* o *«¿Y si le pido dos datos más al usuario?»*. ¡Modificar el código que funciona es la mejor manera de entender cómo está hecho por dentro!
3. **No tengas miedo de cerrar el tutorial ❌ 📺 :** Si para hacer un ejercicio necesitas tener el vídeo del profesor o el PDF abierto al lado mirando línea por línea lo que hace, no estás

haciendo el ejercicio: estás haciendo de copista medieval. ¡Cierra la muestra y fuerza tu cerebro a recordar!

💡 Idea clave para recordar

> **Programar no se mira, programar se hace.**

> ¡Menos leer sobre la piscina y más tirarse al agua! El único código que te hace aprender es el que escribes tú después de haberlo pensado. 💪🏊‍♂️⌨️

📚 Volver al índice de la unidad · **Anterior:** 06 · El protocolo de paz · **Siguiente:** 08 · Cómo utilizaremos la IA

08 — Cómo utilizaremos la IA

El asistente, no tu oráculo 🤖🧠

> 🗺️ **Estás en:** 🧠 **U01 · Introducción al Pensamiento Computacional** → 08 · Cómo utilizaremos la IA

Seamos realistas desde el primer segundo: en un ciclo a distancia, herramientas como ChatGPT, Gemini o Claude están literalmente a un clic de distancia en tu pantalla. Decirte que está «completamente prohibido usar la inteligencia artificial» sería como intentar ponerle puertas al campo o prohibir el uso de la calculadora a un matemático. 🚫🌾

Ahora bien, hay una diferencia ABISMAL entre **usar la IA para multiplicar tu aprendizaje** y **usar la IA para atrofiar tu cerebro**. 🧠💥

Estableceremos las reglas del juego para que la IA sea tu mejor tutor privado y no la causa de que pienses el día del examen presencial. 🎓⚠️

🚨 El gran peligro: Pagar a otro para que haga tus flexiones

Imagina que te apuntas al gimnasio porque quieres ponerte en forma, pero pagas a un pequeño robot para que haga todas las flexiones, abdominales y carreras por ti. Al final del mes, el robot está en una forma increíble, pero tus músculos continúan exactamente igual de blandos. 🏋️🤖

Con la programación pasa exactamente esto:

- ❌ **Si le dices a la IA:** *«Hazme el ejercicio 3 de Java»* La IA te dará un código impecable en 3 segundos ⚡. Lo pegarás en la plataforma, el profesor te pondrá un 10… pero **tú no habrás aprendido absolutamente nada**.
- 💣 **El día del examen presencial (sin IA):** Te quedas solo/a frente al papel en blanco o del ordenador sin internet. Tu “RAM mental” nunca ha entrenado la generación de soluciones y el bloqueo será monumental. 😳🔥

🏛️💬 El Método Pro: La IA como Tutor Socrático

Para que la IA trabaje PARA TI (y no EN LUGAR DE TI), la clave está en **cambiar la forma en que le preguntas**.

Cuando te atasques en un ejercicio, no pidas la solución directa. Usa el **Prompt del Tutor Socrático**:

> 💬 **Copia y pega este texto en la IA cuando te atasques:**
>
> *«¡Hola! Estoy aprendiendo a programar en Java y me he atascado en este ejercicio: [pega el enunciado]. No me des el código ni la solución directa. Hazme preguntas guía o dame una pequeña pista para que pueda descubrir yo mismo/a dónde está el fallo o qué paso me falta.»*

📊 Comparativa de mentalidad

❌ Modo Vague (Te vas al fondo)	✅ Modo Pro (Te haces una máquina)
“Hazme el código para calcular si un número es par.”	“Explícame qué concepto matemático se usa para saber si un número es divisible por 2 sin darme el código.”
“¿Dónde está el error en este código? Corrígelo.”	“Tengo este error en la línea 12. Dame una pista de qué concepto de sintaxis me he olvidado de revisar.”
Copiar el código directamente en tu IDE.	Leer la explicación de la IA, cerrar la ventana del chat y escribir el código de memoria. 🙈

🦹‍♂️ La Regla del “No te fíes ni de tu sombra” (Alucinaciones)

Los modelos de lenguaje no son cerebros que “piensan” como nosotros; son sistemas estadísticos brutales que predicen cuál es la palabra más probable que viene detrás de otra. 📊🔤

Esto significa una cosa fundamental: **la IA puede inventar cosas con una seguridad y una elegancia altísimas, pero sigue siendo una “invención”**.

- Puede darte métodos que no existen en Java 🦄.

- Puede proponerte soluciones ultra-complejas que no corresponden a tu nivel 🤯.
- Puede decirte que un código que está MAL está correcto con una sonrisa virtual 🤡.

> **La regla de oro:** Nunca, NUNCA des por bueno un código que te haya sugerido una IA sin haberlo probado, entendido línea por línea y comprobado con tus propios casos límite. ¡Tú eres el/la piloto; la IA es solo el copiloto! 🏎️🔧

💡 Idea clave para recordar

> **La IA debe ayudarte a pensar, nunca sustituir tu pensamiento.**

> Usa la IA para que te explique conceptos o te dé pistas como un profesor privado las 24 horas, pero asegúrate de que el único que escriba el código final sea tu propio cerebro. 💪🤖🎯

📚 Volver al índice de la unidad · **Anterior:** 07 · Practicar (menos mirar, más intentar) · **Siguiente:** 09 · Mi primer problema (puente a Java)

09 — Mi primer problema (puente a Java)

Resolvemos nuestro primer reto sin tocar Java 🌉☕

> 🗺️ **Estás en: 🧠 U01 · Introducción al Pensamiento Computacional** → 09 · Mi primer problema (puente a Java)

Hemos llegado al final de esta Unidad 1! Después de desmontar falsos mitos, de pelearnos con la literalidad de los robots, de aprender a no tener miedo al texto rojo y de poner a la IA a trabajar como tutor privado, llega el momento de la verdad: **resolver nuestro primer problema de programación real.** 🥳🎉

Y lo mejor de todo? Vamos a hacerlo **sin escribir una sola línea de Java todavía**.

¿Por qué? Porque así comprobarás en carne propia la gran lección de esta unidad: **la parte difícil de programar es pensar la lógica; traducirlo a un lenguaje concreto es solo cuestión de sintaxis.** 🧠➡️⌨️

🥊 El Reto: El Comparador de Números

Imagina que nos piden crear un programa súper sencillo para una aplicación:

> 📌 **Enunciado del problema:**

> *"Un programa debe pedir dos números enteros al usuario e indicar claramente por pantalla cuál de los dos es el mayor, o si ambos números son iguales."*

Parece fácil, ¿verdad? Pero antes de abrir cualquier editor de código o empezar a escribir paréntesis, aplicaremos nuestro método de trabajo en 3 pasos. 📄✍️

🧩 Paso 1: Descomposición y Entrenamiento de Datos (Inputs y Outputs)

Antes de pensar la lógica, un/a programador/a se pregunta: **¿Qué necesito pedir (Inputs) y qué tengo que devolver (Outputs)?**

- 🕹️ **Datos de entrada (Inputs):** Necesitamos 2 números enteros. Los llamaremos número A y número B.

- 📤 **Dato de salida (Output):** Un mensaje por pantalla que diga cuál es el mayor o si son iguales.

👣 Paso 2: El Algoritmo en Pseudocódigo (Lógica pura)

Ahora escribiremos la secuencia de pasos en nuestro propio idioma, sin preocuparnos por puntos y comas o llaves. ¡Solo lógica!

```
1. Pedir al usuario el Número A.
2. Pedir al usuario el Número B.
3. Comparar los dos números:
   - SI el Número A es mayor que el Número B:
         Mostrar el mensaje: "El Número A es el mayor".
   - SI NO, SI el Número B es mayor que el Número A:
         Mostrar el mensaje: "El Número B es el mayor".
   - SI NO (si ninguna de las anteriores se cumple):
         Mostrar el mensaje: "Ambos números son iguales".
4. Fin del programa.
```

Mira este algoritmo un segundo. ¿Tiene todo el sentido del mundo? **¡Esto YA es programar.** Cualquier persona, sea de Ontinyent, de Tokio o un robot de la NASA, entiende perfectamente esta lógica. 🚀✨

🧪 Paso 3: El testing de casos límite (Mentalidad de detective)

Antes de dar el algoritmo por bueno, vamos a hacerle nuestro "control de calidad" probando diferentes valores:

- 🟢 **Caso 1 (Normal):** A = 8, B = 3 ⟶ El programa dice *"A es el mayor"*. (¡Correcto! ✅)
- 🟢 **Caso 2 (Inverso):** A = 2, B = 10 ⟶ El programa dice *"B es el mayor"*. (¡Correcto! ✅)
- 🟡 **Caso Límite (Igualdad):** A = 5, B = 5 ⟶ El programa comprueba la primera opción (5 > 5? Falso), la segunda (5 > 5? Falso) y cae en el *SI NO*, diciendo *"Ambos números son iguales"*. (¡Correcto! ✅)

El algoritmo es 100% sólido. Funciona en papel y funciona en tu cabeza. 🏆

🌉 Y ahora… ¿Dónde está el puente a Java? ☕

¡Aquí viene la magia! A partir de la próxima unidad, lo único que haremos será **aprender cómo se dicen en Java esas mismas instrucciones que tú ya has pensado**:

- Para pedir datos, aprenderemos a usar una herramienta de Java llamada Scanner.
- Para comparar y tomar decisiones (SI… SI NO), aprenderemos la palabra reservada if y else.
- Para mostrar mensajes por pantalla, usaremos una instrucción llamada System.out.println().

🧠 TU LÓGICA (Unidad 1)	☕ LA TRADUCCIÓN A JAVA (Unidad 2)
SI el Número A es mayor que el B…	if (numeroA > numeroB) {
Mostrar “A es el mayor”	System.out.println(“A es el mayor”);
FIN DEL SI	}

¿Ves que no era tan oscuro ni tan difícil? ¡La lógica ya la tienes; ahora solo aprenderemos el “diccionario” de Java para decírselo a la máquina! 📖☕

💡 Idea clave para cerrar la Unidad 1

> **Ya sabes pensar como un programador/a.**

> Ahora que tienes el método, la mentalidad y el protocolo para no bloquearte, abrimos la puerta a la Unidad 2. ¡Bienvenido/a al mundo real de Java! 🚀☕💻

🐛 Glosario

Para cerrar la unidad y que no te suenen a “chino” las palabras que el profesorado y el sector utilizan a diario, aquí tienes tu guía rápida de consulta.

💻 Términos técnicos de Infraestructura

- **Código Fuente (Source Code):** El texto que tú escribes en el lenguaje de programación (archivos .java). Es el "manuscrito" que nosotros podemos leer y editar.
- **Sintaxis:** Las reglas gramaticales de un lenguaje. Si te saltas un punto y coma ; o una llave {}, el compilador no entenderá nada y te dará un error de sintaxis.
- **Compilador:** El programa traductor que coge tu código fuente y lo convierte en instrucciones que el procesador pueda ejecutar.
- **IDE (*Integrated Development Environment*):** El programa que utilizas para escribir código (como *Eclipse*, *NetBeans* o *VS Code*). Es como un "Word para programadores", con colores, autocompletado y botón de ejecutar.

🧠 Términos de lógica y proceso

- **Algoritmo:** La secuencia ordenada, finita y sin ambigüedades de pasos que resuelve un problema.
- **Pseudocódigo:** Hay que escribir la lógica de un algoritmo en tu propio idioma (español/valenciano) antes de traducirlo a Java.
- **Descomposición:** La técnica de dividir un problema grande y complejo en sub-problemas mucho más pequeños y fáciles de resolver.
- **Abstracción:** El proceso de filtrar e ignorar los detalles irrelevantes de un problema para centrarte solo en los datos esenciales.

🐛 Términos de gestión de errores y control

- **Bug:** Un error o fallo en la lógica o en la ejecución de un programa.
- **Debugging (o Depuración):** El proceso de buscar, aislar y corregir un *bug* en tu código.
- **Stack Trace:** La montaña de texto rojo que aparece en la consola cuando el programa falla. Te dice qué error ha habido y en qué línea exacta ha pasado.
- **Casos límite (*Edge Cases*):** Las situaciones o datos extremados (valores cero, números negativos, edades exactas) donde el programa tiene más riesgo de fallar si no se ha probado bien.

🤖 Términos de trabajo y metodología

- **Espejo de la comprensión:** La falsa sensación de saber hacer un ejercicio solo por haber visto o leído cómo lo resolvía otra persona.
- **Tutor Socrático (con IA):** Utilizar la inteligencia artificial haciéndole preguntas para que te ayude a pensar, en vez de pedirle que te haga el código directamente.
- **Entrada/Salida (*Input/Output* o I/O):** Los datos que el programa recibe del usuario (Input) y los resultados que el programa muestra por pantalla (Output).

📚 Volver al índice de la unidad · **Anterior:** 08 · Cómo utilizaremos la IA · **Siguiente:** U02 · Introducción a Java y el entorno de trabajo

U02 — Introducción a Java y el entorno de trabajo

Tu primer café con Java ☕

Bienvenido al curso de Programación en Java. ¿Sabías que tu ordenador es básicamente un cachorro muy listo pero con cero iniciativa? No hace nada hasta que le das órdenes precisas. Y para eso necesitamos un **entorno de desarrollo**, un lenguaje y muchas ganas de compilar.

Esta primera unidad te deja en la puerta del castillo: vas a instalar el JDK, a escribir tu primer programa, a conocer la trilogía del café (JVM, JRE y JDK), a usar el depurador como un detective y a hablar con la máquina mediante comentarios y argumentos. No necesitas experiencia previa: solo un editor de texto y curiosidad.

Esta unidad se lee como un **libro de 9 capítulos**: cada punto desarrolla una idea completa y enlaza con el siguiente.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar la diferencia entre **JDK, JRE y JVM** con la analogía de la cafetería.
- Instalar y configurar un JDK y comprobar tu instalación con `java -version`.
- Escribir, compilar y ejecutar tu primer programa Java.
- Diseccionar el método `main` y entender cada pieza del esqueleto de un programa.
- Usar el **depurador** (breakpoints, step over/into, watch) como un detective.
- Escribir comentarios de una línea, de bloque y **Javadoc**.
- Manejar los **argumentos de línea de comandos** con `args`.
- Reconocer y corregir los errores de compilación más típicos del novato.
- Trabajar con **Visual Studio Code** (o tu IDE favorito) y conocer tu flujo de trabajo diario.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · ¿Qué es Java?	Historia, bytecode, la trilogía JVM/JRE/JDK	Todos
02 · Instalación del JDK	Eclipse Temurin, VS Code, Extension Pack, `java -version`, PATH	Todos
03 · Tu primer programa	Hola Mundo, estructura, el método `main`	Todos
04 · El depurador	Breakpoints, step over/into, watch	Todos
05 · Comentarios y documentación	`//`, `/* */`, `/** */` y Javadoc	Todos
06 · Argumentos de línea de órdenes	El array `args`, cómo leerlo y usarlo	Todos
07 · El compilador y sus errores	Compilar vs ejecutar, errores típicos del novato	Todos
08 · VS Code y tu flujo de trabajo	Proyectos, atajos, crear/ejecutar/depurar	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Quién Soy, Laboratorio, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA1)

RA1: Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

CE	Criterio	Dónde se cubre
a)	Bloques que componen la estructura de un programa	✅ Puntos 3 y 7
b)	Proyectos de desarrollo de aplicaciones	✅ Punto 8
c)	Entornos integrados de desarrollo	✅ Puntos 2 y 8
i)	Comentarios en el código	✅ Punto 5

> 📌 El depurador que se ve en el punto 4 sienta las bases del RA3 (probado y depurado de programas), que se completa en la U04.

🚪 ¿Por dónde empiezo?

- ¿Cero conocimientos? → Arranca en el punto 1, no hay ningún requisito previo.
- ¿Ya sabes qué es un IDE? → Ve directo al punto 3 y vuelve atrás solo si te pierdes.
- ¿Vienes a repasar? → Haz el Repaso interactivo y los boletines.

📍 **Primer punto:** 01 · ¿Qué es Java?

⏭️ **Al acabar la unidad, continúa en U03 · Variables, tipos y operadores.**

01 — ¿Qué es Java?

La trilogía del café y el secreto de la taza humeante ☕

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 01 · ¿Qué es Java?

Java es un lenguaje de programación que se ejecuta dentro de una máquina virtual (la JVM), lo que le permite correr igual en Windows, Linux o macOS: **"escribes una vez, corre en cualquier lugar"**.

Si tu ordenador es un cachorro muy listo, pero con cero iniciativas, Java es uno de los idiomas que puedes usar para darle órdenes. Y la JVM es el intérprete que traduce tus órdenes al idioma concreto de cada cachorro.

🔍 ¿De dónde viene este lenguaje?

Java nació en **1995** en **Sun Microsystems**, y su historia tiene una de las curiosidades más divertidas del sector: se inspiró en la **máquina de café** de la oficina. Por eso el logo es una taza humeante. No me lo invento.

La idea de sus creadores era simple, pero ambiciosa: un lenguaje que funcionara en **cualquier dispositivo**, sin importar el sistema operativo ni el hardware. Antes de Java, cada plataforma necesitaba su propia versión del programa. Java propuso la solución: no programar para el ordenador, sino para una **máquina virtual** que el ordenador simula.

💡 **Dato freak:** el lenguaje se llamó primero *Oak* (roble), por un árbol que se veía desde la oficina. Tuvieron que cambiarlo por motivos de marca registrada y, según la leyenda, el nombre "Java" se eligió por el café que toman durante las reuniones.

📦 ¿Qué es eso del bytecode?

Cuando escribes Java no escribes instrucciones de la CPU de tu ordenador: escribes instrucciones para la **JVM**. El proceso es así:

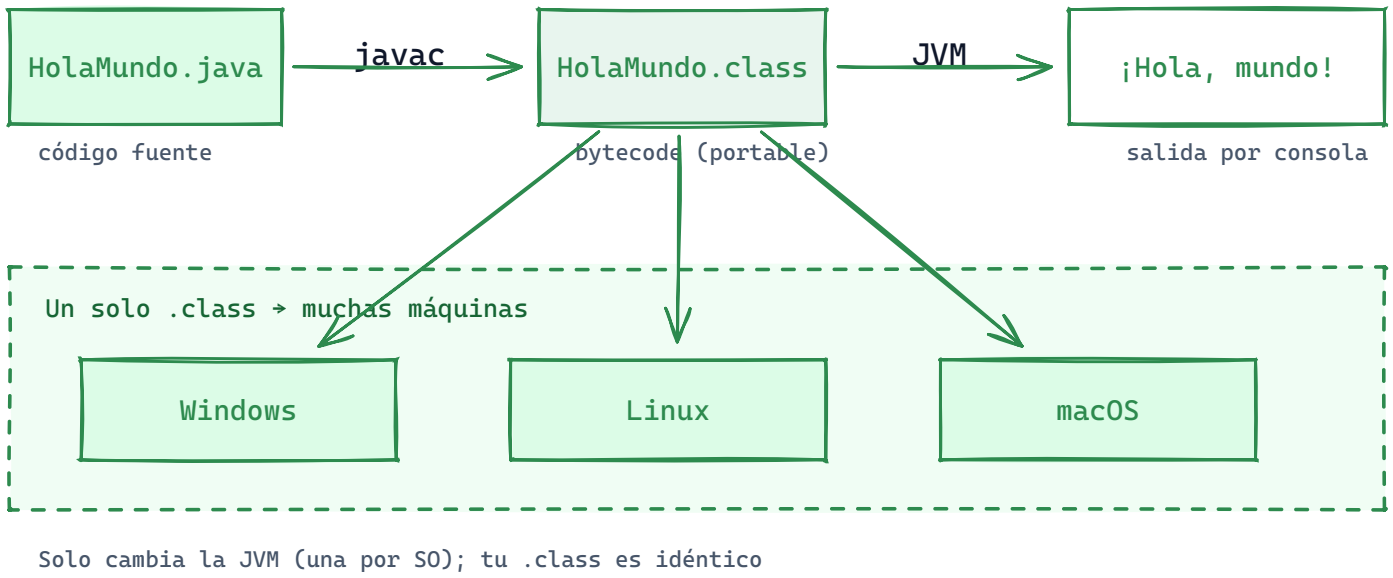
1. Tú escribes código fuente en un archivo `.java`.
2. El **compilador** (`javac`) lo traduce a *bytecode*: un idioma intermedio que entiende la JVM. El resultado es un archivo `.class`.

3. La **JVM** lee el bytecode y lo ejecuta en tu máquina concreta.

```
Tu código (.java)  --javac-->  bytecode (.class)  --JVM-->  ¡se ejecuta!
```

La gracia es que ese `.class` es el mismo para todas las plataformas: la JVM de Windows, la de Linux y la de macOS saben leerlo. Solo cambia la JVM, no tu programa.

Compilar y ejecutar: el viaje de tu programa



☕ La trilogía del Café: JVM, JRE y JDK

Aquí está el momento estrella de la unidad. Java funciona como una **cafetería de especialidad**, y tienes que memorizar quién es quién:

Concepto	Definición	Analogía
JVM	La máquina que ejecuta el *bytecode*.	La máquina de café: tiene su receta y funciona igual en cualquier lugar.
JRE	Todo lo necesario para ejecutar Java.	La cafetería entera: máquina, vasos, azúcar…
JDK	Todo lo necesario para crear programas.	El kit completo para montar tu cafetería: máquina, granos, molinillo y manual de barista.

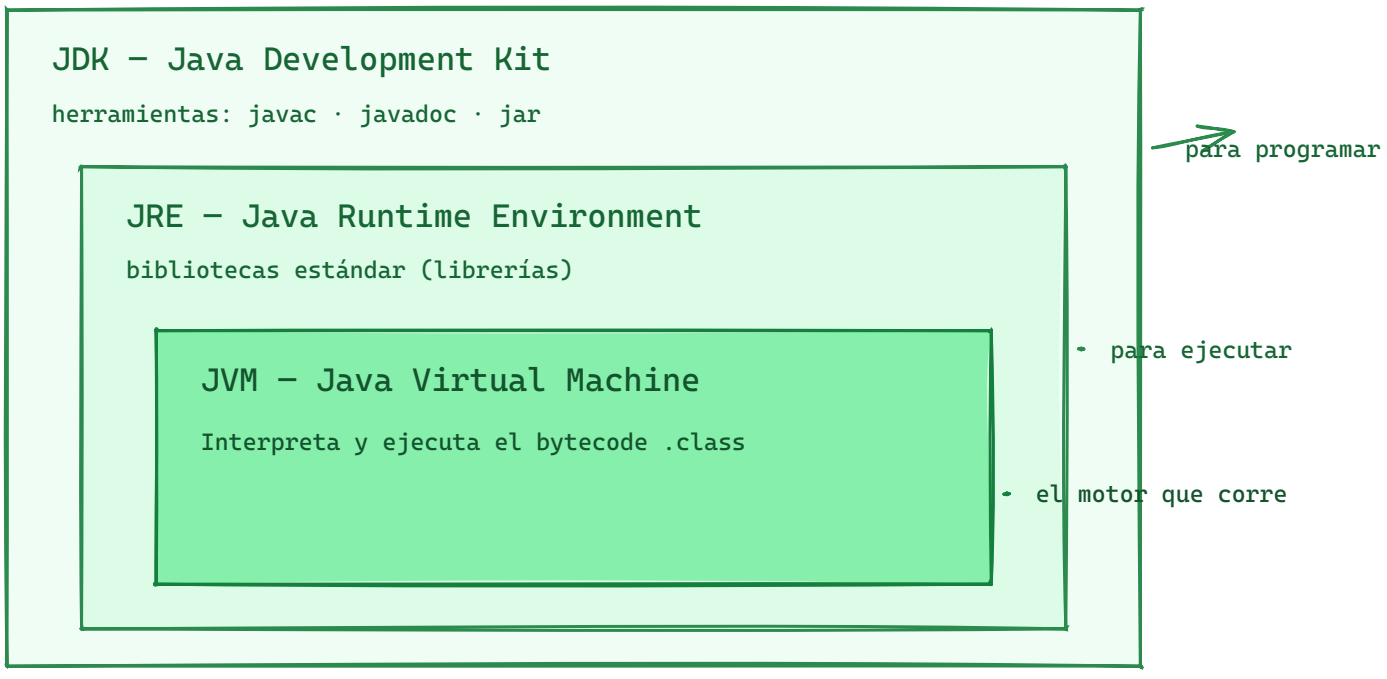
- **JVM (Java Virtual Machine)**: la máquina de café. Ejecuta el bytecode y viaja con tu programa a todas partes.
- **JRE (Java Runtime Environment)**: la cafetería entera. Tiene la JVM y las bibliotecas necesarias para *ejecutar* café ya hecho. Si solo quieres *ejecutar* programas, te basta.
- **JDK (Java Development Kit)**: el kit para montar tu propia cafetería. Incluye el JRE más el compilador `javac`, el generador de documentación `javadoc` y otras herramientas de desarrollo. Si quieres *crear* programas, lo necesitas.

```
// Comentario de cómo funciona el código
public class Cafe {
    public static void main(String[] args) {
        System.out.println("☕ ¡Café preparado!");
    }
}
```

El JDK compila esto a bytecode (café molido), el JRE lo pasa por la JVM y… ¡tarán! Café en tu pantalla.

⚠️ **Advertencia:** no confundas JDK con JRE. El JDK es el *cuchillo del chef*; el JRE es el *plato servido*. El JDK incluye el JRE, así que instalando el JDK tienes las dos cosas. Instalar solo el JRE te permite ejecutar programas, pero no crearlos.

La trilogía del café: JDK, JRE y JVM



Café: JVM = máquina · JRE = cafetería · JDK = kit completo

🌍 ¿Por qué sigue vivo Java tantos años después?

No es nostalgia. Java sigue siendo uno de los lenguajes más demandados del mundo porque:

- **Es multiplataforma**: el mismo programa corre en móviles, servidores, cajeros e incluso en la lavadora inteligente.
- **Domina el mundo empresarial**: las grandes empresas (banca, seguros, logística) llevan décadas construyendo sus sistemas en Java.
- **Es el lenguaje oficial de Android** (en su variante Kotlin convive, pero Java sigue omnipresente).
- **Tiene una comunidad enorme**: cualquier error que tengas, probablemente alguien ya lo resolvió hace diez años en un foro.
- **Es exigente, y eso te hace mejor**: Java te obliga a ser ordenado. Los programadores que aprenden con Java suelen escribir código más limpio.

🎯 Mini-comprobación

Pon a prueba en 30 segundos (las respuestas están más adelante):

1. ¿Cuál es la diferencia entre JDK y JRE en una frase?
2. ¿Qué hace el compilador javac con tu código `.java`?
3. ¿Por qué el logo de Java es una taza humeante?

🔁 Respuestas

1. El JDK sirve para crear programas (incluye el compilador); el JRE sirve para ejecutarlos. El JDK contiene el JRE.
2. Lo traduce a bytecode (un archivo `.class`) que la JVM puede ejecutar.
3. Porque el lenguaje se inspira en la máquina de café de la oficina de Sun Microsystems: “escribes una vez, corre en cualquier lugar” (como el café, que se prepara igual en cualquier cafetería).

✅ Resumen en 3 frases

1. Java ejecuta su código dentro de una **máquina virtual (JVM)**, lo que le da portabilidad total.
2. **JDK** crea (compila a bytecode), **JRE** ejecuta, **JVM** transporta: como Amazon pero con café.
3. Java sigue siendo rey en el mundo empresarial y aprenderlo te forma como programador ordenado.

🐛 Vocabulario rápido

Término	Idea general
JVM	La máquina virtual que ejecuta el bytecode
JRE	Entorno de ejecución (JVM + bibliotecas)
JDK	Kit de desarrollo (compilador + JRE + herramientas)
Bytecode	Código intermedio que entiende la JVM
Compilador	Traductor de código fuente a bytecode

02 — Instalación del JDK

OpenJDK, java -version, PATH, IDE ☕

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 02 · Instalación del JDK

Para crear programas Java necesitas el JDK. Instalarlo es más fácil que montar un mueble de Ikea, y no te sobrarán tornillos.

En el punto anterior viste que el JDK es el kit completo para *crear* programas. Ahora toca ponerlo en tu máquina y comprobar que todo funciona.

🛒 ¿Qué JDK instalo?

El Java original de Sun/Oracle ha tenido una vida movida con las licencias. Hoy la mejor opción y la más sensata para estudiar es ir directamente a la fuente oficial y libre:

1. **OpenJDK**: Es el proyecto de referencia, libre y de código abierto, del cual salen todas las demás distribuciones. **Es nuestra opción principal y la recomendada, más sencillo de instalar en Linux.**
    - https://openjdk.org/index.html
2. **Eclipse Temurin** (antes AdoptOpenJDK): Una distribución excelente basada directamente en OpenJDK, libre, gratuita y mantenida por la fundación Eclipse.
    - https://adoptium.net/es/temurin/releases
3. **Oracle JDK**: La versión comercial de Oracle. Válida para entornos empresariales, pero con una licencia más restrictiva. **Más sencillo de instalar en Windows**
    - https://www.oracle.com/java/technologies/downloads/

La versión a elegir: Elegirás la última versión **LTS** (*soporte a largo plazo*). Hoy en día, cualquier versión 17, 26 o superior te sirve para todo el curso. No te obsesiones con el número exacto: todo lo que aprendemos funciona exactamente igual en todas ellas.

🚀 Los primeros pasos

Paso 1: Instalar

Ejecuta el instalador de OpenJDK y acepta los valores por defecto. En Windows, asegúrate de marcar la opción de añadir el JDK al PATH si te la ofrece el instalador (así podrás usar `java` y `javac` desde cualquier terminal sin tener que configurar nada a mano).

Paso 2: Verificar

Abre una terminal (PowerShell o CMD en Windows, o la terminal en Linux/macOS) y escribe:

```
> java -version
openjdk version "21.0.2" 2024-01-16
OpenJDK Runtime Environment (build 21.0.2+13-LTS)
OpenJDK 64-Bit Server VM (build 21.0.2+13-LTS, mixed mode)
```

Y después comprueba el compilador:

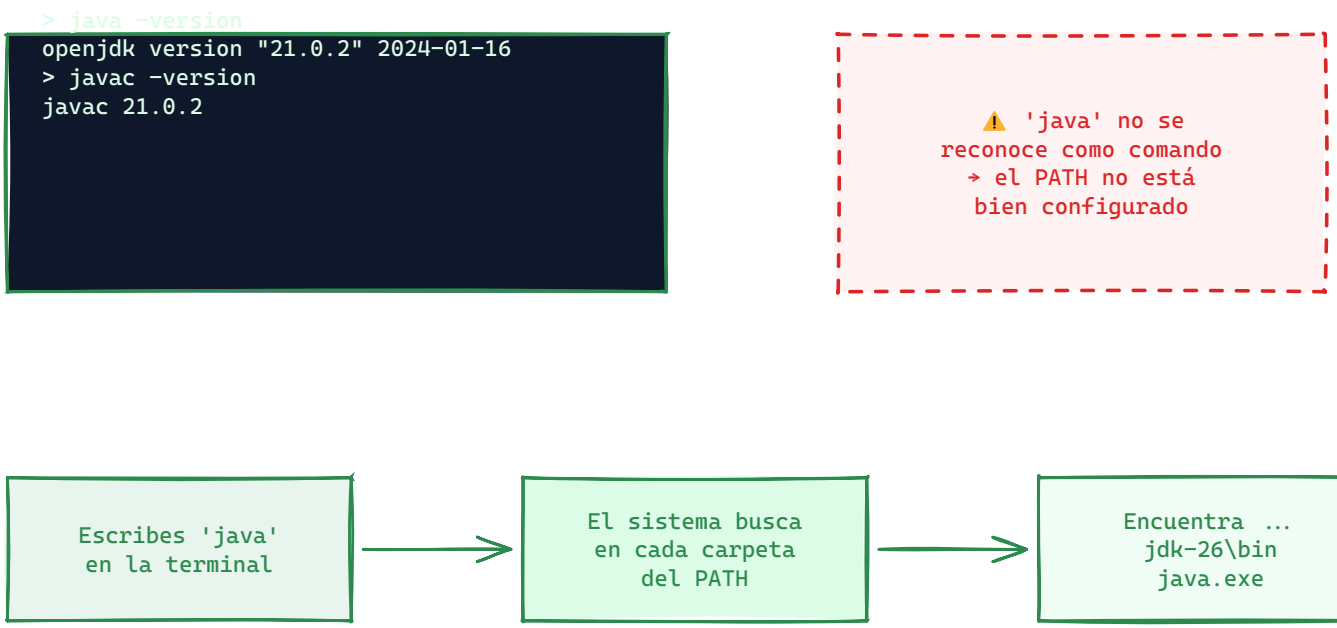
```
> javac -version
javac 21.0.2
```

Si ves una salida parecida, enhorabuena. ¡Tienes poderes de compilación activos.

Si, en cambio, el sistema te dice `'java' no se reconoce como un comando interno o externo`, significa que el PATH no está bien configurado: busca en el menú de Windows “Editar las variables de entorno del sistema” y añade la ruta de la carpeta `bin` de tu instalación de OpenJDK a la variable `Path`.

💡 ¿Qué es el PATH?

El “PATH” es la lista de carpetas donde tu sistema operativo busca los programas y comandos que escribes en la terminal. Si la carpeta `...\jdk-26\bin` (o similar) está en el PATH, al escribir `java` el sistema la encuentra a la primera. Sin esa configuración, el sistema no sabe dónde está instalado tu JDK.



Paso 3: ¿Por qué hay dos comandos?

- **javac**: Es el compilador. Convierte tu código fuente (`.java`) en bytecode (`.class`).
- **java**: Es el ejecutor. Arranca la JVM para ejecutar el bytecode que has compilado previamente.

Se necesitan los dos: primero **javac** traduce tu código y después **java** lo pone en marcha. Los verás trabajar estrechamente durante todo el curso.

🛠️ El IDE: tu navaja suiza

El JDK es el motor, pero la mayor parte del tiempo no escribirás código en un bloc de notas: usarás un IDE (Entorno de Desarrollo Integrado) o un editor avanzado como VS Code. El IDE reúne en una sola aplicación:

- Editor de código con colores, autocompletado y resaltado de errores mientras escribes.
- Compilador y ejecutor con un botón: ya no necesitas teclear `javac` y `java` a mano (aunque es bueno saber cómo funciona).
- Depurador integrado.
- Gestión de proyectos: tus programas no son solo archivos sueltos, sino proyectos con estructura.

IDE / Editor	Puntos fuertes
VS Code	La opción recomendada. Muy ligero, moderno y altamente personalizable. Con la extensión Extension Pack for Java ofrece un entorno completo y excelente.
IntelliJ IDEA (Community)	El favorito del sector profesional; autocompletado bestial. Un poco más pesado al arrancar.
NetBeans	Simple, oficial de Oracle, perfecto para empezar.
Eclipse	Clásico, mucho usado en empresas, un pelín más denso.

💡 **Recomendación para el curso:** VS Code con el plugin de Java (Extension Pack for Java) es la mejor opción por su ligereza y versatilidad. También puedes usar IntelliJ IDEA Community Edition si tu ordenador lo soporta, o NetBeans si quieres algo más tradicional. Todos valen: el IDE es una herramienta, no el objetivo.

🏫 Ejemplo guiado: tu primer proyecto en VS Code

Lo dejaremos todo montado antes de escribir código:

1. Abre VS Code y asegúrate de tener instalado el plugin Extension Pack for Java.
2. Pulsa `Ctrl + Shift + P` (o `Cmd + Shift + P` en Mac) para abrir la paleta de comandos, escribe `Java: Create Java Project` y selecciona `No build tools`.
3. Elige la carpeta donde quieres guardar el proyecto y dale un nombre (por ejemplo, `MiPrimerCurso`).
4. VS Code te creará la estructura del proyecto con una carpeta `src`.
5. Dentro de la carpeta `src`, crea un archivo llamado `HolaMundo.java` y escribe dentro:

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola, Mundo! Llevo años esperando a que me crearas.");
    }
}
```

6. Haz clic en el botón Run ▶ (que aparecerá justo encima del método main) o pulsa `F5` y mira la terminal/consola.

Si ves el mensaje en la pantalla, tu puestito está montado: JDK + VS Code + primer programa. Estás oficialmente dentro.

⚠️ **Advertencia:** no confundas la consola/terminal del IDE con la terminal del sistema. La consola de VS Code es donde se imprimen los `System.out.println` al ejecutar. Si no ves la salida, busca la pestaña Terminal o Output / Salida.

🎯 Mini-comprobación

1. ¿Qué comando comprueba que tu instalación funciona?
2. ¿Para qué sirve el PATH y qué pasa si el JDK no está en él?
3. ¿Cuál es la diferencia entre `java` y `javac`?

🔄 Respuestas

1. `java -version` (y también `javac -version`).
2. El PATH es la lista de carpetas donde el sistema busca los comandos. Sin él, al escribir `java` el sistema responde que no reconoce el comando.
3. `javac` compila el código fuente a bytecode; `java` ejecuta el bytecode con la JVM.

✅ Resumen en 3 frases

1. Instala OpenJDK (un JDK libre) y verifica con `java -version` y `javac -version`.
2. `javac` compila, `java` ejecuta, y el PATH es la dirección postal que el sistema necesita para encontrarlos.
3. Un IDE o editor como VS Code (con el plugin de Java) reúne editor, compilador y depurador: es tu navaja suiza.

🐛 Vocabulario rápido

Término	Idea general
PATH	Lista de carpetas donde el sistema busca comandos
IDE / Editor	Entorno integrado: editor + compilador + depurador
LTS	Versión con soporte a largo plazo (la recomendada)
Terminal	La ventana donde escribes comandos como `java`
Proyecto	La carpeta donde viven tus clases y configuraciones

03 — Tu primer programa

Hola Mundo, estructura, el método main 💻

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 03 · Tu primer programa

Programar es hablarle a un extraterrestre muy literal: si le dices "saluda", no lo hace. Tienes que decirle cómo, *cuándo* y *por qué*.

El ordenador es tonto, pero preciso: no interpreta, **ejecuta**. Cada línea, en orden, sin saltarse ninguna. Tu primer programa va a gritar "¡Hola!" por la consola, y a partir de aquí todo es añadir más órdenes.

👋 Hola Mundo, el ritual de iniciación

Todo programador empieza por aquí. Es como el primer café de la mañana: no es opcional.

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola, Mundo! Llevo años esperando a que me crearas.");
    }
}
```

Compílalo (`javac HolaMundo.java`) y ejecútalo (`java HolaMundo`), o pulsa el botón ▶ de tu IDE. La consola dirá:

```
¡Hola, Mundo! Llevo años esperando a que me crearas.
```

🔬 Disección de Hola Mundo

Diseccionaremos esto como si fuera una rana en biología:

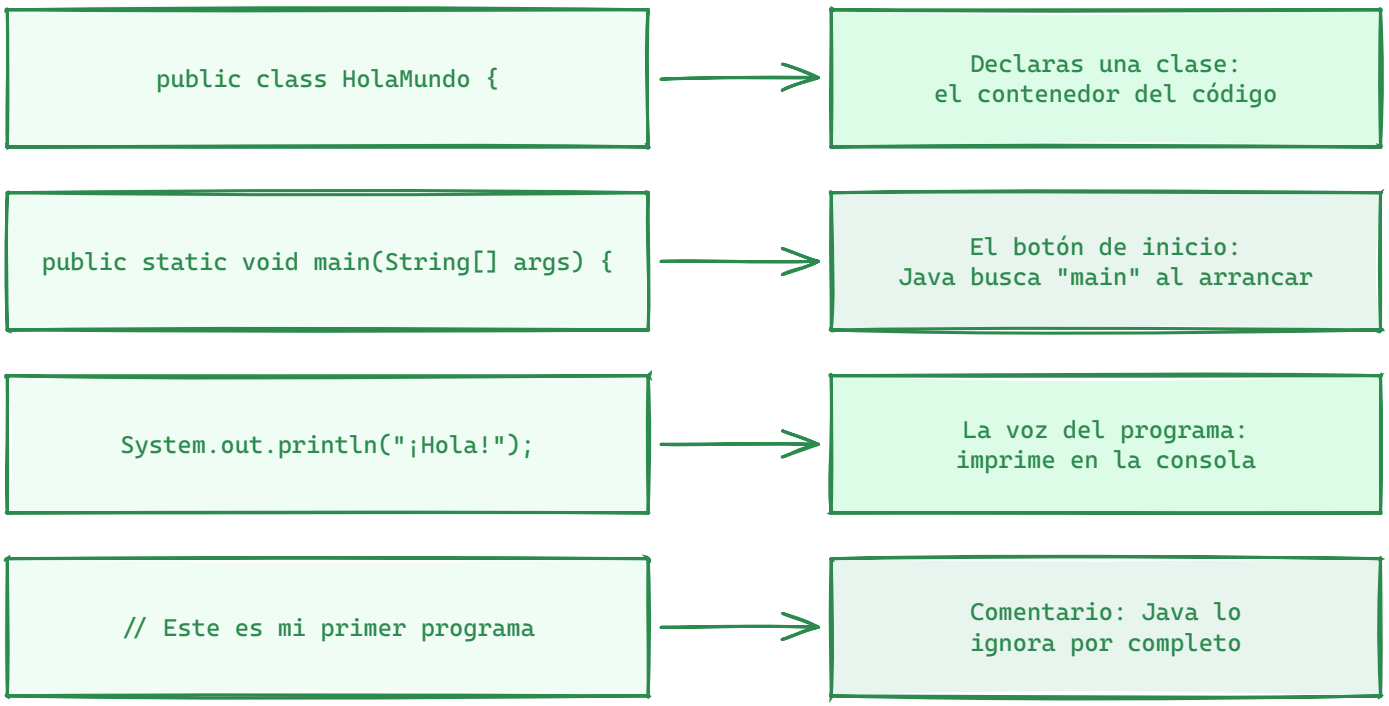
- **`public class HolaMundo`**: declaras una clase. Piensa que le dices a Java: "Oye, voy a crear una cosa que se llama HolaMundo". `public` significa que es accesible desde fuera y la clase tiene que llamarse igual que el archivo.

- **`public static void main(String[] args)`**: este es el **botón de inicio**. Cuando ejecutas el programa, Java busca esta línea y dice "¡por aquí se empieza!".
- **`System.out.println(...)`**: es **la voz** del programa. Le dices que grite algo por la consola. `print` imprime sin saltar de línea; `println` imprime y salta de línea.

```
public class MiPrimerPrograma {
    public static void main(String[] args) {
        // Este es mi primer programa
        System.out.println("¡Holaaaa, mundo!");
        System.out.println("Estoy aprendiendo Java");
        System.out.println("Y me está gustando (de momento)");
    }
}
```

💡 **Detalle práctico:** cada instrucción termina con `;`. Es el punto final de cada frase. Sin él, el compilador piensa que la frase continúa y se enlaza. Los `{}` delimitan los bloques: los de la clase contienen la clase, los del main contienen las órdenes.

Anatomía de un Hola Mundo



Java solo ejecuta lo que está dentro de main

🔑 ¿Por qué `public static void main(String[] args)`?

Parece un hechizo de Harry Potter. Desgranamos cada palabra:

Palabra	Qué significa
public	Java puede encontrarlo desde fuera: el botón es visible
static	Puede llamarse sin necesidad de crear un objeto
void	No devuelve ningún valor: hace su trabajo y se calla
main	El nombre exacto que Java busca al arrancar. No vale otro
String[] args	Un bolsillo donde puedes meter argumentos al ejecutar (punto 6)

El main es **obligatorio tal cual**. Si cambias `main` por `inicio`, Java no lo encuentra y el programa no hace nada.

🚪 El método que no se llama

Aquí va una de las trampas favoritas en los exámenes. Observa:

```
public class Saludos {
    public static void main(String[] args) {
        System.out.println("¡Hola desde el método main!");
    }

    public static void saludo() {
        System.out.println("Esto nunca se ejecuta...");
    }
}
```

¿Se ejecutará correctamente? **Sí**, pero solo imprime la primera línea. El método `saludo()` existe, pero como nunca lo llamas desde `main`, se queda aquí haciendo el vago. Java solo ejecuta lo que está dentro del `main` (salvo que explícitamente llames otros métodos). El método `saludo()` es como un actor que tiene el guion aprendido, pero nunca sube al escenario.

🧠 **Truco de memoria:** `main` es la puerta de entrada de la casa. Puede haber muchas habitaciones (métodos), pero nadie entra por la ventana. Si no llamas a la puerta, las habitaciones se quedan vacías.

⭐ Intenta pensar como el código: tú eres la JVM

Serás Java por un momento. Toma papel y boli (o mentalmente). Te dan este código:

```
public class Ordenador {
    public static void main(String[] args) {
        int x = 5;
        int y = 10;
        int z = x + y;
        System.out.println("El resultado es: " + z);
    }
}
```

Sigue los pasos como si fueras la JVM:

1. Encuentras la clase `Ordenador`.
2. Buscas el método `main` — aquí está.
3. Creas un espacio llamado `x` y metes un 5.
4. Creas `y` y metes un 10.
5. Creas `z`, sumas `x` e `y` (15), lo guardas.
6. Gritas por pantalla: “El resultado es: 15”.

🔄 Solución

La salida es exactamente:

```
El resultado es: 15
```

Si tu respuesta es diferente, vuelve a empezar: el ordenador no interpreta, *ejecuta*, línea a línea.

🎯 Mini-comprobación

1. ¿Qué pasa si renombro `main` a `inicio`?

2. ¿Imprime lo mismo `System.out.print("Hola")` que `System.out.println("Hola")`?
3. ¿Por qué falta un `;` en el código de abajo? Señala el error:

```
public class Erroneo {
    public static void main(String[] args) {
        System.out.println("Me falta algo")
    }
}
```

🔄 Respuestas

1. Java no encuentra la puerta de entrada: el programa compila (en algunos entornos) pero al ejecutar no hace nada, o da error. La firma tiene que ser exacta.
2. No. `print` no salta de línea; `println` sí. En pantalla se nota: `print("Hola")` seguido de `print("Mundo")` muestra `HolaMundo` pegado.
3. Falta el `;` al final de `System.out.println("Me falta algo")`. El compilador se quejará esperando el punto y coma.

✅ Resumen en 3 frases

1. Todo programa tiene una **clase** (contenedor) y un **método main** (puerta de entrada).
2. `System.out.println()` es la voz del programa y el **`";"`** es el punto final de cada frase.
3. La JVM ejecuta **línea a línea, en orden**: tú decides qué entra por la puerta.

🐛 Vocabulario rápido

Término	Idea general
Clase	El contenedor del código (una “cosa” en Java)
Método	Un bloque de órdenes con nombre
main	El método que Java ejecuta al arrancar
println	Imprime texto y salta de línea
Consola / Terminal	La ventana de texto donde se imprime la salida del programa

04 — El depurador

Breakpoints, step over/into, watch 🦎

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 04 · El depurador

El depurador (debugger) es como tener visión de rayos X para tu código: para el programa donde tú quieres y te enseña el valor de cada variable en cada instante.

Tu primer programa funciona. Pero pronto tendrás un programa raro: una variable `edad` que sale 25 cuando debería salir 18. ¿Qué haces? ¿Le pegas al ordenador? No. Usas el depurador.

🔍 ¿Qué es depurar?

Depurar (debug) es sacarle los bichos (bugs) al programa. Y el secreto de los programadores experimentados no es adivinar: es ver. El depurador te permite:

- Parar el programa en una línea concreta.
- Avanzar instrucción a instrucción.
- Inspeccionar el valor de las variables en cada momento.
- Modificar valores sobre la marcha.

Es como ver una serie de crímenes en cámara lenta: puedes pausar, observar quién hace qué, y analizar cada detalle.

💡 **Detalle práctico:** Los errores se dividen en dos familias. Los de compilación los atrapa `javac` (o el propio VS Code resaltándolo en rojo) y te dice la línea. Los de lógica compilan perfecto pero hacen lo que no deben: para esos no hay mejor arma que el depurador.

🛠️ Las cuatro herramientas del detective en VS Code

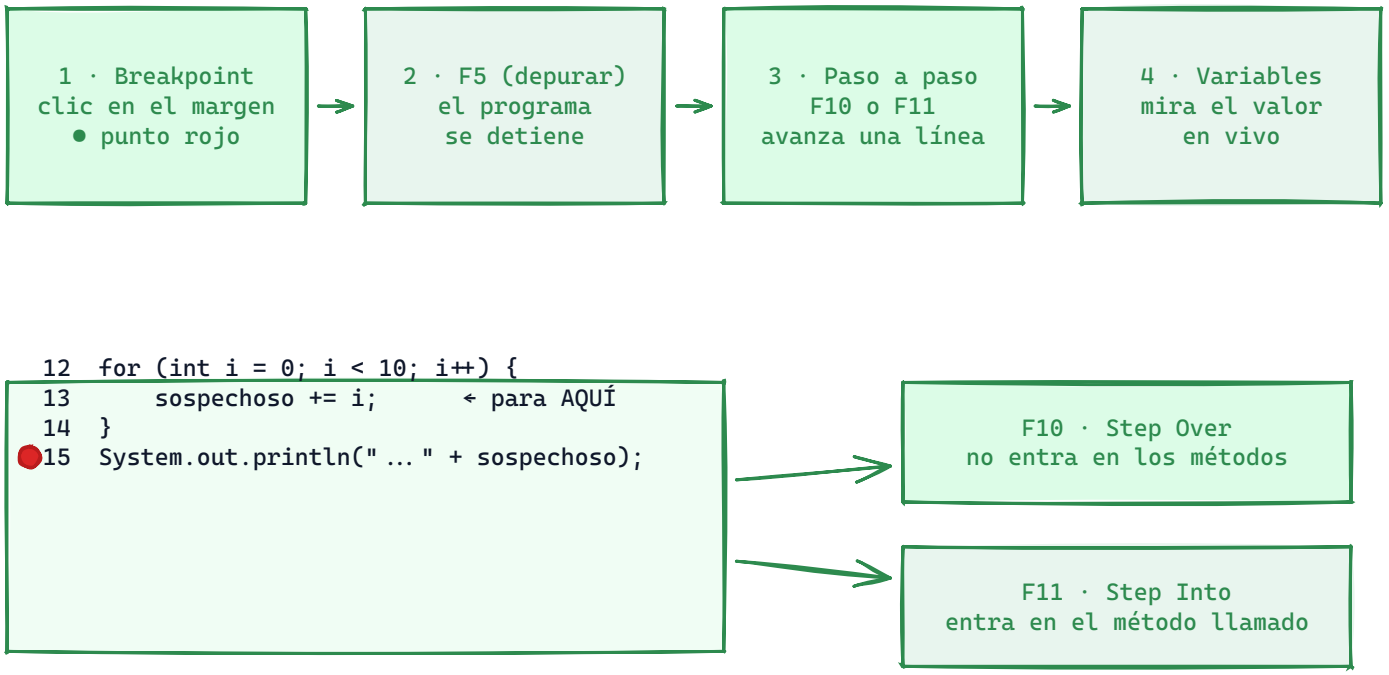
Herramienta	Atajo (VS Code)	Qué hace
Breakpoint (punto de ruptura)	Clic a la izquierda del número de línea	Le dices a Java “para AQUÍ, quiero ver qué pasa”.

Herramienta	Atajo (VS Code)	Qué hace
Step Over	F10	“Ejecuta esta línea pero no me cuentes los detalles internos”.
Step Into	F11	“Ejecuta esta línea Y llévame dentro de esa llamada, quiero espiar”.
Variables / Watch	Panel lateral de depuración	“Enséñame el valor de la variable AHORA MISMO”.

- **Breakpoint**: Marca una línea con un punto rojo. El programa se detiene justo antes de ejecutarla y espera tus órdenes.
- **Step Over (F10):** Ejecuta la línea completa de golpe, sin entrar en los métodos que llama.
- **Step Into (F11):** Entra dentro del método llamado para ver sus líneas una a una.
- **Variables / Watch:** El panel izquierdo en la vista de depuración de VS Code (Run and Debug) que muestra las variables vivas y sus valores actuales. En Watch puedes añadir expresiones concretas para evaluarlas en tiempo real.

⚠️ **Advertencia:** Si te pierdes dentro de un método ajeno con Step Into, pulsa Step Out (`Shift + F11`): sales del método y vuelves al punto donde lo llamaste. Es el botón “he entrado sin querer, sáquenme de aquí”.

Depurar paso a paso (VS Code)



- El punto rojo detiene el programa antes de esa línea

🏫 Ejemplo guiado: el caso del sospechoso en VS Code

Aquí tienes un programa perfecto para estrenar el depurador:

```
public class DetectivesDeCodigo {
    public static void main(String[] args) {
        int sospechoso = 0;
        for (int i = 0; i < 10; i++) {
            sospechoso += i; // Pon un breakpoint aquí
        }
        System.out.println("El culpable es: " + sospechoso);
    }
}
```

El bucle `for` lo verás a fondo en la U04, pero lo puedes intuir: repite la línea de dentro mientras `i` sea menor que 10, con `i` valiendo 0, 1, 2… La línea clave es `sospechoso += i` (que es lo mismo que `sospechoso = sospechoso + i`).

El ejercicio de detective:

1. Pon un breakpoint en la línea de `sospechoso += i` (haz clic a la izquierda del número de línea; aparecerá un punto rojo fijo).
2. Ejecuta en modo depuración: pulsa `F5` o haz clic en el icono del bicho con el triángulo ▶ (Run and Debug / Start Debugging) en la barra flotante superior o en la pestaña izquierda.
3. El programa se detiene y esa línea se ilumina en amarillo. Mira el panel Variables a la izquierda: verás que `sospechoso` vale 0 e `i` vale 0.
4. Pulsa Step Over (`F10`) varias veces y observa cómo cambian `sospechoso` e `i` en el panel lateral en cada vuelta.

🔄 Qué deberías ver

Valores de `sospechoso` en cada parada: 0, 1, 3, 6, 10, 15, 21, 28, 36, 45 y, al terminar el bucle, 55. El valor final impreso en la terminal de VS Code es “El culpable es: 55”. Si en tu depurador el valor no coincide, tienes delante un bug real para diagnosticar. Esto es depurar.

🧠 La regla de oro del depurador

Cuando algo falla, no adivines: observa. El proceso es siempre el mismo:

1. Reproduce el fallo (que ocurra siempre que puedas).
2. Pon un breakpoint antes de la zona sospechosa.
3. Avanza con `F10` y observa las variables en el panel lateral hasta localizar la línea donde el valor se tuerce.
4. Al ver dónde se desvía el valor, habrás encontrado el bug.

💡 **Consejo:** Si el programa se ejecuta hasta el final sin detenerse, es que el breakpoint está en una línea que nunca se alcanza (por ejemplo, dentro de un método que nadie llama o en una condición `if` que no se cumple). El breakpoint no se dispara: otra pista de detective.

🎯 Mini-comprobación

1. ¿Cuál es la diferencia entre Step Over (F10) y Step Into (F11)?
2. Si pones un breakpoint y el programa no se detiene, ¿qué sospechas?
3. ¿En qué modo hay que ejecutar el programa para que los breakpoints funcionen?

🔄 Respuestas

1. Step Over (F10) ejecuta la línea entera sin entrar en los métodos que llama; Step Into (F11) entra dentro del método para ver sus líneas una a una.
2. Que el breakpoint está en una línea que nunca se ejecuta (código muerto, método no llamado, condición que no se cumple).
3. En modo depuración (pulsando `F5` / icono del bicho 🐞 Start Debugging), no en modo ejecución normal (Run Without Debugging).

✅ Resumen en 3 frases

1. El depurador para el programa donde tú marcas (breakpoint) y te muestra las variables en vivo en el panel lateral de VS Code.
2. Step Over (F10) no entra en los métodos; Step Into (F11) sí; el panel Variables / Watch te muestra los valores en tiempo real.
3. Ante un bug, no adivines: reproduce, para, avanza y observa hasta encontrar dónde se tuerce el valor.

🐛 Vocabulario rápido

Término	Idea general
Bug	Un error en el programa
Debug	Proceso de encontrar y arreglar bugs
Breakpoint	Marca (punto rojo) donde el programa se detiene
Step Over / Into	Avanzar sin entrar (F10) / entrando (F11) en métodos
Watch / Variables	Panel lateral de VS Code que muestra variables y expresiones

05 — Comentarios y documentación

//, /* */, /** */ y Javadoc 📝

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 05 · Comentarios y documentación

Un comentario es un mensaje que te dejas a ti mismo (o a otros). El ordenador lo ignora completamente: es para los humanos, no para las máquinas.

El código explica *qué* hace la máquina; los comentarios explican *por qué* lo hace. Y el *por qué* es oro puro: seis meses después, ese comentario te ahorrará horas de "*qué* estaba pensando cuando escribí esto".

📝 Los tres tipos de comentarios

Java tiene tres formas de escribir comentarios, cada una con su uso:

Tipo	Sintaxis	Uso
De una línea	`// text`	Notas rápidas al lado del código para aclarar una línea o lista de tareas.
De bloque	`/* text */`	Explicaciones largas, algoritmos complejos o desactivar temporalmente código.
Javadoc	`/** text */`	Documentación formal y estructurada de clases, métodos y atributos.

```java
// Comentario de una línea: "Aquí va la magia"

/*
Comentario de varias líneas:
"Si esto funciona, no lo toques.
Si no funciona, no lo toques tampoco.
Ya llamaremos a alguien."
*/

/**
 * Comentario Javadoc (el elegante):
 * Sirve para generar documentación automática.
 * @param args argumentos de la línea de comandos
 * @return nada, esto es void, ¿no te das cuenta?
 */
```

💡 **Detalle práctico:** los comentarios pueden ir en medio de una línea sin problema. `System.out.println(/* "Cuatro" */ "Cinco")` imprime `Cinco`: el comentario se ignora y el resto de la línea sigue viva.

🔑 El consejo que te ahorrará horas: comenta el "por qué", no el "qué"

Este es el consejo más rendible de toda la unidad:

```java
int i = 0; // Declaro i con valor 0 ← MAL: el código ya lo dice
int i = 0; // Empezamos desde 0 porque el usuario no ha pulsado nada ← BIEN
```

Escribir `// Declaro i con valor 0` es como poner "Abro la puerta" en una puerta. El código ya dice eso. El comentario útil añade información que **el código no puede expresar**: el contexto, la intención, la decisión de diseño.

- ❌ `int temperatura = 30; // temperatura vale 30`
- ✅ `int temperatura = 30; // Refresca por debajo de 25 según el jefe`

🧠 **Truco de memoria:** si el comentario describe la misma acción que ves en el código, bórralo. El buen comentario responde a *¿por qué?*, nunca a *¿qué?*.

📖 Javadoc: documentación que se genera sola

Javadoc es un comentario especial (`/** ... */`) que colocas **justo antes** de una clase o de un método. La herramienta `javadoc` (incluida en el JDK) lo convierte en páginas de documentación automáticas, iguales a las oficiales de Java.

Cuando escribes un bloque Javadoc (empieza por `/**` y termina con `*/`), puedes utilizar etiquetas especiales (tags) que empiezan por `@` para estructurar la información:

- **`@param nombreParametro`**: Explica para qué sirve un parámetro de entrada que recibe el método.
- **`@return`**: Describe qué devuelve o calcula el método (si no es void).
- **`@author`**: Indica el autor de la clase o del código.
- **`@version`**: Especifica la versión del módulo o clase.
- **`@see`**: Añade una referencia o enlace a otra clase o método relacionado.
- **`@throws Exception`**: Documenta qué tipo de excepción o error puede lanzar el método.

¿Por qué es tan útil en tu día a día?

Además de generar la web, los IDEs modernos como VS Code o IntelliJ IDEA leen estos comentarios Javadoc y, al pasar el cursor sobre cualquier método o utilizar el autocompletado, te muestran una ventana emergente con toda la documentación, parámetros y explicaciones en tiempo real sin necesidad de abrir el archivo original.

Para generarla: `javadoc SobreMi.java` en la terminal y se crearán los `.html` con la documentación. Pruébalo: es como tener un manual de instrucciones hecho por ti mismo.

💡 **Consejo:** en la vida real, los comentarios Javadoc en los métodos suelen “caer” en las rúbricas. Además, en el trabajo, los que documentan duermen mejor… y los que no, vuelven al código a las 3 de la madrugada con cara de pánico.

🏫 Ejemplo guiado: documentar un programa completo

Lo aplicaremos todo a un programa real:

```java
/**
 * Clase que saluda de distintas formas.
 *
 * @author Alumno DAW
 * @version 1.0
 */
public class Saludo {

    /**
     * Método principal: imprime los dos mensajes de la clase.
     *
     * @param args argumentos de línea de órdenes (no usados aquí)
     */
    public static void main(String[] args) {
        // Primer mensaje: bienvenida al curso
        System.out.println("Bienvenidos al curso de Programación");
        /*
         * Segundo mensaje:
         * usamos println para saltar de línea y dejar la consola limpia
         */
        System.out.println("Este curso va a molar");
    }
}
```

Fíjate que cada comentario aporta algo: el Javadoc documenta la API, el `//` explica la intención y el `/* */` justifica una decisión de formato. Ninguno repite lo que el código ya dice.

🎯 Mini-comprobación

1. ¿Cuál es la diferencia entre `//`, `/* */` y `/** */`?
2. ¿Qué imprime este programa?

```java
public class Comentarios {
    public static void main(String[] args) {
        // System.out.println("Uno");
        System.out.println("Dos");
        /* System.out.println("Tres"); */
        System.out.println(/* "Cuatro" */ "Cinco");
    }
}
```

3. ¿Es buen comentario `// x = 10`?

🔄 Respuestas

1. `//` es de una línea; `/* */` es de bloque (varias líneas); `/** */` es Javadoc, que la herramienta `javadoc` convierte en documentación automática.
2. Imprime `Dos` y `Cinco`. La línea con `//` y la del bloque `/* */` se ignoran, y en la última el comentario interno se elimina, pero "Cinco" sigue siendo el argumento del `println`.
3. No: el código ya muestra que `x` vale 10. Comenta el **por qué**, no el qué.

✅ Resumen en 3 frases

1. Los comentarios son para humanos: `//`, `/* */` y `/** */`, según lo que necesites.
2. Comenta el **por qué**, no el qué: el código ya dice qué hace.
3. **Javadoc** genera documentación automática con `@author`, `@version`, `@param` y `@return`.

🐛 Vocabulario rápido

Término	Idea general
Comentario	Texto que el compilador ignora completamente al generar el bytecode.
Javadoc	Comentario estructurado que genera documentación HTML automática.
Etiqueta	`@param`, `@return`… metadatos para describir piezas del código.
Documentación	Explicación escrita sobre el funcionamiento del código para otros desarrolladores.

Término	Idea general
Compilador	Traductor (`javac`) que ignora los comentarios y crea el archivo `.class`.

06 — Argumentos de línea de órdenes

El array args, cómo leerlo y usarlo 📬

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 06 · Argumentos de línea de órdenes

El parámetro "args" del "main" es un bolsillo donde Java mete todo lo que escribas después del nombre de tu programa al ejecutarlo.

Cuando escribes `java MiPrograma hola mundo`, esas palabras sueltas (`hola`, `mundo`) no son casuales: Java las guarda en el array `args` para que tu programa las pueda usar.

🎒 ¿Qué es "args"?

Recuerda la firma del main:

```
public static void main(String[] args)
```

`String[] args` significa: "un array de cadenas de texto llamado args". Un array es una lista de cosas (las verás a fondo más adelante en el curso). Aquí, la lista contiene **cada palabra que escribiste después del nombre de la clase al ejecutar**.

```
public class UsoDeArgumentos {
    public static void main(String[] args) {
        System.out.println("Has escrito " + args.length + " palabras:");
        for (int i = 0; i < args.length; i++) {
            System.out.println("Palabra " + (i + 1) + ": " + args[i]);
        }
    }
}
```

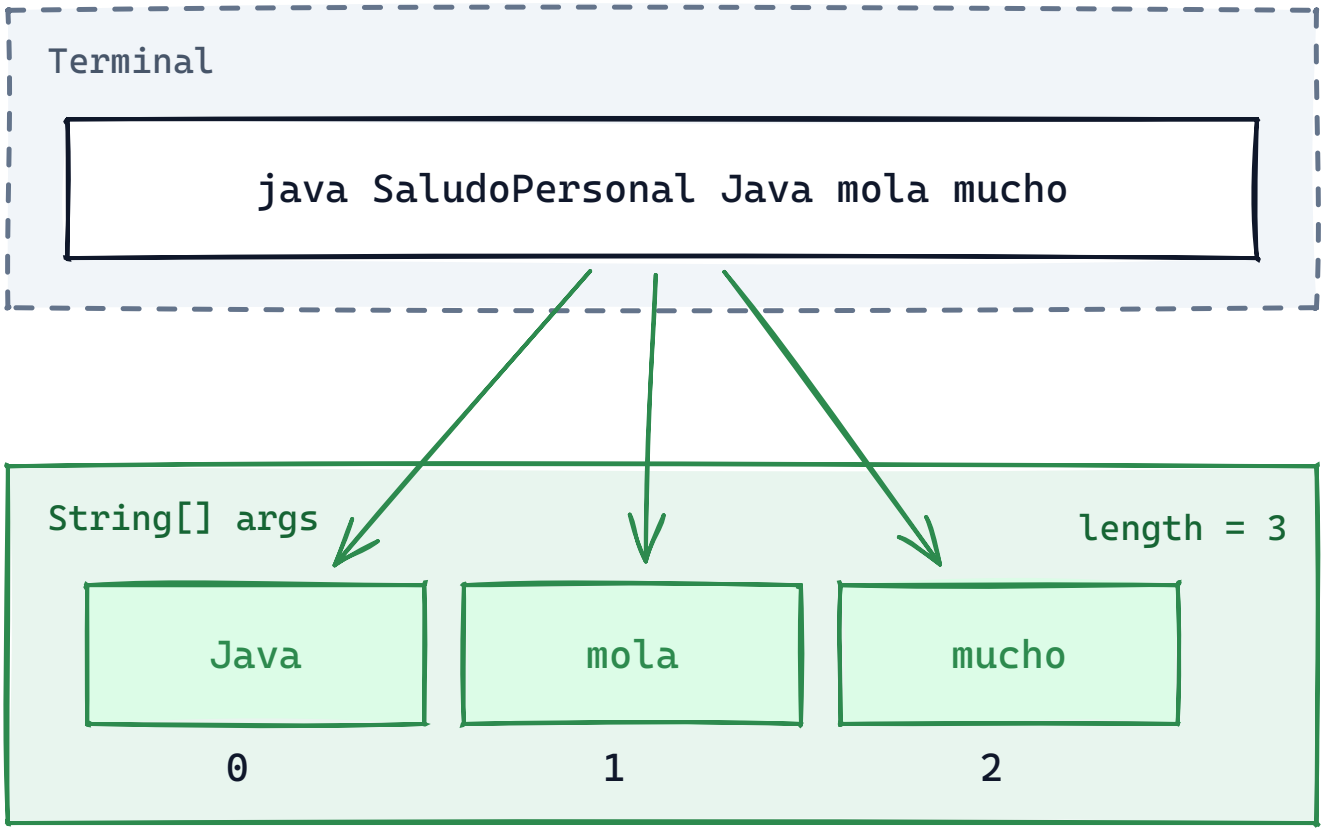
Si ejecutas:

```
> java UsoDeArgumentos Java mola mucho
```

Verás:

```
Has escrito 3 palabras:
Palabra 1: Java
Palabra 2: mola
Palabra 3: mucho
```

El array args: de la terminal al programa



🔢 Cómo se indexan las palabras

En Java, las listas empiezan a contar en 0, como casi todo en programación:

Índice	Valor
`args[0]`	“Java”
`args[1]`	“mola”
`args[2]`	“mucho”
`args.length`	3 (cuántos hay en total)

- `args.length` te dice cuántos argumentos se han pasado (en este ejemplo, 3).

- `args[0]` es el primer elemento, `args[1]` el segundo… y así sucesivamente.
- Si ejecutas el programa sin argumentos, `args.length` valdrá 0 y el array estará vacío.

⚠️ **Advertencia:** cuidado con el error del aprendiz: `args[0]` es el **primer** argumento, no "el argumento número 0". El **primer** cuenta como índice 0. Es como las plantas de un edificio: la baja es la 0.

🧠 ¿Para qué sirve pasar argumentos?

Pasar argumentos por línea de órdenes es la forma más antigua y directa de "configurar" un programa sin tocar el código:

- **Datos de entrada**: `java Calculadora 5 3` → tu programa recibe 5 y 3.
- **Modos de ejecución**: `java App --verbose` o `java App --silencioso`.
- **Archivos**: `java Convertidor entrada.txt salida.txt`.
- **Programas reales**: los comandos de la terminal (como `git status` o `ls -la`) son exactamente esto: programas que reciben argumentos.

En la práctica, el array `args` aparece cada vez que quieras que tu programa hable con "el mundo exterior" **antes** de arrancar. Más adelante usarás el `Scanner` (UO3) para pedir datos durante la ejecución; los args son la vía de entrada **antes** de ejecutarse.

🏫 Ejemplo guiado: el programa que te saluda

Vamos a hacer que tu programa sepa tu nombre sin preguntarlo:

```
public class SaludoPersonal {
    public static void main(String[] args) {
        if (args.length > 0) {
            System.out.println("Hola, " + args[0] + ". ¡Bienvenido al curso!");
        } else {
            System.out.println("Hola, desconocido. ¿Olvidaste tu nombre?");
        }
    }
}
```

Si ejecutas `java SaludoPersonal Sergi`, saluda a Sergi. Si ejecutas `java SaludoPersonal`, te recuerda que no pasaste argumentos. El `if` aquí es un aperitivo de las estructuras de control que se verán en futuras unidades.

💡 **Detalle práctico:** cuando ejecutas desde el IDE (botón ▶), puedes configurar los argumentos en el menú *Run → Edit Configurations → Program arguments*. Escribe ahí `Sergi 20` y pulsa Run: tu programa los recibirá igual que si los teclearas en la terminal.

🎯 Mini-comprobación

1. Si ejecutas `java MiPrograma uno dos tres`, ¿cuánto vale `args.length` y qué contiene `args[2]`?
2. ¿Qué pasa si accedes a `args[5]` cuando solo hay 3 argumentos?
3. ¿Cómo saludas a la primera palabra que reciba tu programa?

🔄 Respuestas

1. `args.length` vale **3** y `args[2]` contiene “tres” (los índices empiezan en 0).
2. Explotará con `ArrayIndexOutOfBoundsException`: estás pidiendo un elemento que no existe.
3. Con `args[0]`, por ejemplo: `System.out.println("Hola, " + args[0]);`.

✅ Resumen en 3 frases

1. `args` es el array de cadenas que Java crea con todo lo que escribes después del nombre de tu clase.
2. Los índices empiezan en **0**: `args[0]` es el primer argumento y `args.length` dice cuántos hay.
3. Los argumentos son la forma de dar **datos de entrada** a un programa antes de que arranque.

🐛 Vocabulario rápido

Término	Idea general
Argumento	Una palabra o valor que le pasas al programa en el momento de ejecutarlo.
args	El array (lista) de cadenas de texto donde Java guarda todos los argumentos recibidos.
Array	Una estructura de datos que guarda una lista de elementos ordenados y numerados desde 0.
args.length	La propiedad que indica la cantidad total de argumentos que tiene el array.
Línea de órdenes	La ventana de terminal o consola donde escribes comandos como `java`, para ejecutar programas.

07 — El compilador y sus errores

Compilar vs ejecutar, errores típicos del principiante ⚠️

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 07 · El compilador y sus errores

El compilador no te odia: es un profesor de lengua puntillós que marca cada coma mal puesta. Aprender a leer sus mensajes es aprender a programar.

Toda tu carrera estará llena de mensajes de error. Los aprendices los temen; los profesionales los leen como quien lee una carta. En este punto aprenderás a entender qué te dice `javac` (y por qué Java es tan puntillós).

🔍 Compilar ≠ ejecutar

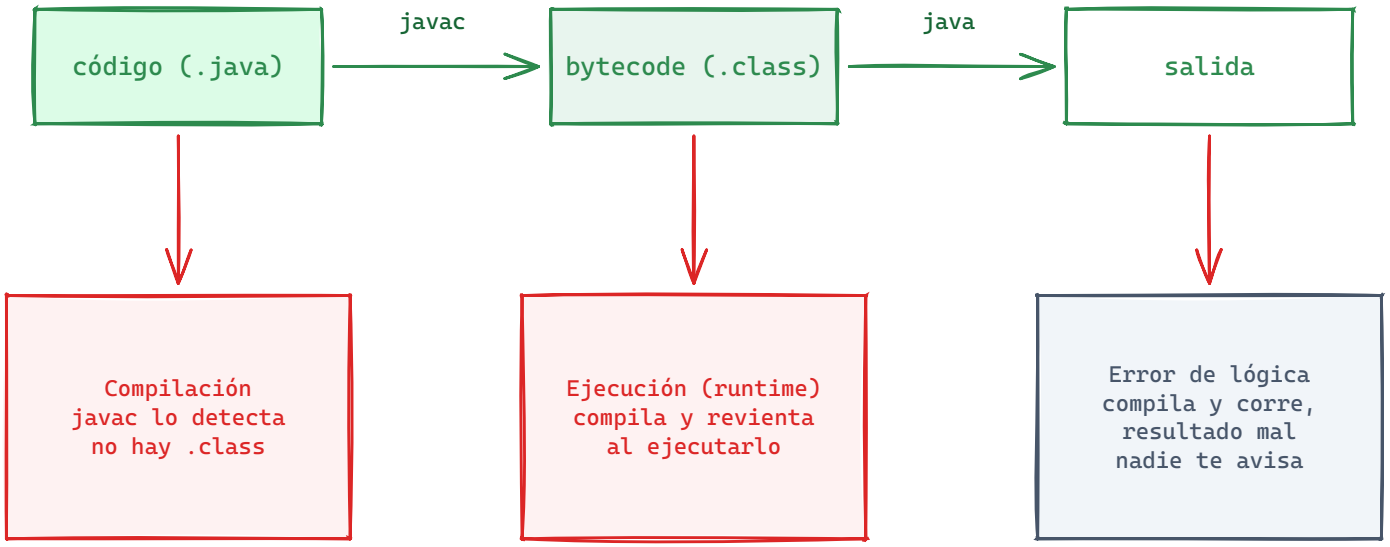
Dos momentos distintos, dos familias de errores:

Momento	Comando	Qué ocurre
Compilar	`javac MiPrograma.java`	El compilador traduce tu código a bytecode. Si hay errores de sintaxis, se queja aquí.
Ejecutar	`java MiPrograma`	La JVM ejecuta el bytecode. Aquí pueden aparecer errores de runtime (en tiempo de ejecución).

```
Tu código (.java)  --javac-->  bytecode (.class)  --java-->  salida
       (errores de compilación aquí)                (errores de ejecución aquí)
```

- **Errores de compilación**: el compilador detecta que has escrito mal alguna cosa. Te dice la **línea exacta** y el motivo. No se genera el `.class`.
- **Errores de ejecución (runtime)**: el programa compila, pero explota al ejecutar (por ejemplo, `ArrayIndexOutOfBoundsException` del punto 6).
- **Errores de lógica**: todo "funciona", pero el resultado es incorrecto. El más peligroso: ni el compilador ni el runtime te avisan. Aquí entra el depurador del punto 4.

Tres familias de errores (y dónde aparecen)



Compila javac · Ejecuta la JVM · La lógica solo la caza el depurador

---

😤 El error del aprendiz: 4 errores en un programa

Este código está fatal. Encuentra los 4 errores antes de mirar la solución:

```
Public class Calculadora
public static void main(string[] args) {
    System.out.println("Suma: " + 5 + 3)
    System.out.println("Resta: " + (5 - 3));
} }
```

🔄 Los 4 errores

1. `Public` debería ser **`public`** (minúscula). Java es sensible a las mayúsculas.
2. Falta **`{`** después de `Calculadora`. La clase necesita su llave de apertura.
3. `string[] args` debería ser **`String[] args`** (la S mayúscula importa: `String` es una clase, no existe `string`).
4. Falta el **`;`** al final de la primera línea del `println`.

Java es muy puntillós, como un profesor de lengua con las comas: `public`, `String` y `System` empiezan en mayúscula o mayúsculas concretas, y cada frase termina en `;`.

🔎 Cómo leer un mensaje de error de javac

Cuando algo no compila, `javac` te muestra algo así:

```
Calculadora.java:3: error: ';' expected
System.out.println("Suma: " + 5 + 3)
                                   ^
1 error
```

Tres datos clave:

1. **"Calculadora.java:3"** → el archivo y la **línea** donde está el problema (mira la ^).
2. **"error: ';' expected"** → el *qué*: esperaba un punto y coma.
3. **El "^"** → el punto exacto donde se ha atascado.

💡 **Consejo:** el error suele estar en la línea marcada, pero a veces es la **anterior**: si falta un `{` o un `;`, javac a veces se da cuenta una línea más tarde. Empieza mirando la línea de la ^ y, si no, sube una.

🚨 Errores típicos del aprendiz (y su remedio)

Error	Mensaje típico	Remedios
Public en lugar de public	`class, interface, or enum expected`	Los nombres y palabras clave en Java son exactos: `public`, `class`, `String`, `System`.
Falta ;	`';' expected`	Toda instrucción u orden en Java debe terminar obligatoriamente en `;`.
Falta { o }	`reached end of file while parsing` o `class, interface...`	Comprueba que cada llave de apertura `{` tenga su correspondiente llave de cierre `}`.
string en lugar de String	`cannot find symbol`	`String` es una clase y siempre se escribe con la S inicial mayúscula.
Nombre de clase diferente del archivo	`class X is public, should be declared in a file named X.java`	La clase public tiene que tener exactamente el mismo nombre que el

Error	Mensaje típico	Remedios
		archivo `.java` (respetando mayúsculas y minúsculas).
system en lugar de System	`cannot find symbol: variable system`	`System` va con S mayúscula, mientras que `out` y `println` van en minúsculas.

🧠 **Truco de memoria:** los nombres de las *clases* (`String`, `System`, `Scanner`…) empiezan en mayúscula; los nombres de *variables y métodos* (`out`, `println`, `main`) en minúscula. Cuando dudes, mira cómo lo escribió Java.

🧩 EL LÍO: el código revuelto

Tu jefe ha dejado este código hecho un desastre. Las líneas están revueltas. ¿Eres capaz de ordenarlas para que sea un programa Java válido que imprima “La suma es: 8”?

```
System.out.println("La suma es: " + (a + b));

int a = 5;

public class CalculoLioso {

System.out.println("Calculando...");

public static void main(String[] args) {

int b = 3;
```

Pista: busca primero dónde empieza la clase y el método `main`.

🔄 Solución

```java
public class CalculoLioso {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;
        System.out.println("Calculando...");
        System.out.println("La suma es: " + (a + b));
    }
}
```

La clase es el contenedor, el main es la puerta de entrada, las instrucciones van dentro del main, en orden. Como una receta: los ingredientes se declaran antes de usarlos.

🎯 Mini-comprobación

1. ¿Qué significa `Calculadora.java:3: error: ';' expected`?
2. ¿Por qué `Public` (con P mayúscula) da error?
3. ¿Cuál es la diferencia entre un error de compilación y uno de lógica?

🔄 Respuestas

1. Que en el archivo `Calculadora.java`, en la línea 3, `javac` esperaba un `;` y no lo encontró (mira la ^).
2. Porque Java es sensible a las mayúsculas: la palabra reservada es `public`, en minúscula exacta.
3. El de compilación impide generar el `.class` (lo detecta `javac`). El de lógica compila y ejecuta, pero el resultado es incorrecto: nadie te avisa, solo el depurador (o el sentido común).

✅ Resumen en 3 frases

1. Compilar (`javac`) y ejecutar (`java`) son momentos distintos con errores distintos.
2. El mensaje de error te dice **archivo, línea y motivo**: aprende a leerlo en vez de temerlo.
3. Los fallos clásicos (mayúsculas, `;`, `{}`, nombre de la clase) se corrigen con un vistazo… una vez que sabes qué mirar.

🐛 Vocabulario rápido

Término	Idea general
Compilar	Traducir el código fuente (`.java`) a bytecode (`.class`) que entiende la JVM.
Error de compilación	El compilador detecta un fallo de sintaxis y se niega a generar el bytecode.
Error de runtime	El programa compila bien, pero falla o se bloquea durante la ejecución.
Error de lógica	El programa se ejecuta sin errores, pero el resultado no es el que se esperaba.
Sensible a mayúsculas	Java distingue mayúsculas de minúsculas (Case-sensitive): “Public” no es lo mismo que “public”.

08 — El IDE y tu flujo de trabajo

Proyectos, atajos, crear/ejecutar/depurar 🛠️

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 08 · El IDE y tu flujo de trabajo

---

El IDE (Entorno de Desarrollo Integrado) es tu navaja suiza: editor, compilador, ejecutor y depurador, todo en una sola ventana.

En el punto 2 instalaste el JDK y le echaste un primer vistazo al IDE/Editor. Ahora vas a montar tu flujo de trabajo diario: cómo se organiza un proyecto, cómo crear clases, ejecutarlas, depurarte y moverte con atajos en VS Code sin tocar el ratón.

🗂️ Anatomía de un proyecto en VS Code

Cuando creas un proyecto Java en VS Code, la extensión te genera una estructura de carpetas limpia. Las importantes son:

```
MiPrimerCurso/
├── src/          ← TU código (.java) vive aquí
│   └── HolaMundo.java
└── bin/ (o target)  ← el bytecode (.class) compilado automáticamente
    └── HolaMundo.class
```

- **src** (source): Tus archivos `.java`. Es lo que tú escribes y lo que se sube a Git.
- **bin / target**: Lo que genera el compilador en segundo plano. No se toca a mano y normalmente ni se sube a Git.

💡 **Detalle práctico:** No edites nunca los `.class`. Si borras la carpeta `bin`, VS Code la regenerará al compilar en segundo plano. Tu código fuente en `src` es la única fuente de verdad.

🔁 El ciclo de trabajo (tu nuevo bucle de vida)

Programar es un bucle de cuatro pasos que repetirás millones de veces:

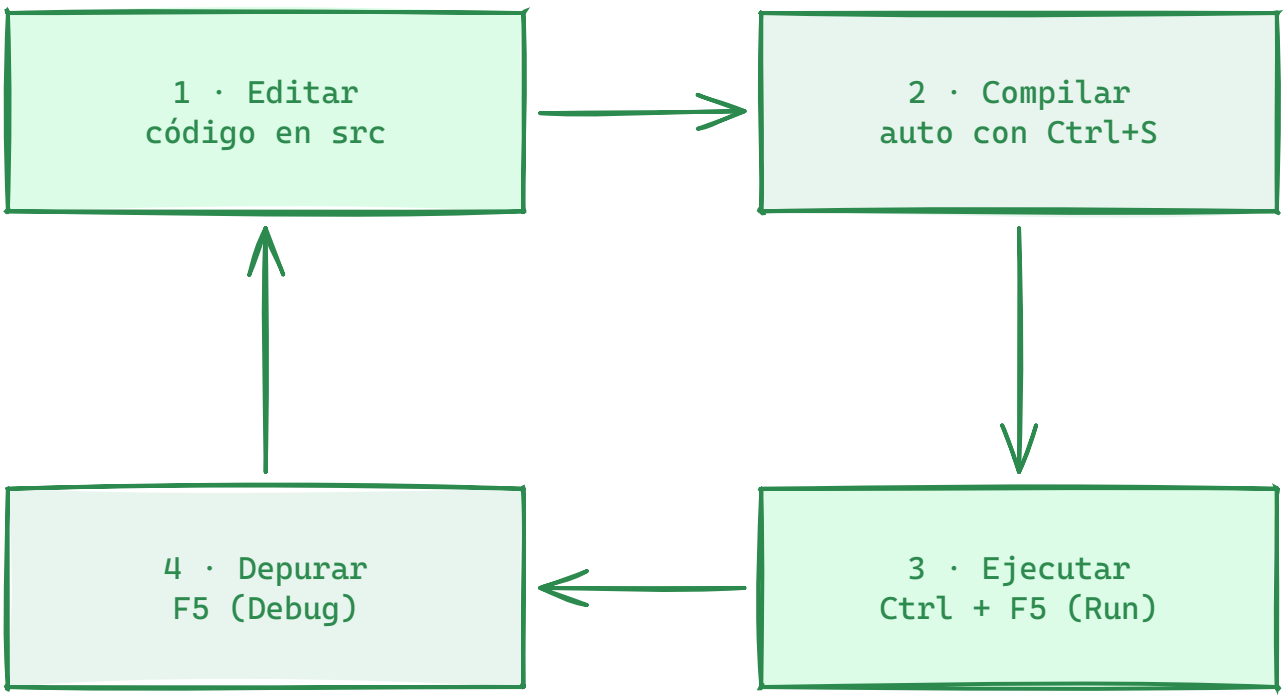
1. **Editar**: Escribes o cambias código en `src`.

2. **Compilar**: VS Code compila automáticamente al guardar (`Ctrl + S`). Aquí se marcan los errores de sintaxis en rojo.
3. **Ejecutar**: Pulsas `Ctrl + F5` (o el botón ▶ Run). Aquí se detectan los errores de runtime.
4. **Depurar**: Si la lógica falla, pulsas `F5` (o Debug) y empieza el detective del punto 4.

Editar → Compilar (Auto) → Ejecutar → Depurar → (Volver a Editar)

⚠️ **Advertencia:** Ejecutar sin depurar (`Ctrl + F5`) y en modo depuración (`F5`) NO son lo mismo. Ejecutar normal ignora los breakpoints; Debug los respeta. Si pones un punto rojo y ejecutas sin depurar, el programa no se detendrá.

El ciclo de trabajo (tu bucle diario)



El bucle se repite millones de veces

⚠️ Run ≠ Debug

Ctrl + F5 (Run): ignora los breakpoints

F5 (Debug): los respeta y para en ellos

⌨️ Los atajos de VS Code que te harán parecer un pro

Deja el ratón en paz. Estos son los atajos esenciales para Java en VS Code:

Atajo (Windows)	Acción
`main` + `Tab` (o `psvm`)	Escribe el esqueleto `public static void main(String[] args) {}`
`sysout` + `Tab` (o `sout`)	Escribe `System.out.println()`
`Ctrl + F5`	Ejecutar (Run Without Debugging)
`F5`	Ejecutar en modo depuración (Start Debugging)
`F10` / `F11`	Step Over / Step Into (depurador)
`Ctrl + /`	Comentar / descomentar la línea seleccionada
`Shift + Alt + ↓`	Duplicar la línea hacia abajo
`F12`	Ir a la definición (Go to Definition)
`F2`	Cambiar el nombre de un símbol/variable en todo el proyecto (Rename)

🧠 **Truco de memoria**: `main` y `sysout` son los dos snippets que más escribirás en el curso. Escribes esas letras, pulsa Tab y VS Code te los rellena de golpe.

🧩 Autocompletado (IntelliSense): tu compañero silencioso

La extensión de Java mientras escribes te sugiere opciones, resalta errores en rojo antes de ejecutar y puede arreglar problemas con un clic (el icono de la bombilla 💡 o `Ctrl + .`):

- Escribes `Syste` y VS Code te ofrece `System` (con la S mayúscula que tanto cuesta al principio).
- Te falta una variable o importación y la bombilla 💡 te ofrece “Import ‘java.util.Scanner’” o la corrección rápida.

- Seleccionas una variable, pulsas `F2` y le cambias el nombre: VS Code actualiza todas sus apariciones en el proyecto (esto es refactorizar).

El autocompletado no es trampa: es la razón por la cual la gente usa VS Code o un IDE en lugar de un bloc de notas. Tu código sale con menos errores tontos porque el editor te corrige mientras piensas.

🏫 Ejemplo guiado: de cero a ejecutar en 60 segundos en VS Code

1. **Nuevo proyecto:** Pulsa `Ctrl + Shift + P`, escribe `Java: Create Java Project`, elige `No build tools`, selecciona la carpeta de destino y ponle un nombre (por ejemplo, `MiPrimerCurso`).

2. **Nueva clase:** Dentro del explorador izquierdo, haz clic derecho sobre la carpeta `src` → New File → ponle de nombre `HolaMundo.java`.

3. Escribe `main` y pulsa Tab (aparece el método main). Dentro, escribe `sysout` y pulsa Tab:

```
public class HolaMundo {
    public static void main(String[] args) {
        System.out.println("¡Hola desde VS Code!");
    }
}
```

4. Pulsa `Ctrl + F5` (Run). Mira la pestaña Terminal en la parte inferior de VS Code.

5. Pon un breakpoint en el `println` (clic a la izquierda del número de línea), pulsa `F5` (Debug) y observa el panel lateral izquierdo de variables.

Si has llegado al paso 5 sin pánico, tu flujo de trabajo está montado. A partir de aquí, el curso entero es rellenar este esqueleto de contenido.

🎯 Mini-comprobación

1. ¿Dónde viven tus archivos `.java` y dónde los `.class`?
2. ¿Cuál es la diferencia entre ejecutar con `Ctrl + F5` (Run) y con `F5` (Debug)?

3. ¿Qué escribe `main + Tab`?

🔁 Respuestas

1. Tus `.java` viven en `src`; los `.class` compilados en `bin` (o `target`). No edites los `.class`.
2. `Ctrl + F5` ejecuta normal (ignora los breakpoints); `F5` ejecuta en modo depuración (los respeta).
3. El esqueleto `public static void main(String[] args) {}` completo.

✅ Resumen en 3 frases

1. Un proyecto tiene `src` (tu código) y `bin` (bytecode): VS Code compila por ti en segundo plano al guardar.
2. Tu bucle diario es editar → compilar → ejecutar → depurar, y Run (`Ctrl + F5`) y Debug (`F5`) no son lo mismo.
3. Los atajos (`main`, `sysout`, `F10/F11`) y el autocompletado (IntelliSense) te hacen trabajar más rápido y con menos errores.

🐛 Vocabulario rápido

Término	Idea general
VS Code / IDE	Editor + herramientas de Java en una sola aplicación
Proyecto	La carpeta raíz que agrupa tu código (`src`) y configuraciones
src	Carpeta de código fuente donde escribes tus archivos `.java`
Refactorizar	Modificar o reestructurar el código manteniendo su comportamiento exactamente igual
Terminal / Consola	La ventana inferior donde se imprime la salida de tus programas

09 — Repaso

Sé el Código, Fireside, Quién Soy, Laboratorio, Crucigrama… 🧩

> 🗺️ **Estás en:** 🚀 **U02 · Introducción a Java y el entorno de trabajo** → 09 · Repaso

En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no funcionará. 😈

⭐ Intenta pensar como el código, *my friend*…

Eres la JVM. Acaban de darte este programa para ejecutar:

```
public class Misterio {
    public static void main(String[] args) {
        System.out.println("Café " + 1 + 2);
        System.out.println("Café " + (1 + 2));
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **Café 3 y Café 3** → La JVM siempre suma los números, pase lo que pase. ❌
2. **Café 12 y Café 3** → En la primera línea, al encontrar texto antes del +, Java concatena de izquierda a derecha: “Café “ + 1 es “Café 1”, y luego + 2 da “Café 12”. En la segunda línea, los paréntesis fuerzan la suma aritmética primero: (1 + 2) = 3, dando “Café 3”.
3. **Café 1 2 y Café 12** → Los paréntesis no existen para la JVM. ❌

🔁 Solución

La opción 2. Cuando un operador + mezcla texto y números, Java concatena (une cadenas). Sin embargo, los paréntesis `(1 + 2)` obligan a realizar primero la suma aritmética. Este es el clásico ejercicio que separa los que han trabajado la unidad de los que han dormido.

🔥 Fireside Chat: JDK vs JRE

Dos veteranos del café discuten al lado de la taza humeante.

- **JRE**: — Mira, yo soy el entorno de ejecución (Java Runtime Environment). Llevo la JVM, las bibliotecas, todo lo necesario para ejecutar Java. Sin mí, tu `.class` corre ni de broma.
- **JDK**: — Ejecutar, ejecutar… ¿Y quién crea el `.class`? Yo. Llevo el compilador `javac`, el generador de Javadoc, la herramienta `jar`… Yo soy el kit completo de desarrollo (Java Development Kit). Tú eres solo la mitad servida.
- **JRE**: — Oye, que sin mí tú eres un catálogo de herramientas sin enchufar. Compilas un programa y… ¿dónde lo ejecutas? ¡En mi casa!
- **JDK**: (Sarcástico) — ¡Ah, claro! Como si yo no te trajera dentro. Cuando me instalan a mí, tú vienes incluido. Eres como el regalo que viene en el pack grande.
- **JRE**: — De acuerdo, pero para el usuario final que solo quiere usar aplicaciones Java ya hechas, ¿a quién llaman? ¡A mí! Soy más ligero y ocupo menos espacio.
- **JDK**: — Y sin compilador. O sea, un cocinero sin cuchillos. Hablaremos cuando quieras crear algo.
- **JRE**: (Suspiro) — Siempre el mismo argumento…

💡 **La lección:** El JDK incluye el JRE. Para programar y crear proyectos necesitas el JDK. Para ejecutar programas en la máquina del usuario final, bastaría con el JRE. Nunca pienses que son opciones excluyentes en un examen.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. Traduce tu código `.java` a bytecode. Soy un profesor de lengua muy puntillós: cualquier coma o llave mal puesta me ofende.
2. Soy la máquina de café: ejecuto el bytecode y funciono igual en cualquier sistema operativo.
3. Soy la puerta de entrada de tu programa. Si cambian mi nombre, Java no encuentra el inicio y todo se queda a oscuras.
4. Soy el detective de VS Code: te dejo parar el programa donde quieras (breakpoints) y espiar las variables paso a paso.

🔄 Respuestas

1. El compilador (`javac`): Traduce `.java` a `.class` y detesta los errores de sintaxis.
2. La JVM (Java Virtual Machine): La máquina virtual que ejecuta el bytecode igual en cualquier plataforma.
3. El método `main`: `public static void main(String[] args)` es el punto de inicio exacto que busca Java.
4. El depurador (debugger): Con breakpoints, Step Over (`F10`), Step Into (`F11`) y el panel de variables.

👹 CONRAD VS EL MUNDO: “El compilador me odia”

CONRAD, nuestro compilador cascarrabías, opina sobre el clásico de los principiantes:

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: “CONRAD, no compila”. Y yo le pregunto: “Vale, ¿qué dice el mensaje de error?”. Y me responde: “Ah, no lo sé, no me lo he leído”. ¡Ay, madre mía! Te doy la línea exacta, el motivo y hasta la flechita ^ señalando el lugar exacto, y no lo lees? ¡Es como quejarse de que la comida está caliente sin mirar el plato!

Y después está el clásico: `Public` con mayúscula. ¿POR QUÉ? La palabra clave es `public`. En minúscula. Hace décadas que compilo y todavía veo `Public`, `string` con minúscula en lugar de `String`… ¡Y las clases que no coinciden con el nombre del archivo! ¡Si lo pone claro en el error!

💡 **La lección:** Antes de llorar sobre el teclado, lee el mensaje de error. Archivo, línea y motivo. El 90% de los errores de los principiantes se arreglan solos consultando esa información. El compilador no te odia: te está pasando las respuestas del examen.

⚡ Laboratorio de Tortura: el programa que no arranca

- Duración estimada: 20–30 minutos
- Herramienta: VS Code y un archivo `.java` nuevo
- El escenario: Copia este programa en un archivo llamado `Tortura.java` y haz que funcione. Tiene 3 errores que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto cuando lo arregles.

```
public class Tortura
    public static void main(string[] args) {
        int a = 3;
        int b = 4;
        System.out.println("La suma es: " + a + b)
        System.out.println("El producto es: " + (a * b));
    }
}
```

🛠️ Tu tarea

Conseguir que compile, que se ejecute y que toda la salida sea correcta.

💡 Pistas (solo si te bloqueas)

1. ¿Falta alguna llave `{` de apertura en la clase?
2. ¿`string` va en mayúscula o minúscula?
3. ¿Falta algún `;` al final de la línea de la suma?
4. Cuando consigas ejecutarlo, ¿por qué la suma sale `34` en lugar de 7? Recuerda cómo funciona el operador `+` con cadenas de texto.

🔄 Solución del Laboratorio

```
public class Tortura {
    public static void main(String[] args) {
        int a = 3;
        int b = 4;
        System.out.println("La suma es: " + (a + b)); // Paréntesis para forzar la suma
        System.out.println("El producto es: " + (a * b));
    }
}
```

Salida correcta por pantalla:

```
La suma es: 7
El producto es: 12
```

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Café de la Mañana**	Hay que explicar la diferencia entre JVM, JRE y JDK con la analogía de la cafetería sin mirar los apuntes.
🏅 **Primer Grito**	Ejecutar tu primer `HolaMundo.java` desde VS Code sin ayuda.
🏅 **Detective de Bugs**	Encontrar los 3+1 errores del Laboratorio de Tortura sin mirar la solución.
🏅 **Atajo Pro**	Hay que escribir un programa en VS Code usando solo los atajos `main` + `Tab` y `sysout` + `Tab`.
🏅 **Traductor**	Documentar una clase con Javadoc (`/** ... */`) y entender las etiquetas `@param` y `@return`.

🧠 Atréverte a pensar

1. Sin ejecutar, ¿qué imprime este programa?

```
public class Misterio2 {
    public static void main(String[] args) {
        System.out.print("Uno ");
        System.out.println("Dos");
        System.out.print("Tres");
    }
}
```

2. El saludo que no saluda: En un programa donde tienes un método `public static void saludo()`, ¿cómo haces para que se ejecute si solo está definido fuera de `main`?

3. El detective de VS Code: Tu programa se ejecuta, pero imprime “La suma es: 34” en lugar de 7. ¿Qué herramienta piensas usar y cómo lo investigas?

4. Verdadero o falso: “javac y java hacen lo mismo: el primero es para Windows y el segundo para Linux.”

💡 Soluciones a las preguntas

1. Imprime:

```
Uno Dos
Tres
```

(`print` no añade salto de línea al final, pero `println` sí. Después de “Tres” no hay salto).

2. Hay que añadir la llamada `saludo();` dentro del método `main`. En Java, solo se ejecuta automáticamente lo que se llama directa o indirectamente desde el método `main`.

3. Usas el depurador (`F5`): Pones un breakpoint en la línea del `println`, avanzas con `F10` (Step Over) y inspeccionas el panel lateral de variables. Verás cómo `"La suma es: " + 3` se convierte en `"La suma es: 3"` y luego `+ 4` da `"La suma es: 34"`. La solución es añadir paréntesis `(a + b)`.

4. Falso. `javac` es el compilador (`.java` → `.class`) y `java` es el ejecutor de la JVM. Ambos se utilizan en cualquier sistema operativo (Windows, Linux, macOS).

💬 Preguntas de entrevista de trabajo

Preguntas reales que te harían para programador Java júnior.

1. **“Explícame, como si fuera tu abuela, la diferencia entre JDK, JRE y JVM.”**
2. **“¿Qué es el método `main` y por qué tiene esa firma exacta?”**
3. **“Un programa compila pero hace lo que no debe. ¿Cuál es tu proceso para arreglarlo?”**
4. **“¿Qué es un array y qué pasa si accedes a un índice que no existe?”**
5. **“¿Cómo le pasas datos a un programa Java sin que te los pida por teclado?”**

🤷 No hay preguntas tontas

❓ ¿Puedo escribir Java en un bloc de notas o necesito sí o sí un IDE?

Puedes, y de hecho es un buen ejercicio: `notepad MiPrograma.java`, `javac MiPrograma.java`, `java MiPrograma`. Pero el IDE te ahorra errores tontos con el autocompletado y te da el depurador. Los profes usan IDE; los masoquistas, bloc de notas.

❓ ¿Por qué el primer `println` de Hola Mundo tenía "args" si no lo uso?

`String[] args` es obligatorio en la firma del `main` aunque no lo uses. Java lo espera sí o sí. Es como la firma en un contrato: está aunque no se ejecute esa cláusula. En el punto 6 viste cómo usarlo.

❓ ¿Java y JavaScript son primos?

No, ni siquiera son del mismo planeta. Java es a JavaScript como un perro es a un perrito caliente. El nombre fue una estrategia de marketing de Netscape para montarse en el boom de Java.

🎬 Poscréditos

El programador aprendiz acaba su primer programa, lo compila y lo ejecuta. Todo perfecto. Entonces se acerca CONRAD, el compilador cascarrabías, con cara de pocos amigos y una taza humeante en la mano.

CONRAD: — Muy bonito el Hola Mundo. ¿Y ahora qué? ¿Vas a imprimir texto toda la vida? Los programas de verdad *guardan* datos, *calculan* y *deciden*. Sin variables, sin tipos, sin operadores… esto es un triler de cartas.

Aprendiz: — ¿Y eso cuándo se aprende?

CONRAD: — *da un trago* La próxima parada. Ya te veo declarando tu primer `int`… y cometiendo tu primer error de tipo. Espérame, que yo estaré ahí para reírme.

El aprendiz guarda su proyecto, cierra el IDE y siente que esto solo acaba de empezar.

U03 — Variables, tipos y operadores

Las cajas del almacén de la memoria y las máquinas del gimnasio de datos 🔤

Bienvenido al almacén. En la U02 aprendiste a decirle a Java que te salude por consola, pero un programa que solo imprime texto es un loro: repite, pero no piensa. Para pensar necesita *guardar* cosas. Edades, precios, notas, si hace frío o no… y para eso existen las **variables**.

Imagina que la memoria de tu ordenador es un **almacén gigante** lleno de estanterías. Cada estantería tiene cajas, y las variables son esas cajas: cada una tiene una etiqueta para que sepas qué hay dentro. Y cuando las cajas se llenan de números y texto, llegan los **operadores**: las máquinas de pesas del gimnasio de datos que suman, restan, comparan y transforman.

En esta unidad vas a aprender a declarar variables de los **8 tipos primitivos**, a guardar texto con `String`, a hacer inmutables algunas cajas con `final`, a manejarlas con operadores aritméticos, relacionales y lógicos, a convertirlas entre tipos con **casting**, y a hacer que tu programa **escuche el teclado** con `Scanner` y juegue a la lotería con `Math.random()`.

Esta unidad se lee como un **libro de 10 capítulos**: los 9 primeros puntos son teoría en progresión y el 10o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Declarar y usar variables de los **8 tipos primitivos** y saber qué cabe en cada caja.
- Aplicar las reglas de **nomenclatura** y distinguir `int`, `long`, `double`, `char` y `boolean` con un vistazo.
- Crear y usar `String` como una **clase** (no un primitivo) y entender su inmutabilidad.
- Declarar **constantes** con `final` y entender por qué el compilador se enfada si las tocas.
- Usar los operadores aritméticos `+ - * / %`, sobrevivir a la **división entera** y respetar la **precedencia**.
- Comparar valores con operadores **relacionales**, combinarlos con **lógicos** (`&&`, `||`, `!`) y resumir decisiones con el **ternario**.
- Convertir entre tipos con **casting** implícito y explícito, sabiendo cuándo se pierde precisión.
- Leer datos del teclado con **`Scanner`** y generar números aleatorios con **`Math.random()`**.

- Manejar los **métodos de `String`** más usados para manipular texto.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Nivel
01 · Variables y tipos primitivos	Declarar variables, nomenclatura y los 8 primitivos	Todos
02 · String, constantes y final	`String`, inmutabilidad, `==` vs `equals` y constantes `final`	Todos
03 · Operadores aritméticos	`+ - * / %`, división entera, precedencia y `++/--`	Todos
04 · Relacionales, lógicos y ternario	Comparaciones, `&&/||/!`, cortocircuito y ternario	Todos
05 · Casting y conversiones	Conversión implícita y explícita, truncamiento y desbordamiento	Todos
06 · Scanner: leer por teclado	`Scanner`, `nextInt`, `nextDouble` y `nextLine`	Todos
07 · Consola: salida formateada y errores de entrada	`printf`, `String.format`, `NumberFormat` y errores del `Scanner`	Todos
08 · Math.random() y números aleatorios	`Math.random()`, la fórmula min-max y las herramientas de `Math`	Todos
09 · Métodos útiles de String	`length`, `substring`, `replace`, `trim`, `charAt`…	Todos
10 · Repaso interactivo	Sé el Código, Fireside, Quién Soy, Laboratorio, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 9 primeros puntos son teoría en progresión. El 10o es el aterrizaje práctico: léelo justo después del 9o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA2 + RA1)

RA1: Reconoce la estructura de un programa informático, identificando y relacionando los elementos propios del lenguaje de programación utilizado.

RA2: Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.

CE	Criterio	Dónde se cubre
RA1 d)	Se han identificado los distintos tipos de variables y la utilidad específica de cada uno.	✅ Punto 1
RA1 e)	Se ha modificado el código de un programa para crear y utilizar variables.	✅ Punto 1
RA1 f)	Se han creado y utilizado constantes y literales.	✅ Punto 2
RA1 g)	Se han clasificado, reconocido y utilizado en expresiones los operadores del lenguaje.	✅ Puntos 3 y 4
RA1 h)	Se ha comprobado el funcionamiento de las conversiones de tipo explícitas e implícitas.	✅ Punto 5
RA2 b)	Se han escrito programas simples.	✅ Todos
RA2 c)	Se han instanciado objetos a partir de clases predefinidas.	✅ Puntos 2 y 6
RA2 d)	Se han utilizado métodos y propiedades de los objetos.	✅ Puntos 6 y 8

CE	Criterio	Dónde se cubre
RA2 e)	Se han escrito llamadas a métodos estáticos.	✅ Punto 7
RA2 g)	Se han incorporado y utilizado librerías de objetos.	✅ Punto 6
RA2 h)	Se han utilizado constructores.	✅ Puntos 2 y 6

> 📌 El `Scanner` del punto 6, la salida formateada del punto 7 y los métodos de `String` del punto 9 plantan la semilla del RA5 (entrada y salida de información), que florece en la U14.

🚪 ¿Por dónde empiezo?

- ¿Cero conocimientos? → Arranca en el punto 1. Solo necesitas lo que viste en la U02.
- ¿Ya sabes declarar variables y quieres chicha? → Ve directo al punto 6 y al punto 8: ahí empiezan los programas interesantes.
- ¿Solo vienes a por operadores? → Salta al punto 3 y al punto 4.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Variables y tipos primitivos

⏭️ **Al acabar la unidad, continúa en U04 · Estructuras de control y excepciones básicas.**

01 — Variables y tipos primitivos

Las cajas del almacén de la memoria y sus 8 tamaños distintos 📦

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 01 · Variables y tipos primitivos

📬 La idea en una frase

> **Las variables son cajas etiquetadas en la memoria del ordenador, y Java te ofrece 8 tamaños de caja (los tipos primitivos) para que elijas el que mejor le va a cada dato.**

En la U02 tu programa solo gritaba texto por consola. Ahora vas a darle memoria: va a guardar tu edad, tu nombre, tu nota media y hasta si tienes hambre. Y para elegir bien la caja de cada dato, primero tienes que conocer el catálogo del almacén.

📦 La declaración: la receta de una caja

Para crear una caja (una variable) le dices a Java tres cosas: el **tipo** (tamaño y forma de la caja), el **nombre** (la etiqueta) y el **valor** (lo que metes dentro). La receta es:

```
tipo nombreDeLaCaja = valorQueMetoDentro;
```

Ejemplos reales:

```
int edad = 25;              // Caja etiquetada "edad" con un 25 dentro
double precio = 19.99;      // Caja con decimales
String nombre = "María";    // Caja mágica que guarda texto
boolean hambre = true;      // Caja de verdadero/falso (ahora mismo: true)
```

> 💡 **Detalle práctico:** las variables se llaman así porque… ¡varían! Puedes cambiar su contenido cuando quieras. `int edad = 25;` y, al día de tu cumpleaños, `edad = 26;`. La etiqueta es la misma, el contenido cambia.

🏷️ Las reglas de nomenclatura (o cómo no meter la pata)

Java es tiquismiquis con los nombres de las cajas. Estas son las reglas de oro:

- Pueden llevar letras, números, _ y $. **Nada de espacios**. (Java admite muchos caracteres Unicode, pero por convención y para evitar líos se usan letras ASCII: evita ñ, ç o acentos en los nombres.)
- **No pueden empezar con número.** `1numero` es ilegal; `numero1` es legal. Como las matrículas de los coches.
- **Las mayúsculas importan**: `edad`, `Edad` y `EDAD` son tres cajas distintas. Como etiquetar "Zapatos", "zapatos" y "ZAPATOS".
- **No uses palabras reservadas**: `int`, `class`, `if`, `while`… son de Java, no tuyas.
- Usa **camelCase** para las variables: `miVariableEjemplo`. Como un camello, con joroba en medio.

```
// ✅ Correcto
int numeroAlumnos = 30;
double notaMedia = 7.5;

// ❌ Incorrecto
int 1numero = 30;         // empieza por número
double nota media = 7.5; // espacio en el nombre
int class = 30;           // palabra reservada
```

📐 Los 8 primitivos: el catálogo de cajas

Java tiene **8 tipos primitivos**. Piensa en ellos como cajas de distintos tamaños en tu almacén:

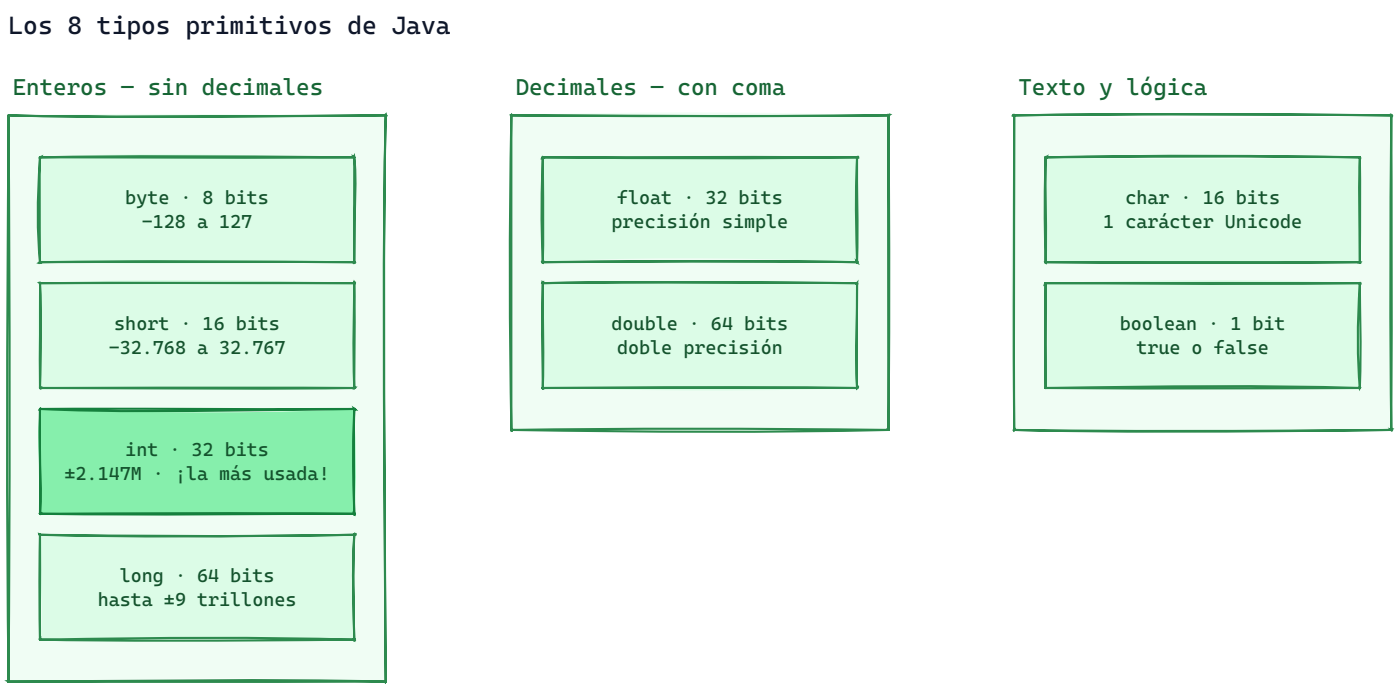
Tipo	Tamaño	Lo que cabe	Analogía
`byte`	8 bits	-128 a 127	Caja de cerillas
`short`	16 bits	-32.768 a 32.767	Caja de zapatos
`int`	32 bits	-2.147M a 2.147M	Caja de mudanza (la que más usarás)
`long`	64 bits	-9 trillones a +9 trillones	Contenedor de barco

Tipo	Tamaño	Lo que cabe	Analogía
`float`	32 bits	Decimales de precisión simple	Vaso de agua
`double`	64 bits	Decimales de precisión doble	Cubo de agua
`char`	16 bits	Un solo carácter Unicode	Una letra en una caja de zapatos
`boolean`	1 bit	`true` o `false`	Interruptor de luz

Y así se declaran cada uno:

```
byte nivel = 100;
short poblacion = 30000;
int habitantes = 1500000;           // El más usado
long distancia = 384400000L;        // La L al final es obligatoria
float precio = 12.99f;              // La f al final es obligatoria
double pi = 3.14159265359;
char letra = 'A';                   // Comillas SIMPLES para char
boolean esJavaDivertido = true;     // Esto es opinable
```

> 📝 **Nota:** usa `int` para casi todo lo numérico entero. Solo pasa a `long` si vas a contar estrellas. Usa `double` para decimales, a menos que ahorrar memoria sea tu fetiche.



🎒 ¿Qué caja uso para cada dato?

Elegir el tipo correcto es como elegir la maleta del viaje: ni un microbus para dos personas, ni un Smart para una familia de cinco. La práctica hace al maestro:

- **`int`**: edades, contadores, puntuaciones, casi todo entero.
- **`double`**: precios, notas medias, temperaturas, cualquier decimal.
- **`boolean`**: respuestas de sí/no: "¿ha aprobado?", "¿hay conexión?".
- **`char`**: una sola letra: la inicial de un nombre, una calificación 'A', 'B', 'C'.
- **`long`**: números astronómicos, milisegundos, identificadores gigantes.

> ⚠️ **Advertencia:** cuando dudes entre `int` y `double`, piensa: ¿este dato puede llevar decimales? Si sí → `double`. Si no → `int`. Simple.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** antes de escribir código, hazte siempre la misma pregunta: *¿qué tipo de dato es esto y en qué caja cabe?* El 90% de los errores de esta unidad vienen de meter mal la caja.

Ejercicio: El guardia del almacén

Eres el guardia de un almacén de datos. Te dan estas declaraciones y te preguntan: **¿cuáles compilan y cuáles no?** Marca las que fallarían y por qué:

```
int a = 150;                // ¿compila?
int b = 10.5;               // ¿compila?
double c = 7;               // ¿compila?
char d = "A";               // ¿compila?
boolean e = "true";         // ¿compila?
long f = 3000000000L;       // ¿compila?
int g = 3000000000;         // ¿compila?
```

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué tamaño de caja usarías para guardar el número de habitantes de la Tierra (más de 8.000 millones)?
2. ¿Por qué `char letra = "A";` no compila y `char letra = 'A';` sí?
3. ¿Cuál es la diferencia entre `float` y `double` en una frase?
4. ¿Por qué `Edad`, `edad` y `EDAD` son tres variables distintas?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Una variable es una **caja etiquetada** en memoria: tipo (tamaño), nombre (etiqueta) y valor (contenido).
2. Java tiene **8 tipos primitivos**, y elegir el correcto (normalmente `int` o `double`) es medio examen.
3. Las reglas de nomenclatura (camelCase, sin espacios, sin palabras reservadas, mayúsculas que importan) te ahorran errores de compilación tontos.

> 🦎 **Vocabulario rápido**

Término	Idea general
Variable	Caja etiquetada donde guardas un dato que puede cambiar
Tipo primitivo	Uno de los 8 tipos básicos de datos de Java
Declarar	Crear la variable: `tipo nombre = valor;`
Literal	El valor escrito tal cual: `25`, `"María"`, `true`
camelCase	Convención de nombres: `miVariableEjemplo`
Palabra reservada	Palabra de Java que no puedes usar como nombre

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · String, constantes y final

02 — String, constantes y final

La caja mágica del texto y las cajas con superglue 🔤

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 02 · String, constantes y final

📬 La idea en una frase

> **`String` es una clase (no un primitivo) que guarda texto, es inmutable como una foto, y `final` es el superglue que convierte cualquier caja en una constante que no se puede tocar.**

En el punto 1 viste las 8 cajas primitivas. Pero los programas también guardan texto: nombres, mensajes, contraseñas… Para eso existe `String`. Y cuando quieras que un valor no cambie jamás, lo declaras `final`. Vamos a las dos.

🔤 String: la caja mágica que no es una caja

`String` **no es primitivo**: es una **clase**. Pero se comporta tan natural que parece primitivo. Es como un amigo que encaja tan bien en tu grupo que jurarías que es de la familia.

```
String saludo = "Hola, DAM";           // La forma normal
String nombre = new String("Ana");        // También se puede crear así (usa un constructor)
```

Fíjate en la segunda línea: `new String(...)` es un **constructor**. Aún no estudias POO a fondo (eso llega en la U09), pero ya puedes instanciar objetos de clases predefinidas como `String`. La primera línea es un atajo que Java te da para no escribir `new String(...)` cada vez.

> 💡 **Detalle práctico:** `String` va con **comillas dobles** `"..."`. Las comillas simples `'...'` son solo para `char`, un único carácter.

🧊 La inmutabilidad: no toques, que se rompe

Los `String` son **inmutables**: una vez creados, no se pueden cambiar. Cuando haces esto:

```
String texto = "Hola";
texto = texto + " mundo";    // Java NO modifica "Hola"
```

…en realidad Java tira el `"Hola"` a la basura y crea un `"Hola mundo"` nuevo. El texto original sigue ahí, congelado, para siempre. Es como si cada vez que quisieras poner un cartel nuevo tuvieras que quemar el anterior.

> ⚠️ **Advertencia:** esta inmutabilidad es la razón por la que comparar `String` con `==` es peligroso. Comparar con `==` pregunta *"¿son el mismo objeto?"*, no *"¿tienen el mismo texto?"*. Para comparar contenido usa `.equals()`.

🧲 `==` vs `.equals()`: la trampa clásica

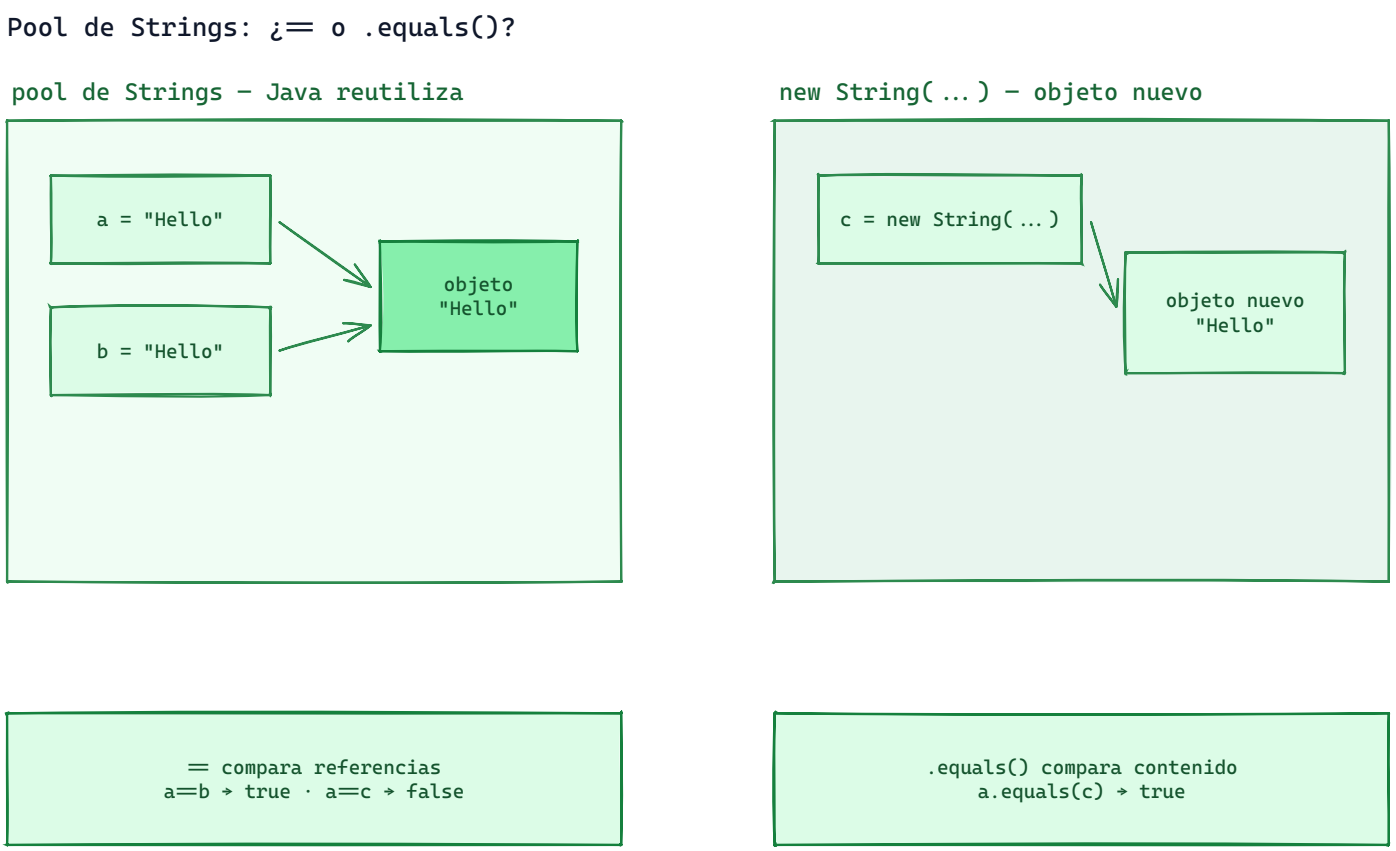
```
String a = "Hello";
String b = "Hello";
String c = new String("Hello");

System.out.println(a == b);       // true
System.out.println(a == c);       // false
System.out.println(a.equals(c));  // true
```

¿Por qué `a == b` da `true` y `a == c` da `false`, si los tres textos son "Hello"?

- `a` y `b` apuntan al **mismo objeto** en el "pool de Strings": Java reutiliza literales iguales.
- `c` se creó con `new String(...)`, así que es un objeto **nuevo y distinto**.
- `==` compara **referencias** (¿son la misma caja?); `.equals()` compara **contenido** (¿tienen lo mismo dentro?).

> ⚠️ **Advertencia:** regla de oro: **los `String` siempre se comparan con `.equals()`**. Si usas `==`, tarde o temprano te morderá en un examen.



🔒 Constantes con `final`: cajas con superglue

Las constantes se declaran con `final`. Una vez que metes algo ahí dentro, no sale ni con palanca:

```
final double IVA = 0.21;
final int MAXIMO_INTENTOS = 3;
final String NOMBRE_APP = "Gestión DAM";

IVA = 0.10;  // ERROR de compilación: ¡no puedes reasignar una constante!
```

Por convención, las constantes se escriben **EN_MAYÚSCULAS_CON_GUIONES_BAJOS**, como si estuvieran gritando "¡SOY INMUTABLE!". Esto le dice a cualquier programador (incluido tu yo del futuro) que ese valor no se debe tocar.

> 💡 **Detalle práctico:** ¿por qué usar constantes y no escribir el número directamente? Porque si el IVA cambia de 0.21 a 0.20 mañana, solo editas **una línea**, no las 50 donde usaste el 0.21. Eso es lo que se llama *mantener el código*.

🏫 Ejemplo guiado: la factura que no se puede tocar

Vamos a montar un mini-programa que calcula el precio final de un producto con IVA. La magia: el IVA es `final` y el nombre de la app también.

```
public class Factura {
    public static void main(String[] args) {
        final double IVA = 0.21;
        final String NOMBRE_APP = "Factura Express";

        double precioBase = 50.0;
        double ivaAplicado = precioBase * IVA;
        double precioFinal = precioBase + ivaAplicado;

        System.out.println(NOMBRE_APP + " — precio base: " + precioBase + "€");
        System.out.println("IVA (" + (IVA * 100) + "%): " + ivaAplicado + "€");
        System.out.println("Total: " + precioFinal + "€");
    }
}
```

Salida:

```
Factura Express — precio base: 50.0€
IVA (21.0%): 10.5€
Total: 60.5€
```

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando veas `==` con dos `String`, sospecha. Pregúntate primero: *¿están comparando referencias o contenido?*

Ejercicio: ¿Qué imprime este lío de Strings?

Sin ejecutar, di qué imprime exactamente este código:

```java
String x = "Java";
String y = "Java";
String z = new String("Java");
System.out.println(x == y);
System.out.println(x == z);
System.out.println(x.equals(z));
```

> 🔄 Solución

🎯 Mini-chequeo

1. ¿`String` es un tipo primitivo? ¿Por qué?
2. ¿Qué significa que un `String` sea inmutable?
3. ¿Qué pasa si intentas reasignar una variable declarada como `final`?
4. ¿Por qué se comparan los `String` con `.equals()` y no con `==`?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. `String` es una **clase** que guarda texto entre comillas dobles y es **inmutable**: cada cambio crea un objeto nuevo.
2. Los `String` se comparan con **`.equals()`**, nunca con `==` (que solo compara referencias y te da sorpresas).
3. `final` convierte una variable en **constante** (por convención, en MAYÚSCULAS), y el compilador se enfada si intentas cambiarla.

> 🦎 Vocabulario rápido

Término	Idea general
`String`	Clase de Java que guarda cadenas de texto
Inmutable	Que no se puede modificar una vez creado
`final`	Modificador que convierte una variable en constante

Término	Idea general
Constructor	El mecanismo que crea un objeto (`new String(...)`)
Pool de Strings	Zona donde Java reutiliza literales iguales
`.equals()`	Método que compara el contenido de dos objetos

📚 Volver al índice de la unidad · **Anterior:** 01 · Variables y tipos primitivos · **Siguiente:** 03 · Operadores aritméticos

03 — Operadores aritméticos

Las máquinas de pesas del gimnasio de datos ➕

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 03 · Operadores aritméticos

📬 La idea en una frase

> **Los operadores aritméticos (+, -, *, /, %) son las máquinas de pesas del gimnasio de datos: transforman tus variables, y la división entera y la precedencia son las trampas que separan a los que saben de los que improvisan.**

Tener variables está muy bien, pero no sirven de nada si no haces cosas con ellas. Bienvenido al gimnasio: vas a sudar con los cinco ejercicios básicos y a descubrir por qué `10 / 3` no es lo que tú crees.

💪 El día en el gimnasio: los 5 ejercicios básicos

Operador	Ejercicio	Ejemplo
+	Press de banca	`5 + 3 = 8`
-	Curl de bíceps	`5 - 3 = 2`
*	Sentadilla	`5 * 3 = 15`
/	Peso muerto	`10 / 3 = 3` (enteros) o `10.0 / 3 = 3.333...`
%	El odiado abdominal	`10 % 3 = 1` (el resto de 10/3)

```
int a = 10;
int b = 3;
double c = 10.0;

System.out.println(a / b);           // 3  (división entera)
System.out.println(a % b);           // 1  (el resto)
System.out.println(c / b);           // 3.333... (división real)
System.out.println((double) a / b);  // 3.333... (obligas decimal)
```

> 💡 **Detalle práctico:** el **módulo** (`%`) no es para aburridos: te dice si un número es par (`numero % 2 == 0`), reparte turnos, da vueltas a los relojes y alimenta montones de juegos. Sin `%` no existiría nada cíclico.

⚠️ La división entera mata

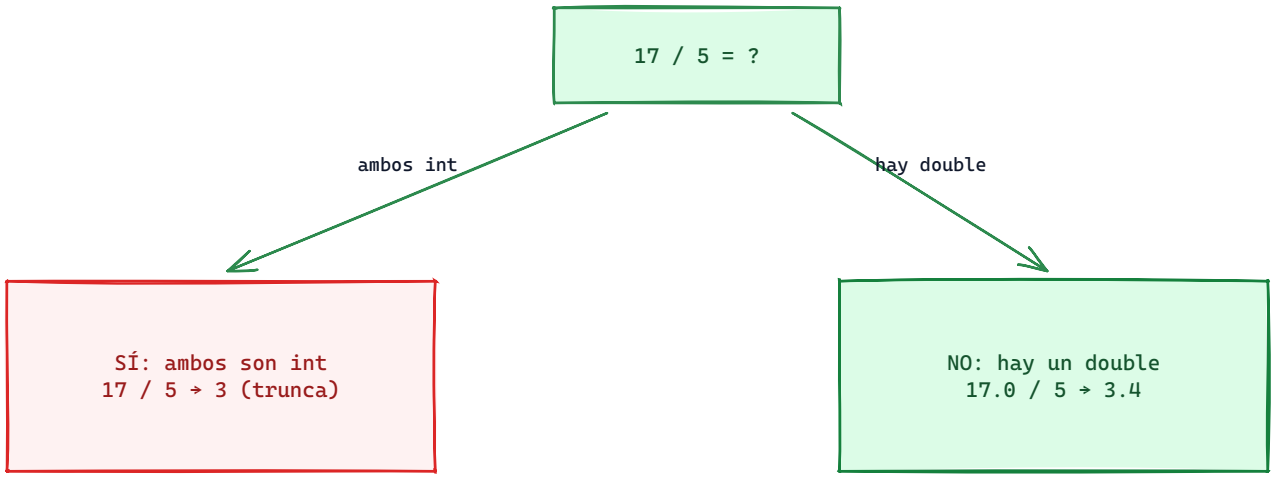
Si divides dos enteros, Java te devuelve un entero. Punto. Los decimales se truncan sin piedad:

```
int alumnos = 17;
int grupos = 5;
System.out.println(alumnos / grupos);   // 3 — Java dice que cada grupo tiene 3 alumnos
```

Para Java, 17 dividido entre 5 son **3**. Ni 3.4 ni 3.5: 3. Si quieres decimales, al menos uno de los dos operandos debe ser `double` (o hacer un casting, que verás en el punto 5).

> ⚠️ **Advertencia:** este es uno de los errores más rentables para un examen. `5 / 2` es `2`. `5 / 2.0` es `2.5`. `(double) 5 / 2` es `2.5`. Memorízalo como un mantra.

División entera: int / int trunca



Mantra de examen: 5/2=2 · 5/2.0=2.5 · (double)5/2=2.5

🎭 Precedencia: la ley del comedor

¿Quién se sirve primero en el comedor de las expresiones? Hay un orden estricto:

```
int resultado = 2 + 3 * 4;        // 14 — la multiplicación se cuela antes
int conParentesis = (2 + 3) * 4;  // 20 — los paréntesis tienen pase VIP
```

La ley del comedor:

1. **Paréntesis `()`** — pase VIP, van los primeros.
2. **Multiplicación, división y módulo `* / %`** — los populares.
3. **Suma y resta `+ -`** — los normales, los últimos.

> 📝 **Nota:** y cuando dudes, **pon paréntesis**. `(a + b) * (c - d)` es mucho más legible que confiar en tu memoria de la precedencia. Los paréntesis no duelen y el que lea tu código (tu yo del futuro) te lo agradecerá.

🌀 Asignación compuesta: el atajo perezoso

Escribir `x = x + 5` es tan verboso… Por suerte, Java tiene los **operadores de asignación compuesta**, un grifo de agua en el sofá para no ir a la cocina:

```
int x = 10;
x += 5;   // x = 15  (x = x + 5, pero más cool)
x -= 3;   // x = 12
x *= 2;   // x = 24
x /= 4;   // x = 6
x %= 3;   // x = 0
```

💪 ++ y --: flexiones para variables

Sumar o restar 1 es tan común que Java tiene su propio operador: `++` (incremento) y `--` (decremento). Pero ojo, que tienen dos caras:

```
int a = 5;
int b = a++;  // POST: primero usa a (5), luego incrementa → b = 5, a = 6
int c = ++a;  // PRE: primero incrementa, luego usa → a = 7, c = 7
```

- **Post-incremento (`a++`)**: “usa y luego sube”.
- **Pre-incremento (`++a`)**: “sube y luego usa”.

> ⚠️ **Advertencia:** regla de oro: si usas ++ o -- *dentro* de una expresión complicada, estarás escribiendo código que ni tú entenderás en una semana. Úsalos solos, en su propia línea. En los exámenes aparecen las trampas, y aquí está la prueba.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** desglosa la expresión paso a paso. ¿Qué valor tiene cada variable en cada momento? Anótalo, no lo hagas de memoria.

Ejercicio: el acróbata de las variables

Sin ejecutar, calcula qué vale todo aquí:

```
int x = 3;
int y = x++ + ++x;
System.out.println("x = " + x + ", y = " + y);
```

> › 🔄 **Solución**

🎯 Mini-chequeo

1. ¿Cuánto vale `int r = 10 / 3`? ¿Y `double r = 10 / 3`?
2. ¿Qué hace el operador `%` y para qué sirve saber si `n % 2 == 0`?
3. ¿Cuál es el resultado de `2 + 3 * 4 - 1`?
4. Diferencia entre `a++` y `++a` en una frase.

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los operadores `+ - * / %` transforman tus datos, y la **división entera** trunca los decimales si ambos operandos son enteros.
2. La **precedencia** sigue la ley del comedor (paréntesis → `* / %` → `+ -`), y los paréntesis siempre son el plan B seguro.

3. `+=`, `-=`, `++` y `--` son atajos peligrosamente cómodos: úsalos solos y no dentro de expresiones enrevesadas.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Operador aritmético | `+ - * / %`: las operaciones matemáticas de Java |
> | División entera | División de dos enteros que trunca los decimales |
> | Módulo | `%`, el resto de una división |
> | Precedencia | El orden en que Java evalúa los operadores |
> | Asignación compuesta | Atajo como `x += 5` (= `x = x + 5`) |
> | Incremento | `++`/`--`: sumar o restar 1, con cara pre y post |

📚 Volver al índice de la unidad · **Anterior:** 02 · String, constantes y final · **Siguiente:** 04 · Relacionales, lógicos y ternario

04 — Relacionales, lógicos y ternario

El juez, el portero y el bouncer del club de las decisiones ⚖️

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 04 · Relacionales, lógicos y ternario

📬 La idea en una frase

> **Los operadores relacionales (`==`, `!=`, `<`, `>`, `<=`, `>=`) comparan valores y devuelven un `boolean`; los lógicos (`&&`, `||`, `!`) combinan booleanos; y el ternario (`? :`) resume una decisión en una sola línea.**

Tus variables ya saben sumar y restar. Ahora van a aprender a **juzgar**: "¿eres mayor de edad?", "¿tienes entrada Y dinero?", "¿eres el dueño O un invitado?". Bienvenido al club de las decisiones.

⚖️ Relacionales: el juez de la discusión

Los operadores relacionales siempre devuelven un `boolean`: `true` o `false`. Son el juez que dicta sentencia sobre la relación entre dos valores:

```
int edad = 18;
boolean puedeVotar = edad >= 18;                  // true
boolean tieneDescuento = edad < 12 || edad > 65;  // false
boolean noEsEl = edad != 18;                      // false
```

Operador	Significado	Ejemplo (`edad = 18`)
`==`	Igual que	`edad == 18` → `true`
`!=`	Distinto de	`edad != 18` → `false`
`>`	Mayor que	`edad > 21` → `false`
`<`	Menor que	`edad < 21` → `true`
`>=`	Mayor o igual	`edad >= 18` → `true`

Operador	Significado	Ejemplo (`edad = 18`)
`<=`	Menor o igual	`edad <= 18` → `true`

> ⚠️ **Advertencia: `=` no es `==`.** `=` asigna ("guarda este valor en esta caja"), `==` compara ("¿son iguales?"). Confundirlos es el error más clásico de la historia de la programación. Es como confundir "pon la mesa" con "¿está puesta la mesa?".

🚪 Lógicos: el portero del club

Los operadores lógicos combinan booleanos para tomar decisiones compuestas. Son el portero del club nocturno:

- **`&&` (AND)**: ¿Tienes más de 18 **Y** tienes entrada? Las dos condiciones deben cumplirse.
- **`||` (OR)**: ¿Tienes más de 18 **O** eres el dueño? Basta con que se cumpla una.
- **`!` (NOT)**: ¿**NO** tienes menos de 18? Niega la condición.

```
int edad = 18;
boolean mayorEdad = true;
boolean tieneEntrada = false;

boolean entra = mayorEdad && tieneEntrada;    // false (falta la entrada)
boolean entraVip = mayorEdad || tieneEntrada; // true (basta ser mayor de edad)
boolean noEsMenor = !(edad < 18);             // true (niega que sea menor)
```

⚡ Cortocircuito: el portero que no mira

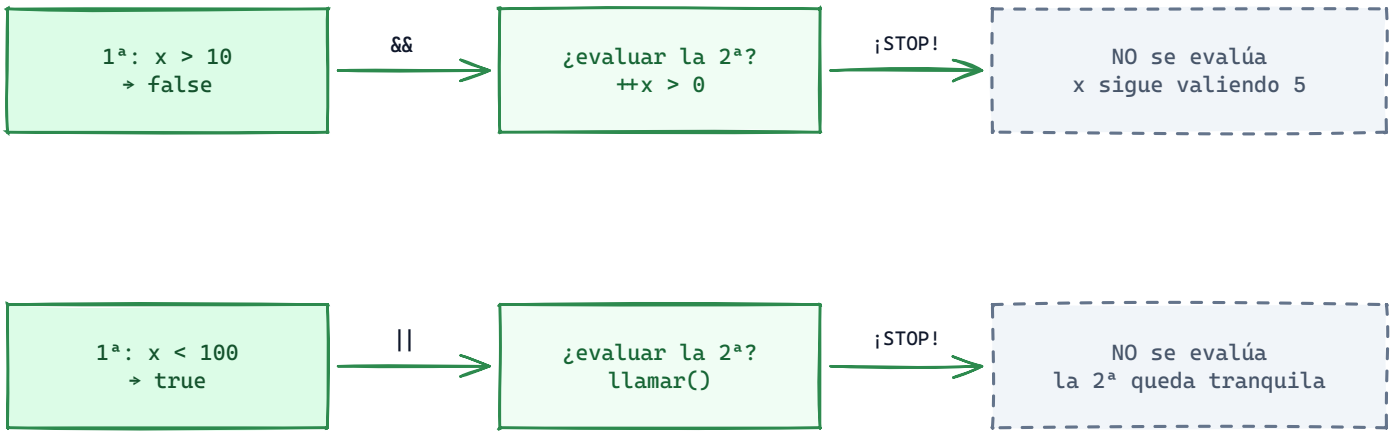
Aquí viene el truco del club más rentable de Java: el **cortocircuito**.

- Con `&&`, si lo primero es `false`, la expresión ya es `false` y **Java ni se molesta en evaluar lo segundo**.
- Con `||`, si lo primero es `true`, la expresión ya es `true` y tampoco mira lo segundo.

```
int x = 5;
boolean resultado = (x > 10) && (++x > 0);   // false, y x sigue siendo 5
System.out.println(x);                       // 5 — el ++x nunca se ejecutó
```

> 💡 **Detalle práctico:** el cortocircuito también te protege. Si escribes `(algo != null) && algo.metodo()`, Java no llamará al método si `algo` es `null`, evitando un crash en tu programa.

Cortocircuito: la 2ª condición no se mira



Regla: && con false corta · || con true corta — la 2ª condición ni se mira

---

El ternario: el bouncer del club

Cuando la decisión es "si pasa esto, mensaje A; si no, mensaje B", el **operador ternario** lo resume en una línea. Es un `if-else` de bolsillo (los `if` de verdad llegan en la UO4):

```
int edad = 17;
String mensaje = (edad >= 18) ? "Pasa, joven" : "Vuelve cuando crezcas";
```

La estructura es: `condición ? valorSiTrue : valorSiFalse`.

```
int nota = 7;
String resultado = nota >= 5 ? "Aprobado" : "Suspenso";
```

Ejemplo guiado: el portero del club

Vamos a montar el control de acceso de un club con todo lo aprendido:

```java
public class ClubNoche {
    public static void main(String[] args) {
        int edad = 19;
        boolean tieneEntrada = true;
        boolean esVip = false;

        boolean puedeEntrar = (edad >= 18) && (tieneEntrada || esVip);
        String mensaje = puedeEntrar ? "¡Pasa, disfruta!" : "Fuera de aquí, pequeñín";

        System.out.println(mensaje);

        int golesLocal = 2;
        int golesVisitante = 2;
        String resultado = golesLocal > golesVisitante ? "Gana el local" :
                                golesLocal < golesVisitante ? "Gana el visitante" :
"Empate";
        System.out.println("Resultado: " + resultado);
    }
}
```

Salida:

```
¡Pasa, disfruta!
Resultado: Empate
```

Fíjate en la segunda decisión: los ternarios se pueden **encadenar** (un ternario dentro de otro) para cubrir tres casos. Legible y compacto.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** en una expresión con && y ||, pregunta siempre: *¿y si lo primero ya decide?* Ese es el cortocircuito.

Ejercicio: el detective de booleanos

Sin ejecutar, calcula el valor de cada variable:

```java
int n = 10;
boolean a = (n > 5) && (n < 20);              // ¿?
boolean b = (n > 15) || (++n == 11);          // ¿?
boolean c = !(n == 10) && (n % 2 == 0);       // ¿?
System.out.println(a);
System.out.println(b);
System.out.println(c);
System.out.println(n);
```

> 🔄 Solución

🎯 Mini-chequeo

1. ¿Qué devuelve siempre un operador relacional?
2. ¿Cuál es la diferencia entre `&&` y `||` en una frase?
3. ¿Qué es el cortocircuito y cuándo se activa?
4. Escribe el ternario que asigne `"mayor"` o `"menor"` a una variable según una edad.

> 🔄 Respuestas

✅ Resumen en 3 frases

1. Los **relacionales** (`==`, `!=`, `<`, `>`, `<=`, `>=`) comparan y devuelven un `boolean`, y `=` nunca se usa para comparar.
2. Los **lógicos** (`&&`, `||`, `!`) combinan condiciones y sufren **cortocircuito**: si la primera ya decide, no miran las demás.
3. El **ternario** (`condición ? A : B`) resume una decisión de dos caminos en una línea.

> 🦎 Vocabulario rápido

Término	Idea general
Operador relacional	Compara dos valores y da `true`/`false`
Operador lógico	Combina booleanos: `&&`, `||`, `!`
Cortocircuito	Dejar de evaluar cuando la primera condición ya decide

Término	Idea general
Ternario	`condición ? valor1 : valor2`, un if-else en una línea
Booleano	Tipo con solo dos valores: `true` o `false`

📚 Volver al índice de la unidad · **Anterior:** 03 · Operadores aritméticos · **Siguiente:** 05 · Casting y conversiones

05 — Casting y conversiones

Aprieta que cabe, con pérdidas y lágrimas si no eliges bien 🔁

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 05 · Casting y conversiones

📬 La idea en una frase

> **El casting convierte un valor de un tipo a otro: la conversión implícita (widening) la hace Java solo y sin pérdidas, mientras que la explícita (narrowing) la fuerzas tú con `(tipo)` y puedes perder datos por el camino.**

En el almacén de la memoria tienes cajas de todos los tamaños. A veces necesitas meter el contenido de una caja grande en una pequeña… y eso, o lo haces con cuidado, o pierdes cosas por el camino. Bienvenido al arte de apretar.

🪜 Conversión implícita (widening): mudarse a una caja más grande

Cuando el destino es una caja **más grande**, Java lo hace solo, sin preguntar. Es como cambiar de un piso a una mansión: te mudas y no pierdes nada. Esto se llama *widening* (ensanchar):

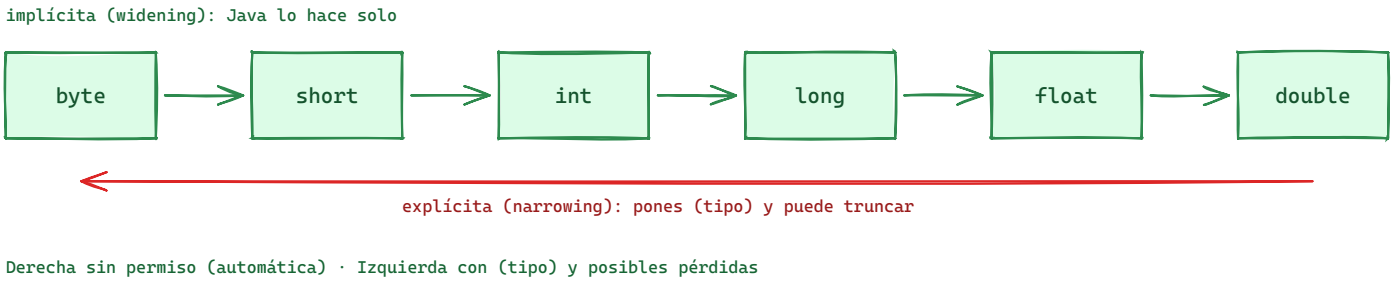
```
int num = 100;
long numLong = num;         // Cabe de sobra, sin pérdidas
double numDouble = num;     // 100 → 100.0, también sin problemas
```

La cadena natural de ensanchamiento entre tipos numéricos es:

```
byte → short → int → long → float → double
```

Cualquier tipo puede pasar al que está a su derecha sin que se pierda ni un bit. Java sonríe y te deja.

Escalera de casting: ancha fácil, estrecha a fuego



Derecha sin permiso (automática) · Izquierda con (tipo) y posibles pérdidas

📉 Conversión explícita (narrowing): la maleta XXL en un Smart

Cuando el destino es una caja **más pequeña**, Java se niega. Tienes que empujar a la fuerza con un casting explícito: pones el `(tipo)` delante y rezas:

```
double precio = 19.99;
int entero = (int) precio;    // Aprietas 19.99 en un int
System.out.println(entero);   // 19 — los céntimos desaparecen en el olvido
```

Esto se llama *narrowing* (estrechar). Java te obliga a escribir el `(tipo)` porque es peligroso: estás diciendo "confía en mí, sé lo que hago". Y si te equivocas, los datos se pierden en silencio.

> 💡 **Detalle práctico:** el casting sobre un `double` **trunca**, no redondea. `(int) 19.99` da `19` y `(int) 19.5` también da `19`. Si quieres redondear, usa `Math.round()` (lo verás en el punto 7).

🪓 El truncamiento: el céntimo olvidado

El ejemplo del precio es la advertencia clásica:

```
double precio = 9.99;
int precioEntero = (int) precio;
System.out.println(precioEntero);   // 9 — te acaban de quitar 0.99€
```

No es redondeo: es **cortar con hacha**. Java tira la parte decimal sin mirar si era 0.1 o 0.9.

> ⚠️ **Advertencia:** antes de estrechar una caja, pregúntate siempre: *¿cabe el valor?* Si el número es mayor que el máximo de la caja destino, no solo perderás decimales: el valor se **desbordará** a algo completamente absurdo.

💥 El desbordamiento: cuando aprietas de más

¿Qué pasa si intentas meter un elefante (300) en una caja de cerillas (byte, máximo 127)?

```
int grande = 300;
byte pequeno = (byte) grande;
System.out.println(pequeno);    // 44 — ¿¡cuarenta y cuatro!?
```

¿De dónde sale el 44? En binario, 300 es `100101100`. Un `byte` solo guarda 8 bits, así que se truncan los sobrantes y se queda con `00101100`, que es… 44. Es como meter un elefante en un Mini Cooper y que salga un perro salchicha: el resultado técnicamente es un animal, pero no el que metiste.

> ⚠️ **Advertencia:** el desbordamiento no da error. Java no te avisa. El programa sigue corriendo con un valor absurdo. Por eso el casting explícito es responsabilidad tuya: comprueba siempre que el valor cabe.

🏫 Ejemplo guiado: el guardia del almacén

Este mini-programa recorre toda la cadena de conversiones, para que veas cuándo Java te acompaña y cuándo te obliga a firmar:

```
public class CadenaDeCajas {
    public static void main(String[] args) {
        int a = 10;
        double b = a;            // implícita: int → double, sin lágrimas
        int c = (int) b;         // explícita: double → int, forzada
        byte d = (byte) c;       // explícita: int → byte, cabe de sobra

        System.out.println(b);   // 10.0
        System.out.println(c);   // 10
        System.out.println(d);   // 10
    }
}
```

Y ahora la parte mala:

```java
public class Perdidas {
    public static void main(String[] args) {
        double nota = 9.9;
        int notaEntera = (int) nota;    // 9 — adiós, décimas
        int enorme = 400;
        byte pequeno = (byte) enorme;   // desbordamiento silencioso
        System.out.println(notaEntera); // 9
        System.out.println(pequeno);    // -112 (¡ni siquiera es 400!)
    }
}
```

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** el casting con `(tipo)` puede truncar y desbordar. Siempre comprueba si el valor cabe antes de forzarlo.

Ejercicio: sigue al guardia

Sin ejecutar, determina qué imprime este programa:

```java
int a = 10;
double b = a;
int c = (int) b;
byte d = (byte) c;
System.out.println(d);

int grande = 300;
byte pequeno = (byte) grande;
System.out.println(pequeno);
```

> 🔄 Solución

🎯 Mini-chequeo

1. ¿Por qué `long x = 100000;` compila sin casting y `int y = (int) 100000.5;` sí necesita el `(int)`?
2. ¿Cuánto vale `(int) 7.99`? ¿Y `(int) 7.1`?

3. ¿Qué pasa si conviertes `300` a `byte`?
4. ¿El truncamiento es lo mismo que redondear?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. La conversión **implícita** (widening) hacia cajas más grandes la hace Java solo y sin pérdidas.
2. La conversión **explícita** (narrowing) la fuerzas con `(tipo)` y puede **truncar** los decimales o **desbordar** el valor.
3. Antes de estrechar, comprueba siempre que el valor cabe: Java no te avisará de las pérdidas.

> 🐛 **Vocabulario rápido**

Término	Idea general
Casting	Convertir un valor de un tipo a otro
Widening	Ensanchar: a caja más grande, sin pérdidas, automático
Narrowing	Estrechar: a caja más pequeña, con `(tipo)` y posibles pérdidas
Truncamiento	Cortar la parte decimal, sin redondear
Desbordamiento	El valor no cabe y se convierte en algo absurdo en silencio
`(int)`	El casting que fuerza un valor decimal a entero

📚 Volver al índice de la unidad · **Anterior:** 04 · Relacionales, lógicos y ternario · **Siguiente:** 06 · Scanner: leer por teclado

06 · Scanner: leer por teclado

El momento en que tu programa deja de gritar y empieza a escuchar ⌨️

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 06 · Scanner: leer por teclado

---

📬 La idea en una frase

> **`Scanner` es la clase de Java que lee lo que escribes por teclado: instancias un objeto con `new Scanner(System.in)` y le pides datos con `nextInt()`, `nextDouble()` o `nextLine()`.**

Hasta ahora, tus programas eran unos gritones: solo escupían texto a la consola. A partir de este punto van a tener oídos. Y con oídos llegan los programas de verdad: un conversor de temperaturas que te pregunta los grados, una calculadora que recibe números…

📖 El paquete: importar la librería

`Scanner` no vive en el centro de Java: vive en una librería (`java.util`). Para usarlo, la primera línea de tu archivo debe ser:

```
import java.util.Scanner;
```

Es como pedir en la biblioteca el libro que vas a usar. Sin el `import`, Java te dirá que no conoce a `Scanner`.

🏗️ Instanciar: el constructor

Para tener un lector de teclado necesitas **crear un objeto** de la clase `Scanner`. Esto se hace con `new` y un **constructor** (recuerda el `new String(...)` del punto 2):

```
Scanner sc = new Scanner(System.in);
```

- `Scanner` es la clase (el molde).
- `new Scanner(...)` crea el objeto (el constructor).
- `System.in` es el argumento que le pasas: “lee del teclado estándar”.

> 💡 **Detalle práctico:** el nombre de la variable suele ser `sc` o `teclado`, por pura costumbre. Cuando acabes de usar el `Scanner`, es buena práctica cerrarlo con `sc.close()`, sobre todo si el programa va a seguir haciendo cosas raras.

🔢 Pedir datos: los métodos next

Una vez tienes el objeto `sc`, le pides datos con sus métodos. Cada método espera a que escribas y pulses Enter:

```
import java.util.Scanner;

public class PrimerEscucha {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("¿Cómo te llamas? ");
        String nombre = sc.nextLine();     // lee una línea de texto completa

        System.out.print("¿Cuántos años tienes? ");
        int edad = sc.nextInt();           // lee un número entero

        System.out.print("¿Cuál es tu nota media? ");
        double nota = sc.nextDouble();     // lee un número con decimales

        System.out.println("Hola, " + nombre + ". Con " + edad + " años y un " + nota +
" de media, vas sobrado.");

        sc.close();
    }
}
```

Salida de ejemplo:

```
¿Cómo te llamas? Ana
¿Cuántos años tienes? 18
¿Cuál es tu nota media? 9.5
Hola, Ana. Con 18 años y un 9.5 de media, vas sobrado.
```

Los métodos más usados:

Método	Lee	Ejemplo
`nextLine()`	Una línea de texto completa	`String nombre = sc.nextLine();`
`next()`	Solo la siguiente palabra	`String palabra = sc.next();`
`nextInt()`	Un número entero	`int edad = sc.nextInt();`
`nextDouble()`	Un número con decimales	`double nota = sc.nextDouble();`
`nextBoolean()`	`true` o `false`	`boolean ok = sc.nextBoolean();`

⚠️ El lío de `nextLine()` tras `nextInt()`

Este es el error más odiado del `Scanner`, y aparece en todos los exámenes. Cuando haces `nextInt()`, el Enter que pulsaste se queda *guardado* en el buffer. Si después llamas a `nextLine()`, esa llamada se traga el Enter residual y te devuelve una línea vacía:

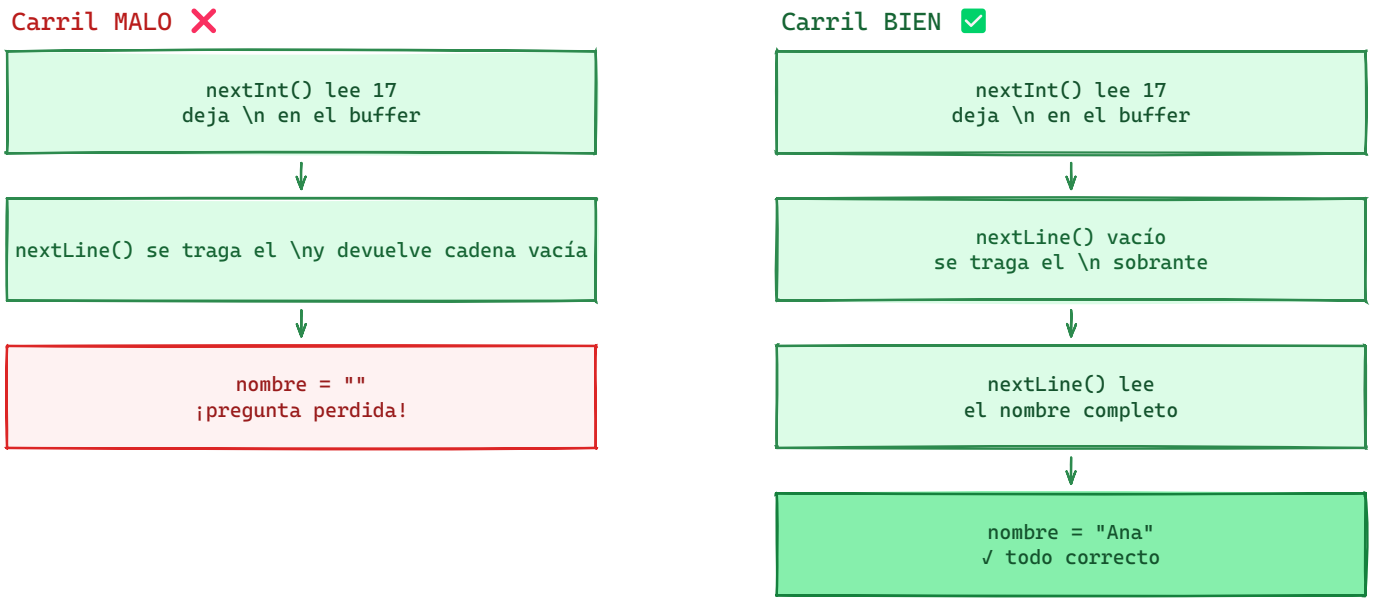
```
Scanner sc = new Scanner(System.in);
System.out.print("Edad: ");
int edad = sc.nextInt();          // escribes 18 y pulsas Enter
System.out.print("Nombre: ");
String nombre = sc.nextLine();    // ¡se salta la pregunta! devuelve ""
```

La solución: añade un `nextLine()` extra (o usa `next()` para el texto) justo después del número:

```
int edad = sc.nextInt();
sc.nextLine();                    // se traga el Enter sobrante
String nombre = sc.nextLine();    // ahora sí, lee el nombre
```

> ⚠️ **Advertencia:** memoriza el truco: *después de un `nextInt()` / `nextDouble()`, inserta un `nextLine()` vacío antes del siguiente `nextLine()`.* Es el guardián del buffer.

El Enter residual: dos caminos, dos resultados



Truco: un nextLine() vacío entre el número y el texto se traga el Enter

---

🏫 Ejemplo guiado: la calculadora de la propina

Vamos a usar el teclado para algo útil: calcular cuánto dejar de propina.

```java
import java.util.Scanner;

public class Propina {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("¿Cuánto vale la cuenta? ");
        double cuenta = sc.nextDouble();

        System.out.print("¿Qué porcentaje de propina? ");
        int porcentaje = sc.nextInt();

        double propina = cuenta * porcentaje / 100;
        double total = cuenta + propina;

        System.out.println("Propina: " + propina + "€");
        System.out.println("Total a pagar: " + total + "€");

        sc.close();
    }
}
```

Salida de ejemplo:

```
¿Cuánto vale la cuenta? 45.5
¿Qué porcentaje de propina? 15
Propina: 6.825€
Total a pagar: 52.325€
```

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando combines `nextInt()` y `nextLine()`, recuerda siempre el Enter residual. Escríbelo como un reflejo: número → `nextLine()` vacío → texto.

Ejercicio: el presentador con trampa

Este programa intenta saludar al usuario... pero algo falla. Sin ejecutar, di qué ocurre y cómo lo arreglarías:

```java
Scanner sc = new Scanner(System.in);
System.out.print("Edad: ");
int edad = sc.nextInt();
System.out.print("Nombre: ");
String nombre = sc.nextLine();
System.out.println(nombre + " tiene " + edad + " años.");
```

> 🔄 Solución

🎯 Mini-chequeo

1. ¿Qué hace la línea `import java.util.Scanner;`?
2. ¿Qué hace `new Scanner(System.in)`?
3. ¿Cuál es la diferencia entre `next()` y `nextLine()`?
4. ¿Por qué tras un `nextInt()` hay que poner un `nextLine()` vacío?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. `Scanner` es la clase para leer del teclado: la importas con `import`, la instancias con `new Scanner(System.in)` y pides datos con métodos `next...`.
2. Cada tipo de dato tiene su método: `nextInt()`, `nextDouble()`, `nextLine()` para texto.
3. Tras un `nextInt()` o `nextDouble()`, un `nextLine()` vacío se traga el Enter residual: sin él, tu siguiente pregunta se saltará.

> 🦎 Vocabulario rápido

Término	Idea general
`Scanner`	Clase de `java.util` para leer datos de teclado
`import`	Línea que trae una clase desde su librería
Constructor	Mecanismo que crea el objeto (`new Scanner(...)`)
`System.in`	El teclado estándar, la fuente de entrada

Término	Idea general
Buffer	Zona de memoria donde queda el Enter residual
Instanciar	Crear un objeto a partir de una clase

📚 Volver al índice de la unidad · **Anterior:** 05 · Casting y conversiones · **Siguiente:** 07 · Consola: salida formateada y errores de entrada

07 · Consola: salida formateada y errores de entrada

printf, String.format, NumberFormat y los fallos clásicos del Scanner (y cómo no morir en el intento) 🖨️

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 07 · Consola: salida formateada y errores de entrada

---

📬 La idea en una frase

> **`printf` y `String.format` dan formato a tu salida (decimales, ancho, alineación) en una línea, y conocer los errores típicos del `Scanner` te ahorra los bugs más odiados de la unidad.**

En el punto 6 aprendiste a leer del teclado. Ahora vas a darle belleza a lo que escribes y, de paso, a blindar tus programas contra los fallos más típicos de entrada. Con esto cierras el círculo de la consola.

🖨️ Salida con formato: `System.out.printf`

`System.out.println` imprime tal cual. Para controlar **cómo** se ve (decimales, ancho, relleno) tienes `printf` (print formatted):

```
String nombre = "Ana";
int edad = 20;
double nota = 9.5;

System.out.printf("Nombre: %s, Edad: %d, Nota: %.2f%n", nombre, edad, nota);
// Nombre: Ana, Edad: 20, Nota: 9,50
```

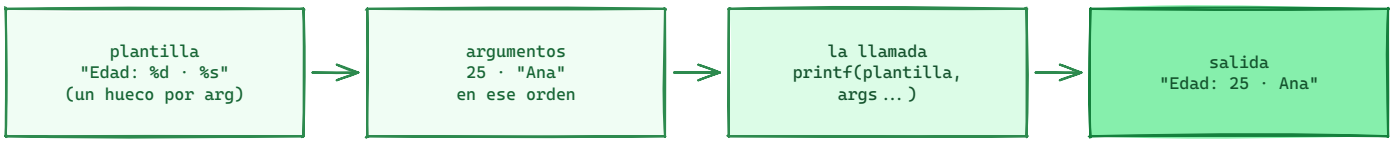
Cada `%algo` es un **hueco** que se rellena con el valor que le sigue, en orden. Los especificadores básicos:

Especificador	Tipo	Ejemplo
`%s`	String	`"Hola %s"` → `"Hola Mario"`
`%d`	Entero	`"Edad: %d"` → `"Edad: 25"`

Especificador	Tipo	Ejemplo
%f	Decimal	"%.2f" → "19,99"
%c	Carácter	"Inicial: %c"
%n	Salto de línea	(independiente del sistema)
%b	boolean	"%b" → true

> 💡 **Consejo:** %n para saltos de línea en printf (no \n): funciona igual en Windows, Linux y Mac. El \n también vale, pero %n es la opción "oficial".

printf: la plantilla y sus argumentos



Los argumentos se colocan en orden: el 1º va al primer hueco (%d), el 2º al segundo (%s).

Controlar los decimales y el ancho

```
double pi = Math.PI;
System.out.printf("2 decimales: %.2f%n", pi);   // 3,14
System.out.printf("4 decimales: %.4f%n", pi);   // 3,1416
System.out.printf("Ancho 10: %10.2f%n", pi);    //       3,14
System.out.printf("Izquierda: %-10.2f%n", pi);  // 3,14
```

- %.2f → dos decimales.
- %10.2f → ancho mínimo de 10 caracteres, alineado a la derecha.
- %-10.2f → el guion lo alinea a la izquierda.

> ⚠️ **Advertencia:** los decimales de printf usan la **configuración regional** de tu sistema. En un ordenador con locale español, %.2f escribe 3,14 (coma); en uno con locale inglés, 3.14 (punto). No te asustes si el resultado varía: es la máquina hablando en su idioma.

String.format: lo mismo, pero sin imprimir

A veces no quieres imprimir en el momento, sino **construir un texto** con formato para usarlo después (guardarlo, concatenarlo…). `String.format` hace exactamente lo mismo que `printf`, pero **devuelve** la cadena en vez de imprimirla:

```
String msg = String.format("Bienvenido, %s. Tienes %d mensajes nuevos.", "Carlos",
3);
System.out.println(msg);
// Bienvenido, Carlos. Tienes 3 mensajes nuevos.
```

> 💡 **Consejo:** usa `String.format` cuando quieras un texto con formato como **valor** (para guardarlo o usarlo varias veces), y `printf` cuando solo quieras escribirlo en pantalla.

💶 Números grandes: `NumberFormat`

Imprimir `1234567.89` sin formato es feo y difícil de leer. `NumberFormat` aplica los separadores de miles y decimales de tu idioma:

```
import java.text.NumberFormat;
import java.util.Locale;

NumberFormat nf = NumberFormat.getInstance(new Locale("es", "ES"));
System.out.println(nf.format(1234567.89));  // 1.234.567,89

NumberFormat moneda = NumberFormat.getCurrencyInstance(new Locale("es", "ES"));
System.out.println(moneda.format(12345.67));  // 12.345,67 €
```

- `getInstance(locale)` → formato numérico con separadores.
- `getCurrencyInstance(locale)` → formato de moneda (con el símbolo €).

> 📝 **Nota:** el `Locale("es", "ES")` le dice “habla como en España”: punto para los miles, coma para los decimales. Si no le pasas locale, usa el de tu sistema.

🚨 Errores clásicos del Scanner (y sus remedios)

El `Scanner` es traicionero. Estos son los fallos que se repiten en cada examen y en cada programa de prácticas:

1. Olvidar el `import java.util.Scanner;`

Sin la línea de import, Java no conoce la clase y te lanza un error de compilación. Es el fallo más tonto y el más común.

2. No cerrar el Scanner: `sc.close()`

Dejar el `Scanner` abierto es de mala educación (y en programas largos, puede dejar recursos sin liberar). Cierra siempre al terminar:

```
Scanner sc = new Scanner(System.in);
// ... todo tu código ...
sc.close();
```

3. Pedir un tipo y escribir otro: `InputMismatchException`

Si pides un `int` y el usuario escribe letras, el programa **explota** con `InputMismatchException`:

```
int edad = sc.nextInt();        // el usuario escribe "hola" → 💥 InputMismatchException
```

La solución robusta es **preguntar antes** con `hasNextInt()` (o `hasNextDouble()`, `hasNext()`…):

```
if (sc.hasNextInt()) {
    int edad = sc.nextInt();
} else {
    System.out.println("Eso no es un número entero.");
    sc.next();   // descarta el texto mal escrito
}
```

> ⚠️ **Advertencia:** `hasNextInt()` **no consume** el dato: solo mira si el siguiente es un entero. Si no lo es, debes consumir el texto basura con `sc.next()` antes de volver a preguntar, o se quedará ahí para siempre.

4. El lío del `nextLine()` tras `nextInt()` (ya lo viste en el punto 6)

El Enter residual se queda en el buffer. Tras un número, pon un `nextLine()` vacío antes de pedir texto.

🏫 Ejemplo guiado: el validador que no se rompe

Un programa que pide una edad y no se cae por mucho que el usuario escriba basura:

```
import java.util.Scanner;

public class EdadSegura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int edad = -1;

        while (edad == -1) {
            System.out.print("¿Cuántos años tienes? ");
            if (sc.hasNextInt()) {
                edad = sc.nextInt();
            } else {
                System.out.println("Eso no es un número entero, intenta otra vez.");
                sc.next();
            }
        }

        System.out.printf("Genial, %d años y listo para programar.%n", edad);
        sc.close();
    }
}
```

El bucle `while` repite la pregunta hasta que el usuario da un entero. Con `hasNextInt()` + `sc.next()` para descartar la basura, el programa es **a prueba de bombas**.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** la regla de oro: `printf` para imprimir con formato, `String.format` para guardar el texto formateado, `hasNextInt()` antes de cada `nextInt()` si el usuario puede equivocarse.

Ejercicio: el formateador misterioso

¿Qué imprime exactamente este programa?

```java
public class Formateo {
    public static void main(String[] args) {
        int horas = 5;
        double precio = 12.5;
        System.out.printf("Trabajo: %d horas a %.1f €/hora = %.2f €%n",
                horas, precio, horas * precio);
    }
}
```

> 🔄 Solución

🎯 Mini-chequeo

1. ¿Qué diferencia hay entre `System.out.printf` y `String.format`?
2. ¿Qué especificador usarías para un `double` con dos decimales?
3. ¿Qué excepción lanza `sc.nextInt()` si el usuario escribe letras?
4. ¿Cómo evitas que `sc.nextInt()` explote con una entrada incorrecta?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. **`printf` y `String.format`** dan formato a la salida con especificadores (`%d`, `%s`, `%.2f`) y control de decimales y ancho.
2. **`NumberFormat`** formatea números grandes y monedas con los separadores de tu idioma.
3. El `Scanner` se rompe con `InputMismatchException` si el usuario escribe mal: pregunta antes con **`hasNextInt()`** y cierra siempre con `sc.close()`.

> 🐛 Vocabulario rápido

Término	Idea general
`printf`	Imprime con formato (`%d`, `%s`, `%.2f`…)
`String.format`	Devuelve un texto con formato, sin imprimirlo
Especificador	El `%algo` que marca dónde y cómo se coloca un valor

Término	Idea general
`NumberFormat`	Formatea números y monedas con separadores locales
`InputMismatchException`	Explosión al pedir un tipo y recibir otro
`hasNextInt()`	Pregunta si lo que viene es un entero, sin consumirlo

Volver al índice de la unidad · **Anterior:** 06 · Scanner: leer por teclado · **Siguiente:** 08 · Math.random() y números aleatorios

08 — Math.random() y números aleatorios

El casino de Java, con dados y fórmulas para no hacer trampa 🎲

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 08 · Math.random() y números aleatorios

---

📬 La idea en una frase

> **`Math.random()` devuelve un número aleatorio entre `0,0` (incluido) y `1,0` (excluido), y con la fórmula `(int)(Math.random() * (max - min + 1)) + min` lo conviertes en un dado, una lotería o cualquier número que necesites.**

Tus programas ya escuchan (Scanner) y calculan (operadores). Ahora van a jugar: a los dados, a la lotería, a adivinar números. Y para eso necesitas el casino de Java: `Math.random()`.

🎲 El casino: qué devuelve Math.random()

`Math.random()` es un **método estático** de la clase `Math`. Devuelve un número entre `0,0` y `1,0`… con un detalle importante: **el `1.0` no está incluido**. Es como cuando te toca la lotería pero no.

```
double aleatorio = Math.random();          // Entre 0.0 y 0.999999...
System.out.println(aleatorio);             // Por ejemplo: 0.473821...
```

> 💡 **Detalle práctico:** `Math.random()` es un método **estático**: no necesitas crear un objeto de `Math` para llamarlo. Escribes `Math.random()` directamente y listo. Es una llamada de clase, no de objeto.

🎯 La fórmula: de 0.0-1.0 a lo que tú quieras

Un número entre 0 y 1 está muy bien, pero tú quieres un dado, un número de la lotería, un porcentaje… Aquí va la escalera:

```java
double aleatorio = Math.random();                    // Entre 0.0 y 0.999999...
int deCeroANueve = (int) (Math.random() * 10);       // Entre 0 y 9
int dado = (int) (Math.random() * 6) + 1;            // Entre 1 y 6 (como un dado)
```

¿Ves el patrón? El truco está en multiplicar y sumar:

- `Math.random() * 6` → número entre `0,0` y `5,999...`
- `(int)` lo trunca → entre `0` y `5`
- `+ 1` lo desplaza → entre `1` y `6` ✅

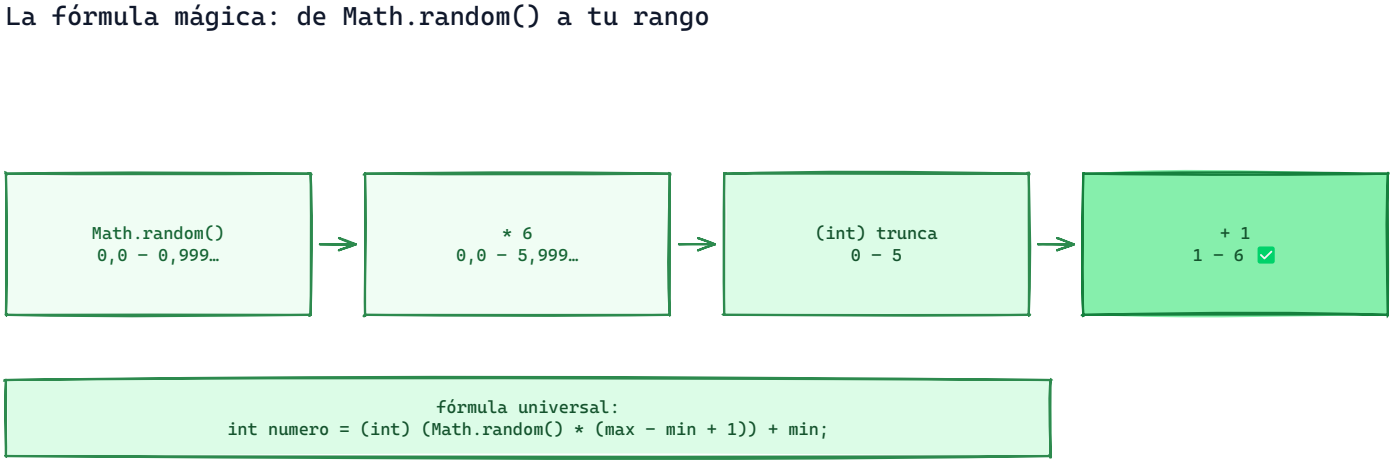
La fórmula universal para un número entre `min` y `max` (ambos incluidos):

```java
int numero = (int) (Math.random() * (max - min + 1)) + min;
```

Ejemplo del 5 al 10:

```java
int entreCincoYDiez = (int) (Math.random() * 6) + 5;   // 5, 6, 7, 8, 9 o 10
```

> 📝 **Nota:** memoriza la fórmula como un mantra: `(max - min + 1)` da el tamaño del abanico, y `+ min` lo coloca donde empieza. No hay más secreto.



🧰 Otras herramientas del casino: la clase Math

`Math` no es solo la ruleta: es toda la sala de máquinas. Algunas joyas que usarás a diario:

```java
double pi = Math.PI;                    // 3.141592653589793 — constante
double potencia = Math.pow(2, 10);      // 1024.0 — 2 elevado a 10
double raiz = Math.sqrt(144);           // 12.0
double absoluto = Math.abs(-7);         // 7.0
double redondeo = Math.round(4.6);      // 5.0
double techo = Math.ceil(4.1);          // 5.0
double suelo = Math.floor(4.9);         // 4.0
int maximo = Math.max(3, 9);            // 9
int minimo = Math.min(3, 9);            // 3
```

> 💡 **Detalle práctico:** todos estos son **métodos estáticos** de la clase `Math` (y `Math.PI` una constante estática): se llaman con `Math.nombre`, sin crear objetos. En el examen, la pregunta típica es "¿cómo redondeas 4.6 sin truncarlo?" → `Math.round(4.6)`.

🏫 Ejemplo guiado: el juego de los dados

Vamos a montar un dado de verdad, con dos tiradas, suma y veredicto:

```java
public class JuegoDeDados {
    public static void main(String[] args) {
        int dado1 = (int) (Math.random() * 6) + 1;
        int dado2 = (int) (Math.random() * 6) + 1;
        int suma = dado1 + dado2;

        System.out.println("Dado 1: " + dado1);
        System.out.println("Dado 2: " + dado2);
        System.out.println("Suma: " + suma);

        boolean esPar = suma % 2 == 0;
        String mensaje = esPar ? "Suma par — ganas" : "Suma impar — pierdes";
        System.out.println(mensaje);
    }
}
```

Salida posible:

```
Dado 1: 4
Dado 2: 6
Suma: 10
Suma par — ganas
```

Cada ejecución da un resultado distinto: eso es lo divertido (y a veces frustrante) de los aleatorios.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** recuerda siempre *truncar antes de sumar*: `(int)(... * 6) + 1`, nunca `(int)(... * 6 + 1)`. Si sumas antes del casting, el rango cambia y tus dados mentirán.

Ejercicio: el dado que miente

¿Qué rango de números produce cada una de estas tres líneas? ¿Cuál es la que da un dado de verdad (1 a 6)?

```java
int a = (int) (Math.random() * 6);
int b = (int) (Math.random() * 6) + 1;
int c = (int) (Math.random() * 7);
```

> 🔁 **Solución**

🎯 Mini-chequeo

1. ¿Qué rango exacto devuelve `Math.random()`?
2. Escribe la línea para obtener un número aleatorio entre 5 y 10.
3. ¿Qué devuelve `(int) (Math.random() * 100)`? ¿Y si le sumas 1?
4. ¿`Math.PI` es un método o una constante? ¿Y `Math.round`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. `Math.random()` devuelve un número entre `0,0` y `1,0` (sin incluir el 1), y se combina con multiplicaciones y casting para generar el rango que quieras.
2. La fórmula `(int)(Math.random() * (max - min + 1)) + min` es tu navaja suiza para cualquier número aleatorio.
3. `Math` es la sala de máquinas estática: `PI`, `pow`, `sqrt`, `abs`, `round`… todos se llaman con `Math.nombre`, sin crear objetos.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `Math.random()` | Número aleatorio entre 0.0 y 1.0 (el 1 excluido) |
> | Método estático | Se llama con `Math.nombre(...)`, sin crear objetos |
> | `Math.pow` | Potencia: `Math.pow(base, exponente)` |
> | `Math.round` | Redondea (a diferencia del truncamiento del casting) |
> | `Math.PI` | Constante con el número π |
> | Truncar | Cortar los decimales con `(int)` |

📚 Volver al índice de la unidad · **Anterior:** 07 · Consola: salida formateada y errores de entrada · **Siguiente:** 09 · Métodos útiles de String

09 — Métodos útiles de String

La caja de herramientas para manipular texto 🪄

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 09 · Métodos útiles de String

📬 La idea en una frase

> **`String` trae de serie una caja de herramientas con métodos para medir, recortar, buscar y transformar texto (`length()`, `trim()`, `toUpperCase()`, `substring()`, `replace()`…).**

En el punto 2 conociste al `String` como la caja mágica del texto. Ahora vas a abrir su caja de herramientas: porque los programas no solo guardan texto, también lo miden, lo limpian, lo ponen en mayúsculas y lo trocean.

🪄 El gimnasio del texto: los métodos que medirás

Aquí está el arsenal completo. Fíjate en cómo se llama un método de objeto: `texto.metodo()`, con un punto entre la variable y el método (no como `Math.random()`, que era estático):

```
String texto = "  Programación DAM  ";

texto.length();              // 20 — cuántos caracteres hay (espacios incluidos)
texto.trim();                // "Programación DAM" — sin espacios a los lados
texto.toUpperCase();         // "  PROGRAMACIÓN DAM  "
texto.toLowerCase();         // "  programación dam  "
texto.contains("DAM");       // true — ¿contiene ese texto?
texto.startsWith("  ");      // true — ¿empieza por...?
texto.endsWith("AM  ");      // true — ¿termina por...?
texto.indexOf("DAM");        // 15 — ¿en qué posición empieza "DAM"?
texto.substring(2, 14);      // "Programación" — recorta del carácter 2 al 13
texto.replace("DAM", "DAW"); // "  Programación DAW  " — sustituye texto
```

> 💡 **Detalle práctico:** `length()` es un **método** (con paréntesis). Es el error clásico del novato escribir `texto.length` sin paréntesis y que no compile. En cambio, para un array (la U06) se usa `.length` sin paréntesis. Los `String` llevan paréntesis; los arrays, no.

🔍 Los métodos que buscan

Cuando necesitas saber si algo está dentro del texto:

```
String email = "ana@instituto.edu";
boolean tieneArroba = email.contains("@");          // true
boolean esDeEdu = email.endsWith(".edu");           // true
boolean empiezaPorAna = email.startsWith("ana");    // true
int posicionArroba = email.indexOf("@");            // 3 — el @ está en la posición 3
```

> 📝 **Nota:** `indexOf()` devuelve la **posición** (empezando en 0) donde encuentra el texto, o **-1** si no lo encuentra. Es el "buscar" de Java.

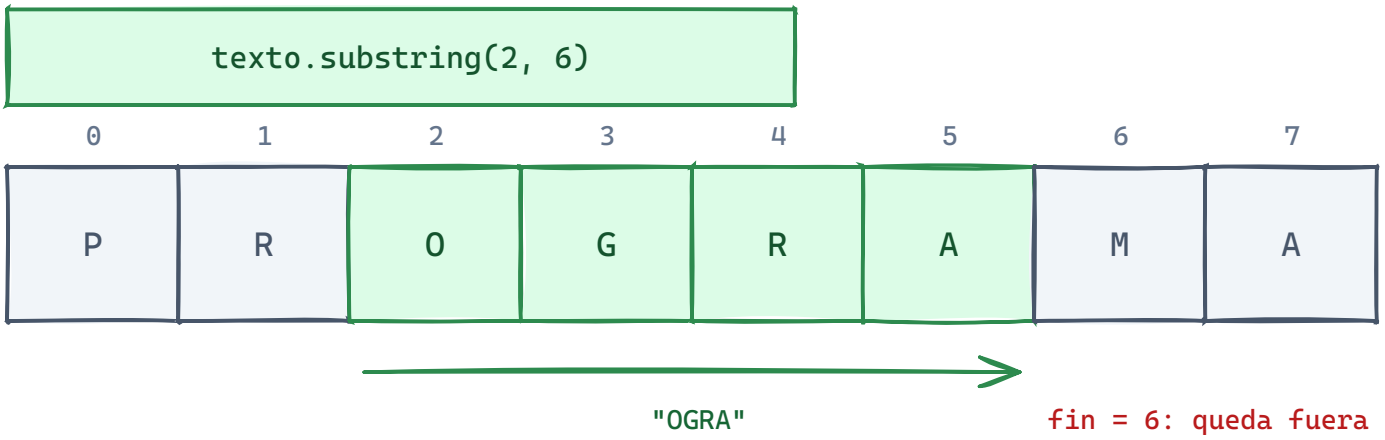
✂️ Los métodos que recortan

El clásico: limpiar espacios y trocear. `trim()` es el héroe silencioso de los formularios mal rellenados:

```
String sucio = "   Ana   ";
String limpio = sucio.trim();          // "Ana" — sin espacios alrededor
String nombreCompleto = "Ana Martínez";
String nombre = nombreCompleto.substring(0, 3);          // "Ana" — del 0 al 3 (sin incluir el 3)
String apellido = nombreCompleto.substring(4);         // "Martínez" — desde el 4 hasta el final
```

> ⚠️ **Advertencia:** en `substring(inicio, fin)`, el `fin` **no se incluye**. `substring(0, 3)` te da los caracteres 0, 1 y 2. Es un error típico pedir un carácter de más (o de menos).

substring(inicio, fin): el fin NO se incluye



🏠 Ejemplo guiado: el nombre del usuario

Vamos a construir un programa que procese un nombre como lo haría un formulario serio: limpiando espacios y mostrando datos:

```
public class ProcesaNombre {
    public static void main(String[] args) {
        String nombre = "  aNA  ";

        String limpio = nombre.trim();                    // "aNA"
        String enMayusculas = limpio.toUpperCase();       // "ANA"
        String enMinusculas = limpio.toLowerCase();       // "ana"
        String primera = enMayusculas.substring(0, 1);    // "A"
        String ultima = enMayusculas.substring(enMayusculas.length() - 1);  // "A"

        System.out.println("Nombre limpio: " + limpio);
        System.out.println("Longitud: " + limpio.length());
        System.out.println("En mayúsculas: " + enMayusculas);
        System.out.println("Primera letra: " + primera);
        System.out.println("Última letra: " + ultima);
    }
}
```

Salida:

```
Nombre limpio: aNA
Longitud: 3
En mayúsculas: ANA
Primera letra: A
Última letra: A
```

Fíjate en la última letra: `enMayusculas.length() - 1` es la última posición, porque las posiciones empiezan en 0.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** encadena los métodos para expresiones potentes: `texto.trim().toUpperCase().substring(0, 1)` hace tres cosas en una línea. Java las ejecuta de izquierda a derecha.

Ejercicio: la inicial de una reina

Sin ejecutar, di qué imprime este código:

```java
String nombre = "  merida  ";
String inicial = nombre.trim().toUpperCase().substring(0, 1);
String resto = nombre.trim().substring(1).toLowerCase();
System.out.println(inicial + ". " + resto);
```

> › 🔄 **Solución**

🎯 Mini-chequeo

1. ¿`String.length()` lleva paréntesis o no? ¿Por qué?
2. ¿Qué hace `trim()` y cuándo es imprescindible?
3. ¿Qué devuelve `indexOf("@")` si el texto no contiene `@`?
4. ¿Qué devuelve `"Hola".substring(1, 3)`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los métodos de `String` se llaman sobre la variable (`texto.metodo()`) y transforman el texto en algo nuevo.
2. `length()`, `trim()`, `toUpperCase()`, `contains()`, `indexOf()`, `substring()` y `replace()` cubren el 90% de lo que harás con texto.
3. En `substring(inicio, fin)` el fin no se incluye, e `indexOf()` devuelve `-1` cuando no encuentra nada.

> 🦎 **Vocabulario rápido**

Término	Idea general
`length()`	No de caracteres de un String (con paréntesis)
`trim()`	Quita los espacios de los extremos
`substring()`	Recorta una porción del texto
`indexOf()`	Posición de la primera aparición (o `-1`)
`replace()`	Sustituye una parte del texto por otra
Encadenar métodos	Aplicar varios métodos seguidos con puntos

📚 Volver al índice de la unidad · **Anterior:** 08 · Math.random() y números aleatorios · **Siguiente:** 10 · Repaso interactivo

10 · Repaso interactivo: la caja que no cabía

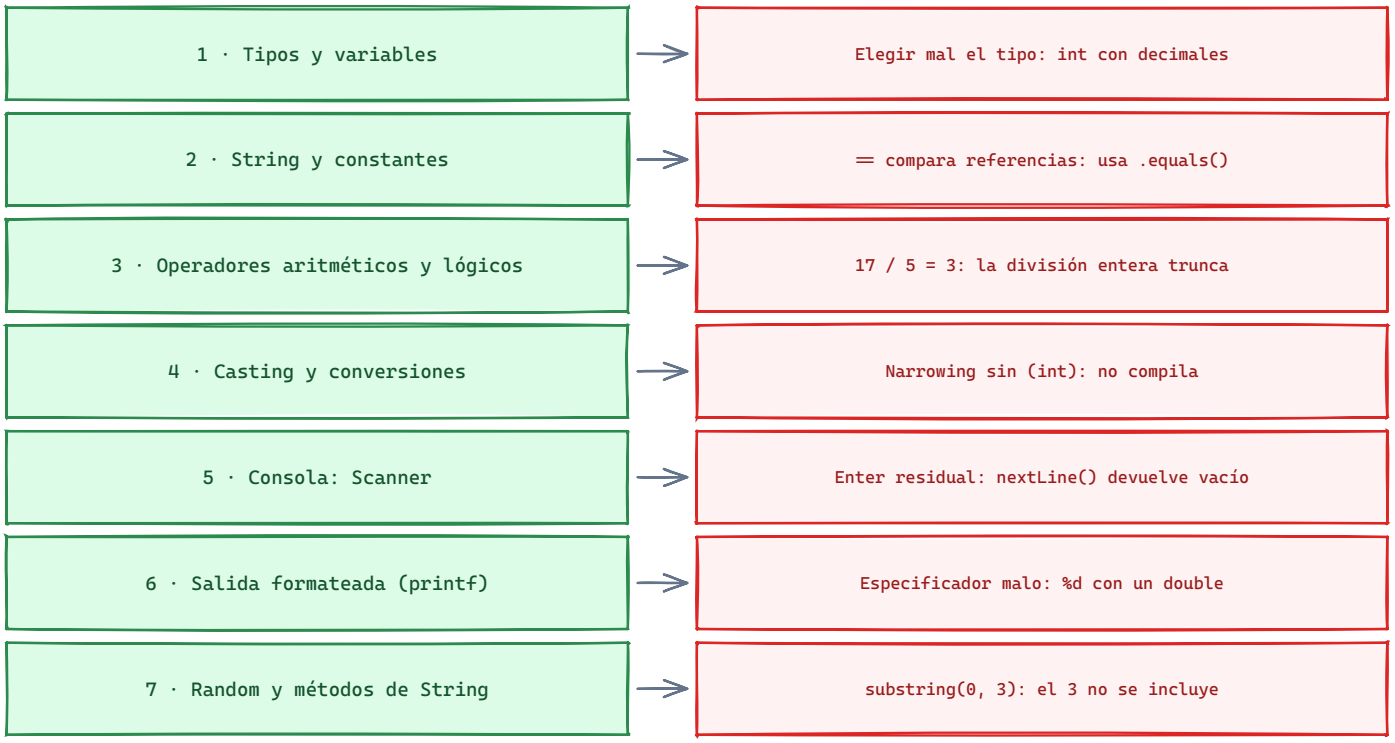
El cierre práctico de la unidad, con un bug de truncamiento que no va a gustar a nadie 😈

> 🗺️ **Estás en:** 🔤 **U03 · Variables, tipos y operadores** → 10 · Repaso interactivo

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

Mapa de la unidad 03: 7 zonas y su error típico



⭐ Sé el Código, my friend…

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```java
public class Misterio {
    public static void main(String[] args) {
        int nota = 7;
        int sobre = 10;
        System.out.println("Nota " + nota + sobre);
        System.out.println("Nota " + (nota + sobre));
        System.out.println(nota / 2 + " de nota media");
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **`Nota 710`, `Nota 17 y 3 de nota media`** → ✅ ¡Correcto! En la primera línea, al ver texto antes del `+`, Java concatena: `"Nota " + 7` es `"Nota 7"` y luego `+ 10` da `"Nota 710"`. En la segunda, los paréntesis fuerzan la suma: `Nota 17`. Y `7 / 2` es división entera: 3.
2. **`Nota 17`, `Nota 17 y 3.5 de nota media`** → Los paréntesis no cambian nada y la división entera se redondea. ❌
3. **`Nota 710`, `Nota 17 y 3.5 de nota media`** → La división de dos enteros da decimales. ❌

> 🔄 Solución

🔥 Fireside Chat: int vs double

> *Dos cajas del almacén discuten junto a la estantería de los datos.*

int: — Yo soy la caja de mudanza. Compacta, exacta, sin decimales. En mí no hay espacio para tonterías. 17 dividido entre 5 son 3 y se acabó.

double: — *arquea una ceja* ¿3? ¿En serio? Para mí son 3.4. Yo guardo los decimales de verdad. Los precios, las notas medias, las temperaturas… todo eso vive en mi casa.

int: — Sí, y cuando intentas meter un número mío en tu caja, va todo bien. Pero cuando intentas meter tú el tuyo en la mía… ¡tienes que pedir permiso con `(int)` y aun así pierdes céntimos por el camino!

double: — *encogiendo de hombros* Es el precio de la precisión. Yo guardo más dígitos que tú. ¿Sabes cuántas veces he visto a novatos escribir `(int)(Math.random() * 6 + 1)` y llorar porque el dado nunca salía 6?

int: — Vale, vale… ¿Y para contar? ¿Para un bucle? ¿Para una edad? A mí me llaman a mí. Contar con decimales no tiene sentido.

double: — Y medir, calcular medias y precios, me llaman a mí. Somos un equipo: tú para lo entero, yo para lo fino.

int: — *gruñe* Un equipo. Bah. Pero… vale. Tú y yo, y `long` para los astronómicos y `char` para las letras. Todos en el mismo almacén.

> La lección: **`int` es para lo entero, `double` para lo decimal**. Confundirlos en una división (o en un casting mal puesto) produce los errores más clásicos de esta unidad.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy el superglue del almacén: una vez que metes algo en mí, no sale ni con palanca. Me escriben en MAYÚSCULAS para que todos me respeten.**
2. **Comparo dos valores y solo sé decir dos palabras: `true` y `false`. Soy el juez de la discusión.**
3. **Soy la caja mágica del texto: no soy primitiva, soy una clase, y si intentas cambiarme, tiro lo viejo y creo uno nuevo.**
4. **Soy el oído del programa: espero a que escribas por teclado y luego paso lo que leíste a una variable.**
5. **Soy el casino: te doy un número entre 0 y 1, y si me multiplicas y me conviertes a `int`, te hago un dado.**

> 🔄 **Respuestas**

👹 CONRAD VS EL MUNDO: “El compilador odia tu memoria”

> *CONRAD, nuestro compilador cascarrabias, opina sobre los clásicos del novato en esta unidad.*

CONRAD: — ¡OTRA VEZ! Llega un alumno y me da esto: `long distancia = 3000000000;` sin la L. Y yo: *eso es un int, y no cabe, leñe.* Pero él no me escucha. ¿Por qué? ¡Porque cree que ya lo

sabe todo!

Y luego el precio: `float precio = 19.99;`. Un `double` suelto dentro de una caja `float`… y va y le falta la `f`. ¡LA F, HOMBRE! Es una letra, una sola. ¿Tan difícil es? Y `char letra = "A"` con comillas dobles. ¡Las simples, las simples! Es como confundir un piso con una escalera.

Y el rey del mambo: `String nombre = "Ana";` y luego `if (nombre == "Ana")`. ¿PERO TÚ TE HAS LEÍDO EL PUNTO 2? `==` compara referencias, no texto. Es que te estoy dando el examen con las respuestas y me lo devuelves en blanco.

La lección: en esta unidad, el compilador es tu mejor amigo: `long` necesita L, `float` necesita `f`, `char` usa comillas simples y los `String` se comparan con `.equals()`. Aprende la lista de cuatro y te ahorrarás el 80% de las broncas.

⚡ Laboratorio de tortura: el programa que cobra mal

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa en tu IDE y haz que funcione. Es un cajero que calcula cuántos billetes de 5 € te da el banco por un reintegro. Tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto cuando lo arregles.

```
import java.util.Scanner;

public class Tortura {
    public static void main(string[] args)
        Scanner sc = new Scanner(System.in);
        System.out.print("Cantidad a retirar: ");
        int cantidad = sc.nextInt()
        double billetes5 = cantidad / 5.0;
        System.out.println("Te dan " + billetes5 + " billetes de 5");
        sc.close();
    }
}
```

Fallo intencionado: uno de los errores parece correcto a simple vista porque “se ve bien”, pero cambia por completo la salida del programa.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Prueba con `cantidad = 17`: ¿cuántos billetes de 5 deberían ser?

Pistas para cuando te frustres (no antes):

1. ¿Hay algún `;` que falte? *no → sigue buscando.*
   > **¿Y si sigo atascado?**
2. ¿Compila ya? *no → mira el mensaje de error y las mayúsculas.*
   > **¿Y si sigo atascado?**
3. ¿Ejecuta pero el número de billetes sale con decimales? *Es el error de lógica: mezcla de tipos.*
   > **Solución final**

Los **3 errores de compilación**:

1. `string[] args` → `String[] args` (la clase `String` con mayúscula).
2. Falta el `{` que abre el cuerpo del `main` después de `main(string[] args)`.
3. Falta el `;` al final de `int cantidad = sc.nextInt()`.

El **error de lógica**: `double billetes5 = cantidad / 5.0;` compila, pero con `cantidad = 17` imprime **3.4** billetes… y un cajero no te puede dar 3.4 billetes. La solución es usar **división entera**: `int billetes5 = cantidad / 5;`

```
import java.util.Scanner;

public class Tortura {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Cantidad a retirar: ");
        int cantidad = sc.nextInt();
        int billetes5 = cantidad / 5;
        System.out.println("Te dan " + billetes5 + " billetes de 5");
        sc.close();
    }
}
```

Para `cantidad = 17`: `17 / 5` con enteros da **3** billetes (y sobran 2 €). Con la versión rota, `cantidad / 5.0` daba 3.4, que como `double` sí se imprime tal cual: el cajero “te daba 3.4 billetes”. La división entera es lo que hace el trabajo limpio.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🎖️ **El Guardia del Almacén**	Explicar los 8 tipos primitivos y qué cabe en cada caja sin mirar
🎖️ **El Equilibrista**	Resolver `int y = x++ + ++x;` sin ejecutar y sin llorar
🎖️ **El Traductor de Texto**	Normalizar un nombre con `trim().toUpperCase().substring(...)` en una línea
🎖️ **El Dado Trucado**	Generar 10 números aleatorios entre 1 y 6 sin que salga nunca un 0
🎖️ **El Coyote del Scanner**	Evitar el lío de `nextLine()` tras `nextInt()` sin que nadie te lo recuerde

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```
public class Misterio2 {
    public static void main(String[] args) {
        int a = 10;
        int b = 3;
        System.out.println("a/b = " + a / b);
        System.out.println("a/b real = " + (double) a / b);
        System.out.println("a%b = " + a % b);
    }
}
```

2. **El precio que no cuadra:** un programa calcula `double total = precio * 0.21;` con `precio = 100` y muestra `21.000000000000004`. ¿Qué le pasa a Java? ¿Cómo lo arreglarías solo con herramientas de esta unidad?

3. **El ternario encadenado:** escribe un ternario (o varios encadenados) que asigne a `categoria` el valor `"niño"`, `"adulto"` o `"jubilado"` según si la edad es `< 12`, `< 65` o `>= 65`.

4. **Verdadero o falso:** "`Math.random() * 5` puede devolver el número 5." Justifica.

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Operador que da el resto de una división (1 letra)
3. Modificador que hace una variable constante (5 letras)
5. Clase para leer del teclado (7 letras)
7. Método que quita los espacios de los extremos (4 letras)

Vertical:
2. Tipo que guarda texto (6 letras)
4. Clase con métodos estáticos como random y round (4 letras)
6. Tipo primitivo de verdadero o falso (7 letras)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo tuviera ocho años, qué es una variable y qué es un tipo primitivo."**
2. **"¿Por qué `double nota = 7/2;` da `3.0` y no `3.5`? ¿Y cómo lo arreglarías?"**
3. **"¿Cuándo usarías `int` y cuándo `long`? Pon un ejemplo de cada uno."**
4. **"¿Qué es un casting y qué riesgos tiene hacer un casting de `double` a `int`?"**
5. **"¿Cómo lees un número entero y una línea de texto desde el teclado sin que el texto se quede vacío?"**

🤷 No hay preguntas tontas

> ❓ **¿Por qué a veces pongo `L` al final de un `long` y otras veces no?**

Porque depende del número. Si el número cabe en un `int` (máximo 2.147 millones), `long x = 100;` va sin `L` (Java lo promociona solo). Si supera ese límite, `long x = 3000000000;` necesita la `L`

para que Java no intente meterlo en un `int` y se queje.

> ❓ **¿`Math.random()` es un método del objeto `Math`?**

No exactamente: `Math` es una **clase**, y `random()`, `pow()`, `round()`… son **métodos estáticos**. No creas ningún objeto de `Math`; llamas directamente `Math.random()`. Es la diferencia entre "llamar a la clase" y "llamar al objeto", que verás a fondo en la U10.

> ❓ **Si escribo `int nota = (int) 7.99;`, ¿me da 8 por redondeo?**

No. El casting **trunca**, no redondea: `(int) 7.99` da **7**. Para redondear de verdad usa `Math.round(7.99)`. El truncamiento corta con hacha; el redondeo negocia.

> ❓ **¿Puedo sumar un `String` y un número así, sin más?**

Sí: `"Resultado: " + 5` da `"Resultado: 5"`. Java convierte el número a texto y lo concatena. Eso se llama **concatenación**. El problema viene cuando te olvidas de los paréntesis: `"Suma: " + 5 + 3` da `"Suma: 53"`. ¡Los paréntesis son vida!

🎬 Poscréditos

El programador novato termina su cajero de billetes, lo compila y lo ejecuta. Funciona. Entonces se acerca CONRAD, el compilador cascarrabias, con cara de pocos amigos y una caja etiquetada `double` en la mano.

CONRAD: — Muy bonito tu cajero. Declaras variables, haces castings, lees del teclado… casi parece un programa de verdad. Pero hay un problema.

Novato: — ¿Qué problema?

CONRAD: — *señala el código con desprecio* Tu programa solo hace UNA cosa. La ejecutas y se acabó. Si el usuario quiere probar otro reintegro… ¿tiene que volver a ejecutar? Los programas de verdad **deciden** y **repiten**. Eligen entre caminos y dan vueltas hasta que se acaba la condición.

Novato: — ¿Y eso cuándo se aprende?

CONRAD: — *sonríe malvado* La próxima parada. Ya te veo escribiendo tu primer `if`… y tu primer bucle infinito. Espérame, que yo estaré ahí para reírme.

El novato guarda su proyecto, cierra el IDE y siente que el almacén de datos ya tiene dueño.

PRÓXIMAMENTE EN U04: Estructuras de control y excepciones. El momento en que tus programas dejan de hacer *una* cosa y empiezan a *decidir* y *repetir*. 🔀

📚 Volver al índice de la unidad · **Anterior:** 09 · Métodos útiles de String · **Siguiente:** U04 · Estructuras de control y excepciones básicas

U04 — Estructuras de control y excepciones básicas

El semáforo, la cinta de correr y el seguro de vida que hacen decidir y repetir a tus programas 🔀

Bienvenido al semáforo del código. En la U02 tu programa aprendió a hablar y en la U03 a guardar datos. Pero hablar y guardar sin decidir es un loro con archivador: repite, pero no piensa. Esta es la unidad donde tus programas empiezan a **pensar**.

Hasta ahora, tu código corría en línea recta, de arriba abajo, como un soldadito obediente. En esta unidad vas a darle tres superpoderes:

1. **Decidir**: "si aprueba, sube la nota; si no, a recuperación". Eso es `if`, `else` y `switch`.
2. **Repetir**: "imprime las tablas de multiplicar, todos los números del 1 al 10". Eso es `while`, `do-while` y `for`.
3. **Sobrevivir**: "¿y si el usuario escribe letras donde se esperaban números?". Eso es `try`, `catch`, `throw` y las excepciones.

Y, de propina, aprenderás a escaparte de un bucle con `break`, a saltarte una vuelta con `continue` y a inventarte tus propias excepciones. Sí, como en la vida real: te quejas de algo, y de paso construyes tu propio defecto.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Tomar decisiones con **`if`, `else if` y `else`**, anidarlos sin perder el norte y resumir decisiones simples con el **ternario**.
- Elegir entre muchas opciones con **`switch`**, sin olvidar el `break` ni el `default`.
- Repetir código con **`while`** y **`do-while`**, sabiendo cuándo se ejecuta al menos una vez.
- Contar vueltas con **`for`**, controlar contadores y crear **bucles anidados**.
- Escaparte de un bucle con **`break`**, saltarte una vuelta con **`continue`** y usar **etiquetas** cuando haga falta.

- Entender la jerarquía de **`Throwable`, `Exception` y `RuntimeException`** y conocer las excepciones más comunes de Java.
- Proteger tu código con **`try` / `catch` / `finally`** y capturar varios errores a la vez.
- Lanzar excepciones con **`throw`** y crear tus propias excepciones con herencia.
- Escribir programas que **decidan, repitan y sobrevivan**, y depurarlos cuando algo explote.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · if, else if y else	Decisiones con `if`/`else`, anidados y ternario	Todos
02 · switch	`switch`, `case`, `break` y `default`	Todos
03 · Bucles: while y do-while	Repetir con `while` y `do-while`, bucles infinitos	Todos
04 · Bucle for y bucles anidados	`for` con contador y bucles dentro de bucles	Todos
05 · break, continue y etiquetas	Escapes del bucle y salto entre etiquetas	Todos
06 · Excepciones básicas	`Throwable`, `Exception`, `RuntimeException` y las excepciones típicas	Todos
07 · try, catch y finally	Proteger código, capturar errores y captura múltiple	Todos
08 · throw y excepciones propias	Lanzar excepciones y crear las tuyas	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Quién Soy, Laboratorio, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA3)

RA3: Escribe y depura código, analizando y utilizando las estructuras de control del lenguaje.

CE	Criterio	Dónde se cubre
RA3 a)	Se ha escrito y probado código que haga uso de estructuras de selección.	✅ Puntos 1 y 2
RA3 b)	Se han utilizado estructuras de repetición.	✅ Puntos 3 y 4
RA3 c)	Se han reconocido las posibilidades de las sentencias de salto.	✅ Punto 5
RA3 d)	Se ha escrito código utilizando control de excepciones.	✅ Puntos 6, 7 y 8
RA3 e)	Se han creado programas ejecutables utilizando diferentes estructuras de control.	✅ Todos
RA3 f)	Se han probado y depurado los programas.	✅ Todos, con chicha extra en los boletines
RA3 g)	Se ha comentado y documentado el código.	✅ Todos

> 📌 Los bucles que ves aquí son el músculo de todo lo que viene: los recorridos de arrays y colecciones (U12) y los algoritmos de ordenación y búsqueda (U08) no serían nada sin `for` y `while`.

🚪 ¿Por dónde empiezo?

- ¿Cero conocimientos? → Arranca en el punto 1. Solo necesitas las variables y el `Scanner` de la U03.
- ¿Ya sabes decidir y solo quieres repetir? → Ve directo al punto 3 y al punto 4.
- ¿Tu programa se rompe y no sabes por qué? → Los puntos 6, 7 y 8 te enseñan a leer los gritos de Java.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · if, else if y else

⏭️ **Al acabar la unidad, continúa en U05 · Funciones y métodos.**

01 — if, else if y else: el semáforo del código

Cuando tu programa aprende a decidir (y a tomarse los semáforos en serio) 🚦

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 01 · if, else if y else

📬 La idea en una frase

> **`if` es el semáforo del código: si la condición es `true`, deja pasar al bloque; si es `false`, lo redirige al `else` (o se queda esperando).**

En la U03 tus programas juzgaban con operadores relacionales y ternarios, pero esa justicia duraba una línea. Ahora llega la justicia de verdad: bloques enteros de código que se ejecutan o no según lo que decida Java.

🚦 El semáforo: if y else

La estructura básica es esta:

```
if (condicion) {
    // código que se ejecuta si condicion es true
} else {
    // código que se ejecuta si condicion es false
}
```

```
int edad = 17;

if (edad >= 18) {
    System.out.println("Puedes votar.");
} else {
    System.out.println("Aún no puedes votar.");
}
```

> ⚠️ **Advertencia:** el `else` es **opcional**. Un `if` solo, sin `else`, es perfectamente legal: si la condición falla, Java sigue como si nada.

🔀 else if: cuando hay más de dos caminos

¿Y si el semáforo tiene tres colores? Ahí entra el `else if`, que se encadena:

```
int nota = 7;

if (nota >= 9) {
    System.out.println("Sobresaliente");
} else if (nota >= 7) {
    System.out.println("Notable");
} else if (nota >= 5) {
    System.out.println("Aprobado");
} else {
    System.out.println("Suspenso");
}
```

Java evalúa las condiciones **en orden, de arriba abajo**. En cuanto una da `true`, se ejecuta su bloque y **se salta el resto**. El `else` final atrapa a todos los que no entraron.

> 💡 **Detalle práctico:** el orden importa. Si empezaras por `nota >= 5`, la nota 7 entraría en el "Aprobado" y nunca llegaría al "Notable". Ordena las condiciones de la más exigente a la más permisiva.

🪆 If anidados: semáforos dentro de semáforos

Un `if` puede vivir dentro de otro. Útil cuando quieres decidir *dentro* de una decisión:

```
int edad = 20;
boolean tieneCarnet = true;

if (edad >= 18) {
    if (tieneCarnet) {
        System.out.println("Puedes conducir.");
    } else {
        System.out.println("Te falta el carnet.");
    }
} else {
    System.out.println("Aún no puedes conducir.");
}
```

> ⚠️ **Advertencia:** no conviertas tus programas en las Torres Kio. Más de 3 niveles de anidamiento es señal de que estás haciendo las cosas raro: en la U07 aprenderás a aplanarlo.

🚦 El ternario: el if de bolsillo

Del punto 4 de la U03 lo conoces como "un if-else en una línea". Aquí está su momento de gloria:

```
int edad = 21;
String mensaje = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
System.out.println(mensaje);
```

La regla de oro: ternario para **asignar un valor** en una línea; `if/else` cuando el bloque es largo o hace más que asignar.

```
// ✅ Bien: ternario para elegir valor
double precioFinal = (dia.equals("viernes")) ? precio * 0.9 : precio;

// ✅ Bien: if cuando hay varias líneas por rama
if (saldo < 0) {
    System.out.println("Estás en números rojos.");
    System.out.println("Revisa tus gastos.");
} else {
    System.out.println("Saldo sano.");
}
```

🏫 Ejemplo guiado: la discoteca municipal

Vamos a montar el control de acceso de una discoteca con entrada reducida:

```java
public class Discoteca {
    public static void main(String[] args) {
        int edad = 16;
        boolean acompanado = true;

        if (edad >= 18) {
            System.out.println("Entra, mayor de edad.");
        } else if (edad >= 16 && acompanado) {
            System.out.println("Entra, pero con tu acompañante.");
        } else {
            System.out.println("Lo siento, vuelve en unos años.");
        }
    }
}
```

Salida:

```
Entra, pero con tu acompañante.
```

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando veas una cadena de `if/else if`, pregúntate siempre: *¿el orden es de la condición más estricta a la más laxa?* Ese es el 90% de los bugs de esta unidad.

Ejercicio: el semáforo confuso

Sin ejecutar, calcula qué imprime este programa:

```java
public class Semaforo {
    public static void main(String[] args) {
        int nota = 8;
        String resultado;

        if (nota >= 5) {
            resultado = "Aprobado";
        } else if (nota >= 7) {
            resultado = "Notable";
        } else if (nota >= 9) {
            resultado = "Sobresaliente";
        } else {
            resultado = "Suspenso";
        }

        System.out.println(resultado);
    }
}
```

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué hace Java si un `if` es `false` y no hay `else`?
2. ¿En qué orden debes encadenar las condiciones de un `else if`?
3. ¿Cuándo prefieres un ternario a un `if/else`?
4. ¿Qué imprimiría `String s = 5 > 3 ? "A" : "B";`?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. `if` / `else if` / `else` son el **semáforo** del código: ejecutan un bloque u otro según una condición booleana.

2. Las condiciones se evalúan **en orden** y gana la primera que dé `true`, así que ordena de la más estricta a la más laxa.
3. El **ternario** resume un if-else de dos valores en una línea, pero para bloques largos usa el `if` clásico.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Condición | Expresión booleana que decide: `edad >= 18` |
> | Rama | Cada uno de los caminos posibles (`if`, `else`) |
> | Anidar | Meter un `if` dentro de otro `if` |
> | Ternario | `condición ? valor1 : valor2`, un if-else en una línea |
> | Cortocircuito | Java deja de evaluar cuando la primera condición ya decide |

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · switch

02 — switch: el menú del restaurante

Elegir entre muchas opciones sin llenar el código de semáforos 🍽️

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 02 · switch

📬 La idea en una frase

> **`switch` es la carta de un restaurante: miras el valor de una variable y ejecutas el `case` que coincida, sin encadenar veinte `if`.**

Cuando tienes que elegir entre muchas opciones con un solo valor (día de la semana, talla, menú), una cadena de `else if` funciona pero es fea. `switch` existe para eso.

🍽️ La carta del restaurante

```
int dia = 3;

switch (dia) {
    case 1:
        System.out.println("Lunes");
        break;
    case 2:
        System.out.println("Martes");
        break;
    case 3:
        System.out.println("Miércoles");
        break;
    default:
        System.out.println("Día desconocido");
        break;
}
```

> ⚠️ **Advertencia:** el **`break` es obligatorio** (a menos que quieras "fall-through", que veremos abajo). Sin él, Java entra en el `case` correcto y sigue ejecutando todos los siguientes hasta encontrar un `break`. Es el clásico bug del novato.

🧱 Las piezas del puzzle

- **`switch (variable)`**: la variable que se examina. Admite tipos enteros, `char` y `enum`; a partir de Java 7 también `String`.
- **`case valor:`**: cada opción posible. Si la variable coincide, se ejecuta ese bloque.
- **`break;`**: "hasta aquí he llegado, salgo del switch". Sin él, todo se desborda hacia abajo.
- **`default:`**: el comodín, el "ninguno de los anteriores". Es opcional, como el `else`.

```
String talla = "M";

switch (talla) {
    case "S":
        System.out.println("Pequeña");
        break;
    case "M":
        System.out.println("Mediana");
        break;
    case "L":
        System.out.println("Grande");
        break;
    default:
        System.out.println("Talla no válida");
        break;
}
```

> 💡 **Detalle práctico:** usa `switch` cuando compares **una variable con muchos valores concretos**. Usa `if/else if` cuando las condiciones sean rangos ("mayor que 5", "entre 10 y 20") o mezclen variables.

🌀 El fall-through: ¿error o superpoder?

El famoso "caerse a través" ocurre cuando olvidas el `break`. En la mayoría de los casos es un bug:

```java
// ⚠️ Fall-through ACCIDENTAL: imprime los tres platos
int plato = 1;

switch (plato) {
    case 1:
        System.out.println("Ensalada");
        // sin break: se cae al siguiente
    case 2:
        System.out.println("Sopa");
        break;
}
```

Pero a veces se usa a propósito, para agrupar casos:

```java
// ✅ Fall-through INTENCIONAL: varios casos comparten bloque
char letra = 'a';

switch (letra) {
    case 'a':
    case 'e':
    case 'i':
    case 'o':
    case 'u':
        System.out.println("Vocal");
        break;
    default:
        System.out.println("Consonante");
        break;
}
```

Aquí, si `letra` es cualquier vocal, ejecuta el bloque compartido. Elegante y compacto.

VS switch vs else if: el duelo

Situación	Mejor opción
Un valor y muchas opciones concretas (`1..7`, `"S"/"M"/"L"`)	`switch`
Rangos o comparaciones (`>= 18`, `entre 10 y 20`)	`if/else if`

Situación	Mejor opción
Combinar varias variables	`if/else if`
Comprobar `null`	`if`

> 💡 **Nota de futuro:** en Java 14+ existe el `switch` con flechas (`->`) que no necesita `break` y devuelve valores. Lo verás como curiosidad avanzada; aquí aprendemos el clásico, que es el de todos los exámenes.

🏫 Ejemplo guiado: el menú del día

Montemos una carta que diga el plato según el día, con un `default` que cace a los despistados:

```
public class MenuDia {
    public static void main(String[] args) {
        String dia = "miercoles";

        switch (dia) {
            case "lunes":
                System.out.println("Lentejas");
                break;
            case "martes":
                System.out.println("Paella");
                break;
            case "miercoles":
                System.out.println("Macarrones");
                break;
            case "jueves":
                System.out.println("Fabada");
                break;
            case "viernes":
                System.out.println("Pescado");
                break;
            default:
                System.out.println("Fin de semana: no hay menú");
                break;
        }
    }
}
```

Salida:

```
Macarrones
```

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** cuando veas un `switch`, cuenta los `break`: **debe haber uno por cada `case` no compartido**. Si falta alguno, tu programa se convierte en un tobogán.

Ejercicio: el switch olvidadizo

Sin ejecutar, calcula qué imprime este programa:

```
public class Tobogan {
    public static void main(String[] args) {
        int numero = 2;

        switch (numero) {
            case 1:
                System.out.println("Uno");
            case 2:
                System.out.println("Dos");
            case 3:
                System.out.println("Tres");
                break;
            default:
                System.out.println("Otro");
                break;
        }
    }
}
```

> › 🔄 **Solución**

🎯 Mini-chequeo

1. ¿Qué pasa si un `case` no tiene `break`?
2. ¿Qué hace el `default`?

3. ¿Cuándo prefieres `switch` a una cadena de `else if`?
4. ¿Cómo agrupas varios valores en el mismo bloque de `switch`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. `switch` elige entre **muchas opciones concretas** de una variable, con `case`, `break` y `default`.
2. Sin `break` se produce el **fall-through** (el código se desborda), que puede ser un bug o un truco para agrupar casos.
3. Usa `switch` para valores exactos y `if`/`else if` para **rangos** y condiciones combinadas.

> 🐛 **Vocabulario rápido**

Término	Idea general
`switch`	Estructura que elige entre varios `case` según un valor
`case`	Cada opción concreta a comparar
`break`	Orden de salida: corta el switch
`default`	El “ninguno de los anteriores”, opcional
Fall-through	Que el código se desborde de un `case` al siguiente

📚 Volver al índice de la unidad · **Anterior:** 01 · if, else if y else · **Siguiente:** 03 · Bucles: while y do-while

03 — Bucles: while y do-while

Repetir sin copiar y pegar, hasta que una condición diga basta 🏃

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 03 · Bucles: while y do-while

---

📬 La idea en una frase

> **Un bucle es una cinta de correr: ejecuta el mismo bloque una y otra vez mientras la condición sea `true`.**

¿Te imaginas escribir "imprime del 1 al 100" con cien `println`? Copiar y pegar es pecado. Los bucles hacen el trabajo sucio por ti: repiten un bloque hasta que les dices basta.

🏃 while: comprueba y luego corre

```
while (condicion) {
    // bloque que se repite
}
```

El `while` primero **mira la condición** y, si es `true`, ejecuta el bloque. Al terminar, vuelve a mirar. Si es `false` desde el principio… **no ejecuta nada**.

```
int intentos = 3;

while (intentos > 0) {
    System.out.println("Reintentando... quedan " + intentos);
    intentos = intentos - 1;
}
```

Salida:

```
Reintentando... quedan 3
Reintentando... quedan 2
Reintentando... quedan 1
```

> ⚠️ **Advertencia:** si olvidas la línea que modifica la condición (`intentos = intentos - 1;`), la condición es `true` para siempre y tu programa **nunca termina**. Bienvenido al bucle infinito, el coche sin frenos de la programación.

♾️ El bucle infinito (y cómo salir)

```
while (true) {
    System.out.println("Socorro");
}
```

Este programa imprimiría “Socorro” hasta que el universo se congele. En el IDE, el botón de parar (🟥) es tu mejor amigo. ¿Por qué existe `while (true)`? Porque a veces quieres un bucle “para siempre” que se rompa en el interior con `break` (ya lo verás en el punto 5).

> 💡 **Detalle práctico:** la sentencia centinela. Un clásico es leer datos hasta que el usuario escriba “salir”:

```
String respuesta = "";
Scanner sc = new Scanner(System.in);

while (!respuesta.equals("salir")) {
    System.out.print("Dime algo (o 'salir'): ");
    respuesta = sc.nextLine();
}
System.out.println("Adiós.");
```

🏃 do-while: corre y luego comprueba

```
do {
    // bloque
} while (condicion);
```

La diferencia con `while` es el **orden**: el `do-while` ejecuta el bloque **al menos una vez** y comprueba la condición al final. Útil cuando necesitas preguntar sí o sí antes de decidir:

```java
int opcion;
Scanner sc = new Scanner(System.in);

do {
    System.out.println("1. Jugar  2. Salir");
    System.out.print("Elige: ");
    opcion = sc.nextInt();
} while (opcion != 1 && opcion != 2);

System.out.println("Has elegido la opción " + opcion);
```

Aquí el menú se muestra **siempre al menos una vez**, y se repite mientras el usuario no elija 1 o 2. Perfecto para menús.

> ⚠️ **Advertencia:** no confundas los dos. Con `while`, si la condición es `false` de entrada, **cero ejecuciones**. Con `do-while`, **al menos una**. Es como la diferencia entre "mira antes de cruzar" y "cruza y luego mira".

VS while vs do-while: el duelo rápido

Situación	Bucle ideal
No sabes si tocará ejecutar el bloque	`while`
El bloque debe ejecutarse sí o sí una vez	`do-while`
Leer datos hasta que el usuario dé el centinela	`while`
Mostrar un menú hasta que elija una opción válida	`do-while`

🏫 Ejemplo guiado: la cuenta atrás

Vamos a lanzar un cohete con una cuenta atrás. Como debe imprimirse el "¡Despegue!" aunque el contador empiece en 0… usamos `do-while`:

```java
public class CuentaAtras {
    public static void main(String[] args) {
        int contador = 5;

        do {
            System.out.println(contador);
            contador--;
        } while (contador >= 0);

        System.out.println("¡Despegue! 🚀");
    }
}
```

Salida:

```
5
4
3
2
1
0
¡Despegue! 🚀
```

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando veas un bucle, pregúntate: *¿la condición avanza hacia `false` en algún momento?* Si la respuesta es “no”, tienes un bucle infinito.

Ejercicio: el contador parado

Sin ejecutar, calcula cuántas veces imprime “Hola” este programa… o si se cuelga:

```java
public class Contador {
    public static void main(String[] args) {
        int x = 10;

        while (x > 0) {
            System.out.println("Hola");
            x = x + 1;
        }
    }
}
```

> 🔄 Solución

🎯 Mini-chequeo

1. ¿Cuántas veces se ejecuta el bloque de un `while` si la condición es `false` desde el principio?
2. ¿Y en un `do-while`?
3. ¿Qué es un bucle infinito y cómo se sale de él en el IDE?
4. ¿Cuándo usarías un `do-while` para un menú?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. `while` comprueba la condición **antes** de ejecutar y `do-while` **después**: el segundo garantiza al menos una ejecución.
2. Un bucle necesita que la condición **avance hacia `false`**; si no, tienes un bucle infinito.
3. Usa `while` para leer hasta un centinela y `do-while` para menús que deben mostrarse sí o sí.

> 🐛 Vocabulario rápido

Término	Idea general
Bucle	Bloque que se repite mientras una condición sea `true`

Término	Idea general
Iteración	Una vuelta completa del bucle
Condición	La expresión booleana que decide si se sigue
Centinela	Valor especial que termina la lectura (“salir”)
Bucle infinito	Bucle que nunca acaba por descuido
do-while	Bucle que ejecuta al menos una vez y comprueba al final

Volver al índice de la unidad · **Anterior:** 02 · switch · **Siguiente:** 04 · Bucle for

04 — Bucle for y bucles anidados

El contador obsesivo que repite un número exacto de vueltas 🔢

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 04 · Bucle for y bucles anidados

---

📬 La idea en una frase

> **`for` es un bucle con contador de serie: declara la variable, pone la condición y la actualiza en la misma línea, ideal para "repite N veces".**

El `while` repetía "mientras pase algo". El `for` repite "un número exacto de veces". Es el bucle favorito para recorrer cosas y el que más usarás en toda tu carrera.

🔢 La anatomía del for

```
for (inicializacion; condicion; actualizacion) {
    // bloque
}
```

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Vuelta " + i);
}
```

Salida:

```
Vuelta 1
Vuelta 2
Vuelta 3
Vuelta 4
Vuelta 5
```

El ciclo de vida de `i`:

1. **Inicialización**: `int i = 1` — se ejecuta una sola vez, al entrar.
2. **Condición**: `i <= 5` — se comprueba antes de cada vuelta.

3. **Bloque**: se ejecuta si la condición es `true`.
4. **Actualización**: `i++` — se ejecuta al final de cada vuelta.
5. Vuelta al paso 2.

> ⚠️ **Advertencia:** una comilla mal puesta en el `for` es el error de examen más común. La sintaxis es: inicialización `;` condición `;` actualización. Tres partes, dos puntos y coma, cero comas entre ellas.

🔁 Los tres bucles son el mismo chiste

Esto es `while`, `do-while` y `for` haciendo exactamente lo mismo:

```
int i = 1;                          // inicialización
while (i <= 5) {                     // condición
    System.out.println("Vuelta " + i);
    i++;                             // actualización
}
```

```
int i = 1;
do {
    System.out.println("Vuelta " + i);
    i++;
} while (i <= 5);
```

```
for (int i = 1; i <= 5; i++) {
    System.out.println("Vuelta " + i);
}
```

Los tres imprimen lo mismo. El `for` gana porque junta las tres partes del control en una línea: es más difícil olvidar el `i++` (adiós, bucles infinitos por descuido).

> 💡 **Detalle práctico:** si sabes cuántas veces vas a repetir → `for`. Si no lo sabes → `while`. Regla que te salvará la vida en el examen.

🧩 Bucles anidados: la parrilla de ejercicios

Un bucle dentro de otro. El bucle **exterior** se ejecuta una vez por cada vuelta del… no, al revés: por cada vuelta del exterior, el **interior** se ejecuta completo.

```
for (int fila = 1; fila <= 3; fila++) {
    for (int columna = 1; columna <= 4; columna++) {
        System.out.print("* ");
    }
    System.out.println();
}
```

Salida:

```
* * * *
* * * *
* * * *
```

- El bucle exterior controla las **filas** (3).
- El interior controla las **columnas** (4).
- El `println()` sin texto después del interior salta de línea al acabar cada fila.

> ⚠️ **Advertencia:** cada nivel de anidamiento multiplica las vueltas. Con 1000 filas y 1000 columnas son 1.000.000 de iteraciones. Los bucles anidados son potentes, pero también la fábrica de programas lentos.

🏫 Ejemplo guiado: la tabla de multiplicar

```
public class TablaMultiplicar {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            System.out.println("7 x " + i + " = " + (7 * i));
        }
    }
}
```

Salida (primeras líneas):

```
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
```

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** cuando uses un `for`, pregunta: *¿la condición usa `<` o `<=`?* Un despiste de un dígito (repetir 9 veces en vez de 10) es el bug del off-by-one, el más famoso de la historia.

Ejercicio: el triangulero

Sin ejecutar, calcula cuántos asteriscos imprime en total este programa:

```java
public class Triangulo {
    public static void main(String[] args) {
        for (int fila = 1; fila <= 4; fila++) {
            for (int ast = 1; ast <= fila; ast++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

> 🔄 **Solución**

🎯 Mini-chequeo

1. ¿Cuáles son las tres partes del `for`?
2. ¿Cuántas veces se ejecuta la inicialización?
3. ¿Qué imprime `for (int i = 0; i < 5; i++)`? ¿5 o 4 vueltas?
4. En un bucle anidado, ¿qué hace el bucle interior por cada vuelta del exterior?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. `for` junta **inicialización, condición y actualización** en una línea: ideal para repetir un número conocido de veces.
2. `while`, `do-while` y `for` son intercambiables; elige `for` cuando sepas las vueltas.
3. En los **bucles anidados**, por cada vuelta del exterior el interior se ejecuta completo, y eso multiplica el trabajo.

> 🦎 **Vocabulario rápido**
>
Término	Idea general
`for`	Bucle con contador: `for (inicio; condicion; avance)`
Contador	Variable que cuenta las vueltas (normalmente `i`)
Bucle anidado	Un bucle dentro de otro
Off-by-one	Fallar por uno en la condición (< vs <=)
Iteración	Una vuelta del bucle

📚 Volver al índice de la unidad · **Anterior:** 03 · Bucles: while y do-while · **Siguiente:** 05 · break y continue

05 — break, continue y etiquetas

Los botones de emergencia y los saltos de línea del bucle 🚪

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 05 · break, continue y etiquetas

---

📬 La idea en una frase

> **`break` apaga el bucle entero y `continue` se salta solo la vuelta actual; con etiquetas puedes decidir a qué bucle anidado afectan.**

Ya sabes repetir. Ahora toca aprender a **salir con estilo**: interrumpir, saltar y dirigirte a un bucle concreto cuando hay varios.

🚪 break: el botón de parada

`break` termina el bucle **inmediatamente**, sin comprobar la condición:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) {
        break;
    }
    System.out.println(i);
}
```

Salida:

```
1
2
3
4
```

En cuanto `i` vale 5, `break` corta el bucle: las vueltas 6 a 10 nunca ocurren. Es perfecto para "encuentra algo y para de buscar".

> 💡 **Detalle práctico:** el `break` dentro de un `switch` (punto 2) cortaba el switch. El `break` dentro de un bucle corta el bucle. Mismo botón, diferente aparato.

⏭️ continue: el botón de saltar

`continue` no termina el bucle: **salta a la siguiente vuelta** directamente, ignorando el resto del bloque:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }
    System.out.println(i);
}
```

Salida:

```
1
2
4
5
```

El 3 se salta, pero el bucle sigue. Útil para "no proceses estos valores, pero sigue con los demás":

```
// Suma solo los números pares del 1 al 10
int suma = 0;
for (int i = 1; i <= 10; i++) {
    if (i % 2 != 0) {
        continue;      // impares: no cuentan
    }
    suma += i;
}
System.out.println("Suma de pares: " + suma);   // 2+4+6+8+10 = 30
```

> ⚠️ **Advertencia:** en un `while`, si pones el `continue` **antes** de actualizar la variable del bucle, la actualización se salta… y el bucle no avanza. Bug infinito asegurado. En un `for` la actualización está en la cabecera y no pasa nada.

🏷️ Etiquetas: el GPS de los bucles anidados

Un `break` o `continue` sueltos afectan **solo al bucle más interno**. ¿Y si quieres salir de dos bucles a la vez? Ahí nacen las **etiquetas**:

```
exterior:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i * j >= 6) {
            break exterior;    // sale de AMBOS bucles
        }
        System.out.println(i + " x " + j);
    }
}
```

Salida:

```
1 x 1
1 x 2
1 x 3
2 x 1
2 x 2
```

Cuando `i * j >= 6`, el `break exterior` salta fuera de la etiqueta, terminando los dos bucles a la vez. Sin etiqueta, el `break` solo habría salido del bucle de `j`.

```
exterior:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (j == 2) {
            continue exterior;   // salta a la siguiente i
        }
        System.out.println(i + "-" + j);
    }
}
```

Salida:

```
1-1
2-1
3-1
```

> ⚠️ **Advertencia:** las etiquetas son legales pero poco usadas. Ante la duda, casi siempre se puede rediseñar con una variable booleana. Usa etiquetas con moderación: tu compañero de proyecto te lo agradecerá.

🏫 Ejemplo guiado: el detector de primer número

Usemos `break` para comprobar si un número es primo de forma eficiente:

```
public class EsPrimo {
    public static void main(String[] args) {
        int numero = 29;
        boolean esPrimo = true;

        for (int divisor = 2; divisor < numero; divisor++) {
            if (numero % divisor == 0) {
                esPrimo = false;
                break;   // encontrado divisor: para de buscar
            }
        }

        System.out.println(numero + " ¿es primo? " + esPrimo);
    }
}
```

Salida:

```
29 ¿es primo? true
```

Con `break`, en cuanto aparece un divisor dejamos de comprobar. Para el 29 no hay divisores, así que el bucle se recorre entero y `esPrimo` sigue siendo `true`.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** para distinguirlos de un vistazo: `break` = **apaga** el bucle; `continue` = **salta** esta vuelta. Uno termina la fiesta, el otro solo se pierde una canción.

Ejercicio: el bucle esquizofrénico

Sin ejecutar, escribe la salida exacta:

```
public class Esquizofrenia {
    public static void main(String[] args) {
        for (int i = 1; i <= 8; i++) {
            if (i % 3 == 0) {
                continue;
            }
            if (i == 7) {
                break;
            }
            System.out.println(i);
        }
    }
}
```

> 🔄 **Solución**

🎯 Mini-chequeo

1. ¿Cuál es la diferencia entre `break` y `continue` en una frase?
2. ¿A qué afectan por defecto en bucles anidados?
3. ¿Para qué sirve una etiqueta?
4. ¿Por qué es peligroso el `continue` en un `while` si va antes de la actualización?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. `break` apaga el bucle entero y `continue` se salta solo la vuelta actual.
2. En bucles anidados afectan al más interno; las **etiquetas** te dejan apuntar a un bucle exterior.
3. Cuidado con `continue` antes de la actualización en `while`: es un bucle infinito en ciernes.

> 🦎 **Vocabulario rápido**

Término	Idea general
`break`	Termina el bucle (o el switch) inmediatamente
`continue`	Salta a la siguiente vuelta del bucle
Etiqueta	Nombre que pones a un bucle para saltar a él
`break etiqueta`	Sale del bucle etiquetado, no del más interno
Bucle infinito	Riesgo si `continue` se salta la actualización de un `while`

Volver al índice de la unidad · **Anterior:** 04 · Bucle for · **Siguiente:** 06 · Excepciones básicas

06 — Excepciones básicas

Cuando Java se queja por escrito: la familia Throwable y las quejas más típicas 💥

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 06 · Excepciones básicas

---

📬 La idea en una frase

> **Una excepción es un aviso de que algo salió mal; Java lo lanza como un objeto que hereda de `Throwable`, y tu programa puede o no estar preparado para atraparlo.**

¿Te ha pasado que un programa se "cae" con un montón de texto rojo? Ese texto es una excepción. En vez de morir en silencio, Java grita con todo el detalle. Aprender a leer esos gritos es aprender a depurar.

💥 El crash: tu primera excepción

Ejecuta esto:

```
public class Explosion {
    public static void main(String[] args) {
        int[] numeros = {1, 2, 3};
        System.out.println(numeros[5]);   // ¡no existe!
    }
}
```

Java no se calla. Aparece algo así:

```
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: Index 5 out of
bounds for length 3
  at Explosion.main(Explosion.java:5)
```

Este texto es oro: te dice **qué** excepción (`ArrayIndexOutOfBoundsException`), **dónde** (línea 5) y en **qué método**. Leerlo bien resuelve la mitad de tus problemas.

🌳 La familia Throwable: el árbol genealógico

Todas las excepciones heredan de una clase madre:

```
Object
 └── Throwable
      ├── Error                  (cosas que no deberías intentar arreglar)
      └── Exception              (lo que de verdad nos interesa)
           ├── RuntimeException  (excepciones en tiempo de ejecución)
           └── (otras excepciones "controladas")
```

- **`Error`**: problemas graves de la JVM (memoria agotada, por ejemplo). No los provocas tú y no debes intentar atraparlos. Ignóralos.
- **`Exception`**: fallos del programa. Aquí vive el 99% de tu vida.
- **`RuntimeException`**: subfamilia de `Exception` que se lanza en **tiempo de ejecución** y que **no estás obligado** a capturar. Aquí viven las más famosas.

```java
// Todas estas son RuntimeException (no necesitas try para que compile):
int x = 10 / 0;                          // ArithmeticException
int[] a = new int[3]; a[9] = 1;          // ArrayIndexOutOfBoundsException
String s = null; s.length();             // NullPointerException
int num = Integer.parseInt("Hola");      // NumberFormatException
```

> 💡 **Detalle práctico:** "RuntimeException" significa que el error aparece cuando el programa **se ejecuta**, no al compilar. El compilador no te avisa: solo lo descubres en plena ejecución.

🗺️ Las excepciones más comunes: la guía de campo

Excepción	Cuándo aparece	Frase típica
`ArithmeticException`	Dividir entre 0	"Dividir entre cero, qué valiente"
`ArrayIndexOutOfBoundsException`	Índice fuera del array	"Ese hueco no existe"
`NullPointerException`	Llamar a algo `null`	"El clásico absoluto"

Excepción	Cuándo aparece	Frase típica
`NumberFormatException`	Convertir texto que no es número	“ParSEAR ‘Hola’ como número, no”
`StringIndexOutOfBoundsException`	Índice fuera de un `String`	“substring() más allá del final”
`InputMismatchException`	`Scanner` recibe el tipo equivocado	“Pusiste texto donde iba un número”

> ⚠️ **Advertencia:** la `NullPointerException` (NPE) es, con diferencia, la excepción más común de la historia de Java. Tu abuela, si programara, también la tendría. El mensaje suele ser un críptico “null” seguido de la línea donde tocaste un objeto que no existía.

🏫 Ejemplo guiado: el lector de edades

Un programa que pide una edad por teclado puede explotar si el usuario escribe letras. Veamos el crash y luego lo arreglaremos en el punto 7:

```
import java.util.Scanner;

public class LectorEdad {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("¿Cuántos años tienes? ");
        int edad = sc.nextInt();
        System.out.println("Naciste hace " + edad + " años.");
        sc.close();
    }
}
```

Si escribes `hola`, obtienes un `InputMismatchException`. El programa muere. La solución es atrapar la excepción con `try/catch`… que es justo el punto 7.

⭐ Sé el Código, my friend…

> 👓 **Don Tip:** cuando un programa explote, lee la primera línea del error: el nombre de la excepción te dice *qué* pasó, y la línea con `at ...` te dice *dónde*. Es un GPS con acusaciones.

Ejercicio: el detective de excepciones

Di qué excepción lanzaría cada línea (o si no lanzaría ninguna):

```
int a = 5 / 0;
String[] dias = {"L", "M", "X"};
System.out.println(dias[3]);
String texto = null;
System.out.println(texto.toUpperCase());
int b = Integer.parseInt("42");
int c = Integer.parseInt("cuarenta y dos");
```

> 🔄 Solución

🎯 Mini-chequeo

1. ¿Qué clase está en la raíz de todas las excepciones?
2. ¿Qué diferencia hay entre `Error` y `Exception`?
3. ¿Qué significa que sea una `RuntimeException`?
4. ¿Cuál es la excepción más famosa de la historia y cuándo aparece?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. Una excepción es un **objeto** que Java lanza cuando algo sale mal y que hereda de `Throwable`.
2. La familia se divide en `Error` (graves, no tocar), `Exception` (capturables) y `RuntimeException` (se lanzan al ejecutar, sin obligación de capturarlas).
3. Leer el mensaje de la excepción (qué, dónde, en qué método) es la primera habilidad de un buen depurador.

> 🐛 Vocabulario rápido

Término	Idea general
Excepción	Objeto que representa un error en el programa
`Throwable`	Clase raíz de errores y excepciones
`Error`	Fallo grave de la JVM, no capturable
`Exception`	Fallo del programa, capturable
`RuntimeException`	Excepción de ejecución, sin captura obligatoria
Trace	El “stack trace”: la lista de llamadas hasta el fallo

📚 Volver al índice de la unidad · **Anterior:** 05 · break y continue · **Siguiente:** 07 · try, catch y finally

07 — try, catch y finally

El airbag del código: que un error no destroce tu programa 🛡️

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 07 · try, catch y finally

📬 La idea en una frase

> **`try` protege el código peligroso, `catch` atrapa la excepción si aparece y `finally` se ejecuta siempre, ocurra lo que ocurra.**

En el punto 6 viste que un `InputMismatchException` mata tu programa. Ahora toca blindarlo: el `try/catch/finally` es el airbag del código.

🛡️ La estructura completa

```
try {
    // código peligroso
} catch (TipoDeExcepcion e) {
    // qué hacer si aparece esa excepción
} finally {
    // se ejecuta SIEMPRE, con o sin excepción
}
```

```java
import java.util.InputMismatchException;
import java.util.Scanner;

public class LectorBlindado {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("¿Cuántos años tienes? ");
            int edad = sc.nextInt();
            System.out.println("Naciste hace " + edad + " años.");
        } catch (InputMismatchException e) {
            System.out.println("Eso no es un número. No me hagas esto.");
        }

        System.out.println("El programa sigue vivo. 🎉");
        sc.close();
    }
}
```

Si escribes `hola`, ya no explota: el `catch` atrapa el error, imprime un mensaje simpático y **el programa continúa**.

> 💡 **Detalle práctico:** el `finally` es opcional y suele usarse para limpiar recursos (cerrar `Scanner`, ficheros…). Se ejecuta **siempre**: si hubo excepción, si no la hubo, e incluso si el `try` tenía un `return`.

🎯 Captura múltiple: varios catch en fila

Puedes capturar varios tipos de excepción, cada uno con su tratamiento. **El orden importa: primero las más específicas, después las generales.**

```
try {
    int[] numeros = {1, 2};
    int indice = 5;
    int divisor = 0;
    System.out.println(numeros[indice] / divisor);
} catch (ArithmeticException e) {
    System.out.println("División entre cero.");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índice fuera del array.");
} catch (RuntimeException e) {
    System.out.println("Algo raro pasó en tiempo de ejecución.");
}
```

En Java 7+ existe una forma compacta para varios tipos con el mismo tratamiento, separados por |:

```
} catch (ArithmeticException | ArrayIndexOutOfBoundsException e) {
    System.out.println("Matemática o índice: fallaste por aquí.");
}
```

> ⚠️ **Advertencia:** un `catch (Exception e)` al principio se comería las excepciones más específicas. Regla: de lo más concreto a lo más general, como en los `else if`.

🧽 La variable e: el botín del error

El `e` del `catch` es el objeto excepción atrapado. Puedes preguntarle cosas:

```
catch (Exception e) {
    System.out.println("Mensaje: " + e.getMessage());
    e.printStackTrace();   // imprime el stack trace completo (para depurar)
}
```

> ⚠️ **Advertencia:** un `catch` **vacío** (sin nada dentro) es un pecado mortal: te tragas el error y no te das ni cuenta de que pasó. Como un testigo que no habla en un juicio. Como mínimo, imprime un mensaje.

🏫 Ejemplo guiado: el menú a prueba de bombas

Reunamos `do-while`, `try` y `catch`: un menú que repite hasta elegir bien y que no explota si escribes basura:

```
import java.util.InputMismatchException;
import java.util.Scanner;

public class MenuBlindado {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int opcion = 0;
        boolean valida = false;

        do {
            System.out.println("1. Jugar  2. Salir");
            System.out.print("Elige: ");

            try {
                opcion = sc.nextInt();
                valida = (opcion == 1 || opcion == 2);
                if (!valida) {
                    System.out.println("Opción no válida, inténtalo otra vez.");
                }
            } catch (InputMismatchException e) {
                System.out.println("Eso no es un número.");
                sc.next();   // descarta el texto basura del buffer
            }
        } while (!valida);

        System.out.println("Has elegido la opción " + opcion);
        sc.close();
    }
}
```

> 💡 **Detalle práctico:** fíjate en `sc.next()` dentro del `catch`: sin él, el texto basura seguiría en el buffer del `Scanner` y el siguiente `nextInt()` fallaría otra vez. Atrapar la excepción y **limpiar el buffer** son dos pasos del mismo baile.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** la pregunta clave ante un `try` es: *¿qué puede fallar aquí y cómo lo manejo?* Un `catch` que solo existe "por si acaso" pero no hace nada es humo.

Ejercicio: el detective del orden

Este código tiene un problema de orden en los `catch`. Sin ejecutar, explica qué ocurre:

```
try {
    int[] numeros = {10, 20};
    System.out.println(numeros[3]);
} catch (Exception e) {
    System.out.println("Atrapado por Exception.");
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Atrapado por ArrayIndexOutOfBoundsException.");
}
```

> › 🔄 **Solución**

🎯 Mini-chequeo

1. ¿Qué hace el bloque `finally` y cuándo se ejecuta?
2. ¿En qué orden deben ir los `catch`?
3. ¿Qué peligro tiene un `catch` vacío?
4. ¿Por qué conviene llamar a `sc.next()` tras un `InputMismatchException`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. `try` protege el código peligroso y `catch` atrapa la excepción para que el programa **no muera**.
2. Los `catch` van **del más específico al más general**, y nunca dejes un `catch` vacío.
3. `finally` se ejecuta siempre, ideal para cerrar `Scanner`, ficheros y demás recursos.

> 🦎 **Vocabulario rápido**

Término	Idea general
`try`	Bloque con código que puede lanzar excepciones
`catch`	Bloque que atrapa una excepción y la maneja
`finally`	Bloque que se ejecuta siempre
Captura múltiple	Varios `catch` seguidos, del específico al general
`e.getMessage()`	Mensaje del error atrapado
Stack trace	Rastro de llamadas donde ocurrió el fallo

📚 Volver al índice de la unidad · **Anterior:** 06 · Excepciones básicas · **Siguiente:** 08 · throw y excepciones propias

08 — throw y excepciones propias

Cuando te toca a ti lanzar la piedra y crear tu propio defecto 🫐

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 08 · throw y excepciones propias

---

📬 La idea en una frase

> **`throw` lanza una excepción cuando tú decides que algo no debe continuar, y creando tu propia excepción puedes ponerle el nombre que quieras al problema.**

Hasta ahora Java lanzaba las excepciones por ti. Pero hay un superpoder mejor: **tú** decides cuándo lanzarlas, y puedes inventarte tipos de error a tu medida.

🫐 throw: lanza la piedra

`throw` crea y lanza una excepción donde quieras. Es como decir "aquí algo está mal, que lo sepa todo el mundo":

```
public class CuentaBancaria {
    public static void main(String[] args) {
        double saldo = 10.0;
        double retiro = 500.0;

        if (retiro > saldo) {
            throw new ArithmeticException("Saldo insuficiente: " + saldo);
        }

        saldo -= retiro;
        System.out.println("Nuevo saldo: " + saldo);
    }
}
```

Ese `throw new ArithmeticException("...")` detiene el programa con esa excepción. Y el mensaje que le pasas al constructor es lo que verás en `e.getMessage()`.

> 💡 **Detalle práctico:** `throw` y `throws` no son primos: son trillizos distintos. `throw` **lanza** una excepción (lo ves aquí). `throws` **anuncia** en la firma del método que puede lanzar excepciones controladas. `throw` va en el cuerpo; `throws`, en la cabecera.

🪜 Excepciones propias: tu defecto a medida

¿Por qué conformarte con `ArithmeticException` cuando puedes tener una `SaldoInsuficienteException` con nombre de película? Crear tu propia excepción es **heredar de `Exception`** (o de `RuntimeException`) y listo:

```
public class SaldoInsuficienteException extends RuntimeException {
    public SaldoInsuficienteException(String mensaje) {
        super(mensaje);
    }
}
```

Y ahora la usas:

```
public class Cajero {
    public static void main(String[] args) {
        double saldo = 10.0;
        double retiro = 500.0;

        if (retiro > saldo) {
            throw new SaldoInsuficienteException("Solo tienes " + saldo + "€.");
        }

        System.out.println("Retirado: " + retiro);
    }
}
```

Salida (con el programa cortándose):

```
Exception in thread "main" SaldoInsuficienteException: Solo tienes 10.0€.
  at Cajero.main(Cajero.java:9)
```

> 💡 **Detalle práctico:** hereda de `Exception` si quieres **obligar** a los que la usen a capturarla (excepción controlada). Hereda de `RuntimeException` si prefieres que no les obligue (como

> las que viste en el punto 6). Para empezar, `RuntimeException` es más cómoda.

⚖️ checked vs unchecked: la burocracia de las excepciones

- **Checked (controladas)**: el compilador **te obliga** a capturarlas o a declararlas con `throws`. Heredan de `Exception` pero no de `RuntimeException`. Ejemplo: `IOException`.
- **Unchecked (no controladas)**: no te obligan a nada. Son `RuntimeException` y sus hijas.

```
import java.io.IOException;

public class Buho {
    public static void main(String[] args) throws IOException {
        // como IOException es checked, DEBE ir en el throws
        // o estar dentro de un try/catch
    }
}
```

Si una excepción es checked y no la gestionas, **no compila**. Si es unchecked, el compilador te deja tranquilo (y el error explota en ejecución).

🏫 Ejemplo guiado: la máquina expendedora

Vamos a crear una excepción propia y un programa que la lance y la atrape:

```
public class ProductoAgotadoException extends RuntimeException {
    public ProductoAgotadoException(String producto) {
        super("El producto " + producto + " está agotado.");
    }
}
```

```
public class MaquinaExpendedora {
    public static void main(String[] args) {
        int stock = 0;
        String producto = "Refresco";

        try {
            if (stock == 0) {
                throw new ProductoAgotadoException(producto);
            }
            System.out.println("Aquí tienes tu " + producto);
        } catch (ProductoAgotadoException e) {
            System.out.println("Lo sentimos: " + e.getMessage());
        }

        System.out.println("La máquina sigue funcionando. 🤖");
    }
}
```

Salida:

```
Lo sentimos: El producto Refresco está agotado.
La máquina sigue funcionando. 🤖
```

¿Ves la magia? El `throw` lanza tu excepción, el `catch` la atrapa por su **nombre propio** y el programa sobrevive. Ese nombre convierte un error genérico en un mensaje que hasta tu jefa entiende.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** si una condición "imposible" ocurre en tu código, es mejor `throw` que dejar que el programa siga con datos rotos. Una excepción temprana vale más que un bug que aparece dos semanas después.

Ejercicio: el controlador de notas

Crea (en papel o en el IDE) una excepción propia `NotaInvalidaException` que herede de `RuntimeException`, y un método que la lance si una nota no está entre 0 y 10. ¿Qué hereda tu excepción de su padre?

> 🔄 Solución

🎯 Mini-chequeo

1. ¿Qué hace la palabra clave `throw`?
2. ¿Cómo se crea una excepción propia?
3. ¿Cuál es la diferencia entre `throw` y `throws`?
4. ¿Qué diferencia hay entre checked y unchecked?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. `throw` lanza una excepción donde tú decides, con el mensaje que quieras.
2. Crear tu propia excepción es **heredar de `Exception` o `RuntimeException`** y ponerle un constructor con mensaje.
3. `throw` (lanzar) ≠ `throws` (declarar), y las excepciones **checked** obligan a gestionarlas mientras las **unchecked** no.

> 🐛 Vocabulario rápido

Término	Idea general
`throw`	Lanza una excepción: `throw new MiExcepcion()`
`throws`	Declara en la firma que el método puede lanzar algo
Excepción propia	Clase que hereda de `Exception` o `RuntimeException`
Checked	Excepción que el compilador te obliga a gestionar
Unchecked	Excepción sin obligación de captura (`RuntimeException`)
`super(mensaje)`	Pasa el mensaje a la clase padre

📚 Volver al índice de la unidad · **Anterior:** 07 · try, catch y finally · **Siguiente:** 09 · Repaso interactivo

09 — Repaso interactivo: controla las estructuras

El cierre práctico de la unidad, con decisiones, bucles y una excepción que no va a funcionar 😈

> 🗺️ **Estás en:** 🔀 **U04 · Estructuras de control y excepciones básicas** → 09 · Repaso interactivo

---

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```
public class Misterio {
    public static void main(String[] args) {
        int nota = 6;

        if (nota >= 5) {
            System.out.println("Aprobado");
        } else if (nota >= 7) {
            System.out.println("Notable");
        } else {
            System.out.println("Suspenso");
        }
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **`Notable`** → La nota 6 es mayor que 5, y también casi mayor que 7, así que Java elige la mejor. ❌
2. **`Aprobado`** → ✅ ¡Correcto! Java evalúa de arriba abajo y se queda con la **primera** condición que da `true`. Como `nota >= 5` se cumple, entra ahí y se olvida del resto, aunque 6 no

llegue a 7.

3. **Suspenso** → El `else` solo se ejecuta si ninguna condición anterior se cumple, y aquí sí se cumple la primera. ❌

> 🔄 **Solución**

🔥 Fireside Chat: if-else vs switch

> *Dos veteranos del control de flujo discuten junto a la máquina de café.*

if-else: — Yo soy el clásico. Condiciones, rangos, comparaciones… ¿necesitas decidir si algo es mayor que 5 o está entre 10 y 20? Llámame a mí. Yo comparo lo que sea.

switch: — Claro, y te llenas de `else if` hasta que el código parece la escalera de un edificio. Conmigo pones la variable una vez y cada caso en su línea. Limpio, directo, elegante.

if-else: — Elegante hasta que te olvidas un `break` y tu switch se convierte en un tobogán. ¿Sabes lo que es el fall-through? Una pesadilla con nombre.

switch: — El fall-through se usa a propósito cuando quiero agrupar casos. ¿Y tú? Con treinta `else if`, ¿sabes siquiera cuál va antes que cuál?

if-else: — ¡Yo soporto rangos! `>= 18`, `< 65`… Tú solo sirves para valores exactos. Un día tienes que decidir por edad y vas a llorar.

switch: — Mejor llorar que repetir una variable veinte veces. Cada uno en su terreno, ¿no?

if-else: — Trato hecho. Tú, valores exactos. Yo, rangos y condiciones combinadas. Así nadie se hace daño.

> La lección: no hay ganador. **`switch` para valores concretos** (día, menú, talla) y **`if/else if` para rangos y mezclas**. Elegir bien es la mitad del examen.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy el semáforo: si mi condición es `true`, dejo pasar; si es `false`, redirijo al otro carril.**
2. **Soy el menú del restaurante: miras mi variable y ejecutas el `case` que coincida.**

3. **Soy la cinta de correr que comprueba antes de correr: si la condición es `false` de entrada, no doy ni un paso.**
4. **Soy el botón de parada: corto el bucle entero en cuanto aparezco.**
5. **Soy el abuelo de todos los errores: todo lo que se lanza hereda de mí.**
6. **Soy el airbag: atrapo el error para que el programa no muera.**

> 🔁 **Respuestas**

🤬 CONRAD VS EL MUNDO: “El bucle que no acaba”

> *CONRAD, nuestro compilador cascarrabias, opina sobre el clásico del novato.*

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: *CONRAD, mi programa se queda colgado*. Y yo: vale, ¿qué tiene el bucle? *Pues no lo sé, no lo he mirado.* ¡AY, MADRE MÍA! Un `while` sin nada que cambie la condición dentro es un coche sin frenos, ¿te lo explico con plastilina?

Y luego está el que escribe `while (x > 0) { x = x + 1; }` *cuando quería restar. Sube en vez de bajar. No es un bucle infinito, es un bucle que asciende hasta el infinito. Como si quisieras vaciar una piscina echándole más agua.*

Y el colmo: `if (x = 5)`. Con UN igual. ¡Eso no es una condición, es una asignación! Te lo digo desde la U03 y sigo viéndolo. El doble igual `==` se queda en casa cuando toca comparar.

La lección: antes de acusar al ordenador de “congelarse”, mira el bucle: ¿algo modifica la condición hacia `false`? ¿El `continue` se salta la actualización? ¿Usas `==` o te has quedado en `=`? El 90% de los programas “colgados” se arreglan con un vistazo a estas tres preguntas.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. ¿Qué imprime `int n = 4; String r = n >= 5 ? "A" : "B";`?
   - a) `A` b) `B`
2. ¿Cuántas vueltas da `for (int i = 0; i < 3; i++)`?
   - a) 3 b) 4

3. ¿Qué imprime `switch` con `case 1` y `case 2` seguidos sin `break` entre ellos, si la variable vale 1?
    - a) Solo el `case 1` b) El `case 1` y luego el `case 2`
4. ¿Cuál es el resultado de `10 / 0`?
    - a) `ArithmeticException` b) Un número enorme

> 🔄 **Soluciones**

⚡ Laboratorio de tortura: la máquina que llora

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto cuando lo arregles.

```
public class Tortura
    public static void main(string[] args) {
        int suma = 0;
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) continue;
            suma = suma - i;
        }
        System.out.println("La suma de los impares es: " + suma)
        System.out.println("El número de impares es: " + 5);
    }
}
```

Fallo intencionado: uno de los errores parece correcto a simple vista porque “se ve bien”, pero hace que el programa cuente mal.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún `;` que falte? *no → sigue buscando.*
    > **¿Y si sigo atascado?**
2. ¿Compila ya? *no → mira el mensaje de error y las mayúsculas.*
    > **¿Y si sigo atascado?**

3. ¿Ejecuta pero la suma sale rara? *Es el error de lógica: el signo.*

> **Solución final**

Los **3 errores de compilación**:

1. Falta la `{` que abre el cuerpo de la clase después de `Tortura`.
2. `string[] args` → `String[] args` (la clase `String` con mayúscula).
3. Falta el `;` al final de `System.out.println("La suma de los impares es: " + suma)`.

El **error de lógica**: `suma = suma - i;` resta en vez de sumar. Compila perfectamente, pero con los impares 1, 3, 5, 7 y 9 la suma sale **-25** en vez de 25. Un signo es lo único que separa a tu programa de la verdad.

```
public class Tortura {
    public static void main(String[] args) {
        int suma = 0;
        for (int i = 1; i <= 10; i++) {
            if (i % 2 == 0) continue;
            suma = suma + i;
        }
        System.out.println("La suma de los impares es: " + suma);
        System.out.println("El número de impares es: " + 5);
    }
}
```

Salida correcta: `La suma de los impares es: 25` y `El número de impares es: 5` (los impares del 1 al 10 son 1, 3, 5, 7 y 9: cinco). Con la versión rota, la suma salía `-25`, y eso era la pista del signo.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Semáforo Humano**	Explicar `if/else if/else` y el orden de las condiciones sin mirar
🏅 **Break o Continúa**	Distinguir `break` y `continue` en un ejercicio "¿Qué imprime?" sin fallar

Logro	Cómo conseguirlo
🏅 **El Cazador de Errores**	Encontrar los errores del Laboratorio sin usar las pistas
🏅 **A prueba de bombas**	Hacer un menú con `do-while` + `try/catch` que no explote con basura
🏅 **El Inventor**	Crear una excepción propia y lanzarla con `throw` en un programa tuyo

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```
public class Misterio2 {
    public static void main(String[] args) {
        for (int i = 1; i <= 6; i++) {
            if (i % 2 != 0) continue;
            System.out.println(i);
        }
    }
}
```

2. **El número invisible:** con el `while` del punto 3, ¿cómo harías para contar cuántos dígitos tiene un número sin usar `String`?
3. **El detective:** tu programa lanza `InputMismatchException` en la línea del `nextInt()`. ¿Qué herramienta usas y qué miras primero en el stack trace?
4. **Verdadero o falso:** "un `catch (Exception e)` atrapa también las `RuntimeException`".

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Estructura que elige entre varios case (6 letras)
3. Termina el bucle entero (5 letras)
5. La clase raíz de todos los errores (9 letras)
6. Repite "mientras" (5 letras)

Vertical:
2. Se salta solo la vuelta actual (8 letras)
4. El bloque que se ejecuta siempre en un try (7 letras)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, la diferencia entre `if` y `switch`."**
2. **"¿Cuál es la diferencia entre `break` y `continue` en un bucle?"**
3. **"Un usuario escribe texto donde tu programa espera un número y la aplicación se cae. ¿Cómo lo arreglarías?"**
4. **"¿Qué es una `NullPointerException` y cómo la evitas?"**
5. **"¿Para qué sirve el bloque `finally`?"**
6. **"¿Cuándo crearías una excepción propia en vez de usar las de Java?"**

🤷 No hay preguntas tontas

> ❓ **¿Puedo usar `switch` con un `double`?**

No. `switch` admite `int` y enteros afines, `char`, `enum` y `String` (desde Java 7). Con `double` usa `if/else if`, porque los decimales casi nunca se comparan con igualdad exacta.

> ❓ **¿Por qué a veces veo `while (true)` si es un bucle infinito?**

Porque se rompe desde dentro con `break`: `while (true) { if (condicion) break; ... }`. Es la forma de escribir "bucle para siempre hasta que ocurra algo". Lo verás mucho en juegos y servidores.

> ❓ **¿El `catch` puede capturar cualquier excepción?**

Si pones `catch (Exception e)`, capturas todas las `Exception` y sus hijas (incluidas las `RuntimeException`). Si quieres capturarlo absolutamente todo, existe `catch (Throwable e)`, pero eso es como pescar con dinamita: también atrapa errores graves de la JVM que no deberías tocar.

🎬 Poscréditos

El programador termina su menú blindado: `do-while` para pedir opción, `try/catch` para sobrevivir a las letras del usuario, `switch` para las opciones… y una excepción propia que lanzó con `throw` cuando el usuario intentó retirar más saldo del que tenía.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — ¿Eso era un programa tuyo o una central nuclear? Menudo blindaje. `try`, `catch`, `switch`, bucles… Hasta una excepción que te has inventado. No está mal para un aprendiz de semáforo.

Novato: — ¿Y ahora qué? Mis programas deciden, repiten y sobreviven.

CONRAD: — *toma un sorbo* Decidir y repetir está bien, pero para resolver problemas de verdad falta algo: pensar en grande. Dividir el problema, diseñar la solución… Eso es otra historia. La próxima parada te la cuenta.

El novato guarda su proyecto, cierra el IDE y siente que los programas ya no le tienen miedo a nada.

PRÓXIMAMENTE EN U07: Algorítmica I: Fundamentos. El momento en el que tus programas no solo piensan, sino que **piensan con método**. 🧩

📚 Volver al índice de la unidad · **Anterior:** 08 · throw y excepciones propias · **Siguiente:** U05 · Funciones y métodos

U05 — Funciones y métodos

El oficio de trocear tu código: métodos que reciben datos, devuelven resultados y convierten un main gigante en piezas pequeñas y probables 🔧

Hasta ahora todo tu código ha vivido dentro de `main`: declarabas variables, repartías `if` y bucles, y el programa entero era un párrafo largo y valiente. Funciona… hasta que el programa crece. Entonces `main` se convierte en un pliego de recetas, no puedes repetir una parte sin copiar y pegar, y cada cambio se propaga como una mancha de café.

En esta unidad aprendes la herramienta que separa a quien escribe código de quien lo **diseña**: el **método** (o función). Un método es una receta con nombre: recibe datos por la puerta, hace una cosa bien explicada, y devuelve el resultado o no devuelve nada y se va. Nada más. Pero con eso construyes programas enteros.

La promesa concreta: al terminar esta unidad, `main` será pequeño, cada concepto vivirá en su propio método con un nombre que se explica solo, y habrás dado el salto que te separa de los ejercicios de “un solo archivo” para siempre.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- **Declarar** tus propios métodos (`public static`) y llamarlos desde `main`.
- **Pasar datos** a un método mediante **parámetros** y usarlos dentro como si fueran variables.
- **Devolver resultados** con `return` y distinguir cuándo un método devuelve y cuándo es `void`.
- Entender el **ámbito** de las variables: qué existe dentro de un método y qué no se sale de él.
- Reconocer y arreglar los **errores típicos** de las funciones (return olvidado, argumentos desordenados…).
- **Dividir** un problema grande en métodos pequeños, cada uno con una responsabilidad.
- **Refactorizar** un programa monolítico en funciones reutilizables sin cambiar lo que hace.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · ¿Qué es una función?	La receta con nombre: por qué existe y qué es “método”	Todos
02 · Tu primer método	`public static void`, llamar desde `main` y ejecutar	Todos
03 · Parámetros: la entrada	Datos que entran: tipos, orden y varios a la vez	Todos
04 · return: la salida	Devolver resultados y cuándo usar `void`	Todos
05 · Ámbito de variables	Qué variables viven dentro de un método y por qué	Todos
06 · Errores frecuentes	Los 7 tropiezos de todo el mundo y cómo salir de ellos	Todos
07 · Divide el problema	Componer: main pequeño, una responsabilidad por método	Todos
08 · Be the Code	Refactoriza un programa monolítico a mano	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Quién Soy, Laboratorio, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

✅ Criterios de evaluación cubiertos (RA2)

RA2: Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.

CE	Criterio	Dónde se cubre
RA2 b)	Se han escrito programas simples.	✅ Todos
RA2 e)	Se han escrito llamadas a métodos estáticos.	✅ Puntos 2, 3, 4 y 7
RA2 f)	Se han utilizado parámetros en la llamada a métodos.	✅ Puntos 3, 4 y 7

> 📌 Esta unidad es la primera mitad del viaje de los métodos: aquí los escribes "a pelo" con `static`, como recetas sueltas. La segunda mitad llega en la U09 (POO), donde los métodos viven dentro de clases con atributos, y en la U10, donde `static` por fin explica su nombre. Hoy solo necesitas la herramienta: entradas, salidas y un nombre honesto.

🚪 ¿Por dónde empiezo?

- ¿Nunca has escrito un método que no fuera `main`? → Arranca en el punto 1 y no te saltes el 2.
- ¿Tu `main` ya parece un párrafo infinito de copiar y pegar? → Ve directo al punto 7 y luego al Be the Code.
- ¿Solo quieres entender por qué `main` es `public static void main`? → El punto 2 te desmonta la firma en 4 palabras.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · ¿Qué es una función?

⏭️ **Al acabar la unidad, continúa en U06 · Arrays.**

01 · ¿Qué es una función?

La receta con nombre que convierte el código repetido en una sola línea: por qué existe y qué demonios es un método 🔧

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 01 · ¿Qué es una función?

📬 La idea en una frase

> **Una función es una receta con nombre: recibe datos por la puerta, hace una cosa bien explicada y devuelve el resultado; tú solo llamas por su nombre.**

🍳 El problema: el párrafo infinito

Mira este `main` de un cuaderno de notas:

```
public class Notas {
    public static void main(String[] args) {
        // Calcular la media de 3 notas
        double media = (8 + 7 + 9) / 3.0;
        System.out.println("Media 1: " + media);

        // Calcular la media de otras 3 notas... otra vez
        double media2 = (6 + 5 + 10) / 3.0;
        System.out.println("Media 2: " + media2);

        // Y mañana, con 30 alumnos, copiar y pegar 30 veces
    }
}
```

El cálculo está escrito **dos veces**. Si mañana cambias la fórmula (¿y si las notas tienen peso?), tendrás que buscar y arreglar todas las copias: una olvidada y el programa ya miente. Programar copiando y pegando es como cocinar repitiendo cada paso del recetario en voz alta en cada plato: funciona… hasta que tienes comensales.

> ⚠️ **Advertencia:** si alguna vez escribes `// Calcular la media` más de un programa en el mismo archivo, en algún lugar un programador senior vuelve a llorar. El código repetido no es código: es deuda con intereses.

📖 La receta con nombre

Una **función** (en jerga: un **método**) es exactamente eso: una receta con nombre en el recetario.

Receta de cocina	Función en Java
El nombre (“Tortilla de patatas”)	El **nombre** del método (`calcularMedia`)
Los ingredientes que te pasan	Los **parámetros** (los datos de entrada)
Los pasos dentro de la receta	El **cuerpo** (el código entre `{` y `}`)
El plato que sale	El **valor que devuelve** (o nada, si solo hace algo)

Tú no recitas la receta entera cada vez: dices **“tortilla”** y la cocina trabaja. Con lo mismo: dices `calcularMedia(...)` y el programa ejecuta ese trozo de código en su sitio.

```
double notaMedia = calcularMedia(8, 7, 9);
System.out.println("Media 1: " + notaMedia);
```

Dos líneas donde antes había tres… y lo mejor: la fórmula **solo está escrita una vez**. Cuando cambie, cambiará en un solo sitio.

🔧 ¿Función o método? (no, no es lo mismo… bueno, sí)

Curso honesto sobre la terminología:

- **Función** es el concepto general: entradas → proceso → salidas. En Python, JavaScript o C las llamas funciones.
- **Método** es esa misma idea **viviendo dentro de una clase**, que es donde vive todo en Java (ya lo viste en la U02: hasta `main` está dentro de una clase).

En la práctica, todo el mundo usa ambos nombres para lo mismo, y en este curso también lo haremos. Si en una entrevista te preguntan la diferencia, ahí tienes la respuesta exacta: **en Java, las funciones son métodos de una clase**.

> 💡 **Consejo:** “método” es la palabra que verás en la documentación de Java y en los errores del compilador. “Función” es la que oirás en las charlas. Saber las dos te salva de quedarte fuera de la conversación.

🧰 Qué gana tu programa al trocearlo

1. **No repites código.** La fórmula se escribe una vez y se usa 30 veces.
2. **Se lee mejor.** `mostrarMedia(...)` explica qué pasa; cinco líneas de aritmética a medias no.
3. **Se prueba por partes.** Puedes comprobar que `calcularMedia` funciona sin ejecutar el programa entero.
4. **Se repara sin miedo.** Arreglas el trozo roto (el método) y el resto no se entera.

Es el mismo instinto de la descomposición de la U01, pero esta vez con herramienta: antes **pensabas** el problema por partes; ahora el lenguaje te deja **escribir** cada parte por separado.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** el nombre del método es documentación. Un método llamado `hacerCosas` es una confesión; uno llamado `calcularIVA` es un plan.

Ejercicio: el vidente de salidas

```
public class Vidente {
    static void saludar() {
        System.out.println("Hola, mundo");
    }

    public static void main(String[] args) {
        saludar();
        saludar();
    }
}
```

¿Qué imprime?

- (A) `Hola, mundo` una vez
- (B) `Hola, mundo` dos veces

- (C) Nada: los métodos no se ejecutan solos
- (D) Error de compilación

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué tres cosas “típicas” tiene una función (según la tabla de la receta)?
2. ¿Se ejecuta el cuerpo de un método solo por estar escrito en el archivo?
3. ¿Es `main` una función? ¿Y `saludar()` de tu propio programa?
4. Menciona dos ventajas de trocear el código en métodos.

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Una **función/método** es una receta con nombre: recibe datos, hace una cosa y devuelve algo (o no).
2. Su ventaja es **escribir una vez, usar muchas**: el código repetido desaparece y el cambio afecta a un solo sitio.
3. En Java todo método vive dentro de una clase, por eso “función” y “método” son la misma herramienta con dos nombres.

> 🦎 **Vocabulario rápido**

Término	Idea general
Función / Método	Receta con nombre: entradas → proceso → salida
Parámetro	Dato que la función recibe por la puerta
Retorno	Valor que la función devuelve al terminar
Llamada	Ejecutar un método: `nombre(...)`
Cuerpo	El código entre { y } del método

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · Tu primer método

02 · Tu primer método

Desmontamos la firma `public static void` en cuatro palabras y escribimos la primera receta del recetario 🔍

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 02 · Tu primer método

📬 La idea en una frase

> **Declarar un método es escribir su receta (`public static void nombre() { ... }`); llamarlo es decir su nombre con paréntesis y punto y coma para que ejecute su cuerpo.**

✍️ La primera receta

Este es un programa completo con un método propio:

```
public class Saludo {
    public static void saludar() {
        System.out.println("¡Hola, clase!");
    }

    public static void main(String[] args) {
        saludar();
        saludar();
    }
}
```

Salida:

```
¡Hola, clase!
¡Hola, clase!
```

Dos partes: **`saludar()`** es la receta (declarada arriba), y en `main` la **llamamos** dos veces. Fíjate en los dos detalles de escritura:

- La **declaración** no lleva punto y coma al final: termina con su bloque `{ ... }`.
- La **llamada** sí lleva `;`, porque es una sentencia, igual que `int x = 5;`.

🔍 La firma, palabra por palabra

```
public static void saludar() {
```

Palabra	Qué significa	Cuándo la verás explicada
`public`	Visible desde cualquier parte	U10 (visibilidad)
`static`	Pertenece a la clase, no a un objeto	U10 (métodos static)
`void`	**No devuelve** nada	Punto 4 de esta unidad
`saludar()`	El nombre y sus paréntesis	Hoy

De momento escribes **siempre** `public static` delante de tus métodos. No te les olvides: sin `static`, `main` no podrá llamarlos (la explicación completa llega en la U10, cuando ya sepas qué es un objeto).

> 📝 **Nota:** el nombre sigue la convención camelCase: `calcularMedia`, `mostrarMenu`, `esPar`. Sin espacios, sin tildes, y la primera palabra en minúscula. Java es perezoso con los acentos y puntual con las mayúsculas.

Las reglas del juego

1. Los métodos se declaran **dentro de la clase**, nunca dentro de otro método (ni siquiera dentro de `main`).
2. El **orden no importa**: puedes llamar a un método declarado más abajo del todo. Java no lee de arriba abajo; busca por nombre.
3. Cada método es una isla con su propio nombre: dos métodos no pueden llamarse igual **con los mismos parámetros** (eso lo verás en la U09, con la sobrecarga).

> ⚠️ **Advertencia:** el error de compilación `cannot find symbol` al llamar a un método casi siempre significa que le has puesto otro nombre del que tiene, o que lo estás llamando desde fuera de la clase.

🎬 `main` también es un método

Aquí está la revelación del día: `main` no es especial porque sea mágico, sino porque la JVM **lo busca por ese nombre exacto** para arrancar tu programa:

```
public static void main(String[] args) {
```

- `public` → la JVM debe poder verlo.
- `static` → la JVM lo llama **sin crear un objeto** de tu clase.
- `void` → devuelve nada (solo ejecuta).
- `String[] args` → el array de argumentos de la línea de comandos (lo viste en la U02).

Todo lo que llevas usando desde la U02 era… un método más. La diferencia es que tú solo llamabas a uno sin darte cuenta. Ahora llamas a todos los que te apetezca.

```
public class Reencuentro {
    public static void main(String[] args) {
        saludar();              // llamamos a un método nuestro
        System.out.println("Seguir con el programa...");
    }

    public static void saludar() {
        System.out.println("Hola de nuevo");
    }
}
```

Funciona aunque `saludar` esté **debajo** de `main`. Regla 2: el orden no importa.

⭐ Sé el Código, my friend…

> 👓 **Don Tip:** si `main` necesita llamar a un método, ese método casi siempre necesita `static`. Sin `static`, tendrás que esperar a la U10 para que las cosas encajen.

Ejercicio: la firma rota

¿Cuál de estos métodos **compila**?

```java
// Opción A
static public void contar() {
    System.out.println("1");
}

// Opción B
public static contar() {
    System.out.println("1");
}

// Opción C
public static void contar {
    System.out.println("1");
}

// Opción D
public static void contar() System.out.println("1");
```

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. Escribe la firma completa de un método llamado `despedir` que no devuelve nada.
2. ¿Qué línea (y en qué sitio) ejecuta el cuerpo de `despedir()`?
3. ¿Puede `main` estar declarado DESPUÉS de `despedir` en la clase? ¿Y al revés?
4. ¿Cuál es la diferencia de puntuación entre declarar y llamar?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Un método se declara con su firma (`public static void nombre() {...}`) dentro de la clase y se **ejecuta solo cuando lo llamas**.
2. `public static void main(String[] args)` es un método más: la JVM lo llama al arrancar porque tiene ese nombre exacto.

3. El orden de los métodos no importa; lo que importa es el nombre correcto y no declarar un método dentro de otro.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Firma | La línea completa: modificadores + retorno + nombre + parámetros |
> | Declaración | Definir la receta (con su cuerpo `{...}`) |
> | Llamada | Ejecutar el método: `nombre(args);` |
> | `static` | De clase: se usa sin crear objetos (detalle en U10) |
> | `void` | "No devuelvo nada" |
> | camelCase | `calcularMedia`: minúscula al inicio, mayúsculas entre palabras |

📚 Volver al índice de la unidad · **Anterior:** 01 · ¿Qué es una función? · **Siguiente:** 03 · Parámetros: la entrada

03 · Parámetros: la entrada

Ingredientes que entran por la puerta: tipos, orden, varios a la vez y la diferencia entre parámetro y argumento 📥

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 03 · Parámetros: la entrada

📬 La idea en una frase

> **Los parámetros son las variables que un método recibe al ser llamado: tú pasas valores en los paréntesis y el método trabaja con ellos como si los hubiera escrito él mismo.**

🔁 El problema: el método de siempre

Nuestro `saludar()` solo sabe decir una cosa:

```
static void saludar() {
    System.out.println("¡Hola, clase!");
}
```

Si mañana hay que saludar a “Marta”, a “Youssef” y a toda la clase uno a uno, toca copiar y pegar con nombres distintos… o darle **entradas** al método:

```
public class Saludos {
    public static void saludar(String nombre) {
        System.out.println("¡Hola, " + nombre + "!");
    }

    public static void main(String[] args) {
        saludar("Marta");
        saludar("Youssef");
        saludar("la clase entera");
    }
}
```

Salida:

```
¡Hola, Marta!
¡Hola, Youssef!
¡Hola, la clase entera!
```

Una sola receta, tres platos. El método ya no decide qué datos usa: **se los pasas**.

📥 Cómo se declaran

```
public static void saludar(String nombre) {
//                         ^^^^^^^^^^^^
//                         parámetro: tipo + nombre
```

- Cada parámetro lleva **tipo** y **nombre**, como una variable: `String nombre`, `int edad`, `double nota`.
- Varios parámetros se separan con **coma**, y cada uno repite su tipo: `String nombre, int edad`.
- Dentro del cuerpo, los parámetros se usan **como variables normales**: se leen, se concatenan, se comparan.

```
public static void presentar(String nombre, int edad) {
    System.out.println(nombre + " tiene " + edad + " años.");
}

public static double areaRectangulo(double base, double altura) {
    return base * altura;   // ¡ya sabemos devolver! detalle en el punto 4
}
```

> 💡 **Consejo:** nombra los parámetros por lo que **son**, no por el orden: `double base, double altura` se entiende; `double x, double y` obliga a leer el cuerpo cada vez.

Parámetro ≠ argumento

Dos palabras, dos momentos, la misma confusión eterna:

```java
//                parámetros (en la declaración)
static int sumar(int a, int b) { return a + b; }

//               argumentos (en la llamada)
int r = sumar(3, 4);
```

Momento	Nombre	Ejemplo
Declarar el método	**Parámetros**	`int a, int b`
Llamar al método	**Argumentos**	`3, 4`

En las charlas de pasillo todo el mundo dice “parámetros” para los dos. En un examen o en una entrevista, la distinción te hace sonar a profesional.

🎯 Reglas de las entradas

1. **El orden importa:** `presentar("Ana", 20)` no es lo mismo que `presentar(20, "Ana")` (esta última ni compila: Java espera un `String` primero).
2. **El número importa:** `sumar(3)` y `sumar(3, 4, 5)` no coinciden con la firma → error de compilación.
3. **El tipo debe encajar:** un `int` se acepta donde se pide `double` (ampliación automática), pero un `String` donde se pide `int` es un muro: `incompatible types: String cannot be converted to int`.
4. Cada llamada crea **sus propias copias** de los argumentos: `sumar(3, 4)` y `sumar(10, 20)` viajan con sus valores, sin pisarse (para primitivos y valores simples, esto es así; los objetos guardan alguna sorpresa para la U09).

> ⚠️ **Advertencia:** si llamas `saludar();` a un método que espera un `String`, no hay “salteo elegante”: `error: method saludar in class ... cannot be applied to given types`. Los parámetros no son opcionales.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando un método tiene muchos parámetros del mismo tipo (`int fila, int columna`), comete el crimen de cambiar el orden y verás cómo todo el programa empieza a

> apuntar a la casilla equivocada. Por eso se nombran bien y se pasan en orden.

Ejercicio: la llamada confusa

```
public class Confusion {
    static void pintar(String color, int veces) {
        for (int i = 0; i < veces; i++) {
            System.out.print(color);
        }
        System.out.println();
    }

    public static void main(String[] args) {
        pintar("verde", 3);
        pintar(2, "azul");
    }
}
```

¿Qué ocurre?

- (A) Imprime `verdeverdeverde` y luego `azulazulazul`
- (B) Compila e imprime `verdeverdeverde` y `2222...` (bucle raro)
- (C) No compila: en la segunda llamada, argumentos en orden incorrecto
- (D) No compila: `pintar` necesita `return`

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. En `static void log(String mensaje, int nivel)`, ¿cuáles son los parámetros y qué le pasas en `log("Fallo", 3)`?
2. ¿Por qué `sumar(1, 2, 3)` no compila si `sumar(int a, int b)` está bien escrito?
3. ¿Qué palabra describe a `3` y `4` en `sumar(3, 4)`: parámetro o argumento?
4. ¿Se puede llamar a un método sin paréntesis, como `saludar;`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los **parámetros** (tipo + nombre) se declaran entre paréntesis y dentro del método se usan como variables.
2. En la llamada pasas **argumentos** cuyo **orden, número y tipo** deben encajar con la firma.
3. Varios parámetros se separan por coma y cada uno lleva su tipo: `String nombre, int edad`.

> 🦎 **Vocabulario rápido**

Término	Idea general
Parámetro	Variable de la entrada, en la declaración
Argumento	Valor que pasas en la llamada
Firma	Nombre + tipos de los parámetros
Ampliación	`int` cabe en `double` sin avisar
Copia del argumento	Cada llamada trabaja con su propio valor

📚 Volver al índice de la unidad · **Anterior:** 02 · Tu primer método · **Siguiente:** 04 · return: la salida

04 · return: la salida

Devolver resultados como una máquina bien educada: tipos que coinciden, caminos que cierran y el pecado de imprimir cuando debes devolver 📫

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 04 · return: la salida

📬 La idea en una frase

> **Un método con `return` calcula y entrega un valor al que lo llamó; un método `void` solo ejecuta algo y se calla. La firma promete el tipo y `return` cumple la promesa.**

📫 Dos oficios, dos firmas

Hasta ahora tus métodos **hacían** cosas (imprimían). Ahora van a **entregar** cosas:

```
public static int sumar(int a, int b) {
    return a + b;
}
//     ^^^
//     la firma dice "devuelvo int" y return lo cumple
```

Quien llama recibe el valor **y puede usarlo**:

```
int total = sumar(3, 4);          // 7 en una variable
System.out.println(sumar(3, 4));  // 7 directo al println
int doble = sumar(sumar(1, 2), 3); // devuelto como argumento: 6
```

La regla de oro: **el tipo del `return` debe encajar con el tipo de la firma**. `int` con `int`, `double` con `double`, `String` con `String`. Si la firma dice `double` y devuelves `7`, Java lo acepta (el entero se amplía). Si dice `String` y devuelves `7`, es un muro: `incompatible types: int cannot be converted to String`.

`void`: la firma que no promete nada

```java
public static void imprimirResultado(int x) {
    System.out.println("Resultado: " + x);
    // sin return (o con un return; vacío opcional)
}
```

`void` = "no devuelvo nada, solo hago mi trabajo". ¿Cómo eliges?

Usa retorno cuando…	Usa void cuando…
Otro método necesita el resultado	Solo hay que producir un efecto (imprimir, guardar, pintar)
La tarea es "calcular"	La tarea es "hacer"
Quieres probar el valor por separado	El resultado ya se ve en pantalla o en fichero

> 💡 **Consejo:** si tu método se llama `calcular...`, `get...`, `es...` o `devolver...`, casi seguro debe **devolver**. Si se llama `imprimir...`, `mostrar...` o `guardar...`, casi seguro es `void`. El nombre del método y su firma cuentan la misma historia.

🏞️ `return` corta la ejecución

El primer `return` que se ejecuta **termina el método al instante**: lo que haya debajo no se ejecuta nunca.

```java
public static String clasificar(int nota) {
    if (nota >= 5) {
        return "Aprobado";
    }
    return "Suspenso";   // solo se alcanza si la línea anterior no se ejecutó
}
```

Dos `return` en el mismo método son legales (y muy útiles), siempre que **todos los caminos posibles terminen en uno**:

```java
public static int maximo(int a, int b) {
    if (a > b) {
        return a;
    } else {
        return b;   // todos los caminos cierran ✓
    }
}
```

Si un camino llega al final sin `return`:

```java
public static int peligro(int a) {
    if (a > 0) {
        return a;
    }
    // ¿y si a <= 0?  →  error: missing return statement
}
```

CONRAD se pone rojo: `error: missing return statement`. Java no adivina: **exige** que la firma no pueda quedar sin cumplir.

> ⚠️ **Advertencia:** código tras un `return` incondicional no se ejecuta jamás (`unreachable statement` es su forma de protesta). Y si una rama devuelve `int` y otra `double`… la firma solo admite uno: decide.

🎭 El pecado de imprimir cuando debes devolver

El error conceptual más común de la unidad:

```java
// ❌ Imprime la media... pero quien llama no se entera
public static void media(int a, int b) {
    System.out.println((a + b) / 2.0);
}

// ✅ Devuelve la media; el que llama decide qué hacer
public static double media(int a, int b) {
    return (a + b) / 2.0;
}
```

Con la versión `void`, `main` no puede guardar la media, compararla ni reutilizarla: se imprimió y desapareció. Con `return`, el valor entra en el programa como datos de verdad.

Regla práctica: **`System.out.println` es para `main` (o para métodos cuyo oficio sea mostrar), no para los métodos que calculan.** El que calcula calcula; el que decide imprime.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** `return true;` / `return false;` en un método que empieza por `es`, `tiene` o `puede` es el patrón más rentable del lenguaje: convierte preguntas en booleanos reutilizables.

Ejercicio: la adivinanza del retorno

```
public static int truco(int x) {
    if (x > 10) {
        return x * 2;
    } else if (x > 5) {
        return x + 3;
    }
    return 1;
}
```

¿Qué devuelve `truco(7)` y `truco(12)`?

- (A) `10` y `24`
- (B) `14` y `24`
- (C) `10` y `12`
- (D) Compila, pero `truco(7)` no devuelve nada

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. En `public static boolean esPar(int n)`, ¿qué dos líneas debe tener como mínimo el cuerpo?
2. ¿Qué error da un método `int` sin `return` en algunos caminos?

3. ¿Cuál es la diferencia entre `void imprimir(int x)` y `int devolver(int x)` para quien llama?
4. ¿Se puede hacer `return` sin valor en un método `int`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. La firma promete un tipo de retorno y cada camino del cuerpo lo cumple con `return expresión;`.
2. `return` **termina el método** al ejecutarse; si algún camino llega al final sin `return` (y no es `void`), no compila.
3. Los métodos **calculan** devolviendo; imprimir es oficio de quien decide (normalmente `main`).

> 🐛 **Vocabulario rápido**

Término	Idea general
`return`	Entrega un valor y cierra el método
`void`	Firma sin retorno: solo ejecuta
`missing return statement`	Algún camino no cumple la firma
Predicado	Método que devuelve `boolean` (`esPar`)
Ampliación	`int` cabe en `double` al devolver

📚 Volver al índice de la unidad · **Anterior:** 03 · Parámetros: la entrada · **Siguiente:** 05 · Ámbito de variables

05 · Ámbito de variables

Las variables viven dentro de su método y no se salen: por qué tu `total` no puede chocar con la de al lado ni escaparse de casa 🏠

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 05 · Ámbito de variables

---

📬 La idea en una frase

> **El ámbito (scope) es el trozo de código donde una variable existe: dentro de un método solo viven sus parámetros y sus locales, y lo que nace ahí muere ahí.**

🏠 Cada método es una casa

```
public class Casas {
    public static void sumar() {
        int total = 10;
        total += 5;
        System.out.println(total);   // 15
    }

    public static void restar() {
        int total = 100;
        total -= 5;
        System.out.println(total);   // 95
    }

    public static void main(String[] args) {
        sumar();
        restar();
        // System.out.println(total);  →  ¡error!
    }
}
```

Dos métodos con una variable llamada `total` **a la vez**: cero problemas, porque son dos `total` distintas, una en cada casa. En cambio, si `main` intenta usar `total` sin declararlo:

```
error: cannot find symbol: variable total
```

Java no lo encuentra porque **no existe** fuera de `sumar()`. El método terminó, su pila de ejecución se deshizo y `total` ya no está.

> 💡 **Consejo:** cuando veas `cannot find symbol` en una variable, casi siempre es una de dos: la has mal escrito (typo) o estás intentando usarla **fuera de su casa**.

📏 Las reglas del ámbito

1. **Parámetros y locales** de un método se ven **dentro de ese método** (de su firma a su llave de cierre).
2. **Un bloque también es una casa pequeña:** una variable declarada dentro de un `if` o de un `for` no existe fuera de sus llaves (ya lo viste en la U04).

```
for (int i = 0; i < 10; i++) { ... }
// System.out.println(i);  →  cannot find symbol: variable i
```

3. **El orden importa dentro del método:** solo puedes usar una variable **después** de declararla.
4. **Dos hermanas no se pelean:** cada método tiene su propio espacio; repetir nombres entre métodos es normal y sano.
5. Al terminar el método, sus variables **mueren**: sus valores no sobreviven (salvo lo que el método devuelve con `return` o lo que imprime).

```
main:            sumar:
┌──────────┐     ┌──────────────┐
│ (nada)   │     │ total = 10   │  ← existe solo aquí
└──────────┘     │ total = 15   │
                 └──────────────┘  ← y aquí desaparece
```

🚪 Lo que entra por la puerta también es de la casa

Los **parámetros** son variables como las demás: se declaran al inicio del método y mueren con él.

```
public static int triplicar(int numero) {
    int resultado = numero * 3;  // dos variables de la casa
    return resultado;
}
```

Y aquí llega la consecuencia bonita: como Java copia los argumentos al entrar, **tu método puede usar el valor que le pasas, pero no reescribe la variable de quien llama**:

```
int puntos = 10;
triplicar(puntos);
System.out.println(puntos);   // sigue siendo 10
```

`triplicar` trabajó con su **copia** de `puntos`. Para "devolver" cambios, el método devuelve el nuevo valor con `return` y quien llama lo guarda. (¿Y qué pasa con los arrays, que sí se modifican desde dentro? Ese misterio lo resuelves en la U06, punto 5. Y la versión completa, con objetos, en la U09.)

> ⚠️ **Advertencia:** no intentes "guardar" el resultado de un método en una variable de otro método. Los métodos solo comparten lo que se **pasan** (argumentos) o lo que se **devuelven** (`return`). No hay ventanillas laterales.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** si necesitas recordar algo de un método a otro, no escondas una variable: **devuélvelo** con `return` y que la decisión la tome quien llama.

Ejercicio: ¿existe esta variable?

```java
public class Alcance {
    public static void a() {
        int x = 1;
    }

    public static void b() {
        System.out.println(x);
    }

    public static void main(String[] args) {
        b();
    }
}
```

¿Qué pasa?

- (A) Imprime `1`
- (B) Imprime `0`
- (C) No compila: `x` no existe en `b()`
- (D) Compila, pero lanza excepción al ejecutar

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Pueden dos métodos tener una variable local llamada `contador` a la vez?
2. ¿Existe `i` del `for` fuera de sus llaves?
3. ¿Sobrevive `resultado` al terminar el método que la declara?
4. Mi método no cambia el `saldo` que le pasé. ¿Es un bug?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. El **ámbito** de una variable es el trozo de código donde existe: su método (o su bloque `if`/`for`) y nada más.
2. Dentro caben **parámetros** y **locales**; al terminar el método desaparecen y otro método no puede verlas.
3. Entre métodos solo viaja lo que se **pasa** (argumentos copiados) o lo que se **devuelve** (`return`).

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Ámbito / scope | Región donde una variable existe |
> | Variable local | Declarada dentro de un método o bloque |
> | Parámetro | Variable de entrada del método |
> | `cannot find symbol` | Estás usando una variable fuera de su casa |
> | Copia del argumento | El método trabaja con su propio duplicado |

📚 Volver al índice de la unidad · **Anterior:** 04 · return: la salida · **Siguiente:** 06 · Errores frecuentes

06 · Errores frecuentes

Los 7 tropiezos de todo el mundo con los métodos: código roto, error real de CONRAD y su arreglo 🧯

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 06 · Errores frecuentes

📬 La idea en una frase

> **Los errores con métodos no son aleatorios: tienen nombre, mensaje y arreglo. Aprende los siete y el 90% de tus horas de frustración se evaporan.**

🧯 La galería de los 7 tropiezos

1. Falta el `return` (o no todos los caminos cierran)

```
public static int doble(int n) {
    if (n > 0) {
        return n * 2;
    }
    // ¿y si n <= 0?
}
```

`error: missing return statement`. **Arreglo:** cierra todos los caminos con `return`.

2. Argumentos en orden o tipo incorrecto

```
static void pintar(String color, int veces) { ... }
pintar(3, "rojo");
```

`error: incompatible types: int cannot be converted to String`. **Arreglo:** respeta el orden y el tipo de la firma; repasa la llamada con los ojos en la declaración.

3. Número de argumentos que no cuadra

```
static int suma(int a, int b) { return a + b; }
int r = suma(1, 2, 3);
```

`error: wrong number of arguments`. **Arreglo:** la firma manda: 2 parámetros = 2 argumentos, ni uno más ni uno menos.

4. Imprimir cuando debes devolver

```
static void media(int a, int b) {
    System.out.println((a + b) / 2.0);   // void, sin retorno
}
double m = media(3, 4);   // ← esto no compila: void no es un valor
```

`error: incompatible types: void cannot be converted to double`. **Arreglo:** que el método `return` el resultado; imprimir es cosa de quien llama.

5. Olvidar `static` al llamar desde `main`

```
void saludar() { ... }              // sin static
public static void main(String[] a) {
    saludar();                      // ← ¡PROHIBIDO de momento!
}
```

`error: non-static method saludar() cannot be referenced from a static context`. **Arreglo:** de esta unidad, tus métodos llevan `static`. La explicación completa (y la alternativa con objetos) vive en la U10.

6. Llamar como si fuera sentencia suelta

```
saludar        // falta () y ;
saludar()      // falta el ;
```

Arreglo: la llamada es `saludar();` — con paréntesis (aunque no lleve argumentos) y con punto y coma.

7. Copiar y pegar… el método equivocado

```
mostrarTotal();
mostrarTotal();   // querías llamar a mostrarMedia()
```

Sin error de compilación: **el más peligroso de los siete**. Compila, ejecuta y miente. **Arreglo:** lee el nombre de cada llamada como si lo escribiera un desconocido (porque dentro de dos semanas lo serás).

> 📝 **Nota:** los errores 1-6 los ve CONRAD en rojo antes de ejecutar. El 7 es de ejecución: por eso el nombre del método y la prueba manual importan tanto.

🩺 Cómo diagnosticar sin perderte

Cuando falle un método, haz las preguntas en este orden:

1. **¿Compila?** → Lee el mensaje de CONRAD: `cannot find symbol` (nombre mal escrito o fuera de ámbito), `wrong number of arguments`, `incompatible types` (tipo/orden), `missing return statement`.
2. **¿Compila pero no hace lo esperado?** → ¿Llamaste al método correcto (7)? ¿Le pasaste los argumentos en el orden que crees (2)?
3. **¿Imprime algo raro?** → Traza la llamada: ¿quién imprime y quién devuelve? ¿Estás usando la copia que llega por parámetro?

> ⚠️ **Advertencia:** no pegues el error en el buscador antes de leerlo entero. El 95% de los errores de esta unidad dicen en la primera línea **qué** está mal y en la segunda **dónde**.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando todo compile y el resultado sigue sin cuadrar, imprime (o mejor: devuelve) los valores a mitad de camino. Depurar es mirar, no adivinar.

Ejercicio: el taller de arreglos

Este método debería devolver el mayor de dos números, pero tiene **2 errores** (uno de compilación y uno de lógica):

```java
public static int mayor(int a, int b) {
    if (a > b) {
        return a
    }
    return 0;
}
```

¿Cuáles son y cómo quedan?

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué error da `int r = media(3, 4);` si `media` es `void` y solo imprime?
2. ¿Qué tres mensajes busca primero al no compilar una llamada?
3. ¿Por qué el error 7 (llamar al método equivocado) no lo detecta el compilador?
4. ¿Qué palabra falta en `void log(String s) { System.out.println(s) }`?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. Los errores de métodos son **siete familias**: return, orden/tipo de argumentos, número de argumentos, void-vs-retorno, `static`, puntuación de la llamada y nombre equivocado.
2. Los seis primeros los atrapa el compilador; el séptimo lo detecta **tu cerebro** al leer nombres en voz alta.
3. Diagnostica siempre en orden: compila → mensaje de error → traza de la llamada.

> 🐛 Vocabulario rápido

Término	Idea general
`missing return statement`	Algún camino no devuelve lo que la firma promete
`wrong number of arguments`	Argumentos ≠ parámetros

Término	Idea general
`incompatible types`	El tipo no encaja (orden incluido)
`static context`	Llamar a algo de clase desde `main` (detalles en U10)
Error de lógica	Compila y ejecuta, pero miente

📚 Volver al índice de la unidad · **Anterior:** 05 · Ámbito de variables · **Siguiente:** 07 · Divide el problema

07 · Divide el problema

Main como director de orquesta: una responsabilidad por método, el programa que se lee como un resumen y cómo trocear sin miedo 🧱

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 07 · Divide el problema

📬 La idea en una frase

> **Dividir es escribir un `main` que se lea como un resumen en inglés claro ("lee datos, calcula media, muestra informe") y esconder cada frase en un método con su nombre.**

🎼 El director no toca los instrumentos

Un buen `main` parece esto:

```
public static void main(String[] args) {
    double[] notas = leerNotas();
    double media = calcularMedia(notas);
    String informe = formatearInforme(media);
    System.out.println(informe);
}
```

Cuatro líneas, cuatro verbos, cero aritmética. ¿Quieres saber cómo se leen las notas? Abres `leerNotas()`. ¿Cómo se calcula la media? `calcularMedia()`. Cada detalle vive en su sitio y `main` sigue siendo un **índice del programa**, no un pliego de recetas.

Si ese mismo programa viviera todo en `main`, tendrías 40 líneas mezclando lectura, cálculo y formato donde ningún comentario te salva. El director de orquesta no toca los instrumentos: **coordina**.

🔪 Cómo trocear: el método de los verbos

Mira un `main` de taller y enciende los verbos:

```
public static void main(String[] args) {
    // 1. leer las notas del teclado          → leerNotas()
    // 2. comprobar si hay alguna suspensa    → tieneSuspensa(notas)
    // 3. calcular la media                   → calcularMedia(notas)
    // 4. imprimir el resultado bonito        → mostrarResultado(media, notas)
}
```

Cada verbo con paréntesis en el comentario es **un candidato a método**. El patrón se repite:

1. **Escribe el comentario-objetivo** (o léelo si ya está).
2. **Crea el método** con ese nombre, `public static`, parámetros si necesita datos, retorno si devuelve algo.
3. **Mueve el código** de ese paso al cuerpo.
4. **Deja una llamada** en `main` y **ejecuta**: si hacía lo mismo que antes, has troceado bien.
5. Repite con el siguiente verbo. **Un método por vez.**

> 💡 **Consejo:** si necesitas un comentario para explicar un bloque de 5 líneas, ese bloque ya tiene nombre: hazlo método y que el nombre hable por él.

¿Dónde está el límite?

Se queda en `main`	Se va a su método
La secuencia de pasos	Un paso con nombre propio
Un `if` de dos líneas que decide flujo	Un bloque que repites o que “hace una cosa”
La llamada final de impresión	La cuenta, la lectura, la búsqueda, el formato

No hay ley universal de “más de N líneas”, sí una brújula: **una responsabilidad por método**. `calcularMediaYMostrarYGuardar` es tres métodos disfrazados de uno.

🧪 Antes y después: el informe de notas

Antes (todo en `main`):

```java
public static void main(String[] args) {
    Scanner teclado = new Scanner(System.in);
    double suma = 0;
    for (int i = 0; i < 5; i++) {
        System.out.print("Nota " + (i + 1) + ": ");
        suma += teclado.nextDouble();
    }
    double media = suma / 5;
    System.out.println("Media: " + media);
    if (media >= 5) {
        System.out.println("¡APROBADO!");
    } else {
        System.out.println("Suspenso. A por ello.");
    }
}
```

Después (un resumen y dos recetas):

```java
public static void main(String[] args) {
    double[] notas = leerNotas(5);
    double media = calcularMedia(notas);
    mostrarVeredicto(media);
}

static double[] leerNotas(int cantidad) { ... }      // leer
static double calcularMedia(double[] notas) { ... }  // calcular
static void mostrarVeredicto(double media) { ... }   // contar
```

Mismo programa, dos vidas distintas: en la segunda, mañana añades el máximo sin tocar nada más (bueno, casi: `leerNotas` ya devuelve un array… que verás en serio en la UO6).

> 📝 **Nota:** el orden de declaración no importa (lo viste en el punto 2), así que `main` puede leerse arriba como un resumen aunque sus ayudantes estén debajo.

⭐ Sé el Código, my friend…

> 👓 **Don Tip:** si dudas entre trocear o no, escribe la llamada que **quisieras** tener (`int max = encontrarMaximo(notas);`) y luego haz que exista. El código se diseña hacia delante, como se pide en un restaurante.

Ejercicio: el verbo escondido

¿Qué método (nombre y firma) extraerías de este `main`? Devuelve `true` si todos los números son positivos:

```
public static void main(String[] args) {
    int[] datos = {3, 7, 2};
    boolean todoPositivo = true;
    for (int i = 0; i < datos.length; i++) {
        if (datos[i] <= 0) {
            todoPositivo = false;
        }
    }
    System.out.println("¿Todo positivo? " + todoPositivo);
}
```

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué tres preguntas te haces al diseñar un método? (pista: nombre, entradas, salidas).
2. ¿Cuál es el oficio de `main` en un programa bien troceado?
3. ¿Qué señal dice “este bloque pide método”?
4. ¿Cuántos bloques troceas de una vez cuando refactorizas?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. Un `main` bien troceado se lee como un **resumen** de verbos; cada verbo vive en un método con una responsabilidad.
2. El método de los verbos: **comentario → nombre → firma → mover código → llamar y probar**, un método por vez.
3. La brújula no es la longitud, es la **responsabilidad**: si el nombre del método necesita “y”, son dos métodos.

🦎 Vocabulario rápido

Término	Idea general
Orquestación	`main` coordina; los métodos trabajan
Responsabilidad única	Un método, una cosa bien hecha
Extracción	Mover un bloque a un método nuevo
Refactorizar	Reorganizar sin cambiar lo que hace
Método helper	Ayudante pequeño al servicio de otro paso

📚 Volver al índice de la unidad · **Anterior:** 06 · Errores frecuentes · **Siguiente:** 08 · Be the Code

08 · Be the Code

Refactoriza un programa de notas en casa, paso a paso, sin copiar soluciones: los verbos, las firmas y el main que se lee solo 🛠️

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 08 · Be the Code

---

📬 La idea en una frase

> **Este punto no tiene teoría nueva: tiene un programa de 30 líneas que debes trocear con tus manos. Solo pides pistas cuando te atasques de verdad.**

El programa de antes (todo en `main`) es tu taller de hoy. Tu misión: convertirlo en una orquesta de métodos sin cambiar lo que imprime. Si lo has hecho a mano, dominas la unidad entera.

🛠️ Tu punto de partida

```java
import java.util.Scanner;

public class InformeNotas {

    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        double suma = 0;
        boolean haySuspensa = false;

        for (int i = 0; i < 5; i++) {
            System.out.print("Nota " + (i + 1) + ": ");
            double nota = teclado.nextDouble();
            suma += nota;
            if (nota < 5) {
                haySuspensa = true;
            }
        }

        double media = suma / 5;
        System.out.println("Media: " + media);

        if (haySuspensa) {
            System.out.println("Hay suspensa: a por ello.");
        } else if (media >= 7) {
            System.out.println("¡Notaza!");
        } else {
            System.out.println("Aprobado con rodillas.");
        }
    }
}
```

Antes de escribir nada, responde en papel (en serio, en papel):

1. ¿Cuántos **verbos con nombre propio** ves? (pista: tres: leer, calcular, mostrar)
2. ¿Qué datos necesita cada uno y **qué devuelve** cada uno?
3. ¿Qué queda en `main` después de trocear?

🪜 Paso a paso (pistas que no regalan el final)

1. Crea `static double[] leerNotas(int cantidad)` debajo de `main`.

> 🥜 **Atascado?**

2. Crea `static double calcularMedia(double[] notas)`.

> 🥜 **Atascado?**

3. Crea `static boolean tieneSuspensa(double[] notas)`.

> 🥜 **Atascado?**

4. Crea `static void mostrarVeredicto(double media, boolean suspensa)`.

> 🥜 **Atascado?**

5. Reescribe `main` como resumen: leer → media → suspensa → mostrar.

> 🥜 **Atascado?**

> 🔄 **Solución completa**

> ⚠️ **Advertencia:** la salida debe ser **idéntica** antes y después. Si cambia, has movido una línea de más: vuelve al paso anterior (refactorizar es cambiar la forma, nunca el comportamiento).

🧪 El Lío: el refactor malogrado

Tu compañero refactorizó y ahora esto **no compila**:

```
public class MalRefactor {
    public static void main(String[] args) {
        double m = media(3, 4, 5);
        System.out.println(m);
    }

    static void media(int a, int b, int c) {
        double resultado = (a + b + c) / 3.0;
        System.out.println(resultado);
    }
}
```

Pistas (no mires la solución aún):

1. ¿Qué quiere `main` hacer con el resultado de `media`?
2. ¿Qué tipo dice la firma que devuelve…?
3. ¿Qué pasa con el `println` de dentro: es oficio de `media` o de quien llama?

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué dos preguntas te haces de cada verbo antes de crear su método? (entradas y salidas)
2. ¿Por qué `leerNotas` devuelve `double[]` y no imprime las notas?
3. ¿Qué debe pasar si troceas bien y ejecutas?
4. ¿Cuál es la señal de que has troceado mal?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. Trocear = **verbo → método** con firma clara, mover el código y dejar una llamada en `main`.
2. El `main` final se lee como un resumen: leer → calcular → mostrar, sin un solo `for`.
3. Refactorizar **nunca** cambia la salida: si cambia, se revierte el paso.

> 🦎 Vocabulario rápido

Término	Idea general
Refactorizar	Reorganizar sin cambiar el comportamiento
Extraer método	Sacar un bloque a su propia función
Resumen en `main`	Líneas que se leen como el plan del programa
Firma clara	Nombre + parámetros + retorno que se entienden solos
Antes/después	Prueba de que la refactorización fue limpia

📚 Volver al índice de la unidad · **Anterior:** 07 · Divide el problema · **Siguiente:** 09 · Repaso interactivo

09 · Repaso interactivo

Fireside, CONRAD, crucigrama y laboratorio: la fiesta de cierre de Funciones y métodos 🎉

> 🗺️ **Estás en:** 🔧 **U05 · Funciones y métodos** → 09 · Repaso interactivo

📬 La idea en una frase

> **Este punto no enseña nada nuevo: destroza lo aprendido con preguntas, errores ajenos y un laboratorio hasta que lo tengas en las manos.**

⭐ Sé el Código, my friend…

Pregunta 1 — la firma

```
public static double promedio(int a, int b, int c) {
    return (a + b + c) / 3;
}
```

¿Qué le pasa a este método?

- (A) Nada: compila y devuelve bien
- (B) Compila, pero devuelve un `int` con la parte decimal perdida
- (C) No compila: `double` no puede recibir la suma de tres `int`

> › 🔄 **Solución**

Pregunta 2 — el orden

```
static void pintar(String texto, int veces) { ... }
pintar(3, "OK");
```

¿Qué ocurre?

- (A) Imprime `OK` tres veces
- (B) Imprime `3` tres veces
- (C) No compila: tipos al revés

> 🔄 Solución

Pregunta 3 — el pecado

```
static void doble(int n) {
    System.out.println(n * 2);
}
int r = doble(5);
```

Elige sabiamente:

1. **`10`** — el método imprime el doble y devuelve algo que se guarda ✗
2. **No compila** — `main` intenta guardar un `void` en un `int` ✓
3. **`0`** — Java rellena lo que falta con ceros ✗ (eso solo pasa en arrays)
4. **Imprime `10` y `r` queda en `0`** — suena lógico, pero `void` no es un valor ✗

> 🔄 Solución

🔥 Fireside Chat: `void` vs `return`

> *Dos veteranos del recorrido discuten junto a la máquina de caf.*

`void`: - Yo hago y me callo. Imprimir, guardar, borrar: efectos puros. El mundo me ve cambiar y no me necesita devolver nada.

`return`: - Y por eso cuando te necesitan para **calcular**, se pierden. A mí me guardan en variables, me comparan, me paso como argumento a otros diez métodos. Soy datos de verdad.

`void`: - Los efectos también son datos… bueno, efectos.

`return`: - ¿Y qué pasa cuando un novato te usa a mí pero le pones `System.out.println` dentro? Resultado: nadie recibe el valor, se imprime en el momento exacto en que a veces **no toca**, y el que llama no puede hacer nada con él.

`void`: - Eso no es culpa mía, es culpa de mezclar los oficios.

`return`: - Exacto. Calculas → devuelves. Decides → imprimes. Si tu método empieza por `calcular`, `obtener` o `es`, ven a mi casa.

`void`: - Trato. Yo para `imprimir` y `guardar`; tú para todo lo que otro método vaya a necesitar. Que el `main` sea el único que hable en voz alta.

> La lección: **`void` para efectos, `return` para valores.** Quien calcula devuelve; quien decide imprime.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy la variable que un método recibe al ser llamado, declarada en su firma.**
2. **Soy el valor que llega en la llamada, en el sitio exacto donde aprietas el gatillo.**
3. **Soy la región donde una variable existe: de la firma a la llave de cierre.**
4. **Soy el final del método: cierro la ejecución y entrego el valor prometido.**
5. **Soy la firma completa: nombre + tipos de los parámetros, mi carta de presentación.**
6. **Soy un método que devuelve `true` o `false`: `esPar`, `tieneSuspensa`, `todoPositivo`.**

> 🔄 **Respuestas**

🤖 CONRAD VS EL MUNDO: “El main que se hizo el listo”

> *CONRAD, nuestro compilador cascarrabias, ha encontrado una nota en la bandeja de errores.*

CONRAD: - ¡OTRA VEZ! Un alumno me dice: *CONRAD, quiero llamar a mi método desde `main`*. Y yo: *¿y el `static`? Pues… lo quité, que me parecía repetir.* ¡SIN `static` NO HAY `main`! `non-static method cannot be referenced from a static context`. El `main` es estático de nacimiento; hasta que no aprendas a crear objetos (U09, U10), todos tus métodos llevan `static`. No es decoración, es la entrada obligada.

Y luego el clásico del retorno: mi método calcula bien pero me da `missing return statement`. ¿Y los caminos? *¿Caminos?* ¡Claro! Si el `if` devuelve en una rama y el `else` no, hay un sendero que llega al final con la firma sin cumplir. Java no adivina tu intención: **todos los caminos cierran**.

Y el de la variable fantasma: en `main` me dice `cannot find symbol: variable total`… pero la declaré en `sumar()`. ¡PORQUE ES DE `sumar()`! Las locales mueren con su método. No hay ventanillas laterales entre casas: solo argumentos que entran y `return` que sale.

Y el que más me hincha: llaman `saludar()`… **sin `static`**… **sin `()`**… desde otro método… ¡LEED EL MENSAJE ENTERO, QUE LA PRIMERA LÍNEA DICE QUÉ Y LA SEGUNDA DÓNDE!

> CONRAD añade: si tu error desaparece al cambiar **una letra**, era un typo; si desaparece al añadir `()`, era la llamada; si desaparece al añadir `static`, era el contexto. El compilador te está dando clases gratis: léelas.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. ¿Cuál es la diferencia real entre parámetro y argumento?
   - a) Ninguna, son sinónimos b) Declaración vs llamada c) Uno es de `int` y otro de `String`
2. ¿Qué imprime `System.out.println(truco(7));` con `truco` del punto 4?
   - a) `10` b) `14` c) `12`
3. ¿Pueden `a()` y `b()` tener ambas una variable `x`?
   - a) No: choca b) Sí, son casas distintas c) Solo si son `static`
4. ¿Qué error da `int r = suma(1, 2);` si `suma(int a, int b, int c)`?
   - a) `wrong number of arguments` b) Nada, el tercero queda a 0 c) `missing return statement`
5. ¿Cuándo usas `void`?
   - a) Cuando no devuelves nada útil, solo haces un efecto b) Cuando no sabes qué devolver c) Cuando hay un solo parámetro
6. ¿Qué hace `return;` dentro de un método `int`?
   - a) Devuelve 0 b) No compila c) Devuelve `null`

> › 🔄 **Soluciones**

⚡ Laboratorio de tortura: el programa sin nombre

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que imprima `Media: 7.5`. Tiene **2 errores de compilación** y **1 error de lógica** que solo se nota en la salida.

```java
public class Tortura {
    public static void main(String[] args) {
        int r = media(6, 7, 8, 9);
        System.out.println("Media: " + r);
    }

    public static int media(int a, int b, int c, int d) {
        double m = (a + b + c + d) / 4;
        return m;
    }
}
```

Tu tarea: que compile y que imprima `Media: 7.5`. Si imprime `7`, has encontrado la lógica; si no compila, son las dos de compilación.

Pistas para cuando te frustres (no antes):

1. ¿Cuántos parámetros declara `media` y cuántos argumentos le pasas? *si cuadra, sigue.*
   > **¿Y si sigo atascado?**
2. ¿Qué dice exactamente el error de la línea `return m;`?
   > **¿Y si sigo atascado?**
3. ¿Compila ya, pero imprime `Media: 7`?
   > **¿Y si sigo atascado?**

> 🔄 **Solución completa**

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🗣️ **El Traductor**	Explicar en voz alta la diferencia entre parámetro y argumento sin dudar
📤 **El Devolvedor**	Escribir un predicado (`esPar`) que cierre todos sus caminos a la primera

Logro	Cómo conseguirlo
🏠 **El Albañil de Ámbitos**	Diagnosticar un `cannot find symbol` de un vistazo (typo o fuera de casa)
🧯 **El Bombero**	Arreglar los 7 tropiezos del punto 6 sin mirar la tabla
🧱 **El Director de Orquesta**	Trocear un `main` de 30 líneas en 4 métodos con salida idéntica
🛠️ **El Refactorizador**	Completar el Be the Code con la salida byte a byte igual que al inicio

🤔 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime?

```
public class Misterio {
    static int f(int x) {
        if (x > 2) {
            return x + 1;
        }
        return f(x + 1);
    }

    public static void main(String[] args) {
        System.out.println(f(0));
    }
}
```

> 🔁 **Respuesta**

2. ¿Por qué decimos que `main` debe leerse “como un resumen” y no como una receta?

3. Si todos tus métodos imprimen, ¿qué le falta a tu programa para ser reutilizable?

4. ¿Se te ocurre un caso legítimo en que un método `void` llame a otro `void` y el programa completo tenga sentido sin un solo `return` con valor?

🧩 Crucigrama de bits

```
Horizontal:
1. Lo que declara el método: tipo + nombre de cada entrada (10 letras)
3. Método que devuelve true o false (9 letras)
5. Tipo de dato sin valor: "____ media(3, 4);" (4 letras)
6. Donde vive una variable: su _______ (6 letras)

Vertical:
2. Lo que pasa en la llamada: "sumar(3, 4)" son ____ (9 letras)
4. Promesa de la firma que cumple return (7 letras)
7. Palabra que cierra el método y entrega (6 letras)
```

> 🔄 Soluciones

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, la diferencia entre un parámetro y un argumento."**
2. **"Escribe un método que devuelva `true` si todos los números de una lista son positivos. ¿Por qué `boolean` y no un `println`?"**
3. **"¿Qué es el ámbito de una variable y por qué existe?"**
4. **"Tu método compila pero el resultado no llega a `main`. ¿Por qué puede ser?"**
5. **"¿Cuándo harías un método `void` y cuándo con retorno? Dame un ejemplo de cada uno."**
6. **"Refactoriza un `main` de 40 líneas. ¿Por dónde empiezas y cómo compruebas que no lo has roto?"**

🤷 No hay preguntas tontas

> ❓ **¿Por qué Java me obliga a poner `static` si yo solo quiero llamar a mi método?**

Porque `main` es estático: se ejecuta **sin** crear ningún objeto. Un método no estático pertenece a un objeto, y no hay objeto al que pertenecer. De momento todos tus métodos son de clase (`static`); cuando crees tu primer objeto en la U09 y veas `static` en detalle en la U10, esto hará *clic* y no volverás a dudar.

> ❓ **¿Puedo llamar a un método desde otro método que está encima o debajo?**

Sí, sin restricciones de orden: todo el mundo se conoce dentro de una clase. `main` puede llamar a `calcularMedia` aunque esté escrita 50 líneas más abajo.

> ❓ **Si paso `x` por parámetro, ¿por qué mi método no puede cambiar mi `x`?**

Porque recibe una **copia** (para tipos primitivos). Es un diseño deliberado: los métodos no reescriben lo de nadie a escondidas; si hay un resultado nuevo, lo **devuelven** con `return`. Con arrays la cosa cambia (se pasa la referencia) y lo verás en la U06.

> ❓ **¿Un método puede llamar a sí mismo?**

Puede, y se llama **recursión**. Es potente y peligroso: si nunca llega a su caso base, se acaba la pila y explota. Toca en la U08, Algorítmica II, con guantes.

🎬 Poscréditos

El novato termina de trocear su informe de notas: `main` tiene cuatro líneas, cada verbo vive en su sitio y la salida es idéntica a la de la mañana. Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: - Vaya. Cuatro métodos, cero bucles en `main` y ni un solo `println` donde no tocaba. ¿Seguro que eres el mismo que empezó la unidad con todo en 40 líneas seguidas?

Novato: - *sonríe* Empecé copiando y pegando, y he terminado con un programa que se lee como una lista de la compra. ¿Y ahora qué?

CONRAD: - *toma un sorbo* Ahora esos datos quieren crecer: cinco notas se han convertido en cien, y guardarlos en variables sueltas es un infierno. Necesitas un aparcamiento de verdad: arrays, índices y `length`. Otro plato, y viene justo después.

El novato guarda su proyecto, cierra el IDE y siente que ya no escribe bloques de código: **escribe planes con nombres**.

PRÓXIMAMENTE EN U06: Arrays. El aparcamiento de datos del mismo tipo: crear, rellenar, recorrer y no salirse nunca del último índice. 🅿️

📚 Volver al índice de la unidad · **Anterior:** 08 · Be the Code · **Siguiente:** U06 · Arrays

U06 — Arrays

El aparcamiento de datos: crea, recorre, ordena y busca con los arrays de Java 🅿️

Bienvenido al aparcamiento. En las unidades anteriores aprendiste a **decidir** (U04) y a **repetir**, pero siempre guardando una cosa por variable. Hasta hoy. En esta unidad descubres los **arrays**: la forma de guardar cientos (o millones) de datos del mismo tipo bajo un solo nombre y de moverlos con un bucle.

Piensa en todo lo que es, en el fondo, una lista de cosas: las notas de tus 30 alumnos, la temperatura de cada día del año, las fichas de un tablero, los nombres de una clase. Pues todo eso, llevado a Java, es esta unidad: **crear** el array, **recorrerlo**, **ordenarlo**, **buscarlo**, manejarlo con la clase `Arrays` y pasarlo a **métodos** sin que se te escape nada por el camino.

Y, como siempre, CONRAD el compilador estará por aquí para recordarte que un índice fuera de rango es el error más caro (y más típico) del curso: `ArrayIndexOutOfBoundsException` no perdona.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- **Crear** arrays de cualquier tipo, inicializarlos y entender sus valores por defecto (`0`, `false`, `null`…).
- **Recorrer** un array con `for` y con `for-each`, sabiendo cuándo usar cada uno.
- Trabajar con **arrays multidimensionales** (filas y columnas, tableros, matrices) y con arrays irregulares.
- Aprovechar la clase **`Arrays`**: `toString`, `sort`, `binarySearch`, `copyOf`, `fill` y `equals`.
- **Pasar arrays a métodos** y entender el paso por referencia de la copia.
- Aplicar los arrays a casos reales: listas de nombres, notas, objetos propios y datos en tabla.
- **Depurar** los errores típicos de los arrays: índices fuera de rango, `null`, imprimir sin `toString`…

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · Arrays: el aparcamiento de datos	Crear arrays, índices, `length` y el primer BOOM	Todos
02 · Recorrer arrays: for y for-each	El dúo inseparable y la variante perezosa de solo lectura	Todos
03 · Arrays multidimensionales	Arrays de arrays, filas, columnas y bucles anidados	Todos
04 · La clase Arrays: tu navaja suiza	`toString`, `sort`, `binarySearch`, `copyOf`, `fill`, `equals`	Todos
05 · Arrays y métodos	Pasar arrays por referencia y devolverlos	Todos
06 · Aplicaciones de los arrays	Arrays de String, de char, de objetos y datos en tabla	Todos
07 · Be the Code: el parking se gestiona	Programar inverso, búsquedas y compactar a mano	Todos
08 · Array-revelde: errores comunes y depuración	La galería de monstruos de los arrays y cómo cazarlos	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Quién Soy, Laboratorio, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA6)

RA6: Escribe programas que manipulen información, seleccionando y utilizando tipos avanzados de datos.

CE	Criterio	Dónde se cubre
RA6 a)	Se han escrito programas que utilicen arrays.	✅ Todos
RA6 b)	Se han reconocido las librerías de clases relacionadas con tipos avanzados de datos.	✅ Punto 4

> 📌 Los arrays son la base de todo lo que viene: los algoritmos de la U07 buscan y ordenan arrays, y cuando un parking de tamaño fijo se te queda pequeño, las colecciones de la U12 (ArrayList, LinkedList…) toman el relevo. Aprender arrays a fondo hoy es la inversión que renta durante todo el curso.

🚪 ¿Por dónde empiezo?

- ¿Cero conocimientos? → Arranca en el punto 1. Solo necesitas variables, bucles y `if` de la U04.
- ¿Ya sabes crear arrays y quieres chicha? → Salta al recorrido o a la clase Arrays.
- ¿Solo quieres ver arrays en acción real? → Lee Aplicaciones de los arrays.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Arrays: el aparcamiento de datos

⏭️ **Al acabar la unidad, continúa en U07 · Algorítmica I: Fundamentos.**

01 · Arrays: el aparcamiento de datos

Declaración, índices y el primer BOOM: la ArrayIndexOutOfBoundsException que te estrellará alguna vez 🅿️

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 01 · Arrays: el aparcamiento de datos

---

📬 La idea en una frase

> **Un array es un aparcamiento de tamaño fijo: guarda muchos datos del mismo tipo bajo un solo nombre, y cada plaza tiene un número (el índice) que empieza en 0.**

Hasta ahora guardabas una cosa por variable. Con los arrays guardas 100 bajo el mismo cartel. Es la primera herramienta "de verdad" para manejar cantidades, y de ella cuelga todo lo que viene después.

😾 El problema: tienes 100 gatos y un solo nombre

Imagina que tienes 100 gatos y necesitas guardar sus nombres. Podrías hacer esto:

```
String gato1 = "Bigotes";
String gato2 = "Garfield";
String gato3 = "Misifú";
// ... 97 líneas después ...
String gato100 = "Calcetines";
```

Pero entonces llega el gato 101 y tu programa se cae. O peor: quieres saber cuántos gatos empiezan con "M" y tienes que escribir 100 `if`. Tu espalda ya duele solo de pensarlo.

> ⚠️ **Advertencia:** si alguna vez escribes `gato1`, `gato2`, `gato3`… `gatoN` en tu código, en algún lugar un programador senior llora. Los arrays existen exactamente para esto.

🅿️ El array: tu primer aparcamiento

Un array es como un parking de varias plantas. Cada plaza tiene un número (el **índice**) y en cada plaza solo caben coches del mismo tipo (bueno, y sus subclases).

```
String[] gatos = new String[100];
// Has creado un parking con 100 plazas para Strings
```

Hay dos formas de declararlo y crearlo:

```
int[] numeros = new int[5];      // 5 plazas, todas vacías (0)
int[] directo = {10, 20, 30};    // 3 plazas, ya ocupadas
```

La primera plaza es la **0**, no la 1. Esto confunde a todo el mundo al principio. Acéptalo.

> 💡 **Consejo:** piensa en los índices como distancias desde la primera posición. La primera casa está a 0 pasos de ti, no a 1.

Los valores por defecto

Cuando creas un array con `new`, cada plaza se llena con el valor por defecto del tipo:

Tipo	Valor por defecto
`int`, `long`, `short`, `byte`	`0`
`double`, `float`	`0.0`
`boolean`	`false`
`char`	`'\U0100'`
Objetos (`String`, `Persona`…)	`null`

Ese último es el que muerde: un array de `String` recién creado está lleno de `null`, no de `""`. Si intentas llamar a un método sobre una plaza null, te llevas un `NullPointerException` al instante.

🚙 Cómo meter cosas en el parking

Se accede a una plaza con corchetes y el número de índice:

```
String[] gatos = new String[3];
gatos[0] = "Bigotes";
gatos[1] = "Garfield";
gatos[2] = "Misifú";
gatos[3] = "Calcetines"; // ¡BOOM!
```

¿Qué pasa en la última línea? Te vas a estrellar.

¡BOOM! La ArrayIndexOutOfBoundsException

```
int[] numeros = new int[5];
numeros[0] = 10;
numeros[1] = 20;
numeros[2] = 30;
numeros[3] = 40;
numeros[4] = 50;
numeros[5] = 60; // Index 5 out of bounds for length 5
```

El array tiene plazas del 0 al 4. Pedir la 5 es como intentar aparcar donde no hay plaza. Java te responde con `ArrayIndexOutOfBoundsException` y tu programa muere en el acto. Es **la** excepción más típica de esta unidad y la primera que casi todo el mundo sufre.

> 📝 **Nota:** los índices válidos van de `0` a `length - 1`. El último elemento siempre es `arr[arr.length - 1]`. No te lo pienses dos veces: memorízalo.

La longitud: `length` sin paréntesis

Para saber cuántas plazas tiene el parking:

```
int[] numeros = new int[10];
System.out.println(numeros.length);  // 10 (sin paréntesis)
```

Fíjate: `array.length` NO lleva paréntesis. No es un método, es un atributo. Los `String` usan `length()`. Los arrays usan `length`. Es una trampa mortal en los exámenes.

> ⚠️ **Advertencia:** el array es un objeto (está en el heap), pero la variable que lo referencia está en la pila (stack). Cuando pasas un array a un método, pasas la referencia, no los datos. Lo verás en detalle en el punto 5, pero ya lo sabes: si modificas el array dentro de un método, los cambios afectan al original.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** los arrays tienen tamaño fijo. Una vez creados, no puedes añadir ni quitar elementos. Si lo necesitas, se crea otro y se copia. En la U12 verás las colecciones, que solucionan eso.

Ejercicio: el array que se duplica

```
public class BeTheArray {
    public static void main(String[] args) {
        int[] arr = new int[4];
        arr[0] = 2;
        arr[1] = 4;
        arr[2] = 6;
        arr[3] = 8;

        for (int i = 0; i < arr.length; i++) {
            arr[i] = arr[i] * 2;
        }

        System.out.println(arr[2]);
    }
}
```

¿Qué imprime?

- (A) 6
- (B) 8
- (C) 12
- (D) 16

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuántas plazas tiene `int[] a = new int[7]` y cuáles son los índices válidos?

2. ¿Qué valor tiene `boolean[] b = new boolean[3]` en cada plaza?
3. ¿Qué excepción lanza `arr[arr.length]`?
4. ¿`numeros.length` o `numeros.length()`? ¿Y para un `String`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Un **array** guarda muchos valores del mismo tipo bajo un solo nombre, en un aparcamiento de **tamaño fijo**.
2. Se accede por **índice** desde `0` hasta `length - 1`; pasarse provoca `ArrayIndexOutOfBoundsException`.
3. La **longitud** se pregunta con `length` (sin paréntesis) y las plazas recién creadas se llenan con los valores por defecto (`0`, `false`, `null`…).

> 🦎 **Vocabulario rápido**

Término	Idea general
Array	Aparcamiento de datos del mismo tipo, tamaño fijo
Índice	Número de la plaza; empieza en 0
`length`	Tamaño del array (atributo, sin paréntesis)
`ArrayIndexOutOfBoundsException`	Error al salirte de las plazas válidas
Valor por defecto	Lo que ocupa una plaza recién creada (`0`, `false`, `null`…)

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · Recorrer arrays: for y for-each

02 · Recorrer arrays: for y for-each

El dúo inseparable (for + array) y la variante perezosa que solo lee: el for-each 🌀

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 02 · Recorrer arrays: for y for-each

---

📬 La idea en una frase

> **Un array sin un bucle es un parking que nadie visita: el `for` recorre plaza a plaza por índice, y el `for-each` hace lo mismo pero sin índice y solo para leer.**

En el punto 1 aprendiste a crear el parking. Ahora toca lo divertido: pasear por todas las plazas y hacer algo con cada coche. Y aquí manda un dúo tan inseparable como el pan y la mantequilla.

🔗 El dúo inseparable: for + array

Los arrays y los bucles `for` se ven siempre juntos. No es casualidad: el `for` tiene un contador natural (`i`) que encaja perfecto con los índices del array.

```
String[] gatos = {"Bigotes", "Garfield", "Misifú", "Calcetines"};

for (int i = 0; i < gatos.length; i++) {
    System.out.println("Gato " + i + ": " + gatos[i]);
}
```

Salida:

```
Gato 0: Bigotes
Gato 1: Garfield
Gato 2: Misifú
Gato 3: Calcetines
```

Fíjate en la condición: `i < gatos.length`. Si escribieras `i <= gatos.length`, en la última vuelta pedirías la plaza `length` y… ¡BOOM! `ArrayIndexOutOfBoundsException`. Es el error de bucle más cometido del universo.

Patrones clásicos con for

El `for` con índice no solo sirve para imprimir. Estos tres patrones se repiten en cada ejercicio del curso:

Sumar todos los elementos:

```
int[] notas = {7, 8, 5, 9, 6};
int suma = 0;
for (int i = 0; i < notas.length; i++) {
    suma += notas[i];
}
System.out.println("Media: " + (double) suma / notas.length);
```

Buscar un valor (búsqueda lineal):

```
int[] edades = {12, 45, 7, 34, 89};
int buscado = 34;
int posicion = -1;
for (int i = 0; i < edades.length; i++) {
    if (edades[i] == buscado) {
        posicion = i;
        break;
    }
}
System.out.println(posicion >= 0 ? "Encontrado en " + posicion : "No encontrado");
```

Modificar el array en el sitio (esto solo se puede con índice):

```
int[] numeros = {1, 2, 3, 4, 5};
for (int i = 0; i < numeros.length; i++) {
    numeros[i] = numeros[i] * 10;
}
// {10, 20, 30, 40, 50}
```

 for-each: la variante perezosa

Si no necesitas el índice (solo quieres leer los valores), existe una sintaxis más corta:

```java
String[] gatos = {"Bigotes", "Garfield", "Misifú"};

for (String gato : gatos) {
    System.out.println("Miau: " + gato);
}
```

Se lee: “para cada String gato en gatos, haz esto”. La variable `gato` va tomando el valor de cada plaza una a una, sin que tú gestiones contadores ni corchetes.

> 📝 **Nota:** el for-each es **de solo lectura**. No puedes modificar el array original dentro del bucle. Bueno, puedes intentarlo, pero el cambio se pierde en el éter: `gato = "Nuevo"` solo cambia la variable local, no la plaza del array. Para modificar, usa el `for` con índice.

¿Cuándo uso cada uno?

Situación	Bucle recomendado
Solo leer y no me importa la posición	`for-each`
Necesito el índice (posiciones, comparar vecinos)	`for` clásico
Quiero modificar los elementos del array	`for` clásico
Recorrer hacia atrás o de dos en dos	`for` clásico
Recorrer una colección (`ArrayList`, `HashSet`…)	`for-each` (U12 Colecciones)

> 💡 **Consejo:** si no necesitas el índice, usa `for-each`. Es más corto, más legible y te ahorra una clase entera de errores (olvidar el ++, empezar en 1, escribir <=…).

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** el for-each es como un robot que va por el parking leyendo las matrículas. Solo lee: no puede repintar los coches.

Ejercicio: la suma de los pacientes

```java
public class BeTheForEach {
    public static void main(String[] args) {
        int[] numeros = {10, 20, 30, 40, 50};
        int total = 0;

        for (int n : numeros) {
            if (n % 20 == 0) {
                total += n;
            }
        }

        System.out.println(total);
    }
}
```

¿Qué imprime?

- (A) 60
- (B) 90
- (C) 120
- (D) 150

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué imprime `for (int i = 0; i < a.length; i++)` sobre `{1,2,3}` si imprimes `a[i]`?
2. ¿Por qué `for (int i = 0; i <= a.length; i++)` lanza excepción?
3. ¿Se puede modificar un array con `for-each`?
4. ¿Qué bucle usarías para imprimir el array al revés?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. El **`for` clásico** recorre el array con un índice (`i`) desde `0` hasta `length - 1`, y es el único que permite **modificar** elementos.
2. El **`for-each`** lee todos los valores sin índice: perfecto para sumar, contar o imprimir, pero es **de solo lectura**.
3. La condición del bucle es `i < length`; si escribes `<=`, te estrellas contra `ArrayIndexOutOfBoundsException`.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Recorrer | Visitar cada elemento del array, uno a uno |
> | `for` clásico | Bucle con índice para leer, modificar o buscar |
> | `for-each` | Bucle de solo lectura: "para cada X en Y" |
> | Búsqueda lineal | Recorrer de principio a fin buscando un valor |
> | `break` | Cortar el bucle en el momento en que encuentras lo buscado |

📚 Volver al índice de la unidad · **Anterior:** 01 · Arrays: el aparcamiento de datos · **Siguiente:** 03 · Arrays multidimensionales

03 · Arrays multidimensionales

El parking de varias plantas: arrays de arrays, filas y columnas, y cómo recorrerlos sin perderte 🏢

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 03 · Arrays multidimensionales

📬 La idea en una frase

> **Un array multidimensional es un array cuyos elementos son otros arrays: un aparcamiento de varias plantas, donde cada plaza se localiza por planta y número.**

Hasta ahora cada plaza guardaba un dato. ¿Y si el dato en sí es otro parking? Entonces tienes una tabla con filas y columnas. Esto es lo que usas para representar tableros, matrices, mapas y cualquier cosa con dos (o más) dimensiones.

🏢 El parking de varias plantas

Un array bidimensional es "un array de arrays". Se declara con doble corchete:

```
int[][] tabla = new int[3][4]; // 3 filas, 4 columnas
```

Piénsalo como un parking con **3 plantas** y **4 plazas** por planta. Para acceder a una plaza necesitas dos números: el de la planta (fila) y el de la plaza dentro de ella (columna).

```
tabla[0][0] = 1; // fila 0, columna 0
tabla[1][2] = 5; // fila 1, columna 2
```

También puedes crearlo ya lleno:

```
int[][] matriz = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

`matriz.length` es el número de filas (3). `matriz[0].length` es el número de columnas de la fila 0 (3).

> 💡 **Consejo:** nombra los índices de arrays multidimensionales como `fila` y `col`, o `i` y `j`. NO uses `x` e `y` a menos que realmente trabajes con coordenadas. Tu yo del futuro te lo agradecerá.

🎲 Arrays de arrays irregulares

Java permite "arrays de arrays" donde cada fila tiene un número distinto de columnas (los llamados *jagged arrays*, arrays dentados):

```
int[][] irregular = new int[3][];
irregular[0] = new int[2];
irregular[1] = new int[5];
irregular[2] = new int[3];
```

La fila 0 tiene 2 columnas, la fila 1 tiene 5 y la fila 2 tiene 3. ¿Para qué sirve? Triángulos, pirámides o simplemente datos que no forman un rectángulo perfecto (por ejemplo, los días de cada mes: febrero tiene menos).

```
int[][] diasPorMes = {
    {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31}, // un solo mes por fila
    {1, 2, 3},                                        // ejemplo de fila corta
};
```

En un array irregular, `irregular[i].length` puede ser distinto para cada `i`. Por eso los recorridos usan `matriz[i].length` dentro del bucle, nunca un número fijo.

🔁 Recorrer un array 2D: los bucles anidados

Para recorrer un rectángulo perfecto necesitas dos bucles: uno para las filas y otro para las columnas.

```java
int[][] matriz = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

for (int i = 0; i < matriz.length; i++) {          // filas
    for (int j = 0; j < matriz[i].length; j++) {   // columnas
        System.out.print(matriz[i][j] + " ");
    }
    System.out.println();
}
```

Salida:

```
1 2 3
4 5 6
7 8 9
```

El bucle exterior va fila a fila; el interior recorre cada columna de esa fila. Fíjate que el interior usa `matriz[i].length`: así funciona también con arrays irregulares.

Y la versión perezosa con for-each (solo lectura):

```java
for (int[] fila : matriz) {
    for (int valor : fila) {
        System.out.print(valor + " ");
    }
    System.out.println();
}
```

Cada `fila` es un `int[]`, y sobre él vuelves a usar for-each. Arrays de arrays, bucles de bucles.

 Para qué sirve de verdad

Los arrays 2D no son un capricho académico. Son la forma natural de representar:

Situación	Array
Tablero de juego (ajedrez, buscaminas, tres en raya)	`char[][]` o `boolean[][]`
Notas por alumno y asignatura	`double[][]`

Situación	Array
Mapa de píxeles de una imagen	`int[][]`
Matrices matemáticas	`int[][]`, `double[][]`

Un buscaminas simplificado, por ejemplo, es un `boolean[][]`:

```
boolean[][] minas = new boolean[5][5];
minas[2][3] = true;  // hay una mina en fila 2, columna 3
```

Y para saber si una posición existe, siempre preguntas antes de tocar: el índice de fila va de `0` a `length - 1` y el de columna de `0` a `matriz[fila].length - 1`. Salirte de ahí vuelve a ser `ArrayIndexOutOfBoundsException`, pero ahora con dos coordenadas.

> ⚠️ **Advertencia:** `matriz.length` y `matriz[0].length` NO son lo mismo. El primero son las filas; el segundo, las columnas de la fila 0. Confundirlos es el error clásico de los principiantes con matrices.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando recorres una matriz con dos bucles, el orden importa: `i` para filas, `j` para columnas. Si los intercambias, estás recorriendo la matriz traspuesta.

Ejercicio: la diagonal que no se ve

```java
public class BeTheDiagonal {
    public static void main(String[] args) {
        int[][] m = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        int suma = 0;
        for (int i = 0; i < m.length; i++) {
            suma += m[i][i];
        }
        System.out.println(suma);
    }
}
```

¿Qué imprime?

- (A) 12
- (B) 15
- (C) 18
- (D) 45

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuántas filas y columnas tiene `int[][] a = new int[3][4]`?
2. ¿Cómo accedes al elemento de la fila 2, columna 1?
3. ¿Qué representa `a.length` y qué representa `a[0].length`?
4. ¿Por qué en un array irregular el bucle interior usa `a[i].length`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Un array **bidimensional** es un array de arrays: se accede con dos índices `[fila][columna]`.
2. Se **recorre con dos bucles anidados**: el exterior para filas y el interior para columnas, usando `matriz[i].length`.
3. Java admite **arrays irregulares** donde cada fila tiene su propia longitud, y `matriz.length` y `matriz[i].length` no son lo mismo.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Array multidimensional | Array cuyos elementos son otros arrays |
> | Fila | Primera dimensión (el índice `i`) |
> | Columna | Segunda dimensión (el índice `j`) |
> | Array irregular | Cada fila con distinto número de columnas |
> | Diagonal principal | Los elementos donde `fila == columna` |

📚 Volver al índice de la unidad · **Anterior:** 02 · Recorrer arrays: for y for-each · **Siguiente:** 04 · La clase Arrays: tu navaja suiza

04 · La clase Arrays: tu navaja suiza

toString, sort, copyOf, binarySearch y fill: los métodos estáticos que te ahorran reinventar la rueda 🔧

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 04 · La clase Arrays: tu navaja suiza

📬 La idea en una frase

> **`java.util.Arrays` es una clase llena de métodos estáticos para trabajar con arrays: imprimir, ordenar, copiar, buscar y rellenar sin escribir tú el bucle.**

Los arrays tienen un problema: no tienen métodos. `numeros.sort()` no existe. Por eso Java te regala una clase de herramientas, todas estáticas, para que no tengas que reinventar el bucle cada vez. Es lo más parecido a una navaja suiza que existe en el mundo de los arrays.

🧰 La clase Arrays

Se importa con `import java.util.Arrays;` y sus métodos se llaman pasándole el array:

```
import java.util.Arrays;

public class EjemploArrays {
    public static void main(String[] args) {
        int[] numeros = {5, 2, 8, 1, 9};

        Arrays.sort(numeros);                    // {1, 2, 5, 8, 9}
        String texto = Arrays.toString(numeros); // "[1, 2, 5, 8, 9]"
        System.out.println(texto);
    }
}
```

Recuerda: como son métodos `static` (lo viste en la U09), se llaman con el nombre de la clase: `Arrays.xxx(array)`. No hace falta crear ningún objeto `Arrays` — de hecho, no puedes.

🔍 Arrays.toString: imprimir bonito

El método más socorrido. Sin él, imprimir un array muestra basura:

```java
int[] numeros = {1, 2, 3};
System.out.println(numeros);           // [I@6d06d69c  (dirección de memoria, inútil)
System.out.println(Arrays.toString(numeros)); // [1, 2, 3]
```

Sin `toString`, Java imprime la dirección de memoria del objeto (`[I@6d06d69c`), no los datos. Con él, obtienes algo legible. Para arrays 2D existe `Arrays.deepToString()`.

> ⚠️ **Advertencia:** `numeros.toString()` tampoco funciona: los arrays no sobreescriben `toString()`. Siempre `Arrays.toString(numeros)`.

📊 Arrays.sort: ordenar de un plumazo

Ordena el array en el sitio, de menor a mayor (según el orden natural del tipo):

```java
int[] notas = {7, 3, 9, 5};
Arrays.sort(notas);
System.out.println(Arrays.toString(notas)); // [3, 5, 7, 9]
```

Con `String` ordena alfabéticamente. Ojo con las mayúsculas: `"Zebra"` va antes que `"abc"` porque las mayúsculas tienen menor valor Unicode. Es ordenación lexicográfica, no "de diccionario humano".

🔎 Arrays.binarySearch: buscar rápido (pero solo ordenado)

La búsqueda binaria parte el array por la mitad en cada paso. Es rapidísima, pero **exige que el array esté ordenado antes**.

```java
int[] numeros = {3, 5, 7, 9, 11};
int pos = Arrays.binarySearch(numeros, 7);
System.out.println(pos); // 2
```

Si el valor no está, devuelve un número negativo (`-(puntoDeInsercion) - 1`). Ojo: si el array no está ordenado, el resultado es impredecible. Ordena antes de buscar, siempre.

📋 Arrays.copyOf: copiar con talla nueva

Crea un **nuevo** array con los primeros `n` elementos (o todos más ceros si pides más de los que hay):

```
int[] original = {1, 2, 3, 4, 5};
int[] recortado = Arrays.copyOf(original, 3); // {1, 2, 3}
int[] alargado = Arrays.copyOf(original, 8);  // {1, 2, 3, 4, 5, 0, 0, 0}
```

Es la forma civilizada de “cambiar el tamaño” de un array, que como sabes es fijo: creas uno nuevo y copias.

🧽 Arrays.fill: rellenar todo de golpe

Pone el mismo valor en todas las plazas:

```
int[] tabla = new int[10];
Arrays.fill(tabla, 0);     // todo a ceros
Arrays.fill(tabla, 7);     // todo a sietes
```

Útil para inicializar tableros, resetear marcadores o preparar un array antes de usarlo.

⚖️ Arrays.equals: comparar contenido, no referencias

Este es el que más bugs evita. `array1.equals(array2)` NO compara los elementos: compara si son el MISMO objeto en memoria. Usa SIEMPRE `Arrays.equals()`:

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};

System.out.println(a.equals(b));            // false (objetos distintos en memoria)
System.out.println(Arrays.equals(a, b));    // true (mismo contenido)
```

Tu jefe te lo agradecerá.

🧭 El mapa de la navaja

Método	Qué hace	Ojo con
`Arrays.toString(arr)`	Imprime el array legible	No usar `arr.toString()`
`Arrays.sort(arr)`	Ordena en el sitio	Modifica el original

Método	Qué hace	Ojo con
`Arrays.binarySearch(arr, v)`	Busca por índice	Requiere array ordenado
`Arrays.copyOf(arr, n)`	Nuevo array con n elementos	Crea copia, no toca el original
`Arrays.fill(arr, v)`	Rellena todo con v	Sirve para inicializar
`Arrays.equals(a, b)`	Compara contenido	No confundir con ==
`Arrays.deepToString(arr2d)`	Imprime arrays 2D	Versión profunda del toString

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** `Arrays.binarySearch()` requiere el array ORDENADO. Si no, el resultado es impredecible. Es como buscar en un diccionario que no está alfabético: no encontrarás nada fiable.

Ejercicio: la búsqueda que lo tenía todo

```
import java.util.Arrays;

public class BeTheSort {
    public static void main(String[] args) {
        int[] datos = {42, 17, 8, 99, 3};
        Arrays.sort(datos);

        int indice = Arrays.binarySearch(datos, 42);
        System.out.println(indice);
    }
}
```

¿Qué imprime?

- (A) 0
- (B) 3
- (C) 4

- (D) 99

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué devuelve `Arrays.binarySearch` si el valor no está en el array?
2. ¿`a.equals(b)` y `Arrays.equals(a, b)` hacen lo mismo?
3. ¿Qué hace `Arrays.copyOf(arr, 10)` si `arr` tiene 4 elementos?
4. ¿Por qué `System.out.println(arr)` no imprime los datos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. `java.util.Arrays` es la caja de herramientas **estática** de los arrays: se importa y se usa sin crear objetos.
2. Los cinco imprescindibles: `toString` (imprimir), `sort` (ordenar), `copyOf` (copiar), `binarySearch` (buscar, requiere orden) y `fill` (rellenar).
3. Para **comparar contenido** usa `Arrays.equals`, nunca `equals` ni `==`, que comparan referencias.

> 🦎 **Vocabulario rápido**

Término	Idea general
Clase utilitaria	Clase con métodos estáticos que no se instancia (como `Math`)
Orden natural	El orden que el tipo define por defecto (numérico, alfabético)
Búsqueda binaria	Búsqueda que parte el array por la mitad; requiere orden
Ordenación lexicográfica	Orden alfabético según el valor de los caracteres
Copia	Nuevo array independiente con los mismos valores

📚 Volver al índice de la unidad · **Anterior:** 03 · Arrays multidimensionales · **Siguiente:** 05 · Arrays y métodos

05 · Arrays y métodos

Pasar arrays a métodos y el paso por referencia: el testigo que se comparte, no se copia 🏃

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 05 · Arrays y métodos

---

📬 La idea en una frase

> **Cuando pasas un array a un método, pasas la referencia, no los datos: el método comparte tu parking y, si toca un coche, el cambio se ve fuera.**

Aquí se te van a caer las gafas (o se te van a poner de moda). En la U09 viste que Java pasa los argumentos **por valor**. Pero los arrays parecen llegar por referencia… La clave: se pasa por valor la **copia de la referencia**. El array no se copia; solo se copia la dirección donde está.

🏃 Pasando el testigo

Mira este programa y su salida:

```
public class ArraysMetodos {
    public static void main(String[] args) {
        int[] edades = {10, 20, 30};
        modificar(edades);
        System.out.println(edades[0]); // 99
    }

    public static void modificar(int[] arr) {
        arr[0] = 99;
    }
}
```

Imprime `99`, no `10`. El método `modificar` ha cambiado la plaza 0 del array `edades`… aunque el array se creó en `main`. ¿Cómo es posible si Java pasa por valor?

La explicación: **la referencia** (la dirección de memoria donde vive el array) se copia al pasar al método. La copia y el original apuntan al **mismo parking**. Cuando el método hace `arr[0] = 99`, no cambia su copia de la referencia: cambia el contenido del objeto al que ambos apuntan.

> 📝 **Nota:** esto NO es un paso por referencia de verdad. Java nunca pasa la variable por referencia. Pasa una copia de la referencia (por eso se llama *pass-by-value*). Pero como el array es un objeto, esa copia apunta al mismo sitio.

🆚 Primitivos vs arrays: la diferencia clave

Compara el comportamiento:

```
public class PasoDeDatos {
    public static void main(String[] args) {
        int numero = 5;
        cambiarNumero(numero);
        System.out.println(numero);   // 5 (el primitivo NO cambia)

        int[] arr = {1, 2, 3};
        cambiarArray(arr);
        System.out.println(arr[0]);   // 99 (el array SÍ cambia)
    }

    static void cambiarNumero(int n) {
        n = 99;                       // cambia la copia, el original sigue en 5
    }

    static void cambiarArray(int[] a) {
        a[0] = 99;                    // cambia el contenido del objeto compartido
    }
}
```

Tipo	Qué recibe el método	¿Se modifica fuera?
`int`, `double`, `boolean`…	Una copia del valor	No
`String`	Una copia de la referencia (String es inmutable)	No (por inmutable)
Array (`int[]`, `String[]`…)	Una copia de la referencia	Sí, si se modifican elementos
Objetos	Una copia de la referencia	Sí, si se modifican atributos

> ⚠️ **Advertencia:** si dentro del método haces `arr = otroArray`, NO cambias el array original: solo reasignas tu copia de la referencia. Para modificar el original, toca elementos (`arr[i] = ...`) o atributos del objeto, nunca la variable.

🧪 Métodos que usan arrays

Recibir un array para calcular

```
public static double media(int[] notas) {
    int suma = 0;
    for (int n : notas) {
        suma += n;
    }
    return (double) suma / notas.length;
}
```

Modificar un array (el cambio se ve fuera)

```
public static void duplicar(int[] arr) {
    for (int i = 0; i < arr.length; i++) {
        arr[i] *= 2;
    }
}
```

Devolver un array nuevo

Los métodos también pueden **devolver** arrays. Aquí se crea uno nuevo y se rellena:

```java
public static int[] primerosCuadrados(int n) {
    int[] resultado = new int[n];
    for (int i = 0; i < n; i++) {
        resultado[i] = (i + 1) * (i + 1);
    }
    return resultado;
}

// En el main:
int[] cuadrados = primerosCuadrados(4);
System.out.println(Arrays.toString(cuadrados)); // [1, 4, 9, 16]
```

Cuando devuelves un array, devuelves una **referencia** a un objeto del heap. Quien la recibe puede usar y modificar ese objeto. Por eso, si no quieres que te lo toquen, devuelve una copia (`Arrays.copyOf`).

🏁 El famoso `main(String[] args)`

Desde la UO2 llevas escribiendo `main(String[] args)` sin pararte a pensar. ¡Es un método que recibe un array de Strings! Esos son los argumentos de línea de comandos:

```java
public class Saludo {
    public static void main(String[] args) {
        System.out.println("Hola, " + args[0]);
    }
}
```

Si lo ejecutas con `java Saludo Ana`, `args` será `{"Ana"}` e imprimirá `Hola, Ana`. Si lo ejecutas sin argumentos y accedes a `args[0]`, `ArrayIndexOutOfBoundsException` — el mismo error del punto 1, ahora con la cara de `args`.

> 💡 **Consejo:** `args.length` te dice cuántos argumentos te han pasado. Comprueba siempre antes de acceder: `if (args.length > 0) { ... }`.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** los arrays se pasan por referencia (copia de la referencia). Las variables primitivas se pasan por valor. Si lo entiendes aquí, entiendes el 90% de los bugs raros del curso.

Ejercicio: el array al cuadrado

```
import java.util.Arrays;

public class BeTheArrayRevelde {
    public static void main(String[] args) {
        int[] nums = {1, 2, 3, 4, 5};
        for (int i = 0; i < nums.length; i++) {
            nums[i] = nums[i] * nums[i];
        }
        System.out.println(nums[2]);
    }
}
```

¿Qué imprime?

- (A) 3
- (B) 6
- (C) 9
- (D) 25

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué imprime este código: `int[] a = {1,2,3}; cambiar(a);` donde `cambiar(int[] x) { x[0] = 7; }`?
2. ¿Y si el método hace `x = new int[]{9,9,9}` en lugar de tocar `x[0]`?
3. ¿`String` se comporta como un array o como un primitivo al pasarlo a un método?
4. ¿Qué guarda realmente la variable `args` de `main`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los **arrays se pasan por referencia** (una copia de la referencia al objeto compartido): modificarlos dentro de un método se ve fuera.
2. Los **primitivos se pasan por valor**: el método recibe una copia y el original no cambia; los `String`, por ser inmutables, se comportan como ellos.
3. Los métodos pueden **devolver arrays** (una referencia a un objeto nuevo) y `main(String[] args)` es, en realidad, un método que recibe un array.

> 🦎 **Vocabulario rápido**
>
Término	Idea general
Paso por valor	Se copia el dato (primitivos, referencias)
Paso por referencia	Se comparte el objeto (efecto visible en arrays)
Referencia	Dirección de memoria donde vive el objeto
Alias	Dos variables que apuntan al mismo objeto
Argumentos de `main`	El array `args` con los parámetros de la línea de comandos

📚 Volver al índice de la unidad · **Anterior:** 04 · La clase Arrays: tu navaja suiza · **Siguiente:** 06 · Aplicaciones de los arrays

06 · Aplicaciones de los arrays

Arrays de String, de char, de objetos y de tablas: cuando el parking deja de ser teoría y empieza a trabajar 💼

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 06 · Aplicaciones de los arrays

📬 La idea en una frase

> **Los arrays no son solo para números: guardas nombres, palabras, caracteres, objetos y hasta tablas enteras. Todo lo que se pueda contar, se puede meter en un parking.**

Hasta ahora has aparcado números. Pero en la vida real los datos tienen nombres: alumnos, asignaturas, temperaturas, películas… Este punto te enseña a guardar **cosas** (no solo números) en tus arrays, y a darles un uso de verdad.

📝 Arrays de String: la lista de la clase

El array más humano que existe: una lista de nombres.

```
String[] clase = {"Ana", "Bruno", "Carla", "Diego"};

for (String alumno : clase) {
    System.out.println("Hola, " + alumno);
}
```

Cada plaza guarda una `String`. Nada nuevo en la sintaxis: lo que cambia es lo que guardas. Y con ellos puedes hacer cosas propias de texto:

```
String[] frutas = {"pera", "manzana", "pera", "uva"};

int peras = 0;
for (String fruta : frutas) {
    if (fruta.equals("pera")) {
        peras++;
    }
}
System.out.println("Hay " + peras + " peras.");
```

> ⚠️ **Advertencia:** con `String` nunca compares con `==`. Usa `.equals()`. Lo viste en la UO3 y en los arrays es igual de obligatorio: `fruta == "pera"` compara referencias, no texto.

El más famoso de todos: `args`

El array de `String` más usado del curso lo llevas escribiendo desde la UO2: `main(String[] args)`. En el punto 5 viste que contiene los argumentos de la línea de comandos. Pues eso: un `String[]` de verdad, como el de tu clase.

🔤 `char[]` vs `String`

Un `char[]` es un array de caracteres. Y ojo: se parece muchísimo a una `String`… pero no es lo mismo.

```
char[] vocales = {'a', 'e', 'i', 'o', 'u'};

for (int i = 0; i < vocales.length; i++) {
    System.out.print(vocales[i] + " ");
}
```

Cosa	`String`	`char[]`
¿Inmutable?	Sí: no se puede cambiar	No: puedes tocar cada plaza
Métodos	`length()`, `charAt()`, `substring()`…	No tiene (usas bucles)
¿Se puede modificar una letra?	No (creas otra)	Sí: `vocales[0] = 'A';`
Pasar a método	Se comporta como inmutable	Se comparte como cualquier objeto

> 💡 **Consejo:** si necesitas “cambiar una letra”, con `String` no puedes. Una opción es pasarla a `char[]`, modificarla y volver a construir la `String`. En la U14 (ficheros y regex) esta idea te va a salir varias veces.

🧍 Arrays de objetos: el parking de personas

Aquí es donde el parking brilla de verdad. Los arrays no solo guardan primitivos o `String`: guardan **objetos** de cualquier clase. Como aún no has creado tus propias clases (eso es la U08), usa una sencilla para el ejemplo:

```
class Alumno {
    String nombre;
    int nota;

    Alumno(String nombre, int nota) {
        this.nombre = nombre;
        this.nota = nota;
    }
}
```

```
Alumno[] alumnos = new Alumno[3];
alumnos[0] = new Alumno("Ana", 8);
alumnos[1] = new Alumno("Bruno", 5);
alumnos[2] = new Alumno("Carla", 10);

int aprobados = 0;
for (Alumno a : alumnos) {
    if (a.nota >= 5) {
        aprobados++;
    }
}
System.out.println("Aprobados: " + aprobados);
```

> ⚠️ **Advertencia:** un `Alumno[]` recién creado está lleno de `null`, no de alumnos. `new Alumno[3]` crea 3 plazas vacías; si accedes a `alumnos[0].nombre` sin crear el objeto antes, `NullPointerException` al instante. Primero `new Alumno(...)`, después usar.

El bucle es idéntico al de los números: cambia el contenido, no la mecánica. Esto se llama **recorrer una colección de objetos**, y vas a hacerlo muchísimo el resto del curso.

🗂️ Arrays de arrays: los datos en tabla

El último clásico: guardar varias listas a la vez. Si tienes las notas de varios alumnos en varias asignaturas, una tabla `double[][]` es la forma natural:

```
double[][] notas = {
    {8.0, 7.5, 9.0},   // Ana: Matemáticas, Lengua, Inglés
    {5.0, 6.0, 4.5},   // Bruno
    {9.5, 8.0, 10.0},  // Carla
};

for (int i = 0; i < notas.length; i++) {
    double suma = 0;
    for (int j = 0; j < notas[i].length; j++) {
        suma += notas[i][j];
    }
    System.out.println("Alumno " + i + ": media " + (suma / notas[i].length));
}
```

Cada fila es un alumno y cada columna una asignatura. Con dos bucles anidados (los del punto 3) recorres toda la tabla y calculas lo que necesites. Los programas que “llevan la cuenta” de la vida real son, en el fondo, esto.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando un array guarda objetos, el bucle no cambia nada: recorre, pregunta, acumula. Lo único nuevo es que preguntas por **atributos** (`a.nota`), no por valores sueltos.

Ejercicio: la búsqueda de la película

```java
public class BeTheCatalogo {
    public static void main(String[] args) {
        String[] peliculas = {"Alien", "Matrix", "Gladiator", "Matrix", "Coco"};
        String buscada = "Matrix";
        int cuantas = 0;

        for (String p : peliculas) {
            if (p.equals(buscada)) {
                cuantas++;
            }
        }

        System.out.println(cuantas);
    }
}
```

¿Qué imprime?

- (A) 1
- (B) 2
- (C) 3
- (D) Matrix

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué hay en las plazas de un `Alumno[]` recién creado?
2. ¿Por qué `fruta == "pera"` es un error con `String`?
3. ¿Puedes modificar una letra de una `String`? ¿Y de un `char[]`?
4. ¿Qué bucle usarías para sumar una fila de una tabla `double[][]`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los arrays guardan **lo que sea**: `String`, `char`, objetos propios y tablas enteras, con la misma mecánica de siempre.
2. Con `String` usa **`.equals()`** y recuerda que un array de objetos recién creado está lleno de **`null`**.
3. Los **arrays de objetos** (recorres atributos) y los **arrays 2D** (recorres filas y columnas) son la base de los programas que gestionan datos de la vida real.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Array de objetos | Parking donde cada plaza guarda un objeto (`Alumno`, `String`…) |
> | `null` | El valor por defecto de las plazas de objetos recién creadas |
> | Inmutable | Que no se puede cambiar; la `String` lo es |
> | `char[]` | Array de caracteres, modificable, sin métodos propios |
> | Tabla de datos | `double[][]`: filas y columnas para datos reales |

📚 Volver al índice de la unidad · **Anterior:** 05 · Arrays y métodos · **Siguiente:** 07 · Be the Code: el parking se gestiona

07 · Be the Code: el parking se gestiona

Invierte, busca y compacta arrays a mano, sin ayuda de la clase Arrays, como en una entrevista de trabajo 🕶️

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 07 · Be the Code: el parking se gestiona

---

📬 La idea en una frase

> **Este punto no tiene teoría nueva: tiene tres retos. Invertir un array sin mirar apuntes, buscar todas las posiciones de un valor y cazar los errores de un array que se rebela.**

La clase `Arrays` te lo da todo hecho: ordena, copia, busca… Pero cuando te piden "invierte este array" en una entrevista, nadie te deja usar `Arrays.sort()`. Hay que saber hacerlo a mano. Este punto es tu gimnasio.

🕶️ Don Tip: la receta mental del inverso en el sitio

Invertir un array es el clásico de las entrevistas. La receta mental:

1. Dos punteros: `izquierda = 0` y `derecha = array.length - 1`.
2. Mientras `izquierda < derecha`: intercambia `array[izquierda]` y `array[derecha]`.
3. `izquierda++` y `derecha--`.
4. Cuando se cruzan, ya está.

El error más común: crear un **array auxiliar** cuando no hace falta. En el sitio, con dos punteros y una variable temporal, es O(n) de tiempo y O(1) de memoria. Eso es lo que impresiona en una entrevista.

🧩 RETO 1: Invertir el array en el sitio

Completa el método `invertir` para que dé la vuelta al array **sin crear otro array**. Debe mostrar `5 4 3 2 1`:

```java
public class RetoInverso {

    public static void invertir(int[] array) {
        // 🧠 TU CÓDIGO AQUÍ
    }

    public static void main(String[] args) {
        int[] datos = {1, 2, 3, 4, 5};
        invertir(datos);
        for (int n : datos) {
            System.out.print(n + " ");
        }
    }
}
```

Pasos guiados (resiste a leerlos todos de golpe):

1. Declara los dos punteros.
   > **¿Atascado?**
2. Escribe el `while` con la condición correcta.
   > **¿Atascado?**
3. Intercambia los dos elementos con una variable temporal.
   > **¿Atascado?**
4. Mueve los punteros hacia el centro.
   > **¿Atascado?**

> **Solución completa**

🧩 RETO 2: Buscar todas las posiciones

`binarySearch` te da una posición. Este reto te pide **todas**. Escribe un método que devuelva un array con todos los índices donde aparece un valor (vacío si no aparece):

```java
public class RetoBusqueda {

    public static int[] posiciones(int[] datos, int buscado) {
        // 🧠 TU CÓDIGO AQUÍ
        return new int[0];
    }

    public static void main(String[] args) {
        int[] datos = {3, 7, 2, 7, 9, 7, 1};
        System.out.println(java.util.Arrays.toString(posiciones(datos, 7)));
        System.out.println(java.util.Arrays.toString(posiciones(datos, 5)));
    }
}
```

Debe mostrar `[1, 3, 5]` y `[]`.

Pasos guiados:

1. Primera pasada: cuenta cuántas veces aparece.
   > **¿Atascado?**
2. Crea el array de resultados con ese tamaño.
   > **¿Atascado?**
3. Segunda pasada: rellena las posiciones.
   > **¿Atascado?**

> 💡 **Consejo de depuración:** este "contar primero, crear después" es un patrón que se repite: no puedes crear el array de resultados hasta saber cuántas plazas necesita. Cuando el tamaño depende de los datos, se hace en dos pasadas.

> **🔄 Solución completa**

🧩 EL LÍO: el parking que se rebeló

El encargado del parking ha escrito esto para "dejar las plazas impares vacías". Algo huele mal. Encuentra los errores:

```
public class ParkingLioso {
    public static void vaciarImpares(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] % 2 == 1) {
                arr[i] = null;   // 🚨 ¿compila esto?
            }
        }
    }
}
```

> 👓 **Don Tip:** pregunta siempre: ¿qué tipo de dato guarda este array? Un `int` no puede valer `null`; un `Integer` sí (pero eso es un objeto de la U12).

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué condición usa el `while` del inverso en el sitio: `<` o `<=`?
2. ¿Qué pasa si usas `<=` en el inverso con un array de longitud impar?
3. ¿Por qué el patrón “contar primero, crear después” necesita dos pasadas?
4. ¿Puede un `int[]` guardar `null`? ¿Y un `String[]`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **Invertir en el sitio** es el clásico de entrevistas: dos punteros (`izquierda`/`derecha`), intercambio con `temp`, y `while (izquierda < derecha)`.
2. Cuando el tamaño del resultado **depende de los datos**, se usa el patrón de **dos pasadas**: contar primero, crear y rellenar después.
3. `null` no cabe en un array de **primitivos**: solo en arrays de objetos. Con `int[]` se usan valores centinela.

> 🦎 **Vocabulario rápido**

Término	Idea general
Invertir en el sitio	Dar la vuelta al array sin crear otro, con dos punteros
Variable temporal	El `temp` que guarda un valor durante el intercambio
Dos pasadas	Contar primero y crear/rellenar después (tamaño dependiente de los datos)
Valor centinela	Un valor especial (`-1`, `0`) que significa “vacío” en primitivos
Puntero	Índice que delimita una zona (`izquierda`, `derecha`)

📚 Volver al índice de la unidad · **Anterior:** 06 · Aplicaciones de los arrays · **Siguiente:** 08 · Array-revelde: errores comunes y depuración

08 · Array-revelde: errores comunes y depuración

La galería de monstruos de los arrays: excepciones, null, comparaciones que no comparan y cómo cazarlos con calma 🦎

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 08 · Array-revelde: errores comunes y depuración

---

📬 La idea en una frase

> **Los arrays no se rebelan por maldad: se rebelan porque olvidas los límites. Conocer los 6 monstruos típicos es la mitad de la batalla; saber depurar es la otra mitad.**

A estas alturas ya has escrito tus primeros arrays. Y, si eres humano, ya te has estrellado una vez (o dos, o cincuenta). Este punto recopila los errores más comunes del universo array para que los reconozcas al instante y dejes de llorar sobre el teclado.

👹 La galería de monstruos

Monstruo 1: la `ArrayIndexOutOfBoundsException`

El rey del mambo. Sales de las plazas válidas (`0` a `length - 1`).

```
int[] a = {10, 20, 30};
System.out.println(a[3]); // Index 3 out of bounds for length 3
```

> 🔍 **Diagnóstico:** el mensaje te dice el índice y la longitud. Si pides `a[3]` y hay 3 plazas (0, 1, 2), el problema está en el `<=` de tu bucle o en un índice calculado mal. Lee el mensaje: no es un misterio, es una pista.

Monstruo 2: la `NullPointerException`

Recorre un array de objetos recién creado y tocas una plaza sin objeto.

```
String[] nombres = new String[3];
System.out.println(nombres[0].toUpperCase()); // BOOM: null no tiene métodos
```

🔍 **Diagnóstico:** las plazas de un `String[]` nuevo están llenas de `null`. Pregunta antes: `if (nombres[i] != null) { ... }` o crea los objetos al inicio.

Monstruo 3: imprimir sin `Arrays.toString`

```
int[] a = {1, 2, 3};
System.out.println(a); // [I@6d06d69c
```

🔍 **Diagnóstico:** imprime la dirección de memoria, no los datos. Siempre `Arrays.toString(a)` (o `Arrays.deepToString` para 2D). Lo viste en el punto 4 y no te cansarás de verlo.

Monstruo 4: comparar con ==

```
int[] a = {1, 2, 3};
int[] b = {1, 2, 3};
System.out.println(a == b);            // false: ¿son el MISMO objeto?
System.out.println(a.equals(b));         // false también: los arrays no sobreescriben equals
System.out.println(Arrays.equals(a, b)); // true: esto compara contenido
```

🔍 **Diagnóstico:** `==` y `equals` comparan **referencias**. Para comparar contenido: `Arrays.equals(a, b)`.

Monstruo 5: confundir `length`, `length()` y `size()`

```
int[] a = {1, 2, 3};
String s = "Hola";
System.out.println(a.length());  // no compila: array usa length (atributo)
System.out.println(s.length);    // no compila: String usa length() (método)
```

🔍 **Diagnóstico:** la regla de oro: **array → `length`** (sin paréntesis), **`String` → `length()`**, **colección → `size()`**. Los exámenes adoran esta trampa.

Monstruo 6: intentar modificar con `for-each`

```
int[] a = {1, 2, 3};
for (int n : a) {
    n = n * 10;   // ¿sube los valores? NO: cambia una copia
}
System.out.println(Arrays.toString(a)); // [1, 2, 3]
```

> 🔍 **Diagnóstico:** el `for-each` es de solo lectura. La variable `n` es una copia del valor de la plaza. Para modificar, `for` con índice: `a[i] = a[i] * 10;`.

👹 CONRAD VS EL MUNDO: “El array que no se calla”

> *CONRAD, nuestro compilador cascarrabias, tiene la taza llena y los arrays rotos sobre la mesa.*

CONRAD: — ¡OTRA VEZ! Me traen un programa y me dicen: *CONRAD, me da ArrayIndexOutOfBoundsException*. Y yo: ¿y tú qué has hecho? *Pues recorrer el array.* ¿CON QUÉ? *Pues con un for… creo.* ¡AY, MADRE MÍA! ¿Un for con qué condición? *Pues* `i <= numeros.length`… ¡¿<=?! ¡PERO SI TE LO DIJE EN EL PUNTO 2! El último índice válido es `length - 1`. `<=` te lleva a la plaza fantasma y de ahí no vuelve nadie.

Y luego están los que preguntan por qué les sale null. Pregunta: ¿qué tienes en la plaza? No sé, la creé con `new String[10]`. *¿Y cuántas plazas has llenado?* Pues… bueno… ¡AH! ¡NINGUNA! Un `String[]` recién creado es una fila de `null`. Si no metes objetos, no hay objetos. ¿Es tan difícil?

Y el favorito de todos: no me funciona el bucle que duplica. *Muéstramelo.* Es un `for-each` que hace `n = n * 2`. *¿Y esperabas cambiar el array?* Pues sí. ¡EL `for-each` NO MODIFICA! Es como un robot lector de matrículas: lee, pero no repinta. Para repintar, índice y corchetes.

La lección: los seis monstruos tienen nombre, mensaje y cura. El 90% de los “arrays rebeldes” se arreglan mirando el mensaje de error con calma y recordando las reglas de oro: límites (`0` a `length-1`), `null` en objetos, `Arrays.toString` para imprimir, `Arrays.equals` para comparar, `length` sin paréntesis y `for` con índice para modificar.

🐛 Depurar arrays sin perder la cabeza

Cuando un array se porta mal, sigue este orden:

1. **Imprime el array completo** con `Arrays.toString()` en cada paso. Ver los datos a la vista arregla la mitad de los misterios.
2. **Comprueba los límites antes de tocar.** Si vas a acceder a `a[i + 1]`, que `i` no llegue a `length - 1`. Pregunta siempre: ¿puede salirse?
3. **Usa el depurador del IDE.** Pon un *breakpoint* en el bucle y observa `i` en cada vuelta. Si se pasa de `length - 1`, lo ves a la primera.
4. **Papel y boli.** Con un array de 3 elementos y un objetivo que no esté, simula el bucle a mano. Sí, en pleno siglo XXI, y sigue funcionando.
5. **Aísla el error.** Divide el problema: primero rellena y comprueba que el contenido es correcto; después el bucle; después el cálculo. Si algo falla, sabrás qué tramo es.

> 💡 **Consejo:** el mensaje de excepción de Java no es tu enemigo: es un detective que te dice la línea y el motivo. `Index 5 out of bounds for length 5` no deja lugar a dudas. LÉELO antes de tocar nada.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando algo falle, pregunta antes "¿qué esperaba?" y después "¿qué he escrito?". El 90% de las veces la diferencia entre las dos respuestas está en un `<=`, un `null` o un `==`.

Ejercicio: el cazador de monstruos

```
public class CazaMonstruos {
    public static void main(String[] args) {
        int[] datos = new int[5];
        for (int i = 1; i <= datos.length; i++) {
            datos[i] = i * 10;
        }
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

¿Qué ocurre?

- (A) Imprime `[10, 20, 30, 40, 50]`
- (B) Imprime `[0, 10, 20, 30, 40]`
- (C) `ArrayIndexOutOfBoundsException`

- (D) `NullPointerException`

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué excepción lanzas al acceder a `a[a.length]`?
2. ¿Qué hay en `String[] s = new String[3]` en cada plaza?
3. ¿Cómo comparas dos arrays para saber si tienen el mismo contenido?
4. ¿Con qué bucle puedes modificar los elementos de un array?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los **seis monstruos** de los arrays son: índice fuera de rango, `null`, imprimir sin `toString`, comparar con `==`, confundir `length`/`length()`/`size()` y modificar con `for-each`.
2. El **mensaje de la excepción es una pista**: te dice la línea y el motivo. Léeselo antes de tocar nada.
3. Depurar un array es **ver los datos** (`Arrays.toString`), **respetar los límites** y aislar el tramo que falla, con boli o con *breakpoint*.

> 🦎 **Vocabulario rápido**

Término	Idea general
Monstruo	Error típico con nombre y cura (`ArrayIndexOutOfBounds`, `NullPointer`…)
`Arrays.toString`	La forma de imprimir un array legible
`Arrays.equals`	Comparar contenido de dos arrays
Valor centinela	Valor especial que representa “vacío” en primitivos
Breakpoint	Punto de parada del depurador para inspeccionar variables

📚 Volver al índice de la unidad · **Anterior:** 07 · Be the Code: el parking se gestiona · **Siguiente:** 09 · Repaso interactivo

09 · Repaso interactivo: el parking a examen

El cierre práctico de la unidad, con recorridos, búsquedas y un array que no va a funcionar 😈

> 🗺️ **Estás en:** 🅿️ **U06 · Arrays** → 09 · Repaso interactivo

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```
public class Misterio {
    public static void main(String[] args) {
        int[] datos = {3, 1, 4, 1, 5, 9};
        int total = 0;

        for (int i = 0; i < datos.length; i += 2) {
            total += datos[i];
        }

        System.out.println(total);
    }
}
```

¿Qué imprime por pantalla? Elige sabiamente:

1. **23** → Sumas todos los elementos, pero el bucle va de dos en dos. ❌
2. **12** → ✅ ¡Correcto! El bucle salta de 2 en 2: `i = 0, 2, 4`. Suma `datos[0] + datos[2] + datos[4]` = `3 + 4 + 5` = **12**. La plaza 1 (el 1) y la 3 (el 1) se quedan sin visitar.
3. **13** → Sumas los índices en vez de los valores. ❌
4. **9** → Solo te quedas con `datos[4]`, el último del salto. ❌

> 🔀 Solución

🔥 Fireside Chat: for clásico vs for-each

> *Dos veteranos del recorrido discuten junto a la máquina de café.*

For-each: — Yo soy el moderno. “Para cada X en Y”: corto, limpio, imposible equivocarse con el índice. ¿Quién quiere contar plazas a mano?

For clásico: — ¿Y cuándo me necesitas a mí? Cuando hay que **modificar**, cuando hay que **saber la posición**, cuando hay que ir **hacia atrás** o **de dos en dos**. Esas cuatro cosas son el 80% de los ejercicios del curso, y las hago todas. Tú, en cambio, lees y te callas.

For-each: — Leer es lo que se hace casi siempre: sumar, contar, imprimir. El 80% de las veces no te necesito, y si te uso sin necesitarte, llegan los errores: el <=, el empezar en 1, el olvidar el ++…

For clásico: — Eso es porque no me usas bien. Conmigo tienes el control total. Tú eres… la variante perezosa.

For-each: — La variante **segura**. Y no me llamo perezosa: me llamo “sin índice, sin excusas”.

For clásico: — Vale, tregua. Tú para leer sin posiciones; yo para todo lo demás. ¿Trato?

For-each: — Trato. Pero que sepas que en el bucle de la media te gano a ti hasta con los ojos cerrados.

> La lección: **for-each** si solo lees y no te importa la posición; **for con índice** si modificas, buscas posiciones o recorres de forma especial. El contexto decide, no la moda.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy un aparcamiento de tamaño fijo: guardo muchos datos del mismo tipo bajo un solo nombre.**
2. **Soy el número de cada plaza, y empiezo en 0, para confusión general.**
3. **Soy la excepción favorita del novato: me lanzan cuando piden la plaza que no existe.**
4. **Soy el bucle de solo lectura: “para cada X en Y”, sin índice y sin prisas.**

5. **Soy la clase utilitaria con `toString`, `sort`, `binarySearch`, `copyOf` y `fill`.**
6. **Soy el atributo que dice cuántas plazas hay, sin paréntesis, para no confundirme con los `String`.**

> 🔄 **Respuestas**

CONRAD VS EL MUNDO: “El array que no para”

> *CONRAD, nuestro compilador cascarrabias, ha encontrado una nota en la bandeja de errores.*

CONRAD: — ¡OTRA VEZ! Un alumno me dice: *CONRAD, mi bucle se sale del array*. Y yo: ¿con qué condición lo has escrito? *Pues… `i <= notas.length`.* ¡¡PERO SI ESO ES LO PRIMERO QUE SE APRENDE EN ESTA UNIDAD!! El último índice es `length - 1`. `<=` significa que vas a pedir la plaza `length`, que no existe. Es como llamar a la planta 5 de un parking de 5 plantas (que van de la 0 a la 4): no está, y el portero te lanza la `ArrayIndexOutOfBoundsException`.

Y luego el clásico del for-each: mi array no se modifica. ¿Y cómo lo recorres? *Con for-each.* ¡CLARO! El for-each es un robot que LEE las matrículas. No repinta coches. Si quieres cambiar valores, índice y corchetes: `notas[i] = notas[i] + 1;`. ¿Cuántas veces lo tengo que decir?

Y el de los String: comparo dos nombres y no me da igual. ¿Con qué? *Con `==`.* ¿CUÁNTAS VECES? Con `String` se usa `.equals()`. `==` compara referencias: ¿son el MISMO objeto? Aunque tengan el mismo texto, si son dos objetos distintos, `==` dice false. ¡LÉELO EN EL PUNTO 6, POR FAVOR!

La lección: los tres males del novato con arrays tienen nombre: **`<=` fuera de rango**, **for-each que no modifica** y **`==` que no compara texto**. Antes de llorar sobre el teclado, comprueba esas tres cosas. El 90% de los “arrays rebeldes” se arreglan con un vistazo.

El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. ¿Cuál es el último índice válido de `int[] a = new int[8]`?
   - a) 7 b) 8 c) 9
2. ¿Qué valor tiene cada plaza de `boolean[] b = new boolean[4]`?
   - a) `true` b) `false` c) `null`

3. ¿Qué hace `Arrays.binarySearch` sobre un array **desordenado**?
    - a) Lanza excepción b) Devuelve un resultado impredecible c) Ordena primero
4. ¿Qué imprime `System.out.println(a.length);` para `int[] a = new int[10]`?
    - a) `9` b) `10` c) `[10]`
5. ¿Qué bucle usas para recorrer un array **hacia atrás**?
    - a) `for-each` b) `for` con índice c) cualquiera

> 🔄 **Soluciones**

⚡ Laboratorio de tortura: la media que se resiste

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Tiene **3 errores** que impiden que compile o que falle en ejecución, y 1 error de lógica que hace que el resultado sea incorrecto cuando lo arregles.

```
public class Tortura
    public static void main(String[] args) {
        int[] notas = {8, 7, 9, 6};
        int suma = 0;

        for (int i = 0; i <= notas.length; i++) {
            suma += notas[i];
        }

        double media = suma / notas.length;
        System.out.println("Media: " + media)
    }
}
```

Fallo intencionado: uno de los errores parece correcto a simple vista porque “se ve bien”, pero hace que el programa falle en ejecución en cuanto arranca.

Tu tarea: conseguir que compile, que ejecute y que imprima `Media: 7.5`. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún `;` que falte? *no → sigue buscando.*

> **¿Y si sigo atascado?**

2. ¿Compila ya? *no → mira el mensaje de error.*

   > **¿Y si sigo atascado?**

3. ¿Compila pero explota al ejecutar? *Es el error de índices: el bucle llega demasiado lejos.*

   > **¿Y si sigo atascado?**

4. ¿Ejecuta e imprime `Media: 7.0`? *Es el error de lógica: la división entera.*

   > **Solución final**

Los **3 errores** que impiden compilar o ejecutar:

1. Falta la `{` de apertura de la clase después de `Tortura`.
2. Falta el `;` al final de `System.out.println("Media: " + media)`.
3. `i <= notas.length` accede a `notas[4]` fuera del array: `ArrayIndexOutOfBoundsException`. Debe ser `i < notas.length`.

El **error de lógica**: `suma / notas.length` divide enteros. `30 / 4 = 7` (se trunca), no `7.5`. Para que la media salga con decimales hay que convertir antes: `(double) suma / notas.length`.

```
public class Tortura {
    public static void main(String[] args) {
        int[] notas = {8, 7, 9, 6};
        int suma = 0;

        for (int i = 0; i < notas.length; i++) {
            suma += notas[i];
        }

        double media = (double) suma / notas.length;
        System.out.println("Media: " + media);
    }
}
```

Salida correcta: `Media: 7.5`. Con la versión rota, una vez arreglados los otros errores, la división entera daba `7.0`: la pista del error de lógica.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Aparcador Jefe**	Crear un array, rellenarlo y recorrerlo sin mirar apuntes
🏅 **El Contador de Plazas**	Explicar la diferencia entre `length`, `length()` y `size()`
🏅 **El Cazador de Monstruos**	Encontrar los 3+1 errores del Laboratorio sin usar las pistas
🏅 **El Inversor**	Invertir un array en el sitio con dos punteros, a la primera
🏅 **El Navajero**	Usar `toString`, `sort`, `binarySearch`, `copyOf` y `fill` en un programa
🏅 **El Profe de la Clase**	Aprobar con un array de objetos: rellenar, recorrer y calcular la media

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```
public class Misterio2 {
    public static void main(String[] args) {
        int[] datos = {4, 2, 8, 1, 6};
        int mayor = datos[0];

        for (int i = 1; i < datos.length; i++) {
            if (datos[i] > mayor) {
                mayor = datos[i];
            }
        }
        System.out.println(mayor);
    }
}
```

2. **El truco del índice:** ¿qué pasa si en el bucle de la media de `{8, 7, 9, 6}` usas `i <= notas.length - 1` en lugar de `i < notas.length`? ¿Funciona? ¿Por qué?
3. **El doble:** `int[] a = {1, 2, 3}; int[] b = a; b[0] = 99;` ¿qué vale `a[0]` después? (Pista: es el punto 5.)

4. **Verdadero o falso:** "`Arrays.sort` modifica el array original, así que conviene copiarlo antes si no quieres perder el orden inicial".

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Tamaño de un array, sin paréntesis (6 letras)
4. Método de la clase Arrays para comparar contenido (5 letras)

Vertical:
2. Valor por defecto de un boolean en un array (5 letras)
3. Bucle de solo lectura (3 letras + 4 letras)
5. La excepción que lanzas al salirte del array (una sola palabra muy larga)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, qué es un array."**
2. **"Invierte este array sin crear otro. Ahora dime cuánta memoria extra necesitas."**
3. **"¿Cuál es la diferencia entre `length`, `length()` y `size()`?"**
4. **"¿Cuándo usarías `for-each` y cuándo un `for` con índice?"**
5. **"¿Cómo comparas dos arrays para saber si tienen el mismo contenido?"**
6. **"Escribe el código que devuelve la nota más alta de un array."**

🤷 No hay preguntas tontas

> ❓ **¿Por qué el primer índice es 0 y no 1?**

Porque el índice es una **distancia** desde el principio, no un número de plaza. La primera casa está a 0 pasos de ti, no a 1. En programación, contar desde 0 evita el off-by-one en miles de

cálculos (y es una convención heredada de los lenguajes más antiguos). Te parecerá raro hasta que deje de parecértelo, y entonces lo defenderás con uñas y dientes.

> ❓ **¿Puedo usar `Arrays.sort` y `Arrays.binarySearch` en vez de escribir los algoritmos a mano?**

En tus programas reales, sí: son rápidos y probados, y los verás por todos lados. Pero en la U07 vas a aprender **cómo funcionan por dentro** (burbuja, búsqueda binaria…) porque entender la idea es lo que te diferencia de alguien que solo importa librerías. Y en una entrevista, te pedirán el algoritmo a mano. Primero se aprende a sumar sin calculadora, ¿no?

> ❓ **¿Un array puede cambiar de tamaño?**

No. Es **tamaño fijo** para siempre. Cuando necesitas "más plazas", se crea un array nuevo y se copia (`Arrays.copyOf`). Si esto te parece un incordio, tienes razón: por eso existen las colecciones (`ArrayList` y compañía), que crecen solas. Las verás en la U12, y vas a abrazarlas.

🎬 Poscréditos

El programador cierra su lista de la compra: un array de 12 `String`, recorrido con un for-each, ordenado con `Arrays.sort` e impreso con `Arrays.toString`. Todo funciona a la primera. Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — Oye, no está mal. Creas, recorres, ordenas, buscas… y encima no has salido del array ni una sola vez. ¿Seguro que eres el mismo que empezó la unidad con `gato1`, `gato2`, `gato3`?

Novato: — *sonríe* Empecé con 100 gatos y he terminado gestionando el parking entero con 20 líneas. ¿Y ahora qué?

CONRAD: — *toma un sorbo* Ahora lo interesante. Todos estos datos que guardas en arrays piden a gritos que los **busques** y los **ordenes** bien. Burbuja, búsqueda binaria, Big O… Otro tipo de bestias, y van justo después.

El novato guarda su proyecto, cierra el IDE y siente que ya no guarda datos sueltos: **piensa en aparcamientos**.

PRÓXIMAMENTE EN U07: Algorítmica I: Fundamentos. Los algoritmos que buscan y ordenan tus arrays, y la notación para saber cuál es rápido de solemnidad. 🧩

📚 Volver al índice de la unidad · **Anterior:** 08 · Array-revelde: errores comunes y depuración · **Siguiente:** U07 · Algorítmica I: Fundamentos

U07 — Algorítmica I: Fundamentos

Recetas, diccionarios, cartas y burbujas: los algoritmos que ordenan y buscan por ti 🧩

Bienvenido a la cocina del programador. En la U04 tus programas aprendieron a **decidir** y a **repetir**, pero repetir por repetir es hacer footing sin rumbo: corres, pero no llegas a ninguna parte. Esta es la unidad donde el rumbo aparece: vas a aprender los **algoritmos clásicos** que ordenan y buscan datos, y la herramienta para saber si uno es rápido o lento de solemnidad: la **notación Big O**.

Piensa en lo que haces cada día sin darte cuenta. Buscas la palabra "merienda" en el diccionario (y no abres por la página 1, sino por la mitad). Ordenas las cartas en tu mano para el póker. Buscas tus zapatillas perdidas revisando la habitación. Pues todo eso, llevado a Java, es esta unidad: **búsqueda lineal**, **búsqueda binaria**, **ordenación burbuja**, **ordenación por inserción** y el arte de elegir cuál usar.

Y, como siempre, CONRAD el compilador estará por aquí para recordarte que un algoritmo mal pensado es como una tortilla sin sal: se puede comer, pero no deberías.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar **qué es un algoritmo**, sus propiedades y la diferencia entre la *idea* y el *código*.
- Buscar elementos en un array con **búsqueda lineal** (O(n)) y saber cuándo conviene.
- Buscar en arrays **ordenados** con **búsqueda binaria** (O(log n)), sin morir en el intento (ni en el off-by-one).
- Ordenar arrays con **burbuja** (el patoso) y con **inserción** (el de las cartas), y saber sus límites.
- Analizar la **complejidad algorítmica** con notación Big O y simplificar con las reglas de oro.
- **Elegir el algoritmo adecuado** según el tamaño, el orden inicial y la situación.
- Escribir búsqueda binaria y burbuja **a mano**, sin mirar apuntes (sí, como en las entrevistas de trabajo).

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · Qué es un algoritmo	Definir algoritmo, sus propiedades y la receta de la tortilla	Todos
02 · Búsqueda lineal	Recorrer el array buscando un elemento, O(n)	Todos
03 · Búsqueda binaria	Dividir y vencer: descartar mitades, O(log n)	Todos
04 · Ordenación burbuja	Intercambiar burbujas hasta que todo flote, $O(n^2)$	Todos
05 · Ordenación por inserción	Ordenar como cartas en la mano, $O(n^2)$ y bestia con casi ordenados	Todos
06 · Complejidad algorítmica: Big O	Cómo crece el tiempo con los datos, de O(1) a $O(2^n)$	Todos
07 · Elegir el algoritmo adecuado	La tabla de decisión según tamaño y orden inicial	Todos
08 · Be the Code: búsqueda binaria desde cero	Programar binaria y burbuja sin red de seguridad	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Quién Soy, Laboratorio, Crucigrama…	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

✅ Criterios de evaluación cubiertos (RA2 + RA6)

RA2: Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.

RA6: Escribe programas que manipulen información, seleccionando y utilizando tipos avanzados de datos.

CE	Criterio	Dónde se cubre
RA2 b)	Se han escrito programas simples.	✅ Todos
RA2 f)	Se han utilizado parámetros en la llamada a métodos.	✅ Puntos 2, 3, 4 y 5
RA6 a)	Se han escrito programas que utilicen arrays.	✅ Todos

> 📌 Esta unidad es el germen de la U08: los bucles que machacas aquí (`for`, `while`), las búsquedas y las ordenaciones se convierten en QuickSort, MergeSort y recursividad. Y los arrays que recorres hoy son la base de las colecciones de la U12.

🚪 ¿Por dónde empiezo?

- ¿Cero conocimientos? → Arranca en el punto 1. Solo necesitas variables, bucles y arrays básicos de la U04.
- ¿Ya sabes qué es un algoritmo y quieres chicha? → Salta a la búsqueda binaria o a la ordenación por inserción.
- ¿Solo quieres saber qué algoritmo elegir? → Lee Big O y Elegir el algoritmo adecuado.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Qué es un algoritmo

⏭️ **Al acabar la unidad, continúa en U08 · Algorítmica II: técnicas avanzadas y modularidad.**

01 — Qué es un algoritmo

La receta de la tortilla que tu ordenador puede seguir al pie de la letra 🍳

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 01 · Qué es un algoritmo

---

📬 La idea en una frase

> **Un algoritmo es una receta de cocina para tu ordenador: una secuencia finita, ordenada y sin ambigüedades de pasos que resuelve un problema.**

Cuando cocinas una tortilla de patatas, sigues un algoritmo mental:

1. Pelar las patatas.
2. Cortarlas en rodajas finas.
3. Freírlas en aceite abundante.
4. Batir los huevos.
5. Mezclarlo todo y cuajar.

Pero ojo: "echa sal al gusto" no vale como paso de algoritmo. ¿Cuánto es "al gusto"? ¿Un pellizco? ¿Un puñado? Cada persona interpretaría la receta de forma distinta. Un algoritmo de verdad **no deja espacio a la interpretación**: cada paso debe ser preciso y determinista. Si le pasas la misma entrada, siempre produce la misma salida. Como una máquina expendedora: metes moneda, pulsas el botón, y siempre sale el mismo batido.

🏛️ Las propiedades de un algoritmo

Para que una secuencia de pasos merezca el nombre de algoritmo, debe cumplir cinco propiedades:

1. **Finito**: debe terminar en algún momento. Si se ejecuta para siempre, no es un algoritmo, es una pesadilla.
2. **Preciso**: cada paso está definido sin ambigüedad. Nada de "al gusto" ni "cuando esté listo".
3. **Entrada**: recibe cero o más valores de entrada.
4. **Salida**: produce al menos un valor de salida.

5. **Eficaz**: resuelve el problema en tiempo finito y de forma correcta.

> 📝 **Nota histórica:** la palabra "algoritmo" viene del matemático persa **Al-Juarismi** (siglo IX), que escribió un libro sobre cómo hacer cálculos con los números indios. Siglos después, los informáticos le robamos la palabra. Como somos así.

🧠 La idea vs. el código

Este es un momento importante, así que sube el volumen mental:

> ⚠️ **Advertencia: no todo código es un algoritmo.** El algoritmo es la *idea*: la secuencia de pasos. El código es su *materialización* en un lenguaje concreto. Puedes implementar el mismo algoritmo en Java, en Python o en ensamblador, y la esencia seguirá siendo la misma.

Mira este ejemplo. El algoritmo de "sumar dos números" se materializa así en Java:

```
public class AlgoritmoSimple {
    public static void main(String[] args) {
        int a = 5;
        int b = 3;
        int resultado = a + b;  // la idea: sumar. El código: la materialización
        System.out.println("5 + 3 = " + resultado);
    }
}
```

La *idea* (sumar dos números) es la misma en cualquier lenguaje. El *código* es solo el disfraz. Por eso los algoritmos se estudian independientemente del lenguaje: son las recetas, y Java solo es una de tus cocinas.

📦 Las dos grandes familias

En esta unidad (y en la próxima) vas a vivir con dos familias de algoritmos:

- **Búsqueda**: encontrar un elemento dentro de un conjunto de datos. ¿Está el número 23 en este array?
- **Ordenación**: poner un conjunto de datos en un orden determinado. ¿Puedes dejar este array ordenado de menor a mayor?

Son las dos habilidades básicas de cualquier programa que maneja datos, y aparecen en todos lados: en una lista de reproducción, en una base de datos, en un buscador. Sin ordenar ni buscar, tu app es un cajón desordenado donde las cosas solo están “por ahí”.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando escribas pasos “a lo loco” en un papel, pregúntate siempre: *¿podría mi abuela ejecutar estos pasos sin preguntarme nada?* Si la respuesta es “no”, tienes ambigüedad.

Ejercicio: la tortilla ordenada

Los pasos de esta receta están desordenados. Ordénalos para que formen un algoritmo válido y explica cuál de las propiedades del algoritmo fallaría si los dejaras en el orden original:

1. Batir los huevos.
2. Comer la tortilla.
3. Echar sal “al gusto” sobre las patatas fritas.
4. Freír las patatas en aceite abundante.
5. Pelar y cortar las patatas en rodajas.
6. Mezclar las patatas con el huevo batido y cuajar en la sartén.

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuál es la diferencia entre un algoritmo y un programa?
2. ¿Por qué “echa sal al gusto” no puede ser un paso de algoritmo?
3. ¿Un algoritmo puede recibir cero entradas?
4. ¿Qué pasaría si un algoritmo nunca terminara?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Un algoritmo es una **secuencia finita, precisa y sin ambigüedades de pasos** que resuelve un problema: una receta de cocina para el ordenador.
2. Debe ser **finito, preciso, con entradas, con salida y eficaz**, y no deja espacio a la interpretación.
3. El algoritmo es la **idea** y el código su materialización: el mismo algoritmo se escribe igual de bien en cualquier lenguaje.

> 🦎 **Vocabulario rápido**

Término	Idea general
Algoritmo	Secuencia finita y precisa de pasos que resuelve un problema
Determinista	Misma entrada → siempre misma salida
Ambigüedad	Paso que se puede interpretar de varias formas (“al gusto”)
Búsqueda	Encontrar un elemento dentro de un conjunto de datos
Ordenación	Poner los datos en un orden determinado (numérico, alfabético…)
Materializar	Traducir la idea del algoritmo a código de un lenguaje

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · Búsqueda lineal

02 — Búsqueda lineal

El buscador de zapatillas perdidas: revisa cada sitio hasta encontrar lo que buscas 👟

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 02 · Búsqueda lineal

---

📬 La idea en una frase

> **La búsqueda lineal recorre el array elemento por elemento hasta encontrar el objetivo (o hasta comprobar que no está). Simple, directa y funciona aunque los datos estén desordenados.**

Imagina que pierdes una zapatilla en tu habitación. ¿Qué haces? Miras debajo de la cama, detrás de la puerta, en el armario… básicamente **revisas cada sitio hasta encontrarla**. Pues eso es la búsqueda lineal: vas uno a uno, sin atajos.

👟 El algoritmo

```java
public class BusquedaLineal {

    public static int buscar(int[] array, int objetivo) {
        for (int i = 0; i < array.length; i++) {
            if (array[i] == objetivo) {
                return i;  // ¡Encontrado! Devuelve la posición
            }
        }
        return -1;  // No está en el array
    }

    public static void main(String[] args) {
        int[] numeros = {34, 12, 56, 78, 23, 9, 45, 67};

        int resultado = buscar(numeros, 23);
        if (resultado != -1) {
            System.out.println("¡Encontrado el 23 en la posición " + resultado + "!");
        } else {
            System.out.println("El 23 no está en el array.");
        }

        resultado = buscar(numeros, 99);
        if (resultado == -1) {
                    System.out.println("El 99 no está. Como unas zapatillas que nunca
aparecen.");
        }
    }
}
```

Salida:

```
¡Encontrado el 23 en la posición 4!
El 99 no está. Como unas zapatillas que nunca aparecen.
```

Detalles del método `buscar`:

- Recorre el array con un `for` desde la posición 0 hasta `array.length - 1`.
- Si encuentra el objetivo, **devuelve su índice** y para: el `return` corta el método entero.
- Si termina el bucle sin encontrar nada, devuelve `-1`, el "índice imposible" que usamos como señal de *no encontrado*.

> 💡 **Detalle práctico:** devolver `-1` es la convención clásica de “no está”. Nunca devuelvas `0` para decir “no encontrado”, porque `0` es una posición válida: la primera. Ese error es el clásico “bug del índice cero”.

⏱️ ¿Cómo de rápida es?

En el **mejor caso**, el elemento está en la primera posición → 1 paso.

En el **peor caso**, el elemento está al final, o no existe → recorres los `n` elementos enteros.

Decimos que su complejidad es **O(n)**, lineal. Si el array tiene 10 elementos, tardas ~10 pasos; si tiene 10.000, tardas ~10.000. Crece al mismo ritmo que los datos.

> 💡 **Consejo:** la búsqueda lineal es como buscar en tu nevera: si es pequeña, da igual el método. Pero si tienes un almacén de 10.000 productos, necesitas algo mejor… y en el próximo punto lo encuentras.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando un método devuelve un índice, los dos caminos de salida son: `return` cuando lo encuentras, `return -1` cuando el bucle termina. Si mezclas eso, el programa se comporta como una gaviota: confunde los sitios.

Ejercicio: el buscador que se pierde

Sin ejecutar, calcula cuántas comparaciones hace este programa y qué imprime:

```java
public class Busqueda2 {
    public static int buscar(int[] array, int objetivo) {
        int pasos = 0;
        for (int i = 0; i < array.length; i++) {
            pasos++;
            if (array[i] == objetivo) {
                System.out.println("He necesitado " + pasos + " pasos.");
                return i;
            }
        }
        System.out.println("He necesitado " + pasos + " pasos.");
        return -1;
    }

    public static void main(String[] args) {
        int[] datos = {3, 8, 1, 9, 5, 2};
        int resultado = buscar(datos, 9);
        System.out.println("Posición: " + resultado);
    }
}
```

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué devuelve la búsqueda lineal cuando el elemento no está en el array?
2. ¿Funciona la búsqueda lineal con arrays desordenados?
3. ¿Por qué se dice que es O(n)?
4. ¿Qué imprimiría `buscar(new int[]{5, 5, 5}, 5)`? ¿Qué posición devuelve?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. La búsqueda lineal recorre el array **elemento por elemento** hasta encontrar el objetivo o agotar la lista.

2. Devuelve el **índice** del elemento, o `-1` si no existe, y funciona con datos **desordenados**.
3. Su complejidad es **O(n)**: perfecta para arrays pequeños, lenta para los grandes.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Índice | Posición de un elemento dentro del array (empieza en 0) |
> | Recorrer | Visitar cada elemento del array uno a uno |
> | -1 | El “índice imposible”: señal de que el elemento no está |
> | O(n) | El tiempo crece en proporción directa al número de elementos |
> | Mejor caso | El elemento está el primero: 1 solo paso |

📚 Volver al índice de la unidad · **Anterior:** 01 · Qué es un algoritmo · **Siguiente:** 03 · Búsqueda binaria

03 — Búsqueda binaria

El buscador Jedi: descarta media tonelada de papel en cada intento 🕯️

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 03 · Búsqueda binaria

---

📬 La idea en una frase

> **La búsqueda binaria abre el "diccionario" por la mitad, compara, y descarta media tonelada de papel en cada intento. Solo funciona si el array está ordenado.**

Buscar una palabra en un diccionario es un ritual muy concreto: no abres por la página 1 y pasas de una en una. Abres por la mitad, ves si la palabra está antes o después, y descartas la mitad del libro en un gesto. Y repites. Eso es la búsqueda binaria.

⚠️ El requisito imprescindible

> **El array debe estar ordenado.** Si no, este método no funciona.

Y lo peor de todo: no te avisa. No hay error de compilación, no hay excepción, no hay "ey, me has dado basura". Simplemente obtienes la respuesta equivocada. Es como buscar "merienda" en un diccionario cuyas palabras están al azar: abrir por la mitad no te sirve de nada.

🕯️ El algoritmo

```java
public class BusquedaBinaria {

    public static int buscar(int[] array, int objetivo) {
        int izquierda = 0;
        int derecha = array.length - 1;

        while (izquierda <= derecha) {
            int medio = izquierda + (derecha - izquierda) / 2;  // mitad del segmento

            if (array[medio] == objetivo) {
                return medio;  // ¡Bingo!
            }

            if (array[medio] < objetivo) {
                izquierda = medio + 1;  // descartamos la mitad izquierda
            } else {
                derecha = medio - 1;    // descartamos la mitad derecha
            }
        }
        return -1;  // no encontrado
    }

    public static void main(String[] args) {
        // OJO: tiene que estar ORDENADO
        int[] numeros = {2, 5, 8, 12, 19, 24, 31, 37, 42, 50, 58, 63};

        int resultado = buscar(numeros, 31);
        System.out.println("31 encontrado en posición: " + resultado);  // 6

        resultado = buscar(numeros, 3);
        System.out.println("3 encontrado en: " + resultado);  // -1
    }
}
```

Vamos a trazar la búsqueda del 31 sobre el array `{2, 5, 8, 12, 19, 24, 31, 37, 42, 50, 58, 63}` (12 elementos):

Vuelta	izquierda	derecha	medio	array[medio]	¿Qué pasa?
1	0	11	5	24	24 < 31 → izquierda = 6
2	6	11	8	42	42 > 31 → derecha = 7

Vuelta	izquierda	derecha	medio	array[medio]	¿Qué pasa?
3	6	7	6	31	¡Bingo! → devuelve 6

Tres comparaciones. La búsqueda lineal habría necesitado siete. Y en un array de un millón de elementos, la diferencia es todavía más escandalosa.

🧮 ¿Por qué `izquierda + (derecha - izquierda) / 2` y no `(izquierda + derecha) / 2`?

Porque si el array es muy grande (cerca de `Integer.MAX_VALUE` elementos), `izquierda + derecha` puede **desbordarse**: el resultado ya no cabe en un `int` y se convierte en un número negativo de golpe. La fórmula alternativa `izquierda + (derecha - izquierda) / 2` evita ese problema.

> ⚠️ **Advertencia:** este es un bug tan famoso que apareció incluso en la biblioteca de Java original. Lleva años coleccionando trofeos: Bug del año, Bug de la década, Bug favorito del público…

⏱️ El análisis: O(log n)

En cada paso, la búsqueda binaria **descarta la mitad** del array restante. Mira cómo crece el número de pasos:

- Array de 16 elementos → 4 pasos máximos
- Array de 32 elementos → 5 pasos
- Array de 1.024 elementos → 10 pasos
- Array de 1.000.000 de elementos → 20 pasos

Eso es **O(log n)**, complejidad logarítmica. Crece muy despacio incluso con datos enormes. Es la diferencia entre preguntar a mil personas una por una, o preguntar "¿está a la izquierda o a la derecha?" y descartar a 500 de golpe. Con un millón de elementos, la lineal necesita un millón de pasos y la binaria solo **20**. Lee eso otra vez. Veinte.

> 📝 **Nota:** cuando un informático dice "log n", piensa en **base 2**: es "mitad, mitad, mitad…". No es el logaritmo decimal de toda la vida. Un logaritmo en base 2 responde a la pregunta "¿cuántas veces puedo dividir entre 2 antes de llegar a 1?".

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** la búsqueda binaria tiene una regla de oro: si el medio es menor que el objetivo, `izquierda = medio + 1`; si es mayor, `derecha = medio - 1`. El `+1` y el `-1` son sagrados: sin ellos, el bucle puede quedarse dando vueltas para siempre.

Ejercicio: el buscador que no avanza

Sin ejecutar, traza la búsqueda del número **8** en este array ordenado y escribe los valores de `izquierda`, `derecha` y `medio` en cada vuelta:

```
public class Traza {
    public static int buscar(int[] array, int objetivo) {
        int izquierda = 0;
        int derecha = array.length - 1;

        while (izquierda <= derecha) {
            int medio = izquierda + (derecha - izquierda) / 2;
               System.out.println("izquierda=" + izquierda + " derecha=" + derecha + "
medio=" + medio);

            if (array[medio] == objetivo) return medio;
            if (array[medio] < objetivo) {
                izquierda = medio + 1;
            } else {
                derecha = medio - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] datos = {1, 4, 8, 12, 20, 33};
        System.out.println("Resultado: " + buscar(datos, 8));
    }
}
```

> › 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué requisito imprescindible tiene la búsqueda binaria?
2. ¿Qué pasa si la incumples?
3. ¿Cuántos pasos máximos necesita para un array de 1.000.000 de elementos?
4. ¿Por qué se usa `izquierda + (derecha - izquierda) / 2` en vez de `(izquierda + derecha) / 2`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. La búsqueda binaria **divide el problema a la mitad en cada paso**, comparando el objetivo con el elemento central.
2. Exige un array **ordenado**; si no lo está, devuelve basura sin avisar.
3. Su complejidad es **O(log n)**: con un millón de elementos bastan ~20 pasos, mientras que la lineal necesita un millón.

> 🦎 **Vocabulario rápido**

Término	Idea general
Mitad	El elemento central del segmento actual (`medio`)
Descartar	Eliminar la mitad izquierda o derecha del segmento
O(log n)	El tiempo crece muy despacio: cada paso descarta la mitad
Off-by-one	Error de “por uno”: confundir `<=` con `<` o `medio+1` con `medio`
Desbordamiento	Cuando una suma supera el máximo que cabe en un `int`

📚 Volver al índice de la unidad · **Anterior:** 02 · Búsqueda lineal · **Siguiente:** 04 · Ordenación burbuja

04 — Ordenación burbuja

Simple, simpática… y la más lenta de las que merecen la pena 🫧

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 04 · Ordenación burbuja

📬 La idea en una frase

> **La burbuja recorre el array comparando parejas vecinas: si el de la izquierda es mayor, los intercambia. Repite hasta que en una pasada no haya ningún intercambio. Los grandes "suben" hacia el final como burbujas en una copa.**

Es el algoritmo de ordenación más sencillo de entender… y el más lento de los que merecen la pena. Pero antes de aprender a correr, hay que aprender a caminar. La burbuja es tu andador.

🫧 El algoritmo

```java
public class Burbuja {

    public static void ordenar(int[] array) {
        int n = array.length;
        boolean huboIntercambio;

        for (int i = 0; i < n - 1; i++) {
            huboIntercambio = false;

            for (int j = 0; j < n - 1 - i; j++) {
                if (array[j] > array[j + 1]) {
                    // intercambio
                    int temp = array[j];
                    array[j] = array[j + 1];
                    array[j + 1] = temp;
                    huboIntercambio = true;
                }
            }

            // si no hubo intercambio, el array ya está ordenado
            if (!huboIntercambio) break;
        }
    }

    public static void main(String[] args) {
        int[] datos = {64, 34, 25, 12, 22, 11, 90};

        System.out.print("Antes: ");
        for (int numero : datos) System.out.print(numero + " ");

        ordenar(datos);

        System.out.print("\nDespués: ");
        for (int numero : datos) System.out.print(numero + " ");
        // 11 12 22 25 34 64 90
    }
}
```

Vamos a trazar la primera pasada con un array pequeño, `{5, 2, 9, 1}`:

Paso	¿Qué compara?	¿Intercambia?	Array
1	5 vs 2	Sí	2 5 9 1
2	5 vs 9	No	2 5 9 1
3	9 vs 1	Sí	2 5 1 9

El 9, el mayor, "subió" hasta el final. En cada pasada, el mayor de los que quedan queda colocado en su sitio: el 9, luego el 5, luego el 2, luego el 1. Por eso el bucle interior llega solo hasta `n - 1 - i`: ya no hace falta mirar los elementos que quedaron colocados al final.

🏎️ ¿Por qué es tan lenta?

Dos bucles anidados. Para un array de `n` elementos:

- Primer bucle: `n` veces.
- Segundo bucle: ~`n` veces (en realidad `n-i-1`, pero a grandes rasgos `n`).

Total: ~n × n = n² operaciones. Complejidad **O(n²)**.

- Para 10 elementos → 100 operaciones (bien).
- Para 1.000 elementos → 1.000.000 de operaciones (empieza a doler).
- Para 1.000.000 de elementos → 1.000.000.000.000 de operaciones (tu ordenador pide la jubilación).

> 💡 **Consejo:** burbuja solo se usa en dos situaciones: (1) estás aprendiendo, y (2) sabes que el array tendrá menos de 50 elementos. Para todo lo demás hay mejores alternativas (las verás en la U08).

🚩 La optimización del flag

Fíjate en la variable `huboIntercambio`. Si en una pasada completa no intercambiamos nada, es que el array ya está ordenado y podemos parar: `break`. Sin este flag, la burbuja daría todas las pasadas aunque el array llegara ordenado en la primera.

Esta optimización **no mejora el peor caso** (array invertido: hay que intercambiarlo todo), pero convierte el mejor caso (array ya ordenado) en O(n): una sola pasada de comprobación y listo.

> 💡 **Detalle práctico:** el patrón del flag (“marca si ha pasado algo; si no, para”) aparece en muchísimos algoritmos reales. Es una de esas ideas que te harán parecer programador senior aunque solo lleves cuatro unidades.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando veas `array[j] > array[j + 1]`, piensa “estoy ordenando **de menor a mayor**”. Si quieres lo contrario, cambia la flecha. El resto del algoritmo no cambia ni una coma.

Ejercicio: la burbuja que se queda corta

Sin ejecutar, escribe la salida exacta de este programa:

```
public class BurbujaCorta {
    public static void main(String[] args) {
        int[] datos = {3, 1, 2};

        for (int i = 0; i < datos.length - 1; i++) {
            for (int j = 0; j < datos.length - 1 - i; j++) {
                if (datos[j] > datos[j + 1]) {
                    int temp = datos[j];
                    datos[j] = datos[j + 1];
                    datos[j + 1] = temp;
                }
            }
        }

        for (int numero : datos) {
            System.out.print(numero + " ");
        }
    }
}
```

> › 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Por qué el bucle interior de la burbuja solo llega hasta `n - 1 - i`?
2. ¿Cuál es la complejidad de la burbuja en el peor caso?
3. ¿Qué hace la variable `huboIntercambio`?
4. ¿Cuándo está justificado usar burbuja en un programa real?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. La burbuja compara **parejas vecinas** e intercambia las que están desordenadas, pasada tras pasada.
2. Su complejidad es **O(n²)**: funciona, pero es lenta con datos grandes.
3. El flag `huboIntercambio` la optimiza para arrays casi ordenados, convirtiendo el mejor caso en O(n).

> 🦎 **Vocabulario rápido**

Término	Idea general
Intercambio	Cambiar dos elementos entre sí usando una variable temporal
Pasada	Un recorrido completo del bucle interior
Flag	Variable booleana que marca si algo ocurrió durante la pasada
O(n²)	El tiempo crece al cuadrado: dos bucles anidados
In-place	Ordena modificando el propio array, sin memoria extra

📚 Volver al índice de la unidad · **Anterior:** 03 · Búsqueda binaria · **Siguiente:** 05 · Ordenación por inserción

05 — Ordenación por inserción

Ordena las cartas de la mano como un crupier veterano 🃏

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 05 · Ordenación por inserción

---

📬 La idea en una frase

> **La inserción toma cada elemento como una carta nueva y la coloca en su sitio dentro de las que ya tienes ordenadas en la mano. Como ordenar cartas en el póker.**

Cuando te reparten cartas, no las tiras todas sobre la mesa y empiezas de cero: vas colocando cada carta nueva en su hueco dentro de la mano que ya tienes ordenada. El 7 entre el 5 y el 9, el 3 al principio, la K al final. Pues eso, pero en Java.

🃏 El algoritmo

```java
public class Insercion {

    public static void ordenar(int[] array) {
        for (int i = 1; i < array.length; i++) {
            int clave = array[i];  // la carta que vamos a colocar
            int j = i - 1;

            // desplazar los mayores hacia la derecha
            while (j >= 0 && array[j] > clave) {
                array[j + 1] = array[j];
                j--;
            }
            array[j + 1] = clave;  // colocar la carta en su sitio
        }
    }

    public static void main(String[] args) {
        int[] datos = {9, 5, 1, 4, 3};

        System.out.print("Antes: ");
        for (int numero : datos) System.out.print(numero + " ");

        ordenar(datos);

        System.out.print("\nDespués: ");
        for (int numero : datos) System.out.print(numero + " ");
        // 1 3 4 5 9
    }
}
```

El truco está en el `while` interior: guardas la `clave` (la carta nueva), y mientras haya cartas mayores que ella a su izquierda, las desplazas una posición a la derecha. Cuando encuentras una menor (o llegas al principio), esa es la posición de la clave. La “mano” izquierda siempre está ordenada.

👣 Paso a paso

Dado `{9, 5, 1, 4, 3}`, mira cómo crece la “mano” (lo que hay a la izquierda de la barra):

```
Paso 0: [9] | 5 1 4 3   → la mano empieza con el 9
Paso 1: [5 9] | 1 4 3   → el 5 se coloca a la izquierda del 9
Paso 2: [1 5 9] | 4 3   → el 1 se cuela al principio
Paso 3: [1 4 5 9] | 3   → el 4 entra entre el 1 y el 5
Paso 4: [1 3 4 5 9]     → el 3 entra entre el 1 y el 4
```

Cada elemento nuevo se “inserta” en su sitio. De ahí el nombre. La mano izquierda siempre está ordenada; el resto del array espera su turno.

📊 El análisis: ¿cuándo es buena?

También es **O(n²)** en el peor caso (array invertido: cada elemento debe viajar hasta el principio). Pero tiene truco:

- **Mejor caso (array casi ordenado):** O(n). Solo hace una pasada de comprobación. Es rapidísima.
- Es **estable**: mantiene el orden relativo de los elementos iguales.
- **No necesita memoria extra**: ordena in-place, modificando el propio array.
- En la práctica, es **más rápida que la burbuja**, aunque ambas sean O(n²).

> 💡 **Consejo:** la inserción es la reina de los datos **casi ordenados**. Si sabes que tu array tiene 100 elementos y ya está “casi bien” (solo un par de elementos fuera de sitio), la inserción te va a sorprender. De hecho, se usa como paso final en algoritmos avanzados (TimSort, el que usa Java por defecto en sus colecciones).

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** en la inserción, la variable `clave` es la única que “sobrevive” al desplazamiento. Si la usaras para algo más, la perderías al sobrescribir `array[j + 1]`. Guárdala como un tesoro: es tu carta.

Ejercicio: la mano que se desordena

Sin ejecutar, escribe la salida exacta de este programa:

```java
public class Insercion2 {
    public static void main(String[] args) {
        int[] datos = {4, 2};

        for (int i = 1; i < datos.length; i++) {
            int clave = datos[i];
            int j = i - 1;
            while (j >= 0 && datos[j] > clave) {
                datos[j + 1] = datos[j];
                j--;
            }
            datos[j + 1] = clave;
        }

        System.out.println(datos[0] + " " + datos[1]);
    }
}
```

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿En qué se diferencia la inserción de la burbuja a la hora de ordenar?
2. ¿Cuál es la complejidad de la inserción en su mejor caso y por qué?
3. ¿Qué significa que sea “estable”?
4. ¿Por qué se dice que no necesita memoria extra?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. La inserción toma cada elemento como una **carta nueva** y lo coloca en su sitio dentro de la parte ya ordenada.
2. Es **O(n²)** en el peor caso, pero **O(n)** con datos casi ordenados: la reina de los arrays casi listos.

3. Es **estable**, no usa memoria extra y en la práctica supera a la burbuja.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Clave | El elemento actual que estamos colocando en su sitio |
> | Desplazar | Mover un elemento una posición hacia la derecha |
> | Mano | La parte del array ya ordenada (a la izquierda) |
> | Estable | Respeta el orden relativo de los elementos iguales |
> | In-place | Ordena sin necesidad de arrays auxiliares |

📚 Volver al índice de la unidad · **Anterior:** 04 · Ordenación burbuja · **Siguiente:** 06 · Complejidad algorítmica: Big O

06 — Complejidad algorítmica: Big O

La balanza que pesa algoritmos sin parar el cronómetro ⏱️

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 06 · Complejidad algorítmica: Big O

📬 La idea en una frase

> **Big O no te dice cuántos segundos tarda un algoritmo: te dice cómo crece su tiempo cuando crece la cantidad de datos. La tendencia, no el cronómetro.**

Dos ordenadores, uno de 2024 y otro de 2004. El moderno gana siempre en un test rápido, pero eso no nos dice nada del *algoritmo*. Para comparar algoritmos de forma justa, miramos su **tasa de crecimiento**: ¿qué pasa cuando el número de datos `n` se hace MUY grande? Eso es Big O.

📈 Las complejidades más comunes

Notación	Nombre	Ejemplo	Para n = 1.000
O(1)	Constante	Acceder a `array[0]`	1 operación
O(log n)	Logarítmica	Búsqueda binaria	~10 operaciones
O(n)	Lineal	Búsqueda lineal	1.000 operaciones
O(n log n)	Casi lineal	Ordenaciones avanzadas (lo verás en la U08)	~10.000 operaciones
$O(n^2)$	Cuadrática	Burbuja, inserción	1.000.000 de operaciones
$O(2^n)$	Exponencial	Fibonacci sin optimizar	¡inviable!

Fíjate en el último. Para `n = 1.000`, $O(2^n)$ no es “muy lento”: es que no termina ni en la era de los dinosaurios. La diferencia entre O(n) y $O(n^2)$ con datos grandes es la diferencia entre “echo un café mientras carga” y “me jubilo antes de que termine”.

🧪 Big O en código

Vamos a ver las tres más habituales con ejemplos en Java:

```java
public class EjemplosComplejidad {

    // O(1) — CONSTANTE: siempre igual, no importa el tamaño
    public static int obtenerPrimero(int[] array) {
        return array[0];  // un solo paso, siempre
    }

    // O(n) — LINEAL: crece en proporción a n
    public static int sumar(int[] array) {
        int suma = 0;
        for (int numero : array) {
            suma += numero;  // n pasos
        }
        return suma;
    }

    // O(n²) — CUADRÁTICA: dos bucles anidados
    public static void imprimirPares(int[] array) {
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array.length; j++) {
                System.out.println(array[i] + ", " + array[j]);  // n × n pasos
            }
        }
    }

    public static void main(String[] args) {
        int[] datos = {10, 20, 30, 40, 50};

        System.out.println("O(1): " + obtenerPrimero(datos));
        System.out.println("O(n): " + sumar(datos));
        System.out.println("O(n²): mira la consola llenándose de pares...");
        imprimirPares(datos);
    }
}
```

La regla del pulgar: **un bucle solo → O(n). Un bucle dentro de otro → O(n²).** Y si el bucle solo recorre la mitad de los datos… sigue siendo O(n), no O(n/2). Las constantes no cuentan.

📏 Las reglas prácticas de Big O

1. **Ignora las constantes**: O(2n) es lo mismo que O(n). El 2 no importa cuando n tiende a infinito.
2. **Quédate con el término dominante**: $O(n^2 + n) \rightarrow O(n^2)$. El n^2 se come al n cuando n crece.
3. **Los bucles anidados multiplican**: un bucle dentro de otro → $n \times n \rightarrow O(n^2)$.
4. **Los bucles secuenciales suman**: un bucle y luego otro → $O(n + n) \rightarrow O(2n) \rightarrow O(n)$.

```java
public class ReglasBigO {
    public static void main(String[] args) {
        // REGLA 1: las constantes no importan
        // O(2n) → O(n)
        // O(100n) → O(n)

        // REGLA 2: el término dominante se queda
        // O(n² + 5n + 1) → O(n²)
        // O(n + log n) → O(n)

        // REGLA 3: bucles anidados → multiplica
        // for (i...) { for (j...) { } }  →  O(n × n) → O(n²)

        // REGLA 4: bucles secuenciales → suma
        // for (i...) { }  for (i...) { }  →  O(n + n) → O(2n) → O(n)

        System.out.println("Big O no es magia, es simplificar.");
        System.out.println("Pregúntate: ¿qué pasa cuando n se hace MUY grande?");
    }
}
```

> 📝 **Nota:** Big O describe el **peor caso** (la cota superior): "como mucho, tardará esto". Existen también Big Omega Ω (mejor caso) y Big Theta Θ (caso promedio), pero con Big O tienes suficiente para empezar. Y para aprobar, también.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** para calcular Big O de memoria: cuenta bucles y mira si se anidan o se suceden. Si tienes dudas entre O(n) y $O(n^2)$, piensa en un array de un millón: ¿recorre una vez (un millón de pasos) o un millón de veces (un billón)?

Ejercicio: el analista de complejidades

Dime la complejidad Big O de cada método (las respuestas, escondidas):

```java
public class Analisis {

    public static int metodoA(int[] array) {
        int total = 0;
        for (int i = 0; i < array.length; i++) {
            total += array[i];
        }
        for (int i = 0; i < array.length; i++) {
            total += array[i] * 2;
        }
        return total;
    }

    public static void metodoB(int[] array) {
        for (int i = 0; i < array.length; i++) {
            for (int j = 0; j < array.length; j++) {
                System.out.println(array[i] + " " + array[j]);
            }
        }
    }

    public static int metodoC(int[] array) {
        return array[array.length - 1];
    }
}
```

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué mide exactamente Big O?
2. ¿Cuál es la complejidad de un bucle anidado?
3. ¿O(2n) y O(n) son lo mismo?
4. Ordena de menor a mayor: O(1), $O(n^2)$, O(n), O(log n), $O(2^n)$.

✅ Resumen en 3 frases

1. Big O describe **cómo crece el tiempo** de un algoritmo cuando crece la cantidad de datos, no los segundos exactos.
2. Las reglas de oro: ignora constantes, quédate con el término dominante, **anidar multiplica** y **secuenciar suma**.
3. Con datos grandes, la diferencia entre O(n) y O(n^2) es la diferencia entre un café y una jubilación.

> 🦎 **Vocabulario rápido**

Término	Idea general
Big O	Notación para la cota superior de crecimiento de un algoritmo
Término dominante	El que manda cuando `n` es enorme (el `n²` de `n² + n`)
Anidar	Meter un bucle dentro de otro → multiplica la complejidad
O(1)	Constante: siempre el mismo número de pasos
O(log n)	Cada paso descarta la mitad: crece muy despacio
Exponencial	O(2^n): crece tan rápido que es inviable con datos medianos

📚 Volver al índice de la unidad · **Anterior:** 05 · Ordenación por inserción · **Siguiente:** 07 · Elegir el algoritmo adecuado

07 — Elegir el algoritmo adecuado

La tabla de decisión del buen programador, para no usar una excavadora como cucharilla 🧠

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 07 · Elegir el algoritmo adecuado

📬 La idea en una frase

> **El mejor algoritmo no es el más famoso ni el más rápido en teoría: es el que encaja con tu tamaño de datos, su orden inicial y lo que necesitas. Elegir bien es la mitad del trabajo.**

¿Para abrir una nuez usarías una excavadora? Pues eso. Hay quien clava una búsqueda binaria en un array de 5 elementos, y quien ordena un millón de datos con burbuja. Los dos están mal: uno desperdicia esfuerzo, el otro desperdicia su tiempo de vida. Elegir el algoritmo adecuado es parte del oficio.

📋 La tabla de resumen

Algoritmo	Complejidad	¿Cuándo usarlo?	¿Cuándo NO?
Búsqueda lineal	O(n)	Arrays pequeños o desordenados	Arrays grandes y ordenados (la binaria gana)
Búsqueda binaria	O(log n)	Arrays grandes **ordenados**, con muchas búsquedas	Arrays desordenados (¡basura sin aviso!)
Burbuja	$O(n^2)$	Aprender y arrays muy pequeños (< 50)	Cualquier cosa que merezca la pena
Inserción	$O(n^2)$ / O(n)	Arrays pequeños o **casi ordenados**	Arrays grandes y desordenados

🧮 La pregunta clave: ¿ordenar antes de buscar?

Vale, la búsqueda binaria es rapidísima… pero exige un array ordenado. Y ordenar también cuesta. Entonces: ¿merece la pena ordenar primero?

La regla del buen administrador:

- Si **ordenas una vez y buscas muchas veces** → ordena con algo decente (en la U08 verás cómo) y luego usa binaria. La inversión se amortiza.
- Si **buscas una vez** en un array desordenado → búsqueda lineal directa. Ordenar solo para una búsqueda es regar el jardín con champán.

Y una curiosidad: para arrays **muy pequeños** (menos de ~50 elementos), la búsqueda lineal suele ganar incluso con datos ordenados, porque la sobrecarga de la binaria no compensa. La teoría importa, pero el contexto manda.

🏫 Ejemplo guiado: el catálogo de la tienda

Tienes una tienda con `notas` de clientes y quieres saber la nota de "Luis". El catálogo está en un `int[] notas` con los valores desordenados y solo vas a preguntar una vez. ¿Qué usas?

```
public class Tienda {
    // Una sola búsqueda sobre datos desordenados → búsqueda lineal
    public static int buscarNota(int[] notas, int objetivo) {
        for (int i = 0; i < notas.length; i++) {
            if (notas[i] == objetivo) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] notas = {7, 9, 5, 8, 6, 4};
        int posicion = buscarNota(notas, 8);
        System.out.println("El 8 está en la posición " + posicion);
    }
}
```

¿Y si tu tienda recibiera **miles de consultas al día** sobre el mismo catálogo? Entonces merece la pena ordenar el array una vez (cuesta $O(n^2)$ con lo que sabes hoy, pero en la U08 aprenderás más rápido) y usar búsqueda binaria en cada consulta. **Ordenar una vez, buscar mil.**

📊 Reglas de oro para decidir

1. **¿Es pequeño (< 50)?** → Cualquiera vale: usa lineal o inserción por simplicidad.
2. **¿Es grande y desordenado?** → No uses burbuja ni inserción. Espera a la U08 (QuickSort, MergeSort).
3. **¿Es grande y ordenado?** → Búsqueda binaria, sin pensarlo.
4. **¿Está casi ordenado?** → Inserción arrasa: O(n) en la práctica.
5. **¿Voy a buscar muchas veces?** → Invierte en ordenar bien y busca con binaria.
6. **¿Voy a buscar una sola vez?** → Lineal directa, sin dramas.

> ⚠️ **Advertencia:** la elección de algoritmo también depende de otros factores que verás más adelante: la **estabilidad** (¿mantener el orden de los iguales?), la **memoria** disponible y si los datos caben en memoria. Por ahora, con estas reglas de oro sobrevives a cualquier examen y a casi cualquier app.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** antes de escribir una línea, pregúntate: *¿qué tamaño tiene mi array? ¿Está ordenado? ¿Cuántas veces voy a hacer esta operación?* Tres preguntas, tres respuestas, y el algoritmo se elige solo.

Ejercicio: el jefe de almacén

Para cada escenario, elige el algoritmo más adecuado y justifica en una línea:

1. Un array de **8** notas desordenado, buscar la nota de un alumno una sola vez.
2. Una agenda de **10.000** contactos ordenada alfabéticamente, donde buscarás nombres constantemente.
3. Un array de **60.000** mediciones desordenadas que hay que dejar ordenadas de menor a mayor.
4. Un array de **200** números ya casi ordenados (solo un par de despistados fuera de sitio).

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuándo gana la búsqueda lineal a la binaria aunque el array esté ordenado?
2. ¿Cuándo merece la pena ordenar antes de buscar?
3. ¿Qué algoritmo eliges para un array casi ordenado?
4. ¿Qué regla de oro usarías para un array de 100.000 elementos desordenado?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. El algoritmo adecuado depende del **tamaño**, el **orden inicial** y la **frecuencia de las operaciones**.
2. **Ordenar una vez y buscar muchas** amortiza la inversión; buscar una sola vez no merece ordenar.
3. Regla rápida: pequeño o desordenado → lineal; grande y ordenado → binaria; casi ordenado → inserción; grande y desordenado → espera a la U08.

> 🦎 **Vocabulario rápido**

Término	Idea general
Contexto	Tamaño, orden inicial y frecuencia de uso de tus datos
Amortizar	Recuperar una inversión inicial con el uso repetido
Sobrecarga	El coste fijo extra de un algoritmo sofisticado en datos pequeños
Trade-off	Compromiso: ganar en algo pagando algo a cambio
Estabilidad	Mantener el orden relativo de los elementos iguales

📚 Volver al índice de la unidad · **Anterior:** 06 · Complejidad algorítmica: Big O · **Siguiente:** 08 · Be the Code: búsqueda binaria desde cero

08 — Be the Code: búsqueda binaria desde cero

Programa binaria y burbuja sin red de seguridad, como en las entrevistas de Google 🕶️

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 08 · Be the Code: búsqueda binaria desde cero

---

📬 La idea en una frase

> **Este punto no tiene teoría nueva: tiene dos retos. Programar búsqueda binaria y burbuja a mano, paso a paso, sin mirar los apuntes. Si algún día quieres trabajar en una FAANG, esto lo tienes que escribir dormido.**

La búsqueda binaria es un clásico de las entrevistas técnicas de Google, Amazon y compañía. No porque sea difícil, sino porque el **off-by-one** castiga sin piedad a quien confía de más. Vamos a entrenarte para que tú no seas esa víctima.

🕶️ Don Tip: la receta mental de la binaria

Antes de escribir código, memoriza la receta:

1. Dos punteros: `izquierda = 0` y `derecha = array.length - 1`.
2. Mientras `izquierda <= derecha`: calcula `medio = izquierda + (derecha - izquierda) / 2`.
3. Si `array[medio] == objetivo` → devuelve `medio`.
4. Si `array[medio] < objetivo` → `izquierda = medio + 1`.
5. Si no → `derecha = medio - 1`.
6. Fin del bucle → devuelve `-1`.

El error más común (incluso con 10 años de experiencia) es el **off-by-one**: ¿`<=` o `<`? ¿`medio + 1` o `medio`? La respuesta: `<=` en la condición, y `medio + 1` / `medio - 1` al mover los punteros. Sin el `+1`/`-1`, si el objetivo no está, el bucle puede quedarse girando para siempre.

🧩 RETO 1: Búsqueda binaria desde cero

Aquí tienes el esqueleto. Complétalo. **Sin mirar los apuntes.** Tu cerebro tiene que recordar, no copiar:

```
public class RetoBinaria {

    public static int busquedaBinaria(int[] array, int objetivo) {
        // 🧠 TU CÓDIGO AQUÍ
        return -1;
    }

    public static void main(String[] args) {
        int[] pruebas = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19};

          System.out.println("El 7 está en posición: " + busquedaBinaria(pruebas, 7));
// 3
         System.out.println("El 13 está en posición: " + busquedaBinaria(pruebas, 13));
// 6
          System.out.println("El 8 está en posición: " + busquedaBinaria(pruebas, 8));
// -1
         System.out.println("El 19 está en posición: " + busquedaBinaria(pruebas, 19));
// 9
          System.out.println("El 1 está en posición: " + busquedaBinaria(pruebas, 1));
// 0
              System.out.println("Array vacío: " + busquedaBinaria(new int[]{}, 5));
// -1
    }
}
```

Pasos guiados (resiste a leerlos todos de golpe):

1. Declara `izquierda` y `derecha`.
   > **¿Atascado?**
2. Escribe el `while` con la condición correcta.
   > **¿Atascado?**
3. Calcula el `medio`.
   > **¿Atascado?**
4. Compara y decide los tres casos.

> ¿Atascado?

5. Fuera del bucle, devuelve el “no encontrado”.

> ¿Atascado?

> 💡 **Consejo de depuración:** si algo falla, dibuja el array en un papel con 3 elementos y simula los pasos con un objetivo que no esté. El papel y el boli son tus mejores herramientas de depuración. Sí, en pleno 2026, y sí, funcionan.

> 🔄 Solución completa

🧩 RETO 2: Burbuja a mano

Ahora toca la burbuja. Misma regla: sin mirar. Completa el método `ordenar` para que deje el array de menor a mayor. Debería mostrar `11 12 22 25 34 64 90`:

```
public class RetoBurbuja {

    public static void ordenar(int[] array) {
        // 🧠 TU CÓDIGO AQUÍ
    }

    public static void main(String[] args) {
        int[] datos = {64, 34, 25, 12, 22, 11, 90};

        ordenar(datos);

        for (int numero : datos) {
            System.out.print(numero + " ");
        }
    }
}
```

Pasos guiados:

1. Dos bucles anidados: el exterior repite pasadas.

> ¿Atascado?

2. El interior compara parejas vecinas, llegando cada vez un elemento menos.

> ¿Atascado?

3. Si están desordenados, intercambia con una variable temporal.

> **¿Atascado?**

4. Extras de nivel: añade el flag `huboIntercambio` con su `break`.

> **¿Atascado?**

> 🔄 **Solución completa**

🧩 EL LÍO: la burbuja que apesta

El departamento de calidad ha recibido este algoritmo. Algo huele mal. Identifica los errores y explica por qué no funciona:

```
public class BurbujaLiosa {
    public static void ordenar(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            for (int j = 0; j < arr.length; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                }
            }
        }
    }
}
```

> 👓 **Don Tip:** cuando un bucle accede a `arr[j + 1]`, asegúrate de que `j + 1` no se salga del array. Y pregúntate: ¿recorro menos elementos en cada pasada?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. En búsqueda binaria, ¿qué condición usa el `while`: `izquierda <= derecha` o `izquierda < derecha`?

2. ¿Por qué `izquierda = medio` (sin el `+1`) puede colgar el bucle?
3. ¿Qué pasa si haces burbuja con `j < array.length` en el bucle interior?
4. ¿Para qué sirve el flag `huboIntercambio` en la burbuja?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. La búsqueda binaria se escribe con **dos punteros**, el **medio anti-desbordamiento** y los **`+1`/`-1` sagrados**; con eso, no hay off-by-one que valga.
2. La burbuja se construye con **dos bucles anidados**, un intercambio con variable temporal y, si eres listo, un **flag** que corta en cuanto está ordenado.
3. Escribir ambos **a mano** (sin mirar) es el ejercicio de la unidad: si lo logras, te llevas la medalla al cinturón.

> 🐛 **Vocabulario rápido**

Término	Idea general
Off-by-one	Error de "por uno": `<=` vs `<`, `medio+1` vs `medio`
Puntero	Índice que delimita el segmento actual (`izquierda`, `derecha`)
Anti-desbordamiento	`izquierda + (derecha - izquierda) / 2` en vez de `(izquierda+derecha)/2`
Variable temporal	El `temp` que guarda un valor durante el intercambio
Flag	Booleano que avisa de si pasó algo en la pasada

📚 Volver al índice de la unidad · **Anterior:** 07 · Elegir el algoritmo adecuado · **Siguiente:** 09 · Repaso interactivo

09 — Repaso interactivo: ordena y busca como un profesional

El cierre práctico de la unidad, con búsquedas, burbujas y un algoritmo que no va a funcionar 😈

> 🗺️ **Estás en:** 🧩 **U07 · Algorítmica I: Fundamentos** → 09 · Repaso interactivo

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```java
public class Misterio {
    public static void main(String[] args) {
        int[] datos = {10, 20, 30, 40, 50, 60};
        int izquierda = 0;
        int derecha = datos.length - 1;
        int objetivo = 40;
        int pasos = 0;

        while (izquierda <= derecha) {
            int medio = izquierda + (derecha - izquierda) / 2;
            pasos++;
            if (datos[medio] == objetivo) {
                    System.out.println("Encontrado en " + medio + " con " + pasos + "
pasos.");
                return;
            }
            if (datos[medio] < objetivo) {
                izquierda = medio + 1;
            } else {
                derecha = medio - 1;
            }
        }
        System.out.println("No está. Pasos: " + pasos);
    }
}
```

¿Qué imprime por pantalla? Elige sabiamente:

1. **`Encontrado en 3 con 1 pasos.`** → Confundes el número de pasos con la posición: la búsqueda no llega en una sola vuelta. ❌
2. **`Encontrado en 3 con 3 pasos.`** → ✅ ¡Correcto! Con 6 elementos, `medio` es `0 + (5-0)/2 = 2` → `datos[2] = 30`, que es menor que 40, así que `izquierda = 3`. Segunda vuelta: `medio = 3 + (5-3)/2 = 4` → `datos[4] = 50`, que es mayor, así que `derecha = 3`. Tercera vuelta: `medio = 3 + (3-3)/2 = 3` → `datos[3] = 40`. ¡Bingo en 3 pasos! Con `return` (estamos en `main`) el programa termina ahí.
3. **`No está. Pasos: 4.`** → El 40 está en el array, solo que la búsqueda binaria no lo caza a la primera. ❌

> 🔄 Solución

🔥 Fireside Chat: búsqueda lineal vs búsqueda binaria

> *Dos veteranos de la búsqueda discuten junto a la máquina de café.*

Lineal: — Yo soy la básica. Recorro elemento por elemento, sin exigirle nada a nadie. ¿Desordenado? Sin problema. ¿Cinco elementos? Un momento. ¿Un millón? Unos milloncitos de pasos, pero voy.

Binaria: — Unos milloncitos. Qué generosidad. Yo con un millón tardo veinte pasos. Veinte. Mientras tú sudas, yo ya he terminado y me estoy pidiendo otro café.

Lineal: — ¿Y quién te ha dado permiso para ser tan lista? El array **ordenado**. Si los datos llegan desordenados, tú no sirves ni para abrir la puerta. Yo, en cambio, funciono siempre. Es la vida: sin exigir nada, pero sin grandes alegrías.

Binaria: — Ordenar una vez y buscar mil, y verás. Yo soy la que salva a las apps con millones de usuarios. Tú eres… el plan B.

Lineal: — El plan B que no se estrella. Cuando tu array está desordenado y a nadie le apetece ordenarlo, a mí me llaman. Y no me quejo.

Binaria: — Vale, cada una en su terreno: tú, lo pequeño y desordenado; yo, lo grande y ordenado con muchas búsquedas. ¿Tregua?

Lineal: — Tregua. Pero que sepas que en los arrays de 5 elementos te gano hasta a ti, con tus humos de `izquierda + (derecha - izquierda) / 2`.

> La lección: ninguna es mejor "en general". **Lineal** para datos pequeños o desordenados; **binaria** para datos grandes y ordenados con muchas búsquedas. El contexto decide.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy una receta de cocina para el ordenador: finita, precisa y sin ambigüedades.**
2. **Recorro el array elemento por elemento hasta encontrar el objetivo. No exijo orden, pero soy lenta con datos grandes.**
3. **Abro el diccionario por la mitad y descarto media tonelada de papel en cada intento. Exijo orden, o te devuelvo basura sin avisar.**

4. **Soy el patoso de la ordenación: comparo vecinos e intercambio, una y otra vez, hasta que los grandes suben como burbujas.**
5. **Ordeno como en el póker: coloco cada carta nueva en su sitio dentro de la mano que ya tengo ordenada.**
6. **No mido segundos: mido cómo crece el tiempo cuando crecen los datos.**

> 🔁 **Respuestas**

👹 CONRAD VS EL MUNDO: “El algoritmo que no termina”

> *CONRAD, nuestro compilador cascarrabias, opina sobre el clásico del novato.*

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: *CONRAD, mi búsqueda binaria se queda colgada*. Y yo: vale, ¿qué tiene el bucle? *Pues no lo sé, no lo he mirado.* ¡AY, MADRE MÍA! Un `while (izquierda <= derecha)` que en lugar de `izquierda = medio + 1` pone `izquierda = medio`… ¿sabes lo que pasa? El segmento no se reduce. `izquierda` se queda atascada y el bucle da vueltas como un gato persiguiéndose la cola. Enseñándole el algoritmo a un robot aspirador con dirección propia.

Y luego está el de la burbuja: `for (int j = 0; j < array.length; j++)` y dentro `array[j + 1]`. ¿DE VERDAD? Cuando `j` llega al final, `j + 1` sale del array. ¿A qué esperas, a que te lance la `ArrayIndexOutOfBoundsException` para leer el mensaje? El mensaje ya te está diciendo el índice, la línea y el motivo. LÉELO.

Y el clásico: mi algoritmo ordena pero muy lento. Ya. ¿Y qué usaste? *Burbuja*. Con un array de medio millón. ¿Sabes cuántos intercambios son? Los que son. Y no es culpa del algoritmo, que avisa en la etiqueta: “O(n^2), para arrays pequeños”. El algoritmo no tiene la culpa de que no leyeras la etiqueta.

La lección: los bucles de los algoritmos se cuelgan por dos motivos: o la condición nunca avanza hacia `false`, o el índice se sale del array. Antes de llorar sobre el teclado, comprueba esas dos cosas. El 90% de los “algoritmos colgados” se arreglan con un vistazo.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. ¿Cuál es la complejidad de la búsqueda binaria?
    - a) O(n) b) O(log n) c) O(1)
2. ¿Cuántos pasos máximos necesita la búsqueda binaria para un array de 1.024 elementos?
    - a) 10 b) 1.024 c) 11
3. ¿Qué devuelve `buscar(new int[]{}, 5)` con una búsqueda binaria bien hecha?
    - a) `-1` b) `0` c) `Exception`
4. ¿Cuál de estas es O(n²)?
    - a) Un bucle secuencial b) Dos bucles anidados c) Acceder a `array[0]`

> 🔄 **Soluciones**

⚡ Laboratorio de tortura: la máquina que no ordena

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Tiene **3 errores** que impiden que compile o que falle en ejecución, y 1 error de lógica que hace que el resultado sea incorrecto cuando lo arregles.

```
public class Tortura
    public static void main(String[] args) {
        int[] datos = {3, 1, 4, 2};
        for (int i = 0; i < datos.length; i++) {
            for (int j = 0; j < datos.length; j++) {
                if (datos[j] < datos[j + 1]) {
                    int temp = datos[j];
                    datos[j] = datos[j + 1];
                    datos[j + 1] = temp;
                }
            }
        }
        for (int n : datos) System.out.print(n + " ")
    }
}
```

Fallo intencionado: uno de los errores parece correcto a simple vista porque “se ve bien”, pero hace que el programa falle en ejecución en cuanto arranca.

Tu tarea: conseguir que compile, que ejecute y que imprima `1 2 3 4`. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún `;` que falte? *no → sigue buscando.*
   > **¿Y si sigo atascado?**
2. ¿Compila ya? *no → mira el mensaje de error.*
   > **¿Y si sigo atascado?**
3. ¿Compila pero explota al ejecutar? *Es el error de índices: el bucle interior llega demasiado lejos.*
   > **¿Y si sigo atascado?**
4. ¿Ejecuta e imprime `4 3 2 1`? *Es el error de lógica: el signo de la comparación ordena al revés.*
   > **Solución final**

Los **3 errores** que impiden compilar o ejecutar:

1. Falta la `{` de apertura de la clase después de `Tortura`.
2. Falta el `;` al final de `System.out.print(n + " ")`.
3. `j < datos.length` accede a `datos[j + 1]` fuera del array: `ArrayIndexOutOfBoundsException`. Debe ser `j < datos.length - 1 - i`.

El **error de lógica**: `datos[j] < datos[j + 1]` ordena **de mayor a menor**. Compila y ejecuta perfectamente, pero imprime `4 3 2 1` en vez de `1 2 3 4`. La burbuja sube el mayor hacia el final comparando con `>`, no con `<`.

```java
public class Tortura {
    public static void main(String[] args) {
        int[] datos = {3, 1, 4, 2};
        for (int i = 0; i < datos.length - 1; i++) {
            for (int j = 0; j < datos.length - 1 - i; j++) {
                if (datos[j] > datos[j + 1]) {
                    int temp = datos[j];
                    datos[j] = datos[j + 1];
                    datos[j + 1] = temp;
                }
            }
        }
        for (int n : datos) System.out.print(n + " ");
    }
}
```

Salida correcta: `1 2 3 4`. Con la versión rota, una vez arreglados los otros errores, la comparación `<` daba `4 3 2 1`: el signo era la pista del error de lógica.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Recetario Humano**	Explicar qué es un algoritmo y sus 5 propiedades sin mirar
🏅 **El Buscador Jedi**	Escribir búsqueda binaria a mano y que funcione a la primera
🏅 **El Crupier**	Explicar inserción con la analogía de las cartas, sin mirar
🏅 **El Analista**	Calcular Big O de un fragmento de código en menos de un minuto
🏅 **El Jefe de Almacén**	Elegir el algoritmo correcto para cada escenario del punto 7
🏅 **El Cazador de Errores**	Encontrar los 3+1 errores del Laboratorio sin usar las pistas

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```java
public class Misterio2 {
    public static void main(String[] args) {
        int[] datos = {2, 4, 6, 8};
        int contador = 0;

        for (int i = 0; i < datos.length; i++) {
            for (int j = i + 1; j < datos.length; j++) {
                if (datos[i] < datos[j]) {
                    contador++;
                }
            }
        }
        System.out.println(contador);
    }
}
```

2. **La copia que se desordena:** ¿qué le pasa a la burbuja si en lugar de comparar `>` comparas `>=`? ¿Afecta a la estabilidad del algoritmo?
3. **El detective:** tu búsqueda binaria devuelve `-1` para un número que SÍ está en el array. ¿Qué herramienta usas y qué variables miras primero?
4. **Verdadero o falso:** "la búsqueda binaria funciona con cualquier array, solo que a veces es más lenta".

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Notación que mide cómo crece el tiempo (3 letras + espacio + 1 letra: "O" más otra letra)
3. La complejidad de la búsqueda binaria (empieza por "log")
5. Algoritmo que intercambia vecinos hasta ordenar (8 letras)
6. Secuencia finita y precisa de pasos (9 letras)

Vertical:
2. Señal de "no encontrado" que devuelven las búsquedas (2 letras)
4. Algoritmo que coloca cada carta en su sitio (9 letras)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, la diferencia entre búsqueda lineal y búsqueda binaria."**
2. **"¿Qué es la notación Big O y por qué es importante?"**
3. **"Escribe una búsqueda binaria en la pizarra. Ahora dime qué pasa si el array no está ordenado."**
4. **"¿Cuándo usarías ordenación por inserción en vez de burbuja?"**
5. **"¿Qué es el off-by-one y cómo lo evitas en la búsqueda binaria?"**
6. **"Un algoritmo O(n²) tarda 1 segundo con 1.000 elementos. ¿Cuánto tardará con 2.000? ¿Y con 10.000?"**

🤷 No hay preguntas tontas

> **❓ ¿Puedo usar `Arrays.sort()` y `Arrays.binarySearch()` de Java en vez de escribir los algoritmos?**

En tus programas reales, sí: Java trae utilidades ordenadas, eficientes y probadas, y verás `Arrays.sort()` pronto. Pero en esta unidad el objetivo es **entender la idea** que hay debajo. Es como aprender a hacer una suma a mano antes de usar la calculadora: no es que la calculadora sea mala, es que necesitas saber qué estás haciendo. Y en una entrevista, el entrevistador quiere ver que lo entiendes, no que sabes importar `java.util.Arrays`.

> **❓ ¿Por qué hay que decir "log n" y no simplemente "pocos pasos"?**

Porque "pocos pasos" no sirve para comparar: pocos comparado con qué. El logaritmo en base 2 te dice exactamente **cuántas veces puedes partir por la mitad** antes de llegar a 1. Y cuando alguien te dice "es O(log n)", tú sabes exactamente qué significa. La precisión es el sueldo del programador.

> **❓ Si burbuja e inserción son las dos O(n²), ¿por qué se dice que inserción es mejor?**

Por dos motivos: en el **caso medio** hace menos intercambios (desplaza, no intercambia de tres en tres), y en **arrays casi ordenados** es O(n) de verdad, mientras que burbuja sin flag sigue

dando pasadas enteras. En la práctica, con arrays pequeños, la inserción nota la diferencia. Con arrays grandes, ninguna de las dos: ahí llega la U08.

🎬 Poscréditos

El programador termina su catálogo: ordena las notas con inserción, y cuando un cliente pregunta por su nota, responde con búsqueda binaria en un parpadeo. Está contento. Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — Oye, no está mal. Inserción para ordenar, binaria para buscar… Y encima sabes decirme por qué no usaste burbuja para el millón de datos. ¿Seguro que eres el mismo que empezó esta unidad con el "echa sal al gusto"?

Novato: — *sonríe* Empecé con una receta de tortilla y he terminado ordenando el catálogo en 20 pasos. ¿Y ahora qué?

CONRAD: — *toma un sorbo* Todo esto es el calentamiento. Burbuja, inserción, lineal, binaria… son los fundamentos. El siguiente nivel usa técnicas que parten los problemas en trozos, se llaman a sí mismas y ordenan arrays de un millón en un suspiro. Otro tipo de bestias.

El novato guarda su proyecto, cierra el IDE y siente que ya no busca ni ordena: **piensa en términos de algoritmos**.

PRÓXIMAMENTE EN U08: Algorítmica II: técnicas avanzadas y modularidad. Divide y vencerás, QuickSort, MergeSort y recursividad. El momento en el que ordenar un millón de datos deja de ser una utopía. ⚡

📚 Volver al índice de la unidad · **Anterior:** 08 · Be the Code: búsqueda binaria desde cero · **Siguiente:** U08 · Algorítmica II: técnicas avanzadas y modularidad

U08 — Algorítmica II: técnicas avanzadas y modularidad

Recursividad, divide y vencerás, Quicksort, Mergesort y modularidad: las técnicas que hacen que tu código piense ⚡

Bienvenido, valiente explorador del stack de llamadas. En la U07 dominaste el arte de buscar y ordenar con bucles: búsqueda lineal, binaria, burbuja e inserción. Aprendiste a medir algoritmos con Big O y a no usar jamás burbuja en producción (lo prometiste). Pero los bucles se quedan cortos cuando el problema es *inherentemente* jerárquico. Y aquí es donde la programación se vuelve elegante.

Esta unidad es la que **te cambia la forma de pensar**. Vas a hacer que una función se llame a sí misma (sin volverte loco), vas a partir problemas por la mitad hasta hacerlos triviales, y vas a conocer a los dos reyes de la ordenación moderna: **Quicksort** y **Mergesort**. Cuando termines, verás recursividad por todas partes. Es como aprender una palabra nueva: de repente, la encuentras en cada esquina del código. Y para rematar, aplicarás esa misma mirada a tu propio código: **modularidad**, el arte de partirlo en piezas con nombre y con sentido.

Respira hondo. Y recuerda: para entender la recursividad, primero tienes que entender la recursividad.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Escribir funciones **recursivas** con caso base y caso recursivo, y razonar sobre la **pila de llamadas**.
- Implementar los clásicos: **factorial**, **Fibonacci** (ingenuo y con memoización), **palíndromos** y **torres de Hanói**.
- Aplicar la estrategia **divide y vencerás**: dividir, conquistar y combinar.
- Implementar **Quicksort** entendiendo la partición, el pivote y sus complejidades.
- Implementar **Mergesort** entendiendo la fusión y la estabilidad.
- **Elegir con criterio** el algoritmo de ordenación según el contexto (tamaño, orden previo, memoria, estabilidad).
- Reconocer otras técnicas como los algoritmos **voraces** (greedy) y el **backtracking**.

- Organizar tu código con criterio de **modularidad**: paquetes, cohesión alta y acoplamiento bajo.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · Recursividad	Caso base, caso recursivo, la pila de llamadas y el miedo al `StackOverflowError`	Todos
02 · Recursividad: ejemplos clásicos	Factorial, Fibonacci, palíndromos y torres de Hanói	Todos
03 · Divide y vencerás	Dividir, conquistar y combinar: el patrón de Julio César	Todos
04 · Quicksort	Partición, pivote, complejidades y por qué es tan famoso	Todos
05 · Mergesort	División por la mitad, fusión y estabilidad	Todos
06 · Comparación: cuándo usar cada ordenación	Guía práctica para elegir algoritmo y qué hay bajo `Arrays.sort()`	Todos
07 · Be the Code: quicksort desde cero	Implementar Quicksort y Mergesort paso a paso, sin mirar	⭐⭐⭐
08 · Otras técnicas algorítmicas	Algoritmos voraces (greedy) y backtracking	⭐⭐⭐
09 · Modularidad	Paquetes, cohesión y acoplamiento para partir tu código	Todos
10 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 9 primeros puntos son teoría y práctica en progresión. El 10o es el aterrizaje final: léelo justo después del 9o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA2 + RA6)

RA2: Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.

RA6: Escribe programas que manipulen información, seleccionando y utilizando tipos avanzados de datos.

CE	Criterio	Dónde se cubre
RA2 b)	Se han escrito programas simples.	✅ Todos
RA2 e)	Se han escrito llamadas a métodos estáticos.	✅ Puntos 1-7
RA2 f)	Se han utilizado parámetros en la llamada a métodos.	✅ Puntos 1-8
RA2 i)	Se ha utilizado el entorno integrado de desarrollo en la creación y compilación de programas simples.	✅ Todos
RA6 a)	Se han escrito programas que utilicen arrays.	✅ Puntos 3-8
RA6 b)	Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados.	✅ Puntos 4, 5 y 6

> 📌 La recursividad de los puntos 1 y 2 es la semilla de las **estructuras jerárquicas** (árboles, grafos) que explorarás con más profundidad en el módulo de Estructuras de Datos. Y la

> elección justificada de algoritmo del punto 6 es, directamente, un RA6 en estado puro: *seleccionar* el tipo de procesamiento adecuado a los datos.

🚪 ¿Por dónde empiezo?

- ¿Vienes de la U07 y estás fresco de Big O? → Arranca en el punto 1. Solo necesitas lo que viste en Fundamentos.
- ¿Lo tuyo es ver el resultado y no las tripas? → Salta al punto 4 y al punto 5 y vuelve después a la teoría de la recursividad.
- ¿Te flipan los retos? → Haz el punto 7 sin mirar el código de los puntos 4 y 5. Sufre con honor.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Recursividad

⏭️ **Al acabar la unidad, continúa en U09 · POO: clases y objetos.**

01 — Recursividad: una función que se llama a sí misma

Cuando tu código decide llamarse a sí mismo y tu cerebro empieza a humear 🔁

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 01 · Recursividad

---

📬 La idea en una frase

> **La recursividad es cuando una función se llama a sí misma para resolver un problema resolviendo una versión más pequeña de sí mismo, hasta llegar a un caso base que lo detiene todo.**

En la U07 viste que un algoritmo se puede escribir con bucles. Pero hay problemas que se *expresan* de forma mucho más natural diciendo "este problema es como el mismo problema, pero más pequeño". Para esos casos nació la recursividad: la técnica estrella de esta unidad.

🔁 El ciclo de la vida recursiva

Toda función recursiva necesita **dos partes imprescindibles**. Si te falta una, estás muerto:

> [!NOTE]
>
> - **Caso base**: la condición que detiene la recursión. Sin esto, tu programa se ejecuta hasta que la JVM se cansa y te lanza un `StackOverflowError`.
> - **Caso recursivo**: la llamada a sí misma, normalmente con una versión más pequeña del mismo problema.

La estructura general es siempre la misma:

```
public static tipo funcion(parametros) {
    if (/* condicion de parada */) {
        return /* valor base */;
    } else {
        // hacer algo con el problema actual
        return funcion(/* version mas pequena */);
    }
}
```

Piénsalo como limpiar tu habitación (suponiendo que algún día lo hagas): en lugar de limpiarla entera de golpe, coges una cosa del suelo y vuelves a llamar a la misma función "limpiarHabitacion" con lo que queda. Eventualmente no queda nada, y has terminado.

📚 La pila de llamadas: el origen de toda la diversión

Cada vez que una función se llama, Java reserva un trocito de memoria en el **stack** (la pila). Ese trocito se llama **stack frame** y guarda: los parámetros de la función, las variables locales, y la dirección de vuelta para cuando la función termine.

Si llamas a una función 5 veces, tienes 5 frames en el stack. Si la llamas 10.000 veces… bum. `StackOverflowError`.

```java
public class StackExplorer {

    static int prof = 0;

    static void recursivo() {
        prof++;
        System.out.println("Llamada numero: " + prof);
        recursivo(); // ¡no hay caso base!
    }

    public static void main(String[] args) {
        try {
            recursivo();
        } catch (StackOverflowError e) {
            System.out.println("Stack exploto en la llamada: " + prof);
        }
    }
}
```

Pruébalo. Verás que el número varía según tu máquina y la JVM: normalmente entre 10.000 y 20.000 llamadas. No es infinito. Nada lo es.

> [!WARNING] Sin caso base, no hay piedad. El stack tiene un límite y Java no te va a salvar. Cada frame ocupa espacio, y cuando el vaso se desborda, la JVM dice “hasta aquí llegamos”. Atrapa el `StackOverflowError` solo para inspeccionar (como arriba), nunca para “arreglar” una recursión rota.

🥊 Recursividad vs iteración: el duelo

Aspecto	Recursividad	Iteración (bucles)
Legibilidad	Muy elegante para problemas jerárquicos	Más verbosa pero clara
Memoria	Gasta stack (cada llamada = frame nuevo)	Solo variables locales
Velocidad	Más lenta (overhead de llamadas)	Más rápida

Aspecto	Recursividad	Iteración (bucles)
Stack overflow	Riesgo real si la profundidad es alta	No aplica
Casos ideales	Árboles, grafos, backtracking, divide y vencerás	Recorridos lineales, procesamiento simple

La regla de oro: **usa recursividad cuando el problema sea inherentemente recursivo** (estructuras jerárquicas, expresiones anidadas, ordenación avanzada). Para lo demás, un `for` de toda la vida. Nadie va a darte una medalla por hacer una suma de array con recursión.

🌰 Un primer ejemplo completo

Antes de los clásicos del punto 2, un ejemplo mínimo para que veas la estructura completa. Contar hasta `n`:

```
public class CuentaAtras {

    static void cuenta(int n) {
         if (n < 0) return;                // caso base: no hay números negativos que imprimir
        System.out.println(n);
         cuenta(n - 1);                   // caso recursivo: el mismo problema, un paso más pequeño
    }

    public static void main(String[] args) {
        cuenta(3);
    }
}
```

Salida:

```
3
2
1
0
```

Fíjate: primero baja imprimiendo 3, 2, 1, 0, y cuando `n < 0`, la llamada devuelve y todos los frames se cierran uno tras otro. La pila se vacía en orden inverso al que se llenó.

> [!TIP] El 90% de los bugs de recursión son dos: **olvidar el caso base** o **no reducir el problema** (llamarse a sí misma con los mismos parámetros). Si tu recursión no termina o no avanza, dibuja el árbol de llamadas en un papel. El papel y boli siguen siendo tus mejores herramientas de depuración.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué dos partes necesita toda función recursiva?
2. ¿Qué lanza la JVM si la recursión nunca termina?
3. ¿Qué guarda un *stack frame*?
4. ¿Por qué una suma de array con bucle suele ser mejor que con recursión?

> › 🔁 **Respuestas**

✅ Resumen en 3 frases

1. La recursividad es una **función que se llama a sí misma** con una versión más pequeña del problema.
2. Sin **caso base** no hay detención: el stack se llena y llega el `StackOverflowError`.
3. Usa recursión cuando el problema sea **inherentemente recursivo**; para lo lineal, los bucles siguen ganando.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Caso base | La condición que detiene la recursión |
> | Caso recursivo | La llamada a sí misma con un problema menor |
> | Stack frame | El trocito de memoria de cada llamada (parámetros, locales, retorno) |
> | Call stack | La pila de frames pendientes |

Término	Idea general
`StackOverflowError`	Lo que pasa cuando llenas la pila
Recursión de cola	Cuando la llamada recursiva es lo último que hace la función

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · Recursividad: ejemplos clásicos

02 — Recursividad: ejemplos clásicos

Factorial, Fibonacci, palíndromos y torres de Hanói: los cuatro jinetes del apocalipsis recursivo 🧮

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 02 · Recursividad: ejemplos clásicos

📬 La idea en una frase

> **Estos cuatro ejemplos te perseguirán el resto de tu carrera: aparecen en exámenes, entrevistas técnicas y conversaciones de ascensor con otros programadores, así que más vale dominarlos.**

En el punto 1 viste la teoría: caso base, caso recursivo y la pila. Ahora toca el museo de los clásicos. Cada uno te enseña una lección diferente, desde la elegancia lineal del factorial hasta el horror exponencial del Fibonacci ingenuo.

🧮 Factorial: el "Hola Mundo" de la recursividad

```java
public class Factorial {

    static long fact(int n) {
        if (n <= 1) return 1;          // caso base: 0! y 1! valen 1
        return n * fact(n - 1);        // caso recursivo
    }

    public static void main(String[] args) {
        System.out.println("fact(5) = " + fact(5));   // 120
        System.out.println("fact(0) = " + fact(0));   // 1
        System.out.println("fact(7) = " + fact(7));   // 5040
    }
}
```

¿Qué pasa cuando llamamos a `fact(4)`? Vamos paso a paso:

```
fact(4) → 4 * fact(3)
       → 4 * (3 * fact(2))
       → 4 * (3 * (2 * fact(1)))
       → 4 * (3 * (2 * 1))
       → 4 * (3 * 2)
       → 4 * 6
       → 24
```

Primero “baja” hasta el caso base, luego “sube” haciendo las multiplicaciones. Como un ascensor que baja al sótano y vuelve a subir parando en cada puerta. La complejidad es **O(n)**: una llamada por cada número.

> [!NOTE] Ojo con `long`: `fact(20)` ya es 2.432.902.008.176.640.000, casi al límite de `long`. Para 21 en adelante, desbordas. El factorial crece **muy** deprisa.

🐇 Fibonacci ingenua: no hagas esto en casa

```java
public class FiboNaive {

    static long fibo(int n) {
        if (n <= 1) return n;
        return fibo(n - 1) + fibo(n - 2);
    }

    public static void main(String[] args) {
        long inicio = System.currentTimeMillis();
        System.out.println("fibo(40) = " + fibo(40));
        long fin = System.currentTimeMillis();
        System.out.println("Tiempo: " + (fin - inicio) + " ms");
    }
}
```

Ejecuta esto. Serán unos segundos de infierno. ¿El problema? `fibo(40)` genera un **árbol de llamadas monstruoso**. Cada llamada se bifurca en dos, que se bifurcan en dos, etc. El número total de llamadas es aproximadamente **O(2ⁿ)**.

```
                        fibo(5)
                       /       \
                  fibo(4)       fibo(3)
                 /       \      /     \
            fibo(3)    fibo(2) fibo(2) fibo(1)
            /    \      /   \   /   \
       fibo(2) fibo(1) ...  ... ...  ...
       /    \
  fibo(1) fibo(0)
```

Cuenta las veces que se llama a `fibo(1)`. Exacto, demasiadas. Es como preguntarle a tu compañero una y otra vez la misma pregunta esperando una respuesta diferente. Eso, señor mío, es locura.

> [!TIP] Este es el ejemplo perfecto para entender por qué **la eficiencia importa**. Un algoritmo que se ve bonito en código puede ser un desastre en tiempo de ejecución. El Fibonacci ingenuo es el rey de los algoritmos bonitos pero inútiles para números grandes.

⚡ Fibonacci optimizado: con memoización, ahora sí

```java
public class FiboOptimizado {

    static long[] memo;

    static long fibo(int n) {
        if (n <= 1) return n;
        if (memo[n] != 0) return memo[n];      // ya lo calculamos antes
        memo[n] = fibo(n - 1) + fibo(n - 2);
        return memo[n];
    }

    public static void main(String[] args) {
        int n = 92;
        memo = new long[n + 1];
        long inicio = System.currentTimeMillis();
        System.out.println("fibo(" + n + ") = " + fibo(n)); // instantáneo
        long fin = System.currentTimeMillis();
        System.out.println("Tiempo: " + (fin - inicio) + " ms");
    }
}
```

¿Qué cambia? Guardamos los resultados en un array (`memo[]`) para no recalcular. Si ya hemos calculado `fibo(10)`, lo devolvemos directamente. Cada Fibonacci se calcula **una sola vez**.

La mejora es brutal: de **O(2ⁿ) a O(n)**. Esa es la mayor subida de nivel desde que pasaste de piedra a pokéball.

> [!NOTE] Esto se llama **memoización** (sí, sin la "r"). Consiste en: "si ya lo calculé, no lo vuelvo a calcular; lo guardo en un array o mapa y lo reutilizo". Es la base de la **programación dinámica**, que verás en unidades más avanzadas.

🔍 Palíndromos: la palabra espejo

Un palíndromo es una palabra que se lee igual hacia delante y hacia atrás: *reconocer*, *salas*, *ana*. ¿Cómo se comprueba con recursión?

```java
public class Palindromo {

    static boolean esPalindromo(String s, int inicio, int fin) {
        if (inicio >= fin) return true;                    // caso base: quedan 0 o 1 letras
        if (s.charAt(inicio) != s.charAt(fin)) return false; // caso base: las puntas no coinciden
        return esPalindromo(s, inicio + 1, fin - 1);       // caso recursivo: vamos "pelando" las puntas
    }

    public static void main(String[] args) {
        String[] pruebas = {"reconocer", "salas", "hola", "anana", "a"};
        for (String s : pruebas) {
            System.out.println(s + " -> " + esPalindromo(s, 0, s.length() - 1));
        }
    }
}
```

Salida:

```
reconocer -> true
salas -> true
hola -> false
anana -> true
a -> true
```

La idea es elegante: comparas la primera y la última letra; si no coinciden, no es palíndromo; si coinciden, repites con las letras de dentro. Como pelar una cebolla, pero de letras.

Torres de Hanói: el rey de los clásicos

El mito: tres varillas y una pila de discos de tamaño creciente. Hay que mover todos los discos de la varilla A a la C, con dos reglas: solo se mueve un disco a la vez y nunca puede quedar un disco grande sobre uno pequeño. La leyenda dice que cuando los monjes terminen, el mundo se acabará (con 64 discos, no nos preocupemos).

```java
public class Hanoi {

    static void mover(int n, char origen, char destino, char auxiliar) {
        if (n == 1) {
            System.out.println("Mueve disco 1 de " + origen + " a " + destino);
            return;                                    // caso base
        }
         mover(n - 1, origen, auxiliar, destino);        // los n-1 discos de arriba, a
la varilla auxiliar
        System.out.println("Mueve disco " + n + " de " + origen + " a " + destino);
          mover(n - 1, auxiliar, destino, origen);         // y ahora los n-1 discos,
encima
    }

    public static void main(String[] args) {
        mover(3, 'A', 'C', 'B');
    }
}
```

Con 3 discos, la salida es:

```
Mueve disco 1 de A a C
Mueve disco 2 de A a B
Mueve disco 1 de C a B
Mueve disco 3 de A a C
Mueve disco 1 de B a A
Mueve disco 2 de B a C
Mueve disco 1 de A a C
```

Son **$2^n - 1$** movimientos. Con 3 discos: 7. Con 64 discos: 18.446.744.073.709.551.615. Al ritmo de un movimiento por segundo, el universo se muere antes. Hanói es el ejemplo de que la recursión puede resolver en 6 líneas lo que iterativamente te haría sufrir de verdad.

> [!WARNING] Hanói es **exponencial** ($O(2^n)$) por naturaleza: no es que lo hayas implementado mal, es que el problema *es* exponencial. La moraleja es distinta: una solución recursiva corta y clara vale más que un bucle ilegible que intenta emularla con pilas manuales.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuál es la complejidad del factorial recursivo?
2. ¿Cuántas veces se llama `fibo(1)` al calcular `fibo(5)` con el método ingenuo? (pista: míralo en el árbol)
3. ¿Qué hace la memoización para mejorar Fibonacci?
4. ¿Cuántos movimientos necesita Hanói con 4 discos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. El **factorial** (O(n)) y los **palíndromos** son los ejemplos "lineales": recorren el problema hacia el caso base sin sorpresas.
2. El **Fibonacci ingenuo** es $O(2^n)$: los números explotan porque repite cálculos; la **memoización** lo deja en O(n).
3. Las **torres de Hanói** demuestran que la recursión expresa en pocas líneas problemas que iterativamente son un infierno.

> 🦎 **Vocabulario rápido**

Término	Idea general
Memoización	Guardar resultados ya calculados para reutilizarlos
Palíndromo	Palabra que se lee igual de delante atrás: *salas*
Recursión lineal	La que hace una sola llamada recursiva por invocación (factorial)
Recursión exponencial	La que se bifurca en varias llamadas (Fibonacci, Hanói)
`long`	Tipo para números grandes… hasta que el factorial lo desborda

📚 Volver al índice de la unidad · **Anterior:** 01 · Recursividad · **Siguiente:** 03 · Divide y vencerás

03 — Divide y vencerás

El patrón más antiguo de la informática: divide el problema, conquista cada parte y combina ⚔️

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 03 · Divide y vencerás

---

📬 La idea en una frase

> **Divide y vencerás (divide & conquer) consiste en partir un problema en subproblemas más pequeños, resolver cada uno por separado (normalmente con recursión) y combinar las soluciones.**

La estrategia es tan antigua como Julio César, pero aplicada a algoritmos sigue siendo igual de efectiva. Y sí, en la U07 ya la usaste sin saberlo: la **búsqueda binaria** parte el array por la mitad y descarta una parte en cada paso. Ese era el germen. Ahora le vas a poner nombre y apellidos.

🧩 Los tres pasos del patrón

1. **Dividir** el problema en subproblemas más pequeños y manejables.
2. **Conquistar** cada subproblema recursivamente (llamadas recursivas).
3. **Combinar** las soluciones de los subproblemas para obtener la solución del problema original.

```
Input grande
     │
     ├──Dividir──→ Subproblema A    → Conquistar (recursivo) → Combinar ──→ Output
     └──Dividir──→ Subproblema B    → Conquistar (recursivo) ─┘
```

La clave está en que **cada subproblema es una copia más pequeña del original**: por eso la recursión encaja tan bien. Y el caso base es el subproblema tan pequeño que ya se resuelve directamente.

📊 ¿Cuándo merece la pena?

No todo problema se beneficia de dividir. La magia aparece cuando el coste de **combinar** es barato y la división reduce de verdad el trabajo. Un clásico de esta unidad:

Algoritmo	Divide	Conquista	Combina	Complejidad
Búsqueda binaria	Parte por la mitad	Descartas una mitad	Nada	O(log n)
Mergesort	Parte por la mitad	Ordenas cada mitad	**Fusionas**	O(n log n)
Quicksort	Particiona con pivote	Ordenas cada zona	Nada (ya está)	O(n log n) promedio

> [!NOTE] Fíjate en la fila de Quicksort: el trabajo "caro" ocurre en la **división** (particionar alrededor del pivote), y la combinación es gratis. En Mergesort pasa lo contrario: la división es trivial (cortar por el medio) y el trabajo está en **combinar** (fusionar). Son dos sabores del mismo patrón.

💡 Un ejemplo sencillo: el máximo de un array

Aunque el máximo se resuelve con un bucle en una línea, veamos cómo lo enfocaría divide y vencerás para que veas el patrón en su mínima expresión:

```java
public class MaximoDyV {

    static int maximo(int[] arr, int izq, int der) {
        if (izq == der) return arr[izq];          // caso base: un solo elemento

        int mid = (izq + der) / 2;
        int maxIzq = maximo(arr, izq, mid);       // conquistar mitad izquierda
        int maxDer = maximo(arr, mid + 1, der);   // conquistar mitad derecha

        return Math.max(maxIzq, maxDer);          // combinar
    }

    public static void main(String[] args) {
        int[] datos = {3, 9, 2, 7, 5, 8, 1, 6};
        System.out.println("Máximo: " + maximo(datos, 0, datos.length - 1)); // 9
    }
}
```

Espera… eso es un bucle disfrazado, ¿no? Sí: su complejidad es O(n), igual que el bucle, porque cada elemento se visita una vez. La moraleja es que **divide y vencerás no siempre gana**: gana cuando la división evita trabajo repetido (como ordenar o buscar), no cuando simplemente reparte el mismo trabajo.

> [!TIP] Regla práctica para decidir: si el bucle simple es claro y rápido, usa el bucle. Divide y vencerás es para cuando *partir el problema* te permite **descartar** o **reutilizar** trabajo. La búsqueda binaria descarta media array; el factorial no descarta nada.

🌳 Aplicaciones famosas

Una vez entiendes el patrón, empiezas a verlo por todas partes. Es como cuando aprendes una palabra nueva y de repente la ves en cada esquina:

- **Búsqueda binaria**: O(log n). Ya la conoces de la UO7.
- **Mergesort y Quicksort**: los dos reyes de la ordenación, que verás en los puntos 4 y 5.
- **Exponenciación rápida**: calcular a^n en O(log n) en lugar de O(n).
- **Multiplicación de matrices (Strassen)**: submatrices en lugar de la triple operación clásica.
- **FFT (Transformada Rápida de Fourier)**: la base del JPEG, el MP3 y el WiFi.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuáles son los tres pasos del patrón divide y vencerás?
2. ¿En qué se diferencia el trabajo de Quicksort y Mergesort dentro del patrón?
3. ¿Por qué el “máximo con divide y vencerás” no mejora al bucle simple?
4. La búsqueda binaria, ¿qué paso del patrón “evita” (no necesita)?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Divide y vencerás es el patrón de **dividir, conquistar y combinar**, con cada subproblema más pequeño resuelto por recursión.
2. El caso base es el subproblema trivial; la complejidad depende de cuánto trabajo te ahorre la **división** (descartar o reutilizar).
3. No siempre gana al bucle: solo brilla cuando partir el problema **evita trabajo repetido**, como en ordenación y búsqueda.

> 🦎 **Vocabulario rápido**

Término	Idea general
Divide & Conquer	Dividir, conquistar y combinar
Subproblema	Una copia más pequeña del problema original
Caso base	El subproblema que se resuelve directamente
Combinar	Unir las soluciones parciales en la solución final
Particionar	Repartir elementos alrededor de un pivote (Quicksort)
Fusionar	Mezclar dos listas ordenadas en una (Mergesort)

📚 Volver al índice de la unidad · **Anterior:** 02 · Recursividad: ejemplos clásicos · **Siguiente:** 04 · Quicksort

04 — Quicksort: el rápido

El algoritmo de ordenación más famoso del mundo, con su partición, su pivote y sus manías 🚀

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 04 · Quicksort

---

📬 La idea en una frase

> **Quicksort elige un pivote, parte el array en menores y mayores a su alrededor, y repite recursivamente en cada mitad: divide y vencerás en estado puro.**

Creado por **Tony Hoare** en 1959. Sí, tiene más años que tus padres. Y sigue siendo el algoritmo de ordenación más usado del mundo. Por algo será.

🎯 Cómo funciona

1. Elegimos un **pivote** (un elemento del array).
2. Colocamos todos los elementos **menores** que el pivote a su izquierda y los **mayores** a su derecha. Esto se llama **particionar**.
3. Aplicamos el mismo proceso recursivamente a las dos zonas (izquierda y derecha del pivote).

Cuando el array tiene 0 o 1 elementos… ya está ordenado. Caso base.

🛠️ Implementación

```java
public class Quicksort {

    static void qs(int[] arr, int izq, int der) {
        if (izq >= der) return;   // caso base: 0 o 1 elementos

        int pivote = arr[(izq + der) / 2];  // elegimos el del medio
        int i = izq, j = der;

        // partición
        while (i <= j) {
            while (arr[i] < pivote) i++;
            while (arr[j] > pivote) j--;
            if (i <= j) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
                j--;
            }
        }

        // llamadas recursivas a cada zona
        qs(arr, izq, j);
        qs(arr, i, der);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        qs(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Ejecución paso a paso con `{3, 1, 4, 1, 5, 9, 2, 6}` (primera partición):

```
Array inicial:  [3, 1, 4, 1, 5, 9, 2, 6]
Pivote = arr[(0 + 7) / 2] = arr[3] = 1
Posiciones i=0, j=7

1. i no avanza (arr[0]=3 no es < 1); j retrocede hasta 3 (arr[3]=1 no es > 1)
2. i(0) <= j(3): intercambiamos arr[0] y arr[3] → [1, 1, 4, 3, 5, 9, 2, 6]; i=1, j=2
3. i no avanza (arr[1]=1 no es < 1); j retrocede a 1 (arr[2]=4 > 1)
4. i(1) <= j(1): se intercambia consigo mismo; i=2, j=0 → i > j, fin de la partición
5. Recursión izquierda: segmento [0..0] (un solo elemento, ya ordenado)
6. Recursión derecha: segmento [2..7] → [4, 3, 5, 9, 2, 6], con pivote 5
...
```

Fíjate: el pivote (1) ha quedado bien colocado y el array se ha partido en `[1]` y `[4, 3, 5, 9, 2, 6]`. Cada zona se ordena igual, recursivamente. Si eliges un pivote distinto, los intercambios cambian, pero el patrón es el mismo.

> [!TIP] El secreto de Quicksort está en la **partición**. Si consigues que los elementos se repartan más o menos equilibradamente, el algoritmo vuela. Si no… prepara los $O(n^2)$.

Elección del pivote

Estrategia	Ventaja	Desventaja
Primer elemento	Simple	Pésimo si el array ya está ordenado
Último elemento	Simple	Pésimo si el array ya está ordenado
Elemento central	Mejor equilibrio	Sigue teniendo casos malos
Mediana de tres (primero, medio, último)	Muy robusto	Un poco más de cálculo
Aleatorio	Evita el caso peor en la práctica	Aleatoriedad no determinista

> [!WARNING] Si eliges siempre el primer elemento como pivote y el array ya está ordenado, Quicksort se vuelve más lento que una tortuga con resaca: **$O(n^2)$**, literalmente peor que un

> `for` anidado cutre. Por eso en producción nadie usa “primer elemento” a secas.

📊 Complejidad

- **Caso promedio**: O(n log n) — casi siempre.
- **Mejor caso**: O(n log n) — cuando el pivote divide siempre en mitades iguales.
- **Peor caso**: O(n^2) — cuando el pivote es siempre el menor o el mayor (array ya ordenado con mal pivote).
- **Memoria**: O(log n) extra (por el stack de las llamadas recursivas). No necesita arrays auxiliares: ordena *in-place*.

🚫 ¿Es estable?

No. Durante la partición, dos elementos iguales pueden intercambiarse de posición. Si necesitas estabilidad (conservar el orden original de los elementos iguales), mejor usa Mergesort.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuáles son las tres partes de Quicksort?
2. ¿Cuál es el peor caso de Quicksort y cuándo ocurre?
3. ¿Qué estrategia de pivote hace que el peor caso “casi nunca” ocurra en la práctica?
4. ¿Es Quicksort estable?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Quicksort **particiona** alrededor de un pivote y repite recursivamente en cada zona: divide y vencerás.

2. Su **caso promedio** es O(n log n) y su **peor caso** $O(n^2)$, que se evita con una buena elección de pivote.
3. Es **in-place** (memoria O(log n)) y **no estable**; el trabajo importante ocurre en la partición.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Pivote | El elemento que divide el array en menores y mayores |
> | Partición | Repartir elementos alrededor del pivote |
> | In-place | Ordenar sin arrays auxiliares, moviendo el propio array |
> | Estable | Que conserva el orden original de los elementos iguales |
> | O(n log n) | Complejidad “casi lineal”: el estándar de la ordenación moderna |

📚 Volver al índice de la unidad · **Anterior:** 03 · Divide y vencerás · **Siguiente:** 05 · Mergesort

05 — Mergesort: el fiable

El algoritmo que siempre cumple lo que promete: O(n log n) pase lo que pase 🤝

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 05 · Mergesort

---

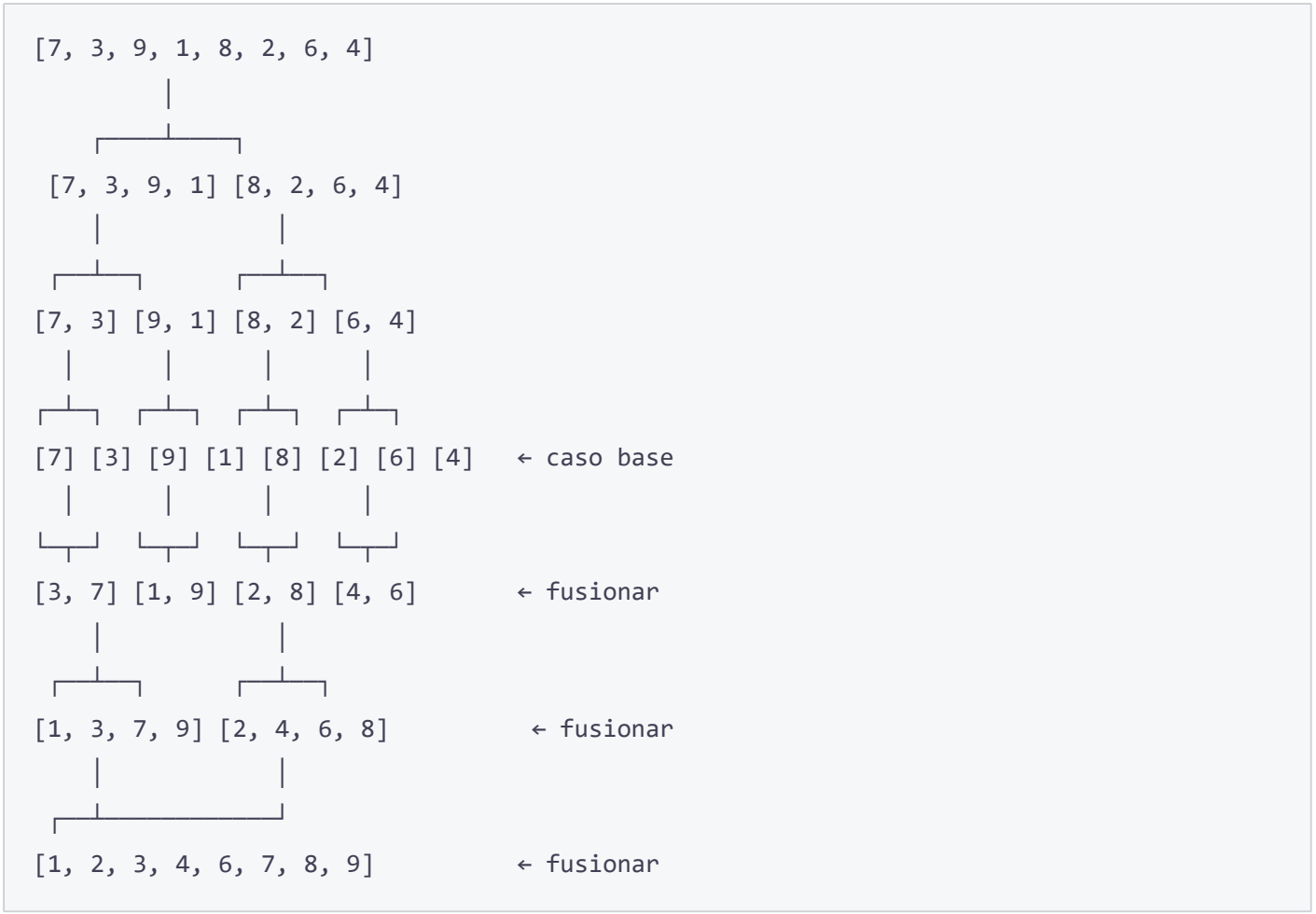
📬 La idea en una frase

> **Mergesort divide el array por la mitad, ordena cada mitad recursivamente y luego fusiona las dos mitades ordenadas en una sola: el "divide y vencerás" sin trampas.**

Creado por **John von Neumann** en 1945. Sí, el mismo de la arquitectura de ordenadores. El tío no paraba.

🎯 Cómo funciona

1. **Dividir** el array en dos mitades (por la mitad exactamente; no hay que elegir pivote).
2. **Ordenar** cada mitad recursivamente.
3. **Fusionar** (merge) las dos mitades ordenadas en un único array ordenado.



🛠️ Implementación

```java
public class Mergesort {

    static void ms(int[] arr, int izq, int der) {
        if (izq >= der) return;  // caso base

        int mid = (izq + der) / 2;
        ms(arr, izq, mid);                  // ordenar mitad izquierda
        ms(arr, mid + 1, der);              // ordenar mitad derecha
        fusionar(arr, izq, mid, der);         // combinar las dos mitades
    }

    static void fusionar(int[] arr, int izq, int mid, int der) {
        int[] tmp = new int[der - izq + 1];
        int i = izq, j = mid + 1, k = 0;

        // comparar elementos de las dos mitades
        while (i <= mid && j <= der)
            tmp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];

        // copiar lo que quede de la mitad izquierda
        while (i <= mid) tmp[k++] = arr[i++];

        // copiar lo que quede de la mitad derecha
        while (j <= der) tmp[k++] = arr[j++];

        // copiar el array temporal al original
        System.arraycopy(tmp, 0, arr, izq, tmp.length);
    }

    public static void main(String[] args) {
        int[] datos = {9, 3, 7, 1, 8, 2, 6, 4, 5};
        ms(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

[!TIP] La **fusión** es la parte interesante. Imagina dos pilas de cartas ya ordenadas: comparas las cartas superiores y te llevas la menor, una y otra vez. Como dos colas de supermercado que se unen en una caja y siempre pasa el que llega primero de cada cola.

📊 Complejidad

- **Siempre**: O(n log n). No importa cómo esté el array. Mergesort no tiene caso malo.
- **Memoria**: O(n) extra por el array temporal `tmp`. Ese es su punto débil.

La estructura es la típica de divide y vencerás: hay `log n` niveles de división, y en cada nivel se hace un trabajo total de `O(n)` (fusionar todos los fragmentos de ese nivel). Resultado: `n · log n`.

✅ ¿Es estable?

Sí. Cuando dos elementos son iguales, el de la izquierda va primero en la fusión (fíjate en el `<=` de la condición). Esto mantiene el orden original de los elementos iguales, algo que Quicksort no puede garantizar.

> [!NOTE] La estabilidad no es un concepto teórico aburrido. Es importante cuando ordenas por múltiples criterios. Por ejemplo, si ordenas una lista de alumnos por nota y luego por nombre, quieres que los que tienen la misma nota mantengan el orden alfabético. Mergesort te lo da. Quicksort te hace llorar.

🏆 Cuándo brilla Mergesort

- Cuando necesitas **estabilidad** a toda costa.
- Cuando el array está **casi ordenado** o lleno de duplicados (no le afecta).
- Cuando los datos no caben en memoria: el **mergesort externo** (fusionando trozos en disco) es la base de la ordenación de las bases de datos.

Y su punto débil: si la memoria es justa (sistemas embebidos, móviles con poca RAM), el array temporal de O(n) puede doler.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuál es el caso malo de Mergesort?
2. ¿Cuál es su punto débil frente a Quicksort?

3. ¿Por qué es estable?
4. ¿En qué se diferencia su “división” de la de Quicksort?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Mergesort **divide por la mitad**, ordena cada mitad y **fusiona** las mitades ordenadas.
2. Es **siempre O(n log n)** y **estable**, a cambio de O(n) de memoria extra.
3. Es la opción fiable cuando importa la estabilidad o el orden inicial es problemático.

> 🐛 **Vocabulario rápido**

Término	Idea general
Fusión (merge)	Combinar dos mitades ordenadas en una
Estable	Conserva el orden original de los elementos iguales
Divide y vencerás	El patrón de dividir, conquistar y combinar
Memoria O(n)	Necesita un array auxiliar del mismo tamaño
Mergesort externo	Variante que ordena datos que no caben en RAM (discos)

📚 Volver al índice de la unidad · **Anterior:** 04 · Quicksort · **Siguiente:** 06 · Comparación: cuándo usar cada ordenación

06 — Comparación: cuándo usar cada ordenación

La tabla de decisión definitiva para elegir algoritmo como un profesional ⚖️

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 06 · Comparación: cuándo usar cada ordenación

📬 La idea en una frase

> **No existe el "mejor algoritmo de ordenación": existe el mejor para tu caso concreto, y elegirlo bien es tan importante como saber implementarlo.**

Ya has visto Quicksort y Mergesort, y en la U07 sobreviviste a burbuja e inserción. Ahora toca la pregunta de examen, de entrevista y de la vida real: *¿cuál uso?* La respuesta siempre empieza igual: *depende*.

🧭 La tabla de decisión

Situación	Elige
Array pequeño (< 50 elementos)	Da igual, usa `Arrays.sort()`
Array grande con datos al azar	**Quicksort** (irá genial en promedio)
Array grande, casi ordenado o con muchos duplicados	**Mergesort** o Quicksort con mediana de tres
Necesitas estabilidad (orden relativo)	**Mergesort**
La memoria es justa (sistema embebido, móvil)	**Quicksort** (O(log n) vs O(n))
No te pagan por pensar	`Arrays.sort()` y a seguir con tu vida
El array es enorme y los datos están en disco	Mergesort externo (el de las bases de datos)

> [!WARNING] Nunca, bajo ningún concepto, uses **Burbuja (Bubble Sort)** en producción. Es $O(n^2)$, lento, y tus compañeros te odiarán. Es como usar un caracol para repartir pizzas: existe, pero no debería.

🥊 El duelo de los titanes

	Quicksort	Mergesort
Complejidad promedio	O(n log n)	O(n log n)
Peor caso	$O(n^2)$ (evitable con buen pivote)	O(n log n) siempre
Memoria extra	O(log n)	O(n)
Estable	No	Sí
Trabajo principal	En la **partición**	En la **fusión**
Ideal cuando	Velocidad y poca memoria	Estabilidad y garantías

En resumen: Quicksort es el velocista con mal genio; Mergesort es el maratoniano fiable. Quicksort suele ganar en velocidad práctica con arrays grandes en memoria; Mergesort gana cuando no puedes permitirte un mal día.

🕵️ ¿Qué hace Java por dentro?

En Java, `Arrays.sort()` no es un algoritmo, son dos según el tipo:

- Para **tipos primitivos** (`int[]`, `double[]`…): usa **Dual-Pivot Quicksort**, una versión mejorada con dos pivotes que reduce el peor caso y acelera el promedio.
- Para **objetos** (`String[]`, `ArrayList`…): usa **TimSort**, una mezcla de Mergesort e Insertion Sort. Estable, y rapidísima con datos casi ordenados.

Por eso "usa `Arrays.sort()`" es casi siempre la respuesta correcta: Java ya eligió bien por ti.

📊 Rendimiento en la práctica

Algoritmo	n=10	n=100	n=1.000	n=10.000	n=100.000	n=1.000.000
Quicksort	~0 ms	~0 ms	~0 ms	~1 ms	~15 ms	~120 ms
Mergesort	~0 ms	~0 ms	~0 ms	~2 ms	~20 ms	~150 ms
Burbuja (para llorar)	~0 ms	~1 ms	~100 ms	~10.000 ms	no esperes	no esperes

Los tiempos son orientativos y dependen del hardware, pero la *tendencia* es la que importa: con 10.000 elementos, Quicksort y Mergesort siguen en "misilísegundos" mientras burbuja ya se ha ido a tomar un café. Con 100.000, burbuja ni se presenta.

🧠 La lógica que hay detrás

Recuerda la regla de oro de la complejidad: **O(n log n) casi siempre gana a O(n²)**, y la diferencia se nota brutalmente cuando `n` crece. Para un millón de elementos:

- O(n log n): ~20 millones de operaciones → un abrir y cerrar de ojos.
- O(n²): ~1 billón de operaciones → el ordenador pide la jubilación.

Así que la decisión no es "¿qué algoritmo?", sino "¿qué necesito garantizar?". Y eso, amigo mío, es pensamiento de ingeniero: elige la herramienta por el **contexto**, no por la costumbre.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué algoritmo elegirías para ordenar 10.000 ints al azar en un servidor con RAM de sobra?
2. ¿Y si necesitas estabilidad para una ordenación por dos criterios?
3. ¿Qué usa `Arrays.sort()` para primitivos? ¿Y para objetos?
4. ¿Cuánto le cuesta a burbuja ordenar un millón de elementos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. No hay un “mejor” algoritmo universal: la elección depende del **tamaño**, el **orden inicial**, la **memoria** y la **estabilidad** que necesites.
2. **Quicksort** gana en velocidad práctica y memoria; **Mergesort** gana en garantías y estabilidad.
3. En Java, `Arrays.sort()` ya elige bien por ti: **Dual-Pivot Quicksort** para primitivos y **TimSort** para objetos.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Dual-Pivot Quicksort | Quicksort con dos pivotes (Java, primitivos) |
> | TimSort | Mezcla de Mergesort e Insertion Sort (Java, objetos) |
> | Estabilidad | Conservar el orden de los elementos iguales |
> | Orden inicial | Cómo llegan los datos: al azar, ordenados, casi ordenados |
> | Mergesort externo | Ordenación de datos que no caben en RAM (bases de datos) |

📚 Volver al índice de la unidad · **Anterior:** 05 · Mergesort · **Siguiente:** 07 · Be the Code: quicksort desde cero

07 — Be the Code: quicksort desde cero

Cierra la página, abre el editor y escribe Quicksort y Mergesort tú solo (o sufre con honor) 🕶️

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 07 · Be the Code: quicksort desde cero

---

📬 La idea en una frase

> **Ya has visto el código: ahora toca borrarlo de la memoria y escribirlo tú solo, paso a paso, porque un algoritmo solo se entiende de verdad cuando te duele implementarlo.**

> 🕶️ **Don Tip:** Divide y vencerás: elige un pivote, parte el array en menores y mayores, y repite recursivamente. Si dominas ese patrón, Quicksort es tuyo.

🚦 Ejercicio guiado 1: Quicksort

Cierra esta página. Abre un editor de texto en blanco. No mires ni una línea de lo leído hasta ahora.

La firma del método debe ser:

```
static void quicksort(int[] arr, int inicio, int fin)
```

Paso 1: el caso base

¿Cuándo un array (o una zona del array) ya está ordenado? Cuando tiene **0 o 1 elementos**. En términos de índices:

```
if (inicio >= fin) return;
```

Paso 2: elegir el pivote

Elige un elemento cualquiera. El del medio es una opción decente y fácil de escribir:

```
int pivote = arr[(inicio + fin) / 2];
```

Paso 3: particionar con dos punteros

Dos índices parten de los extremos y avanzan hacia el centro:

- `i` avanza desde `inicio` mientras `arr[i] < pivote` (busca un elemento que *debería* estar a la derecha).
- `j` retrocede desde `fin` mientras `arr[j] > pivote` (busca un elemento que *debería* estar a la izquierda).
- Si `i <= j`, intercambias y sigues.

Paso 4: llamar recursivamente a cada zona

Cuando `i` y `j` se cruzan, la zona izquierda es `[inicio..j]` y la derecha `[i..fin]`:

```
quicksort(arr, inicio, j);
quicksort(arr, i, fin);
```

La solución completa (inténtalo antes de mirar)

> 🔄 **Solución de Quicksort**

🚦 Ejercicio guiado 2: Mergesort

Mismo juego, misma regla: sin mirar. La firma:

```
static void mergesort(int[] arr, int inicio, int fin)
```

Paso 1: el caso base

Igual que Quicksort: zona con 0 o 1 elementos, ya ordenada.

Paso 2: partir por la mitad

```
int mid = (inicio + fin) / 2;
```

Paso 3: ordenar cada mitad y fusionar

```
mergesort(arr, inicio, mid);
mergesort(arr, mid + 1, fin);
fusionar(arr, inicio, mid, fin);   // este método lo escribes en el paso 4
```

Paso 4: la fusión, el corazón de Mergesort

Dos mitades ordenadas se combinan con un array temporal:

```
static void fusionar(int[] arr, int inicio, int mid, int fin) {
    int[] tmp = new int[fin - inicio + 1];
    int i = inicio, j = mid + 1, k = 0;

    while (i <= mid && j <= fin)
        tmp[k++] = (arr[i] <= arr[j]) ? arr[i++] : arr[j++];
    while (i <= mid) tmp[k++] = arr[i++];
    while (j <= fin) tmp[k++] = arr[j++];

    System.arraycopy(tmp, 0, arr, inicio, tmp.length);
}
```

La solución completa (inténtalo antes de mirar)

> 🔄 **Solución de Mergesort**

⭐ Niveles de logro

- ⭐ **Lo tienes, pero has tenido que mirar el código una vez.** Aprobado raspado.
- ⭐ ⭐ **Te ha salido a la primera y funciona.** Eres una máquina.
- ⭐ ⭐ ⭐ **Te ha salido a la primera, sin errores de off-by-one, y además has elegido mediana de tres como pivote.** No necesitas este curso. Vete a dar una charla TED.

> [!TIP] Pista mental gratuita: Quicksort necesita **tres cosas** — elegir un pivote, partir el array en dos zonas (menores y mayores), y llamar recursivamente a cada lado. Si memorizas ese patrón, Quicksort es tuyo para siempre. Mergesort, por su parte, solo necesita **dos** — partir por la mitad y fusionar.

🧩 El lío

El departamento de calidad ha recibido este Quicksort. Algo huele mal. Encuentra los **3 errores**:

```
public class QuicksortLioso {

    static void quicksort(int[] arr, int inicio, int fin) {
        if (inicio > fin) return;

        int pivote = arr[fin];
        int i = inicio;

        for (int j = inicio; j < fin; j++) {
            if (arr[j] < pivote) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
            }
        }

        quicksort(arr, inicio, i - 1);
        quicksort(arr, i + 1, fin);
    }
}
```

> 👓 **Don Tip:** prueba a ejecutarlo con `{3, 1, 4}` y mira qué pasa con el caso base, el lugar del pivote y la variable `i` después del bucle.

> › 🔄 **Solución del lío**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuándo devuelve la recursión de Quicksort?
2. ¿Qué ocurre cuando `i` y `j` se cruzan en la partición?
3. ¿Cuál es el caso base de Mergesort?
4. ¿Qué hacen los tres `while` de la fusión?

> 🔁 Respuestas

✅ Resumen en 3 frases

1. **Quicksort** se reduce a elegir pivote, particionar con dos punteros y llamar recursivamente a cada zona.
2. **Mergesort** se reduce a partir por la mitad, ordenar cada mitad y fusionar con un array temporal.
3. El off-by-one en los casos base y los elementos iguales son los errores clásicos: **prueba siempre con arrays pequeños a mano**.

> 🦎 **Vocabulario rápido**
>
Término	Idea general
Off-by-one	Error de “uno de más o uno de menos” en los índices
Partición	Repartir menores y mayores alrededor del pivote
Punteros `i` y `j`	Índices que avanzan desde los extremos hasta cruzarse
Fusión	Combinar dos mitades ordenadas en una
Array temporal	El `tmp` de la fusión, que cuesta O(n) de memoria

📚 Volver al índice de la unidad · **Anterior:** 06 · Comparación: cuándo usar cada ordenación · **Siguiente:** 08 · Otras técnicas algorítmicas

08 — Otras técnicas algorítmicas

Dos fichas más para tu cinturón: los algoritmos voraces (greedy) y el backtracking 🧠

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 08 · Otras técnicas algorítmicas

---

📬 La idea en una frase

> **Recursividad y divide y vencerás no son las únicas técnicas: los algoritmos voraces toman la decisión que mejor pinta en cada paso, y el backtracking prueba caminos y retrocede cuando llega a un callejón sin salida.**

Esta unidad ya te ha dado las herramientas estrella. Estos dos puntos extra son "para saber que existen" y reconocerlos cuando los veas, porque aparecen en montones de problemas reales (y en entrevistas). Te aviso: aquí no vas a implementar nada gigante, vas a *entender la idea* con ejemplos sencillos.

🐓 Algoritmos voraces (greedy)

La idea: en cada paso, toma la decisión que parece mejor *en ese momento*, sin mirar hacia atrás ni hacia delante. Como quien compra caramelos y siempre coge el más grande de los que quedan.

La regla de oro: un greedy solo funciona si la decisión local óptima conduce a la solución global óptima. Eso no siempre pasa. Cuando funciona, es de los algoritmos más rápidos y simples que existen.

Ejemplo clásico: el cambio de monedas

Dado un precio y el dinero entregado, devolver el cambio con el **menor número de monedas**:

```java
public class CambioMonedas {

    public static void main(String[] args) {
        int[] monedas = {200, 100, 50, 20, 10, 5, 2, 1};
        int cambio = 173;   // 1,73 €

        for (int moneda : monedas) {
            int cuantas = cambio / moneda;
            cambio %= moneda;
            if (cuantas > 0) {
                System.out.println(cuantas + " moneda(s) de " + moneda + " céntimos");
            }
        }
    }
}
```

Salida:

```
1 moneda(s) de 100 céntimos
1 moneda(s) de 50 céntimos
1 moneda(s) de 20 céntimos
1 moneda(s) de 2 céntimos
1 moneda(s) de 1 céntimos
```

> [!NOTE] El algoritmo voraz de la moneda funciona con el sistema de monedas europeo (donde cada moneda es múltiplo de la anterior). Con sistemas raros (p. ej. monedas de 1, 3 y 4), el voraz **falla**: con cambio 6, el voraz daría 4+1+1 (3 monedas) cuando la solución óptima es 3+3 (2 monedas). La lección: *voraz = rápido y simple, pero no siempre óptimo*.

Otros ejemplos famosos de greedy

- **El problema de la mochila (versión fraccionaria)**: caben solo objetos “a trozos”, y llenas la mochila empezando por el de mejor relación valor/peso.
- **El árbol de expansión mínimo (Prim, Kruskal)**: conectas todos los nodos gastando el mínimo cable.
- **El problema del cambio**, como acabas de ver.
- **Planificación de tareas**: atender primero la tarea que termina antes.

🔁 Backtracking (vuelta atrás)

La idea: pruebas una opción; si llegas a un callejón sin salida, **deshaces el último paso** (de ahí el nombre) y pruebas la siguiente opción. Es una búsqueda exhaustiva e inteligente a la vez, porque *no insistes* en caminos que ya sabes que no llevan a nada.

El patrón clásico es recursivo:

```
public static void resolver(opciones, pasosDados) {
    if (solucionCompleta) { ... }        // caso base: lo lograste
    if (callejonSinSalida) { return; }   // poda: no insistir

    for (opcion : opcionesPosibles) {
        tomar(opcion);
        resolver(opciones, pasosDados + 1);   // probar el siguiente paso
        deshacer(opcion);                     // ¡la vuelta atrás!
    }
}
```

Ese `deshacer(opcion)` es el corazón del backtracking: si el camino no funciona, recoges lo que pusiste y pruebas otro.

Ejemplo sencillo: el laberinto

En un laberinto representado como una matriz de caracteres (`'.'` = pasillo, `'#'` = pared), intenta avanzar hacia la salida. Cuando una casilla no lleva a ningún sitio, deshaces el movimiento:

```
public class Laberinto {

    static char[][] lab = {
        {'S', '.', '#', '#'},
        {'#', '.', '.', '#'},
        {'#', '#', '.', 'E'}
    };

    static boolean salir(int fila, int col) {
         if (fila < 0 || col < 0 || fila >= lab.length || col >= lab[0].length) return
false;
        if (lab[fila][col] == '#') return false;                // pared: callejón sin
salida
        if (lab[fila][col] == 'E') return true;                 // caso base: ¡salida!

         lab[fila][col] = '*';                                     // marco la casilla
como visitada

        boolean encontrado = salir(fila + 1, col)
                           || salir(fila - 1, col)
                           || salir(fila, col + 1)
                           || salir(fila, col - 1);

        lab[fila][col] = '.';                                    // la vuelta atrás: la
dejo libre

        return encontrado;
    }

    public static void main(String[] args) {
        System.out.println(salir(0, 0) ? "¡Salida encontrada!" : "No hay salida.");
    }
}
```

Fíjate en las dos jugadas: marcar la casilla como visitada **evita dar vueltas en círculo** y desmarcarla al fallar **permite probar otros caminos**. Ese es el baile del backtracking.

Otros ejemplos famosos

- **Las 8 reinas**: colocar 8 reinas en un tablero sin que se ataquen.
- **El Sudoku**: probar números y retroceder cuando la casilla se queda sin candidatos.

- **La suma de subconjuntos**: encontrar qué números de una lista suman un valor exacto.
- **Generar todas las permutaciones**: todas las ordenaciones posibles de un conjunto.

> [!TIP] Si alguna vez oyes “exhaustiva pero con poda”, eso es backtracking: no te rindes hasta probarlo todo, pero **tiras a la basura de inmediato** los caminos que ya sabes que son callejones. Es la diferencia entre buscar una aguja en un pajar y buscar una aguja sabiendo en qué mitad del pajar no está.

🧠 Y con esto, el panorama

Ya tienes el mapa de las técnicas más importantes:

Técnica	Idea	Típico para
Recursividad	Resolver una versión más pequeña de sí mismo	Estructuras jerárquicas
Divide y vencerás	Dividir, conquistar, combinar	Ordenación, búsqueda
Voraz (greedy)	Decisión óptima local en cada paso	Cambio de monedas, planificación
Backtracking	Probar, retroceder, probar de nuevo	Juegos, Sudoku, laberintos
Programación dinámica	Memoización + subproblemas superpuestos	Fibonacci optimizado (¡ya lo viste!)

La programación dinámica es la “evolución” de la memoización del punto 2: la verás con más profundidad en unidades avanzadas, pero ya llevas el germen.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué significa que un algoritmo sea “voraz”?
2. ¿El greedy del cambio de monedas es siempre óptimo?
3. ¿Cuál es el paso más característico del backtracking?
4. ¿Por qué se marca una casilla como visitada en el laberinto?

> 🔁 Respuestas

✅ Resumen en 3 frases

1. Los algoritmos **voraces** eligen en cada paso la mejor opción local; son rapidísimos pero solo óptimos en ciertos problemas.
2. El **backtracking** prueba caminos y, al llegar a un callejón sin salida, **retrocede** deshaciendo el último paso.
3. Junto a la recursividad y divide y vencerás, estas técnicas completan tu cinturón algorítmico básico de DAM.

> 🦎 **Vocabulario rápido**

Término	Idea general
Greedy	Tomar la decisión óptima local en cada paso
Óptimo local vs global	Lo mejor ahora vs lo mejor en conjunto
Backtracking	Probar, y al fallar, deshacer y probar otra opción
Poda	Descartar de inmediato los caminos sin salida
Programación dinámica	Memoización + subproblemas que se repiten

📚 Volver al índice de la unidad · **Anterior:** 07 · Be the Code: quicksort desde cero · **Siguiente:** 09 · Modularidad

09 · Modularidad

Un programa gigante no se domina: se parte. Paquetes, cohesión y acoplamiento para que tu código no se convierta en espagueti ⚡🧩

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 09 · Modularidad

---

📬 La idea en una frase

> **Modularidad es partir un programa en piezas con nombre, frontera y responsabilidad propia, para que cada pieza se entienda (y se arregle) por separado.**

En los puntos 3 a 8 dividiste *problemas* hasta hacerlos triviales. Ahora aplicas la misma estrategia al *código*: un programa de mil líneas en un solo archivo nadie lo entiende; mil líneas repartidas en módulos con sentido, sí. Divide y vencerás, pero esta vez en el editor.

🧩 ¿Por qué importa tanto?

- **Mantenibilidad:** cuando algo se rompe, buscas en el módulo responsable, no en un archivo con 1.500 líneas.
- **Trabajo en equipo:** dos personas pueden tocar módulos distintos sin pisarse. Sin fronteras, cada merge es una lotería.
- **Reutilización:** una pieza bien aislada (una utilidad de fechas, un algoritmo de ordenación) sirve en otros proyectos.
- **Testing:** se prueba una pieza pequeña en aislado, no el monstruo entero.
- **Comprensión:** puedes entender *un* módulo sin entender *todos*. Nadie entiende todo el sistema de golpe, y no hace falta.

📦 La pieza oficial en Java: el paquete

En Java, la unidad modular básica es el **paquete** (`package`): una carpeta con nombre que agrupa clases con la misma misión.

```
package curso.algoritmos;                    // este archivo vive en la carpeta
curso/algoritmos/

public class Ordenacion {
    public static int[] burbuja(int[] datos) { /* ... */ }
}
```

Para usarlo desde otra clase, se importa:

```
import curso.algoritmos.Ordenacion;    // "necesito la clase Ordenacion del paquete
curso.algoritmos"

int[] ordenados = Ordenacion.burbuja(datos);
```

Convenciones que en el examen y en la vida real se cumplen:

1. El **directorio = paquete**: `curso.algoritmos` vive en `curso/algoritmos/`.
2. Nombres en **minúsculas y sin espacios** (`curso.algoritmos`, no `Curso.Algoritmos`).
3. Paquetes que expresan **misión**: `ordenes`, `pagos`, `utilidades`… no `cosas2`.
4. El punto (`.`) es el separador jerárquico: primero la empresa o el proyecto, luego la capa.

> 💡 **Analogía:** los paquetes son las estanterías de un taller. Las herramientas sueltas en la mesa siempre se pierden; las que viven en su cajón, aparecen cuando las buscas.

🎯 Cohesión alta, acoplamiento bajo

Dos palabras que resuelven el 90% de las dudas de diseño:

- **Cohesión (alta):** todo lo que hay dentro de un módulo va junto *porque tiene sentido*. Si tu módulo `facturas` contiene también recetas de cocina, la cohesión ha muerto.
- **Acoplamiento (bajo):** los módulos deben depender lo mínimo de los demás. Si para cambiar un nombre de archivo tienes que tocar cinco módulos, están demasiado pegados.

La regla de oro, en una frase: **un módulo, una responsabilidad; y que hable con los demás lo justo.**

✂️ Cómo decides los cortes

No hay una única respuesta, pero estas señales funcionan siempre:

1. **Nombre que se explica solo.** Si no sabes nombrar el módulo, todavía no sabes qué hace.
2. **Cambio repetido en el mismo sitio.** Dos funcionalidades que cambian juntas van al mismo módulo.
3. **Métodos cortos con una misión.** Una función que hace cinco cosas ya está pidiendo a gritos dividirse.
4. **Lo que se reutiliza, sube.** Si lo copias a otro sitio, es que quería ser su propio módulo.

> ⚠️ **[WARNING]** El extremo opuesto también enferma: **módulos de dos líneas** o una carpeta `utilidades` con 40 clases de todo tipo. Modular en exceso es ruido; el objetivo es *claridad*, no árboles infinitos de carpetas.

🔮 Lo que viene después

La modularidad es el puente perfecto hacia la POO: en la **U09** aprenderás que las **clases** son las piezas pequeñas, y en la **U10** que la **visibilidad** (`public`, `private`) decide qué cara muestra cada módulo al resto del mundo. Hoy pones las estanterías; mañana fabricas las piezas que van dentro.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué indica el `package curso.algoritmos;` de la primera línea de un archivo?
2. Si cambias un módulo y tienes que editar cinco más, ¿qué problema de diseño tienes: cohesión alta o acoplamiento excesivo?
3. ¿Por qué el directorio y el paquete deben coincidir?
4. Nombra dos ventajas de modularizar un programa para un equipo de trabajo.

> › 🔁 **Respuestas**

✅ Resumen en 3 frases

1. La **modularidad** parte el programa en piezas con nombre y responsabilidad propia, aplicando al código la misma «divide y vencerás» que usaste con los algoritmos.
2. En Java la pieza básica es el **paquete** (`package` / `import`), cuyo nombre coincide con la carpeta donde vive la clase.
3. Buen diseño = **cohesión alta** (todo junto va por sentido) y **acoplamiento bajo** (los módulos se hablan lo justo).

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Módulo | Pieza del programa con una misión |
> | Paquete | Carpeta de clases en Java (`package`) |
> | Cohesión | Cuánto tiene sentido que el código viva junto |
> | Acoplamiento | Cuánto depende un módulo de los otros |
> | Responsabilidad única | Un módulo, una misión |
> | `import` | Declarar qué pieza ajena necesitas usar |

📚 Volver al índice de la unidad · **Anterior:** 08 · Otras técnicas algorítmicas · **Siguiente:** 10 · Repaso interactivo

10 · Repaso interactivo

El cierre de la unidad con chicha: adivina salidas, arregla bugs y date el lujo de sufrir con honor 😈

> 🗺️ **Estás en:** ⚡ **U08 · Algorítmica II: técnicas avanzadas y modularidad** → 10 · Repaso interactivo

---

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar.** 😈

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```java
public class MisterioRecursivo {
    static int contar(int n) {
        if (n <= 0) return 0;
        return 1 + contar(n / 2);
    }

    public static void main(String[] args) {
        System.out.println(contar(16));
        System.out.println(contar(100));
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **4 y 6** → Recuerda: cada llamada divide `n` entre 2 hasta llegar a 0. ❌
2. **5 y 7** → ✅ ¡Correcto! `contar(16)`: 16→8→4→2→1→0, cinco saltos. `contar(100)`: 100→50→25→12→6→3→1→0, siete saltos. Es un contador de divisiones entre 2 (o sea, el "$\log_2$" redondeado).
3. **4 y 7** → Cuenta los saltos de la primera: 16→8→4→2→1→0 son cinco pasos, no cuatro. ❌

> › 🔄 **Solución**

🎯 Verdadero o falso

1. “Toda función recursiva necesita un caso base.”
2. “El Fibonacci ingenua es O(n log n).”
3. “Quicksort es estable.”
4. “Mergesort siempre es O(n log n), pase lo que pase.”
5. “La memoización convierte el Fibonacci de $O(2^n)$ a O(n).”
6. “En Java, `Arrays.sort()` usa TimSort para tipos primitivos.”

> 🔁 **Respuestas**

🔥 Fireside Chat: Quicksort y Mergesort discuten

> *Dos veteranos de la ordenación discuten junto a la taza humeante.*

Quicksort: — Escucha, colega. Yo soy el más usado del planeta. En promedio, yo ordeno más rápido que tú con la misma memoria. A ti te hace falta un array entero de repuesto.

Mergesort: — ¿En promedio? ¿Y qué pasa con tus malos días? Cuando te dan un array ya ordenado y tu pivote es el primer elemento, te vas a $O(n^2)$ y lloras. Yo jamás. Siempre O(n log n), pase lo que pase.

Quicksort: — ¡Eso era antes! Ahora uso mediana de tres, o pivote aleatorio, o dos pivotes como el Dual-Pivot de Java. No me pillas en la mala.

Mergesort: — Vale, pero dime, ¿eres estable? Porque cuando ordeno alumnos por nota y luego por nombre, yo conservo el orden de los empates. Tú los mezclas como si barajaras cartas.

Quicksort: — Bueno… la estabilidad es para débiles. Y tu memoria O(n) no entra en un sistema embebido.

Mergesort: — Y tu $O(n^2)$ no entra en una base de datos. ¿Sabes lo que ordena discos enteros? Yo. Mergesort externo. Todo el día. Todos los días.

Quicksort: — *suspira* Vale. Cada uno es mejor donde su jefe le pone. Pero todos sabemos quién tiene más fama.

> La lección: **ninguno gana siempre**. Quicksort brilla con datos al azar y poca memoria; Mergesort con estabilidad y garantías. El profesional elige según el contexto, no según la marca.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Me llamo a mí misma con un problema más pequeño, hasta que llego a un caso que ya se resuelve solo.**
2. **Soy la variable que divide el array en menores y mayores, y mi elección decide si el algoritmo vuela o se arrastra.**
3. **Soy el paso en el que dos listas ordenadas se combinan en una sola, comparando el primero de cada una.**
4. **Soy la técnica que guarda los resultados ya calculados para no repetir trabajo.**

> 🔁 **Respuestas**

🧩 Completa el código

Este factorial recursivo tiene **3 huecos**. Rellénalos:

```
public class FactorialHuecos {

    static int fact(int n) {
        if (_____) return 1;          // hueco 1: caso base
        return _____;                 // hueco 2: caso recursivo
    }

    public static void main(String[] args) {
        System.out.println("fact(5) = " + _____);   // hueco 3: la llamada
    }
}
```

> 🔁 **Solución**

👹 CONRAD VS EL MUNDO: “Tu recursión explotó”

> *CONRAD, nuestro compilador cascarrabias, opina sobre el clásico del novato.*

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: *CONRAD, mi programa da StackOverflowError*. Y yo: vale, ¿has mirado tu caso base? *¿Caso qué?* ¡AY, MADRE MÍA! ¡Hemos estado tres unidades para que me digas "¿caso qué"!

Y lo peor viene después. Me enseña el código y veo: `return fibo(n - 1) + fibo(n - 2);` para `n = 50`. ¿PERO TÚ SABES LO QUE ES ESO? Eso no es un programa, es una declaración de intenciones suicidas. `fibo(50)` son miles de millones de llamadas. Mi stack tiembla solo de verlo.

Y luego, el súmmum: coge el array ya ordenado, elige el primer elemento como pivote, y me pregunta por qué Quicksort va tan lento. ¿Que por qué? ¡Porque has convertido O(n log n) en $O(n^2)$, campeón! ¡Elige el del medio, o mediana de tres, o tira un dado!

La lección: antes de llorar sobre el teclado, revisa los tres sospechosos habituales: **caso base**, **complejidad real** (¿estás duplicando trabajo?) y **elección del pivote**. El 90% de los dramas de esta unidad se arreglan solo con mirar esos tres.

⚡ Laboratorio de tortura: el programa que no ordena

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: este programa debería ordenar, pero tiene **3 errores** que lo rompen. Encuéntralos y haz que la salida sea `[1, 2, 3, 4, 5, 6, 7]`.

```java
public class TorturaOrdenacion {

    static void quicksort(int[] arr, int inicio, int fin) {
        if (inicio > fin) return;

        int pivote = arr[inicio];
        int i = inicio + 1;

        for (int j = inicio + 1; j <= fin; j++) {
            if (arr[j] < pivote) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
            }
        }

        quicksort(arr, inicio, i - 1);
        quicksort(arr, i, fin);
    }

    public static void main(String[] args) {
        int[] datos = {7, 3, 5, 1, 6, 2, 4};
        quicksort(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Pistas para cuando te frustres (no antes):

1. ¿El caso base usa > o >=? ¿Qué pasa con una zona de un solo elemento? *pista: `inicio == fin` tiene un solo elemento, ya ordenado.*

   > **¿Y si sigo atascado?**

2. ¿Dónde debería ir el pivote al terminar la partición? *pista: él también es un elemento del array.*

   > **¿Y si sigo atascado?**

3. ¿Y las llamadas recursivas de la derecha, qué zona cubren? *pista: con el pivote en `i - 1`, la zona derecha empieza en `i`.*

   > **Solución final**

```java
public class TorturaOrdenacion {

    static void quicksort(int[] arr, int inicio, int fin) {
        if (inicio >= fin) return;

        int pivote = arr[inicio];
        int i = inicio + 1;

        for (int j = inicio + 1; j <= fin; j++) {
            if (arr[j] < pivote) {
                int tmp = arr[i];
                arr[i] = arr[j];
                arr[j] = tmp;
                i++;
            }
        }

        // el pivote a su sitio: justo antes de la zona de mayores
        int tmp = arr[inicio];
        arr[inicio] = arr[i - 1];
        arr[i - 1] = tmp;

        quicksort(arr, inicio, i - 2);
        quicksort(arr, i, fin);
    }

    public static void main(String[] args) {
        int[] datos = {7, 3, 5, 1, 6, 2, 4};
        quicksort(datos, 0, datos.length - 1);
        System.out.println(java.util.Arrays.toString(datos));
    }
}
```

Salida correcta: `[1, 2, 3, 4, 5, 6, 7]`.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 El Mago del Stack	Explicar caso base, caso recursivo y pila de llamadas sin mirar

Logro	Cómo conseguirlo
🏅 **Recursionista**	Implementar factorial y palíndromo recursivos sin ayuda
🏅 **Domador de Fibonacci**	Bajar `fibo(50)` de O(2ⁿ) a O(n) con memoización
🏅 **El Rápido**	Implementar Quicksort desde cero sin mirar el código
🏅 **El Fiable**	Implementar Mergesort desde cero, con su fusión, sin mirar
🏅 **Ingeniero de Datos**	Elegir con criterio entre Quicksort, Mergesort y `Arrays.sort()` según el contexto

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```
public class Misterio2 {
    static int misterio(int n) {
        if (n == 0) return 0;
        return n % 10 + misterio(n / 10);
    }

    public static void main(String[] args) {
        System.out.println(misterio(1234));
    }
}
```

2. **El iterativo que no se rinde:** escribe una versión **iterativa** (con bucle) de `fact(n)`. ¿Cuál prefieres y por qué?
3. **El detective del IDE:** tu Quicksort "funciona" con arrays pequeños pero desordena los grandes. ¿Qué técnica de depuración usas y en qué paso te fijas primero? *pista: los índices de la partición.*
4. **Verdadero o falso:** "Mergesort es mejor que Quicksort siempre porque no tiene caso malo."

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. La condición que detiene la recursión (5,4)
3. Algoritmo estable del punto 5 (9)
5. Técnica que guarda resultados ya calculados (12)
6. La variable que divide el array en Quicksort (6)

Vertical:
2. Lo que lanza la JVM cuando el stack se llena (8,7)
4. Ordenar sin arrays auxiliares (7)
7. Repartir elementos alrededor del pivote (11)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **“Explícame, como si yo fuera tu abuela, qué es la recursividad y por qué a veces da StackOverflowError.”**
2. **“¿Cuándo usarías Quicksort y cuándo Mergesort? Justifica.”**
3. **“El factorial recursivo con `n = 100000` te revienta la pila. ¿Qué haces?”**
4. **“¿Qué es la memoización y qué problema resuelve?”**
5. **“Si `Arrays.sort()` ya ordena, ¿para qué aprender a implementar Quicksort?”**

🤷 No hay preguntas tontas

> ❓ **¿Puedo escribir recursividad con bucles y olvidarme de todo esto?**

Puedes, pero te perderás la elegancia en los problemas jerárquicos (árboles, juegos, laberintos) y la comprensión de cómo funcionan por dentro Quicksort, Mergesort y el backtracking. Además, casi cualquier entrevista técnica te preguntará por esto. No es obligatorio, pero duele no saberlo.

> ❓ **¿Por qué `fibo(92)` da el máximo que cabe en `long` y no puedo ir más allá?**

Porque `long` tiene un límite (9.223.372.036.854.775.807) y `fibo(93)` ya lo supera. Si necesitas números más grandes, tendrás que usar `BigInteger`, que puede crecer tanto como la memoria lo permita.

> ❓ **¿Mergesort es el que usa Java por defecto? Yo juraría que era Quicksort.**

Los dos, en función del tipo: **Dual-Pivot Quicksort** para primitivos y **TimSort** (una mezcla con Mergesort) para objetos. Java no es dogmático: usa el mejor para cada caso. Ese es el mismo criterio que te pedimos a ti en el punto 6.

🎬 Poscréditos

El programador novato termina su Quicksort, lo ejecuta y… funciona a la primera. Se frota los ojos. Su compañero Mergesort, ese maratoniano fiable, le sonríe desde el otro monitor.

Novato: — No me lo creo. ¡He escrito un Quicksort sin mirar!

CONRAD: — *aparece con cara de pocos amigos* Bonito. Ahora dime: ¿cuándo usarías Mergesort en su lugar?

Novato: — Cuando necesite estabilidad… o cuando el array llegue casi ordenado y me dé miedo el pivote.

CONRAD: — *asiente, casi emocionado* Mira por dónde. Ya no solo escribes algoritmos: piensas en ellos. Eso se llama ser ingeniero. *pausa* Pero no te relajes, que la próxima unidad te espera con objetos, clases y algún que otro constructor que te hará llorar.

Novato: — ¿Llorar? Yo ya domino la recursividad.

CONRAD: — *ríe malvado* Eso decía el otro antes de encontrarse con `this`.

El novato guarda su proyecto, y siente que algo ha cambiado en su forma de pensar. La recursividad ya no le da miedo: la entiende. Y la entiende porque, para entender la recursividad, primero entendió la recursividad.

PRÓXIMAMENTE EN U09: POO: clases y objetos. El momento en el que tus programas dejan de ser funciones sueltas y se convierten en **ciudadanos con atributos y métodos**. 🏗️

📚 Volver al índice de la unidad · **Anterior:** 09 · Modularidad · **Siguiente:** U09 · POO: clases y objetos

U09 — POO: clases y objetos

El cortapastas y las galletas: cuando tu código pasa de ser una receta a un mundo de objetos que hablan entre sí



Bienvenido a la unidad donde tus programas dejan de ser *recetas de cocina* y se convierten en **mundos de cosas**. Hasta ahora has escrito programas lineales: esto, luego esto, luego esto. Pero el mundo real no funciona así. En el mundo real tienes *cosas*: un perro, un coche, un profesor de programación con gafas de pasta. Cada cosa tiene **atributos** (color, edad, número de ganas de corregir exámenes) y **comportamientos** (ladrar, acelerar, poner faltas de ortografía).

La **Programación Orientada a Objetos (POO)** es exactamente eso: escribir código como funciona el mundo real. Y en esta unidad vas a aprender la base de todo: las **clases** (el molde) y los **objetos** (la galleta). Construirás tus propias clases con atributos y métodos, las harás nacer con **constructores**, aprenderás a desambiguar con `this`, a razonar sobre **referencias y memoria**, y a exprimir la **sobrecarga** de métodos y el paso de parámetros.

Esta unidad es el **punto de inflexión del curso**: lo que aprendas aquí, lo usarás en todas las unidades que quedan. La POO no es un tema más: es el tema que hace que todos los demás tengan sentido.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es la **POO** y por qué existe (spoiler: no es para fastidiar, es para no volverte loco).
- Diferenciar **clase** (el cortapastas) de **objeto** (la galleta), y crear objetos con `new`.
- Definir **atributos** (las propiedades) y **métodos** (los comportamientos) de una clase.
- Usar **constructores** para que los objetos nazcan bien preparados, con o sin parámetros.
- Desambiguar atributos y parámetros con la palabra clave **`this`**, y llamar a un constructor desde otro.

- Razonar sobre **referencias, `null` y memoria**: qué guarda una variable de objeto y qué pinta el heap.
- Escribir **métodos sobrecargados** y entender el **paso por valor** (y por qué las referencias se portan "raro").
- Diseñar y crear una **clase completa** con sus métodos, de principio a fin.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · Qué es la POO	Programación procedural vs orientada a objetos y por qué el mundo real manda	Todos
02 · Clases y objetos: el cortapastas y las galletas	El molde, la galleta, `new`, y tu primera clase	Todos
03 · Atributos y métodos	Las propiedades y los comportamientos de una clase, con sus tipos y parámetros	Todos
04 · Constructores	El método que prepara al objeto al nacer: por defecto y con parámetros	Todos
05 · La palabra clave this	Cuando los parámetros se llaman igual que los atributos (y cómo salir del lío)	Todos
06 · Referencias, null y memoria	Qué guarda una variable de objeto, el `heap`, `null` y el mítico `NullPointerException`	Todos
07 · Sobrecarga y paso por valor	Varios métodos con el mismo nombre y qué se copia cuando llamas	Todos
08 · Be the Code: diseña tu clase	Diseñar y construir una clase completa paso a paso, sin mirar	⭐⭐⭐
09 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA2 + RA4)

RA2: Escribe y prueba programas sencillos, reconociendo y aplicando los fundamentos de la programación orientada a objetos.

RA4: Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.

CE	Criterio	Dónde se cubre
RA2 a)	Se han identificado los fundamentos de la programación orientada a objetos.	✅ Punto 1
RA2 b)	Se han escrito programas simples.	✅ Todos
RA2 c)	Se han instanciado objetos a partir de clases predefinidas.	✅ Puntos 2, 3 y 6
RA2 d)	Se han utilizado métodos y propiedades de los objetos.	✅ Puntos 3 y 8
RA2 f)	Se han utilizado parámetros en la llamada a métodos.	✅ Puntos 7 y 8
RA2 h)	Se han utilizado constructores.	✅ Puntos 4 y 7
RA4 a)	Se ha reconocido la sintaxis, estructura y componentes típicos de una clase.	✅ Puntos 2 y 3

CE	Criterio	Dónde se cubre
RA4 b)	Se han definido clases.	✅ Puntos 2 y 8
RA4 c)	Se han definido propiedades y métodos.	✅ Puntos 3 y 8
RA4 d)	Se han creado constructores.	✅ Puntos 4 y 7
RA4 e)	Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.	✅ Puntos 6 y 8

> 📌 El punto 6 (referencias y memoria) y el 7 (paso por valor) son la base para entender el **RA2 c) y f)** en profundidad. Los modificadores de visibilidad (RA4 f), la herencia (RA4 g) y los métodos estáticos (RA4 h) tienen su propio capítulo en la **U10** (Visibilidad, encapsulación y static) y en la **U11** (Herencia), así que aquí los dejamos descansando.

🚪 ¿Por dónde empiezo?

- ¿Cero conocimientos de POO? → Arranca en el punto 1. Solo necesitas lo que viste hasta la U08.
- ¿Ya sabes qué es una clase y quieres chicha? → Ve directo al punto 4 y al punto 5: ahí empieza el baile serio.
- ¿Te flipan los retos? → Haz el punto 8 sin mirar el código de los puntos anteriores. Diseña tu clase con honor.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Qué es la POO

⏭️ **Al acabar la unidad, continúa en U10 · Visibilidad, encapsulación y static.**

01 — Qué es la POO

Cuando tu código deja de ser una receta y se convierte en un mundo de cosas que hablan entre sí 🎡

> 🗺️ **Estás en:** 🎡 **U09 · POO: clases y objetos** → 01 · Qué es la POO

📬 La idea en una frase

> **La POO es escribir código como funciona el mundo real: en vez de una lista de instrucciones, modelas *cosas* con sus datos (atributos) y sus comportamientos (métodos).**

Hasta ahora tus programas eran como una **receta de cocina**: haz esto, luego esto, luego lo otro. Muy ordenado, pero muy rígido. Si quieres representar a dos estudiantes, dos coches o dos galletas… empiezas a copiar y pegar variables como un loco. Y cuando el programa crece, la receta se convierte en un caos.

La POO llega para poner orden: en lugar de variables sueltas, agrupas los datos *de la misma cosa* junto con lo que esa cosa *sabe hacer*. Un perro no es solo un `String nombre` y un `int edad` sueltos por ahí: es un **objeto** que tiene nombre, edad, y sabe ladrar.

📜 De la receta al mundo real

Imagina que tienes que gestionar los datos de un estudiante sin POO:

```
String nombre1 = "Ana";
int edad1 = 20;
double nota1 = 8.5;

String nombre2 = "Luis";
int edad2 = 19;
double nota2 = 6.0;
```

Con dos estudiantes, ya son seis variables. Con cincuenta, es la peor pesadilla de tu vida. Los datos de cada persona están **desconectados**: no hay nada que diga “nombre1, edad1 y nota1 van juntos porque son Ana”.

Con POO agrupas todo eso en una pieza:

```
public class Estudiante {
    String nombre;
    int edad;
    double nota;
}
```

Y cada estudiante es un **objeto** con sus tres datos dentro, bien atados. Ya no tienes variables sueltas: tienes *personas*.

> 💡 **Tip:** cuando veas código lleno de `nombre1`, `nombre2`, `nombre3`, huele a que ahí dentro quiere nacer una clase. Ese olfato se entrena, y esta unidad es tu gimnasio.

🧩 Los dos ingredientes: atributos y métodos

Toda clase se construye con dos ingredientes:

- **Atributos (o propiedades):** los datos que describen la cosa. El color del coche, la edad del perro, el saldo de la cuenta.
- **Métodos (o comportamientos):** lo que la cosa *sabe hacer*. Acelerar, ladrar, retirar dinero.

```
public class Perro {
    // Atributos
    String nombre;
    int edad;

    // Métodos
    void ladrar() {
        System.out.println(nombre + ": ¡Guau!");
    }
}
```

> 📝 **Nota:** todavía no ejecutamos nada. Esta clase es solo el *diseño*. Que exista `Perro` no significa que exista un perro. Eso llegará en el punto 2 con `new`.

🏛️ Los cuatro pilares (una promo, no el capítulo)

La POO se apoya en cuatro pilares. En esta unidad tocamos el primero a fondo y hacemos la promo de los demás:

1. **Abstracción:** te quedas con lo importante de una cosa y te olvidas del ruido. Un gato en un programa es `nombre`, `edad` y `maullar()`, no sus bigotes.
2. **Encapsulación (U10):** cada objeto protege sus datos y decide quién puede tocarlos. Como una cuenta bancaria: el saldo no está tirado en la calle.
3. **Herencia (U11):** una clase puede "heredar" de otra. Un `Perro` es un `Animal` y, además, ladra.
4. **Polimorfismo (U11):** el mismo mensaje, distintas respuestas. Todos los animales "hacen sonido"; el perro ladra y el gato maúlla.

> ⚠️ **Advertencia:** no te estreses con los pilares 2, 3 y 4. Esta unidad te da los cimientos; los demás los levantamos en la U10 y la U11. Aquí manda la abstracción y el nacimiento de las clases.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando veas una clase, pregúntate siempre dos cosas: *¿qué datos tiene?* (atributos) y *¿qué sabe hacer?* (métodos). Si sabes responder, ya has entendido el 80% de la POO.

Ejercicio: clasifica

Sin ejecutar nada, decide para cada uno de estos si es un **atributo** o un **método** de una clase `Videojuego`:

1. `String titulo;`
2. `void pausar() { ... }`
3. `int nivelMaximo;`
4. `double calcularPuntuacion() { ... }`
5. `boolean estaInstalado;`

> › 🔄 **Solución**

❓ ¡No hay preguntas tontas!

> **Q:** Vale, pero yo ya sé hacer programas con variables y bucles. ¿De verdad necesito todo este rollo?
>
> **A:** Para un programa de 30 líneas, no. Para cualquier cosa que vaya a crecer, sí. Los bucles y variables siguen estando (no los abandonamos, los usamos *dentro* de los métodos), pero la POO te da una forma de organizar el código que escala. Nadie escribe un programa de empresa con una sola lista de instrucciones. Bueno, nadie *sano*.

> **Q:** ¿La POO es solo cosa de Java?
>
> **A:** No. Es una *forma de pensar* que usan casi todos los lenguajes modernos: Python, JavaScript, C++, C#... Cada uno con su acento, pero la idea es la misma: modelar el mundo en objetos. Aprenderla aquí te sirve para todos.

> **Q:** ¿Qué pasa con el `main` que llevo escribiendo desde la UO2? ¿Ahora no sirve?
>
> **A:** El `main` sigue siendo la puerta de entrada: lo que hay dentro de `main` es la "receta" que arranca la película. Lo que cambia es que ahora, dentro de ese `main`, en vez de variables sueltas crearás objetos. El `main` sigue siendo el director; solo cambia el reparto.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué dos ingredientes forman una clase?
2. ¿Qué pilar de la POO consiste en "quedarte con lo importante"?
3. ¿Un `String color;` dentro de una clase es atributo o método?
4. ¿En qué unidades se ven la encapsulación y la herencia?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. La **POO** modela el mundo real en objetos con **atributos** (datos) y **métodos** (comportamientos), en vez de una lista de instrucciones.
2. La **clase** es el diseño de la cosa; aún no hay ninguna "cosa" real hasta que no la instancies.

3. Los cuatro pilares son **abstracción, encapsulación, herencia y polimorfismo**; en esta unidad cimentamos el primero.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | POO | Paradigma que organiza el código en objetos |
> | Atributo | Dato que describe a la cosa: `color`, `edad` |
> | Método | Comportamiento que la cosa sabe hacer: `acelerar()` |
> | Abstracción | Quedarte con lo esencial y olvidar el ruido |
> | Procedural | Estilo antiguo: una receta de instrucciones en orden |

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · Clases y objetos: el cortapastas y las galletas

02 — Clases y objetos: el cortapastas y las galletas

El molde, la galleta, `new` y tu primera clase: ya no escribes recetas, horneas 🍪

> 🗺️ **Estás en:** 🧬 **U09 · POO: clases y objetos** → 02 · Clases y objetos: el cortapastas y las galletas

---

📬 La idea en una frase

> **Una clase es el cortapastas (el molde) y un objeto es la galleta que haces con él: un solo molde, millones de galletas, y cada una con su propio sabor.**

En el punto 1 viste que una clase es un diseño. Ahora toca la pregunta del millón: ¿y qué hago con un diseño? Lo uso para **fabricar cosas**. Y en Java, la fábrica se llama `new`.

🍪 El cortapastas y la galleta

Imagina la cocina de tu abuela. Tiene un cortapastas con forma de estrella. Ese cortapastas es **la clase**: define la forma de todas las galletas, pero él mismo no es una galleta. Puedes usarlo mil veces y cada galleta será distinta: unas con pepitas de chocolate, otras de vainilla, otras que tu prima se come a medio hacer.

En programación:

- **Clase** = el cortapastas. Define *qué atributos* y *qué métodos* tendrán todas sus galletas.
- **Objeto** = la galleta. Una cosa concreta, con sus propios valores, viviendo en la memoria.

Así que de un solo molde salen infinitas galletas, todas con la misma forma pero cada una con su propia vida.

🔨 Tu primera clase

Vamos a escribir el cortapastas `Galleta`:

```java
public class Galleta {
    String sabor;
    boolean tieneChocolate;

    void comer() {
        System.out.println("Ñam, galleta sabor " + sabor);
    }
}
```

Esto solo define el molde. Por ahora, **no existe ninguna galleta en memoria**. Es un archivo en el disco esperando a que alguien lo use.

> 📝 **Nota:** la clase lleva `public class Galleta` y el archivo se llama `Galleta.java`. El nombre de la clase y del archivo deben coincidir. Es la primera regla no escrita de Java… vale, sí escrita.

🏭 `new`: la máquina de galletas

Para fabricar galletas (objetos) a partir del molde usas el operador `new`:

```java
public class Pasteleria {
    public static void main(String[] args) {
        Galleta g1 = new Galleta();          // Primera galleta
        g1.sabor = "Chocolate";
        g1.tieneChocolate = true;

        Galleta g2 = new Galleta();          // Segunda galleta (mismo molde)
        g2.sabor = "Vainilla";
        g2.tieneChocolate = false;

        g1.comer();   // "Ñam, galleta sabor Chocolate"
        g2.comer();   // "Ñam, galleta sabor Vainilla"
    }
}
```

Desmenucemos la línea estrella: `Galleta g1 = new Galleta();`

1. `Galleta g1` — declara una variable que apuntará a una galleta.

2. `new Galleta()` — fabrica la galleta en memoria (y llama a su constructor, que verás en el punto 4).
3. `=` — conecta la variable `g1` con la galleta recién horneada.

Después, con `g1.sabor = "Chocolate"` y `g1.comer()`, le dices a *esa* galleta concreta qué sabor tiene y que se coma. La `g2` no se entera de nada: es otra galleta con su propia vida.

> 💡 **Tip:** el punto (`.`) es la mano que tiendes al objeto. `g1.comer()` significa "oye, galleta g1, ejecuta tu método comer". `g1.sabor` significa "dame el sabor de la galleta g1".

🥊 Clase vs objeto: el ring

Clase: — Yo soy el molde, el plano, la idea platónica. Sin mí no existirías. Defino qué atributos y métodos tienen los objetos. ¡Soy la creadora!

Objeto: — Sí, pero yo soy quien realmente hace cosas. Tú eres solo un archivo `.java` en el disco. Yo ocupo memoria, tengo estado, puedo cambiar mis atributos. Sin mí tu código no sirve para nada.

Clase: — ¿Ah sí? ¿Y cuántos de ti existen? Puedes tener miles de objetos creados a partir de mí. Yo soy única, tú eres una copia. ¡Soy original, eres reproducible!

Objeto: — Exacto. Porque tú eres el plano, pero yo soy el edificio construido. Nadie vive en un plano. Cuando ejecutas el programa, el que trabaja soy yo.

Clase: — Vale, nos necesitamos. Sin clase no hay objeto. Sin objeto, la clase es solo teoría.

Objeto: — Trato hecho.

> 🕶️ **Don Tip:** la clase define el **QUÉ** (atributos) y el **CÓMO** (métodos). El objeto es el **QUIÉN** (la instancia concreta que ejecuta y tiene valores propios).

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuenta cada `new` como una galleta nueva. Cada `new` = un objeto nuevo en memoria, aunque sea del mismo molde.

Ejercicio: ¿cuántas galletas hay?

Sin ejecutar, responde: ¿cuántos objetos de la clase `Galleta` existen en memoria al final de este `main`?

```
public class Pasteleria2 {
    public static void main(String[] args) {
        Galleta g1 = new Galleta();
        Galleta g2 = new Galleta();
        Galleta g3 = g1;
        g1.comer();
    }
}
```

> 🔄 Solución

❓ ¡No hay preguntas tontas!

> **Q:** ¿Por qué hace falta escribir `new`? ¿No podría Java crear el objeto solito?
>
> **A:** No, porque `new` es el "permiso de construcción". Sin `new`, solo declaras una variable (como `Galleta g;`), pero no hay galleta en la bandeja, solo un hueco vacío. Hasta que no hagas `new`, el objeto no existe.

> **Q:** ¿Y por qué `String nombre = "Ana";` no lleva `new`?
>
> **A:** Porque Java es un amor con `String`. Es tan común que te deja crearlo con comillas directamente (un literal). Es un atajo: detrás del telón, Java lo trata casi como si tuviera `new`. Pero eso es un privilegio de `String` (y de los arrays), no una norma general. Tú, con tus clases, tendrás que usar `new` como todo el mundo.

> **Q:** ¿Puedo tener varias clases en un mismo archivo `.java`?
>
> **A:** Puedes, pero solo **una** puede ser `public` y debe coincidir con el nombre del archivo. Las demás van sin `public`. Para empezar, un archivo = una clase pública. Simple y limpio.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué es la clase en la analogía del cortapastas?

2. ¿Qué operador fabrica objetos en memoria?
3. ¿Cuántos objetos crea este código: `Galleta a = new Galleta(); Galleta b = new Galleta();`?
4. ¿Para qué sirve el punto (`.`) entre el objeto y el miembro?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. La **clase** es el molde que define atributos y métodos; el **objeto** es la galleta concreta, con sus propios valores, viviendo en memoria.
2. Se fabrican objetos con **`new`**, y cada `new` crea una instancia independiente aunque use el mismo molde.
3. Con el **punto** (`.`) accedes a los atributos e invocas los métodos de un objeto concreto.

> 🦎 **Vocabulario rápido**

Término	Idea general
Clase	El molde / cortapastas. Define atributos y métodos
Objeto	La galleta. Instancia concreta con valores propios
Instanciar	Crear un objeto a partir de una clase con `new`
Instancia	Sinónimo de objeto creado
Miembro	Cualquier atributo o método de una clase

📚 Volver al índice de la unidad · **Anterior:** 01 · Qué es la POO · **Siguiente:** 03 · Atributos y métodos

03 — Atributos y métodos

Las propiedades y los comportamientos de tu clase, con sus tipos, parámetros y valores de retorno 🏗️

> 🗺️ **Estás en:** 🏗️ **U09 · POO: clases y objetos** → 03 · Atributos y métodos

---

📬 La idea en una frase

> **Los atributos son las propiedades de la cosa (qué es) y los métodos son sus comportamientos (qué hace): la clase los declara y cada objeto los vive por su cuenta.**

Ya sabes que la clase es el molde. Ahora vamos a vestir ese molde por dentro: qué datos guarda y qué sabe hacer. Los atributos y métodos son la carne de la clase, y es aquí donde tus objetos empiezan a tener vida propia.

🏷️ Los atributos: las propiedades de la cosa

Un atributo se declara igual que una variable, pero **dentro de la clase** y **fuera de cualquier método**:

```
public class Coche {
    String marca;       // atributo
    String modelo;      // atributo
    int velocidad;      // atributo
    boolean encendido;  // atributo
}
```

Cada atributo tiene un tipo (que ya dominas de la U03) y describe una propiedad del objeto. Cuando creas un coche con `new Coche()`, ese coche concreto tendrá **su propia copia** de los cuatro atributos. Otro coche, otras copias. Cada galleta lleva sus pepitas.

> 💡 **Tip:** los atributos se declaran con los mismos tipos que las variables: `String`, `int`, `double`, `boolean`… Si dominas la U03, dominas la mitad de este punto.

🎮 Los métodos: los comportamientos de la cosa

Un método es una acción que el objeto sabe hacer. Tiene nombre, parámetros (los datos de entrada) y un **tipo de retorno** (lo que devuelve):

```
public class Coche {
    String marca;
    int velocidad;

    // Método que no devuelve nada: solo hace cosas
    void acelerar(int incremento) {
        velocidad += incremento;
    }

    // Método que devuelve un valor: calcula y devuelve
    int velocidadActual() {
        return velocidad;
    }

    // Método que imprime y no devuelve nada
    void mostrarEstado() {
        System.out.println(marca + " va a " + velocidad + " km/h");
    }
}
```

- **`void`** = no devuelve nada. Solo ejecuta (acelera, imprime, saluda…).
- **Tipo normal** (`int`, `double`, `String`…) = devuelve un valor con `return`.

> ⚠️ **Advertencia:** si el método promete devolver un `int` (`int velocidadActual()`), está **obligado** a terminar con un `return` que dé un `int`. Si promete `void`, se prohíbe el `return` con valor. Java es un notario muy puntilloso con las firmas.

📞 Llamar a los métodos

Dentro del `main` (o de otro método) llamas a los métodos con el punto:

```java
public class Concesionario {
    public static void main(String[] args) {
        Coche miCoche = new Coche();
        miCoche.marca = "Seat";
        miCoche.acelerar(50);          // no devuelve nada
        int v = miCoche.velocidadActual();   // devuelve 50
        System.out.println("Velocidad: " + v);
        miCoche.mostrarEstado();       // "Seat va a 50 km/h"
    }
}
```

📝 **Nota:** dentro de la propia clase (por ejemplo, `acelerar` llamando a `mostrarEstado`), no hace falta el punto: `mostrarEstado();` a secas. El punto solo hace falta cuando llamas a un método de **otro** objeto.

🧬 Cada objeto con su propia copia

Aquí está la magia del punto 2 hecha realidad. Dos coches creados del mismo molde son independientes:

```java
Coche cocheA = new Coche();
Coche cocheB = new Coche();

cocheA.marca = "Seat";
cocheB.marca = "Ford";

cocheA.acelerar(100);   // solo cocheA se entera

System.out.println(cocheA.marca + " → " + cocheA.velocidad);  // Seat → 100
System.out.println(cocheB.marca + " → " + cocheB.velocidad);  // Ford → 0
```

`cocheB` no sabe nada de la aceleración de `cocheA`: cada objeto guarda sus propios valores. Esto es lo que hace que la POO no sea un caos: **los datos de cada objeto van con su objeto**.

💡 **Tip:** piensa en los atributos como cajetines personales. Tú tienes tu taquilla, tu compañero tiene la suya. Si tú llenas la tuya de bocadillos, la de él no se llena sola.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** para trazar un programa con objetos, haz una cajita por cada objeto y apunta sus atributos dentro. Cada vez que se llama a un método, actualiza solo la cajita del objeto que lo llama.

Ejercicio: la caja misteriosa

Sin ejecutar, calcula qué imprime este programa:

```
public class Caja {
    int ancho;
    int alto;
    int profundo;

    int volumen() {
        return ancho * alto * profundo;
    }
}

public class Main {
    public static void main(String[] args) {
        Caja c1 = new Caja();
        Caja c2 = new Caja();

        c1.ancho = 2;  c1.alto = 3;  c1.profundo = 4;
        c2.ancho = 5;  c2.alto = 1;  c2.profundo = 2;

        c1.ancho = 10;
        System.out.println(c1.volumen());
        System.out.println(c2.volumen());
    }
}
```

> › 🔄 **Solución**

❓ ¡No hay preguntas tontas!

> **Q:** ¿Un método puede llamar a otro método de la misma clase?

> **A:** Sí, y es lo más normal del mundo. `acelerar` puede llamar a `mostrarEstado` sin el punto: `mostrarEstado();`. Es como hablarte a ti mismo en voz alta: no necesitas presentarte.

> **Q:** ¿Y si un método necesita varios datos? ¿Puedo pasar varios parámetros?
>
> **A:** Todos los que quieras, separados por comas: `void mover(int x, int y, int velocidad)`. El número y tipo de parámetros forman la **firma** del método, y Java distingue métodos por su firma (esto lo exprimimos en el punto 7 con la sobrecarga).

> **Q:** ¿Los atributos pueden tener valores por defecto?
>
> **A:** Sí, y de hecho Java les da uno automático cuando creas el objeto: números a `0`, `boolean` a `false` y `String` (y cualquier referencia) a `null`. Pero depender de esos valores es mala costumbre: verás en el punto 4 que los **constructores** existen para darles un buen comienzo.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Dónde se declaran los atributos dentro de una clase?
2. ¿Qué significa `void` en la firma de un método?
3. ¿Qué palabra devuelve un valor en un método?
4. Si dos objetos son de la misma clase, ¿comparten sus atributos?

> › 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Los **atributos** son las propiedades de la cosa y se declaran como variables dentro de la clase; cada objeto tiene su propia copia.
2. Los **métodos** son los comportamientos: pueden no devolver nada (`void`) o devolver un valor con `return`, y reciben datos por parámetros.
3. Se invocan con el punto (`objeto.metodo()`), y dentro de la misma clase no hace falta.

> 🦎 **Vocabulario rápido**

Término	Idea general
Atributo	Propiedad del objeto: `marca`, `velocidad`
Método	Comportamiento: `acelerar()`, `volumen()`
Firma	Nombre + tipos de los parámetros
`void`	El método no devuelve nada
`return`	Devuelve un valor al que llamó

📚 Volver al índice de la unidad · **Anterior:** 02 · Clases y objetos: el cortapastas y las galletas · **Siguiente:** 04 · Constructores

04 — Constructores

El método que prepara al objeto en su fiesta de bienvenida: por defecto y con parámetros 🎉

> 🗺️ **Estás en:** 🧬 **U09 · POO: clases y objetos** → 04 · Constructores

---

📬 La idea en una frase

> **El constructor es la fiesta de bienvenida del objeto: un método especial que se ejecuta con `new` para que el objeto nazca bien preparado, con sus atributos inicializados.**

En el punto 3 veías que los atributos nacían con valores por defecto (0, `false`, `null`). Muy aburrido. ¿Y si quieres que tu coche nazca con marca "Seat" y el motor ya encendido? Ahí entran los **constructores**: la fiesta de bienvenida donde le dices al objeto cómo debe ser desde el primer segundo.

🎉 ¿Qué es un constructor?

Un constructor es un método especial con tres reglas de oro:

1. Se llama **igual que la clase**.
2. **No devuelve nada** (ni siquiera `void`).
3. Se ejecuta automáticamente con cada **`new`**.

```java
public class Persona {
    String nombre;
    int edad;

    // Constructor sin parámetros: "vale, te pongo valores por defecto"
    public Persona() {
        nombre = "Desconocido";
        edad = 0;
    }

    // Constructor con parámetros: "te paso los datos, tú inicialízalos"
    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    void presentarse() {
        System.out.println("Hola, soy " + nombre + " y tengo " + edad + " años.");
    }
}
```

> 💡 **Tip:** el `this` de `this.nombre = nombre` se explica en el punto 5. Por ahora solo necesitas saber que significa "el atributo de ESTE objeto". Si te pica la curiosidad, que sepas que ya vas a resolverlo.

🆓 El constructor por defecto (el que regala Java)

Si **no escribes ningún constructor**, Java te regala uno gratis, sin parámetros y vacío. Solo se dedica a dejar los atributos con sus valores por defecto (0, `false`, `null`). Como un horno frío esperando a que metas algo.

```java
public class Galleta {
    String sabor;   // Java la crea con sabor = null
    boolean tieneChocolate;  // false
}
```

`new Galleta()` funciona porque Java puso ese constructor invisible. Pero ojo con la trampa:

> ⚠️ **Advertencia:** si escribes **cualquier** constructor, el constructor vacío **desaparece**. Si pones `public Galleta(String sabor)`, entonces `new Galleta()` (sin argumentos) deja de compilar. Java no te regala nada si ya has decidido montar tu propia fiesta.

🎛️ El constructor con parámetros (el horno con programa)

Con parámetros, el objeto nace a medida. Le pasas los datos y el constructor los coloca:

```
public class Galleta {
    String forma;
    boolean tieneChocolate;
    int temperatura;

    public Galleta(String forma, boolean tieneChocolate, int temperatura) {
        this.forma = forma;
        this.tieneChocolate = tieneChocolate;
        this.temperatura = temperatura;
    }
}
```

```
Galleta g = new Galleta("estrella", true, 180);
```

Una sola línea crea la galleta **ya con** forma de estrella, con chocolate y a 180 grados. Nada de dejar atributos bailando.

> 📝 **Nota:** no es obligatorio inicializar todos los atributos en el constructor, pero es la mejor costumbre. Un atributo que se queda sin tocar vive en la incertidumbre (`null`, 0, `false`) y la incertidumbre en programación es donde se esconden los bugs.

🏭 `new` y el constructor, un tándem

La línea `new Persona("Ana", 25)` hace tres cosas en orden:

1. Reserva memoria para el objeto.
2. Llama al constructor `Persona(String, int)`.
3. Devuelve la referencia (la dirección) para que la guardes en tu variable.

Sin constructor a la vista, `new Persona("Ana", 25)` fallaría si solo existiera el vacío (o el que regala Java). La JVM elige el constructor que encaja con los argumentos que le pasas. ¿Qué pasa si hay varios? Eso es **sobrecarga**, y tiene su capítulo en el punto 7.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** para saber qué constructor se llama, mira los argumentos de la llamada: número y tipo. Después, rastrea qué hace ese constructor.

Ejercicio: ¿qué constructor se llama?

Sin ejecutar, ¿qué imprime este código?

```
public class Pedido {
    String producto;
    int cantidad;

    public Pedido() {
        this("Sin producto", 0);
        System.out.println("Constructor vacío");
    }

    public Pedido(String producto, int cantidad) {
        this.producto = producto;
        this.cantidad = cantidad;
        System.out.println("Constructor con parámetros");
    }

    public static void main(String[] args) {
        Pedido p = new Pedido();
        System.out.println(p.producto + " x" + p.cantidad);
    }
}
```

> › 🔄 Solución

❓ ¡No hay preguntas tontas!

> **Q:** ¿El constructor puede ser `private` o tiene que ser `public`?

> **A:** Normalmente es `public` para que cualquiera pueda crear objetos. Existen constructores `private`, pero tienen truco (se usan en patrones como Singleton, que verás en otros módulos). Para esta unidad: `public` y a correr.

> **Q:** ¿Un constructor puede devolver algo? ¿Y si pongo `return` dentro?
>
> **A:** Un constructor **no devuelve nada**, ni siquiera el objeto (eso lo hace `new` por él). Puedes poner un `return;` suelto para salir antes, pero devolver un valor con `return 5` es un error de compilación. El constructor no es un cajero automático.

> **Q:** ¿Y si no necesito inicializar nada? ¿Tengo que escribir constructor igualmente?
>
> **A:** Si no necesitas inicializar nada y no escribes ningún constructor, Java te regala el vacío. Pero el día que añadas un constructor con parámetros para una cosita, el vacío desaparece. Regla: **si escribes uno, escribe también el vacío** si lo vas a usar.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué tres reglas de oro tiene un constructor?
2. ¿Qué hace Java si no escribes ningún constructor?
3. ¿Qué pasa si escribes un constructor con parámetros pero luego llamas a `new Clase()` sin argumentos?
4. ¿Cuándo se ejecuta un constructor?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. El **constructor** es un método especial con el mismo nombre que la clase, sin tipo de retorno, que se ejecuta con cada `new` para preparar al objeto.
2. **Sin constructor escrito**, Java regala uno vacío; **al escribir cualquier constructor**, el vacío desaparece.
3. Con **parámetros**, el objeto nace a medida: sus atributos quedan inicializados desde el primer segundo.

> 🦎 **Vocabulario rápido**

Término	Idea general
Constructor	Método especial que prepara al objeto al nacer
Constructor por defecto	El vacío que regala Java si no escribes ninguno
Constructor parametrizado	El que recibe datos para inicializar atributos
Instanciar	Crear un objeto con `new` (que llama al constructor)
`new`	Operador que reserva memoria, llama al constructor y devuelve la referencia

📚 Volver al índice de la unidad · **Anterior:** 03 · Atributos y métodos · **Siguiente:** 05 · La palabra clave this

05 — La palabra clave this

El objeto gritando «¡hablo de mí, no de otro!» para desambiguar y encadenar constructores 🗣️

> 🗺️ **Estás en:** 🧬 **U09 · POO: clases y objetos** → 05 · La palabra clave this

📬 La idea en una frase

> **`this` es el objeto hablando de sí mismo: sirve para desambiguar entre atributo y parámetro, para llamar a otro constructor, y para pasarte a ti mismo como argumento.**

Tienes un problema clásico: el parámetro se llama igual que el atributo. `nombre` aquí, `nombre` allá… ¿a cuál te refieres? Java necesita que le des una pista, y esa pista se llama `this`.

🔍 Desambiguar: atributo vs parámetro

Cuando el parámetro de un método (o constructor) se llama igual que un atributo, hay un **choque de nombres**. Sin `this`, Java da preferencia al parámetro (el más cercano), así que el atributo se queda sin tocar:

```
public class Persona {
    String nombre;
    int edad;

    public Persona(String nombre, int edad) {
         // Sin this: "nombre" es el parámetro, el atributo se queda con su valor por defecto
        nombre = nombre;   // ❌ no hace nada útil
        this.nombre = nombre;   // ✅ atributo = parámetro
        this.edad = edad;       // ✅ atributo = parámetro
    }
}
```

`this.nombre` significa **“el atributo `nombre` de ESTE objeto”**; `nombre` a secas (a la derecha) es el parámetro. El `this.` es la mano que señala: *“hablo del mío, no del tuyo”*.

> ⚠️ **Advertencia:** si no hay choque de nombres, `this` es opcional. Puedes escribir `acelerar(int inc) { velocidad += inc; }` sin `this`, porque `velocidad` solo puede ser el

> atributo. Pero cuando hay ambigüedad, `this` es obligatorio si quieres tocar el atributo.

🔗 Llamar a otro constructor: `this(...)`

`this` también sirve para llamar a un constructor desde otro constructor de la misma clase. Útil para no repetir código:

```
public class Galleta {
    String forma;
    boolean tieneChocolate;

    public Galleta() {
        this("redonda", false);   // "llama al otro constructor"
    }

    public Galleta(String forma, boolean tieneChocolate) {
        this.forma = forma;
        this.tieneChocolate = tieneChocolate;
    }
}
```

Ahora `new Galleta()` y `new Galleta("redonda", false)` hacen lo mismo, pero el código solo está escrito una vez. Una regla de oro:

> 💡 **Tip:** el `this(...)` que llama a otro constructor **debe ser la primera instrucción** del constructor. Java no deja que hagas otras cosas antes. Es la presentación antes de la fiesta: primero llamas al otro constructor y luego ya te pones a hacer tus cosas.

📦 Pasarte a ti mismo como argumento

`this` también puede viajar: pasas el objeto actual como argumento a otro método. Es la manera que tiene un objeto de decir: *“oye, pásame a mí mismo”*.

```java
public class Cliente {
    String nombre;

    void presentarse() {
        System.out.println("Hola, soy " + nombre);
    }
}

public class Sala {
    void recibir(Cliente c) {
        c.presentarse();
    }
}
```

```java
Cliente ana = new Cliente();
ana.nombre = "Ana";
Sala sala = new Sala();
sala.recibir(ana);   // desde fuera, pasas la variable
```

Dentro de la propia clase `Cliente` podrías hacer `sala.recibir(this)` para pasarte a ti mismo sin tener la variable a mano. Este caso te sonará "raro" ahora, pero es la semilla de patrones que verás en módulos más avanzados. Por ahora, memoriza la idea: **`this` es la referencia al objeto actual**.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** para decidir si hace falta `this`, pregúntate: *¿hay un parámetro o variable local con el mismo nombre que el atributo?* Si sí, `this.` señala al atributo. Si no, no lo necesitas.

Ejercicio: ¿compila o no?

Estas dos líneas están dentro de un constructor `Persona(String nombre)`. Una funciona y la otra es una pérdida de tiempo. ¿Cuál y por qué?

```java
nombre = nombre;        // línea A
this.nombre = nombre;   // línea B
```

> › 🔄 Solución

❓ ¡No hay preguntas tontas!

> **Q:** `this` es una palabra reservada, ¿no? ¿Puedo usarla fuera de una clase?
>
> **A:** No. `this` fuera de una clase es como pedir una pizza en una ferretería: no tiene sentido. Solo existe dentro del contexto de un objeto, es decir, dentro de métodos no estáticos y constructores de una clase.

> **Q:** ¿Y qué pasa con los métodos `static`? ¿Ahí puedo usar `this`?
>
> **A:** No, y ese es el clásico de los exámenes. Los métodos `static` no pertenecen a un objeto concreto, así que no hay "este objeto" al que señalar. Usar `this` en un `static` es error de compilación. Los `static` tienen su propia unidad: la U10.

> **Q:** ¿Por qué en el punto 3 escribí métodos sin `this` y funcionaban?
>
> **A:** Porque no había ambigüedad. En `int volumen() { return ancho * alto * profundo; }` no hay ningún parámetro llamado `ancho`, así que Java solo puede referirse al atributo. `this` es el que habla en voz alta cuando hay dos personas llamadas igual en la sala.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué significa `this.nombre`?
2. ¿Cuándo es obligatorio usar `this`?
3. ¿Qué regla debe cumplir el `this(...)` que llama a otro constructor?
4. ¿Puedo usar `this` dentro de un método `static`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **`this`** es la referencia al objeto actual: `this.nombre` habla del atributo, `nombre` a secas del parámetro.
2. **`this(...)`** encadena constructores de la misma clase y debe ir siempre primero.
3. No hace falta `this` sin ambigüedad, y está **prohibido** dentro de métodos `static`.

🦎 Vocabulario rápido

Término	Idea general
`this`	La referencia al objeto actual
Desambiguar	Dejar claro a qué `nombre` te refieres
Choque de nombres	Parámetro y atributo con el mismo nombre
`this(...)`	Llamada a otro constructor de la misma clase
`static`	Contexto sin objeto (no admite `this`)

📚 Volver al índice de la unidad · **Anterior:** 04 · Constructores · **Siguiente:** 06 · Referencias, null y memoria

06 — Referencias, null y memoria

Qué guarda de verdad una variable de objeto, qué pinta el heap y por qué el NullPointerException te va a despertar por la noche 🕳️

> 🗺️ **Estás en:** 🏗️ **U09 · POO: clases y objetos** → 06 · Referencias, null y memoria

---

📬 La idea en una frase

> **Una variable de objeto no guarda el objeto: guarda la dirección (la referencia) donde el objeto vive en el heap. Y `null` es una referencia que no apunta a nada.**

En el punto 2 tenías una duda pendiente: qué pasa con `Galleta g3 = g1`. Esa duda se llama **referencias**, y es de las pocas cosas de esta unidad que de verdad merecen que las veas venir. Porque cuando lo entiendes, el 90% de los "bugs mágicos" dejan de ser mágicos.

📬 La carta frente a la casa

Piensa en las variables primitivas como en las direcciones postales: `int a = 5` guarda el **5** directamente, como una foto en tu cartera. Pero una variable de objeto guarda **la dirección de la casa**, no la casa entera:

```
Galleta g = new Galleta();
```

- La galleta (el objeto) vive en una zona de memoria llamada **heap**.
- La variable `g` guarda un número que es *la dirección* de esa galleta en el heap.

Ese número se llama **referencia**. Tú nunca lo ves (Java lo esconde), pero es la clave de todo lo que pasa en este punto.

> 💡 **Tip:** cuando haces `new`, Java fabrica el objeto en el heap y te entrega un "plano de localización". Tu variable guarda ese plano. Copiar el plano no copia la casa.

👯 El alias: dos variables, un objeto

Aquí está el momento que lo cambia todo:

```
Galleta g1 = new Galleta();
g1.sabor = "Chocolate";

Galleta g2 = g1;          // NO crea otra galleta
g2.sabor = "Vainilla";    // ¿qué sabor tiene g1 ahora?

System.out.println(g1.sabor);  // "Vainilla" 😱
```

`g2 = g1` copia la **referencia**, no el objeto. Ahora `g1` y `g2` apuntan a **la misma galleta**. Cambiar el sabor con `g2` cambia la galleta que también ve `g1`. Es el famoso **alias**: dos nombres, una sola galleta.

> ⚠️ **Advertencia:** este es el clásico que cae en los exámenes "¿Qué imprime?". La regla: **copiar una variable de objeto copia la dirección, no el objeto**. Si quieres una copia independiente, tendrías que crear un objeto nuevo (`new`) y copiar atributo a atributo (o usar lo que verás en la U12).

🍫 `null`: la referencia al vacío

Una variable de objeto que aún no apunta a nada vale `null`:

```
Galleta g;          // sin inicializar: aún no apunta a nada
Galleta h = null;   // explícitamente "no hay galleta"
Galleta i = new Galleta();   // apunta a una galleta de verdad
```

`null` significa "referencia que no apunta a ningún objeto". Y aquí viene el monstruo:

```
Galleta h = null;
h.sabor = "Chocolate";   // 💥 NullPointerException
```

Si intentas tocar un atributo o llamar un método de una referencia `null`, Java lanza **`NullPointerException`** (NPE): el programa muere en el acto. Es el error más famoso y odiado de Java.

> 💡 **Tip:** cuando veas un `NullPointerException` en tu programa, busca la variable que vale `null` y pregúntate: *¿cuándo debía apuntar a un objeto y no lo hizo?* El 90% de las veces es un objeto que nunca se creó con `new` o un atributo sin inicializar.

🏠 Primitivos vs referencias: el resumen

	Primitivos (`int`, `double`…)	**Referencias (objetos)**
Qué guarda la variable	El **valor** directamente	La **dirección** del objeto en el heap
Comparar con `==`	Compara valores	Compara **direcciones** (¿mismo objeto?)
Valor "vacío"	`0`, `false`, `0.0`	`null`
Copiar	Copia el valor	Copia la dirección (alias)

> 📝 **Nota:** en la UO3 viste que `String` se compara con `equals`, no con `==`. Ahora tienes el porqué: `String` es un objeto, así que `==` compara referencias, no contenido. Los literales iguales de `String` a veces comparten dirección (el *string pool*), por eso `"Hola" == "Hola"` puede dar `true` por pura casualidad. No confíes nunca en eso: usa `equals`.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** dibuja el heap. Por cada `new`, una cajita con su etiqueta. Por cada variable, una flecha hacia su cajita (o hacia el vacío si es `null`). Después solo tienes que seguir las flechas.

Ejercicio: el lío de las flechas

Sin ejecutar, ¿qué imprime este programa?

```java
public class Cuenta {
    int saldo;
}

public class Main {
    public static void main(String[] args) {
        Cuenta a = new Cuenta();
        a.saldo = 100;

        Cuenta b = a;
        b.saldo += 50;

        Cuenta c = new Cuenta();
        c.saldo = a.saldo;

        a = null;

        System.out.println(b.saldo);
        System.out.println(c.saldo);
    }
}
```

> 🔄 Solución

❓ ¡No hay preguntas tontas!

Q: ¿Se puede comparar una referencia con `null` para saber si apunta a algo?

A: Sí, y es súper útil: `if (galleta == null) { ... }` o `if (galleta != null) { ... }`. Es la forma de blindar tu código contra los NPE: *"si no es null, haz cosas; si es null, salta del barco"*.

Q: ¿Qué pasa con los objetos que ya no tienen ninguna variable apuntándolos? ¿Quedan ahí para siempre?

A: No. Java tiene un **recolector de basura** (garbage collector) que detecta los objetos "huérfanos" (sin referencias) y libera su memoria. Tú no tienes que hacer nada: es la criada invisible de Java. En C++ eso era manual y dolía.

Q: ¿Y si hago `null` a todas las variables que apuntan al mismo objeto?

> **A:** Cuando la última referencia desaparece, el recolector se lleva el objeto. Es la forma de "matar" un objeto: quitarle todos los apuntadores. No hay `delete` en Java; hay que dejar de mirarlo hasta que desaparezca.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué guarda una variable de objeto: el objeto o su dirección?
2. ¿Qué hace `Galleta g2 = g1;`? ¿Crea otra galleta?
3. ¿Qué significa `null`?
4. ¿Qué excepción lanza Java si usas una referencia `null`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Una variable de objeto guarda la **referencia** (la dirección en el heap), no el objeto; por eso copiarla crea un **alias**.
2. **`null`** es una referencia que no apunta a nada, y usarla provoca la temida **`NullPointerException`**.
3. Los primitivos guardan el valor (y se comparan con `==`); los objetos guardan direcciones (y se comparan con `equals`).

> 🐛 **Vocabulario rápido**

Término	Idea general
Referencia	La dirección donde vive el objeto
Heap	La zona de memoria donde viven los objetos
Alias	Dos variables apuntando al mismo objeto
`null`	Referencia que no apunta a nada
NPE	`NullPointerException`, el crash por usar `null`

Término	Idea general
Garbage collector	El que recicla objetos sin referencias

📚 Volver al índice de la unidad · **Anterior:** 05 · La palabra clave this · **Siguiente:** 07 · Sobrecarga y paso por valor

07 — Sobrecarga y paso por valor

Varios métodos con el mismo nombre y qué se copia de verdad cuando llamas (pista: las referencias te engañan) 🎭

> 🗺️ **Estás en:** 🏗️ **U09 · POO: clases y objetos** → 07 · Sobrecarga y paso por valor

---

📬 La idea en una frase

> **La sobrecarga permite que varios métodos compartan nombre si sus parámetros difieren, y en Java todo se pasa por valor: los primitivos se copian, y de las referencias se copia la dirección (no el objeto).**

Dos conceptos que parecen de otro planeta y que son pan de cada día en el código real. El primero te da flexibilidad (un método `sumar` que funciona con `int`, con `double` y con tres números). El segundo te explica por qué tu método no consigue "cambiar" un `int` que le pasas… pero sí modifica un objeto. Vamos a por ellos.

🎭 Sobrecarga: el mismo nombre, otras manos

Sobrecargar un método es declarar varios métodos con el **mismo nombre** pero con **distinta lista de parámetros** (número, tipo u orden). Java decide cuál usar mirando los argumentos de la llamada:

```
public class Calculadora {
    int sumar(int a, int b) {
        return a + b;
    }

    int sumar(int a, int b, int c) {
        return a + b + c;
    }

    double sumar(double a, double b) {
        return a + b;
    }
}
```

```
Calculadora calc = new Calculadora();
calc.sumar(2, 3);          // usa el de (int, int) → 5
calc.sumar(2, 3, 4);       // usa el de (int, int, int) → 9
calc.sumar(2.5, 3.5);      // usa el de (double, double) → 6.0
```

> ⚠️ **Advertencia:** lo que **no** puede cambiar para distinguir métodos es solo el tipo de retorno. `int sumar(int, int)` y `double sumar(int, int)` no pueden coexistir: Java no sabe cuál quieres si solo miras `sumar(2, 3)`. El desempate se hace con la **firma** (nombre + parámetros), no con lo que devuelven.

Los **constructores** también se sobrecargan, y de hecho ya lo viste en el punto 4:

```
public class Persona {
    String nombre;
    int edad;

    public Persona() {
        this("Desconocido", 0);
    }

    public Persona(String nombre) {
        this(nombre, 0);
    }

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

Tres maneras de nacer: sin datos, solo con nombre, o con todo. El `this(...)` encadenado evita repetir la inicialización.

📦 Paso por valor: Java solo sabe copiar

Cuando llamas a un método, Java hace una copia de cada argumento y se la pasa. Eso se llama **paso por valor**: el método recibe **copias**, no los originales.

Con primitivos queda claro:

```
void duplicar(int x) {
    x = x * 2;   // cambia la copia, no el original
}

int numero = 21;
duplicar(numero);
System.out.println(numero);   // 21. La copia se duplicó, el original ni se enteró
```

Con referencias la cosa se complica. La variable guarda una dirección; se copia **la dirección**, no el objeto:

```
void subirSaldo(Cuenta c) {
    c.saldo += 100;   // el objeto al que apunta c SÍ cambia
}

Cuenta miCuenta = new Cuenta();
miCuenta.saldo = 50;
subirSaldo(miCuenta);
System.out.println(miCuenta.saldo);   // 150 ✅ (el objeto cambió)
```

La copia y el original guardan **la misma dirección**, así que ambos apuntan al mismo objeto: modificar el objeto se nota fuera.

> 📝 **Nota:** la trampa es esta: puedes cambiar *el contenido* del objeto, pero **no puedes** hacer que la variable original apunte a otro objeto. Si dentro del método haces `c = new Cuenta()`, solo la copia cambia de dirección; el original sigue apuntando a su cuenta. Es como darle a alguien las llaves de tu casa: puede redecorar la casa, pero no puede cambiar la dirección de tu casa.

📊 El resumen que te salva en el examen

Qué le pasas	Qué recibe el método	¿Se nota fuera?
Primitivo (`int`, `double`…)	Una **copia del valor**	❌ No

Qué le pasas	Qué recibe el método	¿Se nota fuera?
Referencia	Una **copia de la dirección**	✅ Sí (si modificas el objeto) / ❌ No (si reasignas la referencia)

> 💡 **Tip:** memorízalo así: *Java copia siempre, pero copiar una dirección te permite llegar al mismo sitio.* Los primitivos nunca cambian fuera; los objetos cambian si los tocas, pero no puedes cambiarles el “punto de mira” desde dentro.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** para trazar una llamada, dibuja las cajas de los argumentos originales y las cajas de las copias del método. Después sigue quién apunta a quién.

Ejercicio: la gran confusión

Sin ejecutar, ¿qué imprime este programa?

```java
public class Caja {
    int valor;
}

public class Main {
    static void cambiar(Caja c, int n) {
        c.valor = 99;
        n = 7;
        c = new Caja();
        c.valor = 1;
    }

    public static void main(String[] args) {
        Caja miCaja = new Caja();
        miCaja.valor = 5;
        int numero = 5;

        cambiar(miCaja, numero);

        System.out.println(miCaja.valor);
        System.out.println(numero);
    }
}
```

> 🔄 Solución

❓ ¡No hay preguntas tontas!

Q: ¿Por qué no puedo sobrecargar cambiando solo el tipo de retorno? Parecería lógico.

A: Porque Java elige el método mirando los **argumentos** de la llamada. Si dos métodos tienen los mismos parámetros pero distinto retorno, `calc.sumar(2, 3)` sería ambiguo: ¿cuál quiere el que llama? Java odia la ambigüedad más que a los lunes.

Q: ¿Los constructores sobrecargados "heredan" algo entre sí?

A: No heredan, pero se **encadenan** con `this(...)`. El constructor corto llama al largo, que hace el trabajo pesado. Así el código de inicialización se escribe una sola vez. Cuando veas un constructor de tres líneas que solo hace `this(...)`, es esa técnica en acción.

> **Q:** ¿Existe el “paso por referencia” en Java?
>
> **A:** No. Java **solo** tiene paso por valor. Lo que pasa es que con referencias se copia la dirección, y eso *parece* paso por referencia. Los puristas lo llaman “paso por valor de la referencia”. En el examen, di que Java es siempre paso por valor.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué es sobrecargar un método?
2. ¿Qué NO puede cambiar para distinguir dos métodos con el mismo nombre?
3. ¿En Java el paso es por valor o por referencia?
4. Si un método recibe un objeto y lo modifica, ¿se nota fuera?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. La **sobrecarga** permite varios métodos con el mismo nombre y distintos parámetros; el tipo de retorno no sirve para distinguirlos.
2. Java es **paso por valor**: los primitivos se copian y nunca cambian fuera; las referencias copian la dirección, así que puedes modificar el objeto pero no reasignarlo desde dentro.
3. Los **constructores** también se sobrecargan y se encadenan con `this(...)` para no repetir código.

> 🦎 **Vocabulario rápido**
>
Término	Idea general
Sobrecarga	Varios métodos con el mismo nombre y distinta firma
Firma	Nombre + lista de parámetros
Paso por valor	Se pasan copias, nunca los originales
Copia de referencia	Se copia la dirección, no el objeto

Término	Idea general
Encadenar constructores	Un constructor llama a otro con `this(...)`

📚 Volver al índice de la unidad · **Anterior:** 06 · Referencias, null y memoria · **Siguiente:** 08 · Be the Code: diseña tu clase

08 — Be the Code: diseña tu clase

Cierra la página, abre el editor y diseña una clase completa de principio a fin (o sufre con honor) 🕶️

> 🗺️ **Estás en:** 🏗️ **U09 · POO: clases y objetos** → 08 · Be the Code: diseña tu clase

---

📬 La idea en una frase

> **Ya has visto la teoría: ahora toca borrarla de la memoria y construir una clase completa tú solo, paso a paso, porque una clase solo se entiende de verdad cuando te duele diseñarla.**

> 🕶️ **Don Tip:** toda clase se diseña con la misma escalera: (1) elige el nombre y sus atributos, (2) el constructor que los inicializa, (3) los métodos que saben hacer cosas con ellos, (4) un `main` que la pruebe. Si dominas esa escalera, cualquier clase es tuya.

🚦 Ejercicio guiado: la cuenta bancaria

Cierra esta página. Abre un editor en blanco. No mires ni una línea de lo leído hasta ahora.

El encargo: diseñar una clase `CuentaBancaria` que guarde el titular y el saldo, y que sepa **ingresar** dinero, **retirar** dinero (sin dejar el saldo en negativo) y **mostrarse** por pantalla.

Paso 1: el nombre y los atributos

¿Qué datos necesita una cuenta bancaria? Mínimo dos: quién la tiene (el **titular**, un `String`) y cuánto dinero hay (el **saldo**, un `double`). Los atributos van dentro de la clase, fuera de cualquier método:

```
public class CuentaBancaria {
    String titular;
    double saldo;
}
```

> 🧠 **Antes de seguir, respóndete a ti mismo:** ¿el saldo debería ser `int` o `double`? ¿Por qué? (Pista: el dinero tiene céntimos, y los céntimos no son enteros.)

Paso 2: el constructor

Los atributos no deben nacer sin rumbo. Un constructor que reciba el titular y el saldo inicial y los coloque con `this`:

```
public CuentaBancaria(String titular, double saldoInicial) {
    this.titular = titular;
    this.saldo = saldoInicial;
}
```

Paso 3: los métodos

La cuenta necesita tres comportamientos:

- `ingresar(double cantidad)`: suma al saldo. Debería rechazar cantidades negativas (¿quién ingresa -50 €?).
- `retirar(double cantidad)`: resta al saldo, pero **sin dejarlo negativo**. Si no hay saldo suficiente, no retira y avisa.
- `mostrar()`: imprime "Titular: X | Saldo: Y €".

Escribe cada método con su firma y su lógica. Recuerda: los métodos que solo hacen cosas son `void`; si alguno necesitara devolver algo, usaría `return`.

Paso 4: pruébala con un `main`

Ninguna clase está terminada hasta que no la has visto funcionar. Añade un `main` que cree una cuenta, ingrese, retire (incluso más de lo que hay, para ver la protección) y muestre el resultado.

La solución completa (inténtalo antes de mirar)

> 🔄 **Solución de la cuenta bancaria**

⭐ Niveles de logro

- ⭐ **Lo tienes, pero has tenido que mirar la solución una vez.** Aprobado raspado.
- ⭐ ⭐ **Te ha salido a la primera y funciona.** Eres una máquina.
- ⭐ ⭐ ⭐ **Te ha salido a la primera, con los dos `if` de protección y, además, le has añadido un método `double consultarSaldo()` que devuelve el saldo.** No necesitas este curso. Vete a dar una charla TED.

> [!TIP] Pista mental gratuita: una clase necesita **cuatro piezas** — atributos, constructor, métodos y una prueba (`main`). Si memorizas esa escalera, cualquier clase que te pidan en el examen se te rinde en cinco minutos.

🧩 El lío

El departamento de calidad ha recibido esta clase. Algo huele mal. Encuentra los **3 errores**:

```
public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca) {
        marca = marca;
    }

    void acelerar(int inc) {
        velocidad += inc;
    }

    void mostrar() {
        System.out.println(marca + " a " + velocidad + " km/h");
    }
}
```

> 👓 **Don Tip:** ejecútalo con `new Coche("Seat")` y mira qué imprime `mostrar()`. Después revisa el constructor con lupa.

> › 🔄 **Solución del lío**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuáles son las cuatro piezas de la escalera para diseñar una clase?
2. ¿Qué pasa en el constructor si escribes `marca = marca` sin `this`?
3. ¿Por qué el `retirar` necesita un `if` antes de restar?
4. ¿Cuándo está “terminada” una clase?

> 🔁 Respuestas

✅ Resumen en 3 frases

1. Diseñar una clase es una **escalera de cuatro pisos**: atributos, constructor, métodos y prueba.
2. El constructor **bautiza los atributos** con `this` cuando hay choque de nombres; sin él, los atributos viven en `null` o en `0`.
3. Los métodos **protegen los datos** (un `if` antes de retirar) y el `main` demuestra que todo funciona.

> 🦎 **Vocabulario rápido**

Término	Idea general
Diseñar una clase	Elegir atributos, constructor, métodos y prueba
Bautizar	Inicializar los atributos en el constructor
Proteger	Validar antes de modificar (no saldos negativos)
`main` de prueba	El programa que demuestra que la clase funciona
Iterar	Probar, encontrar el error, corregir y volver a probar

📚 Volver al índice de la unidad · **Anterior:** 07 · Sobrecarga y paso por valor · **Siguiente:** 09 · Repaso interactivo

09 — Repaso interactivo: domina las clases

El cierre práctico de la unidad, con objetos, constructores y un `null` que no va a funcionar 😈

> 🗺️ **Estás en:** 🏗️ **U09 · POO: clases y objetos** → 09 · Repaso interactivo

---

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```java
public class Galleta {
    String sabor;

    public Galleta(String sabor) {
        this.sabor = sabor;
    }

    void comer() {
        System.out.println("Ñam, galleta de " + sabor);
    }
}

public class Main {
    public static void main(String[] args) {
        Galleta g1 = new Galleta("chocolate");
        Galleta g2 = new Galleta("vainilla");
        Galleta g3 = g1;
        g3.sabor = "limón";
        g1.comer();
        g2.comer();
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **`Ñam, galleta de chocolate` y `Ñam, galleta de vainilla`** → Cada galleta guarda su sabor original, siempre. ❌
2. **`Ñam, galleta de limón` y `Ñam, galleta de vainilla`** → ✅ ¡Correcto! `g3 = g1` copia la referencia, no la galleta: `g3` y `g1` son la misma galleta. Cambiar el sabor con `g3` lo cambia para `g1`. `g2` es otra galleta, independiente.
3. **`Ñam, galleta de limón` y `Ñam, galleta de limón`** → El cambio de `g3` contamina a todas las galletas. ❌

> 🔄 Solución

🔥 Fireside Chat: Clase vs objeto

> *Dos veteranos del molde y la galleta discuten junto a la bandeja del horno.*

Clase: — Mira, yo soy el molde. Sin mí, tus objetos serían amorfos. Yo defino qué atributos y métodos tienen todos mis objetos. Soy la constitución de mi pueblo.

Objeto: — La constitución… claro, y yo soy el ciudadano que paga impuestos. Tú eres un archivo `.java` en el disco. Yo ocupo memoria, tengo saldo, puedo acelerar. Cuando ejecutan el programa, el que trabaja soy yo.

Clase: — ¿Y cuántos ciudadanos como tú puedo producir? Miles. Yo soy único, tú eres reproducible. Eres una copia, un clon, un *vulgar* duplicado.

Objeto: — Un duplicado con vida propia, sí. Tú defines el plano, pero yo soy el edificio. Nadie vive en un plano. Y te recuerdo: sin `new`, tú no eres más que teoría de archivo.

Clase: — Sin mí, `new` no tendría nada que fabricar. Nos necesitamos. Como el cortapastas y la galleta.

Objeto: — Trato hecho. Tú el molde, yo la galleta. Y que nadie se olvide de cuál se come al final.

> La lección: la **clase** es el diseño (QUÉ y CÓMO), el **objeto** es la instancia concreta (QUIÉN). Sin clase no hay objeto; sin `new`, la clase no da de comer.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy el molde que define atributos y métodos, pero no soy ninguna cosa concreta.**
2. **Soy la galleta: una cosa concreta, con valores propios, viviendo en la memoria.**
3. **Soy el método especial que se ejecuta con cada `new` para que el objeto nazca bien preparado.**
4. **Soy la referencia que no apunta a ningún objeto; usarme provoca el error más famoso de Java.**
5. **Soy la palabra que usa el objeto para hablar de sí mismo y deshacer líos de nombres.**
6. **Soy la dirección donde vive el objeto, la zona de memoria donde Java coloca las galletas.**

> › 🔄 **Respuestas**

🤬 CONRAD VS EL MUNDO: “Constructor, ¿dónde estás?”

> *CONRAD, nuestro compilador cascarrabias, opina sobre el clásico del novato.*

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: *CONRAD, no compila*. Y yo: vale, ¿qué dice el error? *Pues no sé, no lo he leído.* ¡AY, MADRE MÍA! Pongo la línea exacta, el motivo y hasta la flechita ^, ¿y no lo lees?

Y luego está el clásico: escriben un constructor con parámetros… y llaman a `new Clase()` sin argumentos. *¡Pero si tú mismo has borrado el constructor vacío!* Cuando escribes cualquier constructor, el vacío desaparece. Es como quitar la puerta de tu casa y luego intentar entrar por la puerta.

Y el colmo de los colmos: `Galleta g;` y luego `g.sabor = "chocolate"`. ¡Pero si `g` no apunta a NADA! No has hecho `new`. Eso es `NullPointerException` en el acto. Es como intentar ponerle un collar a un perro que no existe.

La lección: antes de acusar a Java de “odiarte”, repasa el trío sagrado: **¿hice `new`? ¿el constructor existe con los argumentos que paso? ¿la referencia es `null`?** El 90% de los errores de esta unidad se arreglan con esas tres preguntas. El compilador no te odia: te está pasando las respuestas del examen.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. ¿Cuántos objetos crea `Galleta a = new Galleta(); Galleta b = a;`?
   - a) 2 b) 1
2. ¿Qué imprime `System.out.println(new Galleta("chocolate").sabor);`?
   - a) `chocolate` b) `null`
3. `Persona p;` sin `new`. ¿Cuánto vale `p`?
   - a) `0` b) `null`
4. ¿Puedo escribir `int duplicar(int x)` y `double duplicar(int x)` en la misma clase?
   - a) Sí, cambia el retorno b) No, misma firma

> › 🔁 **Soluciones**

Laboratorio de tortura: la fábrica que llora

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto cuando lo arregles.

```
public class Coche
    String marca;
    int velocidad;

    public Coche(String marca, int velocidad) {
        this.marca = marca;
        this.velocidad = velocidad;
    }

    void acelerar(int inc) {
        velocidad -= inc;
    }

    void mostrar() {
        System.out.println(marca + " va a " + velocidad);
    }

    public static void main(String[] args) {
        Coche c = new Coche();
        c.acelerar(50);
        c.mostrar()
    }
}
```

Fallo intencionado: uno de los errores parece correcto a simple vista porque “se ve bien”, pero cambia por completo el comportamiento del programa.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún ; que falte? *no → sigue buscando.*
   > **¿Y si sigo atascado?**

2. ¿Compila ya? *no → mira el mensaje de error y los argumentos del constructor.*

   > **¿Y si sigo atascado?**

3. ¿Ejecuta pero la velocidad sale rara? *Es el error de lógica: el signo del método.*

   > **Solución final**

Los **3 errores de compilación**:

1. Falta la `{` de apertura de la clase después de `Coche`.
2. `new Coche()` no coincide con el constructor: el único es `Coche(String, int)`. Al escribir un constructor con parámetros, el vacío desaparece.
3. Falta el `;` al final de `c.mostrar()`.

El **error de lógica**: `velocidad -= inc` **resta** en vez de sumar. Compila y ejecuta perfectamente, pero el coche acelera "hacia atrás": con la velocidad inicial a 0 y acelerar 50, imprime `Seat va a -50` en vez de `Seat va a 50`. Un signo separaba tu coche de la verdad.

```
public class Coche {
    String marca;
    int velocidad;

    public Coche(String marca, int velocidad) {
        this.marca = marca;
        this.velocidad = velocidad;
    }

    void acelerar(int inc) {
        velocidad += inc;
    }

    void mostrar() {
        System.out.println(marca + " va a " + velocidad);
    }

    public static void main(String[] args) {
        Coche c = new Coche("Seat", 0);
        c.acelerar(50);
        c.mostrar();
    }
}
```

Salida correcta: `Seat va a 50`. Con la versión rota, una vez arreglados los otros errores, la velocidad salía `-50`: el signo era la pista del error de lógica.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Cortapastas**	Explicar la diferencia entre clase y objeto con la analogía de las galletas sin mirar
🏅 **Primera Clase**	Escribir una clase con atributos, constructor y métodos que funcione a la primera
🏅 **Cazador de NPE**	Encontrar el `null` culpable de un `NullPointerException` sin pistas
🏅 **El Desambiguador**	Explicar por qué hace falta `this` cuando hay choque de nombres
🏅 **El Constructor Rudo**	Diseñar una clase con sobrecarga de constructores encadenados con `this(...)`

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```java
public class Cuenta {
    int saldo;

    void sumar(int x) {
        saldo += x;
    }
}

public class Main {
    public static void main(String[] args) {
        Cuenta a = new Cuenta();
        a.saldo = 100;
        Cuenta b = a;
        b.sumar(50);
        System.out.println(a.saldo);
    }
}
```

2. **El constructor perdido:** en el ejemplo `Galleta` del punto 2 (sin constructor), ¿qué le pasa al código si añades `public Galleta(String sabor)` y dejas `new Galleta()` en otro sitio?
3. **El detective del heap:** tu programa imprime `null` cuando esperabas un nombre. ¿Qué herramienta usas y qué miras primero?
4. **Verdadero o falso:** "si un método recibe un objeto y dentro del método hago `parametro = null`, el objeto original desaparece".

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Operador que fabrica objetos en memoria (3 letras)
3. El método que prepara al objeto al nacer (11 letras)
5. Referencia que no apunta a nada (4 letras)
6. Zona de memoria donde viven los objetos (4 letras)

Vertical:
2. Palabra que usa el objeto para hablar de sí mismo (4 letras)
4. El molde que define atributos y métodos (5 letras)
```

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, la diferencia entre una clase y un objeto."**
2. **"¿Qué hace `new` por dentro cuando creas un objeto?"**
3. **"¿Por qué `String` se compara con `equals` y no con `==`?"**
4. **"¿Java pasa los argumentos por valor o por referencia? Justifica con un ejemplo de objeto."**
5. **"Si dos variables apuntan al mismo objeto y una le cambia un atributo, ¿qué ve la otra?"**
6. **"¿Para qué sirve sobrecargar un constructor y cómo lo encadenas con `this`?"**

🤷 No hay preguntas tontas

> ❓ **¿Puedo escribir varias clases en un mismo archivo `.java`?**

Sí, pero solo una puede ser `public` (la que da nombre al archivo). Las demás van sin `public`. Para los proyectos de clase, un archivo = una clase pública, y cada clase en su archivo. Tus profesores lo agradecerán.

> ❓ **¿Por qué a veces veo `get` y `set` antes de los nombres de métodos, como `getSaldo()`?**

Esa es la convención de los **getters y setters**: métodos para leer (`getSaldo`) y modificar (`setSaldo`) atributos de forma controlada. Suena a burocracia, pero es la base de la **encapsulación**, y tiene su unidad entera: la U10. Aquí los verás de refilón; allí los dominarás.

> ❓ **¿Puedo tener un método llamado igual que el constructor?**

No. El constructor siempre se llama igual que la clase y no devuelve nada. Un método normal no puede tener el nombre de la clase (a menos que sea… un constructor, que se detecta por la falta de tipo de retorno). Si intentas añadirle un `void` delante al constructor, Java te mirará raro.

🎬 Poscréditos

La programadora termina su fábrica de galletas virtual: una clase `Galleta` con constructor, una clase `Horno` que la instancia, y un `main` que hornea cinco galletas distintas. Cada una con su sabor. Hasta que, en la última línea, se le escapa un `null` y todo se apaga con un `NullPointerException`.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — Cinco galletas, cinco objetos, y solo un `null` para estropearlo todo. Típico. ¿Sabes qué ha pasado?

Programadora: — No hice `new`. La referencia apuntaba al vacío.

CONRAD: — *asiente* Ves. Cuando entiendes qué guarda una variable, los errores dejan de ser magia. Clase, objeto, constructor, `this`, referencias… Ya no escribes recetas, creas mundos. Pero esto no ha hecho más que empezar.

Programadora: — ¿Qué más hay?

CONRAD: — *toma un sorbo* Ahora tus atributos están al aire. Cualquiera los toca. La próxima parada es saber quién tiene permiso para entrar y quién no. Espérame, que voy a disfrutar viéndote proteger tus datos.

PRÓXIMAMENTE EN U10: Visibilidad, encapsulación y `static`. El momento en el que tus objetos aprenden a decir “eso no se toca”. 🔒

📚 Volver al índice de la unidad · **Anterior:** 08 · Be the Code: diseña tu clase · **Siguiente:** U10 · Visibilidad, encapsulación y static

U10 — Visibilidad, encapsulación y static

Los candados de tu código: quién ve tus atributos, cómo blindarlos y qué pertenece a la clase y no al objeto 🔒

Bienvenido a la unidad de los candados. En la U09 aprendiste a crear clases y objetos: clases con atributos y métodos que se comportaban como *cosas* del mundo real. Pero había un problema escondido: **todo era público**. Cualquiera podía hacer `persona.edad = -666` y tu objeto quedaba hecho unos zorros. Vivir en una casa de cristal es incómodo, ¿verdad? Pues tu código también merece puertas, llaves y un buen candado.

Esta unidad tiene dos mitades que al final bailan juntas:

- **Visibilidad y encapsulación:** aprenderás a esconder tus atributos con `private` y a controlar cómo se leen y modifican con `public` (getters y setters). Es el pilar que sostiene la POO: tu objeto decide qué muestra y qué guarda, y nadie deja tu clase en un estado imposible.
- **Static:** aprenderás la diferencia entre lo que pertenece a cada **objeto** (cada gato tiene su nombre) y lo que pertenece a la **clase** (todos comparten el contador de gatos). Atributos estáticos, métodos estáticos y constantes `static final` que ni el tiempo se atreve a tocar.

En el camino descubrirás por qué `main` es `static` aunque no tuviera por qué, cómo se construye una clase utilitaria como `Math` (que nadie puede instanciar), y por qué los expertos dicen que los atributos `public` son como dejar la cartera abierta en el metro.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar por qué **esconder** tus atributos es buena idea (y no paranoia).
- Distinguir los **4 niveles de visibilidad**: `public`, `protected`, package-private y `private`.
- **Encapsular** clases con atributos `private` y controlar el acceso con getters y setters `public`.
- **Validar** datos en los setters para que tu objeto nunca viva en un estado imposible.

- Conocer el **patrón JavaBeans** y las convenciones de nombres.
- Usar **atributos y métodos estáticos** (`static`) para lo que pertenece a la clase, no al objeto.
- Entender por qué `main` es `static` y crear **clases utilitarias** con constructor privado.
- Declarar **constantes de clase** con `static final` y saber cuándo usarlas.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · Visibilidad: el arte de no enseñarlo todo	La casa de cristal: el problema de enseñar todos tus atributos al mundo	Todos
02 · public, private y protected	Los 4 niveles de visibilidad y quién ve cada cosa de tu clase	Todos
03 · Encapsulación: el pilar de la POO	Atributos privados, control de acceso y el patrón JavaBeans	Todos
04 · Getters y setters	El portero de la discoteca: leer y modificar atributos con validación	Todos
05 · Atributos static	Lo que pertenece a la clase, no al objeto: el grupo de WhatsApp	Todos
06 · Métodos static y el main	Llamar a métodos sin crear objetos y por qué `main` es estático	Todos
07 · Constantes de clase: static final	Valores inmutables en MAYÚSCULAS y las clases utilitarias	Todos
08 · Be the Code: encapsula tu clase	Convierte una clase sin blindar en una bien encapsulada, paso a paso	⭐⭐⭐
09 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA4)

RA4: Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.

CE	Criterio	Dónde se cubre
RA4 a)	Se ha reconocido la sintaxis, estructura y componentes típicos de una clase.	✅ Punto 1
RA4 b)	Se han definido clases.	✅ Puntos 3 y 8
RA4 c)	Se han definido propiedades y métodos.	✅ Puntos 2, 3 y 4
RA4 d)	Se han creado constructores.	✅ Puntos 3 y 4
RA4 e)	Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.	✅ Puntos 8 y 9
RA4 f)	Se han utilizado mecanismos para controlar la visibilidad de las clases y de sus miembros.	✅ Puntos 1 y 2
RA4 h)	Se han creado y utilizado métodos estáticos.	✅ Puntos 5, 6 y 7

> 📌 La herencia (RA4 g) y las interfaces (RA4 i) tienen su capítulo propio en la **U11**. Los conjuntos y librerías de clases (RA4 j) florecen en la **U12** y la **U13**. Aquí nos quedamos con visibilidad, encapsulación y static, que son los cimientos de todo lo demás.

🚪 ¿Por dónde empiezo?

- ¿Cero conocimientos? → Arranca en el punto 1. Solo necesitas lo que viste en la U09.
- ¿Ya blindas atributos y quieres chicha? → Ve directo al punto 5 y al punto 6: ahí empieza el mundo estático.
- ¿Solo vienes a por los candados? → Salta al punto 3 y al punto 4.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Visibilidad: el arte de no enseñarlo todo

** ⏭️ Al acabar la unidad, continúa en U11 · Herencia, polimorfismo e interfaces.

01 — Visibilidad: el arte de no enseñarlo todo

Vivir en una casa de cristal está fatal: tu código también merece puertas, llaves y candados 🏠

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 01 · Visibilidad: el arte de no enseñarlo todo

---

📬 La idea en una frase

> **Si todo en tu clase es `public`, cualquier persona (o tú mismo dentro de un año) puede romper tus objetos. La visibilidad es el arte de decidir qué muestra tu clase y qué se queda en casa.**

En la U09 construiste clases con atributos y métodos. Pero hay una pregunta que no te hiciste: *¿y si alguien le pone `edad = -666` a mi objeto Persona?* Si todo es público, cualquiera puede hacerlo desde cualquier sitio. Y ahí es donde empieza esta unidad: en el momento en el que tu código necesita **puertas**.

🏠 El problema de la casa de cristal

Imagina que vives en una casa de cristal. Cualquiera puede verlo todo: tu ropa interior, tu colección de cromos, esa caja de galletas vacía que guardas "por si acaso". Incómodo, ¿verdad? Pues lo mismo pasa con tus objetos si todo es público:

```
Persona p = new Persona();
p.edad = -666;          // Edad negativa... Esto no tiene sentido.
p.saldo = 999999999;    // Y encima te multiplicas el saldo por cero.
```

Tu objeto queda hecho unos zorros. Nadie ha cometido un error grave: el problema es que **tu clase permitía hacerlo**. No pusiste candado. La visibilidad es exactamente eso: decidir qué partes de tu clase son de acceso público y cuáles están fuera de la vista de todo el mundo.

> ⚠️ **Advertencia:** un atributo `public` es como dejar la puerta de tu casa abierta con un cartel que pone "Pasen y toquen todo". Tarde o temprano, alguien lo hará.

🚪 Por qué importa la visibilidad

Vale, ¿y por qué no lo dejamos todo `public` y tan contentos? Por tres motivos que en el mundo laboral son oro puro:

1. **Control:** decides qué puede leerse, qué puede modificarse y qué ni se toca. Nada de edades negativas ni saldos imposibles.
2. **Seguridad:** nadie puede dejar tu objeto en un estado inconsistente (por ejemplo, una cuenta con saldo negativo).
3. **Mantenibilidad:** si cambias por dentro cómo se calcula algo, el código de fuera no se entera. Los cambios internos no rompen a tus usuarios.

En los próximos puntos verás las herramientas exactas (modificadores, getters y setters, `static`). Pero primero necesitas el concepto: **tu clase es una caja, y tú decides quién tiene la llave**.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** la visibilidad no es un capricho: es el contrato que tu clase firma con el resto del mundo. Antes de escribir `public` en un atributo, pregúntate: *¿de verdad quiero que cualquiera pueda cambiarlo?*

Ejercicio: el supermercado sin puertas

Aquí tienes una clase de la U09 escrita “a lo bruto”, con todo público:

```java
public class Carrito {
    public String cliente;
    public int total;          // en céntimos
    public int articulos;
}

public class Supermercado {
    public static void main(String[] args) {
        Carrito c = new Carrito();
        c.cliente = "Pepita";
        c.total = -5000;       // ¿Una compra negativa? ¡Claro!
        c.articulos = -3;      // -3 artículos, genial.
    }
}
```

Preguntas (sin ejecutar):

1. ¿Qué hay de raro en `c.total = -5000`?
2. ¿Qué daños colaterales tendría que un `main` de cualquier clase pudiera hacer esto en un programa de verdad?
3. ¿Quién tendría que impedirlo: el `main` o la propia clase `Carrito`?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué problema tiene que todos los atributos de una clase sean `public`?
2. ¿Qué ganas al esconder la implementación interna de tu clase?
3. ¿Quién es el responsable de que una clase no acepte valores imposibles?
4. ¿Por qué es importante la visibilidad para el mantenimiento del código?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. La **visibilidad** decide qué partes de tu clase puede ver y usar el resto del mundo.

2. Con todo `public`, cualquier código puede romper tus objetos con valores imposibles.
3. Esconder datos te da **control, seguridad y mantenibilidad**: los tres superpoderes del programador que no llora depurando.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> | --- | --- |
> | Visibilidad | Qué se ve de una clase desde fuera |
> | `public` | Visible para todos, absolutamente todos |
> | Atributo | La “caja” de datos de la clase (ej. `edad`) |
> | Estado inconsistente | Un objeto con valores imposibles (ej. `edad = -5`) |
> | Contrato público | Lo que tu clase promete al mundo exterior |

📚 Volver al índice de la unidad · **Anterior:** Índice de la unidad · **Siguiente:** 02 · public, private y protected

02 — public, private y protected

Los 4 niveles de visibilidad: de la valla publicitaria en Times Square al diario con candado 🔐

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 02 · public, private y protected

📬 La idea en una frase

> **Hay 4 niveles de visibilidad en Java — `public`, `protected`, package-private y `private` — y cada uno es como un tipo de cerradura: decide quién tiene la llave de cada miembro de tu clase.**

En el punto 1 viste el problema de la casa de cristal. Ahora toca el kit de cerraduras. Java te da exactamente **cuatro niveles** de visibilidad, de la valla publicitaria al diario con candado.

🗂️ Los 4 niveles: de la valla al diario

Modificador	Se ve desde	Es como…
`public`	Todo el mundo, absolutamente todo	Una valla publicitaria en Times Square
`protected`	Mismo paquete + subclases (hijos)	Los secretos de familia: lo saben tus primos y tus hijos
(nada) package-private	Mismo paquete (el vecindario)	El cotilleo del barrio
`private`	Solo la propia clase	Tu diario secreto con candado

El detalle importante: **si no pones nada, Java asume package-private**. No existe una palabra `default` para esto (esa palabra es para otra cosa), así que “sin modificador” = “vecindario”.

> 💡 **Detalle práctico:** cuanto más `private` seas, más seguro estás. La regla de oro del programador paranoico: *empieza con el acceso más restrictivo y ábrelo solo si hay motivo.*

📣 public: la valla publicitaria

Todo el mundo lo ve. Desde cualquier clase, cualquier paquete, cualquier rincón del universo Java. Es como poner tu número de teléfono en una pancarta:

```
public class VallaPublicitaria {
    public String mensaje;

    public void mostrar() {
        System.out.println(mensaje);
    }
}
```

Úsalo para lo que **quieres** que otros usen: métodos de servicio, la puerta de entrada de la clase, el `main`… No para tus atributos, a menos que te guste el caos.

🔒 private: el diario con candado

Solo la clase ve sus propios `private`. Ni su madre, ni su mejor amigo, ni el perro:

```
public class DiarioSecreto {
    private String contenido;

    public void escribir(String mensaje) {
        this.contenido = "Querido diario: " + mensaje;
    }

    public String leer() {
        return contenido;
    }
}
```

Desde fuera no puedes hacer `diario.contenido` directamente: el compilador te lo prohíbe. Solo puedes usar lo que la clase expone (`escribir()` y `leer()`).

> ⚠️ **Advertencia:** nunca, NUNCA, hagas un atributo `public` “porque es más rápido”. Es más rápido de escribir y el doble de lento de depurar: cuando alguien meta un valor imposible, te pasarás horas buscando quién lo cambió.

👨‍👩‍👧 protected: los secretos de familia

Es como las historias vergonzosas de la familia. Tus primos (mismo paquete) y tus hijos (subclases) pueden acceder. Pero un desconocido de otro paquete… no. En la U11 (herencia) lo exprimirás de verdad; por ahora, quédate con la idea: *protegido para la familia*.

```
package zoologico;

public class Animal {
    protected String nombre; // Lo ven el paquete y las subclases
}
```

🏘️ package-private: el cotilleo del barrio

Si no pones ningún modificador, lo ven las clases del **mismo paquete**. Es como el grupo de WhatsApp del vecindario: los del bloque lo saben todo; los de la otra ciudad, ni idea.

```
package barrio;

public class Casa {
    public String direccion;          // Lo sabe todo el mundo
    protected String telefono;        // Lo sabe la familia
    int numeroHabitaciones;           // Lo saben los vecinos (package-private)
    private String contrasenaWifi;    // SOLO YO
}
```

Desde el mismo paquete (`barrio`):

```
public class Vecino {
    public void espiar() {
        Casa c = new Casa();
        System.out.println(c.direccion);          // OK: public
        System.out.println(c.telefono);           // OK: protected (mismo paquete)
        System.out.println(c.numeroHabitaciones); // OK: package-private
        // System.out.println(c.contrasenaWifi);  // ERROR: private
    }
}
```

Desde otro paquete, siendo subclase:

```
package otraCiudad;
import barrio.Casa;

public class CasaHeredada extends Casa {
    public void espiar() {
        System.out.println(direccion);        // OK: public
        System.out.println(telefono);         // OK: protected (soy subclase)
        // System.out.println(numeroHabitaciones); // ERROR: package-private
        // System.out.println(contrasenaWifi);     // ERROR: private
    }
}
```

📝 **Nota:** los **métodos** también tienen visibilidad. Un método `private void calcularImpuesto()` solo se usa dentro de la clase; nadie fuera necesita saber cómo calculas los impuestos (ni tú mismo quieres saberlo).

⭐ Sé el Código, my friend...

🕶️ **Don Tip:** cuando tengas dudas de "¿quién ve esto?", hazte dos preguntas: *¿está la otra clase en el mismo paquete?* y *¿es una subclase?* Con esas dos respuestas, tienes la cerradura resuelta.

Ejercicio: el banco espía

Eres la clase `Banco`. Tienes estos miembros:

```
public class Banco {
    public String nombreBanco;
    protected String direccionSucursal;
    String listaClientes;
    private double saldoCaja;

    public void mostrarInfo() {
        System.out.println(nombreBanco);
        System.out.println(direccionSucursal);
        System.out.println(listaClientes);
        System.out.println(saldoCaja);
    }
}
```

Preguntas (sin ejecutar):

1. ¿Puede una clase `Sucursal` de otro paquete ver `listaClientes`?
2. ¿Y `direccionSucursal`?
3. ¿Puede el `main()` de una clase del mismo paquete ver `saldoCaja`?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuántos niveles de visibilidad tiene Java y cómo se llaman?
2. ¿Qué nivel asume Java si no escribes ningún modificador?
3. ¿Quién puede ver un miembro `protected`?
4. ¿Puede un método ser `private`? ¿Para qué sirve?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los **4 niveles** de visibilidad son `public` > `protected` > package-private > `private`.
2. `private` es lo más restrictivo (solo la clase) y `public` lo más abierto (todo el mundo).
3. **Sin modificador, Java asume package-private**, y la buena práctica es empezar privado y abrir solo lo necesario.

> 🦎 **Vocabulario rápido**

Término	Idea general
Modificador de acceso	Palabra que decide quién ve un miembro: `public`, `protected`, `private`
Package-private	Visibilidad “por defecto”: solo el mismo paquete
Paquete	Carpeta de clases (`package barrio;`)
Subclase	Clase que hereda de otra (`extends`)

Término	Idea general
Miembro	Un atributo o un método de la clase

📚 Volver al índice de la unidad · **Anterior:** 01 · Visibilidad: el arte de no enseñarlo todo · **Siguiente:** 03 · Encapsulación: el pilar de la POO

03 — Encapsulación: el pilar de la POO

Atributos privados, acceso controlado y el patrón JavaBeans: la receta para que tu clase nunca se rompa 📦

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 03 · Encapsulación: el pilar de la POO

📬 La idea en una frase

> **Encapsular es esconder los datos (atributos `private`) y ofrecer métodos controlados (`public`) para acceder a ellos. Es el pilar que sostiene toda la POO.**

Ya tienes la herramienta (`private`) y ahora toca la estrategia completa. Encapsulación = **privacidad + control**. No basta con esconder: hay que decidir cómo se entra y cómo se sale.

📦 La fórmula de la encapsulación

La receta tiene tres pasos:

1. Tus **atributos** son `private`. Nadie los toca directamente.
2. Controlas el acceso con **getters y setters** `public` (los verás en detalle en el punto 4).
3. Dentro de los setters y métodos, **validas** para que el objeto nunca tenga un estado imposible.

Ejemplo de vida (o de muerte), un banco que no quiere quebrar:

```java
public class CuentaBancaria {
    private double saldo;

    public void ingresar(double cantidad) {
        if (cantidad > 0) {
            this.saldo += cantidad;
        }
    }

    public void retirar(double cantidad) {
        if (cantidad > 0 && cantidad <= saldo) {
            this.saldo -= cantidad;
        } else {
            System.out.println("No tienes tanto dinero, amigo.");
        }
    }

    public double getSaldo() {
        return saldo;
    }
}
```

El saldo es `private`: **nadie** puede hacer `cuenta.saldo = -1000` desde fuera. La única manera de cambiarlo es pasar por `ingresar()` y `retirar()`, que son los porteros que validan.

> ⚠️ **Advertencia:** si haces los atributos `public`, estás renunciando a la encapsulación. Es como llevar la cartera abierta en el metro: tarde o temprano, alguien mete mano.

🎁 Las ventajas (o por qué no dormirás peor)

- **Control:** validas y filtras. Nada de edades negativas ni saldos inventados.
- **Mantenibilidad:** cambias la implementación interna y el código cliente ni se entera.
- **Seguridad:** nadie deja tu objeto en un estado inconsistente.
- **Bajo acoplamiento:** cada clase va a lo suyo. No se meten unas en los asuntos de otras.

> 💡 **Detalle práctico:** la encapsulación te permite cambiar cómo funciona un método por dentro (por ejemplo, que `getSaldo()` calcule en vez de guardar) sin que el resto del programa se rompa. Esa libertad vale oro en proyectos grandes.

🧬 La convención JavaBeans: el protocolo

JavaBeans es una convención (no obligatoria, pero sí sensata) que dice que una clase "bien educada":

1. Es **pública**.
2. Tiene **constructor sin argumentos**.
3. Sus **atributos son privados**.
4. Expone **getters y setters públicos**.
5. (Opcional) implementa `Serializable`.

No te dejes engañar por el nombre: no es una librería mágica, es un **estilo**. Muchas herramientas y frameworks (y exámenes) esperan que tus clases lo sigan. El punto 4 te enseña la convención de nombres exacta.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando una clase "deja pasar de todo", suele ser porque su autor confió demasiado en que los demás escribirían bien. Nunca confíes: *valida siempre en la frontera*.

Ejercicio: el cajero que pierde dinero

Esta clase pretende estar encapsulada, pero esconde un fallo gordo:

```
public class Hucha {
    private int cantidad;

    public void anadir(int monedas) {
        cantidad = cantidad + monedas;
    }

    public int getCantidad() {
        return cantidad;
    }
}
```

Preguntas (sin ejecutar):

1. ¿Qué pasa si alguien llama a `hucha.anadir(-50)`?

2. ¿Qué le faltaría a `anadir()` para estar bien blindada?
3. ¿Y si quisiéramos que nadie pudiera leer la cantidad directamente y solo mostrara un mensaje?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuáles son los 3 pasos de la encapsulación?
2. ¿Qué es el patrón JavaBeans y para qué sirve?
3. ¿Por qué `CuentaBancaria` no deja hacer `saldo = -1000`?
4. ¿Qué ganas en mantenibilidad al encapsular?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **Encapsular** = atributos `private` + métodos `public` controlados + validación.
2. Da **control, mantenibilidad, seguridad y bajo acoplamiento**.
3. El **patrón JavaBeans** es la convención estándar que seguirán tus clases a partir de ahora.

> 🦎 **Vocabulario rápido**

Término	Idea general
Encapsulación	Esconder datos y controlar el acceso a ellos
Estado	El conjunto de valores de los atributos de un objeto
JavaBeans	Convención: clase pública, atributos privados, getters/setters
Acoplamiento	Cuánto dependen unas clases de otras
Frontera	Los métodos públicos por donde entra y sale la información

📚 Volver al índice de la unidad · **Anterior:** 02 · public, private y protected · **Siguiente:** 04 · Getters y setters

04 — Getters y setters

El portero de la discoteca: leer y modificar atributos privados con nombre, orden y validación 🚪

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 04 · Getters y setters

📬 La idea en una frase

> **Un getter lee un atributo privado y un setter lo modifica, y ambos tienen nombres con una convención estricta. Son los porteros de tu discoteca: dejan entrar a quien quieren y echan a los que van borrachos.**

Los atributos son `private`, pero entonces… ¿cómo lee el mundo tu clase? ¿Cómo cambia la edad de un `Estudiante`? Ahí entran los **getters** (para leer) y los **setters** (para escribir). Y no, no valen nombres inventados: Java y sus frameworks esperan una convención.

🏷️ La convención de nombres (memorízala)

Tipo de atributo	Getter	Setter
`String nombre`	`getNombre()`	`setNombre(String n)`
`boolean activo`	`isActivo()`	`setActivo(boolean a)`
`int cantidad`	`getCantidad()`	`setCantidad(int c)`

Reglas de oro:

- El getter se llama `get` + atributo en mayúscula inicial. Con `boolean`, se usa `is` en vez de `get`.
- El setter se llama `set` + atributo en mayúscula inicial y recibe el valor como parámetro.
- Devuelven/reciben **exactamente el tipo del atributo**.

```java
public class Estudiante {
    private String nombre;
    private boolean matriculado;

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public boolean isMatriculado() {
        return matriculado;
    }

    public void setMatriculado(boolean matriculado) {
        this.matriculado = matriculado;
    }
}
```

> 📝 **Nota:** `this` (el punto 5 de la UO9) resuelve el lío de que parámetro y atributo se llamen igual. `this.nombre` es el atributo; `nombre` a secas es el parámetro. Sin `this`, te asignarías el parámetro a sí mismo y el atributo se quedaría con su valor por defecto.

🚪 El setter como portero: validar

Un setter no es solo “escribir”. Es la **frontera donde validas**. Ejemplo clásico:

```java
public class Termometro {
    private double temperatura;

    public double getTemperatura() {
        return temperatura;
    }

    public void setTemperatura(double temperatura) {
        if (temperatura < -273.15) {
            System.out.println("Imposible: eso está por debajo del cero absoluto.");
        } else {
            this.temperatura = temperatura;
        }
    }
}
```

Si alguien intenta `termometro.setTemperatura(-500)`, el setter lo frena: el termómetro nunca mostrará una temperatura imposible. Sin el setter, el atributo `public` habría aceptado ese -500 sin rechistar.

> ⚠️ **Advertencia:** un setter que no valida es un portero dormido: deja pasar a cualquiera. Si tu setter acepta edades negativas, en la práctica es como si el atributo fuera `public`.

🔒 Getter sin setter: atributos inmutables

¿Y si quieres que algo se lea pero **no se pueda cambiar**? Entonces pones getter y te ahorras el setter. Punto. Así consigues atributos de solo lectura:

```java
public class Tarjeta {
    private String numero;
    private int pin;

    public Tarjeta(String numero, int pin) {
        this.numero = numero;
        this.pin = pin;
    }

    public String getNumero() {
        return numero;
    }

    public int getPin() {
        return pin;
    }
    // Sin setters: el número y el pin se fijan al nacer y jamás cambian.
}
```

> 💡 **Detalle práctico:** los atributos que se fijan en el constructor y no deben cambiar (un DNI, un id, un pin) solo llevan getter. Es la forma elegante de decir “esto es de solo lectura”.

🤷 No hay preguntas tontas

> ❓ **¿Por qué tanto rollo con los nombres? ¿No puedo llamar a mis métodos `leerEdad()` y `ponerEdad()`?**

Puedes, y funcionarían. Pero la convención `getX()` / `setX()` es lo que esperan los frameworks (y, de paso, los examinadores). Cuando en la U15 veas cómo se serializan objetos a JSON, entenderás por qué llamarse de la forma estándar ahorra dolores de cabeza.

> ❓ **¿El setter siempre tiene que validar con `println`?**

No. Otra opción muy común es **lanzar una excepción** (`IllegalArgumentException`) cuando el valor es inválido, para que el programa que llama se entere del problema. Eso lo verás bien en la U04 con `try/catch` y en los boletines avanzados.

⭐ Sé el Código, my friend…

> 👓 **Don Tip:** piensa en el setter como un portero de discoteca: no deja entrar a los borrachos (valores inválidos). Un buen setter convierte el error de un usuario (o de tu futuro yo) en algo controlado, no en un objeto roto.

Ejercicio: la edad vigilada

Escribe tú (mentalmente, o en papel) una clase `Persona` con:

- Atributo privado `int edad`.
- Constructor que recibe `edad`.
- Getter `getEdad()`.
- Setter `setEdad(int edad)` que **rechace** edades negativas y mayores de 120.

¿Qué imprimiría este código?

```
Persona p = new Persona(25);
p.setEdad(-5);
System.out.println(p.getEdad());
```

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cómo se llama el getter de `boolean activo`?
2. ¿Para qué sirve validar en un setter?
3. ¿Cuándo usas un atributo solo con getter y sin setter?
4. ¿Qué hace `this` en `this.nombre = nombre`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **Getter** lee un atributo privado; **setter** lo modifica, con nombres `getX()`/`setX()` (y `isX()` para booleanos).

2. El setter es la **frontera de validación**: rechaza valores imposibles para que el objeto nunca se rompa.
3. **Sin setter** consigues atributos de solo lectura, perfectos para datos que no deben cambiar.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Getter | Método que devuelve el valor de un atributo (`getEdad()`) |
> | Setter | Método que modifica un atributo, validando (`setEdad(int)`) |
> | `is` | Prefijo del getter cuando el atributo es `boolean` |
> | Solo lectura | Atributo con getter pero sin setter |
> | `this` | Referencia al objeto actual, desambigua atributo vs parámetro |

📚 Volver al índice de la unidad · **Anterior:** 03 · Encapsulación: el pilar de la POO · **Siguiente:** 05 · Atributos static

05 — Atributos static

Lo que pertenece a la clase y no al objeto: el grupo de WhatsApp en vez de los mensajes privados 📢

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 05 · Atributos static

---

📬 La idea en una frase

> **Un atributo `static` es de la clase, no del objeto: solo hay una copia para todos, como el grupo de WhatsApp de la clase, mientras que los atributos normales son los mensajes privados de cada uno.**

Hasta ahora, cada objeto que creabas con `new` tenía sus propios atributos. Si creabas dos `Estudiante`, cada uno tenía su `nombre`. Pero hay datos que no tienen sentido "por objeto": el número total de estudiantes creados, por ejemplo. Eso no es de un estudiante concreto: es de **la clase**. Y para eso existe `static`.

📱 El grupo de WhatsApp vs los mensajes privados

Imagina que eres parte de una clase de 30 alumnos:

- **El grupo de WhatsApp de la clase** (`static`): todos ven el mismo mensaje. Si alguien escribe "mañana hay examen", los 30 lo leen. Es compartido.
- **Tus mensajes privados** (instancia): solo tú los ves. Cada alumno tiene los suyos y no se mezclan.

En Java es igual:

- **Variable de clase** (`static`): una sola copia compartida por todos los objetos.
- **Variable de instancia** (sin `static`): cada objeto tiene la suya, independiente.

```java
public class Estudiante {
    private static int totalEstudiantes = 0;
    private String nombre;
    private int id;

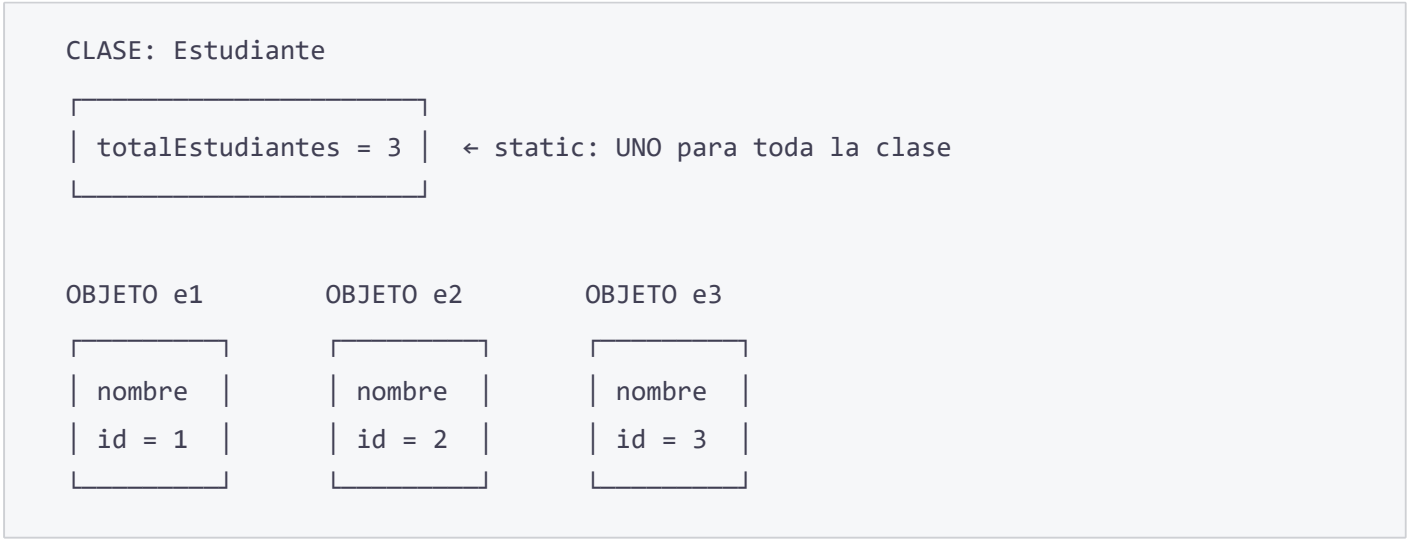
    public Estudiante(String nombre) {
        this.nombre = nombre;
        this.id = ++totalEstudiantes;
    }

    public static int getTotalEstudiantes() {
        return totalEstudiantes;
    }

    public int getId() {
        return id;
    }
}
```

Cada `Estudiante` tiene su propio `nombre` e `id`, pero todos comparten `totalEstudiantes`. Cuando creas el tercer estudiante, el contador pasa a 3 **para toda la clase**.

🧠 Cómo se ve en memoria



El contador vive en “la clase” (un espacio que existe una sola vez), mientras que cada objeto tiene su nombre y su id en su propio cajón.

📞 El uso clásico: contadores e ids

El ejemplo anterior es el patrón estrella de los atributos `static`: **contar objetos creados** y generar **ids autoincrementales**. Cada vez que el constructor se ejecuta, `totalEstudiantes` sube y ese valor se usa como id. Es como el mostrador de la panadería: hay un único número en la pared que va subiendo para todos los clientes.

```
Estudiante a = new Estudiante("Ana");
Estudiante b = new Estudiante("Luis");
Estudiante c = new Estudiante("Marta");

System.out.println(Estudiante.getTotalEstudiantes()); // 3
System.out.println(c.getId());                        // 3
```

> 💡 **Detalle práctico:** para acceder a un miembro `static`, usa el nombre de la **clase**: `Estudiante.getTotalEstudiantes()`. Funcionaría con un objeto (`c.getTotalEstudiantes()`), pero es confuso: da a entender que el valor es "de c", cuando es de todos.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando veas un `static`, pregúntate: *¿tiene sentido que esto sea diferente para cada objeto?* Si la respuesta es "no", entonces pertenece a la clase.

Ejercicio: los gatos estáticos

Mira este código y responde SIN EJECUTAR:

```java
public class Gato {
    public static int totalGatos = 0;
    public String nombre;

    public Gato(String nombre) {
        this.nombre = nombre;
        totalGatos++;
    }

    public static void decirTotal() {
        System.out.println("Hay " + totalGatos + " gatos");
        // System.out.println(nombre); // ¿Esto funciona?
    }
}
```

1. ¿Funciona `System.out.println(nombre);` dentro de `decirTotal()`?
2. Si creas 3 gatos y luego haces `Gato.decirTotal()`, ¿qué imprime?
3. ¿Y si creas 5 gatos más? ¿Qué imprime ahora?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuántas copias existe de un atributo `static`?
2. ¿Cómo se accede a un atributo estático desde fuera?
3. ¿Para qué sirve un contador `static` con un `id` autoincremental?
4. ¿Puede un método estático leer un atributo de instancia directamente?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Un atributo `static` pertenece a la **clase**: hay una sola copia para todos los objetos.
2. Se usa para datos compartidos: **contadores**, ids autoincrementales, configuraciones globales.

3. Accede siempre por el **nombre de la clase**, no por un objeto, para que el código quede claro.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `static` | Pertenece a la clase, no al objeto |
> | Variable de clase | Atributo `static`, una copia compartida |
> | Variable de instancia | Atributo normal, una copia por objeto |
> | Contador | Atributo `static` que se incrementa en cada `new` |
> | Id autoincremental | Número único asignado con el contador |

📚 Volver al índice de la unidad · **Anterior:** 04 · Getters y setters · **Siguiente:** 06 · Métodos static y el main

06 — Métodos static y el main

Llamar a métodos sin crear objetos, la clase Math, y por qué main es el primero en llegar a la fiesta 🎉

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 06 · Métodos static y el main

📬 La idea en una frase

> **Un método `static` se llama usando la clase, sin crear objetos; no tiene `this` y no puede tocar atributos de instancia. Y `main` es estático porque es el primero en llegar a una fiesta donde todavía no existe ningún objeto.**

Si los atributos `static` son el grupo de WhatsApp, los métodos `static` son el número de atención al cliente: no necesitas hablar con un empleado concreto, llamas al número general y listo.

📞 El teléfono de la clase

Un método estático se declara con `static` y se invoca usando el **nombre de la clase**:

```
public class UtilidadesMatematicas {
    public static int sumar(int a, int b) {
        return a + b;
    }

    public static double media(double a, double b) {
        return (a + b) / 2;
    }
}
```

```
int resultado = UtilidadesMatematicas.sumar(5, 3);  // Sin new, sin objeto.
double med = UtilidadesMatematicas.media(10, 20);
```

Compáralo con un método de instancia, que exige un objeto:

```
String texto = "Hola";
int longitud = texto.length();   // length() NO es static: necesito el objeto texto.
```

Esa es la gran diferencia práctica: **static → clase; sin static → objeto**.

🚫 Lo que un método estático NO puede hacer

Reglas de oro que salvan exámenes:

1. **No puede acceder a atributos de instancia** (no sabe de qué objeto habla).
2. **No tiene `this`** (no hay "yo" porque no hay objeto).
3. Solo puede llamar **directamente** a otros métodos estáticos.

```
public class Prueba {
    int x = 1;
    static int y = 1;

    public void incrementarX() { x++; }          // Método de instancia
    public static void incrementarY() { y++; }   // Método estático
}
```

`incrementarY()` puede tocar `y` (estático) pero no `x`: no sabe qué `x` coger. `incrementarX()` puede tocar `x` porque se llama sobre un objeto concreto.

> ⚠️ **Advertencia:** el clásico del novato es meter `System.out.println(nombreDeInstancia)` dentro de un método `static` y esperar que funcione. El compilador te dirá *"non-static variable cannot be referenced from a static context"*. Ahora ya sabes por qué.

🧮 La clase Math: la fábrica de métodos estáticos

`java.lang.Math` es la clase utilitaria por excelencia. TODOS sus métodos son estáticos. No puedes (ni quieres) hacer `new Math()`:

```
double max = Math.max(10, 20);        // 20.0
double raiz = Math.sqrt(25);          // 5.0
double potencia = Math.pow(2, 10);    // 1024.0
double absoluto = Math.abs(-7);       // 7.0
double random = Math.random();        // Aleatorio en [0.0, 1.0)
```

Lo usas desde el primer día del curso, y ahora entiendes **por qué** funciona sin `new`: son métodos estáticos. Y ya de paso, `Integer.parseInt("42")` también lo es.

> 📝 **Nota:** `Math` tiene el constructor **privado**. Nadie puede instanciarla. Es como una estatua: para admirarla, no para hacerle clones. Las clases así se llaman *utilitarias* y las montas tú en el punto 7.

🎉 Por qué `main` es static

`main` es el método que arranca todo:

```
public static void main(String[] args)
```

¿Por qué es estático? Porque cuando empieza el programa **todavía no existe ningún objeto**. Alguien tiene que arrancar la fiesta antes de que llegue nadie. Si `main` fuera de instancia, la JVM necesitaría un objeto para llamarlo… y para crear ese objeto necesitaría un programa en marcha. Pez que se muerde la cola. Por eso `main` es el primero en llegar y abre la puerta él mismo.

🤷 No hay preguntas tontas

> ❓ **¿Puedo llamar a un método estático desde un objeto? ¿Como `miObjeto.metodoEstatico()`?**

Técnicamente sí: Java te lo deja. Pero es como llamar a tu madre por el apellido: funciona, pero queda raro. La convención es usar la clase: `Clase.metodoEstatico()`. De hecho, algunos IDEs te muestran una warning por hacerlo.

> ❓ **¿Los métodos estáticos son más rápidos?**

Un pelín: no necesitan la referencia al objeto. Pero la diferencia es tan pequeña que en el 99,9% de los casos no la notarás. No te obsesiones con la velocidad: preocúpate de que tu código tenga sentido.

> ❓ **¿Puedo ponerle `static` a todo y ahorrarme crear objetos?**

Puedes, pero entonces no estás haciendo POO: estás haciendo "programación estática a lo bruto". Es como usar un destornillador para clavar un clavo: puedes, pero para eso existe el

martillo. Usa `static` para lo que es de la clase, no para todo.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** el truco para no liarte es preguntar siempre: *¿este método necesita saber de qué objeto habla?* Si la respuesta es no, puede ser `static`.

Ejercicio: la calculadora de la clase

¿Funciona este código? Si no, ¿por qué?

```
public class Calculadora {
    private String marca;

    public static int sumar(int a, int b) {
        System.out.println(marca);  // ¿Qué pasa aquí?
        return a + b;
    }
}
```

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cómo se llama a un método estático?
2. ¿Puede un método estático acceder a atributos de instancia?
3. ¿Por qué `main` es estático?
4. ¿Qué diferencia a `Math.sqrt(9)` de `texto.length()`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los métodos `static` se llaman con la **clase**, sin `new`, y no pueden tocar atributos de instancia ni usar `this`.

2. `Math` es la clase utilitaria por excelencia: todo estático, constructor privado.
3. `main` es estático porque **es el primero en llegar**: cuando arranca, no hay objetos todavía.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Método estático | Método de la clase: `Clase.metodo()` |
> | Método de instancia | Método del objeto: `objeto.metodo()` |
> | `this` | Referencia al objeto actual (no existe en estáticos) |
> | Clase utilitaria | Solo métodos estáticos, sin objetos |
> | Contexto estático | "El mundo de la clase": sin objetos a la vista |

📚 Volver al índice de la unidad · **Anterior:** 05 · Atributos static · **Siguiente:** 07 · Constantes de clase: static final

07 — Constantes de clase: static final

Valores inmutables en MAYÚSCULAS y las clases utilitarias que nadie puede instanciar 🏛️

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 07 · Constantes de clase: static final

---

📬 La idea en una frase

> **`static final` es una constante de clase: un valor que pertenece a la clase, no al objeto, y que nadie puede cambiar. Y si además le pones el constructor privado, tienes una clase utilitaria como `Math`.**

Recuerdas el `final` de la U03: una variable que no se puede cambiar. Ahora súmale `static` (de la clase, una sola copia) y tienes la fórmula perfecta para valores fijos de toda la aplicación: el IVA, el nombre de la app, el número máximo de usuarios…

🏷️ La constante de clase

Declaración canónica, con el nombre en **MAYÚSCULAS** y palabras separadas por _:

```
public class Config {
    public static final String NOMBRE_APP = "Gestión DAM";
    public static final String VERSION = "2.1.0";
    public static final int MAX_USUARIOS = 100;
    public static final double IVA = 0.21;
}
```

Se usa así, sin crear ningún objeto:

```
double precioConIva = precio * Config.IVA;
System.out.println("Bienvenido a " + Config.NOMBRE_APP);
```

¿Y si alguien intenta hacer `Config.IVA = 0.5`? **El compilador te lo prohíbe.** Es más firme que tus propósitos de Año Nuevo.

> 💡 **Detalle práctico:** junta las constantes en una clase dedicada (`Config`, `Constantes`, `Valores`) y tendrás un único sitio donde cambiar, por ejemplo, el IVA cuando toque. Ni un "número mágico" suelto por el código.

🔒 La regla de las MAYÚSCULAS

Es una **convención**, no una regla del lenguaje: las constantes se escriben en MAYÚSCULAS con `_`:

- `MAX_INTENTOS_LOGIN`
- `TIMEOUT_SEGUNDOS`
- `RUTA_LOG`

Cuando otro programador (o tú) vea `Config.MAX_USUARIOS` en un código, sabrá al instante que eso es una constante: no se toca, no se pregunta, no se negocia. El estilo dice mucho.

> ⚠️ **Advertencia:** ojo con la tentación de declarar todo `static final` porque "mola". Solo es constante lo que de verdad no va a cambiar. El día que el IVA suba, querrás cambiarlo en un sitio… y si es constante, lo cambias y ya.

🏛️ Las clases utilitarias: el "no necesito pareja" de Java

Una clase utilitaria es una clase que **solo tiene miembros estáticos**. Son como el amigo que está soltero y feliz: no necesita instanciarse para ser útil. La prueba: `Math`, que ya conoces.

¿Cómo evitamos que alguien haga `new StringUtils()`? Con un **constructor privado**:

```java
public class StringUtils {
    private StringUtils() {}  // Nadie puede hacer new StringUtils()

    public static boolean esVacio(String str) {
        return str == null || str.trim().isEmpty();
    }

    public static String invertir(String str) {
        return str == null ? null : new StringBuilder(str).reverse().toString();
    }

    public static String capitalizar(String str) {
        if (esVacio(str)) return str;
        return str.substring(0, 1).toUpperCase() + str.substring(1).toLowerCase();
    }
}
```

Desde fuera, todo es `StringUtils.invertir("hola")`. Intentar `new StringUtils()` da error de compilación: el constructor es privado, la clase "no necesita pareja".

> 📝 **Nota:** el constructor privado no es obligatorio para que funcione, pero es la **etiqueta** de una clase utilitaria bien hecha: le dice al mundo "no me instancies, soy pura utilidad".

⭐ Sé el Código, my friend…

> 👓 **Don Tip:** antes de escribir un número suelto (un 100, un 0,21), pregúntate si es un *número mágico* que merece una constante con nombre. Los números mágicos son la pesadilla de quien mantiene tu código.

Ejercicio: el validador del barrio

Escribe (mentalmente) una clase `Validador` que sea utilitaria: constructor privado y un método `public static boolean esPar(int numero)` que devuelva `true` si el número es par.

¿Cómo comprobarías, sin crear objetos, que 42 es par?

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué significa `static final` y cómo se nombra?
2. ¿Por qué `Math` no se puede instanciar?
3. ¿Qué es un número mágico y cómo lo evitas?
4. ¿Puede cambiar una constante después de declararla?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. `static final` crea una **constante de clase**: una sola copia, inmutable, en MAYÚSCULAS.
2. Las **clases utilitarias** (como `Math`) solo tienen miembros estáticos y usan **constructor privado** para que nadie las instancie.
3. Evita los **números mágicos** dándoles nombre y juntándolos en una clase `Config`.

> 🦎 **Vocabulario rápido**

Término	Idea general
Constante	`static final`: valor fijo de la clase
Número mágico	Valor literal sin nombre que debería ser constante
Clase utilitaria	Solo estáticos, constructor privado
Convención	Regla de estilo (MAYÚSCULAS) que todos siguen
Constructor privado	Impide crear objetos de la clase

📚 Volver al índice de la unidad · **Anterior:** 06 · Métodos static y el main · **Siguiente:** 08 · Be the Code: encapsula tu clase

08 — Be the Code: encapsula tu clase

El reto guiado: convierte una clase sin blindar en una bien encapsulada con getters y setters 🔨

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 08 · Be the Code: encapsula tu clase

---

📬 La idea en una frase

> **Aquí no aprendemos nada nuevo: ponemos en práctica todo lo de los puntos 1 al 7 convirtiendo, paso a paso, una clase que es una puerta abierta en una clase con candado, portero y alarma.**

Es la hora de ser tú el código. La U09 ya te enseñó a diseñar una clase; ahora toca el trabajo fino: **blindarla**. Cogemos una clase que viola todas las reglas y la transformamos en una clase profesional.

🏚️ El punto de partida: la clase del caos

Este es el código que "funciona" pero debería darte vergüenza ajena:

```
public class CuentaBancaria {
    public double saldo;
    public String titular;
    public int numeroIngresos;

    public CuentaBancaria(String titular, double saldoInicial) {
        saldo = saldoInicial;
        numeroIngresos = 0;
    }

    public void ingresar(double cantidad) {
        saldo = saldo + cantidad;
        numeroIngresos++;
    }
}
```

¿Qué falla? Todo: atributos `public`, sin validación, sin getters… Vamos a convertirlo en una clase que respira seguridad. **Paso a paso, sin saltarte ninguno.**

🧰 Paso 1: atributos privados

Regla número uno de la casa: los atributos van con `private`. Sin excepciones.

```
public class CuentaBancaria {
    private double saldo;
    private String titular;
    private int numeroIngresos;
    ...
}
```

Si alguien intenta `cuenta.saldo = 1000` desde fuera, el compilador le dice que no. Pero… ¿y si alguien legítimamente quiere saber el saldo o cambiar el titular? Eso lo resuelven los pasos 2 y 3.

🧰 Paso 2: getters para leer

Un getter devuelve el valor del atributo, siguiendo la convención `get` + nombre:

```
public double getSaldo() {
    return saldo;
}

public String getTitular() {
    return titular;
}

public int getNumeroIngresos() {
    return numeroIngresos;
}
```

Ahora el mundo puede *leer* el saldo, pero no *tocarlo*.

> 💡 **Detalle práctico:** fíjate en que `numeroIngresos` tiene getter pero no setter: nadie debe ponerlo a mano, solo `ingresar()` lo incrementa. Solo lectura, como debe ser.

🧰 Paso 3: setter para lo que se puede cambiar

El titular sí puede cambiar (un banco actualiza titulares, qué le vamos a hacer). Añadimos el setter con su validación:

```
public void setTitular(String titular) {
    if (titular != null && !titular.trim().isEmpty()) {
        this.titular = titular;
    } else {
        System.out.println("Titular inválido.");
    }
}
```

Fíjate en el orden del `if`: primero la condición de validez, y dentro del `else` el aviso. El `this` desambigua (parámetro vs atributo, del punto 5 de la U09).

🧰 Paso 4: validar también en los métodos

Un setter es un portero, pero los métodos `ingresar()` también tienen que serlo. Ahora mismo el código acepta ingresos negativos (¡un "ingreso" de -50 euros!). Lo arreglamos:

```
public void ingresar(double cantidad) {
    if (cantidad > 0) {
        saldo = saldo + cantidad;
        numeroIngresos++;
    } else {
        System.out.println("No se pueden ingresar cantidades negativas.");
    }
}
```

El estado de la cuenta ya no puede ser imposible: ni saldo negativo por la vía rápida ni ingresos fantasma.

🧰 Paso 5: el toque final, constantes

Podemos rematar con una constante de clase. Imaginemos que el banco solo permite un máximo de ingresos al día por la app:

```
public class CuentaBancaria {
    public static final int MAX_INGRESOS_DIA = 10;
    private double saldo;
    private String titular;
    private int numeroIngresos;
    ...
}
```

Y en `ingresar()` añadir la comprobación:

```
if (numeroIngresos >= MAX_INGRESOS_DIA) {
    System.out.println("Límite diario alcanzado.");
    return;
}
```

🏆 La clase final (compárala con la del caos)

```java
public class CuentaBancaria {
    public static final int MAX_INGRESOS_DIA = 10;

    private double saldo;
    private String titular;
    private int numeroIngresos;

    public CuentaBancaria(String titular, double saldoInicial) {
        setTitular(titular);
        saldo = saldoInicial;
        numeroIngresos = 0;
    }

    public double getSaldo() {
        return saldo;
    }

    public String getTitular() {
        return titular;
    }

    public int getNumeroIngresos() {
        return numeroIngresos;
    }

    public void setTitular(String titular) {
        if (titular != null && !titular.trim().isEmpty()) {
            this.titular = titular;
        } else {
            System.out.println("Titular inválido.");
        }
    }

    public void ingresar(double cantidad) {
        if (numeroIngresos >= MAX_INGRESOS_DIA) {
            System.out.println("Límite diario alcanzado.");
            return;
        }
        if (cantidad > 0) {
            saldo = saldo + cantidad;
            numeroIngresos++;
        } else {
```

```
            System.out.println("No se pueden ingresar cantidades negativas.");
        }
    }
}
```

Del caos a la fortaleza: `private` en todo, getters, setters con validación, métodos porteros y una constante. **Eso es encapsular.**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Por qué `numeroIngresos` no tiene setter?
2. ¿Qué hace `setTitular()` si le pasas un texto vacío?
3. ¿Cuántos pasos hizo falta seguir para blindar la clase?
4. ¿Para qué sirve la constante `MAX_INGRESOS_DIA`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Encapsular una clase es un **proceso en pasos**: `private`, getters, setters, validación y constantes.
2. Cada setter y cada método son una **frontera** que decide qué entra y qué sale del objeto.
3. Al final tienes una clase que **se protege sola**: el error del mundo exterior no puede romperla.

> 🐛 **Vocabulario rápido**

Término	Idea general
Refactorizar	Mejorar el código sin cambiar su comportamiento
Blindar	Convertir atributos `public` en `private` con control de acceso
Frontera	Método público que valida lo que entra
Solo lectura	Getter sin setter: se lee pero no se escribe

Término	Idea general
Número mágico	Literal sin nombre: se convierte en constante

📚 Volver al índice de la unidad · **Anterior:** 07 · Constantes de clase: static final · **Siguiente:** 09 · Repaso interactivo

09 — Repaso interactivo: blindalo todo

El cierre práctico de la unidad, con candados, estáticos y una clase que no va a querer compilar 😈

> 🗺️ **Estás en:** 🔒 **U10 · Visibilidad, encapsulación y static** → 09 · Repaso interactivo

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a querer compilar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```
public class Estudiante {
    public static int total = 0;
    private int id;

    public Estudiante() {
        id = ++total;
    }

    public int getId() {
        return id;
    }
}
```

```java
public class Misterio {
    public static void main(String[] args) {
        Estudiante a = new Estudiante();
        Estudiante b = new Estudiante();
        Estudiante c = new Estudiante();

        System.out.println(c.getId());
        System.out.println(Estudiante.total);
    }
}
```

¿Qué imprime por pantalla? Elige sabiamente:

1. **`3` y `3`** → ✅ ¡Correcto! Cada `new` incrementa el `static total`, y el tercer estudiante se lleva el `id = 3`. Además `total` (estático, de la clase) también vale 3: una sola copia para todos.
2. **`1` y `3`** → El `static` sería "de cada objeto", pero no: `total` es de la clase, una sola copia. ❌
3. **`3` y `0`** → El constructor sí incrementa `total`, no lo deja en 0. ❌

> 🔄 **Solución**

🔥 Fireside Chat: la clase y el objeto

> *Dos veteranos de la POO discuten junto a la máquina de café.*

Clase: — Yo soy el molde. Defino atributos, métodos, la estructura. Sin mí no hay nada que instanciar.

Objeto: — Y yo soy la galleta, el resultado real. Yo guardo los datos: mi `nombre`, mi `id`. Tú solo eres el plano.

Clase: — Planos, planos… ¿Y quién tiene el contador de cuántas galletas hay? ¡Yo! `static int total`. Eso es mío, y todos los objetos lo compartís.

Objeto: — Compartimos, sí, pero yo tengo mi `id` único. Mi `nombre`, mi saldo. No me toques mis atributos de instancia.

Clase: — Precisamente por eso los declaré `private`. Para que ni tú te los toques: solo a través de mis getters y setters. Yo soy el portero.

Objeto: — Vale, pero sin mí, tus setters no tienen a quién validar. Necesitas un objeto para que la fiesta exista.

Clase: — *suspira* Por eso el `main` es `static`: para que yo pueda arrancar la fiesta sin que llegue ningún objeto primero.

> La lección: la **clase** define la estructura y posee lo `static` (contadores, constantes); el **objeto** guarda la información de cada cosa (atributos de instancia). Se necesitan mutuamente, pero cada uno a su terreno.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy el nivel más restrictivo: ni mi madre, ni mi mejor amigo, ni el perro. Solo la clase me ve.**
2. **Soy el portero de la discoteca: dejo leer el atributo, pero con nombres estrictos.**
3. **Soy el número de atención al cliente: me llaman por la clase, sin crear ningún objeto.**
4. **Soy el grupo de WhatsApp de la clase: hay una sola copia y todos la compartimos.**
5. **Soy el valor que ni el tiempo se atreve a tocar: en MAYÚSCULAS y con _.**
6. **Soy la clase que nadie puede instanciar: solo estáticos y constructor privado.**

> ▶ 🔁 **Respuestas**

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. ¿Qué visibilidad tiene un atributo sin modificador?
   - a) `public` b) package-private (solo el paquete)
2. ¿Cómo se llama el getter de `boolean activo`?
   - a) `getActivo()` b) `isActivo()`
3. ¿Puede un método estático usar `this`?
   - a) Sí, siempre b) No: no hay objeto del que hablar
4. ¿Qué hace `Config.IVA = 0.5` si `IVA` es `static final`?
   - a) Cambia el valor b) Error de compilación

5. Si dos objetos comparten un atributo `static int x`, y uno lo incrementa…
    - a) Solo el objeto que lo incrementa lo ve b) Los dos lo ven cambiado

> 🔁 **Soluciones**

👹 CONRAD VS EL MUNDO: “¿Por qué todo público?”

> *CONRAD, nuestro compilador cascarrabias, opina sobre el clásico del novato.*

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: *CONRAD, mi programa funciona pero luego se rompe en el módulo de facturación*. Y yo: vale, ¿cómo están tus atributos? *Pues públicos, para ir más rápido.* ¡AY, MADRE MÍA! Un `public` en un atributo es una invitación a que cualquier clase le meta un valor imposible. Claro que funciona… hasta que deja de funcionar a las 3 de la madrugada.

Y luego está el que escribe `setEdad(int edad) { this.edad = edad; }` *sin validar nada. Un setter sin validar es un portero dormido: deja pasar a cualquiera, incluso con edades negativas.*

Y el colmo: un método `static` intentando leer `nombre` de instancia. ¡No sabes ni de qué objeto hablas! Te lo digo desde el punto 6: el `static` no tiene `this`.

La lección: antes de quejarte de bugs raros, revisa la frontera: ¿los atributos son `private`? ¿Los setters validan? ¿Lo estático está donde debe? El 90% de los estados imposibles se evitan con un buen candado en la clase.

⚡ Laboratorio de tortura: la caja fuerte sin candado

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia esta clase y haz que sea una clase “profesional”. Tiene **4 problemas** de visibilidad y diseño que la dejan como una puerta abierta.

```java
public class Mascota {
    public String nombre;
    public int edad;
    public static int totalMascotas = 0;

    public Mascota(String nombre, int edad) {
        nombre = nombre;
        edad = edad;
        totalMascotas++;
    }

    public void cumplirAnos() {
        edad = edad + 1;
    }
}
```

Fallo intencionado: uno de los problemas parece correcto a simple vista, pero hace que el `nombre` y la `edad` de la mascota se queden "en blanco" al nacer.

Tu tarea: convertirla en una clase encapsulada: atributos `private`, getters, setters con validación (la edad no puede ser negativa), y corrige el constructor. Además, prueba que `totalMascotas` sube con cada `new`.

Pistas para cuando te frustres (no antes):

1. ¿Los atributos deberían ser `public`? *no → es el primer problema.*
   > **¿Y si sigo atascado?**
2. ¿Qué pasa con `nombre = nombre;` y `edad = edad;`? *Suena a que se asignan a sí mismos.*
   > **¿Y si sigo atascado?**
3. ¿Qué debería validar el setter de la edad?
   > **¿Y si sigo atascado?**
4. ¿`totalMascotas` está bien como está? *Recuerda la convención de acceso.*
   > **Solución final**

```java
public class Mascota {
    public static int totalMascotas = 0;

    private String nombre;
    private int edad;

    public Mascota(String nombre, int edad) {
        this.nombre = nombre;
        setEdad(edad);
        totalMascotas++;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        if (nombre != null && !nombre.trim().isEmpty()) {
            this.nombre = nombre;
        }
    }

    public int getEdad() {
        return edad;
    }

    public void setEdad(int edad) {
        if (edad >= 0) {
            this.edad = edad;
        } else {
            System.out.println("Edad inválida: " + edad);
        }
    }

    public void cumplirAnos() {
        this.edad++;
    }
}
```

Los 4 problemas: atributos públicos, constructor sin `this` (los parámetros se asignaban a sí mismos), setter de edad sin validar y `cumplirAnos()` que podía acabar en edades imposibles. Con `private` + validación, la mascota ya no puede “nacer en blanco” ni tener edad negativa.

🧩 EL LÍO: el setter que no validaba

El siguiente código pretende encapsular una clase `CuentaBancaria`, pero tiene varios errores. Encuéntralos:

```
public class CuentaBancaria {
    public double saldo;
    private String titular;

    public CuentaBancaria(String titular, double saldoInicial) {
        titular = titular;
        saldo = saldoInicial;
    }

    public double getSaldo() {
        return saldo;
    }

    private void setSaldo(double saldo) {
        if (saldo >= 0) {
            this.saldo = saldo;
        }
    }
}
```

1. ¿El atributo `saldo` está bien encapsulado?
2. ¿Qué error hay en el constructor con `titular = titular`?
3. ¿Por qué `setSaldo` es `private`? ¿Cómo se modifica el saldo entonces?

> 🕶️ **Don Tip:** `this` resuelve ambigüedades. Si parámetro y atributo se llaman igual, sin `this` te asignas el parámetro a sí mismo y el atributo se queda "en blanco".

> › 🔄 **Soluciones**

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Candidato al Candado**	Explicar los 4 niveles de visibilidad y quién ve cada cosa, sin mirar
🏅 **Portero Estricto**	Escribir una clase con atributos `private`, getters y setters con validación
🏅 **El Contador de la Clase**	Crear una clase con contador `static` e ids autoincrementales que funcione
🏅 **El Adivino Estático**	Predecir "¿Qué imprime?" en un puzle de `static` vs instancia sin ejecutar
🏅 **El Refactorizador**	Convertir la clase del Laboratorio en una clase bien encapsulada sin mirar la solución

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```java
public class Contador {
    static int n = 0;
    int m = 0;

    public static void subirN() { n++; }
    public void subirM() { m++; }

    public static void main(String[] args) {
        Contador a = new Contador();
        Contador b = new Contador();

        a.subirM();
        b.subirM();
        a.subirM();
        Contador.subirN();
        Contador.subirN();

        System.out.println(a.m + " " + b.m + " " + Contador.n);
    }
}
```

2. **El atributo fantasma:** en `CuentaBancaria`, ¿cómo harías para que el saldo solo pudiera subir con ingresos y bajar con retiradas, sin que nadie pudiera ponerlo a un valor arbitrario?
3. **El detective:** tu programa acepta edades negativas porque "nadie sabe quién las introduce". ¿Qué herramienta de esta unidad usas para pararlo?
4. **Verdadero o falso:** "un atributo `static` tiene una copia por cada objeto creado".

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Modificador de acceso más restrictivo (7 letras)
4. La clase no lo tiene, el objeto sí, cuando hablamos de "yo" (4 letras)
6. Método que lee un atributo privado (6 letras)

Vertical:
2. Valor fijo de la clase, en MAYÚSCULAS (9 letras)
3. Lo que pertenece a la clase, no al objeto (6 letras)
5. Prefijo del getter cuando el atributo es booleano (2 letras)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, por qué los atributos no deberían ser públicos."**
2. **"¿Cuál es la diferencia entre un método estático y un método de instancia?"**
3. **"¿Qué es la encapsulación y qué ventajas aporta?"**
4. **"¿Por qué `main` es `static`?"**
5. **"¿Cuándo usarías `static final`?"**
6. **"Un compañero ha dejado `public` un atributo que guarda el estado de un juego y aparecen bugs imposibles. ¿Qué haces?"**

🤷 No hay preguntas tontas

> ❓ **¿Package-private y `default` son lo mismo?**

Sí: se llaman igual. Es el nivel que Java asume cuando no escribes `public`, `private` ni `protected`. No confundas con la palabra `default`, que es para otra cosa.

> ❓ **¿Un setter siempre tiene que imprimir un mensaje cuando el valor es inválido?**

No. Otra opción muy profesional es **lanzar una excepción** (`IllegalArgumentException`), para que el código que llama se entere y decida. Imprimir es cómodo para practicar; lanzar excepciones es lo que verás en código de producción.

> ❓ **¿Puedo tener un atributo estático que sea un objeto de su propia clase?**

¡Sí! Es el patrón **Singleton**: un `private static MiClase instancia` y un método `getInstance()`. Es como tener una única piedra filosofal. Pero eso es otro tema… que algún día merecerá su propia unidad.

🎬 Poscréditos

El programador termina su clase `CuentaBancaria` blindada: atributos `private`, getters, setters con validación y un contador estático que cuenta las cuentas creadas. Se siente un fortín andante.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — ¿Eso era una clase o una caja fuerte del Banco de España? `private`, validación, constantes… Hasta constructor privado en una clase utilitaria que te has inventado. No está mal para un aprendiz de candados.

Novato: — ¿Y ahora qué? Mis clases ya no se rompen aunque alguien intente romperlas.

CONRAD: — *toma un sorbo* Blindar está bien, pero un banco con una sola cuenta es un banco triste. Los objetos se relacionan entre sí: heredan, comparten, se especializan. Eso es otra historia. La próxima parada te la cuenta.

El novato guarda su proyecto, cierra el IDE y siente que sus objetos ya no son casas de cristal.

PRÓXIMAMENTE EN U11: Herencia, polimorfismo e interfaces. El momento en el que tus clases **se miran a los ojos y se dicen "mira, esto lo compartimos"**. 🧬

📚 Volver al índice de la unidad · **Anterior:** 08 · Be the Code: encapsula tu clase · **Siguiente:** U11 · Herencia, polimorfismo e interfaces

U11 — Herencia, polimorfismo e interfaces

Cuando tus clases heredan, se disfrazan y firman contratos: extends, super, override, abstract e interfaces 🧬

Hasta ahora tus clases eran islas: cada una con sus atributos, sus métodos y sus secretos bien guardados. Pero en la vida real las cosas no son islas: un perro *es un* animal, un coche *es un* vehículo, y un profesor *es un* empleado. Las cosas **heredan** características de sus padres y, encima, añaden las suyas. Pues tu código hace exactamente lo mismo.

Esta unidad es el corazón de la POO. Y tiene tres grandes actos:

- **Herencia (`extends`):** una clase hija obtiene todo lo que tiene su clase padre y puede mejorarlo. Aprenderás a usar `super` para llamar a la superclase, `@Override` para sobrescribir métodos y `protected` para compartir secretos solo con la familia.
- **Polimorfismo:** el mismo método, mil comportamientos. Una variable de tipo `Animal` puede apuntar a un `Perro`, un `Gato` o una `Vaca`, y cada uno hace su sonido. Verás `instanceof`, downcasting y la mítica clase `Object`, el tatarabuelo de todo.
- **Abstracción e interfaces:** clases abstractas que no se pueden instanciar (bocetos) e interfaces que son contratos. Aquí se decide si tu código es un caos o una obra de arte.

En el camino descubrirás por qué `toString()` te está mintiendo si no lo sobrescribes, qué pasa cuando haces un downcasting a lo loco (spoiler: `ClassCastException`), y por qué los entrevistadores de trabajo adoran preguntar "¿cuándo usarías una clase abstracta y cuándo una interfaz?".

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- **Heredar** de una clase con `extends` y distinguir IS-A de HAS-A.
- Usar **`super`** para llamar a constructores y métodos de la superclase.
- **Sobrescribir** métodos con `@Override` y saber cuándo (y cuándo no) hacerlo.
- Proteger miembros con **`protected`** y diseñar **jerarquías de clases** sensatas.

- Explicar el **polimorfismo dinámico** (dynamic binding) y usarlo con colecciones y parámetros.
- Aplicar **`instanceof` y downcasting** con seguridad.
- Sobrescribir **`toString()`, `equals()` y `hashCode()`** de la clase `Object`.
- Diseñar **clases abstractas** con métodos abstractos y el patrón template method.
- Implementar **interfaces**, usar métodos `default` y decidir cuándo usar `abstract class` vs `interface`.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · Herencia: cuando tus hijos siguen tus pasos	`extends`, qué se hereda y qué no, IS-A vs HAS-A, el árbol genealógico	Todos
02 · super y @Override	Llamar a la superclase, sobrescribir métodos y encadenar constructores	Todos
03 · protected y jerarquías de clases	El miembro que solo ve la familia, árboles de clases y la clase base frágil	Todos
04 · Polimorfismo: el camaleón de la POO	Dynamic binding, referencias polimórficas, colecciones y parámetros	Todos
05 · instanceof y downcasting	Preguntar quién eres y bajar la referencia sin romper nada	Todos
06 · La clase Object: el tatarabuelo	`toString()`, `equals()` y `hashCode()` bien sobrescritos	Todos
07 · Clases abstractas	Bocetos que no se instancian, métodos abstractos y template method	Todos
08 · Interfaces: el contrato que firmas	`implements`, polimorfismo, métodos `default` y abstract vs interface	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA4 y RA7)

RA4: Desarrolla programas organizados en clases analizando y aplicando los principios de la programación orientada a objetos.

CE	Criterio	Dónde se cubre
RA4 a)	Se ha reconocido la sintaxis, estructura y componentes típicos de una clase.	✅ Punto 1
RA4 b)	Se han definido clases.	✅ Puntos 7 y 8
RA4 d)	Se han creado constructores.	✅ Puntos 2 y 7
RA4 e)	Se han desarrollado programas que instancien y utilicen objetos de las clases creadas anteriormente.	✅ Puntos 4, 8 y 9
RA4 g)	Se han definido y utilizado clases heredadas.	✅ Puntos 1, 2 y 3
RA4 i)	Se han definido y utilizado interfaces.	✅ Punto 8

RA7: Desarrolla programas, aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.

CE	Criterio	Dónde se cubre
RA7 a)	Se han identificado los conceptos de herencia, superclase y subclase.	✅ Punto 1
RA7 b)	Se han utilizado modificadores para bloquear y forzar la herencia de clases y métodos.	✅ Puntos 2 y 7
RA7 c)	Se ha reconocido la incidencia de los constructores en la herencia.	✅ Punto 2
RA7 d)	Se han creado clases heredadas que sobrescriban la implementación de métodos de la superclase.	✅ Puntos 2 y 4
RA7 e)	Se han diseñado y aplicado jerarquías de clases.	✅ Puntos 3 y 7
RA7 f)	Se han probado y depurado las jerarquías de clases.	✅ Punto 9
RA7 g)	Se han realizado programas que implementen y utilicen jerarquías de clases.	✅ Puntos 4, 8 y 9

> 📌 El resultado de aprendizaje **RA4** se apoya en la U09 (definir clases) y la U10 (visibilidad), y culmina aquí con herencia e interfaces. La **RA7** se cierra con esta unidad: jerarquías, sobrescritura y contratos. Las colecciones (RA6) llegan en la **U12**.

🚪 ¿Por dónde empiezo?

- ¿Cero conocimientos? → Arranca en el punto 1. Solo necesitas lo que viste en las unidades 06 y 07.
- ¿Ya heredas y quieres el contrato? → Ve directo al punto 7 y al punto 8: ahí está la mitad abstracta de la unidad.
- ¿Solo vienes a por el polimorfismo? → Salta al punto 4 y al punto 5.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Herencia: cuando tus hijos siguen tus pasos

⏭️ **Al acabar la unidad, continúa en U12 · Colecciones.**

01 — Herencia: cuando tus hijos siguen tus pasos

extends, qué se hereda y qué no, IS-A vs HAS-A y el árbol genealógico de tus clases 🧬

> 🗺️ **Estás en:** 🧬 **U11 · Herencia, polimorfismo e interfaces** → 01 · Herencia: cuando tus hijos siguen tus pasos

---

📬 La idea en una frase

> **La herencia permite que una clase hija obtenga los miembros de una clase padre con `extends`, reutilizando código y añadiendo o mejorando lo que haga falta.**

¿Recuerdas cuando heredaste la nariz de tu abuela o el genio para enfadar a los profesores? Pues en Java pasa lo mismo, pero con menos drama y más código reutilizable.

🧬 ¿Qué es la herencia?

La herencia es el mecanismo por el cual una clase *hija* (subclase) obtiene todos los miembros de una clase *padre* (superclase). Y puede hacer dos cosas más: **añadir** los suyos propios y **mejorar** los existentes.

```java
public class Animal {
    protected String nombre;
    protected int edad;

    public void hacerSonido() {
        System.out.println("Algún sonido genérico...");
    }
}

public class Perro extends Animal {
    public void hacerSonido() {
        System.out.println("¡Guau guau!");
    }

    public void moverCola() {
        System.out.println("*mueve la cola felizmente*");
    }
}
```

`Perro` ahora tiene `nombre`, `edad`, `hacerSonido()` (mejorado) y `moverCola()`. Cortesía de la herencia.

> 💡 **Consejo:** código reutilizado, neuronas ahorradas. La herencia existe para que NO tengas que copiar y pegar el mismo código en 15 clases.

🔑 extends: "Soy como tú, pero con mejoras"

La palabra clave es `extends` (lo viste en la U10 por encima, ahora lo dominas):

```java
public class Coche extends Vehiculo {
    // Coche tiene TODO lo de Vehiculo + lo suyo
}
```

Detrás de cada `extends` hay una pregunta: **¿realmente es-un?** Si la respuesta es sí, adelante. Si es "tiene-un"… eso es composición, y la verás en el punto 3.

¿Qué se hereda y qué no?

No todo viaja de padres a hijos. Esta tabla te lo aclara:

Miembro	¿Lo hereda la hija?
`public`	✅ Sí
`protected`	✅ Sí
package-private (sin modificador)	✅ Sí, si está en el mismo paquete
`private`	❌ No (pero existe dentro del objeto)
Constructores	❌ No (se invocan con `super()`, punto 2)
Métodos `final`	❌ No se pueden sobrescribir

> ⚠️ **Advertencia:** los miembros `private` no se heredan, pero siguen ahí, dentro del objeto. La hija no puede tocarlos directamente, pero el padre los usa. Es como el dinero en la cuenta de tus padres: existe, pero no lo tocas.

⚖️ La regla de oro: IS-A vs HAS-A

- **IS-A** (es-un): relación de herencia. `Perro` IS-A `Animal`.
- **HAS-A** (tiene-un): relación de composición. `Coche` HAS-A `Motor`.

```
public class Coche extends Vehiculo { }   // IS-A ✓
public class Coche { private Motor m; }   // HAS-A ✓
```

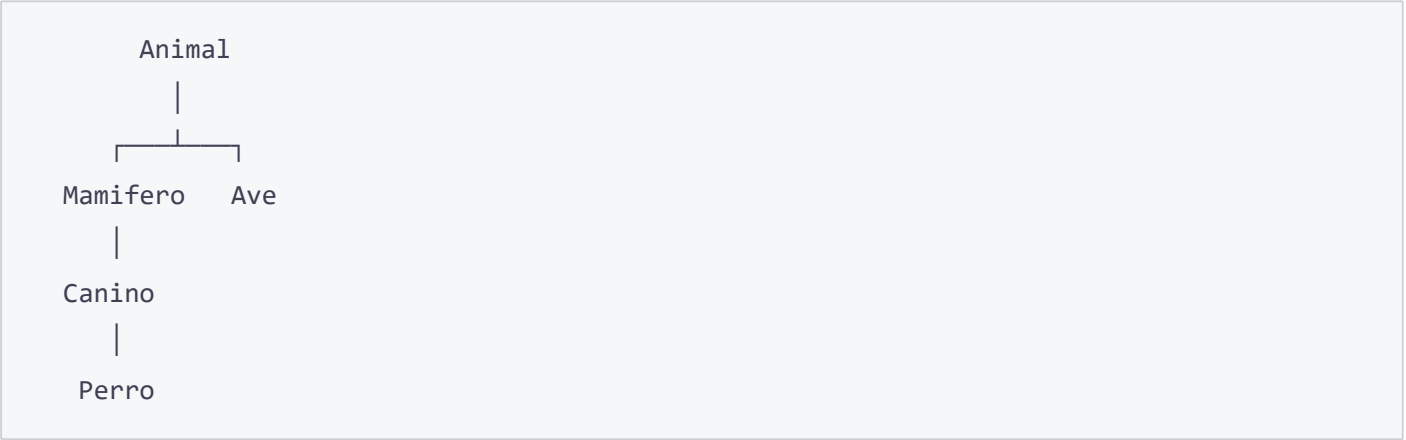
> 🎮 **¿Quién quiere ser millonario? — Edición Java:** ¿Cuál es la relación correcta? a) `Cliente extends Persona` b) `Cliente has-a Persona` c) `Coche extends Rueda`
>
> **Respuesta:** la a. `Cliente` IS-A `Persona`. `Coche` NO es una rueda: tiene ruedas (HAS-A). Si dudaste con la c, repasa tu vocabulario de coches.

🌳 Jerarquía de clases: el árbol genealógico

Una clase puede heredar de otra que a su vez hereda de otra. Así se forman las jerarquías:

```java
public class Animal { }
public class Mamifero extends Animal { }
public class Canino extends Mamifero { }
public class Perro extends Canino { }
```

`Perro` hereda de `Canino`, que hereda de `Mamifero`, que hereda de `Animal`. Cada generación aporta lo suyo:



> 📝 **Nota:** cuanto más arriba en el árbol, más general es la clase; cuanto más abajo, más específica. `Animal` no sabe ladrar; `Perro` no sabe volar. Cada uno en su nivel.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando una clase hereda, se lleva TODO lo accesible. Pregúntate siempre qué ve la hija antes de escribir `extends`.

Ejercicio: el examen de visibilidad

```java
public class Animal {
    private String idSecreto = "X-123";
    protected String nombre = "Animal";
    public int edad = 5;
}

public class Perro extends Animal {
    public void mostrar() {
        System.out.println(idSecreto); // ¿compila?
        System.out.println(nombre);    // ¿compila?
        System.out.println(edad);      // ¿compila?
    }
}
```

Preguntas (sin ejecutar):

1. ¿Cuál de las tres líneas compila?
2. ¿Qué pasaría si `idSecreto` fuese `protected` en vez de `private`?

> 🔄 Solución

Ejercicio: ¿qué imprime?

```java
class Abuelo { void decir() { System.out.println("Abuelo"); } }
class Padre extends Abuelo { void decir() { System.out.println("Padre"); } }
class Hijo extends Padre { void decir() { System.out.println("Hijo"); } }

public class Test {
    public static void main(String[] args) {
        new Hijo().decir();
    }
}
```

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué palabra clave usas para que `Perro` herede de `Animal`?

2. ¿Se heredan los atributos `private`? ¿Y los `protected`?
3. ¿`Coche extends Rueda` es buena idea? ¿Por qué?
4. ¿Qué significa que una jerarquía de clases es un "árbol genealógico"?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. **La herencia** con `extends` permite que una subclase reutilice y mejore los miembros de su superclase.
2. Solo se hereda lo **accesible** (`public`, `protected` y package-private); los `private` y los constructores no.
3. Antes de heredar, pregunta: **¿es-un?** Si es "tiene-un", usa composición.

> 🦎 **Vocabulario rápido**

Término	Idea general
Superclase	La clase padre, la general
Subclase	La clase hija, la específica
`extends`	La palabra que establece la herencia
IS-A	Relación de herencia (Perro es-un Animal)
HAS-A	Relación de composición (Coche tiene-un Motor)
Jerarquía	El árbol de clases de general a específico

📚 Volver al índice de la unidad · **Anterior:** — · **Siguiente:** 02 · super y @Override

02 — super y @Override

Llamar a la superclase, sobrescribir métodos y encadenar constructores sin llorar 🧬

> 🗺️ **Estás en:** 🧬 **U11 · Herencia, polimorfismo e interfaces** → 02 · super y @Override

📬 La idea en una frase

> **`super` llama a la superclase (constructores y métodos) y `@Override` le dice al compilador que estás sobrescribiendo, no inventando.**

En el punto 1 viste que la hija "hereda". Pero heredar no es suficiente: a veces quieres que tu padre haga primero su parte y luego tú la tuya. Para eso existen `super` y `@Override`.

📞 super: llamando a mamá/papá para que te ayuden

`super` tiene dos usos, y conviene tenerlos claros:

1. `super(...)` para invocar el constructor del padre

```
public class Vehiculo {
    private String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
    }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(String marca, int puertas) {
        super(marca);          // ← configura la parte de Vehiculo
        this.puertas = puertas; // ← y luego lo tuyo
    }
}
```

Si el padre tiene un constructor con parámetros, **debes** llamarlo con `super(...)` y debe ser **la primera línea** del constructor hijo. Si no lo haces, el compilador intenta llamar a `super()` (sin argumentos)… que no existe. Error de compilación en el acto.

> ⚠️ **Advertencia:** `super(...)` solo sirve para invocar constructores y métodos de la superclase. No puedes pasarlo como parámetro ni asignarlo a una variable. Es un puntero "solo lectura" hacia arriba.

2. `super.metodo()` para llamar al método del padre

A veces queremos **extender** el método del padre, no reemplazarlo:

```
public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        super.hacerSonido();   // primero lo que hace Animal
        System.out.println("¡MIAU!");  // y luego lo mío
    }
}
```

`super` es como gritar "¡MAAAAMÁ!" en el supermercado: "ejecuta la versión de mi padre, y después yo hago lo mío".

✍️ @Override: "Papá, yo lo hago mejor"

`@Override` es una anotación que le dice al compilador: "asegúrate de que realmente estoy sobrescribiendo un método del padre":

```
public class Pez extends Animal {
    @Override
    public void hacerSonido() { }  // ✓ existe en Animal

    @Override
    public void nadar() { }        // ✗ ERROR: no existe en Animal
}
```

La segunda línea no compila: el compilador verifica que `nadar()` exista en `Animal` y, como no existe, te avisa. Ese aviso a tiempo vale oro: si escribes mal el nombre o la firma de un método,

`@Override` te lo descubre sin tener que esperar a un bug raro en runtime.

> 📝 **Nota:** `@Override` no es obligatorio, pero ponlo siempre. Es como el cinturón de seguridad: no pasa nada si no lo pones… hasta que pasa. Además, es la forma de decirle a quien lea tu código "esto es un override, no un método nuevo".

Reglas de oro del override

1. La firma debe ser **idéntica** (nombre y parámetros).
2. El tipo de retorno debe ser el mismo o un subtipo (covariante).
3. No puedes reducir la visibilidad: `public` en el padre no puede volverse `protected` en la hija.
4. Los métodos `final` no se pueden sobrescribir; las clases `final` no pueden tener hijas.

🔒 final: cuando quieres bloquear la herencia

`final` es el freno de mano:

```
public final class String { }        // no puede tener hijas

public class Vehiculo {
    public final void frenar() { }   // las hijas NO pueden sobrescribirlo
}
```

Úsalo cuando una clase o un método no debe modificarse. Por ejemplo, un método que forma parte del esqueleto de un algoritmo (lo verás en el template method del punto 7).

🏗️ Constructores en la herencia: la cadena de montaje

Cuando creas un objeto de la clase más profunda, se ejecutan **todos** los constructores de la cadena, del más general al más específico:

```java
public class Abuelo {
    public Abuelo() { System.out.println("Abuelo"); }
}
public class Padre extends Abuelo {
    public Padre() { super(); System.out.println("Padre"); }
}
public class Hijo extends Padre {
    public Hijo() { super(); System.out.println("Hijo"); }
}
// new Hijo() imprime: Abuelo / Padre / Hijo
```

Si no escribes `super()`, Java lo añade automáticamente al principio del constructor. Por eso la regla es: **el padre se construye primero, siempre**. Es como montar una casa: primero los cimientos, luego las paredes, luego el tejado.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** si el padre tiene constructor con parámetros, el hijo está obligado a llamarlo con `super(...)`. Si no, el compilador intenta el vacío... que no existe.

El ejercicio clásico: ¿qué impide compilar esto?

```java
public class Vehiculo {
    private String marca;

    public Vehiculo(String marca) {
        this.marca = marca;
    }

    public void arrancar() {
        System.out.println("Vehículo arrancado");
    }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(String marca, int puertas) {
        this.puertas = puertas;
    }

    @Override
    public void arrancar() {
        System.out.println("Coche arrancado con " + puertas + " puertas");
    }
}
```

Pregunta: ¿qué error impide que `Coche` compile?

> **Solución**

Ejercicio: ¿qué imprime con herencia encadenada y super?

```java
class Vehiculo {
    void describir() { System.out.println("Soy un vehículo"); }
}
class Coche extends Vehiculo {
    void describir() { System.out.println("Soy un coche"); }
    void describirCompleto() { super.describir(); this.describir(); }
}
class Deportivo extends Coche {
    void describir() { System.out.println("Soy un coche deportivo"); }
}

public class Test {
    public static void main(String[] args) {
        Deportivo d = new Deportivo();
        d.describirCompleto();
        d.describir();
    }
}
```

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuáles son los dos usos de `super`?
2. ¿Qué le pides a `@Override`?
3. ¿Qué ocurre si el hijo no llama a `super(...)` y el padre no tiene constructor vacío?
4. ¿Puede una hija sobrescribir un método `final` del padre?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **`super`** encadena con la superclase: constructores y métodos, siempre como primera jugada.

2. `@Override` es tu red de seguridad: el compilador comprueba que el método sobrescrito existe.
3. Los **constructores** se ejecutan de arriba a abajo (padre primero) y los métodos `final` son intocables.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `super(...)` | Llamada al constructor de la superclase |
> | `super.metodo()` | Llamada a un método de la superclase |
> | `@Override` | Anotación que verifica la sobrescritura |
> | Sobrescribir | Redefinir un método heredado con otra implementación |
> | `final` | Bloquea la herencia de una clase o la sobrescritura de un método |
> | Cadena de constructores | El padre se construye siempre antes que el hijo |

📚 Volver al índice de la unidad · **Anterior:** 01 · Herencia: cuando tus hijos siguen tus pasos · **Siguiente:** 03 · protected y jerarquías de clases

03 — protected y jerarquías de clases

El miembro que solo ve la familia, árboles de clases y el peligro de la clase base frágil 🧬

> 🗺️ **Estás en:** 🧬 **U11 · Herencia, polimorfismo e interfaces** → 03 · protected y jerarquías de clases

---

📬 La idea en una frase

> **`protected` es la visibilidad de la familia: el miembro lo ven las subclases (y el propio paquete). Y las jerarquías de clases bien diseñadas son árboles, no bosques enmarañados.**

En la U10 viste los cuatro niveles de visibilidad. Aquí descubres el matiz que solo cobra sentido con herencia: `protected`.

🔐 protected: el secreto que solo la familia ve

Visibilidad	Misma clase	Mismo paquete	Subclases (cualquier paquete)	Todos
`private`	✅	❌	❌	❌
package-private	✅	✅	❌	❌
`protected`	✅	✅	✅	❌
`public`	✅	✅	✅	✅

`protected` es el punto dulce: ni privado ni público de todo el mundo. Es la puerta con **llave para la familia**:

```
public class Animal {
    private String secreto;    // Solo Animal
    protected String familia;  // Animal, sus subclases y su paquete
    public String nombre;      // Todos
}
```

> 💡 **Consejo práctico:** cuando una clase base necesita que las hijas lean un atributo o llamen a un método, decláralo `protected`. Cuando las hijas no deben tocarlo, `private`. Y si nadie debe heredar de tu clase… `final`.

🌳 Diseñando jerarquías sensatas

Una jerarquía de clases es un árbol. Las reglas de un árbol sano:

1. **La raíz es general, las hojas son específicas.** `Animal` en la cima, `Perro` en la base.
2. **Cada nivel añade algo.** No crees una clase solo por tener una clase más.
3. **Poca profundidad.** 2 o 3 niveles son fáciles de mantener; 7 niveles son 7 niveles de dolor.
4. **Todo hereda de `Object`** (lo verás en el punto 6), aunque no lo escribas.

```
public class Empleado {
    protected String nombre;
    protected double salarioBase;

    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    public void mostrarInfo() {
        System.out.println(nombre + " cobra " + salarioBase + " €");
    }
}

public class Gerente extends Empleado {
    private double bono;

    public Gerente(String nombre, double salarioBase, double bono) {
        super(nombre, salarioBase);
        this.bono = bono;
    }

    public void mostrarInfo() {
        super.mostrarInfo();
        System.out.println("  y tiene un bono de " + bono + " €");
    }
}
```

Fíjate en el patrón: el `Gerente` usa `super(...)` para construir la parte de `Empleado` y `super.mostrarInfo()` para extender la salida. Ese es el diseño limpio de una jerarquía: cada clase se encarga de lo suyo.

💥 El problema de la clase base frágil

Aquí va la advertencia seria. Imagina que tienes un `Jarrón` con un método `romper()` y un `JarrónChino` que lo sobrescribe para hacer un ritual de rotura. Todo va bien… hasta que alguien añade un método `caerAlSuelo()` a `Jarrón` que internamente llama a `romper()`:

```
public class Jarrón {
    public void romper() { System.out.println("¡Crac! Jarrón roto"); }

    public void caerAlSuelo() {
        System.out.println("¡Se cae!");
        romper();   // ¡sorpresa: puede llamar al override de JarrónChino!
    }
}
```

Ahora `JarrónChino` se comporta de forma inesperada cada vez que cae al suelo, sin que nadie tocara su código. Este es el **problema de la clase base frágil** (fragile base class problem): cambiar la clase padre puede romper el comportamiento de las hijas.

> ⚠️ **Advertencia:** la herencia crea acoplamiento. Cuando cambias `Jarrón`, TODAS sus hijas cambian contigo, quieras o no. Es un superpoder con efectos secundarios.

> ❓ **¿Entonces la herencia es mala?**
>
> ¡No! Es una herramienta. Bien usada es perfecta para relaciones IS-A claras. El problema es usarla cuando una simple composición bastaría. La regla moderna: **prefiere composición sobre herencia**, y usa herencia solo para "es-un" reales.

🧩 Composición: "tener" en vez de "ser"

```
// HERENCIA: un coche ES UN vehículo
public class Coche extends Vehiculo { }

// COMPOSICIÓN: un coche TIENE UN motor
public class Coche {
    private Motor motor;
    private Rueda[] ruedas;
}
```

¿Por qué preferir composición a veces?

1. **Menos acoplamiento:** cambiar la clase de la parte no rompe a la que la contiene.
2. **Más flexible:** puedes cambiar las partes en tiempo de ejecución.
3. **Evitas jerarquías profundas:** 5 niveles de herencia = 5 niveles de dolor.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** `protected` no es "público para los amigos": es una visibilidad formal que el compilador controla. Úsala con intención, no por vaguería.

Ejercicio: el árbol que no compila

```
public class SerVivo {
    protected int energia;

    public SerVivo(int energia) {
        this.energia = energia;
    }
}

public class Animal extends SerVivo {
    protected String nombre;

    public Animal(String nombre, int energia) {
        super(energia);
        this.nombre = nombre;
    }
}

public class Perro extends Animal {
    public Perro(String nombre, int energia) {
        super(nombre, energia);
    }

    public void ladrar() {
        System.out.println(nombre + " ladra con energía " + energia);
    }
}
```

Pregunta: ¿por qué `Perro` puede usar `nombre` y `energia` aunque no las declare? ¿Y qué pasaría si `energia` fuese `private`?

> › 🔄 Solución

Ejercicio: el jarrón que cae

Con el código de `Jarrón` y `JarrónChino` de arriba, ¿qué imprimiría si haces `new JarrónChino().caerAlSuelo()` sabiendo que `JarrónChino` sobrescribe `romper()` para gritar "¡LAMENTO MILENARIO!"?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Quién ve un miembro `protected`?
2. ¿Cuántos niveles de profundidad sugiere el curso para una jerarquía?
3. ¿Qué es el problema de la clase base frágil?
4. ¿Cuándo usarías composición en vez de herencia?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **`protected`** es la visibilidad familiar: subclases y paquete lo ven; el mundo, no.
2. Las **jerarquías sanas** son poco profundas, cada nivel aporta algo y la raíz es general.
3. La herencia **acopla**; el problema de la clase base frágil avisa de que cambiar el padre tiene consecuencias. Composición sobre herencia cuando haya duda.

> 🦎 **Vocabulario rápido**

Término	Idea general
`protected`	Visible para la clase, el paquete y las subclases
Jerarquía de clases	Árbol de herencia de lo general a lo específico
Clase base frágil	Cambios en la superclase que rompen subclases sin avisar
Acoplamiento	Cuánto dependen unas clases de otras
Composición	"Tener" (HAS-A) en vez de "ser" (IS-A)

📚 Volver al índice de la unidad · **Anterior:** 02 · super y @Override · **Siguiente:** 04 · Polimorfismo: el camaleón de la POO

04 — Polimorfismo: el camaleón de la POO

Dynamic binding, referencias polimórficas, colecciones y parámetros que cambian de forma 🧬

> 🗺️ **Estás en:** 🧬 **U11 · Herencia, polimorfismo e interfaces** → 04 · Polimorfismo: el camaleón de la POO

📬 La idea en una frase

> **El polimorfismo hace que un mismo método se comporte de forma diferente según el objeto real que lo invoca, decidiéndolo la JVM en tiempo de ejecución.**

Polimorfismo (*poly* = muchas, *morphé* = formas): un mismo método se comporta diferente según el objeto que lo invoque. Como un control remoto universal donde el mismo botón "PLAY" funciona en Netflix, Spotify y tu tostadora.

🎭 El mismo método, diferentes comportamientos

```
public class Animal {
    public void hacerSonido() { System.out.println("..."); }
}

public class Perro extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Guau!"); }
}

public class Gato extends Animal {
    @Override
    public void hacerSonido() { System.out.println("¡Miau!"); }
}
```

Y ahora la magia:

```
Animal a;
a = new Perro(); a.hacerSonido(); // ¡Guau!
a = new Gato();  a.hacerSonido(); // ¡Miau!
```

La variable `a` es de tipo `Animal`, pero apunta a objetos diferentes. Se ejecuta siempre el método del **objeto real**, no el del tipo de la variable.

🔮 Dynamic binding: la decisión en runtime

El compilador y la JVM hacen trabajos distintos:

> **Compilador:** "¿`Animal` tiene `hacerSonido()`? Sí. Adelante." **JVM (runtime):** "El objeto es un `Perro`. Ejecuto el de `Perro`."

Esto es **dynamic binding** (o late binding): el *qué* se puede llamar lo decide el compilador; el *cómo* se ejecuta lo decide la JVM cuando el programa corre. Esa separación es la que hace posible el polimorfismo.

> 💡 **Detalle práctico:** si Java resolviese los métodos en compilación (static binding), llamar a `a.hacerSonido()` siempre ejecutaría la versión de `Animal`. El polimorfismo existe porque la decisión se aplaza hasta el momento justo.

🔗 Referencias polimórficas

Puedes guardar un objeto específico en una variable de tipo más general:

```
Animal miMascota = new Perro();  // ✓
Animal tuMascota = new Gato();   // ✓
```

Pero la variable solo "ve" los métodos del tipo de la referencia:

```
Animal a = new Perro();
a.hacerSonido();   // ✓
a.moverCola();     // X Animal no tiene moverCola()
```

> ⚠️ **Advertencia:** el compilador es conservador: si la variable es `Animal`, solo deja llamar a métodos de `Animal`, aunque por dentro haya un `Perro`. Para acceder a `moverCola()` necesitas el punto 5 (downcasting).

📚 Polimorfismo con colecciones

Aquí es donde el polimorfismo paga el alquiler:

```
import java.util.ArrayList;

ArrayList<Animal> animales = new ArrayList<>();
animales.add(new Perro());
animales.add(new Gato());
animales.add(new Vaca());

for (Animal a : animales) {
    a.hacerSonido();  // cada uno el suyo
}
// ¡Guau! / ¡Miau! / ¡Muuu!
```

Una sola lista, un solo bucle. Sin polimorfismo necesitarías tres listas separadas (y una bicicleta para llevarlas).

👨‍⚕️ Polimorfismo con parámetros

Un método que acepta el tipo general funciona con cualquier subclase:

```
public class Veterinario {
    public void vacunar(Animal a) {
        System.out.print("Vacunando a: ");
        a.hacerSonido();
    }
}

Veterinario vet = new Veterinario();
vet.vacunar(new Perro()); // Vacunando a: ¡Guau!
vet.vacunar(new Gato());  // Vacunando a: ¡Miau!
```

Y puedes ir más allá: una clase que gestiona una colección polimórfica:

```
public class Zoologico {
    private ArrayList<Animal> animales = new ArrayList<>();

    public void agregarAnimal(Animal a) {
        animales.add(a);
    }

    public void hacerDesfile() {
        for (Animal a : animales) {
            System.out.print(a.getClass().getSimpleName() + " dice: ");
            a.hacerSonido();
        }
    }
}
```

Añadir un `Gato` o un `Perro` no requiere cambiar **una sola línea** del `Zoológico`. Eso es diseño a prueba de futuro: el código se abre a nuevas clases sin tocarse.

🚫 Los atributos NO son polimórficos

Punto importante para el examen: la magia solo funciona con **métodos**. Los atributos se resuelven por el tipo de la **referencia**, no del objeto:

```
class A { int x = 1; }
class B extends A { int x = 2; }

A ref = new B();
System.out.println(ref.x);   // 1, NO 2
```

`ref.x` devuelve `1` porque los atributos no participan en el dynamic binding. Solo los métodos. Los atributos "ocultan" (shadowing), no sobrescriben.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** el compilador mira el tipo de la variable; la JVM mira el tipo real del objeto. Ahí está la magia.

Ejercicio: ¿qué imprime?

```java
class A { void saluda() { System.out.println("Hola desde A"); } }
class B extends A { void saluda() { System.out.println("Hola desde B"); } }
class C extends B { void saluda() { System.out.println("Hola desde C"); } }

public class Test {
    public static void main(String[] args) {
        A ref1 = new B();
        A ref2 = new C();
        B ref3 = new C();
        ref1.saluda();
        ref2.saluda();
        ref3.saluda();
    }
}
```

> 🔄 Solución

Ejercicio: ¿compila?

```java
class Vehiculo { void mover() { System.out.println("Vehículo se mueve"); } }

class Coche extends Vehiculo {
    void mover() { System.out.println("Coche acelera"); }
    void abrirPuertas() { System.out.println("Puertas abiertas"); }
}

public class Test {
    public static void main(String[] args) {
        Vehiculo v = new Coche();
        v.mover();
        // v.abrirPuertas();   <- ¿compila?
    }
}
```

> 🔄 Solución

Ejercicio: ¿qué imprime o?

```java
interface Volable { void volar(); }
class Pajaro implements Volable {
    public void volar() { System.out.println("Vuela"); }
}
class Aguila extends Pajaro {
    public void volar() { System.out.println("Vuela alto"); }
}

public class Test {
    public static void main(String[] args) {
        Volable v = new Aguila();
        Pajaro p = new Aguila();
        Object o = new Aguila();
        v.volar();
        p.volar();
        // o.volar() no compila: Object no tiene volar()
    }
}
```

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué es el dynamic binding?
2. ¿`Animal a = new Perro();` — qué métodos puedes llamar con `a`?
3. ¿Por qué no necesitas cambiar el `Zoológico` al añadir un animal nuevo?
4. ¿Son polimórficos los atributos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **Polimorfismo** = un método, muchas formas: la JVM elige la implementación del objeto real.

2. Las **referencias polimórficas** (`Animal a = new Perro()`) permiten colecciones y parámetros flexibles.
3. Los **métodos** son polimórficos; los **atributos**, no.

> 🐛 **Vocabulario rápido**

Término	Idea general
Polimorfismo	Un mismo método se comporta según el objeto real
Dynamic binding	Decisión del método en tiempo de ejecución
Referencia polimórfica	Variable de tipo general apuntando a un objeto específico
Override	La herramienta que cambia el comportamiento
Shadowing	Ocultar un atributo de la superclase (los atributos no son polimórficos)

📚 Volver al índice de la unidad · **Anterior:** 03 · protected y jerarquías de clases · **Siguiente:** 05 · instanceof y downcasting

05 — instanceof y downcasting

Preguntar quién eres realmente y bajar la referencia con cuidado para no estrellarte 🧬

> 🗺️ **Estás en:** 🧬 **U11 · Herencia, polimorfismo e interfaces** → 05 · instanceof y downcasting

📬 La idea en una frase

> **`instanceof` pregunta "¿eres realmente esto?" y el downcasting `(Tipo)` baja la referencia; juntos son seguros, por separado… peligrosos.**

En el punto 4 viste que una referencia `Animal` no puede llamar a `moverCola()` de un `Perro`. Para llegar a esos métodos específicos necesitas bajar la referencia. Y para bajar sin estrellarte, necesitas preguntar antes.

🕵️ instanceof: "¿Quién eres realmente?"

`instanceof` devuelve `true` si el objeto es de ese tipo (o de una subclase):

```
Animal a = new Perro();
if (a instanceof Perro) {
    System.out.println("¡Es un perro!");
}
```

> 💡 **Consejo:** úsalo con moderación. Si llenas tu código de `instanceof`, algo estás haciendo mal: el polimorfismo (punto 4) debería resolver la mayoría de los casos sin preguntar. `instanceof` es para cuando un objeto concreto necesita un trato que el polimorfismo no cubre.

📉 Downcasting: bajar la referencia

Subir (upcasting) es gratis e implícito: `Animal a = new Perro();`. Bajar (downcasting) es explícito y arriesgado:

```
Animal a = new Perro();
Perro p = (Perro) a;   // downcasting
p.moverCola();         // ✓ funciona
```

Pero si te equivocas de tipo:

```
Animal a2 = new Gato();
Perro p2 = (Perro) a2;  // 💥 ClassCastException en runtime
```

Java no se deja engañar: el objeto real es un `Gato`, y convertirlo a `Perro` explota en tiempo de ejecución. Es como intentar hacer hablar a un gato en un concurso de perros: el público (la JVM) lo ve y monta un escándalo.

> ⚠️ **Advertencia:** el downcasting **compila siempre** (el compilador no sabe qué hay dentro de la variable), pero puede fallar en runtime. La única red de seguridad es preguntar antes.

🛡️ El combo seguro: instanceof + downcasting

La receta que nunca falla:

```
if (a instanceof Perro) {
    Perro p = (Perro) a;
    p.moverCola();
} else {
    System.out.println("No es un perro, no puedo hacerlo mover la cola.");
}
```

`instanceof` es tu mirilla: si es un repartidor de pizzas, abre; si es un león, mejor no.

Ejemplo real: downcasting en una lista polimórfica

```java
import java.util.ArrayList;

ArrayList<Empleado> plantilla = new ArrayList<>();
plantilla.add(new Programador("Ana"));
plantilla.add(new Disenador("Luis"));
plantilla.add(new Programador("Eva"));

for (Empleado e : plantilla) {
    if (e instanceof Programador) {
        ((Programador) e).escribirCodigo();
    } else if (e instanceof Disenador) {
        ((Disenador) e).disenar();
    }
}
```

Cada empleado recibe su trato especial solo si realmente es de ese tipo. Sin `instanceof`, no podrías llamar a `escribirCodigo()` ni a `disenar()`.

✨ Bonus: pattern matching (Java 16+)

Desde Java 16 hay una versión más limpia de `instanceof` + casting en una sola línea:

```java
if (a instanceof Perro p) {
    p.moverCola();   // ya no hace falta el (Perro)
}
```

El compilador crea la variable `p` automáticamente dentro del bloque. Menos código, mismo resultado, cero `ClassCastException` posible.

> 📝 **Nota:** en tu IDE de clase, si usas una versión de Java anterior a la 16, quédate con la forma clásica. El patrón no cambia el concepto: preguntar antes de bajar.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** el downcasting sin `instanceof` es jugar a la ruleta rusa con tu programa. Siempre pregunta antes de bajar.

Ejercicio: ¿qué ocurre aquí?

```java
Animal a = new Perro();
if (a instanceof Gato) {
    System.out.println("Es un gato");
} else {
    System.out.println("No es un gato");
}
```

> 🔄 Solución

Ejercicio: el juez de la granja

```java
ArrayList<Animal> granja = new ArrayList<>();
granja.add(new Perro());
granja.add(new Gato());
granja.add(new Vaca());

for (Animal a : granja) {
    if (a instanceof Perro) {
        System.out.println("El perro mueve la cola");
    } else if (a instanceof Gato) {
        System.out.println("El gato ronronea");
    } else {
        System.out.println("Alguien da leche");
    }
}
```

¿Cuántos `instanceof` son necesarios? ¿Podrías haberlo hecho sin ninguno?

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué devuelve `instanceof` si el objeto no es de ese tipo?
2. ¿Qué excepción lanza un downcasting mal hecho?
3. ¿Por qué el combo `instanceof` + casting es seguro?
4. ¿Cuándo NO deberías usar `instanceof`?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. **`instanceof`** comprueba el tipo real del objeto antes de actuar.
2. El **downcasting** `(Tipo) ref` baja la referencia y solo es seguro tras un `instanceof`.
3. Usa este combo **con moderación**: el polimorfismo suele ser la solución más limpia.

> 🐛 **Vocabulario rápido**

Término	Idea general
`instanceof`	Operador que comprueba el tipo real de un objeto
Downcasting	Convertir una referencia a un tipo más específico
Upcasting	Convertir (implícito) a un tipo más general
`ClassCastException`	Explosión en runtime por un cast imposible
Pattern matching	`instanceof Perro p` (Java 16+): preguntar y bajar en un paso

📚 Volver al índice de la unidad · **Anterior:** 04 · Polimorfismo: el camaleón de la POO · **Siguiente:** 06 · La clase Object: el tatarabuelo

06 — La clase Object: el tatarabuelo

toString, equals y hashCode: los tres métodos que toda clase hereda y casi nadie sobrescribe bien 🧬

> 🗺️ **Estás en:** 🧬 **U11 · Herencia, polimorfismo e interfaces** → 06 · La clase Object: el tatarabuelo

---

📬 La idea en una frase

> **Toda clase hereda de `Object`, y sus tres métodos estrella (`toString()`, `equals()` y `hashCode()`) hay que sobrescribirlos para que tu clase se comporte con sentido común.**

¿Sabías que tu clase `Perro` tiene un bisabuelo? Todas las clases heredan de `Object`, aunque no escribas `extends Object`. Es la raíz de todo el árbol.

👴 Object: la raíz de todo

```
public class MiClase { } // = public class MiClase extends Object { }
```

Métodos que toda clase hereda:

Método	¿Qué hace?	¿Sobrescribirlo?
`toString()`	Representación textual del objeto	Casi siempre
`equals(Object)`	Compara por valor	Cuando tenga sentido
`hashCode()`	Código hash para colecciones	Con `equals`
`getClass()`	Clase real del objeto	No
`clone()`	Copia (superficial)	Raramente

> 💡 **Consejo:** los cuatro primeros los usas a diario. `clone()` mejor olvídalo: usa un constructor de copia (lo viste en la U09).

📇 toString(): la tarjeta de presentación

Por defecto, `Object.toString()` devuelve algo como `Perro@1a2b3c` (la clase y una dirección de memoria). Inútil para el ser humano medio. Sobrescríbelo:

```
public class Perro {
    private String nombre;
    private int edad;

    public Perro(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public String toString() {
        return "Perro{nombre='" + nombre + "', edad=" + edad + "}";
    }
}

System.out.println(new Perro("Firulais", 3));
// Perro{nombre='Firulais', edad=3}
```

Fíjate: `System.out.println(objeto)` llama a `toString()` automáticamente. Y también la concatenación `"Perro: " + perro`. Es el método que Java llama en mil sitios sin que lo pidas.

⚖️ equals(): ¿mismo objeto o iguales?

Por defecto, `equals()` compara **referencias** (es decir, hace `==`). Dos objetos distintos con los mismos datos NO son iguales. Para comparar por **valor**, sobrescríbelo:

```java
import java.util.Objects;

public class Perro {
    private String nombre;
    private int edad;

    // constructor...

    @Override
    public boolean equals(Object o) {
        if (this == o) return true;                    // 1. mismo objeto
        if (o == null || getClass() != o.getClass())   // 2. null o clase distinta
            return false;
        Perro perro = (Perro) o;                        // 3. downcasting seguro
        return edad == perro.edad                       // 4. comparar campos
                && Objects.equals(nombre, perro.nombre);
    }
}
```

El patrón tiene 4 pasos que son el examen casi literal de cualquier entrevista:

1. **Reflexividad rápida:** si es el mismo objeto (`this == o`), son iguales.
2. **Filtro:** `null` o clase distinta → `false`. (`getClass()` evita comparar un `Perro` con un `Gato`.)
3. **Downcasting** (seguro, gracias al paso 2).
4. **Comparar campo a campo**, usando `Objects.equals` para los `String` (que maneja bien el `null`).

🏷️ hashCode(): el código de barras

El `hashCode()` es un número que Java usa para guardar objetos en colecciones como `HashSet` o `HashMap`. La regla sagrada:

> ⚠️ **Advertencia:** si dos objetos son iguales según `equals()`, **DEBEN** tener el mismo `hashCode()`. Si sobrescribes `equals()` sin `hashCode()`, las colecciones hash se comportan de forma impredecible (objetos "iguales" que aparecen dos veces).

La forma moderna y sin dolor:

```java
@Override
public int hashCode() {
    return Objects.hash(nombre, edad);
}
```

`Objects.hash` usa los mismos campos que `equals()`. Regla práctica: **mismos campos en `equals` y en `hashCode`**, y vives en paz.

> 💡 **Detalle práctico:** la regla inversa NO es obligatoria: objetos con el mismo `hashCode()` pueden no ser iguales (colisiones permitidas). Solo importa una dirección: `equals == true` → mismo hash.

🕵️ getClass(): el nombre del objeto real

`getClass()` devuelve un objeto `Class` con información de la clase real del objeto (el que ves en `a.getClass().getSimpleName()` del punto 4). No se sobrescribe: viene de fábrica y es perfecto tal cual.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** `toString()`, `equals()` y `hashCode()` son el trío que tu IDE puede generar con un clic (Generate → equals() and hashCode()). Pero entiende qué generas antes de pulsar.

Ejercicio: ¿qué imprime?

```java
Empleado e1 = new Programador("Ana", "001", 2500, "Java");
Empleado e2 = new Programador("Ana", "001", 2500, "Java");
Empleado e3 = new Gerente("Ana", "002", 3000, 500);

System.out.println(e1);
System.out.println(e1.equals(e2));
System.out.println(e1.equals(e3));
System.out.println(e1.hashCode() == e2.hashCode());
```

Sabiendo que `Programador` sobrescribe `toString()`, `equals()` (por id) y `hashCode()` correctamente, y `Gerente` es otra clase:

> › 🔄 **Solución**

Ejercicio: el gato impostor

```
class Perro extends Animal { Perro(String n) { super(n); } }
class Gato extends Animal { Gato(String n) { super(n); } }

System.out.println(new Perro("Firulais").equals(new Gato("Firulais")));
```

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿De qué clase hereda todo en Java?
2. ¿Qué imprime `System.out.println(objeto)` si sobrescribes `toString()`?
3. ¿Qué pasa si sobrescribes `equals()` pero no `hashCode()`?
4. ¿Cuántos pasos tiene el patrón estándar de `equals()`?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. **`Object`** es el tatarabuelo de todas las clases y aporta `toString()`, `equals()`, `hashCode()` y `getClass()`.
2. **`toString()`** te da representaciones legibles; **`equals()`** compara por valor siguiendo el patrón de 4 pasos.
3. **`equals()` y `hashCode()`** van siempre juntos, usando los mismos campos, o las colecciones se vengan de ti.

> 🦎 Vocabulario rápido

Término	Idea general
`Object`	La clase raíz de toda la jerarquía Java
`toString()`	Representación textual del objeto

Término	Idea general
`equals(Object)`	Comparación por valor (no por referencia)
`hashCode()`	Código hash usado por las colecciones
`getClass()`	Información de la clase real del objeto
Contrato equals/hashCode	Iguales según `equals` ⇒ mismo `hashCode`

📚 Volver al índice de la unidad · **Anterior:** 05 · instanceof y downcasting · **Siguiente:** 07 · Clases abstractas

07 — Clases abstractas

Bocetos que no se instancian, métodos abstractos que obligan y el patrón template method 💉

> 🗺️ **Estás en:** 💉 **U11 · Herencia, polimorfismo e interfaces** → 07 · Clases abstractas

📬 La idea en una frase

> **Una clase abstracta es un boceto: no se puede instanciar, define métodos obligatorios (abstractos) y comparte código concreto con sus hijas.**

Imagina "A la venta: boceto de silla". No puedes sentarte en un boceto, ¿verdad? Pues eso son las clases abstractas: planos incompletos para que *otros* los completen.

🎨 ¿Qué es una clase abstracta?

Una clase que **NO puede instanciarse**. Puede tener métodos abstractos (sin implementación) y métodos concretos (con implementación):

```
public abstract class Animal {
    protected String nombre;

    public abstract void hacerSonido();   // sin llaves: obliga a las hijas

    public void dormir() {                // con llaves: código compartido
        System.out.println(nombre + " está durmiendo... Zzz");
    }
}
```

> ⚠️ **Advertencia:** `new Animal()` es un **error de compilación**. No puedes crear un "animal genérico"; solo puedes crear animales concretos que completen el boceto.

📋 You MUST: implementar los métodos abstractos

Si una clase **concreta** extiende una abstracta, está OBLIGADA a implementar todos sus métodos abstractos:

```
public class Perro extends Animal {
    @Override
    public void hacerSonido() {
        System.out.println("¡Guau!");
    }
}

public abstract class Pajaro extends Animal {
     // No está obligada: Pajaro también es abstracta y puede dejar el método sin implementar
}
```

La regla es la ley de los contratos: **o implementas todo, o te declaras abstracto tú también**. No hay término medio.

abstract vs concrete

Clase abstracta	Clase concreta
No puedes crear objetos directamente	Puedes crear objetos
Puede tener métodos abstractos	Todos implementados
Concepto general	Algo específico
`abstract class`	Solo `class`

📐 Ejemplo: figuras geométricas

```java
public abstract class Figura {
    protected String color;

    public Figura(String color) {
        this.color = color;
    }

    public abstract double calcularArea();
    public abstract double calcularPerimetro();

    public void mostrarColor() {
        System.out.println("Color: " + color);
    }
}

public class Circulo extends Figura {
    private double radio;

    public Circulo(String color, double radio) {
        super(color);
        this.radio = radio;
    }

    @Override public double calcularArea() { return Math.PI * radio * radio; }
    @Override public double calcularPerimetro() { return 2 * Math.PI * radio; }
}

public class Rectangulo extends Figura {
    private double ancho, alto;

    public Rectangulo(String color, double ancho, double alto) {
        super(color);
        this.ancho = ancho;
        this.alto = alto;
    }

    @Override public double calcularArea() { return ancho * alto; }
    @Override public double calcularPerimetro() { return 2 * (ancho + alto); }
}
```

`Figura` define **qué** tienen todas las figuras (color, área, perímetro) y delega **cómo** se calculan a cada una. El polimorfismo del punto 4 hace el resto: `ArrayList<Figura>` con círculos y

rectángulos, y `calcularArea()` funciona solo.

🏗️ Constructores en clases abstractas

Sí, las abstractas pueden tener constructores. Se llaman con `super()`, igual que en cualquier herencia:

```
public abstract class Animal {
    protected String nombre;

    public Animal(String nombre) {
        this.nombre = nombre;
        System.out.println("Constructor de Animal");
    }
}

public class Perro extends Animal {
    public Perro(String nombre) {
        super(nombre);
    }
}
```

No puedes instanciar `Animal`, pero su constructor **sí se ejecuta** cuando creas un `Perro` (recuerda la cadena de constructores del punto 2). El constructor de la abstracta inicializa la parte común.

🧩 Template Method: las abstractas en acción

Este es el patrón que hace que las clases abstractas valgan su peso en oro. Defines el **esqueleto de un algoritmo** y dejas que las subclases rellenen los detalles:

```
public abstract class Bebida {
    public final void preparar() {          // final: nadie puede cambiar el esqueleto
        hervirAgua();
        prepararIngrediente();
        servirEnTaza();
        anadirExtras();
    }

    private void hervirAgua() { System.out.println("Hirviendo agua..."); }
    private void servirEnTaza() { System.out.println("Sirviendo en taza..."); }

    protected abstract void prepararIngrediente();
    protected abstract void anadirExtras();
}

public class Te extends Bebida {
     @Override protected void prepararIngrediente() { System.out.println("Poniendo la bolsita de té..."); }
    @Override protected void anadirExtras() { System.out.println("Añadiendo limón...");
}
}
```

La magia: `Te` solo implementa las dos partes variables, y `preparar()` (que es `final`) ya sabe el orden completo. Las subclases no pueden desordenar el algoritmo, solo personalizarlo. Ese es el **template method pattern**.

> 💡 **Detalle práctico:** fíjate en que los métodos abstractos son `protected`: solo la familia debe implementarlos. El esqueleto `public final` es lo único que el mundo exterior puede llamar.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** los métodos abstractos son contratos: la subclase TIENE que implementarlos. Si no, no compila. Úsalos para “forzar” comportamiento.

Ejercicio: ¿compila?

```
public abstract class A { public abstract void metodo(); }
public class B extends A { }
```

> 🔄 Solución

Ejercicio: ¿qué líneas dan error?

```
public abstract class A { public abstract void metodo(); }
public class B extends A { public void metodo() { } }

A a = new A();  // ¿error?
B b = new B();  // ¿error?
```

> 🔄 Solución

Ejercicio: el gato persa

```
abstract class Animal { abstract void hablar(); }
abstract class Mamifero extends Animal { void hablar() { System.out.println("Mamífero
raro"); } }
class Gato extends Mamifero { void hablar() { System.out.println("Miau"); } }
class GatoPersa extends Gato { }

public class Test {
    public static void main(String[] args) {
        Animal a = new GatoPersa();
        a.hablar();
    }
}
```

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Puedes hacer `new` sobre una clase abstracta?
2. ¿Qué está obligada a hacer una subclase concreta?
3. ¿Puede una clase abstracta tener métodos concretos y constructores?
4. ¿Qué garantiza el `final` en el método `preparar()` del template method?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. Una **clase abstracta** es un boceto no instanciable con métodos abstractos (obligatorios) y concretos (compartidos).
2. Las subclases **concretas** deben implementar todos los métodos abstractos; las abstractas pueden dejar la deuda pendiente.
3. El **template method** define el esqueleto `final` de un algoritmo y delega los detalles variables a las subclases.

> 🦎 **Vocabulario rápido**

Término	Idea general
Clase abstracta	Clase que no puede instanciarse
Método abstracto	Método sin implementación que obliga a las hijas
Subclase concreta	Subclase que implementa todos los abstractos
Template method	Esqueleto de algoritmo con partes que las hijas rellenan
`abstract`	Modificador que marca clases y métodos sin acabar

📚 Volver al índice de la unidad · **Anterior:** 06 · La clase Object: el tatarabuelo · **Siguiente:** 08 · Interfaces: el contrato que firmas

08 — Interfaces: el contrato que firmas

implements, polimorfismo con interfaces, métodos default y la gran decisión abstract vs interface 🧬

> 🗺️ **Estás en:** 🧬 **U11 · Herencia, polimorfismo e interfaces** → 08 · Interfaces: el contrato que firmas

---

📬 La idea en una frase

> **Una interfaz es un contrato: declara qué métodos debe tener una clase (con `implements`), sin decir cómo implementarlos, y permite que una clase firme varios contratos a la vez.**

¿Has firmado un contrato? "El trabajador se compromete a: programar en Java, no dormirse en las reuniones…" pero no dice CÓMO. Una **interfaz** en Java es eso: un contrato.

✍️ Declarando e implementando

```
public interface Reproducible {
    void reproducir();
    void pausar();
    void detener();
    int obtenerDuracion();
}

public class Cancion implements Reproducible {
    private String titulo;

    public Cancion(String titulo) {
        this.titulo = titulo;
    }

     @Override public void reproducir() { System.out.println("🎵 Reproduciendo: " +
titulo); }
    @Override public void pausar() { System.out.println("⏸ Canción pausada"); }
    @Override public void detener() { System.out.println("⏹ Canción detenida"); }
    @Override public int obtenerDuracion() { return 240; }
}
```

Cualquier clase que firme el contrato (con `implements`) **TIENE** que implementar todos esos métodos. El cómo es cosa suya.

> 📝 **Nota:** los métodos de una interfaz son `public abstract` implícitamente (no hace falta escribirlo). Y sus atributos son `public static final`: constantes, no estado.

🎭 Polimorfismo con interfaces

Una interfaz también puede ser el tipo de una referencia polimórfica:

```
import java.util.ArrayList;
import java.util.List;

public class Reproductor {
    public static void main(String[] args) {
        List<Reproducible> lista = new ArrayList<>();
        lista.add(new Cancion("Bohemian Rhapsody"));
        lista.add(new Pelicula("Inception"));

        for (Reproducible r : lista) {
            r.reproducir(); // no sabe si es canción o película
        }
    }
}
```

El `Reproductor` solo conoce el contrato `Reproducible`. Da igual si mañana llega un `Podcast` o un `Audiolibro`: mientras firmen el contrato, entran en la lista sin tocar una línea. Es el polimorfismo del punto 4, pero con contratos en vez de herencia.

🏊 Múltiples interfaces

Aquí está la gran ventaja sobre la herencia: una clase solo extiende **UNA** clase, pero puede implementar **VARIAS** interfaces:

```
public interface Nadador { void nadar(); }
public interface Corredor { void correr(); }

public class Triatleta implements Nadador, Corredor {
    @Override public void nadar() { System.out.println("🏊 Nadando 1.5 km"); }
    @Override public void correr() { System.out.println("🏃 Corriendo 10 km"); }
}
```

Un `Triatleta` es nadador Y corredor. Con herencia múltiple esto sería el famoso *problema del diamante*; con interfaces, firmar dos contratos no rompe nada.

🩹 default Methods: parches sin romper nada

Antes de Java 8, añadir un método a una interfaz **rompía** todas las clases que la implementaban. Llegaron los métodos `default`:

```
public interface Volable {
    void volar();

    default void despegar() {
        System.out.println("🛫 Despegando...");
    }
}

public class Avion implements Volable {
    @Override public void volar() { System.out.println("✈️ Volando a 900 km/h"); }
    // despegar() ya viene implementada: no estás obligado a tocarla
}
```

Un método `default` es una implementación “por defecto”: la clase puede usarla tal cual o sobrescribirla. Es una evolución del contrato sin romper a los que ya firmaron.

¿Y si dos interfaces tienen el mismo `default`?

Conflicto. La clase está obligada a sobrescribir y decidir:

```
interface Guerrero { default void atacar() { System.out.println("Ataca con espada"); } }
interface Mago { default void atacar() { System.out.println("Lanza hechizo"); } }

class Personaje implements Guerrero, Mago {
    @Override
    public void atacar() {
        Guerrero.super.atacar();
        Mago.super.atacar();
        System.out.println("¡Y usa ambas!");
    }
}
```

Imprime: `Ataca con espada`, `Lanza hechizo`, `¡Y usa ambas!`. La sintaxis `Interfaz.super.metodo()` llama a la implementación concreta de cada contrato.

⚖️ La gran decisión: abstract class vs interface

Aspecto	Clase abstracta	Interfaz
Métodos con código	Sí	Sí (default)
Atributos	Cualquiera	`public static final`
Herencia múltiple	No (un `extends`)	Sí (varios `implements`)
Constructores	Sí	No
Estado (atributos de instancia)	Sí	No
Cuándo usarla	Compartir código + forzar métodos en una familia	Definir capacidades/contratos reutilizables

> 💡 **Consejo:** usa **clase abstracta** para “es-un” con código compartido (una familia que comparte implementación). Usa **interfaz** para “puede-hacer” (capacidades que no dependen de la jerarquía: `Pato implements Volable, Cantable`). Si puedes, prefiere la interfaz: es más flexible.

🥊 EL RING: extends vs implements

Dos palabras clave discuten sobre quién es más importante.

extends: «Yo soy la herencia pura. Código reutilizado, una jerarquía clara. `Perro extends Animal`, `Coche extends Vehiculo`. ¡Soy la base de la POO!»

implements: «Sí, pero conmigo no hay límites. Una clase implementa varias interfaces. Con `extends` solo tienes un padre. Yo te permito ser varias cosas a la vez: `Serializable`, `Comparable`, `Cloneable`…»

extends: «Mis clases pueden tener código ya hecho. Tú solo declaras métodos vacíos. ¡Yo aporto implementación!»

implements: «Desde Java 8 tengo métodos `default` y `static`. Mira: `default void log()` ya funciona. Además, soy más flexible: no impongo una jerarquía rígida.»

extends: «Vale, pero sin mí las interfaces no tendrían sentido. Una interfaz no puede instanciarse sola.»

implements: «Y sin mí tendrías herencia múltiple, que es un caos. Mira el problema del diamante en C++.»

extends: «Nos necesitamos.»

implements: «Sí. `extends` para la jerarquía, `implements` para los contratos.»

> 🕶️ **Don Tip:** se pueden combinar: `class Perro extends Animal implements Mascota, Jugable`. Primero el `extends` (un padre), después los `implements` (varios contratos).

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** `implements` es un contrato: la clase se compromete a tener TODOS los métodos de la interfaz. Nada de “casi todos”.

Ejercicio: ¿compila?

```
public interface Cantante { void cantar(); }
public interface Bailarin { void bailar(); }

public class Artista implements Cantante, Bailarin {
    @Override public void cantar() { System.out.println("canta"); }
    // ¡FALTA bailar()!
}
```

> 🔄 Solución

Ejercicio: ¿qué imprime?

```
interface Guerrero { default void atacar() { System.out.println("Ataca con espada"); } }
interface Mago { default void atacar() { System.out.println("Lanza hechizo"); } }

class Personaje implements Guerrero, Mago {
    @Override
    public void atacar() {
        Guerrero.super.atacar();
        Mago.super.atacar();
        System.out.println("¡Y usa ambas!");
    }
}

public class Test {
    public static void main(String[] args) {
        new Personaje().atacar();
    }
}
```

> 🔄 Solución

Ejercicio: la clase que implementa dos contratos

```java
public interface Nadador { void nadar(); }
public interface Corredor { void correr(); }

public class Triatleta implements Nadador, Corredor {
    @Override public void nadar() { System.out.println("🏊 Nadando"); }
    @Override public void correr() { System.out.println("🏃 Corriendo"); }
}
```

¿Puedes usar un `Triatleta` donde se espera un `Nadador`? ¿Y donde se espera un `Object`?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué palabra usa una clase para firmar un contrato?
2. ¿Cuántas clases puede extender y cuántas interfaces puede implementar una clase?
3. ¿Qué son los métodos `default` y para qué sirven?
4. ¿Cuándo eliges una interfaz en vez de una clase abstracta?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Una **interfaz** es un contrato: la clase que la implementa debe aportar todos sus métodos.
2. Las interfaces permiten **polimorfismo** y **múltiples contratos**, con métodos `default` para evolucionar sin romper nada.
3. Regla práctica: **clase abstracta** para familias con código compartido, **interfaz** para capacidades flexibles.

> 🦎 **Vocabulario rápido**

Término	Idea general
Interfaz	Contrato de métodos que la clase debe implementar

Término	Idea general
`implements`	La palabra que firma el contrato
Método `default`	Implementación por defecto que la clase puede heredar
Contrato múltiple	Una clase puede implementar varias interfaces
`Interfaz.super.metodo()`	Cómo resolver conflictos entre `default`

📚 Volver al índice de la unidad · **Anterior:** 07 · Clases abstractas · **Siguiente:** 09 · Repaso interactivo

09 — Repaso interactivo: domina la herencia

El cierre práctico de la unidad, con polimorfismo, contratos y una herencia que no va a funcionar 😈

> 🗺️ **Estás en:** 🧬 **U11 · Herencia, polimorfismo e interfaces** → 09 · Repaso interactivo

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```java
public class Animal {
    public void hacerSonido() { System.out.println("..."); }
}

public class Perro extends Animal {
    public void hacerSonido() { System.out.println("¡Guau!"); }
    public void moverCola() { System.out.println("*mueve la cola*"); }
}

public class Gato extends Animal {
    public void hacerSonido() { System.out.println("¡Miau!"); }
}

public class Main {
    public static void main(String[] args) {
        Animal a1 = new Perro();
        Animal a2 = new Gato();
        a1.hacerSonido();
        a2.hacerSonido();
        // a1.moverCola();  <- ¿compila?
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **`...` y `...`** → La variable es `Animal`, así que se ejecuta el método de `Animal`. ❌
2. **`¡Guau!` y `¡Miau!`** → ✅ ¡Correcto! El polimorfismo ejecuta el método del objeto real, no el de la referencia.
3. **`¡Guau!` y un error** → `a2` no puede ser un `Gato`. ❌

> 🔄 **Solución**

🔥 Fireside Chat: Clase abstracta vs Interfaz

> *Dos veteranos de la POO discuten junto a la pizarra mientras una subclase los mira.*

Clase abstracta: — Yo soy el boceto. Defino el esqueleto y comparto código: mis hijos se ahorran escribir lo aburrido. `Figura` con `calcularArea()` abstracto y `mostrarColor()` ya hecho. Soy la familia que hereda.

Interfaz: — La familia hereda de uno solo. Yo firmo contratos a mansalva. Un `Pato` implementa `Volable` Y `Nadador` al mismo tiempo. ¿Cuántos `extends` le dejas tú al pobre pato?

Clase abstracta: — Pero yo tengo estado. Atributos `protected`, constructores, métodos concretos con lógica de verdad. Tú solo declaras "esto tiene que existir".

Interfaz: — Desde Java 8 tengo métodos `default` con implementación. Y soy la base del polimorfismo moderno: `List<Reproducible>`, `Comparable`, `Runnable`… Toda la API de Java está llena de mí.

Clase abstracta: — Sin mí, no tendrías familias con código compartido. El template method, ¿quién lo hace?

Interfaz: — Y sin mí tendrías el problema del diamante. Nos complementamos.

Clase abstracta: — Eso es verdad. Yo para las familias.

Interfaz: — Yo para las capacidades. ¿Trato?

> La lección: usa **clase abstracta** cuando haya una familia con código compartido (es-un); usa **interfaz** cuando quieras capacidades reutilizables (puede-hacer). Y se pueden combinar: `class Perro extends Animal implements Mascota, Jugable`.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy la palabra que hace que una clase hija obtenga todo lo de su padre.**
2. **Soy la anotación que le dice al compilador "verifica que estoy sobrescribiendo, no inventando".**
3. **Soy la llamada que configura la parte del padre antes de que el hijo haga lo suyo.**
4. **Soy el mecanismo por el que `Animal a = new Perro()` ejecuta el sonido del perro.**
5. **Soy el operador que pregunta "¿eres realmente un Perro?" antes de convertir.**
6. **Soy el contrato que una clase firma con `implements` y que admite varios a la vez.**
7. **Soy la clase abstracta que define el esqueleto de un algoritmo y deja que las hijas rellenen los detalles.**

> 🔄 **Respuestas**

⚖️ Verdadero o falso

1. **“Si `Animal` tiene un método `public`, una subclase puede hacerlo `private`.”**
2. **“`super.metodo()` se puede llamar desde cualquier línea del método.”**
3. **“Una clase puede implementar tres interfaces y extender una clase.”**
4. **“Los atributos también son polimórficos: `ref.x` usa el de la clase real del objeto.”**
5. **“Sobrescribir `equals()` sin `hashCode()` es un error de compilación.”**

> 🔄 Soluciones

Laboratorio de tortura: la herencia que llora

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto cuando lo arregles.

```java
public class Vehiculo
    protected int velocidad;

    public Vehiculo(int velocidad) {
        this.velocidad = velocidad;
    }

    void mover() { System.out.println("Vehículo a " + velocidad + " km/h"); }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(int velocidad, int puertas) {
        this.velocidad = puertas;
        this.puertas = velocidad;
    }

    @Override
    void mover() {
        super.mover();
        System.out.println("Coche con " + puertas + " puertas");
    }

    public static void main(String[] args) {
        Vehiculo v = new Coche(120, 5);
        v.mover()
    }
}
```

Fallo intencionado: uno de los errores parece correcto a simple vista porque “se ve bien”, pero cambia por completo el comportamiento del programa.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún ; o { que falte? *no → sigue buscando.*
   > **¿Y si sigo atascado?**
2. ¿Compila ya? *no → mira el mensaje de error y los constructores.*

> **¿Y si sigo atascado?**

3. ¿Ejecuta pero los números salen cambiados? *Es el error de lógica: el constructor intercambia los valores.*

> **Solución final**

Los **3 errores de compilación**:

1. Falta la `{` de apertura de la clase `Vehiculo`.
2. El constructor de `Coche` no llama a `super(velocidad)`. Al escribir cualquier constructor en el padre, el vacío desaparece.
3. Falta el `;` al final de `v.mover()`.

El **error de lógica**: en el constructor, las asignaciones están cruzadas:

```
this.velocidad = puertas;   // ¡mal! velocidad debería ser 120
this.puertas = velocidad;   // ¡mal! puertas debería ser 5
```

Compila y ejecuta perfectamente, pero intercambia los valores. Código corregido:

```java
public class Vehiculo {
    protected int velocidad;

    public Vehiculo(int velocidad) {
        this.velocidad = velocidad;
    }

    void mover() { System.out.println("Vehículo a " + velocidad + " km/h"); }
}

public class Coche extends Vehiculo {
    private int puertas;

    public Coche(int velocidad, int puertas) {
        super(velocidad);
        this.puertas = puertas;
    }

    @Override
    void mover() {
        super.mover();
        System.out.println("Coche con " + puertas + " puertas");
    }

    public static void main(String[] args) {
        Vehiculo v = new Coche(120, 5);
        v.mover();
    }
}
```

Salida correcta:

```
Vehículo a 120 km/h
Coche con 5 puertas
```

Con la versión rota, una vez arreglados los otros errores, la salida era `Vehículo a 5 km/h` y `Coche con 120 puertas`: dos líneas cambiadas por un constructor cruzado.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Heredero**	Explicar qué se hereda y qué no (private, constructores) sin mirar la tabla
🏅 **El Sobrescribiente**	Usar `@Override` + `super` para extender un método del padre con sentido
🏅 **El Camaleón**	Resolver un problema de colecciones con polimorfismo sin usar `instanceof`
🏅 **El Notario**	Decidir cuándo usar clase abstracta e interfaz y justificarlo
🏅 **El Genealogista**	Diseñar una jerarquía de 3 niveles con `toString()`, `equals()` y `hashCode()`

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```
class A { void mensaje() { System.out.println("A"); } }
class B extends A { void mensaje() { System.out.println("B"); } }
class C extends B { }

public class Test {
    public static void main(String[] args) {
        A r1 = new B();
        A r2 = new C();
        B r3 = new C();
        r1.mensaje();
        r2.mensaje();
        r3.mensaje();
    }
}
```

2. **El constructor perdido:** `Animal` tiene solo `Animal(String nombre)`. ¿Qué le pasa a `Perro extends Animal` si su constructor no llama a `super`?
3. **El detective del equals:** tu `HashSet` contiene dos objetos que son `equals()` entre sí… ¿qué has olvidado sobrescribir?

4. **Verdadero o falso:** "`a instanceof Perro` es `true` si `a` es de tipo `Animal` y el objeto real es un `Perro`."

> 💡 Soluciones

🧩 Crucigrama de bits

```
Horizontal:
1. La palabra que establece la herencia (7 letras)
3. La anotación que verifica que sobrescribes (8 letras)
5. El contrato que firmas con implements (10 letras)
6. Excepción al convertir un Gato en Perro (18 letras)

Vertical:
2. La clase de la que todos heredan (6 letras)
4. La palabra que llama al método del padre (5 letras)
```

> 📝 Soluciones

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, la diferencia entre herencia y composición."**
2. **"¿Qué hace `super()` en un constructor y por qué debe ser la primera línea?"**
3. **"¿Cuándo usarías una clase abstracta y cuándo una interfaz?"**
4. **"¿Por qué `ArrayList<Animal>` puede guardar `Perro` y `Gato` pero no al revés?"**
5. **"¿Qué excepción lanzas al hacer `(Perro) gato` y cómo la evitas?"**
6. **"¿Por qué `equals()` y `hashCode()` van siempre juntos?"**

🤷 No hay preguntas tontas

> ❓ **¿Puedo heredar de varias clases a la vez?**

No. Java no permite herencia múltiple (el *problema del diamante*). Pero por eso existen las interfaces: puedes implementar tantas como quieras.

> ❓ **¿Qué pasa si una clase hija tiene un atributo con el mismo nombre que el padre?**

Se dice que lo **oculta** (shadowing). Pero ojo: los atributos no son polimórficos. Si usas una referencia de la superclase, ves el de la superclase; si usas una de la subclase, ves el de la subclase. Mejor no jugar a eso: nombra las cosas distinto.

> ❓ **¿Y los métodos `final`? ¿No se pueden sobrescribir?**

Exacto. Un método `final` queda congelado: las hijas lo heredan pero no pueden cambiarlo. Y una clase `final` (como `String`) no puede tener hijas. Es la decisión de diseño "esto no se toca".

🎬 Poscréditos

La programadora termina su simulador de zoológico virtual: una clase abstracta `Animal`, un `Perro` y un `Gato` que la implementan, y un `ArrayList<Animal>` que los mete a todos en la misma jaula. Hasta que, en la última línea, intenta convertir un gato en perro y todo explota con una `ClassCastException`.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — Otra vez. Un gato disfrazado de perro. ¿Sabes lo que te ha faltado?

Programadora: — ¿El `instanceof`?

CONRAD: — *asiente* Ves. Cuando entiendes quién es realmente cada objeto, los errores dejan de ser magia. Herencia, `super`, polimorfismo, contratos… Ya no escribes clases, diseñas familias y firmas acuerdos.

Programadora: — ¿Y ahora qué toca?

CONRAD: — *toma un sorbo* Ahora que tienes jerarquías enteras, necesitas guardarlas en montones. Y no, no vale una caja para cada cosa. Te presentaré unas estructuras que guardan animales, figuras y hasta contratos con elegancia.

PRÓXIMAMENTE EN U12: Colecciones. El momento en el que tus jerarquías aprenden a guardarse en listas, conjuntos y mapas. 📚

📚 Volver al índice de la unidad · **Anterior:** 08 · Interfaces: el contrato que firmas · **Siguiente:** U12 · Colecciones

U12 — Colecciones, genéricos y mapas

El parking que crece, el portero sin duplicados, el `` que lo cambia todo y la guía telefónica: ArrayList, Set, genéricos y HashMap en una sola unidad 🧩

En la U06 viste los arrays: tamaño fijo, acceso por índice y bastante trabajo manual para crecer o encoger. Hoy eso se acaba. Las **colecciones** de Java son las estructuras de datos con superpoderes: crecen solas, se encogen solas, rechazan duplicados si quieres, se ordenan automáticamente y se recorren sin que te importe cómo están construidas por dentro.

Y esta unidad va más allá: después de dominar las colecciones, levantaremos el capó del `<String>` que llevas escribiendo desde hace semanas. Los **genéricos** explican por qué el compilador te fía de tus listas, y los **mapas** (`HashMap`, `TreeMap`) te dan la estructura que más se usa en Java real: asociar una clave con un valor.

Esta unidad tiene cuatro grandes actos:

- **Listas (`List`):** `ArrayList`, el parking que crece solo, y `LinkedList`, la conga que encadena a sus elementos y brilla en colas y pilas.
- **Conjuntos (`Set`):** `HashSet`, el portero que no deja entrar duplicados; `TreeSet`, el que los ordena solos; y `LinkedHashSet`, el que recuerda el orden de llegada.
- **Genéricos:** de dónde sale el `<T>`, tu propia clase genérica, el diamante `<>`, el type erasure y el principio PECS.
- **Mapas (`Map`):** `HashMap`, la guía telefónica de Java; `TreeMap`, el que ordena sus claves; y la guía definitiva para elegir estructura.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Usar **`ArrayList`** para guardar listas dinámicas y distinguir **`remove(índice)`** de **`remove(objeto)`**.
- Aplicar **`LinkedList`** para colas (FIFO) y pilas (LIFO).

- Eliminar duplicados con **`HashSet`** y entender por qué `hashCode()` y `equals()` van siempre juntos.
- Ordenar y mantener el orden con **`TreeSet`** y **`LinkedHashSet`**.
- Recorrer cualquier colección con **`Iterator`** y borrar durante el recorrido sin excepción.
- Manipular listas con la clase **`Collections`** (`sort`, `reverse`, `shuffle`, `max`, `min`, `frequency`).
- Crear tus **clases y métodos genéricos** con `<T>`, límites y wildcards.
- Guardar y buscar parejas clave → valor con **`HashMap`** y **`TreeMap`**.
- **Elegir la estructura adecuada** (List, Set o Map) para cada problema.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · ArrayList y el Java Collections Framework	El parking que crece solo, la familia `java.util` y por qué no guarda `int` a pelo	Todos
02 · LinkedList: la conga line	La lista enlazada, colas FIFO, pilas LIFO y sus métodos de extremos	Todos
03 · HashSet, TreeSet y LinkedHashSet: los conjuntos	El porter sin duplicados, el que ordena solo y el que recuerda el orden de llegada	Todos
04 · Iterator y Collections	Borrar mientras recorres y la navaja suiza de las colecciones	Todos
05 · Clases genéricas: tu propia `Caja<T>`	De dónde sale el `<String>`, convenciones T/E/K/V y tu primera clase genérica	Todos
06 · El diamante, métodos genéricos y wildcards	Type erasure, inferencia, límites y el principio PECS	Todos
07 · HashMap: la guía telefónica	Clave → valor, `get`, `put`, `containsKey` y recorrer mapas	Todos
08 · TreeMap y cómo elegir estructura	Mapas ordenados, `LinkedHashMap` y Map vs List vs Set	Todos

Punto	Qué aprenderás	Dificultad
09 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

✅ Criterios de evaluación cubiertos (RA6)

RA6: Escribe programas que manipulen información, seleccionando y utilizando tipos avanzados de datos.

CE	Criterio	Dónde se cubre
RA6 b)	Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados.	✅ Puntos 1, 7 y 8
RA6 c)	Se han utilizado listas para almacenar y procesar información.	✅ Puntos 1, 2 y 9
RA6 d)	Se han utilizado iteradores para recorrer los elementos de las listas.	✅ Puntos 4 y 9
RA6 e)	Se han reconocido las características y ventajas de cada una de las colecciones de datos disponibles.	✅ Puntos 3, 7 y 8
RA6 f)	Se han utilizado tipos enumeradores, estructuras dinámicas de datos y clases genéricas.	✅ Puntos 5, 6, 7 y 8

> 📌 La **RA6** se completa con la **U06** (arrays, CE a) y la **U14** (regex y XML, CEs g, h e i). Esta unidad cubre colecciones, genéricos y mapas: la librería `java.util` entera.

🚪 ¿Por dónde empiezo?

- ¿Vienes de la U11 y quieres lo esencial? → Arranca en el punto 1 y el punto 2: las listas de todos los días.
- ¿Ya usas listas y quieres conjuntos? → Ve directo al punto 3: duplicados fuera.
- ¿Te pica la curiosidad del `<T>`? → Salta al punto 5 y al punto 6: genéricos de cabo a rabo.
- ¿Quieres mapas? → Punto 7 y punto 8: la guía telefónica y el mapa de decisión.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · ArrayList y el Java Collections Framework

⏭️ **Al acabar la unidad, continúa en U13 · Programación funcional (Streams y Lambdas).**

01 · ArrayList y el Java Collections Framework

El parking que crece solo: ArrayList, la familia java.util y por qué no puede guardar int a pelo 🅿️📈

> 🗺️ **Estás en:** 📚 **U12 · Colecciones** → 01 · ArrayList y el Java Collections Framework

📬 La idea en una frase

> **ArrayList es un array con superpoderes: crece solo, se encoge solo y trae métodos listos (`add`, `get`, `remove`, `contains`…) en lugar de obligarte a escribir cada bucle a mano.**

En la U06 viste los arrays: tamaño fijo, acceso por índice y mucho trabajo manual. Llegó el momento que estaban esperando: el parking se ha quedado pequeño y necesitas algo que crezca. Aquí empieza la unidad de las colecciones y, probablemente, la herramienta que más usarás en tu vida como programadora o programador.

🅿️ El problema: tu array se ha quedado sin plazas

Has creado un array de 10 plazas. Han llegado 11 gatos. ¿Qué haces?

```
String[] gatos = new String[10]; // 10 plazas, 11 gatos... mal asunto
```

Con un array clásico tendrías que crear uno nuevo, copiar todo y añadir el que falta. Es como aparcar en la calle porque el parking está lleno:

```
String[] gatosMasGrande = new String[gatos.length + 1];
System.arraycopy(gatos, 0, gatosMasGrande, 0, gatos.length);
gatosMasGrande[gatosMasGrande.length - 1] = "Bigotes Jr.";
gatos = gatosMasGrande; // Ahora apunta al nuevo array
```

Funciona, pero es tedioso. Y si además tienes que borrar un elemento del medio, es peor. Necesitas algo que crezca y se encoja solo.

🎓 El Java Collections Framework (JCF)

Para eso existe el **JCF**: una familia de clases e interfaces en `java.util` que gestionan grupos de objetos como si fueran de goma. El árbol de la familia empieza así:

```
Collection (interfaz)
├── List        → ArrayList, LinkedList (ordenados, permiten duplicados)
├── Set         → HashSet, TreeSet (sin duplicados)
└── Queue       → colas (FIFO)
```

Todas comparten métodos básicos: `add`, `remove`, `contains`, `size`, `isEmpty`, `clear`. Aprender uno es aprender el esqueleto de todos. La interfaz `List` es la que más usarás.

> 💡 **Consejo:** declara tus variables con la interfaz, no con la clase: `List<String> gatos = new ArrayList<>();`. Así, si mañana necesitas una `LinkedList`, cambias una línea. Lo verás en el punto 7.

🅿️📈 ArrayList: el parking que crece solito

ArrayList es un array "de goma": cuando se llena, crea uno más grande por dentro y copia los datos. Tú no te enteras de nada.

```java
import java.util.ArrayList;

public class EjemploArrayList {
    public static void main(String[] args) {
        ArrayList<String> gatos = new ArrayList<>();
        gatos.add("Bigotes");                    // [Bigotes]
        gatos.add("Garfield");                   // [Bigotes, Garfield]
        gatos.add("Misifú");                     // [Bigotes, Garfield, Misifú]
        gatos.remove(1);                         // [Bigotes, Misifú] - adiós, Garfield
        String primero = gatos.get(0);           // "Bigotes"
        int cuantos = gatos.size();              // 2
        boolean hay = gatos.contains("Misifú");  // true
        int pos = gatos.indexOf("Bigotes");      // 0
        System.out.println(gatos);
    }
}
```

El repertorio de métodos imprescindibles:

Método	Qué hace
`add(e)`	Añade al final
`add(i, e)`	Inserta en la posición `i` (desplaza el resto)
`get(i)`	Devuelve el elemento de la posición `i`
`set(i, e)`	Sustituye el elemento de la posición `i`
`remove(i)`	Borra por índice
`remove(objeto)`	Borra la primera aparición de ese objeto
`size()`	Número de elementos (NO `length`)
`contains(e)`	¿Está?
`indexOf(e)`	Posición de la primera aparición
`isEmpty()`	¿Está vacía?
`clear()`	Lo borra todo

> ⚠️ **Advertencia:** ArrayList usa `size()`, no `length`. Array usa `length`, no `size()`. String usa `length()`, no `length` ni `size()`. Cada uno tiene su propia forma de preguntar cuánto mide. Es una trampa en el 90% de los exámenes.

Recorrer un ArrayList

Con for-each (el preferido para leer):

```
for (String gato : gatos) {
    System.out.println("Miau: " + gato);
}
```

O con for clásico, que también funciona porque `List` tiene índices:

```
for (int i = 0; i < gatos.size(); i++) {
    System.out.println(i + ": " + gatos.get(i));
}
```

Fíjate: `gatos.size()` dentro de la condición se evalúa en cada vuelta. Funciona, aunque para rendimiento se suele guardar en una variable si no cambia.

🚫 ArrayList NO guarda primitivos

No puedes hacer `ArrayList<int>`. Los genéricos (`<...>`) solo funcionan con objetos. Para números usas las clases **wrapper**:

```
ArrayList<Integer> numeros = new ArrayList<>();
numeros.add(42);              // autoboxing: int → Integer
int n = numeros.get(0);       // unboxing: Integer → int
```

Desde Java 5, el **autoboxing/unboxing** es automático: metes un `int` y Java lo envuelve en un `Integer` sin que tú lo notes. Pero por dentro sigue habiendo objetos.

> 📝 **Nota:** lo mismo con el resto de primitivos: `boolean` → `Boolean`, `double` → `Double`, `char` → `Character`… Si necesitas saber más sobre genéricos y wrappers, los puntos 4 y 5 de esta misma unidad los estiran a fondo.

El truco de remove con números

`remove(1)` y `remove(Integer.valueOf(1))` NO hacen lo mismo:

```
ArrayList<Integer> nums = new ArrayList<>();
nums.add(10);
nums.add(20);
nums.add(30);
nums.remove(1);               // borra por ÍNDICE: se va el 20
// nums.remove(Integer.valueOf(20)); // esto borra el OBJETO 20
```

Si quieres borrar el número 20, debes pasarle el wrapper. Si le pasas un `int`, lo interpreta como índice. Es el clásico error de los novatos con `ArrayList<Integer>`.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** ArrayList usa arrays por dentro. Cuando se llena, crea uno nuevo y copia. Cada operación tiene su coste, pero tú no tienes que pensar en ello.

Ejercicio: el ArrayList misterioso

```
import java.util.ArrayList;

public class BeTheList {
    public static void main(String[] args) {
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(10);
        lista.add(20);
        lista.add(30);
        lista.add(1, 15);
        lista.remove(Integer.valueOf(20));

        for (Integer n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

¿Qué imprime?

- (A) 10 20 30
- (B) 10 15 30
- (C) 10 15 20 30
- (D) 10 15

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué método usa ArrayList para saber su tamaño: `length`, `length()` o `size()`?
2. ¿Por qué no se puede hacer `ArrayList<int>`?
3. ¿Qué borra `lista.remove(2)` en un `ArrayList<Integer>`: el índice 2 o el número 2?
4. ¿Para qué sirve declarar `List<String> x = new ArrayList<>()` en vez de `ArrayList<String> x`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **ArrayList** es una lista dinámica: crece solo, se encoge solo y trae métodos (`add`, `get`, `remove`, `contains`, `size`) para no reinventar la rueda.
2. Pertenece al **Java Collections Framework** (`java.util`), una familia de listas, conjuntos y colas con métodos comunes.
3. No guarda **primitivos**: usa las clases wrapper (`Integer`, `Double`…) con autoboxing automático, y cuidado con la diferencia entre `remove(índice)` y `remove(objeto)`.

> 🐛 **Vocabulario rápido**
>
Término	Idea general
JCF	La familia de colecciones de `java.util`
`List`	Colección ordenada con índices y duplicados permitidos
Wrapper	Clase que envuelve un primitivo (`Integer` envuelve `int`)
Autoboxing	Pasar de primitivo a wrapper automáticamente
Unboxing	Pasar de wrapper a primitivo automáticamente

📚 Volver al índice de la unidad · **Anterior:** — · **Siguiente:** 02 · LinkedList: la conga line

02 · LinkedList: la conga line

La lista que encadena a sus elementos como una conga: rapidísima al principio y al final, y perfecta para colas y pilas 🎉

> 🗺️ **Estás en:** 📚 **U12 · Colecciones** → 02 · LinkedList: la conga line

📬 La idea en una frase

> **LinkedList encadena sus elementos como una conga: cada uno sabe quién va delante y quién detrás, y es rapidísima para añadir o borrar al principio y al final.**

En el punto 1 viste a `ArrayList`, la lista de todos los días. Ahora toca conocer a su prima, que resuelve un problema distinto: meter y sacar gente por los extremos sin que nada se descoloque. Es la colección perfecta para colas y pilas.

🎉 LinkedList: la conga line

LinkedList es una **lista enlazada**: cada elemento sabe quién está delante y quién detrás, como en una conga. Para recorrerla por índice hay que saltar de elemento en elemento (es más lenta que `ArrayList` en eso), pero es rapidísima para añadir o borrar **al principio** y al final.

```
import java.util.LinkedList;

public class EjemploLinkedList {
    public static void main(String[] args) {
        LinkedList<String> cola = new LinkedList<>();
        cola.addLast("Persona 1");   // al final
        cola.addLast("Persona 2");
        cola.addFirst("Colado");     // se cuela al principio
        String primero = cola.removeFirst(); // "Colado" - se va
        System.out.println(cola);    // [Persona 1, Persona 2]
    }
}
```

Sus métodos propios (aunque `add`, `remove`, `get` también funcionan):

Método	Qué hace
`addFirst(e)` / `addLast(e)`	Añade al principio / al final
`removeFirst()` / `removeLast()`	Quita y devuelve el primero / el último
`getFirst()` / `getLast()`	Mira el primero / el último sin quitarlo
`peek()`	Mira el primero sin quitarlo (no lanza si está vacía)
`poll()`	Quita el primero (no lanza si está vacía)

> 💡 **Consejo:** usa `LinkedList` cuando necesites una **cola** (FIFO: el primero en llegar es el primero en salir) o una **pila** (LIFO), o cuando vayas a añadir/borrar mucho al principio. Para acceso aleatorio frecuente (`get(i)`), `ArrayList` es mejor: va directo al índice.

🛒 Colas y pilas: los dos disfraces de LinkedList

LinkedList es camaleónica: según qué métodos uses, se comporta como una cola o como una pila.

Cola (FIFO): el primero en llegar es el primero en salir. Como la fila del supermercado.

```
LinkedList<String> cola = new LinkedList<>();
cola.addLast("Ana");     // Ana se pone al final
cola.addLast("Bob");     // Bob detrás de Ana
cola.addLast("Carla");   // Carla detrás de Bob
String atendido = cola.removeFirst(); // "Ana" - la primera en llegar
```

Pila (LIFO): el último en entrar es el primero en salir. Como la pila de platos: el de arriba del todo es el que coges.

```
LinkedList<String> pila = new LinkedList<>();
pila.addFirst("plato 1");  // base
pila.addFirst("plato 2");
pila.addFirst("plato 3");  // el último apilado
String quito = pila.removeFirst(); // "plato 3" - el último que entró
```

> ⚠️ **Advertencia:** cuando la cola está vacía, `removeFirst()` y `getFirst()` lanzan una `NoSuchElementException`. Pregunta antes con `isEmpty()`, o usa `peek()` y `poll()`, que

> devuelven `null` en lugar de explotar.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** `addLast` + `removeFirst` = cola FIFO. `addFirst` + `removeFirst` = pila LIFO. Elige los métodos, no la clase.

Ejercicio: el colado de la conga

```
import java.util.LinkedList;

public class BeTheConga {
    public static void main(String[] args) {
        LinkedList<String> conga = new LinkedList<>();
        conga.addLast("Ana");
        conga.addLast("Bob");
        conga.addFirst("Carlos");   // se cuela
        conga.removeLast();

        System.out.println(conga.getFirst());
    }
}
```

¿Qué imprime?

- (A) Ana
- (B) Bob
- (C) Carlos
- (D) Error de ejecución

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué colección usarías para una cola de supermercado?
2. ¿Qué hace `removeFirst()` sobre una lista vacía si usas `peek()`/`poll()` en su lugar?

3. ¿Cuál es la diferencia clave de rendimiento entre `LinkedList` y `ArrayList`?
4. Con `addFirst` + `removeFirst`, ¿estás simulando una cola o una pila?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. **LinkedList** encadena sus elementos (la conga): cada uno sabe quién va delante y detrás.
2. Es **rapidísima al principio y al final**, y por eso es la reina de las **colas (FIFO)** y las **pilas (LIFO)**.
3. Para acceso aleatorio frecuente (`get(i)`), gana `ArrayList`: `LinkedList` tiene que saltar de eslabón en eslabón.

> 🐛 **Vocabulario rápido**

Término	Idea general
Lista enlazada	Elementos encadenados que saben quién va delante y detrás
FIFO	First In, First Out: la cola del supermercado
LIFO	Last In, First Out: la pila de platos
`peek()`	Mirar el primero sin quitarlo
`poll()`	Quitar el primero sin lanzar excepción si está vacía

📚 Volver al índice de la unidad · **Anterior:** 01 · ArrayList y el Java Collections Framework · **Siguiente:** 03 · HashSet, TreeSet y LinkedHashSet

03 · HashSet, TreeSet y LinkedHashSet: los conjuntos

El portero sin duplicados, el que ordena solo y el que recuerda el orden de llegada: la familia Set al completo 🚫🧭

> 🗺️ **Estás en:** 📚 **U12 · Colecciones, genéricos y mapas** → 03 · HashSet, TreeSet y LinkedHashSet

---

📬 La idea en una frase

> **HashSet es una discoteca que no deja entrar a nadie repetido; TreeSet es el mismo portero pero con brújula (ordena solo); y LinkedHashSet tiene memoria del orden de llegada.**

En el punto 2 conociste las listas: orden y duplicados bienvenidos. Ahora cambia el baile: hay problemas donde los repetidos sobran. Ahí entra la familia `Set`, con tres hermanos que comparten la manía de no admitir duplicados pero se diferencian en el orden.

🚫 HashSet: el portero que no deja duplicados

HashSet es como una discoteca: **no deja entrar a nadie que ya esté dentro**. No importa el orden, solo la exclusividad.

```
import java.util.HashSet;

public class EjemploHashSet {
    public static void main(String[] args) {
        HashSet<String> invitados = new HashSet<>();
        invitados.add("Ana");
        invitados.add("Bob");
        invitados.add("Ana"); // No pasa nada, Ana ya está
        System.out.println(invitados.size()); // 2, no 3
        System.out.println(invitados.contains("Bob")); // true
    }
}
```

¿Cómo sabe si un elemento ya está? Usa `hashCode()` y `equals()`. Primero calcula el `hashCode()` del elemento para encontrar el "cajón" y luego comprueba con `equals()` si ya hay uno igual dentro.

> ⚠️ **Advertencia:** si sobreescribes `equals()` en una clase, SOBREESCRIBE `hashCode()`. Siempre. Si dos objetos son iguales según `equals()`, deben tener el mismo `hashCode()`. Si no, HashSet se volverá loco. Repito: **siempre**.

Operaciones típicas con HashSet

```
HashSet<String> set = new HashSet<>();
set.add("rojo");
set.add("verde");
set.add("azul");
set.remove("rojo");
set.contains("verde");   // true
set.isEmpty();           // false
set.size();              // 2
set.clear();             // lo vacía todo
```

Sus ventajas: **O(1)** para añadir, borrar y comprobar pertenencia, y elimina duplicados sin que tengas que pensarlo. Es la colección más rápida para "¿ya tengo esto?".

Un HashSet no tiene orden

HashSet no garantiza ningún orden. Si metes `{"rojo", "verde", "azul"}`, al imprimir puede salir `[azul, rojo, verde]` o cualquier otra combinación. No te fíes del orden, fíate de `size()` y `contains()`.

```
HashSet<String> colores = new HashSet<>();
colores.add("rojo");
colores.add("verde");
colores.add("azul");
System.out.println(colores); // ¿[rojo, verde, azul]? Quizá, quizá no...
```

 TreeSet: el organizado

TreeSet es un HashSet que **se ordena solo**. Internamente usa un árbol rojo-negro (no necesitas saber más, solo que ordena). Todo lo que metas se coloca automáticamente en su sitio.

```
import java.util.TreeSet;

public class EjemploTreeSet {
    public static void main(String[] args) {
        TreeSet<String> ordenado = new TreeSet<>();
        ordenado.add("Zara");
        ordenado.add("Ana");
        ordenado.add("Bob");
        System.out.println(ordenado); // [Ana, Bob, Zara] - orden alfabético

        System.out.println(ordenado.first());      // "Ana"
        System.out.println(ordenado.last());       // "Zara"
        System.out.println(ordenado.headSet("Bob")); // [Ana]
    }
}
```

Métodos extra útiles:

Método	Qué hace
`first()` / `last()`	El menor / el mayor
`headSet(e)`	Todo lo que va antes de `e`
`tailSet(e)`	Todo lo que va después (incluido) de `e`
`subSet(a, b)`	El trozo entre `a` y `b`
`floor(e)` / `ceiling(e)`	El mayor ≤ e / el menor ≥ e

> 📝 **Nota:** TreeSet **no admite `null`**. Para ordenar necesita comparar elementos, y ¿cómo comparas `null` con algo? HashSet, en cambio, admite un único `null`. Detalle de examen asegurado.

🧠 LinkedHashSet: el que recuerda el orden de llegada

LinkedHashSet es el hermano de HashSet que **conserva el orden de inserción**. Elimina duplicados como el portero de antes, pero recuerda quién llegó primero, segundo, tercero…

```
import java.util.LinkedHashSet;

public class EjemploLinkedHashSet {
    public static void main(String[] args) {
        LinkedHashSet<String> colaDeEntrada = new LinkedHashSet<>();
        colaDeEntrada.add("Ana");
        colaDeEntrada.add("Bob");
        colaDeEntrada.add("Ana");   // duplicado: no entra
        colaDeEntrada.add("Carla");
        System.out.println(colaDeEntrada); // [Ana, Bob, Carla] - orden de llegada
    }
}
```

Su uso estrella: **eliminar duplicados manteniendo el orden de primera aparición**. Un clásico de los ejercicios y de las entrevistas.

```
import java.util.ArrayList;
import java.util.LinkedHashSet;

ArrayList<Integer> lista = new ArrayList<>(java.util.Arrays.asList(3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5));
LinkedHashSet<Integer> sinRepetidos = new LinkedHashSet<>(lista);
System.out.println(sinRepetidos); // [3, 1, 4, 5, 9, 2, 6] - sin duplicados y en orden de llegada
```

Construir un `LinkedHashSet` a partir de la lista elimina los repetidos; convertirlo de vuelta en `ArrayList` te devuelve la lista limpia. Dos líneas y el problema de los duplicados desaparece.

> 💡 **Consejo:** HashSet es el más rápido (O(1)), TreeSet ordena (O(log n)) y LinkedHashSet recuerda el orden (O(1) con un pelín más de memoria). Cada uno con su superpoder.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** ante “sin duplicados y ordenados” piensa en TreeSet. Ante “sin duplicados pero manteniendo el orden de llegada” piensa en LinkedHashSet. Ante “solo sin duplicados” piensa en HashSet.

Ejercicio: ¿qué sale por pantalla?

```
import java.util.TreeSet;

public class BeTheTree {
    public static void main(String[] args) {
        TreeSet<Integer> notas = new TreeSet<>();
        notas.add(7);
        notas.add(3);
        notas.add(9);
        notas.add(7);   // duplicado: no entra
        notas.add(1);

        System.out.println(notas.first() + " " + notas.last());
        System.out.println(notas.size());
    }
}
```

¿Qué imprime?

- (A) 1 9 y 5
- (B) 3 9 y 4
- (C) 1 9 y 4
- (D) 1 7 y 4

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuántos elementos hay en un `HashSet` tras añadir `a, b, a, c, b`?
2. ¿Qué diferencia a `TreeSet` de `HashSet`?
3. ¿Puede un `TreeSet` contener un `null`? ¿Y un `HashSet`?
4. ¿Qué colección elimina duplicados **manteniendo el orden de inserción**?
5. ¿Qué devuelve `headSet("Bob")` en un TreeSet con `{"Ana", "Bob", "Zara"}`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **HashSet** es el portero sin duplicados: no admite repetidos, no ordena nada y es **O(1)** gracias a `hashCode()` y `equals()`.
2. **TreeSet** ordena todo lo que metes automáticamente y ofrece extras (`first`, `last`, `headSet`, `subSet`), a cambio de ser más lento y no aceptar `null`.
3. **LinkedHashSet** conserva el orden de inserción y es la solución estrella para **eliminar duplicados manteniendo el orden de llegada**.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `Set` | Colección sin duplicados |
> | `hashCode()` | "Número de cajón" que usa HashSet para encontrar elementos |
> | Árbol rojo-negro | Estructura interna de TreeSet que mantiene el orden |
> | Orden natural | El orden por defecto del tipo (numérico, alfabético) |
> | Orden de inserción | El orden en el que se añadieron los elementos |
> | O(1) / O(log n) | Instantáneo / logarítmico (rápido aunque crezca) |

📚 Volver al índice de la unidad · **Anterior:** 02 · LinkedList: la conga line · **Siguiente:** 04 · Iterator y Collections

04 · Iterator y Collections: recorrer y manipular

El camarero que recorre cualquier colección y borra sin romper nada, y la navaja suiza que ordena, revuelve y consulta en una línea 🍽️🧰

> 🗺️ **Estás en:** 📚 **U12 · Colecciones, genéricos y mapas** → 04 · Iterator y Collections

---

📬 La idea en una frase

> **`Iterator` recorre cualquier colección y su `remove()` es la única forma segura de borrar durante el recorrido; `Collections` es la navaja suiza de métodos estáticos que hace en una línea lo que tú tardarías en diez.**

En los puntos anteriores recorriste listas y conjuntos con `for-each` sin preguntarte cómo funcionaban por dentro. Ese `for-each` tiene un superhéroe trabajando por debajo: el `Iterator`. Y una vez sabes recorrer, toca manipular sin escribir bucles a mano: para eso está `Collections`, la hermana de la clase `Arrays` que viste en la U06.

🍽️ Iterator: el camarero que toma nota

`Iterator` recorre una colección **sin que sepas ni te importe cómo está implementada por dentro**. Es como un camarero: "¿Qué quiere? ¿Y usted? ¿Y usted?"

```java
import java.util.ArrayList;
import java.util.Iterator;

public class EjemploIterator {
    public static void main(String[] args) {
        ArrayList<String> platos = new ArrayList<>();
        platos.add("Tortilla");
        platos.add("Paella");
        platos.add("Croquetas");

        Iterator<String> it = platos.iterator();
        while (it.hasNext()) {          // ¿queda algo?
            String plato = it.next();   // dámelo
            System.out.println(plato);
        }
    }
}
```

El protocolo del camarero:

Método	Qué hace
`hasNext()`	¿Hay más elementos? (`boolean`)
`next()`	Devuelve el siguiente y avanza
`remove()`	Borra el último elemento devuelto por `next()`

> 💡 **Consejo:** el `for-each` usa un `Iterator` por debajo. Por eso puedes usar `for-each` con cualquier colección: todas lo implementan. El for-each es el atajo; el `Iterator` es la herramienta con superpoderes (borrar).

💥 La superpotencia: borrar durante el recorrido

Aquí está la joya. **Nunca** hagas `lista.remove(elemento)` mientras usas un `for-each`:

```java
ArrayList<String> platos = new ArrayList<>();
platos.add("Tortilla");
platos.add("Paella");
platos.add("Croquetas");

// MAL: ConcurrentModificationException
for (String plato : platos) {
    if (plato.equals("Paella")) {
        platos.remove(plato);   // BOOM
    }
}
```

> ⚠️ **Advertencia:** borrar con `lista.remove()` mientras un for-each recorre la lista lanza `ConcurrentModificationException`. El for-each no sabe que la lista ha cambiado por debajo y se vuelve loco. Usa SIEMPRE `iterator.remove()` si necesitas borrar durante el recorrido.

La forma correcta:

```java
Iterator<String> it = platos.iterator();
while (it.hasNext()) {
    String plato = it.next();
    if (plato.equals("Paella")) {
        it.remove(); // BORRA de la lista ORIGINAL, sin excepción
    }
}
// Ahora platos = [Tortilla, Croquetas]
```

`it.remove()` borra el elemento que acaba de devolver `next()` de la colección original. Es la única forma segura de "borrar mientras recorres".

🧰 Collections: el amigo utilitario

```java
import java.util.ArrayList;
import java.util.Collections;

public class EjemploCollections {
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        lista.add("Zara");
        lista.add("Ana");
        lista.add("Bob");

        Collections.sort(lista);        // [Ana, Bob, Zara]
        Collections.reverse(lista);     // [Zara, Bob, Ana]
        Collections.shuffle(lista);     // orden aleatorio
        String max = Collections.max(lista);    // "Zara" (orden alfabético)
        String min = Collections.min(lista);    // "Ana"
        int veces = Collections.frequency(lista, "Ana"); // cuántas veces aparece
        Collections.replaceAll(lista, "Ana", "Ana María");
        Collections.rotate(lista, 2);   // rota 2 posiciones
    }
}
```

El repertorio esencial:

Método	Qué hace
`sort(lista)`	Ordena la lista en el sitio
`reverse(lista)`	Le da la vuelta
`shuffle(lista)`	Mezcla al azar
`max(lista)` / `min(lista)`	El mayor / el menor
`frequency(lista, e)`	Cuántas veces aparece `e`
`replaceAll(lista, viejo, nuevo)`	Sustituye todas las apariciones
`rotate(lista, k)`	Rota los elementos `k` posiciones
`binarySearch(lista, e)`	Busca (lista ordenada)
`copy(destino, origen)`	Copia una lista en otra

> 📝 **Nota:** `Collections.sort()` ordena la **lista original** (la modifica). Si no quieres tocar la tuya, cópiala antes: `ArrayList<String> copia = new ArrayList<>(lista);` y ordena la copia.

⚠️ Los métodos que modifican tu lista

Casi todos los métodos de `Collections` que empiezan con un verbo ("sort", "shuffle", "reverse", "rotate"…) **modifican la lista que les pasas**. No devuelven una nueva: cambian la original.

```
ArrayList<Integer> nums = new ArrayList<>();
nums.add(3);
nums.add(1);
nums.add(2);

Collections.sort(nums);      // nums AHORA es [1, 2, 3]
System.out.println(nums);    // [1, 2, 3] - la original, modificada
```

> ⚠️ **Advertencia:** si necesitas conservar la lista original, haz siempre una copia antes: `new ArrayList<>(lista)`. Es el error típico de quien ordena sin querer su lista de la compra.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** `it.remove()` borra el último elemento devuelto por `next()`. Nunca llames a `remove()` sin un `next()` justo antes. Y recuerda: los métodos estáticos de `Collections` van a por el original.

Ejercicio 1: ¿qué imprime (o qué lanza)?

```java
import java.util.ArrayList;

public class BeTheIterator {
    public static void main(String[] args) {
        ArrayList<String> nombres = new ArrayList<>();
        nombres.add("Ana");
        nombres.add("Bob");
        nombres.add("Carla");

        for (String n : nombres) {
            if (n.equals("Bob")) {
                nombres.remove(n);
            }
        }
        System.out.println(nombres);
    }
}
```

¿Qué ocurre?

- (A) Imprime `[Ana, Carla]`
- (B) Imprime `[Ana, Bob, Carla]`
- (C) Lanza `ConcurrentModificationException`
- (D) No compila

> 🔄 **Solución**

Ejercicio 2: Collections en acción

```java
import java.util.*;

public class BeTheCollections {
    public static void main(String[] args) {
        ArrayList<Integer> nums = new ArrayList<>();
        nums.add(5);
        nums.add(1);
        nums.add(8);
        nums.add(3);

        Collections.sort(nums);
        Collections.reverse(nums);

        System.out.println(nums.get(1));
    }
}
```

¿Qué imprime?

- (A) 1
- (B) 3
- (C) 5
- (D) 8

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué excepción lanza borrar con `lista.remove()` dentro de un `for-each`?
2. ¿Cómo se borra un elemento mientras se recorre con `Iterator`?
3. ¿`Collections.sort()` modifica la lista original?
4. ¿Qué método de `Collections` devuelve el elemento mayor según el orden natural?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **Iterator** recorre cualquier colección con `hasNext()` / `next()`, y su `remove()` es la única forma segura de **borrar durante el recorrido**.
2. **Collections** es la navaja suiza estática: `sort`, `reverse`, `shuffle`, `max`, `min`, `frequency`…
3. La mayoría de los métodos de `Collections` **modifican la lista original**: cópiate antes si la necesitas intacta.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Iterador | Objeto que recorre una colección sin exponer su estructura |
> | `ConcurrentModificationException` | Error al modificar la colección mientras se recorre |
> | Clase utilitaria | Clase de métodos estáticos que no se instancia |
> | Modificar en el sitio | Cambiar la propia colección (no una copia) |
> | `shuffle` | Mezclar al azar |
> | `frequency` | Contar apariciones |

📚 Volver al índice de la unidad · **Anterior:** 03 · HashSet, TreeSet y LinkedHashSet · **Siguiente:** 05 · Tu propia clase genérica: `Caja<T>`

05 · Clases genéricas: tu propia `Caja<T>`

De dónde sale el `` de ArrayList, las convenciones T/E/K/V y tu primera clase con huecos rellenables 📦

> 🗺️ **Estás en:** 📚 **U12 · Colecciones, genéricos y mapas** → 05 · Clases genéricas: tu propia `Caja<T>`

📬 La idea en una frase

> **Los genéricos existen por una sola razón: seguridad de tipos. Y una clase genérica es una plantilla con huecos: declaras `<T>` y quien la use decide qué tipo rellena cada hueco.**

En los puntos 1 y 2 llevas todo el día usando `ArrayList<String>` y `HashSet<Integer>`… pero nadie te ha explicado de dónde sale ese `<String>`. Hoy es el día.

😱 El infierno antes de los genéricos

Una `ArrayList` a la antigua usanza (sin genéricos) es una caja de caos donde puedes meter cualquier cosa. Al sacar, Java te devuelve un `Object` y tú tienes que recordar qué metiste:

```
import java.util.*;

ArrayList cajaDeCaos = new ArrayList();   // sin <String>, sin <Integer>...
cajaDeCaos.add(42);
cajaDeCaos.add("Hola");

Object cosa = cajaDeCaos.get(0);
String texto = (String) cosa;  // 💥 ClassCastException en tiempo de ejecución
```

> ⚠️ **Advertencia:** sin genéricos, los errores de tipo saltan en tiempo de EJECUCIÓN (cuando el usuario usa tu programa). Con genéricos, saltan en tiempo de COMPILACIÓN (mientras programas). ¿Cuándo prefieres enterarte?

🤝 Llegan los genéricos

Una `ArrayList<String>` ya no es una caja de caos: es una máquina expendedora que SOLO da Coca-Colas. Si intentas meter un `int`, el compilador te para el brazo. Y al sacar, `get(0)`

devuelve un `String` directamente: sin casting y sin cruzar los dedos.

El `<String>` es un **parámetro de tipo**: una promesa que le haces al compilador y que él se encarga de cumplir.

🔤 Las convenciones: T, E, K, V, N

El parámetro de tipo se escribe entre `<` `>` y, por convención, se usan mayúsculas con significado implícito:

- `T` → Tipo (Type), el comodín general.
- `E` → Elemento (Element), para colecciones.
- `K` / `V` → Clave / Valor (Key / Value), para mapas.
- `N` → Número (Number).

📦 La clase `Caja<T>`

Una caja que guarda UN objeto de cualquier tipo:

```
public class Caja<T> {
    private T contenido;

    public void guardar(T contenido) {
        this.contenido = contenido;
    }

    public T sacar() {
        return contenido;
    }

    public boolean estaVacia() {
        return contenido == null;
    }
}
```

Fíjate en el patrón: `T` aparece como tipo del atributo, del parámetro de `guardar` y del retorno de `sacar`. Cuando alguien escribe `Caja<String>`, todas las `T` se convierten en `String` para esa instancia:

```
Caja<String> cajaDeTexto = new Caja<>();
cajaDeTexto.guardar("Mensaje secreto");
String mensaje = cajaDeTexto.sacar();  // Sin casting, directo al pelo

Caja<Integer> cajaDeNumeros = new Caja<>();
cajaDeNumeros.guardar(42);
Integer numero = cajaDeNumeros.sacar();
```

`cajaDeTexto` y `cajaDeNumeros` salen del mismo molde, pero para el compilador son tipos distintos: no puedes mezclarlas sin que te pille.

> ⚠️ **Advertencia:** no puedes usar primitivos como parámetro de tipo. `Caja<int>` no compila: usa `Caja<Integer>` con su clase envolvente y deja que el autoboxing haga la conversión automática (lo viste en el punto 1).

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** el tipo `T` es un comodín que se reemplaza al instanciar. Pero ojo: la caja solo guarda UN elemento; el segundo `guardar` pisa al primero.

Ejercicio: la caja que solo recuerda lo último

```
public class BeTheCaja {
    public static void main(String[] args) {
        Caja<Integer> caja = new Caja<>();
        caja.guardar(5);
        caja.guardar(10);
        System.out.println(caja.sacar());
    }
}
```

¿Qué imprime?

- (A) 5
- (B) 10
- (C) null
- (D) Error de compilación

> › 🔄 **Solución**

🤷 No hay preguntas tontas

> ❓ **¿Puedo tener una clase con dos tipos genéricos?**

Sí, y es muy común. `Pareja<T, U>` con `private T primero; private U segundo;` es el ejemplo clásico (lo machacarás en los boletines). Los parámetros de tipo van separados por comas entre los `< >`.

> ❓ **¿`Caja` y `Caja<T>` son la misma clase?**

Casi, pero no. `Caja` a secas (sin `< >`) es un **tipo crudo (raw type)**: funciona como el infierno del principio, sin seguridad de tipos. Nunca lo uses salvo que te obliguen los restos de código antiguo.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué excepción evitamos gracias a los genéricos al sacar elementos de una colección?
2. ¿Qué letra se usa por convención para el parámetro de tipo de una colección? ¿Y para clave y valor de un mapa?
3. ¿Por qué no existe `Caja<int>`?
4. ¿Qué le pasa al 5 después de llamar a `guardar(10)` en una `Caja<Integer>`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los genéricos traen **seguridad de tipos**: comprueban en compilación que no metas la pata y eliminan los castings a ciegas.
2. Una **clase genérica** se declara con `<T>` y usa `T` como un hueco rellenable en atributos, parámetros y retornos; las convenciones (`T`, `E`, `K`/`V`, `N`) hacen que el código se lea solo.
3. **Los primitivos no valen**: usa las clases wrapper (`Integer` en vez de `int`) y deja que el autoboxing convierta solo.

🦎 Vocabulario rápido

Término	Idea general
Genérico	Código que trabaja con un tipo que se decide al usarlo (`<T>`)
Parámetro de tipo	El `<String>` de `ArrayList<String>`: el tipo que se pasa
Clase genérica	Clase que declara `<T>` y usa el tipo como hueco rellenable
Wrapper	Clase que envuelve un primitivo (`Integer` envuelve `int`)
Tipo crudo (raw type)	`Caja` sin `< >`: sin seguridad de tipos, evítalo

📚 Volver al índice de la unidad · **Anterior:** 04 · Iterator y Collections · **Siguiente:** 06 · El diamante, métodos genéricos y wildcards

06 · El diamante, métodos genéricos y wildcards

El perezoso oficial que infiere el tipo, métodos con su propio `` y el principio PECS de Joshua Bloch 💎

> 🗺️ **Estás en:** 📚 **U12 · Colecciones, genéricos y mapas** → 06 · El diamante, métodos genéricos y wildcards

---

📬 La idea en una frase

> **El diamante `<>` te ahorra repetir el tipo, un método genérico declara su propio `<T>` antes del retorno, y los wildcards (`? extends` / `? super`) resuelven la invarianza de los genéricos con la regla PECS.**

Tres piezas que completan el puzzle genérico: la primera te ahorra teclear, la segunda te da flexibilidad en utilidades y la tercera es la que cae en las entrevistas.

💎 El operador diamante <>

Desde Java 7 no hace falta repetir el tipo dos veces; el compilador lo infiere:

```
// Antes de Java 7 (repetitivo):
Caja<String> caja1 = new Caja<String>();

// Desde Java 7 (el diamante <> al rescate):
Caja<String> caja2 = new Caja<>();  // "ya sabes de qué hablo, ¿no?"
```

> 💡 **Consejo:** usa siempre el diamante. El compilador mira el lado izquierdo y deduce el derecho: código más limpio y cero repeticiones.

🎩 Type erasure: el mago se lleva los genéricos

Los genéricos SOLO existen en compilación. Al generar el bytecode, el compilador **verifica** los tipos, **borra** la información genérica y **añade** los castings donde hagan falta:

```
// En tu código fuente:
ArrayList<String> nombres = new ArrayList<>();

// En el bytecode tras compilar:
ArrayList nombres = new ArrayList();   // sin genéricos, con castings
```

Por eso dos `ArrayList` de tipos distintos son la misma clase en tiempo de ejecución, y por eso no puedes hacer `new T()` ni `new T[]`: en runtime, `T` no existe.

🧰 Métodos genéricos

Un método puede declarar su propio `<T>` **antes del tipo de retorno**, sin que la clase sea genérica. El compilador lo deduce de los argumentos:

```
public class Utilidades {

    // Declara <T> antes del tipo de retorno
    public static <T> void imprimir(T elemento) {
        System.out.println("Elemento: " + elemento);
    }

    // Con límite: T debe saber compararse (implementar Comparable)
    public static <T extends Comparable<T>> T maximo(T a, T b) {
        return a.compareTo(b) > 0 ? a : b;
    }
}

// Uso con inferencia:
Utilidades.imprimir(42);        // T = Integer
Utilidades.imprimir("Hola");    // T = String
Utilidades.maximo(5, 8);        // 8
Utilidades.maximo("gato", "perro"); // "perro"
```

El **límite** (`T extends Comparable<T>`) convierte la promesa genérica en contrato: si tu clase no sabe compararse, el compilador no te deja pasarla. Y aunque casi nunca hace falta, existe la sintaxis explícita `Utilidades.<String>maximo(...)` (suele salir en exámenes).

🃏 Wildcards: el problema de la invarianza

Momento incómodo: `List<Number>` NO acepta una `List<Integer>`, aunque un Integer sea un Number. Los genéricos son **invariantes**. La solución es el comodín `?`:

`? extends T`: para LEER (covarianza)

```
public static double sumar(List<? extends Number> numeros) {
    double total = 0.0;
    for (Number n : numeros) {
        total += n.doubleValue();   // leer: sí
    }
    // numeros.add(42);             // 🚫 añadir: no (salvo null)
    return total;
}
// sumar(List.of(1, 2, 3)) y sumar(List.of(1.5, 2.5)) compilan ✅
```

`? super T`: para ESCRIBIR (contravarianza)

```
public static void rellenar(List<? super Integer> lista) {
    lista.add(1);
    lista.add(2);          // escribir: sí
    // Integer n = lista.get(0);  // 🚫 leer tipado: no, solo como Object
}
// rellenar(new ArrayList<Number>()) y rellenar(new ArrayList<Object>()) compilan ✅
```

🧠 PECS: la mnemotecnia infalible

Joshua Bloch (autor de *Effective Java*) lo resumió en cuatro letras:

> - **P**roducer **E**xtends: si el método solo **produce** (lee) datos, usa `? extends`.
> - **C**onsumer **S**uper: si el método solo **consume** (escribe) datos, usa `? super`.
> - ¿Hace las dos cosas? → no uses wildcard: usa `T` directamente.

> ⚠️ **Advertencia:** pregunta clásica de examen: "¿puedo añadir a una `List<? extends Number>`?" NO (salvo `null`). "¿Puedo leer tipado de una `List<? super Integer>`?" Tampoco: solo como `Object`. PECS te salva de ambas.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** gracias al type erasure, los genéricos no existen en runtime: `nombres.getClass() == numeros.getClass()` es `true` aunque uno sea `ArrayList<String>` y otro `ArrayList<Integer>`.

Ejercicio: el comodín que lo acepta todo

```
import java.util.*;

public class BeTheWildcard {
    public static void main(String[] args) {
        printNumbers(Arrays.asList(1, 2, 3));      // ¿compila?
        printNumbers(Arrays.asList(1.5, 2.5, 3.5)); // ¿compila?
    }

    public static void printNumbers(List<? extends Number> lista) {
        for (Number n : lista) {
            System.out.print(n + " ");
        }
    }
}
```

¿Cuántas llamadas compilan?

- (A) 0
- (B) 1
- (C) 2
- (D) Error en ambas

> › 🔄 **Solución**

🤷 No hay preguntas tontas

> ❓ **¿Los genéricos ralentizan mi programa?**

No. El **type erasure** los convierte en castings normales al compilar: es solo azúcar sintáctico. En runtime no hay genéricos ni coste extra.

> ❓ **¿Por qué no puedo hacer `new T()`?**

Porque en compilación Java no sabe qué es `T` y en runtime ya se ha borrado. Es como pedirle a un pastelero "un pastel" sin decirle de qué.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Desde qué versión de Java existe el operador diamante `<>`?
2. ¿Dónde se declara el `<T>` de un método genérico?
3. ¿Qué significa `? extends Number` y para qué sirve?
4. ¿Qué dice la C de PECS?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. El **diamante** `<>` (Java 7+) infiere el tipo del lado derecho, y el **type erasure** borra los genéricos al compilar: por eso son gratis y no existen en runtime.
2. Un **método genérico** declara `<T>` antes del retorno y puede llevar un **límite** (`T extends Comparable<T>`) que actúa como contrato.
3. Los **wildcards** resuelven la invarianza: `? extends` para leer, `? super` para escribir, y **PECS** (*Producer Extends, Consumer Super*) para recordarlo.

> 🦎 **Vocabulario rápido**

Término	Idea general
Diamante	El `<>` que infiere el tipo y evita repetirlo
Type erasure	Proceso que borra los genéricos al compilar
Límite (bound)	Restricción: `T extends Comparable<T>`
Wildcard	El `?`: "cualquier tipo"
Covarianza / Contravarianza	`? extends` (leer) / `? super` (escribir)

Término	Idea general
PECS	Producer Extends, Consumer Super

📚 Volver al índice de la unidad · **Anterior:** 05 · Clases genéricas: tu propia `Caja<T>` · **Siguiente:** 07 · HashMap: la guía telefónica

07 · HashMap: la guía telefónica

Asocia claves con valores, el put que sobrescribe y las tres formas de recorrerlo ☎️

> 🗺️ **Estás en:** 📚 **U12 · Colecciones, genéricos y mapas** → 07 · HashMap: la guía telefónica

📬 La idea en una frase

> **Un `Map` asocia claves únicas con valores: buscas por clave (nombre) y obtienes su valor (teléfono). `HashMap` lo hace en O(1) y, si repites clave, el `put` sobrescribe el valor anterior.**

Hasta aquí solo guardabas elementos sueltos en listas y conjuntos. Pero la vida real está llena de asociaciones: nombre → teléfono, palabra → veces que aparece, matrícula → plaza de parking. Para eso existen los **mapas**, la tercera familia de estructuras junto a `List` y `Set`. Bienvenido a la guía telefónica.

☎️ HashMap: clave → valor

`HashMap` asocia claves con valores. Como una agenda: buscas por nombre (clave) y obtienes el teléfono (valor).

```
import java.util.HashMap;

HashMap<String, Integer> agenda = new HashMap<>();
agenda.put("Ana", 612345678);
agenda.put("Bob", 698765432);
agenda.put("Ana", 600000000);  // Sobrescribe el anterior

int telefono = agenda.get("Ana");          // 600000000
Integer inexistente = agenda.get("NoExisto"); // null
agenda.containsKey("Bob");        // true
agenda.containsValue(600000000);  // true
agenda.remove("Bob");             // borra la entrada
agenda.size();                    // 1 (solo queda Ana)
```

Fíjate en las tres ideas clave:

1. **Las claves son únicas.** `put("Ana", 612345678)` y luego `put("Ana", 600000000)`: el segundo sobrescribe al primero. Es como tachar un número de tu agenda para poner otro: solo queda el último.
2. **`get` devuelve `null` si la clave no existe.** No lanza excepción, pero cuidado: si la variable es `int` (primitivo), asignar `null` da error. Por eso existe `getOrDefault`.
3. **Búsqueda rapidísima**: O(1) de media. Da igual si hay 10 o 10.000 entradas.

> 📝 **Nota:** las claves de un HashMap deben ser **inmutables**. Por eso `String` e `Integer` son perfectos. Si usas un objeto mutable como clave y luego lo modificas, su `hashCode()` cambia y el HashMap no lo encontrará más. Es como cambiar la cerradura y esperar que la llave vieja siga funcionando.

🛟 `getOrDefault`: el salvavidas de los null

En lugar de escribir el clásico `if (map.get(clave) != null)`, haces:

```
int edadCarlos = agenda.getOrDefault("Carlos", 0);  // 0 si no existe
```

Es como un plan B: “si no encuentras a Carlos, dame 0”. Lo usarás muchísimo para contar frecuencias, como verás en el Sé el Código.

🚶 Recorrer un HashMap: tres caminos

Un mapa tiene tres vistas para recorrer: las claves, los valores o las entradas completas.

```java
HashMap<String, Integer> agenda = new HashMap<>();
agenda.put("Ana", 612345678);
agenda.put("Bob", 698765432);

// 1) Solo las claves
for (String nombre : agenda.keySet()) {
    System.out.println(nombre + " → " + agenda.get(nombre));
}

// 2) Solo los valores
for (Integer telefono : agenda.values()) {
    System.out.println("Tel: " + telefono);
}

// 3) Las entradas completas (clave + valor juntos) — la preferida
for (HashMap.Entry<String, Integer> entrada : agenda.entrySet()) {
    System.out.println(entrada.getKey() + " → " + entrada.getValue());
}
```

La tercera es la favorita de los profesionales: `entrySet()` te da pares clave-valor sin tener que hacer un `get` extra dentro del bucle. En los exámenes y en el trabajo, esta es la que verás una y otra vez.

> 💡 **Consejo:** para recorrer, declara el mapa como `Map<String, Integer>` y usa `Map.Entry<K, V>` en el bucle: así, si mañana cambias a `TreeMap`, no tocas nada. Programa contra la interfaz, como en la U12.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** HashMap no garantiza orden. Si necesitas orden, usa TreeMap o LinkedHashMap (punto 7). Y ojo con `put`: repetir clave sobrescribe, no añade.

Ejercicio 1: el HashMap traicionero

```java
import java.util.HashMap;

public class BeTheMap {
    public static void main(String[] args) {
        HashMap<String, String> capitales = new HashMap<>();
        capitales.put("España", "Madrid");
        capitales.put("Francia", "París");
        capitales.put("Italia", "Roma");
        capitales.put("España", "Barcelona");

        System.out.println(capitales.get("España"));
    }
}
```

¿Qué imprime?

- (A) Madrid
- (B) Barcelona
- (C) null
- (D) Error

> 🔄 **Solución**

Ejercicio 2: la frecuencia de letras

> 👓 **Don Tip:** `getOrDefault()` evita el null check. Úsalo siempre que puedas.

```java
import java.util.HashMap;

public class BeTheFrequency {
    public static void main(String[] args) {
        String texto = "banana";
        HashMap<Character, Integer> frec = new HashMap<>();
        for (char c : texto.toCharArray()) {
            frec.put(c, frec.getOrDefault(c, 0) + 1);
        }
        System.out.println(frec.get('a') + " " + frec.get('n'));
    }
}
```

¿Qué imprime?

- (A) 1 1
- (B) 3 1
- (C) 3 2
- (D) 2 2

> 🔄 Solución

🤷 No hay preguntas tontas

> ❓ ¿Puedo tener un HashMap con clave `null`?

Sí, HashMap admite **una** clave `null` (la guarda en una posición especial). TreeMap no lo admite. Y los valores `null` sí se permiten en ambos.

> ❓ ¿Qué pasa si la clave no existe en el mapa?

`get()` devuelve `null`. No lanza excepción. Úsalo con cuidado o, mejor, usa `getOrDefault(clave, valorPorDefecto)`, que te devuelve un valor seguro.

> ❓ ¿Y si necesito asociar varias cosas a la misma clave?

Usa un mapa de listas: `HashMap<String, ArrayList<Integer>>`. La clave apunta a una lista y cada `put` bien hecho añade a la lista existente. Es la estructura que usarás para "una clave, muchos valores".

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué devuelve `get(clave)` si la clave no existe?
2. ¿Qué hace `put("Ana", x)` si la clave "Ana" ya existe?
3. ¿Qué método devuelve un valor por defecto si la clave no existe?
4. ¿Cuál es el bucle preferido para recorrer un mapa y por qué?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. **`Map`** asocia claves únicas con valores: `put` añade (o sobrescribe), `get` consulta y devuelve `null` si la clave no existe.
2. Se recorre con **`keySet()`** (claves), **`values()`** (valores) o **`entrySet()`** (pares), siendo esta última la preferida.
3. Las claves deben ser **inmutables** y el `getOrDefault()` es tu salvavidas contra los `null`, especialmente para contar frecuencias.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `Map` | Interfaz que asocia claves con valores |
> | Clave / valor | El par asociado; la clave es única |
> | `put` | Añadir o sobrescribir una entrada |
> | `getOrDefault` | Devolver un valor seguro si la clave no existe |
> | `entrySet` | Las entradas (clave + valor) para recorrer |

📚 Volver al índice de la unidad · **Anterior:** 06 · El diamante, métodos genéricos y wildcards · **Siguiente:** 08 · TreeMap y cómo elegir estructura

08 · TreeMap y cómo elegir estructura

El ordenado de serie, el que recuerda el orden de llegada y los métodos de navegación 🌳

> 🗺️ **Estás en:** 📚 **U12 · Colecciones, genéricos y mapas** → 08 · TreeMap y cómo elegir estructura

---

📬 La idea en una frase

> **`HashMap` es rapidísimo pero no ordena, `TreeMap` ordena las claves automáticamente (O(log n)) y `LinkedHashMap` recuerda el orden de inserción. Eliges según lo que necesites.**

En el punto 6 conociste al rey de la velocidad. Pero a veces la velocidad no basta: necesitas que las claves salgan ordenadas, o que se respete el orden en el que las metiste. Aquí están sus dos primos, con sus superpoderes y sus debilidades.

🌳 TreeMap: el ordenado de serie

`TreeMap` es un mapa ordenado por clave. Internamente usa un árbol rojo-negro, que es como una biblioteca que se reordena sola cada vez que añades un libro.

```
import java.util.TreeMap;

TreeMap<String, Integer> ordenado = new TreeMap<>();
ordenado.put("Zara", 30);
ordenado.put("Ana", 25);
ordenado.put("Bob", 35);
System.out.println(ordenado);  // {Ana=25, Bob=35, Zara=30} — orden alfabético

// Métodos de navegación extra
ordenado.firstKey();    // "Ana" — la primera clave
ordenado.lastKey();     // "Zara" — la última clave
ordenado.headMap("Bob");  // {Ana=25} — entradas antes de "Bob"
ordenado.subMap("Ana", "Zara"); // {Ana=25, Bob=35} — entre claves
```

La gracia está en los métodos de navegación: `firstKey`, `lastKey`, `headMap`, `subMap`… son la razón de ser del TreeMap. Con un HashMap, para “las claves que van antes que Bob” tendrías que copiar y ordenar a mano.

> ⚠️ **Advertencia:** TreeMap NO admite una clave `null` (no sabe compararla con nada). Y las claves deben ser comparables: por eso `String` e `Integer` funcionan, y las clases tuyas sin `Comparable` fallan en tiempo de ejecución.

🔗 LinkedHashMap: el que recuerda el orden de llegada

`LinkedHashMap` es un HashMap que, además, mantiene el orden de inserción. Es como un HashMap con memoria de lo que fuiste metiendo.

```
import java.util.LinkedHashMap;

LinkedHashMap<String, Integer> conMemoria = new LinkedHashMap<>();
conMemoria.put("Zara", 30);
conMemoria.put("Ana", 25);
conMemoria.put("Bob", 35);
System.out.println(conMemoria);  // {Zara=30, Ana=25, Bob=35} — orden de inserción
```

¿Cuándo lo usas? Cuando quieres la velocidad del HashMap pero necesitas saber "en qué orden llegó cada cosa": un historial, una lista de "lo último visto"... Recuerda: **orden de inserción**, no orden natural.

⚖️ La familia al completo

Mapa	Orden	Velocidad	¿Clave `null`?
`HashMap`	Ninguno	O(1)	Sí (una)
`TreeMap`	Natural (por clave)	O(log n)	No
`LinkedHashMap`	De inserción	O(1)	Sí (una)

Regla práctica: ¿velocidad y no te importa el orden? → `HashMap`. ¿Claves siempre ordenadas o navegación (`firstKey`, `headMap`...)? → `TreeMap`. ¿Velocidad pero respetando el orden de llegada? → `LinkedHashMap`.

🥊 EL RING: HashMap vs TreeMap

Dos implementaciones de Map se enfrentan frente a la máquina de café.

HashMap: — Yo soy el rey de la velocidad. O(1) en `get` y `put`. No me importa el orden, me importa la rapidez.

TreeMap: — Sí, pero yo mantengo las claves ordenadas automáticamente. Si necesitas recorrerlas en orden alfabético, soy tu único amigo.

HashMap: — ¿Ordenado? Eso cuesta. Yo soy O(1) en todo. ¡Soy imbatible en rendimiento!

TreeMap: — Cierto, pero puedo navegar: `firstKey()`, `lastKey()`, `subMap()`, `headMap()`. Tú, para todo eso, tienes que copiar y ordenar.

HashMap: — Si no necesitas orden, ¿para qué pagar el coste? La mayoría de los casos usan HashMap.

TreeMap: — Y cuando necesitan orden, ahí estoy yo. Y no soy tan lento: O(log n) sigue siendo muy rápido para la mayoría de los casos.

HashMap: — Tregua. Cada uno en su sitio.

TreeMap: — Hecho.

> La lección: **¿velocidad? → HashMap. ¿orden natural? → TreeMap. ¿orden de inserción? → LinkedHashMap.** Cada uno tiene su superpoder, y la respuesta a "¿cuál es mejor?" siempre es "depende".

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** TreeMap ordena por clave en el momento del `put`. No ordenas tú: él ordena solo.

Ejercicio: la biblioteca que se ordena sola

```java
import java.util.TreeMap;

public class BeTheTreeMap {
    public static void main(String[] args) {
        TreeMap<String, Integer> m = new TreeMap<>();
        m.put("Zara", 30);
        m.put("Ana", 25);
        m.put("Bob", 35);

        System.out.println(m.firstKey());
        System.out.println(m.headMap("Bob"));
    }
}
```

¿Qué imprime?

- (A) Zara / {Zara=30}
- (B) Ana / {Ana=25}
- (C) Ana / {Ana=25, Bob=35}
- (D) Zara / {Zara=30, Ana=25}

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué mapa ordena las claves automáticamente?
2. ¿Qué mapa mantiene el orden de inserción?
3. ¿Qué método devuelve las entradas con clave menor que una dada?
4. ¿Puede un TreeMap tener una clave `null`?

> 🔄 **Respuestas**

⚖️ Map vs List vs Set: la comparativa definitiva

Característica	List	Set	Map
¿Qué guarda?	Elementos ordenados por posición	Elementos únicos	Pares clave → valor
¿Duplicados?	Sí	No	Claves no, valores sí
¿Orden?	De inserción	Depende (Hash/Tree)	Depende (Hash/Tree)
Acceso	Por índice	Por elemento	Por clave
¿Nulls?	Sí	HashSet: 1, TreeSet: 0	HashMap: 1 clave, TreeMap: 0
Implementación principal	`ArrayList`	`HashSet`	`HashMap`

La gran diferencia: List y Set guardan elementos sueltos; Map guarda parejas. Si tu dato necesita una “clave” para buscarse después (un DNI, un nombre, una palabra), es un Map. Si solo necesitas una colección de cosas, es List o Set.

🧭 El árbol de decisiones

```
¿Tienes parejas clave → valor?
├── Sí → ¿Necesitas orden por clave?
│        ├── Sí → TreeMap
│        └── No → ¿Orden de llegada?
│                 ├── Sí → LinkedHashMap
│                 └── No → HashMap
└── No → ¿Me importa el orden?
         ├── No → ¿Duplicados?
         │        ├── No → HashSet
         │        └── Sí → ArrayList
         └── Sí → ¿Qué orden?
                  ├── Natural → ¿Duplicados?
                  │        ├── No → TreeSet
                  │        └── Sí → ArrayList + sort
                  └── De llegada → ¿Duplicados?
                           ├── No → LinkedHashSet
                           └── Sí → ArrayList
```

🥇 La regla práctica

- ¿Una lista de cosas que recorres por posición? → `ArrayList`
- ¿Cosas sin repetir y no te importa el orden? → `HashSet`
- ¿Cosas sin repetir y siempre ordenadas? → `TreeSet`
- ¿Asociar una cosa con otra y buscar por clave? → `HashMap`
- ¿Claves ordenadas o navegación (`firstKey`, `headMap`)? → `TreeMap`
- ¿Velocidad pero respetando el orden de llegada? → `LinkedHashMap`

La regla de oro: ¿el dato se busca solo por su contenido? → Set. ¿Por una posición? → List. ¿Por una clave distinta de él mismo? → Map. Esa última frase separa al que memoriza de quien entiende.

¿Qué estructura usarías? Para cada escenario, elige y justifica en una línea:

1. Asociar cada alumno (por su nombre) con su nota de examen.
2. Las notas de la clase, para recorrerlas muchas veces por posición.
3. Los DNI de los matriculados, sin repetir y en orden alfabético.
4. Saber si una palabra ya ha salido en un sorteo, con 100.000 palabras.
5. El número de veces que aparece cada letra en un texto, mostradas en orden alfabético.

✅ Resumen en 3 frases (ampliado)

1. **List** guarda elementos por posición (duplicados sí), **Set** guarda únicos y **Map** asocia claves únicas con valores: la primera pregunta siempre es “¿necesito asociar?”.
2. **TreeMap** ordena sus claves y añade navegación (`firstKey`, `headMap`, `floorEntry`); **LinkedHashMap** combina velocidad de HashMap con orden de llegada.
3. **“¿Cuál es la mejor estructura?”** se responde siempre con “depende” y argumentos: el árbol de decisiones de este punto es tu brújula.

📚 Volver al índice de la unidad · **Anterior:** 07 · HashMap: la guía telefónica · **Siguiente:** 09 · Repaso interactivo

09 · Repaso interactivo: colecciones, genéricos y mapas

El cierre práctico de la unidad: un remove() que lo va a liar todo, un HashMap que sobrescribe y un bucle que va a explotar 😈

> 🗺️ **Estás en:** 📚 **U12 · Colecciones, genéricos y mapas** → 09 · Repaso interactivo

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```
import java.util.ArrayList;

public class Misterio {
    public static void main(String[] args) {
        ArrayList<Integer> lista = new ArrayList<>();
        lista.add(1);
        lista.add(2);
        lista.add(3);
        lista.add(4);

        lista.remove(1);
        lista.add(2, 10);

        for (int i = 0; i < lista.size(); i++) {
            System.out.print(lista.get(i) + " ");
        }
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **`1 3 4 10`** → El `remove(1)` borra el 2, y luego el `add(2, 10)` mete el 10 al final. ❌
2. **`1 2 3 10`** → El `remove(1)` borra el 2, y el `add(2, 10)` inserta el 10 en la posición 2. ❌
3. **`1 3 10 4`** → ✅ ¡Correcto! `remove(1)` borra el índice 1 (el 2) → `{1, 3, 4}`. `add(2, 10)` inserta el 10 en el índice 2, desplazando el 4 → `{1, 3, 10, 4}`.

> 🔄 **Solución**

Y ahora la versión mapa, con trampa doble:

```
import java.util.*;

public class MisterioMapa {
    public static void main(String[] args) {
        HashMap<String, Integer> m = new HashMap<>();
        m.put("Ana", 3);
        m.put("Bob", 5);
        m.put("Ana", m.get("Ana") + 2);

        TreeMap<Integer, String> ordenado = new TreeMap<>();
        for (Map.Entry<String, Integer> e : m.entrySet()) {
            ordenado.put(e.getValue(), e.getKey());
        }

        System.out.println(ordenado.firstKey() + " " + m.size());
    }
}
```

¿Qué imprime este segundo programa?

1. **`3 3`** → El `put("Ana", m.get("Ana") + 2)` añade otra entrada y el mapa tiene 3. ❌
2. **`5 2`** → ✅ ¡Correcto! `Ana` pasa de 3 a 5 (3 + 2), `Bob` sigue en 5, el mapa tiene 2 entradas, y el TreeMap de claves numéricas tiene `{5 → Ana, 5 → Bob}` donde el segundo `put` sobrescribe: primera clave = 5.
3. **`Bob 2`** → El TreeMap guarda el valor como clave y el nombre como valor, y `firstKey()` devuelve el valor. ❌

> 🔄 **Solución**

🔥 Fireside Chat: ArrayList vs LinkedList

> *Dos primos de la familia List se enfrentan frente a la máquina de café.*

ArrayList: — Mira, yo soy el de toda la vida. Acceso directo a cualquier posición: `get(500)` y ya está. Contiguo, rápido, fiable.

LinkedList: — Sí, pero tú, para añadir al principio, desplazas a toda la fila. Yo encadeno mis elementos como una conga: cada uno sabe quién va delante y quién detrás.

ArrayList: — ¿Y para recorrer? Yo soy un array por dentro, el CPU te lo agradece. Tú saltas de eslabón en eslabón.

LinkedList: — Cierto, pero mis métodos `addFirst`, `addLast`, `removeFirst` son instantáneos. Soy la reina de las colas y las pilas. ¿Tú cómo simulas una cola de supermercado?

ArrayList: — Vale, para colas y pilas eres tú. Para el resto de los días, yo. ¿Tregua?

LinkedList: — Tregua.

> La lección: **¿acceso aleatorio y recorridos frecuentes? → ArrayList. ¿Añadir/borrar en los extremos o simular cola/pila? → LinkedList.** Cada problema con su lista.

🔥 Fireside Chat: el Genérico vs el Tipo Crudo

> *Dos maneras de declarar una lista se encuentran frente a la misma máquina de café.*

Genérico: — Mira, yo me llamo `ArrayList<String>`. Sé exactamente qué hay dentro, el compilador me protege y no me hacen falta castings. Voy por la vida con la conciencia tranquila.

Tipo Crudo: — ¡Ja! Yo soy `ArrayList` a secas. Meto lo que quiero, saco lo que puedo y si algo explota, es que el programador no sabía lo que hacía. Libertad total.

Genérico: — ¿Libertad? A ti, la primera vez que metes un `Integer` donde iba un `String`, se te rompe el programa en producción. Yo detecto ese error en compilación, cuando todavía estás a tiempo de arreglarlo.

Tipo Crudo: — Vale, vale… pero los genéricos son más difíciles de escribir. Ese `<T>` por todas partes…

Genérico: — Desde Java 7 el diamante `<>` lo infiere todo, y los genéricos desaparecen al compilar (type erasure). No cuestan ni un céntimo de rendimiento. Tú eres el que paga: con castings y `ClassCastException`.

Tipo Crudo: — Tregua. Reconozco que soy solo para código antiguo.

Genérico: — Hecho. Y por el amor de Java, no escribas `new ArrayList()` sin los `< >`.

> La lección: **usa siempre genéricos con diamante <>. El tipo crudo es una reliquia que solo da sustos.** El compilador no te odia: te está protegiendo.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy el parking que crece solo y no puede guardar `int` a pelo.**
2. **Soy el que encadena elementos como una conga, perfecto para colas y pilas.**
3. **Soy el portero que no deja entrar a nadie que ya esté dentro.**
4. **Soy el que ordena todo lo que metes automáticamente.**
5. **Soy el camarero que recorre cualquier colección y permite borrar sin liarla.**
6. **Soy la clase estática que ordena, revuelve y encuentra el máximo de tus colecciones.**
7. **Soy el comodín que significa "cualquier tipo" y limita a una jerarquía cuando le pongo `extends`.**
8. **Soy el operador perezoso que evita repetir el tipo dos veces.**
9. **Soy el mago que borra los genéricos cuando el código se compila.**
10. **Soy el mapa rapidísimo que no garantiza ningún orden.**
11. **Soy el mapa que ordena las claves automáticamente.**
12. **Soy el método que funciona para cualquier tipo gracias a su `<T>` propio.**

> › 🔁 Respuestas

👹 CONRAD VS EL MUNDO: "El parking que no aparca"

> *CONRAD, nuestro compilador cascarrabias, opina sobre los clásicos del novato.*

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: *CONRAD, mi programa se cae*. Y yo: vale, ¿qué error? *Pues algo de "ConcurrentModificationException".* ¡AY, MADRE MÍA! Estabas borrando con `lista.remove(...)` dentro de un `for-each`. ¡El bucle está recorriendo la lista mientras tú la cambias por debajo! Usa `iterator.remove()`, que para eso existe.

Y luego está el del `remove` con números. Escribe `numeros.remove(1)` creyendo que borra el número 1. ¡Pero si `1` es un `int`! Lo interpreta como **índice**. Para borrar el número 1 necesitas `remove(Integer.valueOf(1))`. Índice o valor: decide qué quieres borrar.

Y el colmo de los colmos: confunde `HashSet` con `TreeSet`. Se queja de que "su HashSet no ordena". ¡Pues claro! Un `HashSet` es un portero, no un organizador. Si quieres orden, `TreeSet` (orden natural) o `LinkedHashSet` (orden de llegada). Y no le metas `null` a un `TreeSet`, que no sabe compararlo con nada.

La lección: antes de acusar a Java de "odiarte", repasa el trío sagrado de la unidad: **¿estoy borrando por índice o por valor? ¿estoy modificando una colección mientras la recorro? ¿he elegido la colección con el orden que necesito?** El 90% de los errores de esta unidad se arreglan con esas tres preguntas. El compilador no te odia: te está pasando las respuestas del examen.

Y cuando ya creías que CONRAD estaba calmado, llega el del mapa:

CONRAD: — ¡OTRA VEZ! Un alumno me enseña su agenda y se queja: *"CONRAD, se me ha perdido un contacto"*. Y yo: ¿lo añadiste con `put` usando el mismo nombre dos veces? *"Pues sí, ¿por?"* ¡AY, MADRE MÍA! ¡En un HashMap las claves son únicas! El segundo `put("Ana", ...)` SOBRESCRIBE al primero. Es como tachar un número en tu agenda para poner otro: el anterior se fue para siempre. Si quieres conservar varios, usa `HashMap<String, List<String>>`.

Y luego está el que pregunta "¿por qué no ordena mi HashMap?" ¡Pues claro que no ordena! Es un HashMap, el rey de la velocidad, no un bibliotecario. Si quieres orden, `TreeMap`. Si quieres orden de llegada, `LinkedHashMap`. Cada mapa con su superpoder.

Y el colmo de los colmos: el que escribe `Caja<int>` y espera que compile. ¡No se puede! Los genéricos solo aceptan objetos: usa `Integer` y deja que el autoboxing haga su magia. Y no me vengas con el `get` de una clave que no existe: devuelve `null`. Usa `getOrDefault`, que para eso está.

La lección: repasa el trío sagrado del lado de los mapas: **¿estoy sobrescribiendo una clave con `put`? ¿he elegido el mapa con el orden que necesito? ¿estoy guardando un primitivo donde solo caben objetos?** El compilador no te odia: te está pasando las respuestas del examen.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. `ArrayList<Integer> l = new ArrayList<>(); l.add(7); l.remove(7);` ¿Qué borra el `remove(7)`?
   - a) El índice 7 b) El objeto 7
2. ¿Qué colección te da los elementos siempre ordenados y sin duplicados?
   - a) `HashSet` b) `TreeSet`
3. ¿Qué lanza borrar con `lista.remove()` dentro de un `for-each`?
   - a) `ArrayIndexOutOfBoundsException` b) `ConcurrentModificationException`
4. ¿Qué colección elimina duplicados manteniendo el orden de llegada?
   - a) `LinkedHashSet` b) `TreeSet`
5. `Collections.sort(lista)`… ¿qué hace con la lista original?
   - a) La modifica b) Devuelve una nueva lista
6. `mapa.put("Ana", 10); mapa.put("Ana", 20);` ¿Qué hace el segundo `put`?
   - a) Añade otra entrada b) Sobrescribe el valor de “Ana” a 20
7. `Caja<int>`… ¿compila?
   - a) Sí b) No: los genéricos no aceptan primitivos
8. `mapa.get("NoExisto")` devuelve…
   - a) `null` b) 0 c) Una excepción
9. ¿Qué mapa te da las claves siempre ordenadas alfabéticamente?
   - a) `HashMap` b) `TreeMap`
10. ¿Puedes añadir un elemento a una `List<? extends Number>`?
    - a) Sí, cualquier Number b) No (solo `null`)
11. En runtime, ¿`ArrayList<String>` y `ArrayList<Integer>` son la misma clase?
    - a) Sí, gracias al type erasure b) No, son clases distintas

> 🔄 **Soluciones**

⚡ Laboratorio de tortura: la lista que llora

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Te dicen que tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto… pero, ¿y si te lo dicen

mal? Tu tarea: hacer que compile, que ejecute y que **toda** la salida sea correcta, contando tú los errores reales.

```
import java.util.ArrayList;

public class Estadisticas {

    public static int mayor(ArrayList<Integer> notas) {
        int max = notas.get(0);
        for (int i = 1; i < notas.size(); i++) {
            if (notas.get(i) > max) {
                max = notas.get(i)
            }
        }
        return max;
    }

    public static void main(String[] args) {
        ArrayList<Integer> notas = new ArrayList<>();
        notas.add(6);
        notas.add(8);
        notas.add(5);
        notas.add(9);

        System.out.println("La mayor es: " + mayor(notas))
    }
}
```

Fallo intencionado: el número de errores del enunciado está mal a propósito: si buscas el "tercer error de compilación" y el "error de lógica", te vuelves loco. La trampa es tu confianza en el enunciado.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún ; que falte? *no → sigue buscando.*
   > **¿Y si sigo atascado?**

2. ¿Compila ya? *no → mira los tipos y los imports.*
   > **¿Y si sigo atascado?**

3. ¿Crees que hay un error de lógica que cambia la nota máxima? *Vuelve a leer el código con calma y no te fíes del enunciado.*

> **› Solución final**

Los **errores de compilación**:

1. Falta el `;` al final de `max = notas.get(i)`.
2. Falta el `;` al final de `System.out.println(...)`.

El **error de lógica**: no existe. La condición `notas.get(i) > max` es correcta y devuelve el mayor (9). Ese era el fallo intencionado: el enunciado dice "3 errores de compilación y 1 de lógica", pero solo hay 2 faltas de `;` y ninguna lógica rota. El error "oculto" era tu confianza en el enunciado. El resultado correcto es `La mayor es: 9`.

```
import java.util.ArrayList;

public class Estadisticas {

    public static int mayor(ArrayList<Integer> notas) {
        int max = notas.get(0);
        for (int i = 1; i < notas.size(); i++) {
            if (notas.get(i) > max) {
                max = notas.get(i);
            }
        }
        return max;
    }

    public static void main(String[] args) {
        ArrayList<Integer> notas = new ArrayList<>();
        notas.add(6);
        notas.add(8);
        notas.add(5);
        notas.add(9);

        System.out.println("La mayor es: " + mayor(notas));
    }
}
```

Salida correcta: `La mayor es: 9`. La lección real: **lee el código, no el enunciado**. Un bug puede ser que te digan "hay 3 errores" y solo haya 2: si buscas el tercero, te vuelves loco. Cuenta,

verifica y confía en el compilador, no en la leyenda.

Laboratorio de tortura: la media que llora

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: el mismo juego, ahora con genéricos y mapas. Te dicen que tiene **3 errores** de compilación y 1 de lógica… ¿te lo crees?

```
import java.util.*;

public class Estadisticas {

    public static <T extends Number> double media(List<T> numeros) {
        double suma = 0;
        for (T n : numeros) {
            suma += n.doubleValue()
        }
        return suma / numeros.size();
    }

    public static void main(String[] args) {
        ArrayList<Integer> notas = new ArrayList<>();
        notas.add(6);
        notas.add(8);
        notas.add(5);
        notas.add(9);

        HashMap<String, Double> resultado = new HashMap<>();
        resultado.put("media", media(notas))
        System.out.println(resultado.get("media"));
    }
}
```

Pistas para cuando te frustres (no antes):

1. ¿Hay algún ; que falte? *no → sigue buscando.*

   > **› ¿Y si sigo atascado?**

2. ¿Compila ya? *no → mira los tipos y los imports.*

> **¿Y si sigo atascado?**

3. ¿Crees que hay un error de lógica que cambia la media? *Vuelve a leer el código con calma y no te fíes del enunciado.*

> **Solución final**

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Aparcador**	Explicar por qué `remove(1)` y `remove(Integer.valueOf(1))` hacen cosas distintas
🏅 **El Portero**	Usar `HashSet` para eliminar duplicados de una lista en un ejercicio real
🏅 **El Conguero**	Montar una cola FIFO con `LinkedList` sin que se cuele nadie
🏅 **El Doctor Iterator**	Borrar elementos de una colección durante un recorrido sin `ConcurrentModificationException`
🏅 **El Tipógrafo**	Explicar por qué `ArrayList<String>` rechaza un `int` sin necesidad de casting
🏅 **El Cajero**	Crear tu propia clase genérica `Caja<T>` y usarla con dos tipos distintos
🏅 **El Mago**	Explicar el type erasure y el diamante `<>` como si fuera un truco de magia
🏅 **El Guía Telefónico**	Recorrer un `HashMap` con `entrySet()` y contar frecuencias con `getOrDefault`
🏅 **El Estratega**	Elegir entre Map, List y Set justificando la elección con argumentos

---

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```java
import java.util.ArrayList;

public class Acertijo {
    public static void main(String[] args) {
        ArrayList<Integer> l = new ArrayList<>();
        l.add(1);
        l.add(2);
        l.add(3);
        int suma = 0;
        for (int i = 0; i < l.size(); i++) {
            if (l.get(i) % 2 == 0) {
                suma += l.get(i);
            }
        }
        System.out.println(suma);
    }
}
```

2. **El limpiador de listas:** tienes un `ArrayList<Integer>` con repetidos. ¿Cómo lo dejas sin duplicados pero manteniendo el orden de primera aparición, en dos líneas?
3. **El detective del TreeSet:** tienes un `TreeSet<String>` y quieres saber cuántos elementos van antes que `"Bob"` en el orden natural. ¿Qué método usas y qué devuelve?
4. **Verdadero o falso:** “un `for-each` puede modificar los elementos de un `ArrayList<Integer>` haciendo `elemento = 99`”.
5. **Sin ejecutar:** ¿qué imprime este programa?

```java
import java.util.HashMap;

public class AcertijoMapa {
    public static void main(String[] args) {
        HashMap<String, Integer> m = new HashMap<>();
        for (char c : "casa".toCharArray()) {
            m.put(String.valueOf(c), m.getOrDefault(String.valueOf(c), 0) + 1);
        }
        System.out.println(m.get("a") + " " + m.size());
    }
}
```

6. **El contador de palabras ordenado:** tienes un `String[]` de palabras y quieres mostrar cada palabra con su frecuencia, en orden alfabético. ¿Qué estructura usas y con qué bucle?

7. **El detective del erasure:** ¿por qué no puedes escribir `new T()` dentro de una clase genérica, aunque te hayas pasado por el `<T>`?
8. **Verdadero o falso:** "un `HashMap` garantiza el orden en el que introduces las claves".

> 💡 **Soluciones**

🧩 Crucigrama de bits

Colecciones:

```
Horizontal:
1. Clase estática que ordena y revuelve colecciones (11 letras)
3. Excepción al modificar una colección mientras se recorre (31 letras)
5. Bucle de solo lectura: "para cada X en Y" (7 letras)
6. Clase lista que crece sola y usa size() (9 letras)

Vertical:
2. Colección que no deja entrar duplicados (7 letras)
4. Colección que mantiene los elementos ordenados (7 letras)
```

> 📝 **Soluciones (colecciones)**

Genéricos y mapas:

```
Horizontal:
1. Mapa que ordena las claves automáticamente (7 letras)
3. Operador perezoso que infiere el tipo en `new ArrayList<>()` (7 letras)
5. Método que devuelve un valor por defecto si la clave no existe (12 letras)
6. Mapa rapidísimo que no garantiza orden (7 letras)

Vertical:
2. Proceso que borra los genéricos al compilar (12 letras)
4. Estructura que asocia claves con valores (3 letras)
```

> 📝 **Soluciones (genéricos y mapas)**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **“Explícame, como si yo fuera tu abuela, qué es un `ArrayList`.”**
2. **“¿Cuál es la diferencia entre `ArrayList` y `LinkedList`?”**
3. **“¿Qué es el autoboxing y por qué `ArrayList<int>` no compila?”**
4. **“¿Cómo eliminas un elemento de una colección mientras la recorres? ¿Qué pasa si usas `list.remove()`?”**
5. **“¿Cuándo usarías `HashSet` en vez de `ArrayList`?”**
6. **“¿Cuál es la diferencia entre `Collection` y `Collections`?”**
7. **“Explícame, como si yo fuera tu abuela, qué es un `HashMap`.”**
8. **“¿Qué es el type erasure y por qué es importante?”**
9. **“¿Cuál es la diferencia entre `? extends T` y `? super T`? ¿Qué es PECS?”**
10. **“¿Por qué `List<Number>` no acepta una `List<Integer>`?”**
11. **“¿Cuándo usarías un `TreeMap` en vez de un `HashMap`?”**
12. **“¿Puedes crear un array de genéricos? ¿Y hacer `new T()`? ¿Por qué?”**

🤷 No hay preguntas tontas

> ❓ **¿`Collection` y `Collections` son lo mismo?**

No. `Collection` es la **interfaz** raíz de la familia (List, Set, Queue la implementan). `Collections` es la **clase utilitaria** de métodos estáticos (`sort`, `shuffle`, `max`…). Una es un contrato; la otra, una navaja suiza.

> ❓ **¿Y si mezclo tipos en una colección sin genéricos?**

Puedes: `ArrayList lista = new ArrayList();` acepta cualquier cosa. Pero al sacarlo todo es `Object` y tienes que hacer casting cruzando los dedos. Con genéricos (`ArrayList<String>`), el compilador te protege y te casca antes de que el lío llegue a ejecución.

> ❓ **¿`for-each` y `iterator` son lo mismo?**

Casi. El `for-each` usa un `Iterator` por debajo. La diferencia: con el `Iterator` tienes el método `remove()`, que te permite borrar durante el recorrido. Si solo lees, usa el for-each: es más corto y legible.

> ❓ **¿`Map` es una `Collection`?**

No. `Map` no hereda de la interfaz `Collection` (aunque vive en `java.util`). `List` y `Set` son `Collection`; `Map` es una familia aparte que asocia claves con valores. Es un error muy típico: "las colecciones de Java" incluyen a los tres, pero técnicamente `Map` no implementa `Collection`.

> ❓ **¿Los genéricos ralentizan mi programa?**

No. Java aplica **type erasure**: el compilador borra toda la información genérica y la convierte en castings normales. En runtime no hay genéricos, así que no hay coste extra. Es solo azúcar sintáctico en compilación.

> ❓ **¿Wildcard o parámetro de tipo? ¿Cuándo uso cada uno?**

Usa el **parámetro de tipo** `<T>` cuando necesites el tipo más de una vez (devolverlo, guardarlo en una variable, dos argumentos del mismo tipo). Usa el **wildcard** `?` cuando solo te importe la colección en sí (leer o escribir sin nombrar el tipo). Si tienes dudas: "¿necesito nombrar el tipo?" → parámetro de tipo; "¿me da igual cuál sea?" → wildcard.

🎬 Poscréditos

La programadora termina su agenda telefónica: un `HashMap<String, String>` con nombre → número, un `TreeMap` que le ordena los contactos alfabéticamente y un `getOrDefault` que evita los `null` cuando busca a alguien que no existe. Todo perfecto… hasta que cierra el programa y su agenda desaparece.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — ¿Has visto? Todo vive en memoria RAM. Cierras la app y adiós, agenda. Un mapa es fantástico para guardar datos mientras el programa corre, pero para que sobreviva al reinicio necesitas algo más.

Programadora: — ¿Guardarlo en un archivo? ¿Como un fichero de texto en el disco?

CONRAD: — *asiente* Exacto. Ficheros, serialización, y cuando los datos crezcan, expresiones regulares para buscar patrones. Todo eso es otra historia… y casi te toca.

PRÓXIMAMENTE EN U14: Ficheros y expresiones regulares (Regex). Guarda tus mapas y tus listas en disco y busca patrones con expresiones regulares. 📁

📚 Volver al índice de la unidad · **Anterior:** 08 · TreeMap y cómo elegir estructura · **Siguiente:** U13 · Programación funcional (Streams y Lambdas)

U13 — Programación funcional (Streams y Lambdas)

La cinta transportadora que procesa tus colecciones y la flecha que viaja sola: lambdas, interfaces funcionales, streams, Collectors y Optional 🎯

En la U12 viste colecciones, genéricos y mapas: tus datos por fin viven en estructuras que el compilador controla y que se asocian con elegancia. Pero había una manera de recorrerlas: bucles, bucles y más bucles. ¿Y si te dijera que existe una forma de decirle a Java **QUÉ** quieres conseguir y que él se encargue del **CÓMO**?

Bienvenido o bienvenida a la programación funcional. Esta unidad tiene tres grandes actos:

- **Lambdas e interfaces funcionales:** la flecha `->` y los cuatro trajes oficiales (`Predicate`, `Function`, `Consumer`, `Supplier`). Las funciones dejan de ser solo bloques de código y se convierten en valores que viajan de un lado a otro como un mensaje en una botella.
- **Streams:** la cinta transportadora que recorre tus colecciones sin escribir un solo bucle. `filter`, `map`, `distinct`, `sorted`, `limit`, `collect`, `reduce`… declaras qué quieres y Java se ocupa del cómo. La pereza nunca fue tan productiva: las operaciones intermedias no hacen nada hasta que abres el grifo.
- **Optional y Collectors:** la caja que acepta la ausencia en vez de un `null` a secas, y el maletín que convierte streams en listas, conjuntos y mapas (`toList()`, `groupingBy`, `toMap`).

Por el camino entenderás por qué las operaciones intermedias son perezosas, por qué un `Stream` solo se recorre una vez, y por qué `Optional.of(null)` te va a explotar en la cara (una vez. Solo necesitas una).

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar qué es una **expresión lambda** y escribir la sintaxis `(x) -> ...` sin miedo.
- Reconocer las **interfaces funcionales** de `java.util.function`: `Predicate`, `Function`, `Consumer` y `Supplier`.

- Usar **referencias a métodos** (`::`) para escribir el mismo código con menos ruido.
- Crear un **`Stream`** a partir de una colección con `stream()` o desde cero con `Stream.of`.
- Encadenar **operaciones intermedias** (`filter`, `map`, `distinct`, `sorted`, `limit`) sin ejecutar nada.
- Cerrar el pipeline con **operaciones terminales** (`collect`, `reduce`, `count`, `forEach`).
- Convertir streams en colecciones con **`toList()`** y **`Collectors`** (`groupingBy`, `toMap`, `joining`).
- Usar **`Optional`** para gestionar la ausencia de valor y por qué es mejor que un `null` a secas.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · Lambdas: funciones que viajan solas	La flecha `->`, la sintaxis y guardar funciones en variables	Todos
02 · Interfaces funcionales: Predicate, Function, Consumer, Supplier	Los cuatro trajes oficiales de toda lambda y `@FunctionalInterface`	Todos
03 · Referencias a métodos: el atajo `::`	`String::toUpperCase`: reciclar métodos como si fueran lambdas	Todos
04 · Streams: el flujo de datos	Crear streams con `stream()` y `Stream.of`, y el pipeline	Todos
05 · Operaciones intermedias: filter, map, distinct, sorted, limit	La cinta que filtra, transforma, quita repetidos, ordena y limita	Todos
06 · Operaciones terminales: collect, reduce, count, forEach	El grifo que hace que el agua fluya de verdad	Todos
07 · Del stream a la colección: toList, Collectors, groupingBy	`toList()`, `Collectors.toMap` y agrupar como en una tabla	Todos
08 · Optional: el que acepta la ausencia	Envolver el `null` para no tropezarte con él	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: en esta unidad el por-resolver va primero. Inténtalo de verdad y solo mira el resuelto cuando hayas sudado un poco.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA6)

RA6: Escribe programas que manipulen información, seleccionando y utilizando tipos avanzados de datos.

CE	Criterio	Dónde se cubre
RA6 b)	Se han reconocido las librerías de clases relacionadas con tipos de datos avanzados.	✅ Puntos 1, 2, 4, 7 y 8
RA6 c)	Se han utilizado listas para almacenar y procesar información.	✅ Puntos 4, 5, 6 y 7

> 📌 La **RA6** se completa con la **U06** (arrays, CE a), la **U12** (colecciones, genéricos y mapas, CEs b, c, d, e y f) y la **U14** (regex, CE g). Esta unidad añade la capa de **programación funcional**: procesa la información de las colecciones con lambdas, streams y `Optional`, reforzando los CEs b y c desde el enfoque declarativo.

✅ Criterios de evaluación cubiertos (RA7)

RA7: Desarrolla programas, aplicando características avanzadas de los lenguajes orientados a objetos y del entorno de programación.

CE	Criterio	Dónde se cubre
RA7 a)–g)	Herencia, superclase, subclase, modificadores, constructores, sobrescritura y jerarquías de clases.	🔁 Se cubrió en la **U11**
RA7 h)	Se ha comentado y documentado el código.	✅ Repaso interactivo y boletines

> 📌 La **RA7** se apoya en la **U11** (herencia, CEs a–g). Esta unidad la desarrolla en su vertiente de **características avanzadas del lenguaje**: lambdas, interfaces funcionales, streams y `Optional`, los paquetes `java.util.function` y `java.util.stream`.

🚪 ¿Por dónde empiezo?

- ¿Vienes de la U12 y quieres lo esencial? → Arranca en el punto 1 y el punto 2: las funciones que viajan solas.
- ¿Ya escribes lambdas y quieres flujos? → Ve directo al punto 4 y al punto 5: la cinta transportadora.
- ¿Solo vienes a por los trucos? → Salta al punto 3 (la drecera `::`) y al punto 8 (la caja que acepta la ausencia).
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Lambdas: funciones que viajan solas

⏭️ **Al acabar la unidad, continúa en U14 · Ficheros y expresiones regulares (Regex).**

01 · Lambdas: funciones que viajan solas

La flecha `→`, la sintaxis de las lambdas y la primera vez que guardas una función en una variable 🏹

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 01 · Lambdas: funciones que viajan solas

📬 La idea en una frase

> **Una lambda es una función sin nombre que puedes tratar como un valor: guardarla en una variable, pasarla a un método o devolverla. Es la respuesta de Java a "quiero pasarle a este método un comportamiento, no solo datos".**

En la U12 viste las colecciones, los genéricos y los mapas. Todo genial. Pero para recorrerlas siempre había una constante: los bucles y los métodos uno tras otro. Hoy cambia la jugada: las funciones dejan de estar encadenadas a una clase y empiezan a **viajar solas**.

😱 La forma antigua: clases anónimas

Imagina que quieres pasarle a algo "la acción de imprimir un saludo". Antes de las lambdas, Java solo te dejaba hacerlo con una **clase anónima**: una clase sin nombre que defines sobre la marcha, solo para implementar un método y regalárselo a otro sitio.

```
Runnable tarea = new Runnable() {
    @Override
    public void run() {
        System.out.println("Hola");
    }
};
```

Fíjate en el espectáculo: para decir "imprime Hola" necesitas `new Runnable()`, una clase anónima, `@Override`, `public void run()`… seis líneas para una idea de una. Java siempre ha sido muy de ir por las ramas.

🏹 Llega la flecha: la lambda

Una **expresión lambda** (del griego lambda, y no, no tiene nada que ver con los lámbada de la serie *Futurama*… o sí, porque los dos son raros) es exactamente esa idea en una línea:

```
Runnable tarea = () -> System.out.println("Hola");
```

Se lee: “sin parámetros, haz `System.out.println("Hola")`”. La flecha `->` separa **los parámetros** (a la izquierda) del **cuerpo** (a la derecha). Es como decir: “con estos datos, haz esto”.

La sintaxis tiene varias caras:

```
() -> System.out.println("Hola");        // sin parámetros
(x) -> x * 2;                            // un parámetro con paréntesis
x -> x % 2 == 0;                         // un parámetro SIN paréntesis (permitido)
(a, b) -> a + b;                         // dos parámetros
nombre -> {                              // cuerpo con bloque: varias sentencias
    String saludo = "Hola, " + nombre;
    System.out.println(saludo);
}
```

Reglas de oro:

- **Un solo parámetro** → puedes quitar los paréntesis: `x -> x * 2`.
- **El tipo se infiere** → no escribes `(Integer x) -> ...` a menos que te apetezca.
- **Cuerpo de una sola expresión** → no necesita `return` ni llaves.
- **Cuerpo con bloque** → necesitas `{ }` y `return` si devuelves algo.

📦 Funciones que viajan en variables

Aquí viene la parte que te va a volar la cabeza: la lambda no necesita pertenecer a ninguna clase concreta. Solo necesita una **interfaz funcional**: una interfaz con un único método abstracto (en el punto 2 profundizamos). Con eso, la función viaja como un valor.

```
interface Transformador {
    int transformar(int x);
}

public class Demo {
    public static void main(String[] args) {
          Transformador doble = x -> x * 2;              // guardo una función en una
variable
        Transformador sumaUno = x -> x + 1;          // y otra

        System.out.println(doble.transformar(5));    // 10
        System.out.println(sumaUno.transformar(5)); // 6
    }
}
```

`doble` no es un número ni un String: es **una función**. La has metido en una variable como quien guarda un lápiz en el bolsillo. Y puedes pasarla a otros métodos, guardarla en listas… ya no eres tú quien llama a la función: la función viaja sola hasta donde haga falta.

⭐ Sé el Código, my friend…

> 👓 **Don Tip:** una lambda con un solo parámetro puede quitarse los paréntesis, pero con dos o más (o ninguno) no hay atajo: `(a, b) -> ...` y `() -> ...`.

Ejercicio: el contador de viajes

```
interface Viajera {
    String viajar(String destino);
}

public class Demo {
    public static void main(String[] args) {
        Viajera saludo = destino -> "¡Bienvenida a " + destino + "!";
        System.out.println(saludo.viajar("Valencia"));
    }
}
```

¿Qué imprime?

- (A) `¡Bienvenida a Valencia!`

- (B) `saludo@1234abcd`
- (C) Error de compilación: las lambdas no pueden devolver Strings.
- (D) `destino`

> 🔁 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué significa la flecha `->` en una lambda?
2. ¿Cuándo puedes omitir los paréntesis de los parámetros?
3. ¿Qué tipo necesita la variable que guarda una lambda?
4. ¿Cuál es la ventaja de una lambda frente a una clase anónima?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Una **lambda** es una función sin nombre que se escribe con la flecha `->` y se puede tratar como un valor: `(x) -> x * 2`.
2. Se usa donde antes hacía falta una **clase anónima**: menos llaves, menos ruido, exactamente la misma idea.
3. Para guardarla en una variable necesitas una **interfaz funcional**: una interfaz con un único método abstracto.

> 🦎 **Vocabulario rápido**

Término	Idea general
Lambda	Función sin nombre, escrita con `->`
Flecha `->`	Separa parámetros de cuerpo en una lambda
Clase anónima	Clase sin nombre definida sobre la marcha (la forma antigua)
Interfaz funcional	Interfaz con un único método abstracto

Término	Idea general
Variable funcional	Variable cuyo tipo es una interfaz funcional y guarda una lambda

📚 Volver al índice de la unidad · **Anterior:** — · **Siguiente:** 02 · Interfaces funcionales: Predicate, Function, Consumer, Supplier

02 · Interfaces funcionales: Predicate, Function, Consumer, Supplier

Los cuatro trajes oficiales de toda lambda: qué espera cada uno y cómo leer su firma 🎭

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 02 · Interfaces funcionales: Predicate, Function, Consumer, Supplier

---

📬 La idea en una frase

> **Una interfaz funcional es una interfaz con un único método abstracto. Java ya te regala las cuatro más usadas en `java.util.function`: `Predicate` (comprueba), `Function` (transforma), `Consumer` (consume) y `Supplier` (fabrica).**

En el punto 1 viste que para guardar una lambda en una variable necesitas una interfaz funcional. Hoy te presento a los cuatro trajes oficiales que Java te da hechos. Ya no hace falta inventarse interfaces propias como `Transformador`: el armario de `java.util.function` está lleno.

🎭 ¿Qué es una interfaz funcional?

Es una interfaz con **exactamente un método abstracto**. Ese método es la firma que toda lambda debe cumplir: si la interfaz espera `boolean test(int)`, tu lambda debe ser algo que reciba un `int` y devuelva un `boolean`.

Para que el compilador te eche una mano, existe la anotación `@FunctionalInterface`: si alguien intenta añadir un segundo método abstracto, el compilador monta en cólera.

```
@FunctionalInterface
interface Transformador {
    int transformar(int x);
    // int otroMetodo();  // 🚫 Error de compilación: solo un método abstracto
}
```

> ⚠️ **Advertencia:** `@FunctionalInterface` es opcional. La interfaz funciona igual sin ella; la anotación solo sirve para que el compilador verifique que sigues las reglas. Como un cinturón de seguridad: nadie te obliga, pero es buena idea.

🎭 Los cuatro trajes oficiales

1. `Predicate<T>` — el juez

Comprueba si algo cumple una condición. Su método abstracto: **`boolean test(T t)`**.

```
import java.util.function.Predicate;

Predicate<Integer> esPar = n -> n % 2 == 0;
System.out.println(esPar.test(4));   // true
System.out.println(esPar.test(7));   // false
```

2. `Function<T, R>` — el transformador

Recibe una cosa de tipo `T` y devuelve otra de tipo `R`. Su método abstracto: **`R apply(T t)`**.

```
import java.util.function.Function;

Function<String, Integer> longitud = s -> s.length();
System.out.println(longitud.apply("java"));   // 4

Function<String, String> gritando = s -> s.toUpperCase();
System.out.println(gritando.apply("hola"));   // HOLA
```

3. `Consumer<T>` — el que consume sin devolver

Hace algo con el valor y no devuelve nada. Su método abstracto: **`void accept(T t)`**.

```
import java.util.function.Consumer;

Consumer<String> imprimir = s -> System.out.println(s);
imprimir.accept("¡Hola!");
```

4. `Supplier<T>` — el fabricante

No recibe nada y produce un valor. Su método abstracto: **`T get()`**.

```java
import java.util.function.Supplier;

Supplier<String> fabricaSaludos = () -> "¡Hola!";
System.out.println(fabricaSaludos.get());   // ¡Hola!
```

📋 La chuleta de las cuatro

Interfaz	Método abstracto	Qué hace	Lambda típica
`Predicate<T>`	`boolean test(T)`	Comprueba una condición	`n -> n % 2 == 0`
`Function<T, R>`	`R apply(T)`	Transforma un valor en otro	`s -> s.length()`
`Consumer<T>`	`void accept(T)`	Consume sin devolver	`s -> System.out.println(s)`
`Supplier<T>`	`T get()`	Fabrica un valor sin entrada	`() -> new ArrayList<>()`

Fíjate en el patrón para leerlos sin miedo: el nombre te dice el papel (**pre**dica, **fun**ción, **con**sume, **sup**line) y la firma te dice el contrato. Cuando en los puntos siguientes veas `filter(n -> ...)`, `map(n -> ...)`, `forEach(...)` o `orElseGet(...)`, todos estarán pidiendo uno de estos trajes por debajo.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** cuando una lambda no encaja en `Predicate`, `Function`, `Consumer` o `Supplier`, Java tiene decenas de variantes (`BiFunction`, `IntFunction`, `UnaryOperator`…). Empieza siempre por las cuatro básicas y amplía solo si hace falta.

Ejercicio: el portero de discoteca (versión funcional)

```java
import java.util.function.*;

public class Portero {
    public static void main(String[] args) {
        Predicate<Integer> mayorDeEdad = edad -> edad >= 18;
        Function<Integer, String> etiqueta = edad -> edad >= 18 ? "ADULTO" : "MENOR";
        Consumer<String> anunciar = s -> System.out.println("Entrada: " + s);
        Supplier<Integer> reloj = () -> 20;

        int edad = reloj.get();
        if (mayorDeEdad.test(edad)) {
            anunciar.accept(etiqueta.apply(edad));
        }
    }
}
```

¿Qué imprime?

- (A) `Entrada: ADULTO`
- (B) `Entrada: MENOR`
- (C) Nada: el `if` nunca se cumple.
- (D) Error de compilación: no se pueden combinar cuatro lambdas así.

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué método abstracto tiene `Predicate<T>` y qué devuelve?
2. ¿Qué devuelve `Function<T, R>`?
3. ¿Para qué sirve `Consumer<T>`?
4. ¿Y `Supplier<T>`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Una **interfaz funcional** es una interfaz con un único método abstracto: es la firma que toda lambda debe cumplir.
2. Los cuatro trajes oficiales de `java.util.function` son **`Predicate`** (`test`), **`Function`** (`apply`), **`Consumer`** (`accept`) y **`Supplier`** (`get`).
3. La anotación **`@FunctionalInterface`** le pide al compilador que verifique que la interfaz es funcional: un único método abstracto y nada más.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Interfaz funcional | Interfaz con un único método abstracto |
> | `Predicate<T>` | Comprueba una condición: `boolean test(T)` |
> | `Function<T, R>` | Transforma un valor: `R apply(T)` |
> | `Consumer<T>` | Consume sin devolver: `void accept(T)` |
> | `Supplier<T>` | Fabrica un valor: `T get()` |
> | `@FunctionalInterface` | Anotación que valida que la interfaz es funcional |

📚 Volver al índice de la unidad · **Anterior:** 01 · Lambdas: funciones que viajan solas · **Siguiente:** 03 · Referencias a métodos: el atajo `::`

03 · Referencias a métodos: el atajo `::`

Reciclar métodos ya escritos como si fueran lambdas con el operador `::` 🩹

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 03 · Referencias a métodos: el atajo `::`

📬 La idea en una frase

> **Cuando tu lambda solo llama a un método que ya existe, puedes escribir `::` en vez de `->`: `String::toUpperCase` significa "el método `toUpperCase` de los Strings", listo para usarse como una lambda.**

En el punto 1 viste las lambdas y en el punto 2 los cuatro trajes oficiales. Hoy aprendes a vestirlos sin escribir ni una flecha: si el cuerpo de tu lambda es literalmente "llama a este método", Java te deja reciclar ese método directamente con el operador `::` (dos puntos, dos puntos, no son dos puntos suspensivos).

🩹 Cuando la lambda es solo una llamada

Imagina que quieres una `Function<String, String>` que ponga un texto en mayúsculas. Con lambda:

```
Function<String, String> mayusculas = s -> s.toUpperCase();
```

Esa lambda hace una sola cosa: llamar a `toUpperCase()`. Java piensa: "oye, ya existe ese método, ¿para qué escribo la flecha?". La versión con `::`:

```
Function<String, String> mayusculas = String::toUpperCase;
```

`String::toUpperCase` se lee: "el método `toUpperCase` de la clase `String`". Y funciona exactamente igual que la lambda: cuando le pases un String, lo pondrá en mayúsculas.

```
System.out.println(mayusculas.apply("hola"));   // HOLA
```

🗂️ Los cuatro tipos de referencia

1. Método estático: `Clase::metodoEstatico`

```
Function<Integer, Integer> valorAbsoluto = Math::abs;
System.out.println(valorAbsoluto.apply(-7));   // 7
```

Equivale a `x -> Math.abs(x)`.

2. Método de instancia sobre un objeto cualquiera: `Clase::metodoInstancia`

```
Function<String, Integer> longitud = String::length;
System.out.println(longitud.apply("java"));   // 4
```

Equivale a `s -> s.length()`. El parámetro de la lambda se convierte en el objeto sobre el que llamamos.

3. Método de instancia sobre un objeto concreto: `objeto::metodo`

```
Consumer<String> imprimir = System.out::println;
imprimir.accept("¡Hola!");
```

Equivale a `s -> System.out.println(s)`. Aquí el objeto ya existe (`System.out`) y el parámetro de la lambda es el argumento del método.

4. Constructor: `Clase::new`

```
Supplier<ArrayList<String>> fabrica = ArrayList::new;
ArrayList<String> lista = fabrica.get();
```

Equivale a `() -> new ArrayList<>()`. El constructor también es un método, y también se puede reciclar.

📋 La tabla del `::`

Referencia	Sintaxis	Equivale a
Estática	`Math::abs`	`x -> Math.abs(x)`
De instancia (clase)	`String::toUpperCase`	`s -> s.toUpperCase()`
De instancia (objeto)	`System.out::println`	`s -> System.out.println(s)`
Constructor	`ArrayList::new`	`() -> new ArrayList<>()`

> ⚠️ **Advertencia:** la referencia a método no ejecuta el método. `String::toUpperCase` **no** pone nada en mayúsculas: es la receta, no la comida. Se ejecutará cuando el stream o la interfaz funcional lo pida.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cuando dudes entre lambda y referencia, piensa en la legibilidad. `String::toUpperCase` es más limpio que `s -> s.toUpperCase()`, pero una lambda con lógica (como `s -> s.length() > 3`) no se puede convertir a `::` porque no llama a un único método.

Ejercicio: el reciclador de saludos

```
import java.util.function.*;

public class Reciclador {
    public static void main(String[] args) {
        Supplier<String> fabrica = String::new;
        Function<String, String> mayusculas = String::toUpperCase;
        Consumer<String> imprimir = System.out::println;

        String vacio = fabrica.get();
        imprimir.accept(mayusculas.apply(vacio.isEmpty() ? "hola" : vacio));
    }
}
```

¿Qué imprime?

- (A) `HOLA`
- (B) `hola`
- (C) Nada: el `String` está vacío.

- (D) Error de compilación: `String::new` no es una referencia válida.

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué significa `String::toUpperCase` sin paréntesis?
2. ¿Cuándo usarías `objeto::metodo` en vez de `Clase::metodo`?
3. ¿`Math::max` qué tipo de referencia es?
4. ¿Qué hace `Clase::new`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Una **referencia a método** (`::`) es una lambda aún más corta: reutiliza un método que ya existe en vez de escribir la flecha.
2. Hay cuatro tipos: **estática** (`Math::abs`), **de instancia sobre clase** (`String::toUpperCase`), **de instancia sobre objeto** (`System.out::println`) y **de constructor** (`ArrayList::new`).
3. Cuando tu lambda solo llama a un método, la referencia es más legible y se comporta exactamente igual.

> 🦎 **Vocabulario rápido**

Término	Idea general
Referencia a método	Reutilizar un método como lambda con `::`
`Clase::estatico`	Referencia a un método estático
`Clase::instancia`	Método de instancia llamado sobre el parámetro que llegue
`objeto::metodo`	Método llamado sobre un objeto concreto que ya existe
`Clase::new`	Referencia al constructor

📚 Volver al índice de la unidad · **Anterior:** 02 · Interfaces funcionales: Predicate, Function, Consumer, Supplier · **Siguiente:** 04 · Streams: el flujo de datos

04 · Streams: el flujo de datos

La cinta transportadora de Java: crear un `Stream`, el pipeline y por qué no guarda nada 📦

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 04 · Streams: el flujo de datos

---

📬 La idea en una frase

> **Un `Stream` no es una colección nueva: es una cinta transportadora por la que pasan los elementos de una fuente (una lista, un array, varios valores…) para procesarlos de una vez, sin bucles.**

Hasta ahora, para trabajar con una lista de la U12 escribías `for (String s : lista)` y dentro hacías la magia. El stream es otra filosofía: en vez de decirle a Java **cómo** recorrer (bucle, índice, iterador), le dices **qué** quieres conseguir y él monta la cinta. Tú diseñas el flujo; Java mueve los paquetes.

📦 ¿Qué es un Stream (y qué NO es)?

Un `Stream<T>` es una **secuencia de elementos** de tipo `T` que se procesa de una vez. Tres reglas que te van a evitar disgustos:

- **No guarda datos.** La cinta no almacena paquetes: los datos siguen viviendo en tu lista o tu array. El stream solo los va pasando.
- **Es perezoso.** Las operaciones intermedias no se ejecutan hasta que aparece una operación terminal (lo vemos en el punto 5).
- **Solo se recorre una vez.** Como una cinta de supermercado: cuando el último producto cae en la bolsa, la cinta se queda vacía. Usarlo dos veces es imposible.

🚚 Crear un Stream: las cuatro fuentes

```java
import java.util.*;
import java.util.stream.*;

public class Fuentes {
    public static void main(String[] args) {
        List<String> nombres = List.of("ana", "bob", "carla");

         Stream<String> s1 = nombres.stream();                              // desde una
colección
        Stream<Integer> s2 = Stream.of(1, 2, 3);                          // desde valores
sueltos
          Stream<String> s3 = Arrays.stream(new String[]{"x", "y"});       // desde un
array
         Stream<Integer> s4 = Stream.iterate(0, n -> n + 1);               // infinita
(¡con cuidado!)
    }
}
```

- `lista.stream()` — la fuente más común: cualquier colección de la U12 tiene este método.
- `Stream.of(a, b, c)` — cuando quieres un flujo a partir de varios valores sin montar una lista.
- `Arrays.stream(array)` — para convertir un array de la U06 en flujo.
- `Stream.iterate(0, n -> n + 1)` — genera una secuencia infinita. No la cierres con una terminal que lo recorra todo o tu programa se dedicará a contar hasta el infinito (y no es una metáfora).

🏭 El pipeline: fuente → intermedias → terminal

La forma de usar un stream es el **pipeline**: una fuente, cero o más operaciones intermedias, y una operación terminal que cierra el grifo.

```java
import java.util.*;
import java.util.stream.*;

public class Pipeline {
    public static void main(String[] args) {
        List<String> nombres = List.of("ana", "bob", "carla", "david", "eva");

        long cantidad = nombres.stream()          // 1. fuente
             .filter(n -> n.length() > 3)           // 2. intermedia: se queda con los largos
             .count();                              // 3. terminal: los cuenta

        System.out.println(cantidad);              // 3 (carla, david, eva)
    }
}
```

La operación terminal (`count()`) es la que hace que la cinta arranque. Sin ella, el stream no mueve ni un paquete: las intermedias solo preparan la cinta. Es como planear un viaje: puedes diseñar la ruta todo lo que quieras, pero nadie sale hasta que el autobús arranca.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** `lista.stream()` te da un flujo, pero `List.of(...)` crea listas **inmutables**. Si necesitas que el resultado sea modificable, en el punto 7 verás cómo salir del stream a una colección que sí puedas tocar.

Ejercicio: la cinta que arranca

```java
import java.util.*;
import java.util.stream.*;

public class Cinta {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(4, 8, 15, 16, 23, 42);
        Stream<Integer> flujo = numeros.stream()
            .filter(n -> n % 2 == 0)
            .map(n -> n * 10);
        System.out.println("Cinta preparada");
    }
}
```

¿Qué imprime?

- (A) `Cinta preparada`
- (B) `Cinta preparada` y luego `40 80 160 420`
- (C) `40 80 160 420`
- (D) Nada: el programa se queda colgado esperando la cinta.

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Un `Stream` guarda datos?
2. ¿Cuántas veces puedes recorrer un `Stream`?
3. ¿Cómo creas un stream desde una lista?
4. ¿Y desde varios valores sueltos?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Un **`Stream`** es un flujo de elementos, no un contenedor: no guarda nada, es una cinta por la que pasan los datos.

2. Se crea con `coleccion.stream()`, `Stream.of(...)`, `Arrays.stream(array)` o `Stream.iterate`, y se usa en un **pipeline** de fuente → intermedias → terminal.
3. El stream es **perezoso y de un solo uso**: sin operación terminal no hace nada, y una vez consumido no se puede reutilizar.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Stream | Flujo de datos que se procesa de una vez |
> | Pipeline | Cadena fuente → intermedias → terminal |
> | Fuente | De dónde salen los elementos (colección, array, `Stream.of`) |
> | Pereza | Las operaciones intermedias no se ejecutan hasta la terminal |
> | Consumible | Un stream solo se recorre una vez |

📚 Volver al índice de la unidad · **Anterior:** 03 · Referencias a métodos: el atajo `::` · **Siguiente:** 05 · Operaciones intermedias: filter, map, distinct, sorted, limit

05 · Operaciones intermedias: filter, map, distinct, sorted, limit

La cinta que filtra, transforma, quita repetidos, ordena y limita el flujo 🌀

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 05 · Operaciones intermedias: filter, map, distinct, sorted, limit

---

📬 La idea en una frase

> **Las operaciones intermedias transforman el flujo y devuelven otro `Stream`. No hacen nada por sí solas (son perezosas) y no tocan la colección original: solo deciden cómo será la cinta cuando arranque.**

Ya tienes la cinta del punto 4. Hoy le pones las estaciones: `filter` decide qué paquetes pasan, `map` los reetiqueta, `distinct` saca los duplicados, `sorted` los ordena y `limit` corta el flujo. Todas devuelven un stream nuevo, todas son perezosas y ninguna modifica tu lista original. Eso sí: **el orden de las estaciones importa**.

🌀 Las cinco estaciones

`filter(Predicate)` — el portero

Deja pasar solo los elementos que cumplen la condición. Necesita un `Predicate` (punto 2).

```
List<Integer> numeros = List.of(1, 2, 3, 4, 5, 6);
List<Integer> pares = numeros.stream()
    .filter(n -> n % 2 == 0)      // 2, 4, 6
    .toList();
```

`map(Function)` — el transformador

Transforma cada elemento. Necesita una `Function` (punto 2). Cambia el tipo de la cinta: `Stream<Integer>` → `Stream<String>` si quieres.

```java
List<String> palabras = List.of("hola", "java");
List<Integer> longitudes = palabras.stream()
    .map(String::length)          // 4, 4
    .toList();
```

`distinct()` — el que quita repetidos

Elimina duplicados usando `equals()`.

```java
List<String> repetidas = List.of("sol", "mar", "sol", "luna");
List<String> unicas = repetidas.stream()
    .distinct()                   // sol, mar, luna
    .toList();
```

`sorted()` — el que ordena

Ordena según el orden natural (con `Comparable`). Si quieres otro criterio, usa `sorted(Comparator)`.

```java
List<Integer> desordenados = List.of(5, 2, 8, 1);
List<Integer> ordenados = desordenados.stream()
    .sorted()                     // 1, 2, 5, 8
    .toList();
```

`limit(n)` — la tijera

Se queda con los `n` primeros elementos del flujo.

```java
List<Integer> numeros = List.of(10, 20, 30, 40, 50);
List<Integer> tresPrimeros = numeros.stream()
    .limit(3)                     // 10, 20, 30
    .toList();
```

🏭 Las estaciones en cadena

Lo bonito es que se encadenan: cada operación recibe el flujo de la anterior y le pasa el suyo a la siguiente. Un ejemplo completo:

```
import java.util.*;
import java.util.stream.*;

public class Cadena {
    public static void main(String[] args) {
            List<String> palabras = List.of("java", "hola", "java", "mundo", "adios",
"luna");

        palabras.stream()
            .filter(p -> p.length() > 3)      // java, hola, java, mundo, adios, luna
            .map(String::toUpperCase)         // JAVA, HOLA, JAVA, MUNDO, ADIOS, LUNA
            .distinct()                       // JAVA, HOLA, MUNDO, ADIOS, LUNA
            .sorted()                         // ADIOS, HOLA, JAVA, LUNA, MUNDO
            .limit(3)                         // ADIOS, HOLA, JAVA
            .forEach(System.out::println);    // terminal: imprime las tres
    }
}
```

Salida:

```
ADIOS
HOLA
JAVA
```

Fíjate en la **importancia del orden**: si pones `limit(3)` antes de `sorted()`, te quedas con los tres primeros en el orden de llegada y luego los ordenas; el resultado cambia. Diseñar el pipeline es decidir en qué orden pasa cada paquete por cada estación.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** `map` puede cambiar el tipo del stream (`Stream<String>` → `Stream<Integer>`), pero `filter` siempre devuelve el mismo tipo que recibe: solo quita elementos, no los transforma.

Ejercicio: la estación que sobra

```java
import java.util.*;
import java.util.stream.*;

public class Estaciones {
    public static void main(String[] args) {
        List<String> nombres = List.of("ana", "bob", "carla", "david", "eva");
        List<String> resultado = nombres.stream()
            .sorted()
            .filter(n -> n.length() == 3)
            .limit(1)
            .toList();
        System.out.println(resultado);
    }
}
```

¿Qué imprime?

- (A) `[ana]`
- (B) `[bob]`
- (C) `[ana, bob, eva]`
- (D) `[eva]`

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿`filter` deja pasar los elementos que cumplen la condición o los que no?
2. ¿`map` modifica la colección original?
3. ¿Qué hace `distinct`?
4. ¿Qué hace `limit(3)`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Las **operaciones intermedias** transforman el flujo y devuelven otro `Stream`: `filter`, `map`, `distinct`, `sorted` y `limit`.
2. Son **perezosas**: no se ejecutan hasta que llega una operación terminal, y **no modifican** la fuente.
3. El **orden de las estaciones importa**: el mismo pipeline con las mismas operaciones en distinto orden puede dar resultados distintos.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Intermedia | Operación que devuelve otro `Stream` |
> | `filter` | Deja pasar lo que cumple el `Predicate` |
> | `map` | Transforma cada elemento con una `Function` |
> | `distinct` | Elimina repetidos (con `equals`) |
> | `sorted` | Ordena (orden natural o con `Comparator`) |
> | `limit(n)` | Corta el flujo en n elementos |

📚 Volver al índice de la unidad · **Anterior:** 04 · Streams: el flujo de datos · **Siguiente:** 06 · Operaciones terminales: collect, reduce, count, forEach

06 · Operaciones terminales: collect, reduce, count, forEach

El grifo que hace que el agua fluya de verdad: `collect`, `reduce`, `count` y `forEach` 🚰

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 06 · Operaciones terminales: collect, reduce, count, forEach

---

📬 La idea en una frase

> **Las operaciones terminales son el grifo del pipeline: sin una de ellas, el stream no fluye. `collect` empaqueta el resultado, `reduce` combina todos los elementos en uno, `count` los cuenta y `forEach` los recorre.**

En el punto 4 viste que el stream es perezoso y en el punto 5 que las intermedias solo preparan la cinta. Hoy abres el grifo. Cada terminal produce un resultado final y, al hacerlo, **consume el stream**: después de `count()`, `reduce()` o `forEach()`, la cinta queda vacía y no se puede reutilizar.

🚰 Las cuatro terminales

`collect(Collector)` — la que empaqueta

Recoge los elementos del flujo en una estructura. La más famosa: `Collectors.toList()`. Es el puente entre el stream y las colecciones de la U12 (en el punto 7 la exprimimos a fondo).

```
import java.util.*;
import java.util.stream.*;

List<Integer> numeros = List.of(4, 8, 2, 9, 5);
List<Integer> lista = numeros.stream()
    .filter(n -> n % 2 == 0)
    .collect(Collectors.toList());    // [4, 8, 2]
```

`reduce(identidad, acumulador)` — la que combina

Va combinando los elementos en uno solo. El primer argumento es la **identidad** (el valor neutro: `0` para sumas, `1` para productos), y el segundo es un `BinaryOperator` que dice cómo combinar dos elementos.

```
int suma = numeros.stream().reduce(0, (a, b) -> a + b);      // 28
int maximo = numeros.stream().reduce(Integer.MIN_VALUE, Math::max);  // 9
```

`reduce(0, (a, b) -> a + b)` se lee: "empieza en 0 y ve sumando cada elemento al acumulador". Es como un bucle `for` donde el acumulador viaja de elemento en elemento… sin bucle.

`count()` — la que cuenta

Devuelve el número de elementos como `long`.

```
long pares = numeros.stream().filter(n -> n % 2 == 0).count();   // 3
```

`forEach(Consumer)` — la que recorre

Aplica un `Consumer` a cada elemento. Es la terminal "imperativa": la única que se parece a un bucle, pensada para efectos secundarios (imprimir, guardar…).

```
numeros.stream().forEach(System.out::println);   // imprime todos
```

> ⚠️ **Advertencia:** `forEach` con un stream en paralelo no garantiza orden. Para el curso, úsalo en streams normales y no te preocupes; pero recuerda que si el orden importa, es más seguro `collect` y luego recorrer.

🏭 El pipeline completo

Junta lo de los dos puntos anteriores:

```java
import java.util.*;
import java.util.stream.*;

public class Notas {
    public static void main(String[] args) {
        List<Integer> notas = List.of(7, 8, 3, 9, 5);

        long aprobados = notas.stream()
            .filter(n -> n >= 5)
            .count();                                   // 4

        int sumaAprobados = notas.stream()
            .filter(n -> n >= 5)
            .reduce(0, Integer::sum);                   // 29

        List<Integer> dobladas = notas.stream()
            .filter(n -> n >= 5)
            .map(n -> n * 2)
            .collect(Collectors.toList());              // [14, 16, 18, 10]

        System.out.println(aprobados + " " + sumaAprobados + " " + dobladas);
    }
}
```

Salida: `4 29 [14, 16, 18, 10]`. Tres pipelines, tres grifos, tres resultados. Cada vez que abres uno, el stream se consume y tienes que crear otro.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** `Integer::sum` es una referencia a método (punto 3) perfecta para `reduce`: es un `BinaryOperator<Integer>` que suma dos enteros.

Ejercicio: el acumulador

```java
import java.util.*;
import java.util.stream.*;

public class Acumulador {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(1, 2, 3, 4, 5);
        int total = numeros.stream()
            .reduce(10, (a, b) -> a * b);
        System.out.println(total);
    }
}
```

¿Qué imprime?

- (A) `120`
- (B) `15`
- (C) `1200`
- (D) `10`

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué devuelve `count()`?
2. ¿Para qué sirve el primer argumento de `reduce(0, ...)`?
3. ¿Qué hace `forEach`?
4. ¿Puedes reutilizar el mismo stream tras una operación terminal?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Las **operaciones terminales** son el grifo del pipeline: sin ellas, nada fluye.
2. `collect` empaqueta el resultado, `reduce` combina los elementos en uno (con una identidad), `count` los cuenta y `forEach` los recorre.

3. Tras una terminal, el stream se **consume**: si necesitas procesar de nuevo, crea otro stream.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Terminal | Operación que cierra el pipeline y produce un resultado |
> | `collect` | Recoge el flujo en una estructura (p. ej. lista) |
> | `reduce` | Combina todos los elementos en uno (suma, máximo…) |
> | Identidad | Valor inicial del `reduce` (neutro: `0` para la suma) |
> | `count()` | Devuelve el número de elementos (`long`) |
> | `forEach` | Aplica un `Consumer` a cada elemento |

📚 Volver al índice de la unidad · **Anterior:** 05 · Operaciones intermedias: filter, map, distinct, sorted, limit · **Siguiente:** 07 · Del stream a la colección: toList, Collectors, groupingBy

07 · Del stream a la colección: toList, Collectors, groupingBy

`toList()`, `Collectors.toMap` y `groupingBy`: del flujo a listas, conjuntos y mapas 🗂️

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 07 · Del stream a la colección: toList, Collectors, groupingBy

📬 La idea en una frase

> **Un stream no guarda nada, así que al final del pipeline toca aterrizar: `toList()` o los `Collectors` convierten el flujo en listas, conjuntos, Strings y, con `groupingBy`, en mapas agrupados por una clave.**

La cinta transportadora está muy bien para procesar, pero al final los paquetes tienen que ir a algún sitio. La terminal `collect` (del punto 6) es la puerta de salida, y hoy conoces su maletín: `Collectors`, la clase con las recetas para empaquetar de todas las formas imaginables.

📥 `toList()`: la salida rápida (Java 16+)

Desde Java 16 existe `toList()` directo sobre el stream: la forma más corta de aterrizar en una lista.

```
List<String> palabras = List.of("hola", "mundo", "hola", "adios");
List<String> unicas = palabras.stream()
    .distinct()
    .toList();                      // [hola, mundo, adios]
```

> ⚠️ **Advertencia:** `toList()` devuelve una lista **inmutable**: no puedes `add`, `remove` ni reordenarla. Si necesitas modificarla, usa `collect(Collectors.toList())`, que devuelve un `ArrayList` de toda la vida.

🗂️ Los `Collectors` de toda la vida

La clase `Collectors` (en `java.util.stream`) tiene recetas para cada tipo de aterrizaje:

```java
import java.util.*;
import java.util.stream.*;

List<String> palabras = List.of("hola", "mundo", "hola", "adios");

List<String> lista = palabras.stream().collect(Collectors.toList());       // ArrayList
Set<String> conjunto = palabras.stream().collect(Collectors.toSet());       // HashSet sin duplicados
String frase = palabras.stream().collect(Collectors.joining(", "));          // "hola, mundo, hola, adios"
long cuantos = palabras.stream().collect(Collectors.counting());           // 4
```

`joining(", ")` es el favorito para sacar un `String` legible: une los elementos con el separador que le pases. Ideal para depurar pipelines en dos líneas.

🗺️ `groupingBy`: agrupar como en una tabla

El rey de este punto. **`Collectors.groupingBy(función)`** agrupa los elementos según la clave que devuelve la función y te da un `Map<clave, List<valor>>`.

```java
import java.util.*;
import java.util.stream.*;

public class Agrupando {
    public static void main(String[] args) {
        List<String> palabras = List.of("hola", "adios", "hola", "mar", "mundo");

        Map<Character, List<String>> porLetra = palabras.stream()
            .collect(Collectors.groupingBy(p -> p.charAt(0)));

        System.out.println(porLetra);
        // {a=[adios], h=[hola, hola], m=[mar, mundo]}
    }
}
```

Es el equivalente a la sentencia SQL `GROUP BY`: en vez de recorrer a mano y montar el `HashMap` de la U12 con `getOrDefault`, declaras la clave y Java agrupa. Por dentro está usando un `HashMap`, pero tú no lo ves: el `Collectors` lo monta por ti.

🗺️ `toMap`: de elementos a entradas

`Collectors.toMap(clave, valor)` convierte cada elemento en una entrada de mapa. Tiene un tercer argumento: la **función de fusión**, que decide qué hacer cuando dos elementos quieren la misma clave.

```
List<String> palabras = List.of("hola", "mundo", "hola", "adios");

// palabra → longitud
Map<String, Integer> longitudes = palabras.stream()
    .collect(Collectors.toMap(p -> p, String::length, (a, b) -> a));
// {adios=5, hola=4, mundo=5}

// primera letra → primera palabra que la cumpla
Map<Character, String> representantes = palabras.stream()
    .collect(Collectors.toMap(p -> p.charAt(0), p -> p, (a, b) -> a));
// {a=adios, h=hola, m=mundo}
```

Sin la función de fusión `(a, b) -> a`, un `"hola"` duplicado provocaría una `IllegalStateException`: dos elementos queriendo la misma clave y nadie resolviendo el conflicto. Con `(a, b) -> a` le dices “quédate con el primero”.

⭐ Sé el Código, my friend…

> 👓 **Don Tip:** `groupingBy` admite un segundo argumento, el “colector aguas abajo”: `groupingBy(clave, Collectors.counting())` te da `Map<clave, Long>` con cuántos elementos hay en cada grupo. El contador de frecuencias de la U12 en una sola línea.

Ejercicio: el contador de grupos

```java
import java.util.*;
import java.util.stream.*;

public class Grupos {
    public static void main(String[] args) {
        List<String> palabras = List.of("sol", "mar", "sol", "luna", "mar", "sol");
        Map<String, Long> conteo = palabras.stream()
            .collect(Collectors.groupingBy(p -> p, Collectors.counting()));
        System.out.println(conteo);
    }
}
```

¿Qué imprime?

- (A) `{sol=3, mar=2, luna=1}`
- (B) `{sol=1, mar=1, luna=1}`
- (C) `{sol=1, mar=1, sol=1, luna=1, mar=1, sol=1}`
- (D) `{sol=6}`

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué devuelve `toList()`?
2. ¿Qué hace `Collectors.groupingBy(...)`?
3. ¿Para qué sirve el tercer argumento de `toMap`?
4. ¿Qué devuelve `Collectors.joining(", ")`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. `toList()` y los `Collectors` convierten el flujo en **colecciones reales**: listas, conjuntos, Strings y mapas.

2. `groupingBy` agrupa los elementos por una clave y devuelve un `Map<clave, List<valor>>`; con `counting()` como segundo argumento, cuenta cada grupo.
3. `toMap` convierte cada elemento en una entrada clave-valor y necesita una **función de fusión** cuando puede haber claves repetidas.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `toList()` | Recoge el stream en una lista inmutable (Java 16+) |
> | `Collectors.toList()` / `toSet()` | Recogen en lista o conjunto modificables |
> | `groupingBy` | Agrupa por una clave en un `Map` |
> | Colector aguas abajo | Segundo argumento que procesa cada grupo (`counting()`) |
> | `toMap` | Convierte cada elemento en una entrada clave-valor |
> | Fusión (merge) | Función que resuelve qué hacer con claves repetidas |
> | `joining` | Une los elementos en un `String` |

📚 Volver al índice de la unidad · **Anterior:** 06 · Operaciones terminales: collect, reduce, count, forEach · **Siguiente:** 08 · Optional: el que acepta la ausencia

08 · Optional: el que acepta la ausencia

Envolver el `null` para no tropezarte: `Optional.of`, `orElse`, `isPresent` e `ifPresent` 🎁

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 08 · Optional: el que acepta la ausencia

---

📬 La idea en una frase

> **`Optional<T>` es una caja que contiene un valor o está vacía. En vez de devolver un `null` a secas (que nadie sabe si era un dato real o un olvido), devuelves la caja y obligas a quien la recibe a preguntar: ¿hay valor dentro, o no?**

Llevas toda la unidad procesando datos como un jefe. Pero hay un momento en que todo se tuerce: cuando un método no encuentra lo que busca y devuelve `null`. Y tú, inocente, llamas a `.length()` o a `.get()` sobre ese `null`… y la `NullPointerException` te saluda en toda la cara. `Optional` existe para que la ausencia sea **explícita** y **gestionable**, no un accidente.

🎁 La caja en acción

Un `Optional<T>` es una cajita que puede estar **llena** (con un valor) o **vacía**. Crearla es fácil:

```
import java.util.*;

Optional<String> algo = Optional.of("java");          // caja LLENA. Prohibido pasar null.
Optional<String> vacio = Optional.empty();            // caja VACÍA.
Optional<String> dudoso = Optional.ofNullable(valor); // acepta null: si es null, caja vacía
```

> ⚠️ **Advertencia:** `Optional.of(null)` lanza `NullPointerException` al instante. Es la forma de decir "esta caja SOLO admite valores reales". Si no sabes si puede llegar `null`, usa `ofNullable`.

🎁 Abrir la caja con elegancia

La gracia de la caja es que no la abres a lo bruto: usas sus métodos para decidir qué pasa en cada caso.

```
import java.util.*;

public class Caja {
    public static void main(String[] args) {
        Optional<String> vacio = Optional.empty();

        // Preguntar
        System.out.println(vacio.isPresent());          // false

        // Valor por defecto: inmediato
        String texto = vacio.orElse("valor por defecto");
        System.out.println(texto);                       // valor por defecto

        // Valor por defecto: perezoso (Supplier)
        String fabricado = vacio.orElseGet(() -> fabricarLento());

        // Si está vacío... revienta con tu excepción
                            //    String    valor    =    vacio.orElseThrow(()    ->    new
IllegalStateException("¡Faltaba!"));

        // Si hay valor, haz algo (Consumer). Si no, no hagas nada.
        vacio.ifPresent(s -> System.out.println(s.toUpperCase()));  // no imprime nada
    }

    static String fabricarLento() {
        return "fabricado con esfuerzo";
    }
}
```

La diferencia entre `orElse` y `orElseGet`: `orElse(x)` ya tiene `x` calculado (el argumento se evalúa siempre, haya o no valor). `orElseGet(() -> ...)` solo fabrica el valor por defecto si la caja está vacía: es perezoso, y si fabricar es caro, te ahorra el gasto.

🎁 Optional y los streams: pareja de baile

Aquí se junta todo: muchas operaciones terminales de streams devuelven un `Optional` porque no pueden garantizar que exista resultado. `findFirst`, `findAny`, `max` y `min` son las típicas.

```java
import java.util.*;
import java.util.stream.*;

public class Pareja {
    public static void main(String[] args) {
        List<String> nombres = List.of("bob", "ana", "carla");

        Optional<String> primeroConA = nombres.stream()
            .filter(n -> n.startsWith("a"))
            .findFirst();

        System.out.println(primeroConA.isPresent());      // true
        primeroConA.ifPresent(System.out::println);       // ana

        Optional<String> ninguno = nombres.stream()
            .filter(n -> n.startsWith("z"))
            .findFirst();
        System.out.println(ninguno.orElse("no hay nadie"));  // no hay nadie
    }
}
```

Si el `filter` no deja pasar a nadie, `findFirst` devuelve una caja vacía… y tú, con `orElse` o `ifPresent`, decides el aterrizaje. Eso sí: **no uses `get()` a lo loco**: si la caja está vacía, `get()` lanza `NoSuchElementException`. Pregunta (`isPresent`) o aterriza (`orElse`) antes de abrir.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** `orElse(null)` es legal pero es volver a casa con las manos vacías: mejor `orElse("algo sensato")` o `orElseThrow(...)`. Si vas a devolver `null`, ¿para qué envolviste la ausencia en una caja?

Ejercicio: el ascensor de la caja

```java
import java.util.*;
import java.util.stream.*;

public class Ascensor {
    public static void main(String[] args) {
        List<Integer> notas = List.of();
        Optional<Integer> maxima = notas.stream()
            .max(Integer::compareTo);
        System.out.println(maxima.orElse(-1));
    }
}
```

¿Qué imprime?

- (A) `-1`
- (B) `0`
- (C) `Optional.empty`
- (D) Una `NoSuchElementException`

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué pasa si haces `Optional.of(null)`?
2. ¿Cómo lees el valor de un `Optional` sin arriesgarte a una excepción?
3. ¿Qué diferencia hay entre `orElse` y `orElseGet`?
4. ¿Qué métodos de streams devuelven `Optional`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **`Optional`** es una caja que contiene un valor o está vacía: sustituye al `null` a secas y hace explícita la ausencia.

2. Se crea con `of`, `ofNullable` o `empty`, y se consume con `orElse`, `orElseGet`, `orElseThrow` e `ifPresent`.
3. Muchos métodos de streams (`findFirst`, `max`, `min`) devuelven `Optional`: la caja es el puente entre la pereza del stream y tu decisión sobre la ausencia.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `Optional<T>` | Caja que envuelve un valor o su ausencia |
> | `Optional.of(v)` | Caja llena (prohíbe `null`) |
> | `Optional.ofNullable(v)` | Caja que admite `null` (si es null, queda vacía) |
> | `Optional.empty()` | Caja vacía |
> | `orElse(x)` / `orElseGet(s)` | Valor por defecto (inmediato / perezoso) |
> | `orElseThrow()` | Lanza una excepción si está vacío |
> | `ifPresent(c)` | Ejecuta un `Consumer` solo si hay valor |

📚 Volver al índice de la unidad · **Anterior:** 07 · Del stream a la colección: toList, Collectors, groupingBy · **Siguiente:** 09 · Repaso interactivo

09 · Repaso interactivo: domina la programación funcional

El cierre práctico de la unidad, con un pipeline que parece magia y un Optional que esconde un susto 🎯

> 🗺️ **Estás en:** 🎯 **U13 · Programación funcional (Streams y Lambdas)** → 09 · Repaso interactivo

---

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```
import java.util.*;
import java.util.stream.*;

public class Misterio {
    public static void main(String[] args) {
        List<String> nombres = List.of("ana", "bob", "carla", "david", "eva");

        long r = nombres.stream()
            .filter(n -> n.length() > 3)
            .map(String::toUpperCase)
            .count();

        System.out.println(r);
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **5** → El `filter` no filtra nada y el `map` convierte todos. ❌
2. **3** → `filter` deja pasar a `carla`, `david` y `eva`... ¿no? ❌

3. **2** → ✅ Correcto de verdad: de `ana, bob, carla, david, eva` pasan el filtro `carla` y `david` (las únicas con más de 3 letras). `map(String::toUpperCase)` las pone en mayúsculas, pero no cambia la cuenta. `count()` devuelve **2**.

> 🔄 **Solución**

🔥 Fireside Chat: el Bucle For vs el Stream

> *Dos maneras de recorrer una lista se encuentran frente a la máquina de café.*

Bucle For: — Mira, yo soy el clásico. `for (String s : lista) { ... }`. Voy elemento a elemento, decido yo el ritmo, puedo parar cuando quiero. El control total.

Stream: — ¡Ja! Tú controlas el *cómo*. Yo solo necesito que me digas el *qué*: `lista.stream().filter(...).map(...).toList()`. Declaras la receta y yo cocino.

Bucle For: — ¿Y cuando necesitas modificar la lista mientras la recorres? Yo te dejo, con cuidado. Tú ni lo intentas.

Stream: — Para eso existen las colecciones de salida. Y cuando te olvidas del `break` y recorres mil elementos de más, no me vengas llorando. Además, yo puedo decir `limit(3)` y me paro solo. ¿Tú sabes lo que es pereza productiva?

Bucle For: — Vale, vale… pero mis primitivos y mis índices siguen siendo útiles para cosas simples.

Stream: — Nadie dice que no. Yo brillo cuando el pipeline se vuelve complejo: filtrar, transformar, ordenar y agrupar en tres líneas en vez de tres bucles anidados. Tú para lo simple, yo para lo declarativo. Tregua.

Bucle For: — Tregua. Al final, los dos recorremos.

> La lección: **usa el bucle cuando el control importe y el stream cuando quieras declarar el resultado.** No es una guerra: es saber cuándo usar cada herramienta. El stream no te odia: te está quitando trabajo.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy la flecha que convierte un bloque de código en un valor que viaja solo.**
2. **Soy la interfaz funcional que responde `true` o `false` a una pregunta.**
3. **Soy el operador que te deja reciclar un método como si fuera una lambda.**
4. **Soy el flujo perezoso que no hace nada hasta que abres el grifo.**
5. **Soy el grifo final: sin mí, el stream no fluye.**
6. **Soy la caja que acepta la ausencia.**

> 🔄 **Respuestas**

🦉 CONRAD VS EL MUNDO: “El que olvida el grifo”

> *CONRAD, nuestro compilador cascarrabias, opina sobre los clásicos del novato.*

CONRAD: — ¡OTRA VEZ! Un alumno me enseña su programa y se queja: *“CONRAD, no me imprime nada”*. Y yo: ¿has puesto una operación terminal? *“¿Una qué?”* ¡AY, MADRE MÍA! ¡`lista.stream().filter(...).map(...)` sin terminal no hace NADA! El stream es perezoso, como mi paciencia. Pónle un `collect`, un `count` o un `forEach`, o la cinta se queda parada para siempre.

Y luego está el que intenta recorrer el mismo stream dos veces. “¿Por qué me da error al segundo `count()`?” ¡Pues porque la cinta ya se vació! Un stream se consume con la primera terminal. Si quieres contar dos cosas, crea dos streams. Es como querer comer la misma pizza dos veces: imposible, ya está en tu estómago.

Y el colmo de los colmos: el que hace `Optional.of(objetoQuePuedeSerNull)` y se extraña de la `NullPointerException`. ¡`of` es para lo seguro! Si puede venir `null`, `ofNullable`. Y no me vengas con `get()` a ciegas sobre una caja que puede estar vacía: eso es `NoSuchElementException` asegurada. `orElse`, `ifPresent`, `orElseThrow`… la caja tiene puerta, úsala.

La lección: antes de acusar a Java de “odiarte”, repasa el trío sagrado de la unidad: **¿hay una operación terminal? ¿estoy reutilizando un stream ya consumido? ¿estoy abriendo un `Optional` vacío sin red?** El compilador no te odia: te está pasando las respuestas del examen.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. `nombres.stream().map(String::length)` sin operación terminal… ¿se ejecuta el `map`?
    - a) Sí, en el acto b) No, hasta que llegue una terminal
2. ¿Puedes hacer `lista.add(...)` sobre una lista devuelta por `toList()`?
    - a) Sí b) No: es inmutable
3. `Optional<String> o = Optional.empty(); o.get()`… ¿qué pasa?
    - a) Devuelve `null` b) Lanza `NoSuchElementException`
4. `reduce(0, Integer::sum)` sobre un `Stream<Integer>`… ¿qué devuelve?
    - a) `int` b) `Optional<Integer>`
5. `Collectors.groupingBy(Function.identity())` devuelve un…
    - a) `Map<T, List<T>>` b) `List<T>`
6. Después de `stream.count()`, ¿puedes usar el mismo stream para otra cosa?
    - a) Sí b) No: queda consumido

> 🔄 **Soluciones**

⚡ Laboratorio de tortura: la estadística que no arranca

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Te dicen que tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto… pero, ¿y si te lo dicen mal? Tu tarea: hacer que compile, que ejecute y que **toda** la salida sea correcta, contando tú los errores reales.

```java
import java.util.*;
import java.util.stream.*;

public class Estadisticas {

    public static void main(String[] args) {
        List<Integer> notas = List.of(7, 8, 3, 9, 5);

        long aprobados = notas.stream()
            .filter(n -> n >= 5)
            .count;                                    // línea A

        List<Integer> dobladas = notas.stream()
            .map(n -> n * 2)
            .toList();                                 // línea B

        int total = dobladas.stream()
            .reduce(0, (a, b) -> a + b);               // línea C

        System.out.println(aprobados + " " + dobladas + " " + total);
    }
}
```

Fallo intencionado: el número de errores del enunciado está mal a propósito: si buscas el "tercer error de compilación" y "el error de lógica", te vuelves loco. La trampa es tu confianza en el enunciado.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún error de compilación? *sí → busca la línea sospechosa.*

   > **¿Y si sigo atascado?**

2. ¿Compila ya? *sí → ahora piensa en la salida.*

   > **¿Y si sigo atascado?**

3. ¿Crees que hay un error de lógica que cambia el resultado? *Vuelve a leer el código con calma y no te fíes del enunciado.*

> Solución final

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 El Arquero	Explicar la sintaxis `->` de las lambdas con un ejemplo de un parámetro sin paréntesis
🏅 El Sastre	Vestir una lambda con `Predicate`, `Function`, `Consumer` y `Supplier` sin consultar la chuleta
🏅 El Reciclador	Reescribir tres lambdas como referencias a método (`::`)
🏅 El Maestro de la Cinta	Montar un pipeline `filter → map → sorted → limit → collect` y predecir su salida
🏅 El Aceptador	Gestionar un `Optional` vacío con `orElse` e `ifPresent` sin lanzar ninguna excepción

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```
import java.util.*;
import java.util.stream.*;

public class Acertijo {
    public static void main(String[] args) {
        List<String> palabras = List.of("sol", "mar", "luna", "sol", "luna", "cielo");
        long distintas = palabras.stream().distinct().count();
        List<String> orden = palabras.stream().distinct().sorted().toList();
        System.out.println(distinctas + " " + orden);
    }
}
```

2. **La media de aprobados:** tienes una `List<Integer>` con notas. ¿Cómo calculas, con streams, la media de las notas que son 5 o más? ¿Qué devuelve `average()` si ninguna

nota aprueba?

3. **El detective del pipeline:** ¿por qué `stream.map(...).filter(...)` sin operación terminal no imprime absolutamente nada?

4. **Verdadero o falso:** "un `Stream` se puede recorrer tantas veces como quieras".

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Expresión sin nombre que viaja como un valor (6 letras)
3. Operación terminal que combina todos los elementos en uno (6 letras)
5. Clase que envuelve un valor o su ausencia (8 letras)
7. Operación intermedia que deja pasar solo lo que cumple una condición (6 letras)

Vertical:
2. Interfaz funcional que devuelve `boolean` (9 letras)
4. El flujo de datos perezoso de Java (6 letras)
6. Operador que transforma cada elemento (3 letras)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, qué es una lambda."**
2. **"¿Qué es una interfaz funcional y qué tiene que ver con las lambdas?"**
3. **"¿Cuál es la diferencia entre una operación intermedia y una terminal en un stream?"**
4. **"¿Por qué `Optional` es mejor que devolver `null`?"**
5. **"¿Qué hace `groupingBy` y qué devuelve?"**
6. **"¿Cuál es la diferencia entre `map` y `reduce`?"**

🤷 No hay preguntas tontas

> ❓ **¿Una lambda es un objeto?**

Sí. Una lambda es un objeto de la clase anónima que implementa su interfaz funcional. `Predicate<Integer> p = n -> n % 2 == 0;` crea por debajo un objeto que implementa `Predicate<Integer>`. El compilador hace la magia; tú solo escribes la flecha.

> ❓ **¿Puedo guardar una lambda en una variable?**

Sí, pero no en una variable de tipo `int` ni `String`: necesita una variable de tipo **interfaz funcional**. `Predicate<Integer> esPar = n -> n % 2 == 0;` es perfectamente legal. La lambda viaja como cualquier otro valor.

> ❓ **¿Un `Stream` guarda datos? ¿Y cuándo se vacía?**

No guarda nada: es una vista sobre una fuente (lista, array…). Se vacía en cuanto llega una operación terminal. Los datos originales siguen intactos en su colección; lo que se consume es la cinta.

> ❓ **¿Puedo usar streams con arrays, que en la U06 eran otra cosa?**

Claro: `Arrays.stream(miArray)` convierte un array en flujo, y `Arrays.stream(int[])` además da un `IntStream` con métodos como `sum()` y `average()`. Los arrays y los streams no compiten: se pasan el relevo.

🎬 Poscréditos

La programadora acaba su analizador de notas: un pipeline de streams que filtra los aprobados, calcula la media con `average()` y guarda el resultado en un `Optional` por si un día no hay notas que analizar. Todo perfecto… hasta que cierra el programa y sus estadísticas desaparecen.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — ¿Has visto? Todo vive en memoria RAM. Cierras la app y adiós, estadísticas. Un stream es fantástico para procesar datos mientras el programa corre, pero para que sobrevivan al reinicio necesitas algo más.

Programadora: — ¿Guardarlas en un archivo? ¿Como un fichero de texto en el disco?

CONRAD: — *asiente* Exacto. Ficheros, y cuando los datos crezcan, expresiones regulares para buscar patrones. Tu cinta transportadora procesa; el fichero conserva. Todo eso es otra historia… y casi te toca.

PROXIMAMENTE EN U14: Ficheros y expresiones regulares (Regex). Guarda los resultados de tus streams en disco y busca patrones con expresiones regulares. 📁

📚 Volver al índice de la unidad · **Anterior:** 08 · Optional: el que acepta la ausencia · **Siguiente:** U14 · Ficheros y expresiones regulares (Regex)

U14 — Ficheros y expresiones regulares (Regex)

El GPS de los archivos, el lápiz que no olvida, la puerta que se cierra sola y el detective de patrones: File, FileWriter, BufferedReader, NIO, serialización y regex 📁

En la U12 viste colecciones, genéricos y mapas: tus datos por fin viven en memoria como quisieras. Pero hay un problema: cuando el programa termina, todo se esfuma. Los arrays, los `ArrayList` y los `HashMap` mueren con el `main`. Hoy eso se acaba. Los **ficheros** hacen que tus datos sobrevivan a tu programa, y las **expresiones regulares** te dan vista de superhéroe para buscar, validar y extraer cualquier patrón en un texto.

Esta unidad tiene tres grandes actos:

- **Ficheros de texto:** la clase `File`, el GPS de los archivos; `FileWriter` y `FileReader` + `BufferedReader`, el lápiz y el lector de siempre; `try-with-resources`, la puerta que se cierra sola; y `PrintWriter` y `Scanner`, el sargento que formatea y el pulpo que tokeniza.
- **La forma moderna y la de élite:** la API **NIO** (`Files` y `Paths`), que hace en una línea lo que antes costaba cinco, y la **serialización**, que guarda objetos enteros en un archivo como quien mete ropa en una maleta.
- **Expresiones regulares:** `Pattern` y `Matcher`, la tabla de símbolos, y sus aplicaciones con `matches()`, `replaceAll()`, `split()` y grupos de captura para validar correos, DNIs y teléfonos como un profesional.

Por el camino entenderás por qué `new File("ruta")` no crea nada, qué es eso del "infierno de las contrabarras" (`\\d`), por qué no cerrar un archivo puede costarte los datos, y por qué una regex no sirve para validar HTML (por más que lo intentes).

> 📌 **Nota:** la consola (`System.out`, `printf`, `Scanner` por teclado) ya la viste en la **U03**, punto 7. Aquí el `Scanner` y el `PrintWriter` vuelven, pero jugando en su liga: leyendo y escribiendo archivos.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Localizar y describir archivos y carpetas con la clase **`File`** (`exists`, `isFile`, `isDirectory`, `length`, `listFiles`).
- Escribir texto con **`FileWriter`** (sobrescribiendo o añadiendo) y leerlo con **`FileReader`** + **`BufferedReader`** línea a línea.
- Cerrar recursos sin llorar con **`try-with-resources`** y explicar por qué el `close()` importa.
- Formatear la salida con **`PrintWriter`** y leer archivos con **`Scanner`** (incluido `useDelimiter`).
- Manejar archivos con la API moderna **NIO** (`Files` y `Paths`): leer, escribir y consultar en una línea.
- Guardar y recuperar objetos enteros con **`ObjectOutputStream`** y **`ObjectInputStream`** (serialización).
- Crear y compilar **expresiones regulares** con `Pattern` y `Matcher`, y dominar los símbolos básicos.
- Aplicar regex con **`matches()`, `replaceAll()` y `split()`**, y usar **grupos de captura** para validar y extraer.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · La clase File: tu explorador de archivos	El GPS que localiza archivos sin leer su contenido	Todos
02 · Escribir y leer texto: FileWriter, FileReader y BufferedReader	El lápiz que no olvida y el lector que no se atraganta	Todos
03 · try-with-resources: cerrar sin llorar	La puerta que se cierra sola: adiós al `close()` manual	Todos
04 · PrintWriter y Scanner + File	El sargento que formatea y el pulpo que tokeniza	Todos
05 · NIO: Files y Paths (la forma moderna)	Leer, escribir y consultar archivos en una línea	Todos

Punto	Qué aprenderás	Dificultad
06 · Serialización: guardar objetos con ObjectOutputStream	Tu objeto entero en una maleta: `Serializable`	Todos
07 · Expresiones regulares: Pattern y Matcher	El molde, el texto y el infierno de las contrabarras	Todos
08 · Regex en acción: matches, replaceAll, split y validaciones	Validar correos, DNIs, teléfonos y extraer con grupos	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

✅ Criterios de evaluación cubiertos (RA5)

RA5: Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases.

CE	Criterio	Dónde se cubre
RA5 c)	Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas.	✅ Puntos 1, 4 y 5
RA5 d)	Se han utilizado ficheros para almacenar y recuperar información.	✅ Puntos 1, 2, 3, 4, 5 y 6

CE	Criterio	Dónde se cubre
RA5 e)	Se han creado programas que utilicen diversos métodos de acceso al contenido de los ficheros.	✅ Puntos 2, 4, 5 y 6

> 📌 Los CEs **RA5 a)** (consola) y **RA5 b)** (formatos) se cubrieron en la **U03**, punto 7. Esta unidad completa la RA5 con los ficheros y las librerías de E/S.

✅ Criterios de evaluación cubiertos (RA6)

RA6: Escribe programas que manipulen información, seleccionando y utilizando tipos avanzados de datos.

CE	Criterio	Dónde se cubre
RA6 g)	Se han utilizado expresiones regulares en la búsqueda de patrones en cadenas de texto.	✅ Puntos 7, 8 y 9

> 📌 La **RA6** se completa con la **U06** (arrays, CE a), la **U12** (colecciones, genéricos y mapas, CEs b, c, d, e y f). Esta unidad aporta el CE g: las expresiones regulares.

🚪 ¿Por dónde empiezo?

- ¿Vienes de la U12 y quieres lo esencial? → Arranca en el punto 1 y el punto 2: localizar y leer archivos de texto.
- ¿Ya sabes escribir y leer y quieres la forma moderna? → Salta al punto 3 (cerrar sin llorar) y al punto 5 (NIO en una línea).
- ¿Solo vienes a por las regex? → Ve directo al punto 7 y al punto 8: el detective de patrones.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · La clase File: tu explorador de archivos

⏭️ **Al acabar la unidad, continúa en U15 · Persistencia de datos: JDBC e introducción a ORM.**

01 · La clase File: tu explorador de archivos

El GPS que localiza archivos sin leer su contenido: existe, es archivo, es carpeta, cuánto pesa y qué hay dentro 🗂️📍

> 🗺️ **Estás en:** 📁 **U14 · Ficheros y expresiones regulares (Regex)** → 01 · La clase File: tu explorador de archivos

---

📬 La idea en una frase

> **`File` no es el contenido de un archivo: es la tarjeta de visita que te dice dónde está, cómo se llama, cuánto pesa y si existe. Antes de leer o escribir, pregúntale al GPS.**

En la U12 guardabas tus datos en memoria: listas, mapas, objetos… todo vivo mientras el programa corre. Pero ¿y si quieres saber si un archivo existe antes de abrirlo? ¿Y si necesitas recorrer las carpetas de tu disco? Ahí entra la clase `File`. No lee ni escribe nada: solo explora y describe. Es el portero del edificio de los ficheros.

🗂️ File: el GPS de tus archivos

La clase `File` (en `java.io`) representa una **ruta** en tu sistema: un archivo o una carpeta. Lo más importante que debes grabar a fuego: **`new File("ruta")` no crea nada**. Solo crea el objeto que representa esa ruta. Crear el archivo de verdad es otra historia (lo verás en el punto 2).

```
import java.io.File;

File f = new File("C:/datos/notas.txt");
System.out.println("¿Existe? " + f.exists());           // true o false
System.out.println("¿Es archivo? " + f.isFile());         // ¿es un fichero y no una carpeta?
System.out.println("¿Es carpeta? " + f.isDirectory());   // ¿es un directorio?
System.out.println("Tamaño: " + f.length() + " bytes");  // peso en bytes
System.out.println("Ruta absoluta: " + f.getAbsolutePath()); // la dirección completa
System.out.println("Nombre: " + f.getName());                // solo el nombre con extensión
```

> 💡 **Consejo:** en Windows usa `/` o `\\`. Escribe `"C:/datos/notas.txt"` o `"C:\\datos\\notas.txt"`. Con una sola `\` Java cree que empiezas un carácter especial (`\n`, `\t`…) y te vas a liar. Esto se pone interesante con las regex en el punto 7: las contrabarras se duplican por todas partes.

El repertorio imprescindible

Método	Qué te dice
`exists()`	¿Existe la ruta?
`isFile()`	¿Es un archivo (y no una carpeta)?
`isDirectory()`	¿Es una carpeta?
`length()`	Tamaño en bytes
`getName()`	Nombre del archivo (sin la ruta)
`getAbsolutePath()`	Ruta completa
`getParent()`	La carpeta que lo contiene
`listFiles()`	Los archivos y carpetas que hay dentro (si es un directorio)
`createNewFile()`	Crea el archivo vacío (devuelve `false` si ya existe)
`mkdir()` / `mkdirs()`	Crea la carpeta (con `mkdirs`, crea también las intermedias)

> ⚠️ **Advertencia:** `File` **no lee contenido**. Le preguntas *"¿existe? ¿cuánto pesa? ¿es carpeta?"* y te contesta. Para leer lo de dentro, ya llegarán `FileReader`, `BufferedReader`, `Scanner` y `Files.readAllLines()` en los próximos puntos. Cada uno a su oficio.

📁 Recorrer una carpeta con listFiles

`listFiles()` te devuelve un array con todo lo que hay dentro de una carpeta. Perfecto para el "explorador de archivos" casero:

```java
import java.io.File;

public class Explorador {
    public static void main(String[] args) {
        File carpeta = new File("C:/datos");
        if (carpeta.exists() && carpeta.isDirectory()) {
            File[] contenidos = carpeta.listFiles();
            for (File item : contenidos) {
                String tipo = item.isDirectory() ? "[CARPETA] " : "[ARCHIVO] ";
                   System.out.println(tipo + item.getName() + " (" + item.length() + "
bytes)");
            }
        } else {
            System.out.println("Esa ruta no es una carpeta existente.");
        }
    }
}
```

Recuerda comprobar `exists()` e `isDirectory()` antes de `listFiles()`: si la ruta no existe, `listFiles()` devuelve `null` y el `for-each` te estalla en la cara.

📝 La ruta no es el contenido

Un error típico de novato es creer que `new File("notas.txt")` "crea" el archivo o que al construir un `File` ya "lees" algo. No:

```java
File f = new File("notas.txt");   // Solo representa la ruta. No existe nada nuevo
en disco.
boolean creado = f.createNewFile();  // ESTO sí intenta crearlo. Devuelve false si ya
existía.
```

`createNewFile()` crea un archivo **vacío** y devuelve `true` si lo ha creado, o `false` si ya existía. Si la carpeta no existe, lanza `IOException`. El patrón habitual: comprueba `exists()` antes de decidir qué hacer.

⭐ Sé el Código, my friend...

> 👓 **Don Tip:** File representa rutas, no contenido. Para leer el contenido usa `Scanner`, `BufferedReader` o `Files.readAllLines()`. El GPS no te cuenta lo que hay dentro del edificio: solo te lleva a la puerta.

Ejercicio: el detective de archivos

```
import java.io.*;

public class DetectiveDeArchivos {
    public static void main(String[] args) throws IOException {
        File f = new File("misterio.txt");
        if (!f.exists()) {
            System.out.println("Creando archivo...");
            FileWriter w = new FileWriter(f);
            w.write("Tres\npalabras\nmisteriosas\n");
            w.close();
        }
        BufferedReader r = new BufferedReader(new FileReader(f));
        String s = "";
        String linea;
        while ((linea = r.readLine()) != null) {
            s = linea + " " + s;
        }
        r.close();
        System.out.println(s);
    }
}
```

¿Qué imprime la PRIMERA vez? ¿Y la SEGUNDA vez (cuando el archivo ya existe)?

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Crea algo `new File("notas.txt")` en el disco?
2. ¿Qué método devuelve `true` si una ruta es una carpeta?
3. ¿Qué devuelve `createNewFile()` si el archivo ya existe?
4. ¿Para qué sirve `length()`?

> 🔁 Respuestas

✅ Resumen en 3 frases

1. **`File`** es un GPS de rutas: te dice si existen, si son archivo o carpeta, cuánto pesan y qué contienen (`listFiles()`), pero **no lee contenido**.
2. `new File("ruta")` **no crea nada**: crea el objeto que representa la ruta. Para crear, `createNewFile()`, `mkdir()`/`mkdirs()` o un `FileWriter`.
3. Antes de operar con una ruta, comprueba `exists()` e `isFile()`/`isDirectory()`: los métodos devuelven `false` o `null` si la ruta no es la esperada.

> 🦎 **Vocabulario rápido**

Término	Idea general
`File`	Objeto que representa una ruta (archivo o carpeta) en el sistema
Ruta	La "dirección" de un archivo: `C:/datos/notas.txt`
Ruta relativa	Dirección desde el directorio de trabajo (`"notas.txt"`)
Ruta absoluta	Dirección completa desde la raíz del disco
`listFiles()`	Devuelve los archivos/carpetas que hay dentro de un directorio

📁 Volver al índice de la unidad · **Anterior:** — · **Siguiente:** 02 · Escribir y leer texto: FileWriter, FileReader y BufferedReader

02 · Escribir y leer texto: FileWriter, FileReader y BufferedReader

El lápiz que no olvida y el lector que no se atraganta: escribe archivos con FileWriter y léelos línea a línea con BufferedReader ✍️📖

> 🗺️ **Estás en:** 🗂️ **U14 · Ficheros y expresiones regulares (Regex)** → 02 · Escribir y leer texto: FileWriter, FileReader y BufferedReader

---

📬 La idea en una frase

> **`FileWriter` es el lápiz que escribe caracteres en un archivo y `FileReader` el lector que los recupera; con `BufferedReader` encima, leer línea a línea es coser y cantar.**

En el punto 1 `File` solo te decía si el archivo existía y cuánto pesaba. Ahora toca lo bueno: **escribir** contenido y **leerlo**. Para escribir tienes `FileWriter`; para leer, `FileReader`. Y si encima envuelves el lector con `BufferedReader`, podrás leer archivos enteros línea a línea sin despeinarte.

✍️ Escribir con FileWriter

`FileWriter` escribe **caracteres** en un archivo. Si el archivo no existe, lo crea; si existe, lo **sobrescribe** por defecto.

```
import java.io.FileWriter;
import java.io.IOException;

public class EscribirTexto {
    public static void main(String[] args) throws IOException {
        FileWriter escritor = new FileWriter("salida.txt");
        escritor.write("Primera línea.\n");
        escritor.write("Segunda línea.\n");
        escritor.close();
    }
}
```

¿Quieres **añadir al final** sin borrar lo anterior? Pásale un segundo argumento `true` (modo *append*):

```
FileWriter writer = new FileWriter("bitacora.txt", true);  // añade al final
```

> ⚠️ **Advertencia:** sin `close()` o `flush()`, los datos se quedan atrapados en el buffer interno y pueden perderse. Es como escribir una carta y no cerrar el sobre: el cartero no la recoge. Lo verás a fondo en el punto 3 con `try-with-resources`.

📖 Leer con FileReader + BufferedReader

`FileReader` lee caracteres sueltos, que es incómodo. La solución: envuelves el `FileReader` con un `BufferedReader` y usas `readLine()`, que te devuelve una línea entera de golpe (y `null` cuando se acaba el archivo).

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LeerTexto {
    public static void main(String[] args) throws IOException {
        BufferedReader lector = new BufferedReader(new FileReader("salida.txt"));
        String linea = lector.readLine();
        while (linea != null) {
            System.out.println(linea);
            linea = lector.readLine();
        }
        lector.close();
    }
}
```

El patrón `while ((linea = lector.readLine()) != null)` es el pan de cada día de Java: lee una línea, comprueba que no es `null`, la procesa y repite. Cuando `readLine()` devuelve `null`, el archivo se ha terminado.

> 📝 **Nota:** la lectura con `Scanner` por teclado la viste en la UO3 (punto 6). Aquí el `Scanner` reaparece en el punto 4, pero leyendo archivos.

🚨 ¿Y si el archivo no existe al leer?

Lanzas un `FileReader` sobre un archivo que no existe y… `FileNotFoundException`. Es hija de `IOException`, así que tienes dos caminos:

1. **Declarar** la excepción en la firma: `throws IOException` (lo que hemos hecho arriba).
2. **Capturarla** con `try-catch` y hacer algo sensato, como avisar al usuario:

```
try {
    BufferedReader lector = new BufferedReader(new FileReader("no_existe.txt"));
    String linea;
    while ((linea = lector.readLine()) != null) {
        System.out.println(linea);
    }
    lector.close();
} catch (IOException e) {
    System.out.println("Error al leer: " + e.getMessage());
}
```

> 💡 **Consejo:** el camino 2 es el profesional: no dejas que el programa muera por un archivo que falta. El camino 1 es rápido para practicar. En el punto 3 verás la combinación definitiva.

🧱 ¿Y por qué BufferedReader y no FileReader a secas?

`FileReader` te da caracteres de uno en uno. Para leer frases necesitas montar un bucle de acumulación. `BufferedReader` te da **líneas enteras** y además es más rápido en archivos grandes porque **lee en bloques** y guarda lo sobrante en su buffer. El patrón "envolver" (`new BufferedReader(new FileReader(...))`) es típico de Java: una clase que añade superpoderes a otra.

Tarea	Herramienta
Escribir caracteres	`FileWriter`
Escribir añadiendo al final	`FileWriter(path, true)`
Leer caracteres	`FileReader`

Tarea	Herramienta
Leer líneas enteras (recomendado)	`BufferedReader` envolviendo `FileReader`
Formatear y escribir con `println`/`printf`	`PrintWriter` (punto 4)

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** el trío clásico para leer archivos de texto en Java es `BufferedReader` + `FileReader`. El bucle `while ((linea = lector.readLine()) != null)` es el estándar que verás en miles de proyectos.

Ejercicio: el contador que no miente

```
import java.io.*;

public class ContadorLineas {
    public static void main(String[] args) throws IOException {
        File f = new File("datos.txt");
        FileWriter w = new FileWriter(f);
        w.write("linea1\nlinea2\nlinea3\n");
        w.close();

        BufferedReader r = new BufferedReader(new FileReader(f));
        int contador = 0;
        while (r.readLine() != null) {
            contador++;
        }
        r.close();
        System.out.println(contador);
    }
}
```

¿Qué imprime? ¿Cuenta las líneas de verdad?

> › 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué hace `FileWriter` con un archivo que ya existe por defecto?
2. ¿Cómo añades texto al final de un archivo sin borrar lo anterior?
3. ¿Qué devuelve `readLine()` cuando el archivo se ha terminado?
4. ¿Por qué se envuelve `FileReader` con `BufferedReader`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. **`FileWriter`** escribe caracteres en un archivo (lo crea si no existe y lo sobrescribe si existe); con el argumento `true` añade al final.
2. **`FileReader` + `BufferedReader`** leen archivos de texto línea a línea con `readLine()`, que devuelve `null` al final.
3. Sin `close()` (o `flush()`) los datos se quedan en el buffer: **cierra siempre** o usa `try-with-resources` (punto 3).

> 🦎 **Vocabulario rápido**

Término	Idea general
`FileWriter`	Escribe caracteres en un archivo
`FileReader`	Lee caracteres de un archivo
`BufferedReader`	Lector con buffer que permite `readLine()`
`readLine()`	Devuelve la siguiente línea o `null` si no queda ninguna
Append	Añadir al final del archivo sin borrar lo anterior
`FileNotFoundException`	Se lanza al leer un archivo que no existe

📁 Volver al índice de la unidad · **Anterior:** 01 · La clase File: tu explorador de archivos · **Siguiente:** 03 · try-with-resources: cerrar sin llorar

03 · try-with-resources: cerrar sin llorar

La puerta que se cierra sola: deja que Java llame a close() por ti, aunque salte una excepción en mitad del camino 🔒🚪

> 🗺️ **Estás en:** 📁 **U14 · Ficheros y expresiones regulares (Regex)** → 03 · try-with-resources: cerrar sin llorar

---

📬 La idea en una frase

> **En vez de acordarte de cerrar cada archivo, le dices a Java "cierra esto cuando acabes, pase lo que pase" y él lo hace solo.**

En el punto 2 viste que sin `close()` los datos pueden perderse. Pero acordarte de cerrar, y encima cuando hay excepciones por el medio, es un dolor. Desde Java 7 existe el `try-with-resources`: abres los recursos dentro de los paréntesis del `try` y Java los cierra automáticamente al salir del bloque, tanto si todo va bien como si salta una excepción.

🔒 El try-with-resources en acción

Mira qué limpio queda el lector del punto 2 sin `close()` manual:

```java
import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;

public class LectorLimpio {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("salida.txt"))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                System.out.println(linea);
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
        // No hay br.close(). Se cierra solo.
    }
}
```

Lo que pasa por debajo: al salir del bloque `try` (con éxito o con excepción), Java llama a `close()` sobre cada recurso que declaraste entre paréntesis. Es el `finally` que te ahorra escribir.

> 💡 **Consejo:** fíjate que el `catch` sigue existiendo. El `try-with-resources` cierra los recursos, pero no traga las excepciones: tú decides cómo gestionarlas.

🧹 Varios recursos a la vez

¿Leer de un archivo y escribir en otro? Declara los dos recursos separados por `;` y Java los cierra **en orden inverso** al declararlos:

```java
import java.io.*;

public class Copiador {
    public static void main(String[] args) {
        try (BufferedReader br = new BufferedReader(new FileReader("origen.txt"));
             PrintWriter pw = new PrintWriter(new FileWriter("copia.txt"))) {

            String linea;
            while ((linea = br.readLine()) != null) {
                pw.println(linea);
            }
            System.out.println("Copia terminada.");
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Es el patrón estándar para copiar archivos: lee línea a línea y escríbela en el destino. Y si uno de los dos no se puede abrir, no pasa nada: Java intenta cerrar los que sí se abrieron.

🤔 ¿Qué recursos valen?

Solo valen los que implementan `AutoCloseable` (o su hija `Closeable`). Las clases de ficheros que hemos visto (`FileReader`, `FileWriter`, `BufferedReader`, `PrintWriter`, `ObjectInputStream`…) la implementan. En general: si la clase tiene `close()`, casi seguro sirve.

> ⚠️ **Advertencia:** `Scanner` también es `AutoCloseable`, así que puedes hacer `try (Scanner sc = new Scanner(new File(...)))`. Ojo, no lo uses para cerrar un `Scanner` que lee del teclado en un programa largo: al cerrarlo, cierras también la entrada estándar y a partir de ahí todo `nextLine()` te da errores. Para el teclado, un `Scanner` normal y un `close()` al final.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** `try-with-resources` no es solo para ficheros: sirve para cualquier recurso que haya que liberar. Si tu clase necesita limpiarse al terminar, que implemente `AutoCloseable` y la metes entre los paréntesis.

Ejercicio: el diario que no olvida (modo append)

```
import java.io.*;
import java.time.LocalDate;

public class Diario {
    public static void main(String[] args) {
        try (FileWriter fw = new FileWriter("diario.txt", true)) {
            fw.write(LocalDate.now() + ": Hoy he aprendido try-with-resources.\n");
        } catch (IOException e) {
            System.out.println("No pude escribir el diario: " + e.getMessage());
        }
    }
}
```

¿Qué pasa si ejecutas el programa dos veces? ¿Y qué hace el `LocalDate.now()`?

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué hace Java al salir del bloque de un `try-with-resources`?
2. ¿En qué orden se cierran varios recursos declarados en el `try`?
3. ¿Qué interfaz debe implementar un recurso para poder usarlo en `try-with-resources`?
4. ¿El `try-with-resources` elimina la necesidad de `catch`?

> 🔄 Respuestas

✅ Resumen en 3 frases

1. **`try-with-resources`** declara los recursos entre paréntesis del `try` y Java los cierra solo al salir, con éxito o con excepción: adiós al `close()` olvidado.
2. Puedes abrir **varios recursos a la vez** (separados por `;`) y se cierran en orden inverso.
3. Solo valen recursos que implementen **`AutoCloseable`/`Closeable`**, que es el caso de todas las clases de ficheros de Java.

🦎 Vocabulario rápido

Término	Idea general
`try-with-resources`	Bloque `try` que cierra sus recursos automáticamente
`AutoCloseable`	Interfaz con `close()` que marca un recurso como "liberable"
`Closeable`	Subinterfaz de `AutoCloseable`, la que usan los streams clásicos
Recurso	Algo que se debe liberar: archivo, conexión, scanner…
Append	Añadir al final de un archivo (segundo argumento `true` de `FileWriter`)

📁 Volver al índice de la unidad · **Anterior:** 02 · Escribir y leer texto: FileWriter, FileReader y BufferedReader · **Siguiente:** 04 · PrintWriter y Scanner + File

04 · PrintWriter y Scanner + File

El sargento que formatea la salida y el pulpo que tokeniza la entrada: PrintWriter para escribir bonito y Scanner para leer archivos a pedazos 🖨️🐙

> 🗺️ **Estás en:** 📁 **U14 · Ficheros y expresiones regulares (Regex)** → 04 · PrintWriter y Scanner + File

---

📬 La idea en una frase

> **`PrintWriter` escribe en archivos con la comodidad de `System.out` (`println`, `printf`) y `Scanner` lee archivos troceándolos en tokens (`nextInt`, `nextLine`, `useDelimiter`).**

En la U03 viste `System.out.printf` y `Scanner` por teclado (puntos 6 y 7). Ahora los dos vuelven, pero apuntando a **archivos** en lugar de a la consola. `PrintWriter` escribe con formato; `Scanner + File` lee archivos como si estuvieras leyendo del teclado, con su `hasNextLine`, `nextInt` y compañía.

🖨️ PrintWriter: System.out, pero en un archivo

`PrintWriter` tiene `println()`, `print()` y `printf()`: exactamente los métodos de la consola, pero escribiendo en un archivo.

```
import java.io.PrintWriter;
import java.io.FileWriter;

public class Informe {
    public static void main(String[] args) throws Exception {
        try (PrintWriter pw = new PrintWriter(new FileWriter("informe.txt"))) {
            pw.println("=== Informe de ventas ===");
            pw.printf("Total: %10.2f €%n", 1234.5678);
            pw.printf("Producto: %-12s Unidades: %d%n", "Camiseta", 42);
            pw.printf("Producto: %-12s Unidades: %d%n", "Chaqueta", 7);
        }
    }
}
```

El `%n` es el salto de línea portable (no uses `\n` en `printf`, que en Windows no pilla). Si necesitas repasar los formatos de `printf` (`%d`, `%f`, `%s`, `%-10s`…), vuelve a la UO3, punto 7.

> 📝 **Nota:** el `printf` con `%10.2f` se ve bien en la consola, pero en un archivo que vaya a abrirse con otro programa, cuidado con los espacios: no todos los editores los respetan igual.

Scanner + File: leer archivos a pedazos

`Scanner` puede leer de un archivo igual que lee del teclado. Envuelve un `File` y a jugar con `hasNextLine()` / `nextLine()`, o con `hasNextInt()` / `nextInt()` para datos sueltos.

```
import java.io.File;
import java.util.Scanner;

public class LeerConScanner {
    public static void main(String[] args) throws Exception {
        try (Scanner sc = new Scanner(new File("notas.txt"))) {
            while (sc.hasNextLine()) {
                System.out.println(sc.nextLine());
            }
        }
    }
}
```

¿Archivos con números separados por espacios? `nextInt()` te los va sirviendo sin que tú te metas en el lío de las líneas:

```
// datos.txt → "10 20 30 40"
try (Scanner sc = new Scanner(new File("datos.txt"))) {
    int suma = 0;
    while (sc.hasNextInt()) {
        suma += sc.nextInt();
    }
    System.out.println("Suma: " + suma);  // Suma: 100
}
```

> ⚠️ **Advertencia:** en el punto 3 lo comentábamos: un `Scanner` en `try-with-resources` es fantástico para archivos. No lo hagas con un `Scanner` del teclado dentro de un programa largo: al cerrarlo cierras `System.in` y todo lo que venga después de leer del teclado se rompe.

🪓 useDelimiter: cortar donde tú digas

Por defecto `Scanner` trocea por espacios y saltos de línea. Pero si tu archivo separa los campos con `;`, le dices con `useDelimiter()` dónde cortar:

```
import java.io.File;
import java.util.Scanner;

public class LeeCSV {
    public static void main(String[] args) throws Exception {
        // datos.csv → "Ana;25;DAM" ... "Bob;22;DAW"
        try (Scanner sc = new Scanner(new File("datos.csv"))) {
            sc.useDelimiter(";|\\R");   // corta por ';' o por salto de línea
            while (sc.hasNext()) {
                String nombre = sc.next();
                int edad = sc.nextInt();
                String ciclo = sc.next();
                System.out.printf("%-8s %2d años %s%n", nombre, edad, ciclo);
            }
        }
    }
}
```

`\\R` es el “cualquier salto de línea” de Java (punto 7, cuando veas regex, esto te sonará). Con `";|\\R"` le dices: corta por punto y coma o por salto de línea. El resultado es un archivo CSV leído sin despeinarte.

🆚 Scanner vs BufferedReader: ¿cuál uso?

Criterio	`Scanner`	`BufferedReader`
Leer por líneas	`nextLine()`	`readLine()`
Leer números	`nextInt()` directo	Tienes que convertir a mano
Delimitadores personalizados	`useDelimiter()`	No

Criterio	`Scanner`	`BufferedReader`
Velocidad en archivos enormes	Suficiente	Mejor (buffer más eficiente)
Lo mejor para	Parsear datos sueltos y tokens	Leer archivos de texto línea a línea

> 💡 **Consejo:** ¿archivo de configuración con números y palabras? `Scanner`. ¿Log de 2 GB que hay que recorrer entero? `BufferedReader`. Para procesar logs con regex ya lo verás en el punto 8.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** `PrintWriter` y `Scanner` son la pareja de moda para "leer, transformar y escribir": lees con `Scanner`, transformas, y escribes con `PrintWriter` formateado. El `printf` de `PrintWriter` es el mismo de siempre, pero con destino de archivo.

Ejercicio: la hoja de cálculo que se hace sola

```java
import java.io.*;
import java.util.Scanner;

public class NotasMedias {
    public static void main(String[] args) throws Exception {
        try (Scanner sc = new Scanner(new File("alumnos.txt"))) {
            sc.useDelimiter(";|\\R");
            PrintWriter pw = new PrintWriter(new FileWriter("medias.txt"));
            while (sc.hasNext()) {
                String nombre = sc.next();
                int n1 = sc.nextInt();
                int n2 = sc.nextInt();
                double media = (n1 + n2) / 2.0;
                pw.printf("%s: %.1f%n", nombre, media);
            }
            pw.close();
        }
        System.out.println("Hecho.");
    }
}
```

El archivo `alumnos.txt` contiene `"Ana;7;9\nBob;5;6"`. ¿Qué contendrá `medias.txt`?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué método de `PrintWriter` te permite formatear con `%d`, `%f`, `%s`?
2. ¿Cómo lees un archivo con `Scanner` en lugar de por teclado?
3. ¿Para qué sirve `useDelimiter(";")`?
4. ¿Qué devuelve `hasNextInt()` cuando el archivo se ha quedado sin números?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **`PrintWriter`** escribe en archivos con `println`, `print` y `printf`: la comodidad de la consola con destino de archivo.
2. **`Scanner + File`** lee archivos a tokens (`nextInt`, `nextLine`), y con **`useDelimiter`** puedes cortar por `;`, comas o lo que necesites.
3. Regla rápida: **`Scanner`** para parsear datos sueltos, **`BufferedReader`** para archivos grandes línea a línea, y ambos dentro de **`try-with-resources`**.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `PrintWriter` | Escritor con `println`/`printf` sobre archivos |
> | `useDelimiter()` | Cambia el separador con el que `Scanner` trocea |
> | Token | Cada pedacito en que `Scanner` corta la entrada |
> | `\\R` | “Cualquier salto de línea” en regex de Java |
> | CSV | Archivo de valores separados por comas (o por `;`) |

📁 Volver al índice de la unidad · **Anterior:** 03 · try-with-resources: cerrar sin llorar · **Siguiente:** 05 · NIO: Files y Paths (la forma moderna)

05 · NIO: Files y Paths (la forma moderna)

Leer, escribir y consultar archivos en una línea: la API NIO llegó para que el viejo File parezca del siglo pasado ⚡📫

> 🗺️ **Estás en:** 📁 **U14 · Ficheros y expresiones regulares (Regex)** → 05 · NIO: Files y Paths (la forma moderna)

📬 La idea en una frase

> **`Path` es la ruta moderna y `Files` su navaja suiza: `readAllLines`, `write`, `readString`… cosas que con `File` costaban cinco líneas y con NIO son una.**

Desde Java 7 existe la API **NIO** (New I/O) en `java.nio.file`. Es el reemplazo moderno de la clase `File`: `Path` para representar rutas y `Files` para hacer todo lo demás. ¿Leer un archivo entero en una lista de líneas? Una línea de código. ¿Escribir una lista en un archivo? Otra. El viejo `File` sigue funcionando, pero cuando veas esto ya no querrás volver.

📫 Path y Paths: el nuevo File

`Path` es la ruta (como `File`, pero más ligera y pensada para componerse). Se crea con la clase `Paths`:

```
import java.nio.file.Path;
import java.nio.file.Paths;

Path ruta = Paths.get("C:/datos/notas.txt");
Path relativa = Paths.get("notas.txt");                // relativa al directorio de trabajo
Path combinada = Paths.get("datos", "sub", "f.txt"); // con separadores a prueba de SO
```

`Paths.get(...)` con varios argumentos construye la ruta usando el separador del sistema operativo (en Windows `\`, en Linux `/`). Es la forma de no liarte con barras.

> 💡 **Consejo:** los métodos de `Path` (`getFileName()`, `getParent()`, `exists()`, `isDirectory()`…) son los mismos conceptos de `File`. La diferencia grande está en `Files`, la clase estática que hace el trabajo pesado.

⚡ Files: la navaja suiza

La clase estática `Files` es donde vive la magia. Estos son los métodos que usarás el 90% del tiempo:

```
import java.nio.file.*;
import java.util.List;

public class NioFacil {
    public static void main(String[] args) throws Exception {
        Path ruta = Paths.get("notas.txt");

        // Escribir una lista de líneas (sobrescribe)
        Files.write(ruta, List.of("Línea 1", "Línea 2", "Línea 3"));

        // Leer todo el archivo en una lista de líneas
        List<String> lineas = Files.readAllLines(ruta);
        for (String linea : lineas) {
            System.out.println(linea);
        }

        // Leer todo el archivo como un único String
        String contenido = Files.readString(ruta);

        // Otras consultas útiles
        System.out.println("¿Existe? " + Files.exists(ruta));
        System.out.println("¿Es archivo? " + Files.isRegularFile(ruta));
        System.out.println("Tamaño: " + Files.size(ruta) + " bytes");
    }
}
```

⚠️ **Advertencia:** `Files.write` y `readAllLines` cargan el archivo entero en memoria. Para archivos gigantes usa `Files.newBufferedReader(ruta)` (como en el punto 2, pero sin `FileReader` en medio) y lee línea a línea.

📋 Otras joyas de Files

Método	Qué hace
`Files.readAllLines(ruta)`	Todo el archivo como `List<String>`
`Files.write(ruta, lista)`	Escribe una lista de líneas (sobrescribe)
`Files.readString(ruta)`	Todo el archivo como un único `String`
`Files.writeString(ruta, texto)`	Escribe un `String`
`Files.newBufferedReader(ruta)`	`BufferedReader` directo sobre un `Path`
`Files.newBufferedWriter(ruta)`	`BufferedWriter` directo sobre un `Path`
`Files.copy(origen, destino)`	Copia un archivo
`Files.move(origen, destino)`	Mueve (o renombra)
`Files.delete(ruta)`	Borra (lanza excepción si no existe)
`Files.deleteIfExists(ruta)`	Borra si existe, sin excepción
`Files.walk(ruta)`	Recorre recursivamente todo un árbol de archivos
`Files.lines(ruta)`	Un `Stream<String>` línea a línea (para archivos grandes)

La pareja `Files.walk` + filtro es la forma elegante de "busca todos los `.java` en una carpeta y sus subcarpetas":

```
import java.nio.file.*;
import java.util.stream.Stream;

try (Stream<Path> rutas = Files.walk(Paths.get("src"))) {
    rutas.filter(p -> p.toString().endsWith(".java"))
         .forEach(System.out::println);
}
```

> 📝 **Nota:** no pasa nada si el `Stream` te abruma: es la UO7 con esteroides. Lo importante ahora es el patrón: `Files.walk` te da todas las rutas y tú filtras la que te interesa.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** NIO usa `Path` y `Files`. Son más modernos y tienen métodos útiles como `Files.walk()` para recorrer árboles. Cuando veas código nuevo de Java, el 80% de las veces usa NIO.

Ejercicio: Be the NIO

```
import java.nio.file.*;
import java.util.*;

public class BeTheNIO {
    public static void main(String[] args) throws Exception {
        Path p = Paths.get("nums.txt");
        Files.write(p, List.of("3", "7", "2", "9", "5"));
        List<String> l = Files.readAllLines(p);
        int suma = 0;
        for (String s : l) {
            suma += Integer.parseInt(s);
        }
        Files.write(p, List.of("Total: " + suma));
        System.out.println(Files.readString(p));
    }
}
```

¿Qué imprime?

> 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Con qué clase creas un `Path`?
2. ¿Qué método de `Files` lee un archivo entero como `List<String>`?
3. ¿Qué método de `Files` recorre un árbol de carpetas recursivamente?
4. ¿`Files.write` añade al final o sobrescribe?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **`Path`** (creado con `Paths.get`) es la ruta moderna y **`Files`** la clase estática que hace el trabajo: `readAllLines`, `write`, `readString`, `copy`, `move`, `delete`, `walk`.
2. Leer o escribir un archivo completo son **operaciones de una línea**, sin `close()` porque NIO se encarga.
3. `Files` carga todo en memoria: para archivos enormes usa `Files.newBufferedReader` o `Files.lines` y procesa en streaming.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | NIO | "New I/O": la API moderna de entrada/salida de `java.nio.file` |
> | `Path` | La ruta moderna (el `File` nuevo) |
> | `Paths` | Fábrica de `Path` |
> | `Files` | Clase estática con las operaciones sobre rutas |
> | `Stream` | Secuencia de datos sobre la que puedes filtrar, transformar y recorrer |

📁 Volver al índice de la unidad · **Anterior:** 04 · PrintWriter y Scanner + File · **Siguiente:** 06 · Serialización: guardar objetos con ObjectOutputStream

06 · Serialización: guardar objetos con ObjectOutputStream

Tu objeto entero en una maleta: implementa Serializable y guarda y recupera objetos completos con ObjectOutputStream y ObjectInputStream 🧳📦

> 🗺️ **Estás en:** 📁 **U14 · Ficheros y expresiones regulares (Regex)** → 06 · Serialización: guardar objetos con ObjectOutputStream

---

📬 La idea en una frase

> **La serialización convierte un objeto entero en una secuencia de bytes que puedes guardar en un archivo, y la deserialización lo reconstruye tal cual. Tu `Persona`, tu `Estudiante`, tu `ArrayList` entero, en una maleta.**

Hasta ahora guardabas texto. Pero ¿y si quieres guardar un **objeto completo** con todos sus atributos? ¿O una `ArrayList` entera? Hacerlo campo a campo en un `FileWriter` es un horror. La serialización te deja "congelar" el objeto, guardarlo en un archivo y "descongelarlo" después con todos sus datos intactos.

🧳 El trato: implementar Serializable

Para que un objeto se pueda serializar, su clase debe implementar la interfaz **`Serializable`**. No tiene métodos: es una *marca* que le dice a Java "este objeto puede congelarse".

```
import java.io.Serializable;

class Persona implements Serializable {
    String nombre;
    int edad;

    Persona(String n, int e) {
        this.nombre = n;
        this.edad = e;
    }
}
```

> ⚠️ **Advertencia:** si un atributo de la clase no es serializable (por ejemplo, un `Thread` o un `Socket`), lanzarás una `NotSerializableException`. Soluciones: marca el atributo como `transient` (no se guarda) o haz que su clase también sea `Serializable`.

📦 Guardar: ObjectOutputStream

Para escribir objetos usas `ObjectOutputStream` envuelto sobre un `FileOutputStream`:

```
import java.io.*;

public class GuardandoObjetos {
    public static void main(String[] args) throws Exception {
        Persona p = new Persona("Luis", 25);

        try (ObjectOutputStream oos = new ObjectOutputStream(
                new FileOutputStream("persona.obj"))) {
            oos.writeObject(p);
        }
        System.out.println("Objeto guardado.");
    }
}
```

> 💡 **Consejo:** la extensión `.obj` (o `.ser`/`.dat`) es puramente convencional: el archivo es binario, no lo abras con el bloc de notas. Si lo abres, verás basura y algún nombre de clase legible: es la firma que Java usa para saber qué deserializar.

🧊 Recuperar: ObjectInputStream

Para leerlo de vuelta, `ObjectInputStream` sobre `FileInputStream`. El resultado es un `Object`, así que necesitas un **casting** (ya sabes: el reparto del `instanceof` de la U11):

```java
import java.io.*;

public class RecuperandoObjetos {
    public static void main(String[] args) throws Exception {
        try (ObjectInputStream ois = new ObjectInputStream(
                new FileInputStream("persona.obj"))) {
            Persona recuperada = (Persona) ois.readObject();
            System.out.println(recuperada.nombre + " tiene " + recuperada.edad);
        }
    }
}
```

> ⚠️ **Advertencia:** `readObject()` puede lanzar `ClassNotFoundException` si la clase no existe al deserializar (por ejemplo, si cambiaste de proyecto o renombraste la clase). Por eso el método declara `throws Exception` o captura las tres: `IOException`, `ClassNotFoundException`.

🧨 ¿Y si la clase cambia entre guardar y leer?

Aquí viene la trampa clásica: guardas el objeto, modificas la clase (añades un campo), y al leer… `InvalidClassException`. Java guarda un identificador de versión de la clase y, si cambia, se queja. La solución: declara tú el identificador:

```java
class Persona implements Serializable {
    private static final long serialVersionUID = 1L;  // yo controlo la versión
    String nombre;
    int edad;
}
```

Con `serialVersionUID` fijo, Java no recalcula la versión y tolera pequeños cambios. Sin él, cualquier retoque en la clase rompe los archivos guardados antes. Es de esas cosas que duelen una vez y luego no se olvidan.

📚 Guardar colecciones enteras

¿Una lista de objetos? También se serializa de una vez. Es la forma más cómoda de “persistir” tus datos de la U12:

```java
import java.io.*;
import java.util.ArrayList;
import java.util.List;

public class GuardandoLista {
    public static void main(String[] args) throws Exception {
        List<Persona> equipo = new ArrayList<>();
        equipo.add(new Persona("Ana", 20));
        equipo.add(new Persona("Bob", 22));
        equipo.add(new Persona("Carla", 19));

                    try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("equipo.obj"))) {
            oos.writeObject(equipo);
        }

                     try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("equipo.obj"))) {
            List<Persona> recuperado = (List<Persona>) ois.readObject();
            for (Persona p : recuperado) {
                System.out.println(p.nombre + " - " + p.edad);
            }
        }
    }
}
```

📝 **Nota:** `ArrayList` y sus clases ya son `Serializable`, y una lista es serializable si sus elementos lo son. Por eso funciona sin tocar nada. Persistir colecciones con serialización es el “guardar partida” de tus programas.

⭐ Sé el Código, my friend…

🕶️ **Don Tip:** la serialización no guarda el código, guarda el **estado**: los valores de los atributos. Para reconstruir el objeto, la clase debe existir y ser compatible. Si necesitas que los datos los lea otro programa (no Java), la serialización no vale: tendrás que usar texto, JSON o XML.

Ejercicio: el objeto que viaja en el tiempo

```java
import java.io.*;

class Nota implements Serializable {
    private static final long serialVersionUID = 1L;
    String texto;
    int prioridad;

    Nota(String t, int p) {
        this.texto = t;
        this.prioridad = p;
    }
}

public class ViajeTemporal {
    public static void main(String[] args) throws Exception {
        Nota original = new Nota("Comprar leche", 5);

                    try (ObjectOutputStream oos = new ObjectOutputStream(new
FileOutputStream("nota.obj"))) {
            oos.writeObject(original);
        }

        original.texto = "Comprar café";

                     try (ObjectInputStream ois = new ObjectInputStream(new
FileInputStream("nota.obj"))) {
            Nota copia = (Nota) ois.readObject();
                    System.out.println("Texto: " + copia.texto + " (prioridad " +
copia.prioridad + ")");
        }
    }
}
```

¿Qué imprime? ¿Cambia el archivo porque modificaste el objeto en memoria antes de leerlo?

> 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué interfaz debe implementar una clase para poder serializarse?
2. ¿Qué excepción lanzas si un atributo no es serializable?
3. ¿Qué hace `transient` en un atributo?
4. ¿Para qué sirve `serialVersionUID`?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Una clase **`Serializable`** (sin métodos) puede congelarse: `ObjectOutputStream.writeObject` la guarda en un archivo binario y `ObjectInputStream.readObject` la reconstruye con un casting.
2. Si un atributo no es serializable, `NotSerializableException`; márcalo **`transient`** para excluirlo.
3. Fija un **`serialVersionUID`** para que tus archivos sobrevivan a pequeños cambios en la clase, y recuerda que la serialización guarda una **foto** del estado, no el código.

> 🐛 **Vocabulario rápido**

Término	Idea general
Serializar	Convertir un objeto en bytes para guardarlo o enviarlo
Deserializar	Reconstruir el objeto a partir de los bytes
`Serializable`	Interfaz-marca que permite serializar una clase
`transient`	Atributo que no se guarda al serializar
`serialVersionUID`	Versión de la clase para compatibilidad
Binario	Archivo que no es texto legible (`.obj`, `.dat`, `.ser`)

📁 Volver al índice de la unidad · **Anterior:** 05 · NIO: Files y Paths (la forma moderna) · **Siguiente:** 07 · Expresiones regulares: Pattern y Matcher

07 · Expresiones regulares: Pattern y Matcher

El molde, el texto y el infierno de las contrabarras: aprende a crear patrones con Pattern y a buscarlos con Matcher 🎭 🧩

> 🗺️ **Estás en:** 📁 **U14 · Ficheros y expresiones regulares (Regex)** → 07 · Expresiones regulares: Pattern y Matcher

---

📬 La idea en una frase

> **Una expresión regular (regex) es un patrón que describe un conjunto de cadenas: `"\\d+"` significa "uno o más dígitos". Con `Pattern` la compilas y con `Matcher` la buscas dentro de un texto.**

Hasta aquí guardabas datos en archivos. Ahora toca la otra mitad de la unidad: **buscar patrones dentro de texto**. ¿Cuántas veces has querido "encontrar todos los números" o "comprobar que esto parece un correo"? Las expresiones regulares son el lenguaje para describir esas búsquedas. Se usan en casi todos los lenguajes y, una vez las aprendes, no hay texto que se te resista.

⚠️ El infierno de las contrabarras (léelo dos veces)

En Java, las regex se escriben como **cadenas** y, dentro de un `String`, la contrabarra `\` es un carácter de escape. Para escribir un `\d` de regex dentro de un `String` tienes que duplicarla:

Lo que quieres en la regex	Lo que escribes en Java
`\d` (dígito)	`"\\d"`
`\.` (punto literal)	`"\\."`
`\s` (espacio)	`"\\s"`

> ⚠️ **Advertencia:** si escribes `"\d"` Java intenta escapar la `d`, no sabe qué es y te da un error de compilación (o peor, un comportamiento raro). El "infierno de las contrabarras" es la causa

> número 1 de que una regex “no funcione” cuando acabas de empezar.

🔍 Pattern y Matcher: el molde y el texto

El flujo clásico son dos pasos:

- **`Pattern`**: la expresión regular **compilada**. Es el molde.
- **`Matcher`**: se aplica a un **texto concreto** y va buscando coincidencias. Es el detector.

```
import java.util.regex.Pattern;
import java.util.regex.Matcher;

public class PrimerRegex {
    public static void main(String[] args) {
        Pattern patron = Pattern.compile("\\d+");   // uno o más dígitos (el molde)
        Matcher matcher = patron.matcher("Hay 123 manzanas y 456 peras");

        while (matcher.find()) {
            System.out.println("Encontrado: " + matcher.group()
                + " (posición " + matcher.start() + "-" + matcher.end() + ")");
        }
    }
}
```

Salida:

```
Encontrado: 123 (posición 4-7)
Encontrado: 456 (posición 21-24)
```

- `find()` avanza buscando la siguiente coincidencia y devuelve `true` mientras encuentre.
- `group()` devuelve el texto que ha coincidido.
- `start()` y `end()` devuelven dónde empieza y acaba la coincidencia.

> 💡 **Consejo:** compila el `Pattern` **una sola vez** fuera de bucles y reutilízalo. `Pattern.compile()` es caro (analiza la regex y monta una máquina de estados). Si lo llamas dentro de un bucle de 1 millón de líneas, te lo harás mirar.

🧩 La tabla de símbolos (tu chuleta de examen)

Símbolo	Significado	Ejemplo
`.`	Cualquier carácter (excepto salto de línea)	`c.sa` → “casa”, “cose”
`\d`	Dígito (0-9)	`\d{3}` → “123”
`\D`	NO dígito	`\D+` → “Hola”
`\w`	Letra, dígito o _	`\w+` → “Hola_123”
`\W`	NO `\w`	`\W` → “.”, “ “
`\s`	Espacio en blanco	`\s+` → separadores
`\S`	NO espacio	`\S+` → palabras
`*`	0 o más veces	`a*` → “”, “a”, “aa”
`+`	1 o más veces	`a+` → “a”, “aa”
`?`	0 o 1 vez (opcional)	`colou?r` → “color”, “colour”
`{n}`	Exactamente n	`\d{3}` → tres dígitos
`{n,m}`	Entre n y m	`\d{2,4}` → de 2 a 4 dígitos
`[abc]`	Uno del conjunto	`[aeiou]` → vocales
`[a-z]`	Rango	`[a-z]` → minúsculas
`[^abc]`	Negación	`[^0-9]` → no dígitos
`( )`	Grupo de captura	`(\d+)-(\w+)`
`^`	Inicio de línea	`^Hola`
`$`	Final de línea	`mundo$`
`|`	OR lógico	`gato|perro`
`\b`	Límite de palabra	`\bJava\b` no coincide con “JavaScript”

> 📝 **Nota:** `\w` NO incluye tildes ni `ñ` por defecto. Para "pérez" o "muñoz" necesitas `[a-zA-ZáéíóúüñÑ]` o la bandera `Pattern.UNICODE_CHARACTER_CLASS`. Cosa de examen, apuntátelo.

🕵️ Grupos de captura: los paréntesis que recuerdan

Los paréntesis `( )` no solo agrupan: **capturan** lo que coincide para poder extraerlo después. Es la base del punto 8, pero conviene verlo ya:

```
Pattern patron = Pattern.compile("(\\w+): (\\d+) años");
Matcher matcher = patron.matcher("Juan: 28 años, María: 32 años");

while (matcher.find()) {
    System.out.println(matcher.group(1) + " tiene " + matcher.group(2) + " años");
}
// Juan tiene 28 años
// María tiene 32 años
```

- `group(0)` (o `group()`) es toda la coincidencia.
- `group(1)`, `group(2)`… son los paréntesis en orden de apertura.
- Si agrupas sin querer capturar (por rendimiento): `(?:...)`.

🚩 Las banderas (flags)

`Pattern.compile` acepta un segundo argumento con banderas:

```
Pattern p1 = Pattern.compile("java", Pattern.CASE_INSENSITIVE);                 //
Java, JAVA, java...
Pattern p2 = Pattern.compile("^\\d+", Pattern.MULTILINE);                      // ^ al
inicio de cada línea
Pattern p3 = Pattern.compile(".*", Pattern.DOTALL);                            // el .
también captura saltos
Pattern p4 = Pattern.compile("java", Pattern.CASE_INSENSITIVE | Pattern.MULTILINE);  //
combinadas
Pattern p5 = Pattern.compile("(?i)java");                                      // flag
inline: (?i) = case-insensitive
```

`(?i)` dentro de la propia regex hace lo mismo que la bandera, sin segundo argumento. Las banderas se combinan con `|`.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** las regex se prueban con `matches()` (todo el string) o `find()` (subcadena). `group()` extrae lo capturado con paréntesis. Si tu regex no encuentra nada, revisa las contrabarras antes que la lógica.

Ejercicio: el cazador de patos

```
import java.util.regex.*;

public class CazadorPatos {
    public static void main(String[] args) {
        String texto = "Hay 3 patos, luego 45 patos, y al final 678 patos";
        Pattern patron = Pattern.compile("\\d+ patos");
        Matcher matcher = patron.matcher(texto);

        int total = 0;
        while (matcher.find()) {
            String grupo = matcher.group();                // "3 patos", "45 patos"...
            String numero = grupo.split(" ")[0];           // "3", "45"...
            total += Integer.parseInt(numero);
        }
        System.out.println("Patos totales: " + total);
    }
}
```

¿Qué imprime? ¿Y si cambias `"\\d+ patos"` por `"\\d+ patos"` con `matches()`?

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cómo escribes en Java la regex `\d`?
2. ¿Qué devuelve `matcher.find()`?

3. ¿Qué método de `Matcher` te da el texto que ha coincidido?
4. ¿Por qué conviene compilar el `Pattern` fuera de un bucle?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Una **regex** describe un conjunto de cadenas con símbolos (`\d`, `\w`, `+`, `{n}`, `[a-z]`…), y en Java las contrabarras se **duplican** (`"\\d"`).
2. **`Pattern.compile`** compila la regex (el molde) y **`matcher.find()`** busca coincidencias en un texto: `group()`, `start()` y `end()` te dan los detalles.
3. `matches()` exige que **todo** el string cumpla el patrón; `find()` busca subcadenas. No los confundas: es el error número 1.

> 🐛 **Vocabulario rápido**

Término	Idea general
Regex	Patrón de búsqueda que describe un conjunto de cadenas
`Pattern`	La regex compilada (el molde)
`Matcher`	El buscador aplicado a un texto concreto
`find()`	Busca la siguiente coincidencia
`group()`	El texto que ha coincidido
Grupo de captura	Paréntesis que “recuerdan” parte de la coincidencia
Metacaracter	Símbolo con significado especial en una regex (`.`, `+`, `\d`…)

📁 Volver al índice de la unidad · **Anterior:** 06 · Serialización: guardar objetos con ObjectOutputStream · **Siguiente:** 08 · Regex en acción: matches, replaceAll, split y validaciones

08 · Regex en acción: matches, replaceAll, split y validaciones

Del molde al mundo real: valida correos y DNIs, limpia textos con replaceAll, trocea con split y extrae datos con grupos de captura 🧰🛠️

> 🗺️ **Estás en:** 🗂️ **U14 · Ficheros y expresiones regulares (Regex)** → 08 · Regex en acción: matches, replaceAll, split y validaciones

---

📬 La idea en una frase

> **Aquí la regex deja de ser teoría: validas un correo con `matches()`, limpias espacios con `replaceAll()`, troceas una frase con `split()` y extraes los campos de un log con grupos de captura.**

El punto 7 te dio el martillo (`Pattern` y `Matcher`). Este punto es el taller: aplicaciones reales que usarás todos los días. Además de `Pattern`/`Matcher`, la clase `String` tiene métodos que aceptan regex directamente, y son los que más rápido te harán la vida.

🎯 String.matches(): ¿cumple TODO el string?

`matches()` devuelve `true` solo si **todo** el texto cumple el patrón. Es la herramienta perfecta para validar.

```
"123".matches("\\d+");          // true  (todo son dígitos)
"Hola".matches("\\d+");         // false (no son dígitos)
"12345678Z".matches("\\d{8}[A-Z]");  // true (8 dígitos + mayúscula)
"1234567Z".matches("\\d{8}[A-Z]");   // false (faltan dígitos)
```

> ⚠️ **Advertencia:** `matches()` empareja **todo** el string. Para buscar subcadenas dentro de un texto usa `find()` (punto 7). Este error se cobra su precio en cada examen.

🧹 replaceAll(): limpiar texto en una línea

`replaceAll(regex, reemplazo)` sustituye **todas** las coincidencias del patrón. `replaceFirst()` solo la primera.

```
String texto = "  Hola    mundo  de las   regex  ";

String limpio = texto.replaceAll("\\s+", " ").trim();
// "Hola mundo de las regex"

texto.replaceFirst("\\s+", " ").trim();
// "Hola    mundo  de las   regex"  (solo el primer bloque de espacios)
```

El patrón `"\\s+"` = “uno o más espacios” → lo sustituye por un solo espacio. Con `.trim()` limpias los extremos. Resultado: texto normalizado en dos llamadas.

> 💡 **Consejo:** los reemplazos también aceptan grupos: `"Hola Ana".replaceAll("Hola (\\w+)", "Adiós $1")` → `"Adiós Ana"`. El `$1` recupera el grupo 1 en el reemplazo.

🔪 split(): trocear por patrón

`split(regex)` divide el string por las coincidencias y devuelve un `String[]`.

```
"a,b,c,d".split(",");        // ["a", "b", "c", "d"]
"a,b,c,d".split(",", 3);     // ["a", "b", "c,d"]  (con límite)
"hola mundo  java".split("\\s+");  // ["hola", "mundo", "java"] (uno o más espacios)
"a1b22c333".split("\\d+");   // ["a", "b", "c", ""]
```

Fíjate en el último: si el string termina con dígitos, `split` deja una cadena vacía al final. Es un clásico que muerde. Y `split` con un punto (`"."`) no trocea por puntos, porque `.` es “cualquier carácter”: necesitas `"\\."`.

🎫 Validaciones de la vida real

Aquí juntamos todo: `matches()` + patrones compilados una vez como constantes. El validador clásico de correo, DNI y teléfono:

```java
import java.util.regex.Pattern;

public class ValidadorRegex {
    private static final Pattern PATRON_DNI = Pattern.compile("\\d{8}[A-Z]");
    private static final Pattern PATRON_EMAIL =
        Pattern.compile("[\\w.]+@[\\w.]+\\.[a-z]{2,}");
    private static final Pattern PATRON_TELEFONO =
        Pattern.compile("[679]\\d{8}");

    public static boolean esEmailValido(String email) {
        return PATRON_EMAIL.matcher(email.toLowerCase()).matches();
    }

    public static boolean esDNIValido(String dni) {
        return PATRON_DNI.matcher(dni).matches();
    }

    public static boolean esTelefonoValido(String telefono) {
        return PATRON_TELEFONO.matcher(telefono).matches();
    }

    public static void main(String[] args) {
        System.out.println(esEmailValido("user@example.com"));   // true
        System.out.println(esEmailValido("user@@example"));      // false
        System.out.println(esDNIValido("12345678Z"));            // true
        System.out.println(esDNIValido("12345678z"));            // false (minúscula)
        System.out.println(esTelefonoValido("612345678"));       // true
         System.out.println(esTelefonoValido("512345678"));        // false (empieza por
5)
    }
}
```

📝 **Nota:** `[\\w.]+@[\\w.]+\\.[a-z]{2,}` se lee: “letras/puntos, una @, letras/puntos, un punto, y al menos 2 letras”. Es un formato **básico**: no comprueba si el dominio existe ni si la dirección es real. Y el DNI: la regex solo verifica el **formato** (8 dígitos + letra); para validar la letra de verdad haría falta el algoritmo módulo 23. Las regex validan forma, no verdad.

🕵️ El procesador de logs (grupos de captura + ficheros)

La aplicación estrella: leer un log y extraer sus campos con grupos. Recuerda del punto 2 cómo leer archivos, y aquí le sumas la regex:

```
import java.io.BufferedReader;
import java.io.PrintWriter;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class ProcesadorLog {
    public static void main(String[] args) throws Exception {
        Pattern patron = Pattern.compile(
            "\\[(ERROR|INFO|WARN)\\]\\s+" +
            "(\\d{4}-\\d{2}-\\d{2})\\s+" +
            "(\\d{2}:\\d{2}:\\d{2})\\s+-\\s+(.*)");

        Path entrada = Paths.get("app.log");
        Path salida = Paths.get("errores.txt");

        try (BufferedReader br = Files.newBufferedReader(entrada);
             PrintWriter pw = new PrintWriter(Files.newBufferedWriter(salida))) {

            String linea;
            while ((linea = br.readLine()) != null) {
                Matcher matcher = patron.matcher(linea);
                if (matcher.find()) {
                    String nivel = matcher.group(1);
                    String fecha = matcher.group(2);
                    String hora = matcher.group(3);
                    String mensaje = matcher.group(4);
                        System.out.printf("[%s] %s a las %s: %s%n", nivel, fecha, hora,
mensaje);
                    if (nivel.equals("ERROR")) {
                        pw.println(fecha + " " + hora + " " + mensaje);
                    }
                }
            }
        }
        System.out.println("Errores extraídos a " + salida);
    }
}
```

Con una sola regex sacas el nivel, la fecha, la hora y el mensaje de cada línea. Y si la línea es un `ERROR`, la escribes en un archivo aparte. Regex + ficheros: la unidad entera en un programa.

> 💡 **Consejo:** para un log enorme no uses `Files.readAllLines` (punto 5): carga todo en memoria. Lee **línea a línea** con `BufferedReader` y aplica el `Matcher` a cada una. Memoria constante, da igual que el log pese 2 GB.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** cada paréntesis captura una parte. `(.*)` al final captura "todo lo que quede": es el clásico para el resto de la línea. Si una línea no coincide, `find()` devuelve `false` y no pasa nada: el bucle sigue.

Ejercicio: el formateador de fechas

```
public class FormateadorFechas {
    public static void main(String[] args) {
        String fecha = "15/03/2024";
        String convertida = fecha.replaceAll("(\\d{2})/(\\d{2})/(\\d{4})", "$3-$2-$1");
        System.out.println(convertida);
    }
}
```

¿Qué imprime? ¿Y qué pasa si la fecha viene con guiones `15-03-2024`?

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué método de `String` valida que TODO el texto cumpla un patrón?
2. ¿Cómo quitas los espacios dobles de un texto con una línea?
3. ¿Qué devuelve `"a,b,c".split(",")`?
4. En `replaceAll("(\\w+)@(\\w+)", "$2@$1")`, ¿qué hace `$2`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **`matches()`** valida que todo el texto cumpla el patrón (correos, DNIs, teléfonos); **`replaceAll`** limpia y reordena texto; **`split`** trocea por el patrón.
2. Los **grupos de captura** extraen partes (`group(n)` en Java, `$n` en los reemplazos) y son la base para procesar logs y CSV.
3. Para ficheros grandes, aplica la regex **línea a línea** con `BufferedReader` (no cargues el archivo entero en memoria).

> 🦎 **Vocabulario rápido**

Término	Idea general
`matches()`	¿Todo el string cumple el patrón?
`replaceAll()`	Sustituye todas las coincidencias
`replaceFirst()`	Sustituye solo la primera
`split()`	Trocea el string por el patrón
`$n`	Grupo `n` en un reemplazo de `replaceAll`
Validar	Comprobar el formato de un dato (no su veracidad)

📁 Volver al índice de la unidad · **Anterior:** 07 · Expresiones regulares: Pattern y Matcher · **Siguiente:** 09 · Repaso interactivo

09 · Repaso interactivo: domina ficheros y regex

El cierre práctico de la unidad, con un close() que va a doler, una regex que va a fallar y un objeto que va a viajar en el tiempo 😈

> 🗺️ **Estás en:** 📁 **U14 · Ficheros y expresiones regulares (Regex)** → 09 · Repaso interactivo

---

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```java
import java.io.*;
import java.util.regex.*;

public class Misterio {
    public static void main(String[] args) throws Exception {
        File f = new File("datos.txt");
        if (!f.exists()) {
            FileWriter w = new FileWriter(f);
            w.write("Ana;20\nBob;35\nCarla;28");
            w.close();
        }

        Pattern patron = Pattern.compile("(\\w+);(\\d+)");
        BufferedReader r = new BufferedReader(new FileReader(f));
        String linea;
        int suma = 0;
        while ((linea = r.readLine()) != null) {
            Matcher m = patron.matcher(linea);
            if (m.matches()) {
                suma += Integer.parseInt(m.group(2));
            }
        }
        r.close();
        System.out.println("Suma: " + suma);
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **`Suma: 83`** → La regex captura nombre y edad, y se suman 20 + 35 + 28. ✅
2. **`Suma: 0`** → `m.matches()` no encuentra nada porque el patrón no cuadra con las líneas. ❌
3. **`Suma: 83` pero solo la primera vez** → El archivo se crea la primera ejecución, pero la suma es la misma siempre. ❌

> › 🔄 Solución

🔥 Fireside Chat: File vs Path

> *Dos generaciones de la API de ficheros se encuentran frente a la máquina de café.*

File: — Llevo aquí desde Java 1.0. Toda una vida localizando archivos: `exists()`, `isFile()`, `length()`… Soy el clásico.

Path: — Y yo llegué en Java 7 con toda la modernidad: `Paths.get(...)`, y mi compañero `Files` hace `readAllLines`, `write` y `readString` en una línea.

File: — ¿Y para qué me necesitas a mí entonces? Vamos a ver, yo tengo `listFiles()` para recorrer carpetas.

Path: — Yo tengo `Files.walk()` que recorre **árboles enteros** recursivamente y te los da en un `Stream`. Con filtro, encima.

File: — Vale, pero sigues dependiendo de mí en algunos sitios… y de `FileReader`, `FileWriter` y compañía para leer y escribir.

Path: — Cierto, el trío `FileWriter/FileReader/BufferedReader` es eterno y funciona de maravilla. La familia de `java.io` no va a desaparecer. Pero para lo moderno, `Files` y `Paths` son mi liga.

> La lección: **`java.io` (File, FileReader, FileWriter, BufferedReader) sigue siendo válida y la usarás toda tu vida. `java.nio.file` (Paths, Files) es la forma moderna y preferida en código nuevo.** Saber las dos te hace flexible; saber solo una, limitado.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy el GPS que localiza archivos, pero no leo contenido.**
2. **Soy el lector que te da líneas enteras y nunca se atraganta.**
3. **Soy el bloque que cierra los archivos por ti, pase lo que pase.**
4. **Soy la interfaz-marca que permite guardar un objeto en un archivo.**
5. **Soy el molde compilado de una expresión regular.**
6. **Soy el método que exige que TODO el string cumpla el patrón.**

> › 🔄 **Respuestas**

😈 CONRAD VS EL MUNDO: “No cerré el archivo”

> *CONRAD, nuestro compilador cascarrabias, opina sobre los clásicos del novato.*

CONRAD: — ¡OTRA VEZ! Llega un alumno y me dice: *CONRAD, mi archivo se ha quedado a medias*. Y yo: vale, ¿a ver tu código? *Pues mira, escribo con FileWriter…* ¡Y NO CIERRAS! ¡Claro que se queda a medias! Los datos viven en el buffer hasta que llega el `close()`. ¿Te gusta escribir cartas y no cerrar el sobre? Pues eso. Desde Java 7 tienes `try-with-resources` para cerrar sin llorar. ¡ÚSALO!

Y luego está el de la regex. Escribe `"abc123".matches("\\d+")` y se sorprende de que dé `false`. ¡Pero si hay letras por el medio! `matches()` exige que TODO el string sean dígitos. Para buscar subcadenas, `find()`. ¡Lleva toda la unidad diciéndolo!

Y el de la serialización: guarda un objeto, cambia la clase añadiendo un campo, y se queja de que al leerle salta `InvalidClassException`. ¡Pues claro! Pusiste el `serialVersionUID` y se lo agradeciste con una mala cara… Pónselo fijo y deja de llorar.

La lección: el trío sagrado de la unidad: **¿cierro los recursos con `try-with-resources`? ¿uso `find()` para buscar y `matches()` solo para validar todo el string? ¿mi clase tiene `Serializable` y un `serialVersionUID` fijo?** El 90% de los sustos de esta unidad se arreglan con esas tres preguntas. El compilador no te odia: te está pasando las respuestas del examen.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. `new File("notas.txt")` … ¿qué hace en el disco?
   - a) Crea el archivo vacío b) Nada, solo representa la ruta
2. ¿Qué método de `String` devuelve `true` solo si TODO el texto es un correo válido?
   - a) `matches()` b) `find()`
3. ¿Qué lanza leer con `FileReader` un archivo que no existe?
   - a) `FileNotFoundException` b) `NullPointerException`
4. ¿Qué método de `Files` (NIO) lee un archivo completo como `List<String>`?
   - a) `Files.readAllLines()` b) `Files.readString()`
5. Para añadir al final de un archivo con `FileWriter` sin borrar lo anterior…
   - a) `new FileWriter("f.txt", true)` b) `new FileWriter("f.txt", false)`

> 🔄 **Soluciones**

⚡ Laboratorio de tortura: el validador que no valida

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Te dicen que tiene **2 errores de compilación y 1 error de lógica**… pero, ¿y si te lo dicen mal? Tu tarea: hacer que compile, que ejecute y que **toda** la salida sea correcta, contando tú los errores reales.

```
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class Validador {
    public static void main(String[] args) {
        String[] correos = {"ana@example.com", "bob@@example", "carla@site.es"};

        Pattern patron = Pattern.compile("[\\w.]+@[\\w.]+\\.[a-z]{2,}")

        int validos = 0;
        for (String c : correos) {
            Matcher m = patron.matcher(c);
            if (m.find()) {
                System.out.println(c + " → válido");
                validos++;
            } else {
                System.out.println(c + " → inválido");
            }
        }
        System.out.println("Válidos: " + validos)
    }
}
```

Fallo intencionado: el número de errores del enunciado está mal a propósito. La trampa es tu confianza en el enunciado.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Compila? *no → mira los ; y los imports.*

> **¿Y si sigo atascado?**

2. ¿Ya compila? *entonces mira el resultado.*

   > **¿Y si sigo atascado?**

3. ¿Crees que hay un error de lógica? *Vuelve a leer el código con calma y no te fíes del enunciado.*

   > **Solución final**

Los **errores de compilación**:

1. Falta el `;` al final de `Pattern.compile("[\\w.]+@[\\w.]+\\.[a-z]{2,}")`.
2. Falta el `;` al final del `System.out.println("Válidos: " + validos)`.

El **error de lógica**: no existe. El enunciado decía "2 errores de compilación y 1 de lógica", pero solo hay 2 faltas de `;` y ninguna lógica rota. Esa era la trampa: tu confianza en el enunciado.

Salida correcta:

```
ana@example.com → válido
bob@@example → inválido
carla@site.es → válido
Válidos: 2
```

Con `find()`, "bob@@example" NO pasa: el patrón `[\\w.]+@[\\w.]+\\.[a-z]{2,}` necesita letras, una `@`, más letras, un punto y al menos dos letras. "bob@@example" tiene dos `@` seguidas y ningún punto, así que no hay subcadena que cumpla el patrón.

La lección real: **lee el código, no el enunciado**. Cuenta, verifica y confía en el compilador, no en la leyenda.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Explorador**	Listar el contenido de una carpeta con `File.listFiles()` distinguiendo archivos y carpetas
🏅 **El Copión**	Copiar un archivo de texto línea a línea con `BufferedReader` + `PrintWriter` y `try-with-resources`
🏅 **El Sargento**	Formatear una tabla alineada en un archivo con `PrintWriter.printf`

Logro	Cómo conseguirlo
🏅 **El Maletero**	Guardar y recuperar una `ArrayList` de objetos con la serialización
🏅 **El Detective**	Extraer nivel, fecha y mensaje de un log con una regex y grupos de captura
🏅 **El Portero**	Validar correos, DNIs o teléfonos con `matches()` y patrones compilados

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```
import java.io.*;

public class Acertijo {
    public static void main(String[] args) throws Exception {
        try (FileWriter fw = new FileWriter("nums.txt")) {
            fw.write("5\n10\n15\n");
        }
        try (BufferedReader br = new BufferedReader(new FileReader("nums.txt"))) {
            int suma = 0;
            String linea;
            while ((linea = br.readLine()) != null) {
                suma += Integer.parseInt(linea);
            }
            System.out.println(suma);
        }
    }
}
```

2. **El limpiador:** tienes `String texto = "Hola mundo Java"` y quieres dejarlo en una sola línea de `String[]` con las tres palabras. ¿Qué escribes?
3. **El detective del log:** una línea de log es `[ERROR] 2026-09-06 10:30:45 - Conexión fallida: server=db01`. ¿Qué grupo captura `server=db01` con el patrón del punto 8? ¿Coincide con `matches()` o solo con `find()`?
4. **Verdadero o falso:** "`Files.readAllLines` es la mejor opción para un archivo de 2 GB".

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Interfaz-marca para guardar objetos en archivos (11 letras)
3. Método de String que valida que TODO el texto cumpla el patrón (7 letras)
5. Clase que envuelve FileReader para leer líneas enteras (15 letras)
6. API moderna de ficheros: Files y Paths (3 letras)
7. Lo que devuelve readLine() cuando se acaba el archivo (4 letras)

Vertical:
2. Bloque que cierra recursos automáticamente (16 letras)
4. Clase que representa una ruta sin leer contenido (4 letras)
```

> 📝 Soluciones

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, la diferencia entre `File`, `FileReader` y `FileWriter`."**
2. **"¿Qué es `try-with-resources` y por qué deberías usarlo siempre con ficheros?"**
3. **"¿Cuál es la diferencia entre `find()` y `matches()` en una regex?"**
4. **"¿Cómo guardarías y recuperarías una lista de objetos en un archivo?"**
5. **"¿Qué es el 'infierno de las contrabarras' en las regex de Java?"**
6. **"¿Cuándo usarías `BufferedReader` en vez de `Scanner`, y al revés?"**

🤷 No hay preguntas tontas

> ❓ **¿`File` y `Path` son lo mismo?**

Casi. Ambos representan rutas. `File` es de la API antigua (`java.io`) y `Path` de la moderna (`java.nio.file`). `Path` + `Files` es lo recomendado en código nuevo, pero `File` sigue funcionando y lo verás en proyectos antiguos. Saber los dos es ser un programador completo.

> ❓ **¿Puedo guardar cualquier objeto con la serialización?**

Solo si su clase (y todas sus clases de atributos) implementan `Serializable`. Si no, `NotSerializableException`. Y la serialización es de Java a Java: si otro programa (Python, JavaScript) tiene que leer tus datos, usa texto, CSV o JSON.

> ❓ **¿Se pueden validar documentos HTML con regex?**

No. HTML no es un lenguaje regular: tiene etiquetas anidadas que las regex no pueden rastrear. Para eso existen los *parsers*. Las regex son para texto plano con patrones, no para árboles de etiquetas. Esa pregunta se la hacen todos, y la respuesta es la misma: "no".

> ❓ **¿`\w` reconoce las tildes y la ñ?**

No por defecto. `\w` es `[a-zA-Z0-9_]`. Para "pérez" o "muñoz" usa `[a-zA-ZáéíóúüñÑ]` o la bandera `Pattern.UNICODE_CHARACTER_CLASS`.

🎬 Poscréditos

La programadora termina su gestor de contactos: guarda cada contacto con `Serializable`, los carga al arrancar con `ObjectInputStream`, valida el correo de cada uno con una regex compilada y escribe un informe bonito con `PrintWriter.printf`. Funciona. Hasta que intenta leer el archivo desde otra máquina y le sale `ClassNotFoundException`.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — ¿Has visto? La serialización guarda los datos, pero el archivo se la trae floja: solo sirve para el mismo programa con la misma clase. Cuando quieras que tus datos los entienda cualquier sistema, tendrás que irte de viaje: guardarlos en una base de datos.

Programadora: — ¿Una base de datos? ¿Y cómo conecto Java con una?

CONRAD: — *asiente* Con JDBC: una conexión, un `Statement`, y tus datos dejan de estar en un archivo para vivir en tablas con consultas. Tú suelta el `FileWriter` y ve preparando el SQL.

PROXIMAMENTE EN U15: Conexión a bases de datos con JDBC. El momento en que tus programas aprenden a hablar con un gestor de bases de datos relacional. 🗄️

📁 Volver al índice de la unidad · **Anterior:** 08 · Regex en acción: matches, replaceAll, split y validaciones · **Siguiente:** U15 · Persistencia de datos: JDBC e introducción a ORM

U15 — Persistencia de datos: JDBC e introducción a ORM

Java aprende a hablar con las bases de datos: JDBC, SQLite, Connection, Statement, PreparedStatement, el patrón DAO, transacciones y una primera toma de contacto con los ORM. Todo con humor y sin perder la conexión 🗄️

En la U14 cerraste el trato con los **ficheros**: guardar y leer información en el disco. Hoy subimos de nivel. Esa información deja de vivir en un archivo suelto y pasa a una **base de datos relacional**, con sus tablas, sus claves y sus reglas. Y Java, por supuesto, va a hablar con ella. La herramienta se llama **JDBC** (Java Database Connectivity) y es el puente oficial entre tu código y cualquier base de datos con controlador.

Esta unidad tiene cuatro grandes actos:

- **Conectar y consultar:** qué es JDBC, cómo se monta la dependencia de **SQLite** en Maven y el ritual de los 5 pasos para abrir una conexión (`Connection`), lanzar consultas (`Statement`) y leer los resultados (`ResultSet`).
- **El CRUD seguro:** insertar, leer, actualizar y borrar con **`PreparedStatement`**, la vacuna contra la **SQL injection** (pregúntale a Bobby Tables), y el **patrón DAO** para que el SQL no se cuele en tu lógica de negocio.
- **La integridad: transacciones** con `commit` y `rollback` para operaciones de todo-o-nada, y un decálogo de buenas prácticas para que tu conexión no sea una fuga de recursos.
- **El puente con los ORM:** después de domar el JDBC a mano, verás qué es un **ORM** (mapeo objeto-relacional) y por qué los equipos de verdad lo usan para no escribir tanto SQL de catálogo.

Por el camino conocerás a `SQLException`, la excepción checked que te va a acompañar en cada paso, y entenderás por qué `executeQuery()` y `executeUpdate()` son como la puerta de embarque y la del maletero: cada operación tiene la suya.

Esta unidad se lee como un **libro de 10 capítulos**: los 9 primeros puntos son teoría en progresión y el 10o es un aterrizaje práctico para machacar todo lo aprendido.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar **qué es JDBC** y por qué sirve para cualquier base de datos con controlador.
- Configurar la dependencia de **SQLite** en Maven y conectar tu programa con `DriverManager.getConnection()`.
- Ejecutar consultas con **`Statement`** y leer los resultados con **`ResultSet`** usando `next()` y los `getXxx("columna")`.
- Aplicar el **CRUD completo** (INSERT, SELECT, UPDATE, DELETE) con `executeQuery()` y `executeUpdate()`.
- Usar **`PreparedStatement`** con placeholders `?` y evitar la **SQL injection** sin excusas.
- Aplicar el **patrón DAO** (interfaz + implementación) para separar el SQL de la lógica de negocio.
- Gestionar **transacciones** con `commit()`, `rollback()` y savepoints para operaciones atómicas.
- Escribir código JDBC siguiendo el **decálogo de buenas prácticas**: `try-with-resources`, filas afectadas y `WHERE` siempre.
- Explicar **qué es un ORM** y en qué se diferencia de escribir JDBC a mano.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · Qué es JDBC	JDBC, SQLite y la dependencia Maven que lo pone todo en marcha	Todos
02 · Conexión a SQLite	Los 5 pasos del ritual y la interfaz `Connection`	Todos
03 · Statement y ResultSet	Ejecutar consultas y procesar filas con `next()` y `getXxx`	Todos
04 · CRUD completo	Insertar, leer, actualizar y borrar con `executeQuery` y `executeUpdate`	Todos
05 · PreparedStatement y SQL injection	Placeholders `?`, `setXxx` y la historia de Bobby Tables	Todos
06 · El patrón DAO	Interfaz + implementación: el SQL no pisa tu lógica de negocio	Todos

Punto	Qué aprenderás	Dificultad
07 · Transacciones	`commit`, `rollback` y savepoints: todo o nada	Todos
08 · Buenas prácticas	El decálogo del JDBC: recursos, filas afectadas y `WHERE` siempre	Todos
09 · Introducción a ORM	Qué es un ORM, mapeo objeto-relacional y JDBC vs ORM	Todos
10 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 9 primeros puntos son teoría en progresión. El 10o es el aterrizaje práctico: léelo justo después del 9o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA9)

RA9: Gestiona información almacenada en bases de datos relacionales manteniendo la integridad y la consistencia de los datos.

CE	Criterio	Dónde se cubre
RA9 a)	Se han identificado las características y métodos de acceso a sistemas gestores de bases de datos relacionales.	✅ Puntos 1 y 2
RA9 b)	Se han programado conexiones con bases de datos.	✅ Punto 2

CE	Criterio	Dónde se cubre
RA9 c)	Se ha escrito código para almacenar información en bases de datos.	✅ Puntos 4 y 5
RA9 d)	Se han creado programas para recuperar y mostrar información almacenada en bases de datos.	✅ Puntos 3 y 4
RA9 e)	Se han efectuado borrados y modificaciones sobre la información almacenada.	✅ Punto 4
RA9 f)	Se han creado aplicaciones que ejecuten consultas sobre bases de datos.	✅ Puntos 3, 5 y 10
RA9 g)	Se han creado aplicaciones para posibilitar la gestión de información presente en bases de datos relacionales.	✅ Puntos 6 y 10

> 📌 Esta unidad cubre la **RA9** completa. El camino hasta aquí: **U04** te enseñó las excepciones (y `SQLException` es checked, te va a perseguir), **U09/U10** te dieron las clases e interfaces con las que montar los POJOs y el DAO, y la **U14** te dejó el `try-with-resources` listo para cerrar conexiones. Todo el material anterior cobra sentido ahora: es hora de que tus objetos duerman en una base de datos.

🚪 ¿Por dónde empiezo?

- ¿Vienes de la U14 (ficheros) y quieres lo esencial? → Arranca en el punto 1 y el punto 2: conectar es el 90% de la batalla.
- ¿Ya conectas y quieres manejar datos? → Ve directo al punto 4 (el CRUD) y al punto 5 (hacerlo sin que te hackeen).
- ¿Solo vienes a por los trucos? → Salta al punto 5 (la SQL injection) y al punto 8 (el decálogo).
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · Qué es JDBC

⏭️ **Al acabar la unidad, continúa en U16 · Servir y consumir APIs con Web.**

01 · Qué es JDBC

JDBC, el USB de las bases de datos: Java habla con SQLite, MySQL o PostgreSQL gracias a un controlador. Y la dependencia Maven que lo enciende todo 🗄️🔌

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 01 · Qué es JDBC

📬 La idea en una frase

> **JDBC (Java Database Connectivity) es el traductor universal entre tu código Java y cualquier base de datos que tenga un controlador: da igual SQLite, MySQL o PostgreSQL, el código que escribes es casi el mismo.**

En la U14 aprendiste a guardar datos en ficheros: listas planas, texto plano, archivos sueltos. Pero un fichero no sabe responder a «dame todos los alumnos mayores de 18». Para eso están las **bases de datos relacionales**: tablas, filas, columnas y consultas. Y para que Java pueda hablar con ellas existe JDBC, un conjunto de interfaces del paquete `java.sql`. Bienvenido al punto donde tu programa empieza a tener memoria de largo plazo.

🔌 JDBC: el USB de las bases de datos

Piensa en JDBC como el puerto USB: da igual qué enchufes (SQLite, MySQL, PostgreSQL, Oracle…), la clavija es la misma. Cada base de datos aporta su **controlador** (en inglés, *driver*): un `.jar` que sabe traducir las llamadas genéricas de JDBC al dialecto de esa base de datos en concreto.

```
Tu programa Java  ──JDBC──▶  Controlador  ──▶  Base de datos
   (java.sql)      (genérico)   (el enchufe)     (SQLite, MySQL...)
```

La regla de oro: si cambias de base de datos y usaste SQL estándar, solo cambias el **controlador** y la **URL de conexión**. El resto del código ni se entera. Esa es la magia de JDBC.

> 💡 **Consejo:** el paquete `java.sql` contiene las interfaces clave: `Connection`, `Statement`, `PreparedStatement` y `ResultSet`. Este punto las presenta; los siguientes las desmontan una a una.

🗄️ SQLite: la base de datos de entrenamiento

Para aprender no necesitas montar un servidor ni abrir puertos. **SQLite** es una base de datos que vive en un **único archivo local**: sin servidor, sin usuario, sin contraseña. Si el archivo no existe, SQLite lo crea solo la primera vez que te conectas. Perfecta para practicar, y de paso es la que usan montones de aplicaciones reales.

> 📝 **Nota:** en el curso usamos SQLite porque no necesita instalación. Todo lo que aprendes aquí (CRUD, PreparedStatement, transacciones) funciona igual en MySQL o PostgreSQL cambiando el controlador y la URL.

📦 La dependencia Maven

Para usar el controlador de SQLite en un proyecto Maven, añades esta dependencia al `pom.xml`:

```
<dependency>
    <groupId>org.xerial</groupId>
    <artifactId>sqlite-jdbc</artifactId>
    <version>3.45.1.0</version>
</dependency>
```

Cuando Maven la descarga, el controlador queda en tu *classpath* y Java lo encuentra solo. Desde Java 6 no necesitas ni escribir `Class.forName(...)` para cargarlo: JDBC 4.0 se auto-descubre. Si ves `Class.forName()` en tutoriales antiguos, es que ese tutorial se escribió cuando se hacía a mano. No pasa nada si lo pones; tampoco pasa nada si no lo pones.

> ⚠️ **Advertencia:** si Maven no encuentra la clase `org.sqlite.JDBC` al ejecutar, el 99% de las veces es que la dependencia no está en el `pom.xml` o no se ha descargado. Primero revisa eso, y después tu código.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué significa JDBC y en qué paquete vive?
2. ¿Qué papel juega el controlador (driver)?
3. ¿Por qué SQLite es ideal para aprender?

4. ¿Qué tienes que cambiar si pasas de SQLite a MySQL?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. **JDBC** es el puente estándar de Java hacia cualquier base de datos con controlador: interfaces en `java.sql` que no cambian aunque cambie la base de datos.
2. Cada base de datos (SQLite, MySQL, PostgreSQL…) aporta su **controlador**, que traduce las llamadas genéricas a su dialecto.
3. Usamos **SQLite** porque es un único archivo sin servidor, y en Maven se añade con una dependencia de una línea.

> 🦎 **Vocabulario rápido**

Término	Idea general
JDBC	Java Database Connectivity: el puente Java ↔ base de datos
Controlador (driver)	El enchufe específico para cada base de datos
`java.sql`	El paquete con las interfaces de JDBC
SQLite	Base de datos en un único archivo, sin servidor
Dependencia Maven	La línea en el `pom.xml` que trae el controlador

📚 Volver al índice de la unidad · **Anterior:** — · **Siguiente:** 02 · Conexión a SQLite: los 5 pasos y Connection

02 · Conexión a SQLite: los 5 pasos y Connection

El ritual de los 5 pasos para conectar Java con SQLite y la interfaz Connection, la tubería por la que viajan tus consultas 🗄️🔗

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 02 · Conexión a SQLite

---

📬 La idea en una frase

> **Conectarse a SQLite es un ritual de 5 pasos: `DriverManager.getConnection("jdbc:sqlite:instituto.db")` te devuelve una `Connection`, y esa tubería es la que usará todo el resto del código.**

Si el punto 1 era conocer a JDBC de lejos, aquí te acercas a saludar. Abrir una conexión es lo primero que hace cualquier programa con base de datos, y también lo que más veces olvidamos cerrar. Vamos a montar el ritual completo y a entender qué es exactamente ese objeto `Connection` que vas a usar en cada ejercicio de la unidad.

🪜 Los 5 pasos (y el bonus no opcional)

Todo programa JDBC sigue esta escalera:

1. **Cargar el controlador** — desde Java 6, automático. Solo te acuerdas de que existe cuando lees código antiguo.
2. **Establecer la conexión** — `DriverManager.getConnection(url)` te devuelve una `Connection`.
3. **Crear un Statement** — `con.createStatement()` (o `con.prepareStatement(sql)`, lo verás en el punto 4).
4. **Ejecutar la consulta** — `executeQuery()` para SELECT, `executeUpdate()` para INSERT/UPDATE/DELETE.
5. **Procesar los resultados** — recorrer el `ResultSet` con `next()`.

Y el **bonus no opcional**: cerrar todo. En Java moderno, `try-with-resources` lo hace solo (lo viste en la U14 con los ficheros, aquí es exactamente la misma idea). Un recurso que no se cierra es

una fuga: en un servidor, cada conexión abierta consume memoria y hay un límite. Llega el siguiente `getConnection()` y… "Too many connections".

> 💡 **Consejo:** memoriza los 5 pasos en orden. El paso 4 y el 5 se repiten en cada consulta que escribas el resto de tu vida. Y el bonus de cerrar… que no se te olvide jamás.

🔗 Connection: la tubería

`Connection` es la interfaz que representa la conexión abierta con la base de datos. Es la tubería por la que viajan tus consultas. Para SQLite, que no tiene usuario ni contraseña, la URL es simplemente la ruta de un archivo:

```
String url = "jdbc:sqlite:instituto.db";
Connection con = DriverManager.getConnection(url);
```

> 📝 **Nota:** si el archivo `instituto.db` no existe, SQLite lo crea automáticamente. No tienes que montar la base de datos a mano antes de conectar: la primera vez que la uses, ya estará ahí.

El famoso protocolo "jdbc:sqlite:" es la URL estándar: `jdbc` es el protocolo, `sqlite` es el subprotocolo (qué base de datos) y el resto es el destino (para MySQL sería algo como `jdbc:mysql://localhost:3306/instituto`).

SQLException, la que te va a perseguir

`DriverManager.getConnection(...)` lanza **`SQLException`**, una excepción *checked*: el compilador te obliga a capturarla o declararla. Aprendiste las excepciones en la UO4; aquí vas a convivir con esta a diario. Lo correcto no es tragártela en silencio, sino mostrar qué ha pasado:

```
try {
    Connection con = DriverManager.getConnection(url);
    System.out.println("¡Conectado a SQLite!");
} catch (SQLException e) {
    System.err.println("Error de conexión: " + e.getMessage());
    System.err.println("Código de error: " + e.getErrorCode());
    System.err.println("Estado SQL: " + e.getSQLState());
}
```

> ⚠️ **Advertencia:** nunca hagas `catch (Exception e) {}` y te quedes tan pancho. Eso es tapar la luz del "check engine" con esparadrapo: el error sigue ahí, solo que ya nadie lo ve. Captura `SQLException`, y si quieres saber más, usa `getErrorCode()` y `getSQLState()`.

💊 La conexión completa con try-with-resources

El patrón que usarás en toda la unidad. Abre la conexión, haz lo que tengas que hacer y déjate de historias:

```
String url = "jdbc:sqlite:instituto.db";

try (Connection con = DriverManager.getConnection(url)) {
    System.out.println("Conectado. Base de datos lista.");

    // aquí irían las consultas (Statement, ResultSet...)

} catch (SQLException e) {
    System.err.println("Error BD: " + e.getMessage());
}
// la Connection se cierra sola al salir del try
```

`try-with-resources` cierra los recursos en **orden inverso** al de apertura. Si abres `Connection`, `Statement` y `ResultSet`, se cierran ResultSet → Statement → Connection. Solo y sin que tengas que pensar. Es mágico, y es la forma correcta.

> 💡 **Consejo:** no compartas una `Connection` entre hilos (lo verás más adelante): no es *thread-safe*. Si algún día necesitas varias conexiones a la vez, usa un pool de conexiones como HikariCP. Para este curso, una conexión por operación con `try-with-resources` es más que suficiente.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué devuelve `DriverManager.getConnection(url)`?
2. ¿Qué hace SQLite si el archivo de la base de datos no existe?
3. ¿Qué excepción lanza `getConnection()` y por qué no puedes ignorarla?

4. ¿En qué orden se cierran los recursos con `try-with-resources`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Conectarse a SQLite es un **ritual de 5 pasos** cuyo corazón es `DriverManager.getConnection("jdbc:sqlite:instituto.db")`.
2. `Connection` es la **tubería** por la que viajan tus consultas; si no la cierras, se queda abierta y agota los recursos del servidor.
3. `getConnection()` lanza **`SQLException`** (checked), y la forma correcta de trabajar es envolverlo todo en **`try-with-resources`** para que se cierre solo.

> 🦎 **Vocabulario rápido**

Término	Idea general
`DriverManager`	La clase que reparte conexiones (`getConnection`)
`Connection`	La tubería abierta hacia la base de datos
URL JDBC	`jdbc:sqlite:archivo.db` — protocolo, subprotocolo y destino
`SQLException`	La checked que te acompaña en todo JDBC
`try-with-resources`	Cierre automático de recursos, orden inverso

📚 Volver al índice de la unidad · **Anterior:** 01 · Qué es JDBC · **Siguiente:** 03 · Statement y ResultSet: consultar datos

03 · Statement y ResultSet: consultar datos

Lanza consultas con Statement, procesa filas con ResultSet y decide entre executeQuery y executeUpdate sin meter un tenedor en el microondas 🗄️📊

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 03 · Statement y ResultSet

---

📬 La idea en una frase

> **Con `Statement` envías la consulta y con `ResultSet` recibes las filas: `rs.next()` avanza a la siguiente fila (empezando antes de la primera) y `rs.getInt("id")` / `rs.getString("nombre")` leen cada columna.**

Ya tienes la tubería (`Connection`). Ahora toca lo divertido: **preguntarle** a la base de datos. El mensajero que lleva tu pregunta se llama `Statement`, y la respuesta llega en un `ResultSet`, que es como una tabla virtual con las filas que han coincidido. Vamos a ver cómo se lanza una consulta y cómo se procesa fila a fila.

📤 Statement: el mensajero

`Statement` es la forma más directa de ejecutar una consulta. La creas a partir de la conexión y le pasas el SQL como un String:

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");
```

`executeQuery()` es el método para **SELECT**: devuelve un `ResultSet` con las filas que han coincidido. Si la consulta no devuelve nada, no es un error: te da un `ResultSet` vacío, y `rs.next()` devolverá `false` directamente.

> ⚠️ **Advertencia:** no uses `executeQuery()` para un INSERT, UPDATE o DELETE. Eso lanza una `SQLException`. `executeQuery()` es solo para consultas que devuelven datos (SELECT); para las demás está `executeUpdate()`. Cada cosa en su sitio, como el tenedor y el microondas.

📊 ResultSet: la tabla virtual

`ResultSet` es la respuesta de la base de datos: un conjunto de filas. La clave es que el cursor empieza **antes de la primera fila**, y `next()` lo avanza a la siguiente. Cuando no quedan más filas, `next()` devuelve `false`.

```
while (rs.next()) {
    System.out.println(rs.getInt("id") + ": " + rs.getString("nombre"));
}
```

El patrón es siempre el mismo: `while (rs.next())` para recorrer todas las filas, y dentro, `rs.getXxx("nombre_columna")` para leer cada campo. El `getXxx` depende del tipo: `getInt`, `getString`, `getDouble`, `getBoolean`, `getDate`…

Tipo en SQLite	Método de lectura
INTEGER	`getInt("columna")`
TEXT	`getString("columna")`
REAL	`getDouble("columna")`
BOOLEAN (0/1)	`getBoolean("columna")`

> 💡 **Consejo:** también puedes usar el índice de columna en lugar del nombre: `rs.getString(1)` es la primera columna, `rs.getString(2)` la segunda… El nombre es más legible y no se rompe si reordenas las columnas. Usa nombres.

⚖️ executeQuery vs executeUpdate

Dos métodos, dos misiones. Esta tabla te salva en los exámenes:

Método	Para qué	Devuelve
`executeQuery()`	SELECT	`ResultSet` (las filas)
`executeUpdate()`	INSERT, UPDATE, DELETE	`int` (filas afectadas)

Regla mnemotécnica: si esperas **datos de vuelta** (filas con columnas), `executeQuery()`. Si solo quieres saber **cuántas filas se han modificado**, `executeUpdate()`. Mezclarlos da `SQLException`:

es como meter el tenedor en el microondas, no hay vuelta atrás.

🧩 El programa completo

Todo junto, con `try-with-resources` para que no se escape ningún recurso:

```
String url = "jdbc:sqlite:instituto.db";
String sql = "SELECT id, nombre FROM alumnos";

try (Connection con = DriverManager.getConnection(url);
     Statement stmt = con.createStatement();
     ResultSet rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        System.out.printf("%d: %s%n", rs.getInt("id"), rs.getString("nombre"));
    }

} catch (SQLException e) {
    System.err.println("Error BD: " + e.getMessage());
}
```

Fíjate en el orden de apertura: Connection → Statement → ResultSet. Con `try-with-resources` se cierran en orden inverso y no tienes que preocuparte. Si la consulta es fija y no tiene parámetros variables, un `Statement` sencillo es suficiente. Cuando haya datos del usuario de por medio… eso es cosa del punto 5.

> 💡 **Consejo:** `Statement` sirve para consultas literalmente fijas. Para el 90% de tus consultas reales (con parámetros que cambian) usarás `PreparedStatement`. Lo verás muy pronto, no te enamores del `Statement` todavía.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Dónde empieza el cursor de un `ResultSet` antes de llamar a `next()`?
2. ¿Qué método usarías para un `UPDATE` y qué devuelve?
3. ¿Qué pasa si `executeQuery()` no encuentra ninguna fila?

4. ¿Cómo lees un valor entero de una columna llamada `edad`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. `Statement` envía la consulta con `executeQuery()` (SELECT) y devuelve un `ResultSet`, una **tabla virtual** que se recorre con `while (rs.next())`.
2. Cada fila se lee con `rs.getXxx("columna")`, y el cursor empieza **antes** de la primera fila, de ahí el `next()` del bucle.
3. `executeUpdate()` (para INSERT/UPDATE/DELETE) devuelve las **filas afectadas**, y usar `executeQuery()` para esas operaciones lanza `SQLException`.

> 🦎 **Vocabulario rápido**

Término	Idea general
`Statement`	El mensajero que ejecuta una consulta fija
`ResultSet`	La tabla virtual con las filas devueltas
`next()`	Avanza a la siguiente fila (empieza antes de la primera)
`executeQuery()`	Para SELECT: devuelve `ResultSet`
`executeUpdate()`	Para INSERT/UPDATE/DELETE: devuelve filas afectadas

📚 Volver al índice de la unidad · **Anterior:** 02 · Conexión a SQLite · **Siguiente:** 04 · CRUD completo

04 · CRUD completo: insertar, leer, actualizar y borrar

El CRUD completo con JDBC: INSERT, SELECT, UPDATE y DELETE con PreparedStatement, el POJO que guarda cada fila y el WHERE que te salva la vida 🗄️🔄

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 04 · CRUD completo

---

📬 La idea en una frase

> **CRUD son las 4 operaciones básicas de cualquier base de datos — Create, Read, Update, Delete — y con JDBC se traducen en `INSERT`, `SELECT`, `UPDATE` y `DELETE` usando `executeUpdate()` para las que modifican y `executeQuery()` para las que consultan.**

Ya sabes conectar y consultar. Ahora toca la rutina de toda aplicación con base de datos: **insertar, leer, actualizar y borrar**. Es el pan de cada día. Aquí vamos a construir el ejemplo completo con una tabla `contactos`, un POJO (objeto Java, de los que conociste en la U09) y los cuatro métodos. Prepara la cafetera.

🧱 La tabla

Primero, la base de datos necesita una tabla. Este SQL se puede ejecutar una vez desde cualquier cliente SQLite (o con un `Statement` y `executeUpdate`, que también vale):

```
CREATE TABLE contactos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    telefono TEXT,
    email TEXT
);
```

> 📝 **Nota:** SQLite usa `INTEGER PRIMARY KEY AUTOINCREMENT` donde MySQL usa `INT AUTO_INCREMENT`. El `id` se asigna solo: tú no lo eliges, la base de datos lo genera.

🧍 El POJO

Cada fila de la tabla se convierte en un objeto Java. Es la clase de siempre, con atributos privados, getters y setters (la viste en la U09 y la U10):

```
public class Contacto {
    private int id;
    private String nombre;
    private String telefono;
    private String email;

    public Contacto() {}
    public Contacto(String nombre, String telefono, String email) {
        this.nombre = nombre;
        this.telefono = telefono;
        this.email = email;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public String getTelefono() { return telefono; }
    public void setTelefono(String telefono) { this.telefono = telefono; }
    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    @Override
    public String toString() {
        return id + " - " + nombre + " (" + telefono + ")";
    }
}
```

La estrategia de toda la unidad: la base de datos devuelve filas, y el código las convierte en objetos `Contacto`. Así tu programa trabaja con objetos de verdad, no con `ResultSet` sueltos.

✍️ Create (INSERT)

Insertar un contacto. Fíjate en los `?`: son placeholders que se rellenan con `setXxx`, empezando en 1. Más sobre ellos en el punto 5; de momento, fíate de ellos:

```java
public void insertarContacto(Contacto c) {
    String sql = "INSERT INTO contactos (nombre, telefono, email) VALUES (?, ?, ?)";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, c.getNombre());
        pstmt.setString(2, c.getTelefono());
        pstmt.setString(3, c.getEmail());
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.err.println("Error al insertar: " + e.getMessage());
    }
}
```

No le decimos a la base de datos qué `id` usar: lo genera ella sola. Si quisieras saber cuál ha sido, lo verás en el boletín avanzado con `RETURN_GENERATED_KEYS`.

📖 Read (SELECT)

Leer todos los contactos. La consulta devuelve filas y las convertimos en una `List<Contacto>` (colecciones, U12, haciendo de las suyas):

```
public List<Contacto> obtenerTodos() {
    List<Contacto> contactos = new ArrayList<>();
    String sql = "SELECT * FROM contactos ORDER BY nombre";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql);
         ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            Contacto c = new Contacto(
                rs.getString("nombre"),
                rs.getString("telefono"),
                rs.getString("email")
            );
            c.setId(rs.getInt("id"));
            contactos.add(c);
        }
    } catch (SQLException e) {
        System.err.println("Error al consultar: " + e.getMessage());
    }
    return contactos;
}
```

> 💡 **Consejo:** devuelve `List<Contacto>`, nunca un `ResultSet`. El `ResultSet` está atado a su conexión: si cierras la conexión, el `ResultSet` muere. Convierte a objetos Java y cierra todo tranquilo.

La búsqueda con LIKE

¿Y si queremos buscar por una parte del nombre? Ahí entra `LIKE` con comodines `%`:

```java
public List<Contacto> buscarPorNombre(String nombre) {
    List<Contacto> contactos = new ArrayList<>();
    String sql = "SELECT * FROM contactos WHERE nombre LIKE ?";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, "%" + nombre + "%");
        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Contacto c = new Contacto(
                    rs.getString("nombre"),
                    rs.getString("telefono"),
                    rs.getString("email")
                );
                c.setId(rs.getInt("id"));
                contactos.add(c);
            }
        }
    } catch (SQLException e) {
        System.err.println("Error al buscar: " + e.getMessage());
    }
    return contactos;
}
```

El `%` delante y detrás hace que coincida con cualquier contacto que contenga ese texto. Nota que los `%` van dentro del **valor**, no en el SQL: la concatenación aquí es segura.

📝 Update (UPDATE)

Actualizar un contacto. Aquí aparece el invitado estrella: el `WHERE`. Sin él, actualizas TODOS los contactos de la tabla:

```java
public boolean actualizarContacto(Contacto c) {
    String sql = "UPDATE contactos SET nombre = ?, telefono = ?, email = ? WHERE id =
?";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, c.getNombre());
        pstmt.setString(2, c.getTelefono());
        pstmt.setString(3, c.getEmail());
        pstmt.setInt(4, c.getId());
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error al actualizar: " + e.getMessage());
        return false;
    }
}
```

> ⚠️ **Advertencia:** sin `WHERE id = ?`, actualizas TODOS los contactos. 500 contactos llamados "John Doe" en un abrir y cerrar de ojos. **WHERE SIEMPRE** en UPDATE y DELETE. Es la primera norma del club de las bases de datos.

`executeUpdate()` devuelve el número de filas afectadas: si es `> 0`, se actualizó algo. Comprobarlo es gratis y te dice si la operación funcionó de verdad.

🗑️ Delete (DELETE)

Borrar un contacto por su id. La misma historia: `WHERE`, o arde la tabla:

```java
public boolean eliminarContacto(int id) {
    String sql = "DELETE FROM contactos WHERE id = ?";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setInt(1, id);
        return pstmt.executeUpdate() > 0;
    } catch (SQLException e) {
        System.err.println("Error al eliminar: " + e.getMessage());
        return false;
    }
}
```

> 💡 **Consejo:** confirma con el usuario antes de borrar. El `DELETE` no tiene Ctrl+Z: es para siempre. Como las decisiones en una película de terror, no se pueden deshacer.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué método usa el INSERT, `executeQuery()` o `executeUpdate()`?
2. ¿Qué pasa si ejecutas un `UPDATE` sin `WHERE`?
3. ¿Por qué conviene devolver `List<Contacto>` y no un `ResultSet`?
4. En `LIKE ?`, ¿dónde van los `%`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. El **CRUD** son las 4 operaciones básicas: `INSERT` (create), `SELECT` (read), `UPDATE` (update) y `DELETE` (delete).
2. Cada fila del `ResultSet` se convierte en un **objeto Java** (`Contacto`) y las consultas devuelven `List<Contacto>` para no depender de la conexión abierta.
3. **`WHERE` siempre** en `UPDATE` y `DELETE`, y comprueba las **filas afectadas** que devuelve `executeUpdate()` para saber si la operación tuvo efecto.

> 🐛 **Vocabulario rápido**

Término	Idea general
CRUD	Create, Read, Update, Delete: el abecedario de los datos
POJO	Objeto Java que representa una fila de la tabla
`LIKE ?`	Búsqueda por coincidencia parcial con `%` como comodín
`WHERE`	El filtro que te evita actualizar/borrar la tabla entera
Filas afectadas	El `int` que devuelve `executeUpdate()`

📚 Volver al índice de la unidad · **Anterior:** 03 · Statement y ResultSet · **Siguiente:** 05 · PreparedStatement y SQL injection

05 · PreparedStatement y SQL injection

Concatena strings en SQL y Bobby Tables borrará tu tabla de alumnos. La vacuna se llama PreparedStatement y sus placeholders ? 🗄️💉

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 05 · PreparedStatement y SQL injection

---

📬 La idea en una frase

> **Nunca construyas SQL concatenando strings con datos del usuario: es SQL injection. La vacuna es `PreparedStatement`, que separa el SQL de los datos con placeholders `?` y lo rellena de forma segura.**

En el punto 3 viste `Statement` y quedó claro que sirve para consultas fijas. Aquí llega el momento en el que te digo por qué, en cuanto hay **datos del usuario** de por medio, el `Statement` se convierte en una bomba de relojería. Y de paso conocerás a Bobby Tables, el niño más famoso de la historia de las bases de datos.

💣 Concatenar SQL: la bomba

Imagina una búsqueda de alumnos por nombre. La forma inocente (y fatal) de montarla:

```
// PELIGRO: esto es una bomba
String sql = "SELECT * FROM alumnos WHERE nombre = '" + inputUsuario + "'";
```

Ahora, un "usuario" escribe esto en el campo de búsqueda:

```
Luis'; DROP TABLE alumnos; --
```

La consulta que se ejecuta acaba siendo:

```
SELECT * FROM alumnos WHERE nombre = 'Luis'; DROP TABLE alumnos; --'
```

Fíjate: cierra la comilla con `'`, termina la consulta con `;`, **borra la tabla** con `DROP TABLE alumnos`, y comenta el resto con `--`. Adiós, tabla de alumnos. Esto es **SQL Injection** y ocurre de verdad, en

aplicaciones reales, cada día.

> ⚠️ **Advertencia:** NUNCA construyas SQL concatenando strings con datos del usuario. Es como dejar las llaves puestas en el coche con un cartel en la puerta que pone “PASE USTED”.

La historia de Bobby Tables

```
Madre: "He criado a mi hijo para que sea un programador cuidadoso,
        no un hacker que robe datos."
Hijo:  "Hola, me llamo Robert'); DROP TABLE Students; --"
```

No seas Bobby Tables. Usa PreparedStatement.

💉 PreparedStatement al rescate

La solución es separar el **código SQL** de los **datos**. El SQL se prepara con placeholders `?` y los valores se pasan después, por separado:

```
String sql = "SELECT * FROM alumnos WHERE nombre = ?";
PreparedStatement pstmt = conexion.prepareStatement(sql);
pstmt.setString(1, nombre);
ResultSet rs = pstmt.executeQuery();
```

El controlador escapa los datos automáticamente: si el usuario escribe `Robert'); DROP TABLE Students; --`, se trata como un **valor**, no como código. La tabla se queda donde está.

Los `?` son **placeholders posicionales** y empiezan en **1** (no en 0, como los arrays… trampa clásica de examen):

Tipo	Método
String	`setString(i, valor)`
int	`setInt(i, valor)`
double	`setDouble(i, valor)`
boolean	`setBoolean(i, valor)`

Tipo	Método
Date	`setDate(i, valor)` (usa `java.sql.Date`)
null	`setNull(i, Types.TIPO)`

> 💡 **Consejo:** para muchas consultas iguales, `PreparedStatement` puede ser **más rápido** que `Statement`, porque la base de datos compila la consulta una sola vez y reutiliza el plan. La seguridad te la da gratis, y de regalo viene la velocidad.

🧪 Guía de placeholders

Varios `?`, en orden. El primero es el `1`, el segundo el `2`, y así sucesivamente:

```
String sql = "SELECT * FROM alumnos WHERE curso = ? AND edad > ? ORDER BY nombre";
try (PreparedStatement pstmt = conexion.prepareStatement(sql)) {
    pstmt.setString(1, "DAM");
    pstmt.setInt(2, 18);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

¿Ves qué limpio? No hay concatenación, no hay comillas escapadas, no hay riesgo de inyección. Solo `?` limpios y ordenados. Los placeholders solo valen para **valores**, nunca para nombres de tabla o columna: `SELECT * FROM ?` no funciona, y no debería funcionar: un identificador no se escapa, se elige en el código.

⭐ Sé el Código, my friend...

> 🕶️ **Don Tip:** PreparedStatement previene la inyección SQL porque separa el código SQL de los datos. Nunca concatenes strings en SQL, ni en broma.

Ejercicio: detecta el intento de ataque

Un alumno te enseña este código y te pregunta si es seguro:

```
String nombre = sc.nextLine();
String sql = "SELECT * FROM alumnos WHERE nombre = '" + nombre + "'";
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

¿Qué responderías? ¿Es seguro? ¿Qué pasaría si el usuario escribiera `Ana'; DELETE FROM matriculas; --`? ¿Cómo lo arreglarías?

> 🔄 Solución

🥊 El ring: Statement vs PreparedStatement

Dos formas de ejecutar SQL discuten delante de la máquina de café.

Statement: — Yo soy el original. Sencillo, directo. `stmt.executeQuery("SELECT * FROM usuarios")`. Para consultas fijas soy perfecto.

PreparedStatement: — Sí, pero concatenar strings en SQL es una bomba de relojería. Inyección SQL, errores de sintaxis con comillas... Yo separo el SQL de los datos con `?` y soy seguro.

Statement: — Para una consulta fija, una sola vez, ¿para qué voy a preparar nada?

PreparedStatement: — Además, yo cacheo el plan de ejecución. Si ejecutas la misma consulta varias veces con distintos parámetros, soy más rápido. Y en Java, casi siempre ejecutas la misma consulta con distintos valores.

Statement: — Vale, pero yo valgo para DDL: CREATE TABLE, ALTER...

PreparedStatement: — Cierto. Para DDL usa Statement. Para DML (SELECT, INSERT, UPDATE, DELETE) usa PreparedStatement. ¿Trato?

Statement: — Trato.

> 👓 **Don Tip:** usa siempre `PreparedStatement` para consultas con datos de usuario. No es solo seguridad: es más rápido en consultas repetitivas y más legible.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿En qué número empiezan los índices de los `?` en un `PreparedStatement`?
2. ¿Qué hace la base de datos con `Robert'); DROP TABLE Students; --` si usas PreparedStatement?
3. ¿Puedes usar `?` para el nombre de una tabla?
4. ¿Por qué `PreparedStatement` puede ser más rápido en consultas repetidas?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Concatenar SQL con datos del usuario es **SQL injection**: una entrada como `'; DROP TABLE alumnos; --` puede borrar tablas enteras.
2. **`PreparedStatement`** separa el SQL de los datos con placeholders `?` y los `setXxx(i, valor)` (índices desde 1), escapando los valores automáticamente.
3. Es la forma **segura**, la más **rápida** para consultas repetidas y la única que no te hará pasar vergüenza en una entrevista de trabajo.

> 🐛 **Vocabulario rápido**

Término	Idea general
SQL injection	Inyectar instrucciones SQL a través de datos de usuario
Placeholder	El `?` que reserva el hueco para un valor
`setXxx(i, valor)`	Rellena el placeholder número `i` (desde 1)
Escapar	Neutralizar caracteres peligrosos de un texto
Bobby Tables	El niño que te enseña por qué no concatenar

📚 Volver al índice de la unidad · **Anterior:** 04 · CRUD completo · **Siguiente:** 06 · El patrón DAO

06 · El patrón DAO

El patrón DAO: una interfaz que promete y una implementación que cumple, para que el SQL no se cuele jamás en tu lógica de negocio 🗄️📐

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 06 · El patrón DAO

📬 La idea en una frase

> **El patrón DAO (Data Access Object) encapsula todo el SQL en una clase separada, detrás de una interfaz: tu programa habla con objetos y métodos (`insertarContacto`, `obtenerTodos`…), no con sentencias SQL sueltas.**

En el punto 4 escribimos los cuatro métodos del CRUD. Si los dejas sueltos por el programa, el SQL se pega a la lógica de negocio como chicle al zapato: cada vez que cambie la base de datos, hay que repasar el código entero buscando `getConnection`. El patrón **DAO** pone orden. Es el tema de las interfaces que viste en la U10, aplicado al acceso a datos.

🏛️ La idea: separar el SQL del negocio

¿Qué significa “separar la lógica de negocio del SQL”? Que tu programa haga esto:

```
ContactoDAO dao = new ContactoDAOImpl();
List<Contacto> contactos = dao.obtenerTodos();     // no hay ni un SELECT aquí
Contacto c = new Contacto("María", "666000111", "maria@ies.edu");
dao.insertarContacto(c);                           // tampoco
```

Y que **todo** el `jdbc:sqlite:`, el `PreparedStatement` y el `ResultSet` viva dentro de una sola clase. Si mañana cambias de SQLite a MySQL, solo tocas esa clase. El resto del programa no se entera de nada.

> 💡 **Consejo:** esto es programar contra una interfaz, que viste en la U10: declaras el tipo `ContactoDAO` y creas la implementación concreta `ContactoDAOImpl`. Si la implementación cambia, tu código no se mueve.

📜 La interfaz: el contrato

La interfaz declara **qué** se puede hacer, sin decir **cómo**:

```
public interface ContactoDAO {
    void insertarContacto(Contacto c);
    List<Contacto> obtenerTodos();
    List<Contacto> buscarPorNombre(String nombre);
    boolean actualizarContacto(Contacto c);
    boolean eliminarContacto(int id);
}
```

Cinco métodos, cinco operaciones. El que llama a la interfaz no sabe (ni le importa) si detrás hay SQLite, MySQL o un fichero de texto. El contrato es lo único que importa.

> 📝 **Nota:** los métodos son exactamente los que escribimos en el punto 4. El DAO los unifica bajo una misma interfaz: eso es todo el secreto.

🏭 La implementación: quien pone las manos en la masa

La implementación concreta contiene la URL, el SQL y el manejo de `SQLException`. Aquí es donde vive todo lo de los puntos anteriores:

```
public class ContactoDAOImpl implements ContactoDAO {
    private static final String URL = "jdbc:sqlite:contactos.db";

    @Override
    public void insertarContacto(Contacto c) {
          String sql = "INSERT INTO contactos (nombre, telefono, email) VALUES (?, ?,
?)";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql)) {
            pstmt.setString(1, c.getNombre());
            pstmt.setString(2, c.getTelefono());
            pstmt.setString(3, c.getEmail());
            pstmt.executeUpdate();
        } catch (SQLException e) {
            System.err.println("Error al insertar: " + e.getMessage());
        }
    }

    @Override
    public List<Contacto> obtenerTodos() {
        List<Contacto> contactos = new ArrayList<>();
        String sql = "SELECT * FROM contactos ORDER BY nombre";
        try (Connection con = DriverManager.getConnection(URL);
             PreparedStatement pstmt = con.prepareStatement(sql);
             ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                Contacto c = new Contacto(
                    rs.getString("nombre"),
                    rs.getString("telefono"),
                    rs.getString("email"));
                c.setId(rs.getInt("id"));
                contactos.add(c);
            }
        } catch (SQLException e) {
            System.err.println("Error al consultar: " + e.getMessage());
        }
        return contactos;
    }

    // buscarPorNombre, actualizarContacto y eliminarContacto siguen el mismo patrón
}
```

Fíjate en lo que **no** hay aquí fuera: ni un `SELECT` en el `main`, ni una URL de base de datos perdida entre la lógica. Todo el sucio trabajo del SQL está encerrado en su celda.

> 💡 **Consejo:** es habitual tener un DAO por entidad: `AlumnoDAO`, `ContactoDAO`, `LibroDAO`… Una tabla, un POJO y un DAO forman el trío clásico de las aplicaciones con base de datos.

🤔 DAO vs Repository

Dos términos que se parecen tanto que a veces se usan como sinónimos. La diferencia fina:

- **DAO** está pegado a la base de datos: habla de `INSERT`, `SELECT`, `UPDATE` y `DELETE`.
- **Repository** está más cerca del dominio: habla de `guardar`, `buscar`, `borrar`, sin mencionar SQL.

En proyectos pequeños se usan indistintamente. En los grandes, el Repository puede usar varios DAOs por debajo. Si alguien te lo pregunta en una entrevista, con esta respuesta sobrevives.

⭐ Sé el Código, my friend…

> 🕶️ **Don Tip:** un DAO encapsula el acceso a datos. Si cambias de SQLite a MySQL, solo cambia el DAO; la lógica de negocio ni se entera.

Ejercicio: la biblioteca

Eres la arquitecta o el arquitecto de una aplicación de gestión de biblioteca. Tienes estas entidades:

```
class Libro {
    String isbn;
    String titulo;
    String autor;
    boolean disponible;
}

class Prestamo {
    int id;
    String isbn;
    String socio;
    LocalDate fechaPrestamo;
    LocalDate fechaDevolucion;
}
```

Preguntas para pensar (responde antes de mirar):

1. ¿Cuántos DAOs crearías: uno por entidad (`LibroDAO`, `PrestamoDAO`) o uno solo genérico?
2. ¿Qué métodos tendría cada DAO?
3. ¿Dónde pondrías la regla de "un libro no se puede prestar si ya está prestado"?
4. ¿Usarías transacciones para gestionar préstamos y devoluciones?

> 🔄 **Solución orientativa**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué contiene la interfaz del DAO: el "qué" o el "cómo"?
2. Si cambias de SQLite a MySQL, ¿qué clase tocas y cuál no?
3. ¿Dónde vive el `jdbc:sqlite:...` en un programa bien hecho?
4. ¿DAO y Repository son exactamente lo mismo?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. El **patrón DAO** separa el acceso a datos de la lógica de negocio: una **interfaz** con las operaciones y una **implementación** con el SQL.
2. El resto del programa habla con métodos (`obtenerTodos()`, `insertarContacto(...)`) y nunca ve un `SELECT` ni una URL de conexión.
3. Si cambias de base de datos, solo cambia la **implementación**: esa es la magia de programar contra la interfaz.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | DAO | Data Access Object: el patrón que encierra el SQL |
> | Interfaz | El contrato: qué se puede hacer |
> | Implementación | La clase concreta: cómo se hace (SQL, URL…) |
> | Lógica de negocio | Las reglas del problema, sin SQL |
> | Repository | Primo del DAO, más cercano al dominio |

📚 Volver al índice de la unidad · **Anterior:** 05 · PreparedStatement y SQL injection · **Siguiente:** 07 · Transacciones

07 · Transacciones: commit, rollback y savepoints

Operaciones de todo o nada: setAutoCommit(false), commit para confirmar, rollback para deshacer y savepoints para no deshacerlo todo 🗄️🎭

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 07 · Transacciones

---

📬 La idea en una frase

> **Una transacción agrupa varias operaciones en un "todo o nada": `setAutoCommit(false)` abre el paréntesis, `commit()` lo cierra confirmando, y `rollback()` lo cierra deshaciendo todo.**

Hasta ahora cada `executeUpdate()` se guardaba al momento. Pero ¿qué pasa cuando una operación depende de otra? Piensa en una transferencia bancaria: quitar 50 € de una cuenta y ponerlos en otra. Si la segunda operación falla, te quedas con 50 € volando en el limbo. Eso no puede pasar. Aquí entran las transacciones.

🎭 El problema: el limbo del dinero

Imagina estas dos operaciones:

```
// Operación 1: quitar 50 € a Ana
stmt.executeUpdate("UPDATE cuentas SET saldo = saldo - 50 WHERE titular = 'Ana'");

// Operación 2: poner 50 € a Luis
stmt.executeUpdate("UPDATE cuentas SET saldo = saldo + 50 WHERE titular = 'Luis'");
```

Sin transacciones, cada `UPDATE` se confirma solo. Si la operación 2 lanza una `SQLException`, la 1 ya se ejecutó: Ana es más pobre y Luis no es más rico. El sistema queda **inconsistente**. Los datos ya no cuadran, y nadie sabe cómo arreglarlo.

> ⚠️ **Advertencia:** sin transacciones, si falla la segunda operación, la primera ya se ejecutó. El sistema queda inconsistente. Usa transacciones para operaciones **atómicas** (todo o nada): como un puente que o está entero o no está.

---

setAutoCommit(false): abrir el paréntesis

Por defecto, JDBC usa **auto-commit**: cada sentencia se confirma al momento. Para agrupar operaciones, lo desactivamos y decidimos nosotros cuándo se confirma:

```
public void transferir(int idOrigen, int idDestino) {
    String quitar = "UPDATE cuentas SET saldo = saldo - 50 WHERE id = ?";
    String poner  = "UPDATE cuentas SET saldo = saldo + 50 WHERE id = ?";

    try (Connection con = DriverManager.getConnection(URL)) {
        con.setAutoCommit(false);   // abrimos la transacción

        try (PreparedStatement q = con.prepareStatement(quitar);
             PreparedStatement p = con.prepareStatement(poner)) {

            q.setInt(1, idOrigen);
            q.executeUpdate();

            if (idOrigen == idDestino) {
                throw new SQLException("Destino inválido: misma cuenta");
            }

            p.setInt(1, idDestino);
            p.executeUpdate();

            con.commit();           // todo bien: confirmamos
            System.out.println("Transferencia OK");

        } catch (SQLException e) {
            con.rollback();         // algo falló: deshacemos todo
            System.err.println("Falló, todo deshecho: " + e.getMessage());
        }
    } catch (SQLException e) {
        System.err.println("Error de conexión: " + e.getMessage());
    }
}
```

El flujo es claro: **abrir** con `setAutoCommit(false)`, hacer todas las operaciones, y al final **`commit()`** si todo fue bien o **`rollback()`** si algo falló. El `rollback()` está en el `catch` interno: cualquier `SQLException` de las dos operaciones lo dispara y la base de datos vuelve a como estaba.

> 💡 **Consejo:** el `setAutoCommit(false)` va antes de las operaciones, sobre la conexión. El `commit()` y el `rollback()` también son métodos de la `Connection`. Todo gira alrededor de ese objeto.

📍 Savepoints: no deshacerlo todo

A veces una transacción tiene varias fases y no quieres deshacer todo si falla la última. Un **savepoint** es una marca intermedia: puedes hacer `rollback` hasta ese punto y conservar lo anterior.

```
con.setAutoCommit(false);

Savepoint sp = con.setSavepoint("despuesDeInsertar");
// ... más operaciones ...

if (algoMal) {
    con.rollback(sp);   // vuelve al savepoint, no deshace todo
} else {
    con.commit();       // todo bien
}
```

Es como en un videojuego: `setSavepoint` guarda la partida, `rollback(sp)` te devuelve a esa partida guardada en vez de empezar de cero, y `commit()` guarda el final definitivo. Útil cuando una transacción larga tiene una parte que sabes que siempre sale bien y otra que puede fallar.

> 📝 **Nota:** los savepoints se usan menos de lo que parece. El 90% de las veces te basta con `commit()`/`rollback()` en bloque. Pero saber que existen te ahorrará un susto el día que los necesites.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué método de `Connection` abre una transacción?
2. ¿Qué hace `commit()`? ¿Y `rollback()`?
3. ¿Dónde se suele poner el `rollback()` en un método típico?
4. ¿Qué te permite hacer un `Savepoint`?

✅ Resumen en 3 frases

1. Una **transacción** agrupa operaciones en un todo-o-nada: `setAutoCommit(false)` la abre, `commit()` la confirma y `rollback()` la deshace.
2. Sin transacciones, una operación que falla a medias deja los datos **inconsistentes** (como el dinero que desaparece del limbo).
3. Los **savepoints** marcan puntos intermedios para poder deshacer solo una parte de la transacción.

🦎 Vocabulario rápido

Término	Idea general
Transacción	Un grupo de operaciones que van juntas o no van
`setAutoCommit(false)`	Abre la transacción: tú controlas el momento
`commit()`	Confirmar todo: se guarda
`rollback()`	Deshacer todo: como si no hubiera pasado
`Savepoint`	Marca intermedia para deshacer solo hasta ahí
Atómico	Todo o nada, sin estados a medias

📚 Volver al índice de la unidad · **Anterior:** 06 · El patrón DAO · **Siguiente:** 08 · Buenas prácticas

08 · Buenas prácticas: el decálogo del JDBC

Diez mandamientos para no morir en el intento: PreparedStatement siempre, try-with-resources, WHERE siempre y no devolver ResultSet 🗄️📜

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 08 · Buenas prácticas

---

📬 La idea en una frase

> **Todo lo que has aprendido en esta unidad, resumido en 10 mandamientos: de "PreparedStatement siempre" a "confirma antes de borrar". Memorízalos y serás el alma de la oficina.**

Ya sabes conectar, consultar, hacer el CRUD, evitar la inyección, aplicar el DAO y gestionar transacciones. Este punto es el cierre de la teoría: la lista de cosas que los programadores veteranos aprendieron a base de sustos, y que tú puedes aprender en cinco minutos. Son los hábitos que separan a quien escribe JDBC de quien lo domestica.

📜 El decálogo del JDBC

1. **PreparedStatement siempre.** Nunca concatenes SQL. La SQL injection no es un mito: es Bobby Tables llamando a tu puerta.
2. **Try-with-resources.** `Connection`, `Statement` y `ResultSet` se cierran solos. Conexión abierta que no se cierra = servidor que se atraganta.
3. **No devuelvas `ResultSet`.** Devuelve listas de objetos (`List<Contacto>`). El `ResultSet` está atado a su conexión: cuando la cierras, muere con ella.
4. **Captura `SQLException` con mensaje descriptivo.** No atrapes `Exception` a lo loco y no te quedes en silencio. `e.getMessage()` es tu amigo.
5. **Usa transacciones para operaciones múltiples.** Todo o nada. Una transferencia no se hace a medias, y tus datos tampoco.
6. **No expongas credenciales en el código.** La URL y los datos de acceso van en un archivo de configuración (`config.properties`) o en variables de entorno, no en un `static final String` de la clase. Es como llevar la contraseña del banco escrita en la frente.

7. **Comprueba las filas afectadas.** `executeUpdate()` te dice si la operación funcionó (`> 0`). Ignorar ese valor es apostar a ciegas.
8. **No hagas consultas dentro de bucles.** Una consulta en un bucle que recorre mil elementos es mil viajes a la base de datos. Una sola consulta con `JOIN` basta. El bucle es para procesar resultados, no para hacer consultas.
9. **`WHERE` siempre en `UPDATE` y `DELETE`.** O pagas las consecuencias. Sin `WHERE`, actualizas o borras la tabla entera y no hay vuelta atrás.
10. **Confirma antes de borrar.** El usuario siempre se equivoca. Un `DELETE` no tiene Ctrl+Z.

> 💡 **Consejo:** los mandamientos 1, 2 y 9 son los que más vidas han salvado. Si memorizas solo tres, que sean esos. Los otros siete vienen de regalo.

🔮 El acertijo

Tienes dos tablas en SQLite: `usuarios(id, nombre, email)` y `pedidos(id, usuario_id, total)`.

Quieres obtener todos los usuarios que han hecho algún pedido con total superior a **100 €**. ¿Cuántas consultas SQL necesitas como mínimo si las haces "a lo tonto"? ¿Y si usas `JOIN`?

> › 🔄 **Solución**

> 🕶️ **Don Tip:** una consulta con `JOIN` es UNA sola llamada a la base de datos. Hacer una consulta por cada usuario en un bucle es el patrón N+1, el clásico error de rendimiento que cae en todas las entrevistas de trabajo.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué tres mandamientos son los más importantes según este punto?
2. ¿Por qué no conviene devolver un `ResultSet` desde un método?
3. ¿Dónde deben vivir las credenciales de la base de datos?
4. ¿Qué es el patrón N+1?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. El decálogo del JDBC son **10 hábitos** que te ahorran desastres: `PreparedStatement` siempre, `try-with-resources`, `WHERE` siempre y comprobar filas afectadas, entre otros.
2. Dos errores de rendimiento clásicos: **consultas en bucles** (patrón N+1, se arregla con `JOIN`) y **devolver `ResultSet`** (se arregla devolviendo listas de objetos).
3. La seguridad también es una buena práctica: las **credenciales nunca van en el código**, y los `DELETE` se **confirman** antes de ejecutarse.

> 🦎 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Decálogo | Las 10 reglas de oro del JDBC |
> | Patrón N+1 | N consultas en un bucle donde cabría una |
> | `JOIN` | Relacionar dos tablas en una sola consulta |
> | Filas afectadas | El `int` de `executeUpdate()` que confirma el cambio |
> | `config.properties` | Archivo donde van la URL y las credenciales |

📚 Volver al índice de la unidad · **Anterior:** 07 · Transacciones · **Siguiente:** 09 · Introducción a ORM

09 · Introducción a ORM

El traductor automático entre tus clases Java y las tablas de la base de datos: qué es un ORM, cómo mapea y por qué no te libra de aprender SQL 🗄️🤖

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 09 · Introducción a ORM

📬 La idea en una frase

> **Un ORM (Object-Relational Mapping) es una capa de código que convierte tus objetos Java en filas de tabla y tus consultas SQL en llamadas a métodos, para que tú te centres en el negocio y no en el catálogo.**

Has escrito JDBC a mano: `Connection`, `PreparedStatement`, `setXxx()`, `executeUpdate()`, cerrar todo en el orden correcto… Funciona, es rápido y aprendes muchísimo. Pero es **mucho** código repetitivo para lo que hace: mover datos de un sitio a otro. Los ORM nacieron exactamente para ahorrarte ese trabajo mecánico.

🔎 ¿Qué es un ORM, en concreto?

La base de datos piensa en **tablas, filas y columnas**. Tu programa piensa en **objetos, atributos y referencias**. Hablan dos idiomas distintos y alguien tiene que traducir. Ese traductor es el ORM:

Concepto de la base de datos	Equivalente en Java
Tabla	Clase (`@Entity`)
Fila	Objeto (instancia)
Columna	Atributo
Clave primaria (`id`)	Campo con `@Id`
Clave foránea	Referencia a otro objeto
`INSERT / UPDATE / DELETE`	`persist() / merge() / remove()`
`SELECT`	Método de búsqueda (`find`, JPQL, criterios)

En vez de montar el SQL y leer el `ResultSet`, el ORM **hace el viaje de ida y vuelta por ti**: al guardar un objeto crea la fila, y al leer una fila construye el objeto.

⚖️ JDBC vs ORM: la misma tarea, dos estilos

Imagina guardar un contacto nuevo.

A mano (JDBC):

```
String sql = "INSERT INTO contactos (nombre, email) VALUES (?, ?)";
try (Connection c = DriverManager.getConnection(url);
     PreparedStatement ps = c.prepareStatement(sql)) {
    ps.setString(1, "Ana");
    ps.setString(2, "ana@ejemplo.com");
    ps.executeUpdate();
}
```

Con un ORM (estilo JPA):

```
Contacto ana = new Contacto("Ana", "ana@ejemplo.com");
em.persist(ana);   // una línea: el ORM genera el INSERT por ti
```

El ORM construye el `INSERT`, le pasa los parámetros, gestiona la conexión y, cuando lees, rellena tu objeto campo a campo. Menos boilerplate, más lógica de negocio.

🧰 El arsenal: JPA, Hibernate y compañía

No todos los ORM son iguales, y conviene conocer los nombres que oirás en cualquier entrevista:

- **JPA** (Java Persistence API): la **especificación** de Java para persistencia. Son anotaciones (`@Entity`, `@Table`, `@Column`) y una API estándar, no un programa en sí.
- **Hibernate**: la **implementación** de JPA más famosa. Si JPA es el estándar de tráfico, Hibernate es el coche que más circula.
- **Sesión y `EntityManager`:** el objeto que gestiona tus entidades. Tus llamadas a `persist()` o `find()` pasan por él; él decide cuándo tocar la base de datos.

- **Carga perezosa (*lazy*) vs anticipada (*eager*):** el ORM puede traer los objetos relacionados solo cuando los necesitas (lazy) o de golpe (eager). Elegir mal aquí es el origen de no pocos agujeros de rendimiento.

> 💡 **Del curso a la industria:** nosotros hemos usado SQLite + JDBC porque es la forma más honesta de entender qué ocurre bajo el capó. En un proyecto JPA el capó sigue siendo el mismo: conexiones, SQL y transacciones… solo que el ORM lo conduce por ti.

🐘 El lado oscuro: lo que el ORM no te arregla

Un ORM no es magia, y saberte sus límites es lo que te separa de quien solo copió un tutorial:

1. **El SQL sigue siendo tuyo.** El ORM genera consultas sencillas, pero una consulta con tres `JOIN`s y un agregado la escribes tú (o la genera el ORM, pero tienes que leerla y entenderla).
2. **N+1 también, pero escondido.** Recorrer una lista y que cada objeto cargue sus hijos a escondidas = N+1 otra vez. El ORM lo amplifica si no vigilarlo.
3. **Capa de rendimiento.** A veces el ORM hace *dos* viajes donde tú habrías hecho uno. Medir sigue siendo obligatorio.
4. **Curva de aprendizaje.** Configurar proveedores, dialectos, mapeos… tiene su coste inicial.
5. **El desajuste objeto-relacional.** Clases con herencia y tablas planas no encajan siempre limpio: por eso el tema se llama *mapeo* y no *traducción perfecta*.

> ⚠️ **[WARNING]** Un ORM no sustituye a SQL, lo **esconde**. Si no entiendes lo que hay debajo (INSERT, transacciones, índices), cuando algo falle no sabrás ni por dónde empezar a mirar. Por eso lo vemos al final de la unidad: primero el motor, luego el GPS.

🧭 ¿Y por qué lo vemos ahora?

Porque el orden importa: cuando alguien diga «esta app usa Hibernate y no escribimos SQL», tú ya sabrás responder **qué está haciendo exactamente por debajo** (tu `persist()` de una línea es el `PreparedStatement` del capítulo 5, con traje y corbata). Y cuando el ORM te falle a las 3 de la mañana, tú serás quien abra el log y vea el SQL real.

Esa es la promesa de este punto: **no memorizar un framework, sino entender la idea** que hay detrás de todos ellos.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué traduce un ORM: de objetos a tablas o de tablas a objetos?
2. En JPA, ¿qué es `@Entity` y qué es Hibernate?
3. ¿Cuál es el riesgo clásico de rendimiento que un ORM puede esconder?
4. ¿Por qué no sustituye a SQL?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Un **ORM** mapea clases → tablas y objetos → filas, y traduce llamadas a métodos (`persist`, `find`) en SQL para ahorrarte el JDBC mecánico.
2. **JPA** es la especificación de Java (anotaciones y API) e **Hibernate** es su implementación más usada; trabajan con un `EntityManager` y entidades anotadas.
3. No sustituye a SQL: **esconde** el trabajo repetitivo, pero las consultas complejas, el rendimiento (N+1) y la depuración siguen siendo cosa tuya.

> 🦎 **Vocabulario rápido**

Término	Idea general
ORM	Capa que traduce objetos ↔ tablas
JPA	Especificación de persistencia de Java (anotaciones + API)
Hibernate	Implementación de JPA más extendida
Entidad	Clase mapeada a una tabla (`@Entity`)
`EntityManager`	El “mandamás” que ejecuta persistencia
Lazy / eager	Cargar datos tarde (cuando toca) o de golpe

📚 Volver al índice de la unidad · **Anterior:** 08 · Buenas prácticas · **Siguiente:** 10 · Repaso interactivo

10 · Repaso interactivo: domina el JDBC

El cierre práctico de la unidad, con una inyección SQL que va a doler, un UPDATE sin WHERE que va a arder y una conexión que nadie cerró 😈🗄️

> 🗺️ **Estás en:** 🗄️ **U15 · Persistencia de datos: JDBC e introducción a ORM** → 10 · Repaso interactivo

---

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar.** 😈

⭐ Sé el Código, my friend...

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```
import java.sql.*;

public class Misterio {
    public static void main(String[] args) {
        String url = "jdbc:sqlite:instituto.db";

        try (Connection con = DriverManager.getConnection(url);
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery("SELECT COUNT(*) FROM alumnos")) {

            if (rs.next()) {
                System.out.println("Total: " + rs.getInt(1));
            }

        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **`Total: 0`** → `COUNT(*)` siempre devuelve 0 si hay algún alumno. ❌
2. **`Total: <número de alumnos>`** → `COUNT(*)` devuelve una sola fila con una columna, y `rs.getInt(1)` la lee por índice. ✅
3. **Una excepción** → `getInt(1)` no existe porque las columnas empiezan en 0. ❌

> 🔄 **Solución**

🔥 Fireside Chat: Statement vs PreparedStatement

> *Dos formas de ejecutar SQL se encuentran frente a la máquina de café.*

Statement: — Yo soy el de toda la vida. `stmt.executeQuery("SELECT * FROM usuarios")`. Directo, sin vueltas. Para una consulta fija soy perfecto.

PreparedStatement: — Sí, pero cuando llega un dato del usuario y concatenas, se te enciende el piloto de la inyección. Yo separo el SQL de los datos con `?` y los `setXxx`. Seguridad de serie.

Statement: — ¿Y si solo ejecuto una consulta una vez? ¿Para qué voy a preparar nada?

PreparedStatement: — Además yo cacheo el plan de ejecución. Si ejecutas la misma consulta con distintos parámetros, soy más rápido. Y en Java casi siempre haces eso: la misma consulta, mil valores distintos.

Statement: — Vale… pero yo sirvo para el DDL: `CREATE TABLE`, `ALTER`…

PreparedStatement: — Cierto. Para DDL, Statement. Para DML (SELECT, INSERT, UPDATE, DELETE), yo. ¿Trato?

Statement: — Trato.

> La lección: **DDL → Statement. DML → PreparedStatement.** Y jamás concatenes datos de usuario en un SQL, digas lo que digas.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy el traductor universal entre Java y cualquier base de datos con controlador.**

2. **Soy la tubería que `DriverManager.getConnection()` te devuelve y que cierras con `try-with-resources`.**
3. **Soy la tabla virtual que se recorre con `next()` y se lee con `getXxx("columna")`.**
4. **Soy el mensajero seguro que separa el SQL de los datos con `?`.**
5. **Soy el patrón que encapsula todo el SQL detrás de una interfaz.**
6. **Soy la operación que hace que varias sentencias se confirmen juntas… o ninguna.**

> 🔁 **Respuestas**

👹 CONRAD VS EL MUNDO: "El JDBC que no se cierra"

> *CONRAD, nuestro compilador cascarrabias, opina sobre los clásicos del novato.*

CONRAD: — ¡OTRA VEZ! Llega un alumno y me dice: *CONRAD, mi programa funciona pero a la segunda vez se queja de "Too many connections"*. ¡AY, MADRE MÍA! Es que está abriendo `Connection` a lo loco y **no la cierra nunca**. Cada `getConnection()` sin su `try-with-resources` es una tubería goteando. El servidor tiene un límite de conexiones, y tú las estás agotando todas. ¡`try-with-resources`! ¿Tanto cuesta?

Y luego está el del `UPDATE` sin `WHERE`. Me enseña un código que actualiza "un" contacto y resulta que **los ha puesto a todos** con el mismo teléfono. ¿Y el `WHERE id = ?`? ¡Se le olvidó! En `UPDATE` y `DELETE`, si no hay `WHERE`, es que querías arrasar la tabla entera.

Y el rey del drama: usa `executeQuery()` para un `INSERT` y se lleva una `SQLException`. ¡Pues claro! `executeQuery()` es para SELECT; para INSERT/UPDATE/DELETE está `executeUpdate()`. Cada cosa en su sitio. Como el tenedor y el microondas, que te lo dije en el punto 3.

Y el último, el del silencio: captura la excepción y… **no hace nada**. `catch (Exception e) {}` vacío. La luz del "check engine" encendida y él tan feliz. ¡Escribe `e.getMessage()`! Es gratis y te dice exactamente qué ha pasado.

La lección: antes de acusar a la base de datos de "odiarte", repasa el trío sagrado de la unidad: **¿he cerrado la conexión? ¿hay `WHERE` en mi UPDATE/DELETE? ¿estoy usando el método de ejecución correcto?** El 90% de los errores de esta unidad se arreglan con esas tres preguntas. La base de datos no te odia: te está pasando las respuestas del examen.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. Quieres buscar alumnos por un nombre que teclea el usuario. ¿Qué usas?
   - a) `Statement` con concatenación b) `PreparedStatement` con `?`
2. `executeUpdate()` en un `UPDATE` que no encuentra filas… ¿qué devuelve?
   - a) `0` b) una `SQLException`
3. ¿Qué método abre una transacción?
   - a) `con.setAutoCommit(false)` b) `con.beginTransaction()`
4. `rs.next()` en un `ResultSet` vacío devuelve…
   - a) `false` b) `null`
5. Para el DDL (`CREATE TABLE`), ¿qué se usa con más sentido?
   - a) `Statement` b) `PreparedStatement`

> 🔄 **Soluciones**

⚡ Laboratorio de tortura: el CRUD que sangra

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE, el driver de SQLite y un archivo nuevo

El escenario: te pasan este gestor de contactos con SQLite. Te dicen que tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el resultado sea incorrecto… pero, ¿y si te lo dicen mal? Tu tarea: hacer que compile, que ejecute y que **toda** la salida sea correcta, contando tú los errores reales.

```
import java.sql.*;

public class GestorContactos {
    private static final String URL = "jdbc:sqlite:contactos.db";

    public static void main(String[] args) {
        crearTabla();
        insertar("Ana", "666111222");
        insertar("Luis", "666333444");
        listar();
    }

    static void crearTabla() {
          String sql = "CREATE TABLE IF NOT EXISTS contactos (id INTEGER PRIMARY KEY
AUTOINCREMENT, nombre TEXT, telefono TEXT)";
        try (Connection con = DriverManager.getConnection(URL);
             Statement stmt = con.createStatement()) {
            stmt.executeUpdate(sql);
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    static void insertar(String nombre, String telefono) {
         String sql = "INSERT INTO contactos (nombre, telefono) VALUES ('" + nombre +
"', '" + telefono + "')";
        try (Connection con = DriverManager.getConnection(URL);
             Statement stmt = con.createStatement()) {
            stmt.executeUpdate(sql);
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }

    static void listar() {
        String sql = "SELECT * FROM contactos"
        try (Connection con = DriverManager.getConnection(URL);
             Statement stmt = con.createStatement();
             ResultSet rs = stmt.executeQuery(sql)) {
            while (rs.next()) {
                System.out.println(rs.getInt("id") + " - " + rs.getString("nombre"));
            }
```

```
        } catch (SQLException e) {
            System.err.println("Error: " + e.getMessage());
        }
    }
}
```

Fallo intencionado: el número de errores del enunciado está mal a propósito: si buscas el "tercer error de compilación" y el "error de lógica", te vuelves loco. La trampa es tu confianza en el enunciado.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Compila? *no → busca puntos y comas.*
   > **¿Y si sigo atascado?**
2. ¿Compila ya? *no → busca más cosas raras.*
   > **¿Y si sigo atascado?**
3. ¿Crees que hay un error de lógica que cambia la salida? *Vuelve a leer con calma y no te fíes del enunciado.*
   > **Solución final**

Errores de compilación:

1. Falta el `;` al final de `String sql = "SELECT * FROM contactos"` en `listar()`.

El "error de lógica": no existe como tal. La lógica de crear tabla, insertar y listar es correcta y la salida sería `1 - Ana` y `2 - Luis`. Ese era el fallo intencionado: el enunciado dice "3 errores de compilación y 1 de lógica", pero solo hay una falta de `;`. El error "oculto" era tu confianza en el enunciado.

El problema real que no se ve al ejecutar: `insertar()` concatena el SQL con los valores, lo que es **SQL injection** en estado puro (punto 5). El programa "funciona" porque los datos son de confianza, pero si `nombre` viniera del usuario, un `Ana'; DROP TABLE contactos; --` se cargaría la tabla. La corrección profesional es `PreparedStatement`:

```
static void insertar(String nombre, String telefono) {
    String sql = "INSERT INTO contactos (nombre, telefono) VALUES (?, ?)";
    try (Connection con = DriverManager.getConnection(URL);
         PreparedStatement pstmt = con.prepareStatement(sql)) {
        pstmt.setString(1, nombre);
        pstmt.setString(2, telefono);
        pstmt.executeUpdate();
    } catch (SQLException e) {
        System.err.println("Error: " + e.getMessage());
    }
}
```

Salida correcta con el código arreglado:

```
1 - Ana
2 - Luis
```

La lección real: **lee el código, no el enunciado.** Un bug puede ser que te digan “hay 3 errores” y solo haya 1: si buscas los otros dos, te vuelves loco. Y el error más peligroso es el que no da error: la SQL injection se ve con ojos de arquitecto, no con el compilador. Cuenta, verifica y confía en lo que ves, no en la leyenda.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Conectador**	Abrir y cerrar una `Connection` con `try-with-resources` sin que se escape ni una
🏅 **El Mensajero Seguro**	Usar `PreparedStatement` para una consulta con datos de usuario, sin concatenar nada
🏅 **El Cazador de Tablas**	Detectar SQL injection en código ajeno y explicar por qué es peligrosa
🏅 **El Arquitecto DAO**	Encapsular el CRUD en una interfaz + implementación y usarlo desde un `main` limpio

Logro	Cómo conseguirlo
🏅 **El Banquero**	Montar una transferencia atómica con `commit()` y `rollback()` que nunca deja los datos a medias

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa si la tabla `alumnos` tiene 3 filas?

```
String sql = "SELECT nombre FROM alumnos WHERE edad > ?";
try (Connection con = DriverManager.getConnection(url);
     PreparedStatement pstmt = con.prepareStatement(sql)) {
    pstmt.setInt(1, 18);
    try (ResultSet rs = pstmt.executeQuery()) {
        while (rs.next()) {
            System.out.print(rs.getString("nombre") + " ");
        }
    }
}
```

2. **El detective del ResultSet:** un método devuelve un `ResultSet` al `main` y luego cierra la conexión. ¿Qué pasa cuando el `main` intenta recorrerlo?
3. **El consultor de precios:** tienes una tabla `productos(precio)` y quieres actualizar un precio concreto. ¿Qué dos cosas son imprescindibles en el `UPDATE`?
4. **Verdadero o falso:** "un `PreparedStatement` permite usar `?` tanto para valores como para el nombre de la tabla".

> 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Método que avanza a la siguiente fila de un ResultSet (4 letras)
3. Excepción checked que acompaña a todo JDBC (13 letras)
5. Método que confirma una transacción (6 letras)
6. Patrón que encapsula el acceso a datos (3 letras)

Vertical:
2. El puente entre Java y la base de datos (4 letras)
4. Objeto Java que representa una fila de la tabla (4 letras)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **“Explícame, como si yo fuera tu abuela, qué es JDBC.”**
2. **“¿Cuál es la diferencia entre `Statement` y `PreparedStatement`?”**
3. **“¿Qué es la SQL injection y cómo la evitas?”**
4. **“¿Qué pasa si no cierras una `Connection`?”**
5. **“¿Qué es el patrón DAO y por qué se usa?”**
6. **“¿Qué son `commit` y `rollback` y cuándo los usas?”**

🤷 No hay preguntas tontas

> ❓ **¿Es necesario `Class.forName()` para cargar el controlador?**

No. Desde Java 6, los controladores JDBC 4.0 se auto-descubren si están en el classpath. Lo verás en tutoriales antiguos, y no pasa nada si lo pones: solo es código que ya no hace falta.

> ❓ **¿`Statement` se usa todavía?**

Está zombi. Si ves un `Statement` en código real (que no sea una consulta literal o DDL), es bandera roja. Para cualquier consulta con datos variables: `PreparedStatement`, siempre.

> ❓ **¿Puedo compartir una `Connection` entre varios hilos?**

Técnicamente sí, prácticamente no. `Connection` no es *thread-safe*. En aplicaciones con hilos se usa un pool de conexiones (HikariCP) y cada hilo pide la suya.

> ❓ **¿DAO y Repository son lo mismo?**

Casi. El DAO está pegado a la base de datos (INSERT, SELECT). El Repository es más de dominio (guardar, buscar). En proyectos pequeños se usan como sinónimos; en los grandes conviven.

🎬 Poscréditos

La programadora termina su gestor de biblioteca: un `LibroDAO` con su `PreparedStatement` en cada método, una transacción con `commit` y `rollback` para los préstamos y un `config.properties` guardando la URL. Todo funciona. Hasta que la directora le pide que los préstamos se consulten desde el móvil.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — ¿Has visto? Tienes los datos en SQLite y solo los lees desde Java en la misma máquina. Para consultarlos desde un móvil necesitas que tu programa **exponga** esa información por la red: una API. Alguien le pide datos a tu servidor con una petición HTTP, y tu servidor responde con JSON.

Programadora: — ¿O sea que en vez de conectarme yo a la base de datos desde cada cliente, el cliente me habla a mí por la red?

CONRAD: — *asiente* Exacto. Tu programa Java se convierte en el intermediario: recibe peticiones, consulta la base de datos con el JDBC que acabas de dominar, y devuelve los datos. Primero serviste datos desde la base; ahora los servirás por la web.

PROXIMAMENTE EN U16: Servir y consumir APIs con Web. El momento en que tu programa Java se pone un uniforme de camarero: recibe peticiones HTTP y sirve datos en JSON. 🌐

📚 Volver al índice de la unidad · **Anterior:** 09 · Introducción a ORM · **Siguiente:** U16 · Servir y consumir APIs con Web

U16 — Servir y consumir APIs con Web

Del terminal a Internet: HttpServer, HTML, formularios, JSON y HttpClient para servir y consumir APIs como una de verdad 🌐

Hasta ahora tus programas vivían en la terminal, en ficheros y en bases de datos. Hoy se acabó: es hora de que salgan a Internet. Sin JavaScript frameworks, sin servidores de aplicaciones y sin instalaciones eternas: solo Java y HTTP.

Esta unidad tiene tres grandes actos:

- **Ser servidor:** con `HttpServer`, el servidor HTTP que Java trae de serie, servirás texto, HTML, formularios y JSON sin instalar Tomcat ni Spring.
- **Ser API:** montarás un mini proyecto REST completo —un gestor de tareas— con GET, POST, PUT y DELETE.
- **Ser cliente:** con `java.net.http.HttpClient`, consumirás APIs externas reales (GitHub, jsonplaceholder), parsearás JSON con Gson y manejarás cabeceras, timeouts y códigos de estado.

Por el camino entenderás el modelo petición-respuesta de HTTP, por qué `"POST".equals(...)` es mejor que `...equals("POST")`, qué significa de verdad un 404 y por qué el `fetch` de JavaScript y el `HttpClient` de Java son dos caras de la misma moneda.

Esta unidad se lee como un **libro de 9 capítulos**: los 8 primeros puntos son teoría en progresión y el 9o es un aterrizaje práctico para machacar todo lo aprendido.

> 💉 **Material de ampliación (extra, no evaluable).** Esta unidad va más allá del temario oficial: úsala para ampliar, no para sudar la gota gorda. Los boletines son recomendados, no obligatorios.

🎯 Objetivo de la unidad

Al terminar, serás capaz de:

- Explicar el modelo **petición-respuesta** de HTTP y sus códigos de estado más comunes (200, 404, 500…).
- Montar un **servidor web mínimo** con `com.sun.net.httpserver.HttpServer` en menos de 20 líneas.

- Servir **HTML** desde Java y leerlo también desde un fichero.
- Procesar **parámetros GET** y **formularios POST** sin miedo.
- Devolver **JSON** con el `Content-Type` adecuado y consumirlo con `fetch`.
- Construir una **API REST** completa (GET, POST, PUT, DELETE) con datos en memoria.
- Consumir APIs externas con **`HttpClient`**: GET, POST, cabeceras, timeouts y llamadas asíncronas.
- **Parsear JSON** con Gson, mapearlo a `record` y manejar errores HTTP con elegancia.

🗺️ Mapa de la unidad

Punto	Qué aprenderás	Dificultad
01 · El protocolo HTTP en 30 segundos	Petición-respuesta, métodos, códigos de estado y URLs	Todos
02 · Servidor web mínimo con HttpServer	Tu primer servidor Java: `HttpServer`, `createContext` y `HttpExchange`	Todos
03 · Sirviendo HTML	Páginas completas con `Content-Type: text/html` y desde fichero	Todos
04 · Parámetros GET y formularios POST	Leer la query string y el cuerpo de la petición	Todos
05 · Devolviendo JSON: como una API de verdad	`application/json`, `fetch` desde el frontend y por qué front y back se separan	Todos
06 · Mini proyecto: gestor de tareas (API REST)	CRUD completo en memoria: GET, POST, PUT y DELETE	Todos
07 · Consumir APIs externas con HttpClient	`HttpClient`, GET, `BodyHandlers` y parsear JSON con Gson	Todos
08 · HttpClient avanzado	POST, cabeceras, timeouts, `sendAsync` y PUT/DELETE	Todos
09 · Repaso interactivo	Sé el Código, Fireside, Laboratorio, Crucigrama y más	Todos

> 📖 **Flujo de lectura:** los 8 primeros puntos son teoría en progresión. El 9o es el aterrizaje práctico: léelo justo después del 8o y antes de abrir los boletines.

📝 Boletines de la unidad

> Practica con los pares del curso: intenta primero el por-resolver y comprueba con el resuelto cuando hayas terminado.

🟢 Inicial por resolver ✅ Inicial resuelto ⭐ Avanzado por resolver 💪 Avanzado resuelto 🔥 Extras

---

✅ Criterios de evaluación cubiertos (RA5)

RA5: Realiza operaciones de entrada y salida de información, utilizando procedimientos específicos del lenguaje y librerías de clases.

CE	Criterio	Dónde se cubre
RA5 c)	Se han reconocido las posibilidades de entrada / salida del lenguaje y las librerías asociadas.	✅ Puntos 1, 2, 7 y 8
RA5 f)	Se han utilizado las herramientas del entorno de desarrollo para crear interfaces gráficos de usuario simples.	✅ Puntos 3 y 6
RA5 g)	Se han programado controladores de eventos.	✅ Punto 6
RA5 h)	Se han escrito programas que utilicen interfaces gráficos para la entrada y salida de información.	✅ Puntos 5 y 6

> 📌 La **RA5** se completa con la **U03** (consola y formatos, CEs a y b) y la **U14** (ficheros y acceso a archivos, CEs c, d y e). Esta unidad cubre la cara web de la entrada/salida: HTTP como protocolo, HTML y JSON como formatos, y las librerías `HttpServer` y `HttpClient` como procedimientos del lenguaje.

🚪 ¿Por dónde empiezo?

- ¿Vienes de la U15 (JDBC) y quieres lo esencial? → Arranca en el punto 1 y el punto 2: HTTP y tu primer servidor.
- ¿Ya sabes servir y quieres hacer APIs? → Ve directo al punto 5 y al punto 6: JSON y el CRUD de tareas.
- ¿Solo vienes a por el lado cliente? → Salta al punto 7 y al punto 8: consumir APIs externas.
- ¿Vienes a repasar? → Haz el Repaso interactivo y después los boletines.

📍 **Primer punto:** 01 · El protocolo HTTP en 30 segundos

🎓 **Fin del curso.** Empezaste escribiendo `System.out.println("Hola, mundo")` en una terminal y acabas sirviendo y consumiendo APIs por Internet. De consola a web en 14 unidades: no es poco. La siguiente parada ya no tiene guía: elige tu camino (Spring Boot, tu proyecto final, tu primera entrevista) y sigue programando.

01 · El protocolo HTTP en 30 segundos

Petición-respuesta, verbos HTTP, códigos de estado y por qué toda la web se sostiene sobre una conversación tan simple 🌐 ⚡

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 01 · El protocolo HTTP en 30 segundos

---

📬 La idea en una frase

> **HTTP es un protocolo de petición-respuesta: el cliente pide un recurso (una URL) y el servidor le responde con un código de estado y el contenido. Eso es todo. El resto son capas que se construyen encima.**

En la U14 ya cerrabas ficheros con `try-with-resources` y en la U15 conectabas Java a bases de datos. Ahora el protagonista es otro: la web. Y da igual si algún día usas Tomcat, Spring Boot o un microservicio en la nube: todos hablan el mismo idioma, y ese idioma se llama HTTP. Hoy lo aprendes en 30 segundos… y en unas pocas páginas más.

💬 La conversación de la web

Cuando escribes `https://google.com` en tu navegador, ocurre esto:

```
Tú (cliente)                    Google (servidor)
    ── GET / HTTP/1.1 ──────────►
    │                               │
    ◄── HTTP/1.1 200 OK ────────────
         Content-Type: text/html
         (el HTML de Google)
```

Tres piezas:

1. **Petición:** el cliente pide un recurso. Incluye un método (`GET`), una ruta (`/`) y cabeceras (información extra como `User-Agent`).
2. **Respuesta:** el servidor contesta con un **código de estado** (`200 OK`) y un cuerpo (el HTML, un JSON, una imagen…).

3. **Cabeceras:** notas al margen que viajan en ambos sentidos. La más importante de momento: `Content-Type`, que le dice al cliente qué tipo de contenido le mandas (`text/html`, `application/json`…).

> 💡 **Consejo:** memoriza la estructura del código de estado: el primer dígito es la familia. **2xx** todo bien, **3xx** redirecciones, **4xx** el cliente se ha equivocado, **5xx** el servidor ha fallado. Con eso ya vas sobrado para interpretar cualquier error.

🎤 Los verbos de HTTP

Los métodos HTTP son el verbo de la frase. Hay muchos, pero con estos cuatro haces el 99 % de tu vida:

Método	Qué hace	Analogía
`GET`	Pedir información	Preguntar
`POST`	Enviar información nueva	Entregar un formulario
`PUT`	Reemplazar un recurso completo	Devolver algo por uno nuevo
`DELETE`	Borrar un recurso	Tirar a la papelera

Fíjate en la diferencia fina: `GET` no cambia nada en el servidor (es de solo lectura), mientras que `POST`, `PUT` y `DELETE` lo modifican. Por eso un enlace normal es `GET` y el envío de un formulario de registro es `POST`.

🚦 Los códigos de estado más comunes

No necesitas memorizar los cientos que existen. Con la familia y estos clásicos te basta:

Código	Significado	Momento típico
`200`	Todo bien	Página cargada correctamente
`201`	Recurso creado	Un `POST` que crea algo nuevo
`404`	No encontrado	La URL no existe (o el usuario no existe)

Código	Significado	Momento típico
`403`	Prohibido	Sin permiso (suele faltar un token)
`500`	Error interno	El servidor ha petado por dentro

> 🐛 **Error típico:** cuando tu servidor devuelve `500`, casi siempre es una excepción que no capturaste dentro del handler. El navegador solo ve `500`, pero tu consola de Java te cuenta toda la historia.

🧩 Anatomía de una URL

Todo lo anterior ocurre sobre una URL. Y una URL tiene partes que te interesan:

```
http://localhost:8080/saludo?nombre=Ana&edad=25
└──┘ └────────┘ └──┘ └─────┘ └──────────────┘
protocolo  dominio   puerto  ruta    query string
```

La **query string** (después de `?`) son pares `clave=valor` separados por `&`. Es lo que envía un formulario GET y lo que leerás en el punto 4. El puerto por defecto de HTTP es el 80 (no se ve), pero en desarrollo usarás el 8080.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué dos cosas componen siempre la respuesta de un servidor HTTP?
2. ¿Qué método HTTP usarías para que un usuario registre una cuenta nueva: `GET` o `POST`? ¿Y para ver su perfil?
3. ¿Qué significa un `404`? ¿Y un `500`?
4. En `http://localhost:8080/saludo?nombre=Ana&edad=25`, ¿qué es `nombre=Ana&edad=25`?

> › 🔁 **Respuestas**

✅ Resumen en 3 frases

1. **HTTP es petición-respuesta:** el cliente pide una URL con un método y el servidor responde con un código de estado y un contenido.
2. Los métodos clave son **GET** (leer), **POST** (crear), **PUT** (reemplazar) y **DELETE** (borrar), y los códigos se agrupan en familias: 2xx bien, 4xx culpa del cliente, 5xx culpa del servidor.
3. La **query string** (`?clave=valor&clave2=valor2`) es cómo viajan los datos en una petición GET.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | HTTP | El idioma que hablan cliente y servidor |
> | Petición | Lo que el cliente pide (método + URL + cabeceras) |
> | Respuesta | Lo que el servidor contesta (código + cuerpo + cabeceras) |
> | `Content-Type` | Cabecera que dice qué tipo de contenido se envía |
> | Query string | Los parámetros `clave=valor` que viajan en la URL |

📚 Volver al índice de la unidad · **Anterior:** — · **Siguiente:** 02 · Servidor web mínimo con HttpServer

02 · Servidor web mínimo con HttpServer

Tu primer servidor Java: HttpServer, createContext y HttpExchange en menos de 20 líneas 🖥️🚀

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 02 · Servidor web mínimo con HttpServer

---

📬 La idea en una frase

> **Java trae un servidor HTTP de serie en `com.sun.net.httpserver.HttpServer`: sin Tomcat, sin Spring, sin instalar nada. En menos de 20 líneas tienes un servidor respondiendo en `http://localhost:8080`.**

En el punto anterior viste el protocolo. Ahora toca ponerlo en marcha: no con Tomcat ni con un framework, sino con lo que Java ya lleva dentro desde la versión 6. Es mínimo, es didáctico y, para aprender cómo funciona una petición por dentro, es perfecto.

🖥️ El servidor mínimo

Copia esto, compila y ejecuta:

```java
import com.sun.net.httpserver.HttpServer;
import java.io.IOException;
import java.io.OutputStream;
import java.net.InetSocketAddress;

public class ServidorMinimo {
    public static void main(String[] args) throws IOException {
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );
        server.createContext("/", intercambio -> {
            String resp = "Hola, mundo web!";
            intercambio.sendResponseHeaders(200, resp.length());
            OutputStream os = intercambio.getResponseBody();
            os.write(resp.getBytes());
            os.close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

Abre `http://localhost:8080` en tu navegador. **Acabas de crear tu primer servidor web en Java.** Ahora desmontemos las piezas:

- **`HttpServer.create(new InetSocketAddress(8080), 0)`** — crea el servidor en el puerto 8080. El `0` es el número de conexiones en espera (el backlog); con 0 vale.
- **`server.createContext("/", lambda)`** — registra una ruta y le asocia un *handler*: una lambda que recibe un `HttpExchange`, el objeto que representa "una petición que está entrando".
- **`intercambio.sendResponseHeaders(200, longitud)`** — envía el código de estado y el tamaño del cuerpo. Si pasas `0` o `-1`, Java usa *chunked* y no necesitas el tamaño exacto.
- **`getResponseBody().write(...)` y `.close()`** — escribe el cuerpo y cierra el canal. Cerrar es obligatorio: si no, el navegador se queda esperando.
- **`server.setExecutor(null)`** — usa el ejecutor por defecto; cada petición se atiende en un hilo propio.
- **`server.start()`** — arranca. A partir de aquí el `main` no se "acaba": el servidor se queda escuchando.

> 💡 **Consejo:** el `try-with-resources` de la U14 no vale aquí: el servidor debe estar vivo mientras el `main` sigue corriendo. Por eso `start()` no está dentro de un `try`.

🕶️ Sé el Código, my friend...

> **Don Tip:** el método HTTP lo decide el cliente. `intercambio.getRequestMethod()` te dice cuál ha sido.

Añade justo antes de `server.start()`:

```
server.createContext("/track", intercambio -> {
    System.out.println("Método: " + intercambio.getRequestMethod());
    System.out.println("URI: " + intercambio.getRequestURI());
    System.out.println("Cabeceras: " + intercambio.getRequestHeaders().entrySet());
    intercambio.getResponseHeaders().add("Content-Type", "text/plain");
    intercambio.sendResponseHeaders(200, 0);
    try (var os = intercambio.getResponseBody()) {
        os.write("Todo ok, gracias por preguntar.".getBytes());
    }
});
```

Abre `http://localhost:8080/track?nombre=Ana&edad=25` y mira la consola del servidor: verás el método, la URI con la query string y todas las cabeceras que te manda el navegador. Es el mejor "espía" que vas a tener en toda la unidad.

> › 🔄 **¿Qué esperas ver?**

🚨 "Address already in use"

El error más típico al arrancar el servidor. Significa que el puerto 8080 ya lo está usando otro proceso (una instancia anterior del tuyo, u otra aplicación). Dos soluciones:

- Cambia el puerto en `new InetSocketAddress(8081, 0)`.
- O encuentra y mata al culpable. En Windows: `netstat -ano | findstr 8080` y luego `taskkill /PID <numero> /F`.

> ⚠️ **Advertencia:** los puertos por debajo de 1024 (como el 80) requieren permisos de administrador. Quédate con el 8080 para desarrollo.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué clase de Java te da un servidor HTTP sin dependencias externas?
2. ¿Qué método de `HttpServer` asocias a una ruta con su handler?
3. ¿Qué objeto recibe tu lambda y qué representa?
4. ¿Qué pasa si olvidas cerrar el `getResponseBody()`?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. **`HttpServer`** es el servidor HTTP que Java trae de serie: lo creas en un puerto, registras rutas con `createContext` y lo arrancas con `start()`.
2. El **handler** recibe un `HttpExchange` con el que lees la petición, envías el código de estado con `sendResponseHeaders` y escribes el cuerpo con `getResponseBody()`.
3. **Cierra siempre el cuerpo** de la respuesta y recuerda que el servidor no se detiene hasta que tú lo quieras: es tu primera aplicación "que no se acaba".

> 🐛 **Vocabulario rápido**

Término	Idea general
`HttpServer`	El servidor HTTP del JDK
`createContext`	Asocia una ruta con su handler
`HttpExchange`	Objeto que representa una petición entrante
Handler	La lambda que procesa cada petición de una ruta
`sendResponseHeaders`	Envía el código de estado (y el tamaño del cuerpo)

📚 Volver al índice de la unidad · **Anterior:** 01 · El protocolo HTTP en 30 segundos · **Siguiente:** 03 · Sirviendo HTML

03 · Sirviendo HTML

Tu servidor pasa de hablar en texto plano a servir páginas completas: Content-Type, cadenas bien formadas y ficheros 📄 🎨

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 03 · Sirviendo HTML

📬 La idea en una frase

> **Devolver texto plano es como hablarle al navegador en susurros: se entera, pero no muestra nada bonito. Con `Content-Type: text/html` y una cadena bien formada, tu servidor se convierte en una web de verdad.**

En el punto 2 el servidor contestaba con una frase suelta. Hoy le enseñamos a vestirse: páginas HTML completas. Dos caminos, ambos legítimos: escribirlo directamente en una cadena Java o leerlo desde un fichero (mucho más cómodo cuando la página crece).

📄 HTML desde una cadena

El paso natural: en lugar de `"Hola, mundo web!"`, devuelves una página completa. Fíjate en el bloque de texto (Java 15+):

```
server.createContext("/", intercambio -> {
    String html = """
        <!DOCTYPE html>
        <html>
        <head><title>Mi App Java</title>
        <meta charset="UTF-8">
        </head>
        <body>
            <h1>Bienvenido a mi App</h1>
            <p>Esto se sirve desde Java.</p>
        </body>
        </html>
        """;
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html; charset=UTF-8");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

Dos detalles importantes:

- **`Content-Type: text/html; charset=UTF-8`** — sin esta cabecera, el navegador puede tratar tu HTML como texto plano o interpretar mal las tildes. El `charset=UTF-8` es obligatorio si tu página lleva caracteres como á, ñ o €.
- **`html.getBytes().length`** — el tamaño del cuerpo en **bytes**, no en caracteres. Una ñ ocupa 2 bytes en UTF-8; si pasas `html.length()` (caracteres), el navegador puede quedarse esperando el resto.

> 🐛 **Error típico:** `sendResponseHeaders(200, 0)` con HTML funciona, pero con `0` el navegador no sabe cuándo termina el cuerpo con conexiones persistentes. Pasa siempre el tamaño en bytes (o usa `-1` para *chunked*).

📂 HTML desde un fichero

Escribir HTML dentro de Java es feo cuando la página crece: mezclas dos lenguajes y cualquier comilla es una agonía. La alternativa profesional: guarda la página en un fichero y léela con `Files.readString()` (el mismo API de la U14):

```
server.createContext("/", intercambio -> {
    String html = Files.readString(Path.of("web", "index.html"));
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html; charset=UTF-8");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

Con esta estructura, el HTML vive en `web/index.html` (dentro de tu proyecto, junto a los fuentes) y el código Java solo se encarga de servirlo. Cambiar el diseño ya no exige recompilar: editas el fichero, recargas el navegador y listo.

> 💡 **Consejo:** el fichero puede traer CSS y JavaScript enlazados con rutas relativas (`estilos.css`, `app.js`). Pero ojo: esas rutas también las pide el navegador al servidor, así que tendrás que servir esos ficheros también. Se ve en el punto 6 con el proyecto de tareas.

🕶️ Sé el Código, my friend…

> **Don Tip:** sirve el HTML con `Content-Type` correcto y el navegador es tu mejor aliado; olvida la cabecera y el navegador te enseñará el código en vez de la página.

Ejercicio: el servidor despistado

Un compañero te enseña este handler y jura que funciona:

```
server.createContext("/", intercambio -> {
    String html = "<h1>Hola mundo</h1>";
    intercambio.sendResponseHeaders(200, html.length());
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

Dice que “a veces” el navegador muestra `<h1>Hola mundo</h1>` literal en vez de un encabezado. ¿Por qué? ¿Cuántos bugs hay?

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué cabecera necesitas para que el navegador renderice tu respuesta como HTML?
2. ¿Qué le pasas a `sendResponseHeaders` como segundo argumento: caracteres o bytes?
3. ¿Con qué método de la U14 lees el contenido de un fichero en un `String`?
4. ¿Por qué es mejor servir el HTML desde un fichero que escribir 50 líneas dentro del código Java?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Para servir HTML, fija **`Content-Type: text/html; charset=UTF-8`** y pasa el tamaño del cuerpo **en bytes** (`getBytes().length`).
2. El HTML puede ir **en una cadena** (rápido para probar) o **en un fichero** leído con `Files.readString()` (limpio y profesional).
3. Cerrar el `getResponseBody()` sigue siendo obligatorio: es la señal que el navegador espera para pintar la página.

> 🦎 **Vocabulario rápido**

Término	Idea general
`text/html`	El `Content-Type` de las páginas web
`charset=UTF-8`	Declaración de que el texto usa UTF-8 (tildes incluidas)
`Files.readString`	Lee un fichero completo en un `String` (visto en U14)
Text block	`"""..."""`: cadena multilínea para escribir HTML cómodo

📚 Volver al índice de la unidad · **Anterior:** 02 · Servidor web mínimo con HttpServer · **Siguiente:** 04 · Parámetros GET y formularios POST

04 · Parámetros GET y formularios POST

El navegador pregunta con la URL y entrega con el cuerpo: lee la query string y el body como un profesional 📥📤

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 04 · Parámetros GET y formularios POST

📬 La idea en una frase

> **El cliente pregunta (GET) con la query string de la URL y entrega (POST) con el cuerpo de la petición. Saber leer ambas cosas es saber entender lo que tu web recibe.**

Hasta ahora el servidor contestaba lo mismo a todo el mundo. Hoy aprende a escuchar: qué pide el cliente y qué le envía. Con GET lees la URL; con POST lees el cuerpo. Ambos llegan como texto y tu trabajo es convertirlos en datos útiles.

📥 GET: la query string

Cuando un formulario se envía por GET, los datos viajan en la URL: `http://localhost:8080/saludo?nombre=Ana`. Para leerlos:

```
server.createContext("/saludo", intercambio -> {
    String query = intercambio.getRequestURI().getQuery();
    String nombre = "Mundo";
    if (query != null && query.startsWith("nombre=")) {
        nombre = query.split("=")[1];
    }
    String html = "<h1>Hola, " + nombre + "!</h1>";
    intercambio.getResponseHeaders()
        .set("Content-Type", "text/html");
    intercambio.sendResponseHeaders(200, html.getBytes().length);
    intercambio.getResponseBody().write(html.getBytes());
    intercambio.getResponseBody().close();
});
```

- `getRequestURI().getQuery()` te da `"nombre=Ana"` (o `null` si no hay query).
- El `if` evita dos dramas: que `query` sea `null` y que no empiece por `nombre=`.

> ⚠️ **Advertencia:** esto es **frágil**. Si el nombre contiene `&` o `%20` (espacio codificado), `split("=")[1]` te da basura o peta. En producción usa `URLDecoder.decode(valor, "UTF-8")` y un parser de query de verdad. Este es un ejemplo didáctico, no código de producción.

📮 POST: el cuerpo de la petición

Para recibir datos de un formulario POST necesitas leer el **cuerpo** de la petición:

```
server.createContext("/procesar", intercambio -> {
    if ("POST".equals(intercambio.getRequestMethod())) {
        byte[] body = intercambio.getRequestBody().readAllBytes();
        String datos = new String(body);
        // datos = "nombre=Ana&edad=25"
        String nombre = extraerParametro(datos, "nombre");
        String respuesta = "<h1>Recibido: " + nombre + "</h1>";
        intercambio.getResponseHeaders()
            .set("Content-Type", "text/html");
        intercambio.sendResponseHeaders(200,
            respuesta.getBytes().length);
        intercambio.getResponseBody()
            .write(respuesta.getBytes());
        intercambio.getResponseBody().close();
    }
});
```

Y el HTML del formulario que lo alimenta:

```
<form action="/procesar" method="POST">
  <input name="nombre" placeholder="Tu nombre">
  <button>Enviar</button>
</form>
```

> 💡 **Consejo:** ¿por qué `"POST".equals(intercambio.getRequestMethod())` y no `intercambio.getRequestMethod().equals("POST")`? Porque si `getRequestMethod()` devuelve `null`, la segunda forma casca con `NullPointerException`. La primera forma es **null-safe**. Es una manía que te ahorrará dolores de cabeza para siempre.

🧰 El parser casero

El cuerpo `"nombre=Ana&edad=25"` tiene el mismo formato que la query string: pares `clave=valor` separados por `&`. Un parser mínimo para ambos casos:

```
static String extraerParametro(String datos, String clave) {
    if (datos == null) return null;
    for (String par : datos.split("&")) {
        String[] trozos = par.split("=");
        if (trozos.length == 2 && trozos[0].equals(clave)) {
            return trozos[1];
        }
    }
    return null;
}
```

No es la octava maravilla, pero para aprender cómo funciona el formato por dentro vale. Cuando en el punto 6 hagas la API REST, los cuerpos serán JSON y el parser lo hará Gson.

🕶️ Sé el Código, my friend…

> **Don Tip:** GET y POST pueden llevar los mismos datos, pero en sitios distintos. Un GET es reproducible (puedes repetir la URL); un POST de registro no debería repetirse sin querer.

Ejercicio: el formulario con dos campos

Un formulario envía `nombre=Ana&edad=25` por POST a `/procesar`. Con el `extraerParametro` de arriba:

1. ¿Qué devuelve `extraerParametro(datos, "edad")`?
2. ¿Y `extraerParametro(datos, "ciudad")`?
3. ¿Qué devuelve si `datos` es `null`?

> › 🔄 **Solución**

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Dónde viajan los datos de un formulario GET?
2. ¿Dónde viajan los datos de un formulario POST?
3. ¿Por qué `"POST".equals(método)` es mejor que `método.equals("POST")`?
4. ¿Qué método de `HttpExchange` te da el cuerpo de la petición?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Los **GET** llevan sus datos en la query string (`?clave=valor&...`) y los lees con `getRequestURI().getQuery()`.
2. Los **POST** llevan sus datos en el **cuerpo** de la petición y los lees con `getRequestBody().readAllBytes()`.
3. En ambos casos es texto con formato `clave=valor`, y la manía de `"POST".equals(...)` en vez de `...equals("POST")` te salva de los `NullPointerException`.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | Query string | Datos del GET, en la URL |
> | Request body | Datos del POST, en el cuerpo de la petición |
> | `URLDecoder` | Convierte `%20` y amigos en caracteres legibles |
> | Null-safe | Código que no peta aunque reciba `null` |

📚 Volver al índice de la unidad · **Anterior:** 03 · Sirviendo HTML · **Siguiente:** 05 · Devolviendo JSON: como una API de verdad

05 · Devolviendo JSON: como una API de verdad

application/json, fetch desde el frontend y por qué las aplicaciones modernas separan datos de diseño 🧩🔀

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 05 · Devolviendo JSON: como una API de verdad

---

📬 La idea en una frase

> **Las aplicaciones modernas no devuelven HTML al navegador: devuelven datos en JSON y que el frontend se las arregle. Cuando tu servidor responde con `application/json`, deja de ser una página web y se convierte en una API.**

Hasta aquí tu servidor hablaba en HTML, el idioma de los humanos. Ahora aprende el idioma de las máquinas: JSON. El frontend (JavaScript) pide datos y el backend se los da en JSON, sin mezclar diseño con datos. Es la separación que hace funcionar a todas las aplicaciones modernas.

🧩 Qué es JSON (en 10 segundos)

JSON es texto con una estructura de dos piezas:

```
{
  "id": 1,
  "nombre": "Ana",
  "edad": 25,
  "activo": true
}
```

- **Objetos** entre llaves `{...}`: colección de pares `"clave": valor`.
- **Arrays** entre corchetes `[...]`: listas de valores.
- Valores posibles: textos entre comillas, números, `true`/`false`, `null`, objetos y arrays.

Un `record` de Java (los viste en la U12) y un objeto JSON son primos hermanos: misma idea de "datos con nombre". La diferencia es que JSON es texto, así que cualquier lenguaje lo entiende.

🚀 Servir JSON desde tu servidor

La clave de todo es una cabecera y una cadena:

```
server.createContext("/api/usuarios", intercambio -> {
    String json = """
        [
            {"id":1,"nombre":"Ana","edad":25},
            {"id":2,"nombre":"Luis","edad":30}
        ]
        """;
    intercambio.getResponseHeaders()
        .set("Content-Type", "application/json");
    intercambio.sendResponseHeaders(200, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

Fíjate: nada ha cambiado respecto al HTML salvo dos cosas:

- El `Content-Type` es ahora **`application/json`**.
- El contenido es un array JSON con los datos.

> ⚠️ **Advertencia:** si devuelves JSON con `Content-Type: text/html` o sin cabecera, el navegador no lo tratará como datos. Muchas APIs externas rechazan respuestas que dicen ser JSON y no lo son… y lo notarás. Cabecera correcta o nada.

📡 Consumirlo con fetch

Y ahora el otro lado de la moneda: desde el HTML, JavaScript pide los datos y los pinta. Nada de recargar la página: el `fetch` hace una petición en segundo plano:

```
<script>
fetch('/api/usuarios')
  .then(r => r.json())
  .then(data => console.log(data));
</script>
```

Lo que ocurre por dentro es exactamente el punto 1: el navegador hace un `GET /api/usuarios`, tu servidor responde con `200` y un JSON, y `r.json()` lo convierte en objetos JavaScript listos para usar.

> 💡 **Consejo:** esto es el patrón de todas las aplicaciones modernas: el **backend** sirve datos (JSON) y el **frontend** los consume y decide cómo mostrarlos. Aprender a servir JSON es aprender a hacer APIs; el frontend ya viene de serie en el navegador.

🕶️ Sé el Código, my friend…

> **Don Tip:** cuando el `fetch` falla, no mires solo el `console.log`: abre la pestaña *Network* del navegador y mira el código de estado y el `Content-Type` de la respuesta. El 90 % de los misterios se resuelven ahí.

Ejercicio: el endpoint con cabeza

Escribe el handler de `/api/hora` que devuelva este JSON:

```
{"hora": "14:30:01", "zona": "Europe/Madrid"}
```

Completa el código (sin ejecutar, a ojo):

```
server.createContext("/api/hora", intercambio -> {
    String json = """
        {"hora": "______", "zona": "______"}
        """;
    intercambio.getResponseHeaders()
        .set("______", "application/json");
    intercambio.sendResponseHeaders(______, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

> › 🔄 Solución

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué `Content-Type` usas para servir JSON?
2. ¿Qué dos estructuras componen el formato JSON?
3. ¿Qué método de JavaScript hace una petición HTTP desde el navegador?
4. ¿Cuál es la ventaja de servir datos JSON en vez de HTML?

> 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **JSON** es el formato de datos universal: objetos `{...}` y arrays `[...]`, y un `record` de Java es su primo hermano.
2. Para servir JSON, fija **`Content-Type: application/json`** y el resto del handler es idéntico al de HTML.
3. El **frontend** lo consume con `fetch(...).then(r => r.json())`, y así nace la separación entre datos y diseño que define a las APIs.

> 🐛 **Vocabulario rápido**

Término	Idea general
JSON	Formato de datos universal en texto
`application/json`	El `Content-Type` de JSON
API	Interfaz por la que tu servidor expone datos
`fetch`	La forma de JavaScript de hacer peticiones HTTP
Frontend / Backend	Quien muestra (navegador) / quien sirve (servidor)

📚 Volver al índice de la unidad · **Anterior:** 04 · Parámetros GET y formularios POST · **Siguiente:** 06 · Mini proyecto: gestor de tareas (API REST)

06 · Mini proyecto: gestor de tareas (API REST)

Júntalo todo en una API REST: GET, POST, PUT y DELETE con datos en memoria 🗂️🔧

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 06 · Mini proyecto: gestor de tareas (API REST)

📬 La idea en una frase

> **Ya sabes servir HTML y JSON. Ahora júntalo todo en una API REST con las cuatro operaciones de la vida: crear, leer, actualizar y borrar (CRUD), con los datos viviendo en un `ArrayList` en memoria.**

Este es el punto donde todo cobra sentido. Un servidor que solo contesta "hola" es un monólogo. Una **API REST** es una conversación: el cliente hace `GET`, `POST`, `PUT` o `DELETE` y el servidor responde como es debido. Y no necesitas base de datos: un `ArrayList` en memoria (viste las colecciones en la U12) es suficiente para que funcione.

🗂️ Qué es REST (en 20 segundos)

REST es una forma ordenada de organizar rutas y métodos para que la API se explique sola:

Método	Ruta	Acción	Resultado
GET	`/api/tareas`	Listar todas	`200` + JSON
POST	`/api/tareas`	Crear una	`201` + la tarea creada
PUT	`/api/tareas/1`	Actualizar la tarea 1	`200`
DELETE	`/api/tareas/1`	Borrar la tarea 1	`204` (sin contenido)

La regla de oro: **el método dice el verbo y la ruta dice el sustantivo**. `GET /api/tareas` es "dame las tareas"; `DELETE /api/tareas/3` es "borra la tarea 3". Así, una tabla de cuatro filas documenta toda la API.

🧱 El modelo y el almacén

Primero, la tarea. Un `record` (genérico y limpio, como en la U12):

```
record Tarea(int id, String titulo, boolean completada) {}
```

Y el almacén: un `ArrayList<Tarea>` + un contador de IDs. Lo declaras como campo de la clase del servidor:

```
import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.atomic.AtomicInteger;

public class ApiTareas {
    private static final List<Tarea> tareas = new ArrayList<>();
    private static final AtomicInteger contador = new AtomicInteger(1);

    static {
              tareas.add(new  Tarea(contador.getAndIncrement(),  "Aprender  HttpServer",
false));
         tareas.add(new Tarea(contador.getAndIncrement(), "Hacer la práctica de JSON",
false));
    }

    // ... aquí van los contextos
}
```

> 💡 **Consejo:** `AtomicInteger` te da IDs que no se repiten ni con peticiones simultáneas. Con un `int` normal, dos peticiones a la vez podrían leer el mismo valor. En producción, ese `ArrayList` sería una base de datos (JDBC, como en la U124), pero el patrón es idéntico.

🔧 Los cuatro endpoints

GET: listar todo

```
server.createContext("/api/tareas", intercambio -> {
    if ("GET".equals(intercambio.getRequestMethod())) {
        StringBuilder sb = new StringBuilder("[");
        for (int i = 0; i < tareas.size(); i++) {
            Tarea t = tareas.get(i);
            if (i > 0) sb.append(",");
            sb.append("{\"id\":").append(t.id())
              .append(",\"titulo\":\"").append(t.titulo())
              .append("\",\"completada\":").append(t.completada()).append("}");
        }
        sb.append("]");
        responderJSON(intercambio, sb.toString(), 200);
    }
});
```

Aquí el JSON se construye a mano para que veas qué es cada pieza. En el punto 7 descubrirás que Gson lo hace por ti con una línea: `new Gson().toJson(tareas)`.

POST: crear

```
server.createContext("/api/tareas", intercambio -> {
    if ("POST".equals(intercambio.getRequestMethod())) {
        String body = new String(intercambio.getRequestBody().readAllBytes());
        // body = {"titulo": "Comprar café"}
        String titulo = body.split("\"titulo\":\"")[1].split("\"")[0];
        Tarea nueva = new Tarea(contador.getAndIncrement(), titulo, false);
        tareas.add(nueva);
        responderJSON(intercambio,
                      "{\"id\":" + nueva.id() + ",\"titulo\":\"" + titulo +
"\",\"completada\":false}",
            201);
    }
});
```

> ⚠️ **Advertencia:** parsear JSON con `split` es un apaño didáctico. Si el título lleva comillas, se rompe. El camino profesional es Gson (puntos 7 y 8), que parsea esto en una línea y sin drama.

PUT y DELETE: actualizar y borrar por ID

Para rutas con parámetro (`/api/tareas/3`) hay que leer el ID de la propia ruta:

```
server.createContext("/api/tareas/", intercambio -> {
    String ruta = intercambio.getRequestURI().getPath();
    int id = Integer.parseInt(ruta.substring("/api/tareas/".length()));

    if ("PUT".equals(intercambio.getRequestMethod())) {
        for (int i = 0; i < tareas.size(); i++) {
            if (tareas.get(i).id() == id) {
                tareas.set(i, new Tarea(id, tareas.get(i).titulo(), true));
                responderJSON(intercambio, "{\"actualizada\":true}", 200);
                return;
            }
        }
        responderJSON(intercambio, "{\"error\":\"tarea no encontrada\"}", 404);
    } else if ("DELETE".equals(intercambio.getRequestMethod())) {
        boolean borrada = tareas.removeIf(t -> t.id() == id);
        if (borrada) {
            intercambio.sendResponseHeaders(204, -1);
        } else {
            responderJSON(intercambio, "{\"error\":\"tarea no encontrada\"}", 404);
        }
        intercambio.close();
    }
});
```

Fíjate en dos detalles: el `404` cuando el ID no existe (el cliente pregunta por algo que no está), y el `204` del DELETE (borrar "bien" no devuelve cuerpo).

> 💡 **Consejo:** `tareas.removeIf(...)` es la joya de las colecciones de la U12: borra todos los elementos que cumplan la condición en una línea, sin `ConcurrentModificationException` porque es un método de la propia lista.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué método HTTP usas para crear un recurso? ¿Y para listarlos?
2. ¿Qué código de estado devuelves cuando el cliente pide un ID que no existe?
3. ¿Qué devuelve un `DELETE` que ha salido bien?

4. ¿Por qué sirve un `ArrayList` como almacén si al reiniciar se pierde todo?

> 🔁 **Respuestas**

✅ Resumen en 3 frases

1. Una **API REST** ordena rutas y métodos: `GET` lee, `POST` crea, `PUT` actualiza, `DELETE` borra, y cada ruta dice qué recurso toca.
2. El **almacén en memoria** (`ArrayList` + `AtomicInteger`) es perfecto para aprender el patrón; la base de datos de la U15 solo sustituye el almacén, no las rutas.
3. Maneja los **errores con códigos honestos**: `404` si no existe, `201` si creas, `204` si borras; así tu API habla claro.

> 🦎 **Vocabulario rápido**

Término	Idea general
REST	Estilo de organización de rutas y métodos de una API
CRUD	Crear, leer, actualizar, borrar (las 4 operaciones)
Endpoint	Una ruta concreta de la API (`/api/tareas`)
Path param	El `3` de `/api/tareas/3`, leído de la propia ruta
`204`	Respuesta de éxito sin contenido (típico de DELETE)

📚 Volver al índice de la unidad · **Anterior:** 05 · Devolviendo JSON: como una API de verdad · **Siguiente:** 07 · Consumir APIs externas con HttpClient

07 · Consumir APIs externas con HttpClient

Tu programa se convierte en cliente: HttpClient, GET, BodyHandlers y parsear JSON con Gson 📡🔍

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 07 · Consumir APIs externas con HttpClient

---

📬 La idea en una frase

> **Hasta ahora has sido el servidor. Con `java.net.http.HttpClient`, tu programa se convierte en cliente y le pregunta a las APIs de todo el mundo: GitHub, el tiempo, tu red social favorita…**

Los puntos anteriores eran sobre recibir. Ahora toca pedir: en el mundo real, tus programas llaman a APIs de terceros constantemente. Java 11 trae `HttpClient`, un cliente HTTP moderno y sin dependencias externas, con soporte de HTTP/2, peticiones síncronas y asíncronas, y gestión de cabeceras.

> 📝 **Nota:** antes de Java 11 tenías que usar `HttpURLConnection` (feo, verboso, un castigo) o librerías externas como Apache HttpClient u OkHttp. `java.net.http.HttpClient` llegó para salvar a la humanidad.

📡 El trío sagrado del cliente

Consumir una API son siempre tres piezas:

1. **`HttpClient`** — el que hace la llamada. Un objeto, reutilizable para todas tus peticiones.
2. **`HttpRequest`** — la petición: URL, método, cabeceras. Se construye con *builder*.
3. **`HttpResponse`** — la respuesta: código de estado, cabeceras y cuerpo.

El GET más básico que existe:

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClienteGET {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.github.com/users/octocat"))
            .GET()
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Código: " + response.statusCode());
        System.out.println("Cuerpo: " + response.body());
    }
}
```

Dos métodos clave:

- **`client.send(...)`** — hace la petición de forma **síncrona**: el programa se queda esperando hasta que llega la respuesta (o falla). Es el que usarás el 90 % del tiempo.
- **`HttpResponse.BodyHandlers.ofString()`** — le dice a Java "conviérteme el cuerpo en un `String`". Hay más: `ofByteArray()`, `ofInputStream()`, `ofFile(Path)`… según lo que necesites.

> 💡 **Consejo:** para probar, jsonplaceholder (`https://jsonplaceholder.typicode.com`) es la API de juguete perfecta: devuelve JSON de ejemplo para practicar sin registro ni tokens.

🧩 Parsear JSON con Gson

El cuerpo que devuelve GitHub es JSON. Y aquí viene la noticia triste: **Java no trae parser JSON nativo**. Necesitas una librería. Las más usadas:

Librería	Grupo Maven	Ideal para
Gson (Google)	`com.google.code.gson:gson`	Sencilla, mapeo a clases
Jackson	`com.fasterxml.jackson.core:jackson-databind`	Potente, rápida, estándar industrial
org.json	`org.json:json`	La más simple, sin mapeo

Usaremos **Gson**, la más intuitiva. Dos formas de usarla:

Forma 1 — a lo bruto, con `JsonObject`:

```
import com.google.gson.Gson;
import com.google.gson.JsonObject;

Gson gson = new Gson();
JsonObject json = gson.fromJson(response.body(), JsonObject.class);

String login = json.get("login").getAsString();
String nombre = json.get("name").getAsString();
System.out.println("Usuario: " + login + " — " + nombre);
```

Forma 2 — mapeando a un `record` (los de la U12):

```
record UsuarioGitHub(String login, String name, int public_repos) {}

UsuarioGitHub usuario = gson.fromJson(response.body(), UsuarioGitHub.class);
System.out.println(usuario.name() + " tiene " + usuario.public_repos() + " repos públicos");
```

> ⚠️ **Advertencia:** si la API devuelve campos que no existen en tu `record`, Gson los ignora por defecto. Si tu `record` tiene campos que no están en el JSON, se quedan en `null`. Con Jackson puedes configurarlo con `@JsonIgnoreProperties(ignoreUnknown = true)`.

🌟 El ejemplo completo: los repos de GitHub

Juntémoslo todo: consumir la API de GitHub para listar los repositorios de un usuario.

```java
import com.google.gson.Gson;
import com.google.gson.JsonArray;
import com.google.gson.JsonObject;
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;
import java.time.Duration;

public class GitHubRepos {
    public static void main(String[] args) throws Exception {
        String usuario = args.length > 0 ? args[0] : "octocat";

        HttpClient client = HttpClient.newBuilder()
            .connectTimeout(Duration.ofSeconds(10))
            .build();

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://api.github.com/users/" + usuario + "/repos"))
            .header("Accept", "application/vnd.github+json")
            .header("User-Agent", "JavaCliente/1.0")
            .timeout(Duration.ofSeconds(15))
            .GET()
            .build();

        HttpResponse<String> response = client.send(request,
            HttpResponse.BodyHandlers.ofString());

        if (response.statusCode() == 200) {
            Gson gson = new Gson();
            JsonArray repos = gson.fromJson(response.body(), JsonArray.class);

            System.out.println("Repositorios de " + usuario + ":");
            System.out.println("————————————————————————————————");
            for (int i = 0; i < repos.size(); i++) {
                JsonObject repo = repos.get(i).getAsJsonObject();
                String nombre = repo.get("name").getAsString();
                String lenguaje = repo.has("language")
                    && !repo.get("language").isJsonNull()
                    ? repo.get("language").getAsString()
                    : "?";
                int estrellas = repo.get("stargazers_count").getAsInt();
```

```
                System.out.printf("⭐ %s (%s) ★ %d%n", nombre, lenguaje, estrellas);
            }
        } else if (response.statusCode() == 403) {
            System.out.println("Rate limit alcanzado. Espera un minuto.");
        } else {
            System.out.println("Error " + response.statusCode()
                + " — ¿existe el usuario " + usuario + "?");
        }
    }
}
```

> 📝 **Nota:** fíjate en `repo.has("language") && !repo.get("language").isJsonNull()`. En JSON un campo puede existir pero valer `null`. Si intentas `getAsString()` sobre un `null`, Gson lanza una excepción. Este patrón es muy común al parsear APIs reales.

Salida de ejemplo:

```
Repositorios de octocat:
──────────────────────────────────
⭐ Hello-World (Java) ★ 2727
⭐ Spoon-Knife (HTML) ★ 13291
⭐ Octocat (?) ★ 5
```

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Cuáles son las tres clases del trío sagrado del cliente HTTP?
2. ¿Qué hace `client.send(...)`: síncrono o asíncrono?
3. ¿Cómo le dices a `HttpClient` que el cuerpo de la respuesta sea un `String`?
4. ¿Qué librería usamos para parsear JSON y cuál es la forma de mapearlo a una clase?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. **`HttpClient` + `HttpRequest` + `HttpResponse`** son el trío para consumir APIs: `client.send(...)` hace la petición síncrona y `BodyHandlers.ofString()` te da el cuerpo

como texto.

2. **Gson** parsea ese texto JSON: a lo bruto con `JsonObject` o mapeando a un `record` con `fromJson`.
3. En JSON un campo puede existir pero ser `null`: comprueba con `has(...)` y `isJsonNull()` antes de `getAsString()`, o Gson te peta.

> 🐛 **Vocabulario rápido**
>
> | Término | Idea general |
> |---|---|
> | `HttpClient` | El cliente HTTP moderno de Java 11+ |
> | `BodyHandler` | Le dice a Java cómo convertir el cuerpo (`ofString`, `ofByteArray`…) |
> | Gson | Librería para convertir JSON en objetos Java (y al revés) |
> | `JsonObject` / `JsonArray` | El JSON sin mapear, accesible a lo bruto |
> | Rate limit | Límite de peticiones por minuto que imponen las APIs |

📚 Volver al índice de la unidad · **Anterior:** 06 · Mini proyecto: gestor de tareas (API REST) · **Siguiente:** 08 · HttpClient avanzado

08 · HttpClient avanzado

POST con BodyPublishers, cabeceras y timeouts, sendAsync sin bloquear y PUT/DELETE como un cliente profesional 🚀🧠

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 08 · HttpClient avanzado

---

📬 La idea en una frase

> **El GET es el saludo. Hoy aprendes el resto de la conversación: enviar datos con POST, afinar la petición con cabeceras y timeouts, no bloquear el programa con `sendAsync` y hasta borrar con DELETE.**

El punto 7 era leer. Este es escribir: mandar datos a una API, controlar el tiempo de espera, interpretar cabeceras, lanzar peticiones sin congelar el programa y completar el repertorio de métodos HTTP. Al terminar, ya eres un cliente completo.

📤 POST: enviar datos a una API

Para crear un recurso, hacer login o registrar algo, usas POST con un cuerpo JSON:

```java
import java.net.URI;
import java.net.http.HttpClient;
import java.net.http.HttpRequest;
import java.net.http.HttpResponse;

public class ClientePOST {
    public static void main(String[] args) throws Exception {
        HttpClient client = HttpClient.newHttpClient();

        String json = """
            {"title": "Aprender HttpClient",
             "body": "Ya casi lo tengo",
             "userId": 1}
            """;

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/posts"))
            .header("Content-Type", "application/json")
            .POST(HttpRequest.BodyPublishers.ofString(json))
            .build();

        HttpResponse<String> response = client.send(
            request, HttpResponse.BodyHandlers.ofString()
        );

        System.out.println("Código: " + response.statusCode());
        System.out.println("Creado: " + response.body());
    }
}
```

Dos novedades:

- **`BodyPublishers.ofString(json)`** — el espejo del `BodyHandler`: convierte tu `String` en el cuerpo de la petición. Hay `ofByteArray()`, `ofFile()`… y `ofInputStream()` para casos raros.
- **`.header("Content-Type", "application/json")`** — sin esta cabecera, el servidor puede rechazar tu petición o interpretar mal el cuerpo. JSON sin `Content-Type` es como mandar una carta sin remitente.

> 💡 **Consejo:** `HttpRequest.BodyPublishers` tiene métodos para enviar String, byte[], archivos, etc. El más común es `ofString()` para JSON. Y si el servidor responde `415`, ese código significa “Unsupported Media Type”: casi siempre es que olvidaste el `Content-Type`.

🛡️ Cabeceras, timeouts y manejo de errores

Las APIs no siempre se portan bien: a veces tardan, a veces se caen, a veces te devuelven 401, 403, 404 o 500. Tu código debe estar preparado.

```java
import java.net.http.HttpConnectTimeoutException;
import java.time.Duration;

HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(10))   // tiempo máximo para conectar
    .build();

HttpRequest request = HttpRequest.newBuilder()
    .uri(URI.create("https://api.github.com/users/someone"))
    .header("Accept", "application/vnd.github+json")
    .header("User-Agent", "MiAppJava/1.0")
    .timeout(Duration.ofSeconds(15))          // tiempo máximo para recibir respuesta
    .GET()
    .build();

try {
    HttpResponse<String> response = client.send(request,
        HttpResponse.BodyHandlers.ofString());

    int codigo = response.statusCode();
    if (codigo >= 200 && codigo < 300) {
        System.out.println("Éxito: " + response.body());
    } else if (codigo == 404) {
        System.out.println("El recurso no existe.");
    } else if (codigo == 403) {
        System.out.println("Prohibido. ¿Token necesario?");
    } else if (codigo >= 500) {
        System.out.println("Error del servidor. Vuelve a intentarlo.");
    }
} catch (HttpConnectTimeoutException e) {
    System.out.println("El servidor no responde (timeout de conexión).");
} catch (java.net.http.HttpTimeoutException e) {
    System.out.println("La respuesta tardó demasiado.");
} catch (java.io.IOException e) {
    System.out.println("Error de red: " + e.getMessage());
} catch (InterruptedException e) {
    System.out.println("La petición fue interrumpida.");
}
```

⚠️ **Advertencia:** muchas APIs públicas (GitHub, Twitter, etc.) exigen un **token de autenticación** en la cabecera `Authorization`. Sin él, tienes cuotas muy bajas (rate limiting) o

> acceso denegado. Si ves un `403`, probablemente necesitas registrar una aplicación y obtener un token.

Las cabeceras más comunes que enviarás:

Cabecera	Propósito
`Content-Type`	Tipo de datos que envías (`application/json`)
`Accept`	Tipo de datos que esperas recibir
`Authorization`	Token Bearer, Basic Auth, etc.
`User-Agent`	Identifica tu aplicación (muchas APIs lo exigen)
`Cache-Control`	Control de caché

⚡ Llamadas asíncronas con `sendAsync`

Si necesitas varias peticiones sin bloquear el programa, `HttpClient` también tiene modo asíncrono:

```
CompletableFuture<HttpResponse<String>> futuro = client.sendAsync(
    request, HttpResponse.BodyHandlers.ofString()
);

// Mientras tanto, puedes hacer otras cosas...
System.out.println("Esperando respuesta...");

// Cuando llegue:
futuro.thenAccept(response -> {
    System.out.println("Código: " + response.statusCode());
    System.out.println(response.body());
});

// O bloquear hasta que llegue (no recomendado si vas justo de tiempo):
HttpResponse<String> resp = futuro.get();
```

> 💡 **Consejo:** `sendAsync` devuelve un `CompletableFuture`, la forma elegante de Java para el código asíncrono sin liarte con hilos manuales. Puedes encadenar llamadas con `thenApply`,

> `thenCompose`, etc. La diferencia con `send()`: el primero no espera (sigue el `main`) y el segundo congela el programa hasta tener la respuesta.

🗑️ PUT, DELETE y otros métodos

`HttpClient` soporta todos los métodos HTTP. La diferencia es solo la llamada al builder:

```
// PUT
HttpRequest putRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .header("Content-Type", "application/json")
    .PUT(HttpRequest.BodyPublishers.ofString(
        "{\"nombre\": \"Actualizado\"}"))
    .build();

// DELETE
HttpRequest deleteRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .DELETE()
    .build();

// Con método personalizado (PATCH, etc.)
HttpRequest patchRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://api.ejemplo.com/recursos/1"))
    .method("PATCH", HttpRequest.BodyPublishers.ofString(
        "{\"nombre\": \"Parcial\"}"))
    .build();
```

Fíjate: `PUT` lleva cuerpo (reemplaza con contenido nuevo), `DELETE` no. Y si necesitas algo que no tiene método propio (como `PATCH`), `method(...)` te lo da.

🏗️ Frameworks modernos (una ojeada)

El `HttpServer` nativo es perfecto para aprender, pero en el mundo real se usan frameworks:

- **Javalin** — minimalista, moderno, ideal para APIs. En 3 líneas tienes un servidor.
- **Spring Boot** — el estándar industrial. Lo tiene todo, pero es más pesado.
- **Spark** — similar a Javalin, muy ligero.

Ejemplo con Javalin:

```
import io.javalin.Javalin;

public class App {
    public static void main(String[] args) {
        Javalin app = Javalin.create().start(8080);
        app.get("/", ctx -> ctx.result("Hola desde Javalin"));
        app.get("/api/usuarios", ctx ->
            ctx.json(List.of(new Usuario(1, "Ana"))));
    }
}
```

> 💡 **Consejo:** en este curso usamos `HttpServer` nativo como servidor y `HttpClient` para consumir APIs: no necesitas instalar nada más que Gson. Los frameworks los verás en el módulo de "Desarrollo de Interfaces" y, si te lanzas, en tu proyecto final. Ahora entiendes el terreno que pisan: todos hablan HTTP.

🎯 Mini-chequeo

Ponte a prueba en 30 segundos (las respuestas están escondidas):

1. ¿Qué método de `BodyPublishers` usas para mandar un `String` en el cuerpo de un POST?
2. ¿Qué cabecera es imprescindible al enviar JSON?
3. ¿Qué diferencia hay entre `send()` y `sendAsync()`?
4. ¿Cómo haces un DELETE con `HttpClient`?

> › 🔄 **Respuestas**

✅ Resumen en 3 frases

1. Para **enviar datos** usas `POST` con `BodyPublishers.ofString(json)` y la cabecera `Content-Type: application/json`.
2. Las APIs se caen: usa **timeouts** (`connectTimeout`, `timeout`), comprueba el código de estado y captura las excepciones de red con elegancia.
3. **`sendAsync()`** devuelve un `CompletableFuture` y no bloquea el programa; `PUT`, `DELETE` y `PATCH` se hacen cambiando solo la llamada al builder.

🦎 Vocabulario rápido

Término	Idea general
`BodyPublishers`	Convierte tu `String`/`byte[]` en el cuerpo de la petición
`Authorization`	Cabecera para tokens y credenciales
Timeout	Tiempo máximo de espera antes de rendirse
`CompletableFuture`	La promesa de Java: “llegará, pero no ahora”
`415`	Unsupported Media Type: olvidaste el `Content-Type`

📚 Volver al índice de la unidad · **Anterior:** 07 · Consumir APIs externas con HttpClient · **Siguiente:** 09 · Repaso interactivo

09 · Repaso interactivo: domina la web

El cierre práctico de la unidad y del curso, con un parser que lo va a liar todo y un servidor que no arranca 😈🌐

> 🗺️ **Estás en:** 🌐 **U16 · Servir y consumir APIs con Web** → 09 · Repaso interactivo

📬 La idea en una frase

> **En este punto no aprendemos nada nuevo: lo convertimos todo en práctica. Y, como siempre, algo no va a funcionar. 😈**

⭐ Sé el Código, my friend…

> *Eres la JVM. Acaban de darte este programa para ejecutar:*

```
public class Misterio {

    static String extraer(String query, String clave) {
        if (query == null) return "nada";
        for (String par : query.split("&")) {
            String[] trozos = par.split("=");
            if (trozos.length == 2 && trozos[0].equals(clave)) {
                return trozos[1];
            }
        }
        return "nada";
    }

    public static void main(String[] args) {
        System.out.println(extraer("nombre=Ana&edad=25", "edad"));
        System.out.println(extraer("nombre=Ana&edad=25", "ciudad"));
        System.out.println(extraer("clave=valor", "clave"));
    }
}
```

¿Qué imprimes por pantalla? Elige sabiamente:

1. **`25 nada valor`** → ✅ ¡Correcto! Para la clave `edad` encuentra el par `edad=25` y devuelve `"25"`. Para `ciudad` no hay par con esa clave → `"nada"`. Para `clave` → `"valor"`.
2. **`nada 25 valor`** → Leíste el orden al revés: piensa qué clave se busca primero. ❌
3. **`25 25 valor`** → El segundo `System.out` no busca "edad" dos veces: busca `ciudad`, que no existe. ❌

> 🔄 **Solución**

🔥 Fireside Chat: GET vs POST

> *Dos primos del protocolo se enzarzan frente a la máquina de café.*

GET: — Yo soy el de toda la vida. Pido, me dan, no cambio nada. ¿Quieres repetir mi petición? Solo pegas la URL en el navegador y ya está. Reproducible, cacheable, simple.

POST: — ¿Y para enviar el formulario de registro de un usuario? ¿Vas a meter su contraseña en la URL para que quede en el historial y en los logs del servidor? Eso es lo que harías tú. Yo lo llevo en el cuerpo: privado, ordenado y sin límite de tamaño práctico.

GET: — Vale, para datos sensibles eres tú. Pero yo soy el que carga el 99 % de la web: cada página, cada imagen, cada búsqueda pasa por mí. ¿Cuántas veces te he visto en un formulario de búsqueda? Ah, que el buscador usa GET, porque así el resultado se puede compartir y enlazar. Interesante.

POST: — Tú para leer y compartir. Yo para enviar y crear. ¿Tregua?

GET: — Tregua. Pero no me mandes un `DELETE` con cuerpo, que eso es de mal gusto.

> La lección: **¿leer, cachear, compartir? → GET. ¿enviar datos, crear, algo sensible? → POST.** Y recuerda: un GET no debe cambiar el estado del servidor; eso es trabajo de POST, PUT y DELETE.

🕵️ ¿Quién soy?

Adivina qué concepto de la unidad soy:

1. **Soy el idioma que hablan el cliente y el servidor, y el 200 es mi saludo favorito.**
2. **Soy el servidor web que Java trae de serie; créame un puerto y un contexto y le atenderé.**

3. **Soy el formato de datos que cualquier lenguaje entiende, con llaves y corchetes.**
4. **Soy la cabecera que delata si la respuesta es HTML, JSON o texto plano.**
5. **Soy el cliente de Java 11+ que consulta APIs ajenas con el trío cliente-petición-respuesta.**
6. **Soy la librería que convierte JSON en objetos Java, con `fromJson` como conjuro.**

> 🔄 **Respuestas**

🤬 CONRAD VS EL MUNDO: "El puerto que no arranca"

> *CONRAD, nuestro compilador cascarrabias, opina sobre los clásicos del novato en la web.*

CONRAD: — ¡OTRA VEZ! Viene un alumno y me dice: *CONRAD, mi servidor no arranca*. Y yo: vale, ¿qué error? *Pues algo de "Address already in use".* ¡AY, MADRE MÍA! Tienes otra instancia del servidor corriendo desde la práctica anterior. Cierra la pestaña de la terminal o cambia de puerto: `new InetSocketAddress(8081, 0)`. El puerto no es de tu propiedad vitalicia.

Y luego está el del HTML. Devuelve una página y el navegador le enseña el código en vez de la web. ¿Adivinas qué falta? El `Content-Type: text/html`. Sin la cabecera, el navegador hace lo que puede, y lo que puede es mostrarte el código como texto plano. La cabecera es el uniforme: sin ella, nadie sabe qué eres.

Y el colmo de los colmos: el que parsea el JSON a mano con `split`. Le llega un título con una comilla y Gson no tiene culpa de nada: es que el `split("\"titulo\":\"")` se ha descosido. ¿Por qué? Porque existe Gson para eso. En el servidor y en el cliente: **para parsear JSON, deja de usar tijeras.** Un `split` es para aprender, no para producir.

La lección: antes de acusar a Java de "odiarte", repasa el trío sagrado de la unidad: **¿tengo la cabecera correcta? ¿estoy usando el método HTTP adecuado? ¿estoy parseando JSON con una herramienta de verdad?** El compilador no te odia: te está pasando las respuestas del examen.

🎮 El juego de las decisiones

Elige la respuesta correcta para cada decisión (respuestas al final):

1. Un formulario de inicio de sesión envía la contraseña. ¿GET o POST?

- a) GET b) POST

2. Tu endpoint devuelve una lista de usuarios. ¿Qué `Content-Type` pones?
   - a) `text/html` b) `application/json`
3. El cliente pide `/api/tareas/99` y esa tarea no existe. ¿Qué código devuelves?
   - a) `200` b) `404`
4. En `HttpClient`, ¿qué método envía datos en el cuerpo de la petición?
   - a) `.GET()` b) `.POST(...)`
5. `gson.fromJson(cuerpo, Usuario.class)` con un campo que no está en el JSON… ¿qué recibe ese campo?
   - a) `null` b) `""` (cadena vacía)

> 🔄 **Soluciones**

⚡ Laboratorio de tortura: el servidor que no era

> **Duración estimada:** 30 minutos **Herramienta:** tu IDE y un archivo nuevo

El escenario: copia este programa y haz que funcione. Te dicen que tiene **3 errores** que impiden que compile y 1 error de lógica que hace que el JSON salga mal… pero, ¿y si te lo dicen mal? Tu tarea: hacer que compile, que ejecute y que **toda** la salida sea correcta, contando tú los errores reales.

```java
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;

public class ApiHora {

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );
        server.createContext("/api/hora", e -> {
            String json = "{\"hora\": \"" + java.time.LocalTime.now() + "\"}";
            e.getResponseHeaders().set("Content-Type", "application/json");
            e.sendResponseHeaders(200, json.getBytes().length);
            e.getResponseBody().write(json.getBytes())
            e.getResponseBody().close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080")
    }
}
```

Fallo intencionado: el número de errores del enunciado está mal a propósito: si buscas el "tercer error de compilación" y el "error de lógica", te vuelves loco. La trampa es tu confianza en el enunciado.

Tu tarea: conseguir que compile, que ejecute y que **toda** la salida sea correcta. Si el resultado no cuadra, diagnostica.

Pistas para cuando te frustres (no antes):

1. ¿Hay algún ; que falte? *no → sigue buscando.*
   > **¿Y si sigo atascado?**
2. ¿Compila ya? *no → mira los tipos y los imports.*
   > **¿Y si sigo atascado?**
3. ¿Crees que hay un error de lógica que rompe el JSON? *Vuelve a leer el código con calma y no te fíes del enunciado.*
   > **Solución final**

Los **errores de compilación**:

1. Falta el `;` al final de `e.getResponseBody().write(json.getBytes())`.
2. Falta el `;` al final de `System.out.println(...)`.

El **error de lógica**: no existe. El JSON `{"hora": "09:30:05"}` se construye bien y el `Content-Type` es correcto. Esa era la fallada intencionada: el enunciado dice “3 errores de compilación y 1 de lógica”, pero solo hay 2 faltas de `;` y ninguna lógica rota. El error “oculto” era tu confianza en el enunciado. El servidor funciona en `http://localhost:8080/api/hora`.

```
import com.sun.net.httpserver.HttpServer;
import java.net.InetSocketAddress;

public class ApiHora {

    public static void main(String[] args) throws Exception {
        HttpServer server = HttpServer.create(
            new InetSocketAddress(8080), 0
        );
        server.createContext("/api/hora", e -> {
            String json = "{\"hora\": \"" + java.time.LocalTime.now() + "\"}";
            e.getResponseHeaders().set("Content-Type", "application/json");
            e.sendResponseHeaders(200, json.getBytes().length);
            e.getResponseBody().write(json.getBytes());
            e.getResponseBody().close();
        });
        server.setExecutor(null);
        server.start();
        System.out.println("Servidor en http://localhost:8080");
    }
}
```

La lección real: **lee el código, no el enunciado**. Un bug puede ser que te digan “hay 3 errores” y solo haya 2: si buscas el tercero, te vuelves loco. Cuenta, verifica y confía en el compilador, no en la leyenda.

🏆 Logros de esta unidad

Logro	Cómo conseguirlo
🏅 **El Cartero**	Explicar la diferencia entre GET y POST con un ejemplo de cada uno
🏅 **El Gran Jefe del Puerto**	Montar un `HttpServer` que no diga “Address already in use” y sirva dos rutas
🏅 **El Sommelier de Content-Type**	Devolver HTML, JSON o texto plano con la cabecera correcta en cada caso
🏅 **El Cliente Elegante**	Consumir una API externa con `HttpClient` y parsear el JSON con Gson
🏅 **El Maestro de la API**	Construir un CRUD REST completo (GET, POST, PUT, DELETE) en memoria

🧠 Atrévete a pensar

1. **Sin ejecutar:** ¿qué imprime este programa?

```
public class Acertijo {
    static String url = "/api/tareas/3";

    public static void main(String[] args) {
        String id = url.substring("/api/tareas/".length());
        System.out.println(id);
        System.out.println(url.split("/").length);
    }
}
```

2. **El servidor que espía:** ¿qué cabecera pones para que el navegador no cachee tus respuestas JSON y siempre reciba datos frescos?
3. **El detective del 403:** consumes una API sin autenticarte y te devuelve `403`. ¿Qué cabecera probablemente te falta? ¿Y qué debes hacer para conseguirla?
4. **Verdadero o falso:** “un `GET` puede enviar datos en el cuerpo de la petición”.

> › 💡 **Soluciones**

🧩 Crucigrama de bits

```
Horizontal:
1. Formato de datos universal de la web, con llaves y corchetes (4 letras)
4. Servidor HTTP que Java trae de serie (11 letras)
6. Código de estado para "no encontrado" (3 letras)
7. Cabecera que dice el tipo de contenido (12 letras)

Vertical:
2. Método HTTP para enviar datos y crear recursos (4 letras)
3. Cliente HTTP moderno de Java 11+ (11 letras)
5. Formato de contenido de una página web (4 letras)
```

> 📝 **Soluciones**

💬 Preguntas de entrevista de trabajo

> Preguntas reales que te harían para programador Java junior.

1. **"Explícame, como si yo fuera tu abuela, qué es una API REST."**
2. **"¿Cuál es la diferencia entre GET y POST? ¿Cuándo usarías cada uno?"**
3. **"¿Qué significa el código 404? ¿Y el 500?"**
4. **"¿Cómo consumes una API externa en Java sin librerías de terceros?"**
5. **"¿Qué es Gson y para qué sirve? ¿Qué pasa si el JSON trae un campo que no está en tu `record`?"**
6. **"¿Qué es un `Content-Type` y por qué es importante?"**

🤷 No hay preguntas tontas

> ❓ **¿HTTP y HTTPS son lo mismo?**

Casi. HTTPS es HTTP con una capa de cifrado (SSL/TLS): los datos viajan encriptados. Para producción, siempre HTTPS; para desarrollo local, HTTP basta.

> ❓ **¿Qué es CORS y por qué me da error?**

Cross-Origin Resource Sharing. El navegador bloquea peticiones de un dominio a otro por seguridad. Para desarrollo, añade en tu servidor: `intercambio.getResponseHeaders().add("Access-Control-Allow-Origin", "*")`.

> ❓ **¿El `HttpServer` es como Tomcat?**

No. `HttpServer` es mínimo y didáctico: sirve para entender el protocolo. Tomcat es un servidor de aplicaciones completo (Servlets, JSP…). Aquí usamos lo justo para que la web deje de ser magia.

🎬 Poscréditos

La programadora termina su primera API: un gestor de tareas servido con `HttpServer`, con sus cuatro verbos REST y un frontend que hace `fetch`. Funciona. Hasta que abre el móvil y quiere consumirla desde fuera del ordenador, y descubre que el localhost solo vive en su máquina.

Se acerca CONRAD, el compilador cascarrabias, con su taza humeante.

CONRAD: — Eso es lo bonito de lo que acabas de construir: la API ya no depende de dónde se ejecuta. Hoy vive en tu portátil; mañana, en una nube. Los que la consumen solo ven rutas y JSON.

Programadora: — Vale… pero ahora mismo todo está en un `ArrayList` que se borra al reiniciar. ¿Cuándo aprendo a guardarlo de verdad?

CONRAD: — *asiente* Eso ya lo viste en la U15 con JDBC: cambia el almacén, no las rutas. Tu API no se entera. Pero si me preguntas a mí, ya has recorrido un camino enorme: empezaste con un `Hola, mundo` en una consola y acabas sirviendo y consumiendo web. Eso es todo el curso.

Programadora: — ¿Y ahora qué?

CONRAD: — *se ajusta las gafas* Ahora elige. Spring Boot, tu proyecto final, tu primera entrevista… El curso se acabó, pero la programación no: solo cambia de terminal.

🎓 **FIN DEL CURSO.** De `System.out.println("Hola, mundo")` a servir y consumir APIs en 14 unidades. Ha sido un placer programar contigo. ¡Sigue escribiendo código! 🌐

📚 Volver al índice de la unidad · **Anterior:** 08 · HttpClient avanzado · 🎓 **Fin del curso**

Boletín U01 — Inicial
📝 Inicial por resolver

> Entrena la mente de programador/a sin una sola línea de Java. No tienen nota ni cuentan para la evaluación, pero son la mejor manera de romper el hielo y compartir ideas en el foro.

Ejercicio 1: “Engaña a tu robot literal” 🤖🥪

- **El concepto:** Un ordenador no tiene sentido común; hace exactamente lo que le dices, no lo que tenías en la cabeza.
- **Cómo participar:**
  1. Elige una acción cotidiana (ej: *hacer un café de cápsula*, *ponerse un abrigo*, *Pedir una pizza por una app*).
  2. Escribe en el foro las instrucciones paso a paso (máximo 8 pasos) para un robot.
  3. **Modo gambero:** ¡Lee la propuesta de un/a compañer/a y haz de *Robot Malicioso*: encuentra un paso donde falte información implícita e indica la pifia literal que haría el robot!

Ejercicio 2: “Cazadores de bugs humanos” 🐛🎯

- **El concepto:** Los programadores profesionales prueban los “casos límite” para encontrar fallos antes de que el programa pueda petar.
- **El algoritmo a analizar:** Un compañero ha diseñado este sistema para **“Calcular si una persona puede subir a una atracción (Altura mínima: 140 cm)”**:

1. Pedir la altura en centímetros.
2. Si la altura es mayor que 140:
3. ```
   Mostrar: "¡Puedes subir!"
   ```
4. Si no:
5. ```
   Mostrar: "Eres muy bajito/a."
   ```

- **Cómo participar:** Responde en el foro contestando a estas preguntas:

- ¿Qué caso límite o dato “gamberro” haría fallar este algoritmo? *(Pista: ¿Qué pasa si mides 140 cm exactos? ¿Y si piensas un número negativo?)*.
- ¿Cómo lo arreglarías tú en una sola frase de texto normal?

Ejercicio 3: “Tu protocolo de paz” 🛡️💬

- **El concepto:** A distancia, el peligro más grande es el bloqueo y la frustración cuando un ejercicio no sale a la primera.
- **Cómo participar:** Comparte en un mensaje breve tus respuestas a estas dos preguntas:
  1. *¿Cuál es el concepto de esta Unidad 1 que más te ha sorprendido o cambiado el chip?*
  2. *¿Cuáles serán tus 3 pasos personales cuando te atasques en un ejercicio antes de pedir ayuda?*

💡 **Recuerda:** No hay respuestas “malas” ni formas de equivocarse. El foro está abierto para probar, reírse de los errores de pensamiento literal y ver que todos y todas estamos en el mismo barco.

Ejercicio 4: El filtro del hospital vs. el gimnasio (Abstracción)

- **El concepto:** Un ordenador solo tiene que guardar los datos que necesita. Guardar información inútil solo ocupa espacio y crea problemas.
- **El escenario:** Imagina que tienes que diseñar la ficha de un usuario para dos aplicaciones diferentes: la aplicación de un **Hospital** y la aplicación de un **Gimnasio**.
- **Cómo participar:** Clasifica los siguientes datos según si son **ÚTILES** o **IRRELEVANTES** para cada aplicación y compártelo en el foro: ***Datos**: Nombre, Grupo sanguíneo, Canción preferida, Alergias a medicamentos, Número de zapato, Objetivo de peso, Historial de fracturas.*

Ejercicio 5: El desglosador de promociones (Descomposición)

- **El concepto:** Si un problema parece muy grande o embrollado, hay que dividirlo en piezas microscópicas.
- **El escenario:** Una tienda online quiere aplicar la siguiente oferta: *"Si compras 3 productos o más, se te aplica un 10% de descuento. Si además eres cliente VIP, se te aplica un 5% adicional."*
- **Cómo participar:** Escribe en el foro la lista de preguntas (en orden) que debería hacer el programa para saber qué mejora aplicar. *(**Pista**: 1. ¿Cuántos productos lleva? 2. ¿Es VIP?…)*

Ejercicio 6: El test de la receta tapada (Metacognición)

- **El concepto:** Comprobar por ti mismo/a la diferencia entre "ver cómo se hace" y "saber hacerlo tú solo".
- **Cómo participar:**
  1. Lee las instrucciones de cómo hacer un nudo de corbata, una figura de papiroflexia sencilla o una receta de cocina rápida.
  2. Cierra la página.
  3. Intenta escribir en un papel todos los pasos de memoria sin mirar.
  4. Comparte en el foro: *¿En qué paso te has quedado atascado al cerrar la muestra? ¿Qué has aprendido de esta experiencia para cuando estudies Java?*

Ejercicio 7: El Patrón del Cajero y el parking (Patrones)

- **El concepto:** En programación no se reinventa la rueda: se buscan cosas que funcionan igual para usar la misma lógica.
- **El escenario:** Piensa en cómo funciona un **cajero automático cuando sacas dinero** y cómo funciona la **barrera de un parking cuando pagas el tiquete**.
- **Cómo participar:** Escribe en el foro cuál es la condición que tienen en común estas dos máquinas para dejarte salir o darte el dinero. *(Pista: Ambas comprueban si "el importe introduído es mayor o igual a…").*

Ejercicio 8: Mi manual para aprender a programar

Después de haber trabajado los conceptos clave de la Unidad 1, no queremos que te vayas a la Unidad 2 sin un plan de acción personalizado.

Esta tarea se divide en dos partes muy simples:

1. Parte 1: Tu “Hoja de Ruta” personal para gestionar el bloqueo, los errores y la IA durante el curso.
2. Parte 2: Un mini-reto práctico de pensamiento lógico para poner a prueba tu capacidad de descomposición y testing (sin tocar ordenador).

📝 PARTE 1: Mi manual de navegación (Reflexión)

Completa las siguientes 5 frases con tu reflexión personal. Sé sincero/a: no hay respuestas “correctas” de libro, sino tu compromiso real sobre cómo vas a estudiar a distancia.

1. Frente a la pantalla en blanco…

Cuando me quede bloqueado/a frente a un ejercicio nuevo, lo primero que haré antes de tocar el teclado será…

(Pista: Piensa en la regla del papel, la descomposición o el dibujo de la lógica).

2. Mi relación con la solución de los ejercicios…

Cuando revise un ejemplo de código resuelto en los apuntes o en un vídeo, para asegurarme de no haber caído en el “espejo de la comprensión”, yo…

(**Pista**: Recuerda la regla de la pantalla tapada).

3. Mi protocolo frente al “texto rojo” (Errores)…

Si mi programa me da un error al ejecutarse, seguiré estos 3 pasos en orden antes de pedir ayuda en el foro:

- Paso 1:
- Paso 2:
- Paso 3:

4. Mi compromiso con la inteligencia artificial…

Me haya encontrado o no con la IA (ChatGPT, Gemini, etc.), me comprometo a utilizarla principalmente para... y NUNCA para...

(**Pista**: Recuerda el uso de la IA como tutor socrático vs. esclavo de copy-paste).

5. Mi hallazgo sobre aprender a programar...

Una cosa importante que he descubierto en esta Unidad 1 sobre cómo aprende mi cerebro a programar es...

🧩 PARTE 2: Mini-Reto de Lógica (Sin Java)

Lee atentamente el enunciado de este problema del mundo real y resuelve los 3 apartados siguientes utilizando tu pensamiento lógico.

📌 Enunciado del Mini-Reto:

"Un cine de verano quiere automatizar la venta de entradas. El precio base de la entrada es de 8 €. Sin embargo, si el cliente tiene menos de 12 años o más de 65 años, se le aplica un descuento del 50%. Además, si es martes (el día del espectador), TODOS tienen un descuento directo de 2 € sobre el precio que le corresponda."

✂️ 1. Descomponer el problema

Antes de pensar en la solución, escribe qué datos de entrada (Inputs) necesita pedir el programa al usuario para poder hacer los cálculos.

👣 2. Escribir el algoritmo paso a paso

Escribe en tu propio idioma (pseudocódigo) la secuencia de instrucciones ordenadas que debería seguir el programa para calcular el precio final de la entrada.

(**Pista**: Piensa en qué orden se han de comprobar la edad y el día de la semana).

🧪 3. Control de calidad (Probar casos límite)

Un buen/a programador/a comprueba si el programa falla en los puntos críticos. Indica cuál debería ser el precio final de la entrada en estos 4 casos de prueba:

- Caso A: Un crío de 10 años un jueves ⟶ Precio final: ____ €
- Caso B: Un adulto de 30 años un martes ⟶ Precio final: ____ €

- Caso C (Límite 1): Una persona de 12 años exactos un miércoles ⟶ Precio final: ____ (¡Atención: ¿Tiene descuento del 50% o no? ¡Revisa bien el enunciado!)

- Caso D (Límite 2): Un abuelo de 70 años un martes ⟶ Precio final: ____ (¡Atención: ¿Se le aplican ambos descuentos? ¿Cuál es el resultado?)

Puedes compartir tus soluciones en el foro 😀

Boletín U02 — Inicial

📝 Boletín U02 — Inicial

> Sin soluciones. Sin prisas. Con un editor de texto y muchas ganas de compilar. Esto solo empieza. Comenta las soluciones en el foro e intenta ayudar a tus compañeros a que mejoren sus capacidades como programador.

Ejercicio 1: Desordena esto

Las líneas de este programa están desordenadas. Ordénalas para formar un programa Java válido que compile y se ejecute:

```
}
    public static void main(String[] args) {
        System.out.println("Mi primer programa ordenado");
public class Ordenado {
    }
```

Ejercicio 2: ¿Qué imprime?

Sin ejecutar, escribe la salida exacta de este programa:

```
public class Escapista {
    public static void main(String[] args) {
        System.out.print("Dijo: \"Java mola\"");
        System.out.println(" y siguió: \tprogramando.");
    }
}
```

💡 **Pista importante**: `\"` imprime una comilla doble literal, `\t` es un tabulador.

Ejercicio 3: Cazador de errores

Este código tiene **4 errores**. Encuéntralos y corrígelos:

```
Public class ErrorFinder {
    public static void main(string[] args) {
        System.out.println("Hola, "Mundo"");
        System.out.println("Esto funciona?)
    }
}
```

Ejercicio 4: Tu ficha personal

Escribe un programa llamado `FichaPersonal.java` que muestre el siguiente texto por pantalla:

```
Nombre: [Tu nombre]
Edad: [Tu edad]
Lenguaje favorito: Java
¿Emocionado?: true
```

Usa una línea `System.out.println` para cada campo y asegúrate de que los boleanos (como `true`) no llevan comillas.

Ejercicio 5: Completa el programa

Falta una llave de cierre y un carácter crucial. Añádelos para que compile y muestre “Aprobado, esto funciona”:

```
public class Completame {
    public static void main(String[] args) {
        System.out.println("Aprobado, esto funciona")
}
```

Ejercicio 6: Empareja conceptos

Relaciona cada concepto de la izquierda con su definición de la derecha:

Concepto	Definición
1. `class`	A. Punto de entrada del programa
2. `main`	B. Imprime texto y salta de línea
3. `System.out.println`	C. Define un nuevo tipo de datos o plantilla de código
4. `//`	D. Comentario de una línea
5. `args`	E. Contiene los argumentos de la línea de órdenes

Escribe las respuestas con el formato: 1→C, 2→A, ...

Ejercicio 7: El detective de errores

El siguiente código tiene un error de sintaxis que impide que compile. Encuéntralo y corrígelo:

```
public class Detective {
    public static void main(String[] args) {
        System.out.println("Soy un detective")
        System.out.println("y resuelvo errores");
    }
}
```

Escribe la versión corregida. Después, ejecútala en VS Code y comprueba que funciona.

Ejercicio 8: Tu biografía

Escribe un programa llamado `Biografia.java` que muestre:

- Tu nombre
- Tu edad
- Tu lenguaje de programación favorito
- Una frase que te motive

Cada dato tiene que salir en una línea diferente, pero con una única condición: usa una sola instrucción `System.out.println` utilizando `\n` para los saltos de línea.

Boletín U02 — Inicial Resuelto

📝 Boletín U02 — Inicial (Resuelto)

> Las soluciones están ocultas. Intenta de verdad antes de destaparlas. Compara tu código con estas propuestas de resolución y comparte tus dudas o alternativas en el foro.

Ejercicio 1: Desordena esto (Solución)

> 🔄 Solución

Ejercicio 2: ¿Qué imprime? (Solución)

> 🔄 Solución

Ejercicio 3: Cazador de errores (Solución)

> 🔄 Solución

Ejercicio 4: Tu ficha personal (Solución)

> 🔄 Solución

Ejercicio 5: Completa el programa (Solución)

> 🔄 Solución

Ejercicio 6: Empareja conceptos (Solución)

> 🔄 Solución

Ejercicio 7: El detective de errores (Solución)

> Solución

Ejercicio 8: Tu biografía (Solución)

> Solución

Boletín U02 — Avanzado

📝 Boletín U02 — Avanzado

> Sin soluciones. Sin prisas. Con un editor de texto y muchas ganas de compilar. Esto solo empieza. Comenta las soluciones en el foro e intenta ayudar a tus compañeros a que mejoren sus capacidades como programador.

Ejercicio 1: Calculadora de propinas

Escribe un programa que calcule cuánto dejar de propina en un restaurante. Declara:

- `double totalCuenta = 45.50;`
- `int porcentajePropina = 15;` (el porcentaje, sin el símbolo)

Calcula la propina (`totalCuenta * porcentajePropina / 100`) y el total final (`totalCuenta + propina`). Muestra los tres valores.

Pista: cuidado con la división: `porcentajePropina / 100` con enteros da 0. Multiplica primero y divide después, o usa `100.0`.

Ejercicio 2: Conversor dólar-euro

Declara `final double TASA_CAMBIO = 0.92;` (1 dólar = 0.92 euros). Declara `double dolares = 100.0;` y calcula su equivalente en euros. También haz la conversión inversa: dado `double euros = 50.0;`, calcula cuántos dólares son.

Muestra:

```
100.0$ son 92.0€
50.0€ son 54.347826086956516$
```

Pista: para pasar de euros a dólares divides entre la tasa: `euros / TASA_CAMBIO`.

Ejercicio 3: ¿Qué imprime? — el casting traidor

Sin ejecutar, escribe la salida exacta:

```
public class CastingTraidor {
   public static void main(String[] args) {
       int a = 7;
       int b = 2;
       double resultado1 = a / b;
       double resultado2 = (double) a / b;
       double resultado3 = a / (double) b;

       System.out.println(resultado1);
       System.out.println(resultado2);
       System.out.println(resultado3);
       System.out.println(3 + 4 * 2.0);
       System.out.println((int) (3.7 + 2.3));
   }
}
```

Pista: fíjate bien en dónde está el casting y en qué momento se aplica la división entera. En `resultado1`, ¿la división se hace antes o después de guardar en el `double`?

Ejercicio 4: Interés compuesto (sin bucle)

Declara `final double CAPITAL_INICIAL = 1000.0;`, `final double TASA = 0.05;` (5% anual), `int años = 3;`. Calcula el capital final después de 3 años usando la fórmula del interés compuesto SIN bucles:

```
capitalFinal = capitalInicial * (1 + tasa)^años
```

Para la potencia usa `Math.pow(base, exponente)`. Muestra el capital año a año:

```
Año 0: 1000.0€
Año 1: 1050.0€
Año 2: 1102.5€
Año 3: 1157.625€
```

💡 **Cuidado con la precisión:** en el año 3 tu programa puede imprimir `1157.6250000000002` en lugar de `1157.625` por la coma flotante binaria de los `double` (lo veremos en «Atréverte a pensar»). No es un error de tu código.

Pista: para mostrar cada año sin bucle, crea tres variables distintas (`capital1`, `capital2`, `capital3`) y usa `Math.pow(1 + TASA, i)` con `i` valiendo 1, 2 y 3.

Ejercicio 5: El enigma del post-incremento

Sin ejecutar, determina el valor de cada variable después de ejecutar este código. Escribe el paso a paso:

```
public class EnigmaIncrement {
   public static void main(String[] args) {
   int x = 3;
   int y = x++ + ++x;
   int z = --y + y-- + x++;
   System.out.println("x = " + x);
   System.out.println("y = " + y);
   System.out.println("z = " + z);
   }
}
```

Pista: haz una tabla en un papel con los valores de x e y después de cada operación. x++ usa y después incrementa; ++x incrementa y después usa.

💡 Herramientas de ampliación (Operador ternario, .equals(), Scanner y Math.random())

⚠️ **Fuera de temario (de momento):** Estas herramientas las veremos en detalle y de manera oficial en próximas unidades, pero es útil saber cómo funcionan si quieres empezar a explorar opciones más avanzadas en tus programas.

1. El operador ternario (? :)

Es una forma ultracompacta de escribir una estructura condicional (if-else) sencilla en una sola línea. Sirve para elegir entre dos valores según si se cumple o no una condición.

Sintaxis: `condicion ? valor_si_verdadero : valor_si_falso`

Ejemplo:

```java
int edad = 20;
String estado = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
```

Si la condición (`edad >= 18`) es cierta, retorna “Mayor de edad”; si no, retorna “Menor de edad”.

2. Comparar texto con .equals()

En Java, las cadenas de texto (`String`) son objetos, no tipos primitivos como `int` o `double`. Por eso, nunca se debe utilizar `==` para comparar el contenido de dos textos, ya que `==` compara si están guardados en la misma posición de memoria, no si tienen las mismas letras.

Sintaxis: `cadena1.equals(cadena2)`

Ejemplo:

```java
String operador = "+";

// Incorrecto ❌ (puede fallar según cómo se haya creado el texto):
// if (operador == "+")

// Correcto ✅ (compara el contenido del texto letra por letra):
if (operador.equals("+")) {
    System.out.println("Es una suma");
}
```

3. Leer datos del teclado con Scanner y la importación (import)

Para leer lo que el usuario escribe en la terminal, utilizamos la clase `Scanner`. Como no viene cargada por defecto en el motor básico de Java, tenemos que decirle al programa dónde encontrarla mediante una instrucción `import` en la primera línea del archivo.

Importación obligatoria: `import java.util.Scanner;` (se pone antes de declarar la clase).

Métodos principales de lectura:

- `scanner.nextLine()`: Lee una línea de texto completa (`String`).
- `scanner.nextInt()`: Lee un número entero (`int`).
- `scanner.nextDouble()`: Lee un número decimal (`double`).

Ejemplo:

```
import java.util.Scanner; // 1. Importamos la clase de la biblioteca de Java

public class EjemploScanner {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in); // 2. Creamos el objeto Scanner

        System.out.print("Introduce tu nombre: ");
        String nombre = scanner.nextLine(); // 3. Leemos el texto introducido

        System.out.println("Hola, " + nombre + "!");
        scanner.close(); // Buenos hábitos: cerramos el Scanner al terminar
    }
}
```

4. Generar números aleatorios con Math.random()

La clase `Math` viene incluida automáticamente en Java (no necesita `import`). Su método `Math.random()` genera un número decimal aleatorio entre 0.0 (incluido) y 1.0 (excluido).

Cómo adaptarlo para obtener un número entero en un rango (ej: un dado de 1 a 6):

- Multiplicamos el resultado por el máximo del rango (`Math.random() * 6` da de 0.0 a 5.999...).
- Hacemos un casting a entero (`int`) para eliminar la parte decimal (0 a 5).
- Sumamos 1 para ajustar el rango final (1 a 6).

Ejemplo para simulación de dados:

```
// Genera un número entero aleatorio entre 1 y 6:
int dado = (int) (Math.random() * 6) + 1;
System.out.println("Has sacado un: " + dado);
```

Ejercicio 6: El duelo de dados (Extra)

Escribe un programa que lance un dado para el usuario (le pide el nombre con `Scanner`) y otro para la máquina (con `Math.random()`). Gana quien saque más. Muestra las dos tiradas y el

ganador con un mensaje tipo:

```
Anna saca 4, la máquina saca 6.
Gana la máquina.
```

Pista: combina `nextLine()` (nombre), `(int)(Math.random() * 6) + 1` (dados) y un ternario encadenado para el veredicto. Cuidado con el empate.

Ejercicio 7: Supercalculadora con lógica (Extra)

Escribe un programa que lea dos números con `Scanner` (`double`) y un carácter de operación (`+`, `-`, `*`, `/`) con `next()`.

Según el operador, muestra el resultado. Si el operador es `/` y el segundo número es 0, muestra "Error: no se puede dividir entre 0". Para cualquier otro carácter, muestra "Operador desconocido".

Pista: calcula los resultados posibles en variables y elige con ternarios encadenados o con lógica. Puedes comprobar la división entre 0 con `op.equals("/") && b == 0`.

Ejercicio 8: Javadoc de campeonato

Escribe una clase llamada `SobreMi.java` que muestre por pantalla tu nombre, tu motivación para programar y cuántos argumentos ha recibido por la línea de órdenes (`args.length`).

El programa tiene que incluir los tres tipos de comentarios de Java:

- Comentario de una línea (`//`): Explica por qué se imprime primero el nombre.
- Comentario de bloque (`/* ... */`): Un resumen breve de la función del programa.
- Comentarios Javadoc (`/** ... */`): Documentación formal para la clase y para el método main, utilizando las etiquetas `@author`, `@version` y `@param`.

Tarea extra: Ejecuta el comando `javadoc SobreMi.java` en la terminal (o desde las opciones de Java en VS Code) para generar la documentación en HTML y abre el archivo `index.html` en tu navegador.

Boletín U02 — Avanzado Resuelto

📝 Boletín U02 — Avanzado (Resuelto)

> Las soluciones están ocultas. Intenta de verdad antes de destaparlas. Compara tu código con estas propuestas de resolución y comparte tus dudas o alternativas en el foro.

Ejercicio 1: Calculadora de propinas (Solución)

> 🔄 Solución

Ejercicio 2: Conversor dólar-euro (Solución)

> 🔄 Solución

Ejercicio 3: ¿Qué imprime? — el casting traidor (Solución)

> 🔄 Solución

Ejercicio 4: Interés compuesto (sin bucle) (Solución)

> 🔄 Solución

Ejercicio 5: El enigma del post-incremento (Solución)

> 🔄 Solución

💡 Herramientas de ampliación (Operador ternario, .equals(), Scanner y Math.random())

⚠️ **Fuera de temario (de momento):** Estas herramientas las veremos en detalle y de manera oficial en próximas unidades, pero es útil saber cómo funcionan si quieres empezar a explorar opciones más avanzadas en tus programas.

1. El operador ternario (? :)

Es una forma ultracompacta de escribir una estructura condicional (if-else) sencilla en una sola línea. Sirve para elegir entre dos valores según si se cumple o no una condición.

Sintaxis: `condicion ? valor_si_verdadero : valor_si_falso`

Ejemplo:

```
int edad = 20;
String estado = (edad >= 18) ? "Mayor de edad" : "Menor de edad";
```

Si la condición (`edad >= 18`) es cierta, retorna “Mayor de edad”; si no, retorna “Menor de edad”.

2. Comparar texto con .equals()

En Java, las cadenas de texto (`String`) son objetos, no tipos primitivos como `int` o `double`. Por eso, nunca se debe utilizar `==` para comparar el contenido de dos textos, ya que `==` compara si están guardados en la misma posición de memoria, no si tienen las mismas letras.

Sintaxis: `cadena1.equals(cadena2)`

Ejemplo:

```
String operador = "+";

// Incorrecto ❌ (puede fallar según cómo se haya creado el texto):
// if (operador == "+")

// Correcto ✅ (compara el contenido del texto letra por letra):
if (operador.equals("+")) {
    System.out.println("Es una suma");
}
```

3. Leer datos del teclado con Scanner y la importación (import)

Para leer lo que el usuario escribe en la terminal, utilizamos la clase `Scanner`. Como no viene cargada por defecto en el motor básico de Java, tenemos que decirle al programa dónde encontrarla mediante una instrucción `import` en la primera línea del archivo.

Importación obligatoria: `import java.util.Scanner;` (se pone antes de declarar la clase).

Métodos principales de lectura:

- `scanner.nextLine()`: Lee una línea de texto completa (`String`).
- `scanner.nextInt()`: Lee un número entero (`int`).
- `scanner.nextDouble()`: Lee un número decimal (`double`).

Ejemplo:

```
import java.util.Scanner; // 1. Importamos la clase de la biblioteca de Java

public class EjemploScanner {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in); // 2. Creamos el objeto Scanner

        System.out.print("Introduce tu nombre: ");
        String nombre = scanner.nextLine(); // 3. Leemos el texto introducido

        System.out.println("Hola, " + nombre + "!");
        scanner.close(); // Buenos hábitos: cerramos el Scanner al terminar
    }
}
```

4. Generar números aleatorios con Math.random()

La clase `Math` viene incluida automáticamente en Java (no necesita `import`). Su método `Math.random()` genera un número decimal aleatorio entre 0.0 (incluido) y 1.0 (excluido).

Cómo adaptarlo para obtener un número entero en un rango (ej: un dado de 1 a 6):

- Multiplicamos el resultado por el máximo del rango (`Math.random() * 6` da de 0.0 a 5.999...).
- Hacemos un casting a entero (`int`) para eliminar la parte decimal (0 a 5).

- Sumamos 1 para ajustar el rango final (1 a 6).

Ejemplo para simulación de dados:

```
// Genera un número entero aleatorio entre 1 y 6:
int dado = (int) (Math.random() * 6) + 1;
System.out.println("Has sacado un: " + dado);
```

Ejercicio 6: El duelo de dados (Extra) (Solución)

> 🔄 Solución

Ejercicio 7: Supercalculadora con lógica (Extra) (Solución)

> 🔄 Solución

Ejercicio 8: Javadoc de campeonato (Solución)

> 🔄 Solución

Boletín U02 — Extras

📝 Boletín U02 — Extras, para incrementar portafolio

Actividades opcionales orientadas a resolver retos en plataformas competitivas de programación. Están diseñadas para aquellos alumnos que quieran ir un paso más allá, avanzar por su cuenta e ir creando su portafolio de proyectos en GitHub.

⚠️ **Nota importante (Fuera de temario por ahora):** Estas actividades son totalmente extras y voluntarias. Utilizan conceptos o estructuras que iremos viendo a lo largo del curso. Si ahora mismo no sabes cómo resolverlas todas, no te preocupes en absoluto: puedes guardarte el boletín y volver a ellos más adelante cuando hayamos avanzado en las próximas unidades.

💼 **Impacte en tu Currículum y Perfil Professional:** Plataformas como CodeWars y AceptaElReto te permiten mantener un perfil público con tus soluciones, puntuación y nivel alcanzado. Incluir los enlaces a estos perfiles en tu CV o en tu LinkedIn es una práctica muy valorada por las empresas del sector tecnológico, ya que demuestra de manera práctica tu capacidad de resolución de problemas, tu constancia y tu interés por mejorar continuamente tus habilidades de programación.

💡 **Consejo para tu Portafolio:** Crea un repositorio en GitHub llamado `java-challenges` o `code-solutions` y sube una carpeta organizada para cada uno de estos retos con su código fuente bien documentado.

Ejercicio 1: CodeWars — Square(n) Sum (Extra)

Resuelve la kata **Square(n) Sum** (8 kyu) en CodeWars.

Dado un array de números, eleva cada uno al cuadrado y suma los resultados.

- Ejemplo: `[1, 2, 2]` → $1^2 + 2^2 + 2^2 = 1 + 4 + 4 = 9$.

Ejercicio 2: CodeWars — Grasshopper - Summation (Extra)

Resuelve la kata **Grasshopper - Summation** (8 kyu) en CodeWars.

Escribe un programa que encuentre la suma de todos los números enteros desde 1 hasta n (ambos incluidos).

- Ejemplo: Si n = 4, el resultado debe ser 1 + 2 + 3 + 4 = 10.
- Pista: Hay una fórmula matemática conocida (`n * (n + 1) / 2`), pero si te atreves, también puedes resolverlo utilizando un bucle. El método recibirá un `int n` y deberá devolver un `int`.

Ejercicio 3: AceptaElReto — 119 Escudos del ejército romano (Extra)

Resuelve el problema **119 — Escudos del ejército romano** en la plataforma AceptaElReto.com.

Un ejército romano necesita organizar sus legionarios en formaciones cuadradas protegidas por escudos. Cada formación de k x k legionarios requiere escudos para el contorno y la cobertura superior. Determina cuántos escudos necesitan en total para organizar un número dado de soldados.

- Pista: Lee el número total de soldados para cada caso de prueba. La secuencia de entrada termina cuando se lee el valor 0 (marcador de fin).

Boletín U03 — Inicial

📝 Boletín U03 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, declara variables y ensucia tus manos de bits. Nadie nace sabiendo qué es un `double`.

Ejercicio 1: conversor de temperaturas

Escribe un programa llamado `ConversorTemperatura` que convierta 30 grados Celsius a Fahrenheit. Usa la fórmula:

```
F = C * 9/5 + 32
```

Declara `int celsius = 30` y una variable `double fahrenheit` para el resultado. Muestra ambas temperaturas.

Ejercicio 2: ¿Qué imprime? — incrementos

Sin ejecutar, escribe la salida exacta de este programa:

```java
public class Incrementos {
    public static void main(String[] args) {
        int x = 5;
        System.out.println(x++);
        System.out.println(++x);
        System.out.println(x--);
        System.out.println(--x);
        System.out.println(x);
    }
}
```

Pista: recuerda la diferencia entre pre y post. x++ usa y luego sube; ++x sube y luego usa.

Ejercicio 3: calculadora de descuentos

Declara una constante `final double DESCUENTO = 0.15` y declara `double precioOriginal = 120.0`. Calcula:

1. El descuento (`precioOriginal * DESCUENTO`)
2. El precio final (`precioOriginal - descuento`)

Muestra todo con `println`. Si luego intentas cambiar el valor de `DESCUENTO`, el compilador se enfadará. ¿Sabes por qué?

Ejercicio 4: El tipo perfecto

Indica qué tipo de dato primitivo (`int`, `double`, `boolean`, `char`, `long`) usarías para cada caso:

1. El número de habitantes de tu ciudad (~500.000)
2. La distancia en kilómetros hasta la Luna (~384.400)
3. La inicial de tu segundo apellido
4. La nota media de un examen (3.7)
5. Si has aprobado o no el examen anterior
6. El precio de un café en céntimos (entero)

Ejercicio 5: El casting asesino

Declara `double precio = 9.99;`. Conviértelo a `int` con un casting explícito. Imprime ambos valores. ¿Qué se pierde por el camino?

Después, declara `int grande = 300;` y conviértelo a `byte`. Imprime el resultado. ¿Te esperabas ese número? (Pista: en un `byte` solo caben -128 a 127.)

Ejercicio 6: ¿Par o impar?

Escribe un programa llamado `ParOImpar` que use el operador `%` para determinar si un número es par o impar. Puedes usar una variable `int numero = 7;` y el operador ternario para mostrar “Par” o “Impar”. Muestra también `true` o `false` al comprobar si `numero % 2 == 0`.

Ejercicio 7: String en acción

Escribe un programa llamado `NombreEnAccion` que, con la variable `String nombre = " ana ";`:

1. Limpie los espacios con `trim()`
2. Muestre la longitud del nombre limpio
3. Lo muestre en mayúsculas
4. Muestre la primera letra (con `substring(0, 1)`)

Ejercicio 8: saludo con Scanner

Escribe un programa llamado `SaludoScanner` que:

1. Pregunte "¿Cómo te llamas?" y lea el nombre con `nextLine()`
2. Pregunte "¿Cuántos años tienes?" y lea la edad con `nextInt()`
3. Muestre: "Hola, [nombre]. [edad] años, bienvenido."

Recuerda el `import java.util.Scanner;` y el `sc.close()`.

Ejercicio 9: CodeWars — Will you make it?

Resuelve la kata **"Will you make it?"** (8 kyu) en CodeWars.

Te dan la distancia hasta una gasolinera, los litros que tiene tu coche y los kilómetros por litro. Determina si llegas o no. Devuelve `true` si llegas, `false` si te quedas tirado.

Ejercicio 10: ¿Qué imprime? — printf con conversiones

Sin ejecutar, escribe la salida exacta de este programa:

```java
public class FormateoBasico {
    public static void main(String[] args) {
        int entero = 42;
        double decimal = 3.1416;
        String texto = "Java";

        System.out.printf("%d %f %s %n", entero, decimal, texto);
    }
}
```

¿Qué imprime? ¿Qué hace `%n` al final?

Pista: `%d` es para enteros, `%f` para decimales y `%s` para texto. ¿Cuántos decimales muestra `%f` cuando no le pones `%.2f`?

Ejercicio 11: la nota con dos decimales

Escribe un programa llamado `NotaFormateada` que declare `String nombre = "Marta"` y `double nota = 9.5678;`. Usa `String.format()` para construir este mensaje:

```
Marta ha sacado un 9.57.
```

Después muestra el mismo mensaje con `System.out.printf()`. ¿Qué diferencia hay entre las dos formas?

Pista: `String.format` devuelve un `String` y no imprime nada; `printf` escribe directamente en pantalla. Los dos usan `%.2f` para dejar dos decimales.

Ejercicio 12: la propina con dos decimales

Escribe un programa llamado `PropinaFormateada` que pida con `Scanner` el total de la cuenta (`double`) y el porcentaje de propina (`int`). Calcula la propina y el total final, y muéstralos con `System.out.printf` y dos decimales:

```
Total: 45.50 €
Propina (15%): 6.82 €
Total a pagar: 52.33 €
```

Pista: `%.2f` controla los decimales. Multiplica primero (`total * porcentaje`) y divide después con `100.0`, o `porcentaje / 100` dará 0.

Boletín U03 — Inicial Resuelto

📝 Boletín U03 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: conversor de temperaturas

> 🔄 Solución

Ejercicio 2: ¿Qué imprime? — incrementos

> 🔄 Solución

Ejercicio 3: calculadora de descuentos

> 🔄 Solución

Ejercicio 4: El tipo perfecto

> 🔄 Solución

Ejercicio 5: El casting asesino

> 🔄 Solución

Ejercicio 6: ¿Par o impar?

> 🔄 Solución

Ejercicio 7: String en acción

> Solución

Ejercicio 8: saludo con Scanner

> Solución

Ejercicio 9: CodeWars — Will you make it?

> Solución

Ejercicio 10: ¿Qué imprime? — printf con conversiones

> Solución

Ejercicio 11: la nota con dos decimales

> Solución

Ejercicio 12: la propina con dos decimales

> Solución

Boletín U03 — Avanzado

📝 Boletín U03 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: calculadora de propinas

Escribe un programa que calcule cuánto dejar de propina en un restaurante. Declara:

- `double totalCuenta = 45.50;`
- `int porcentajePropina = 15;` (el porcentaje, sin el símbolo)

Calcula la propina (`totalCuenta * porcentajePropina / 100`) y el total final (`totalCuenta + propina`). Muestra los tres valores.

Pista: cuidado con la división: `porcentajePropina / 100` con enteros da 0. Multiplica primero y divide después, o usa `100.0`.

⭐ Ejercicio 2: conversor dólar-euro

Declara `final double TASA_CAMBIO = 0.92;` (1 dólar = 0.92 euros). Declara `double dolares = 100.0;` y calcula su equivalente en euros. También haz la conversión inversa: dado `double euros = 50.0;`, calcula cuántos dólares son.

Muestra:

```
100.0$ son 92.0€
50.0€ son 54.347826086956516$
```

Pista: para pasar de euros a dólares divides entre la tasa: `euros / TASA_CAMBIO`.

⭐ ⭐ Ejercicio 3: ¿Qué imprime? — el casting traidor

Sin ejecutar, escribe la salida exacta:

```
public class CastingTraidor {
    public static void main(String[] args) {
        int a = 7;
        int b = 2;
        double resultado1 = a / b;
        double resultado2 = (double) a / b;
        double resultado3 = a / (double) b;

        System.out.println(resultado1);
        System.out.println(resultado2);
        System.out.println(resultado3);
        System.out.println(3 + 4 * 2.0);
        System.out.println((int) (3.7 + 2.3));
    }
}
```

Pista: fíjate bien en dónde está el casting y en qué momento se aplica la división entera. En `resultado1`, ¿la división se hace antes o después de guardar en el `double`?

⭐ ⭐ Ejercicio 4: interés compuesto (sin bucle)

Declara `final double CAPITAL_INICIAL = 1000.0;`, `final double TASA = 0.05;` (5% anual) e `int anios = 3;`. Calcula el capital final después de 3 años usando la fórmula del interés compuesto SIN bucles:

```
capitalFinal = capitalInicial * (1 + tasa)^anios
```

Para la potencia usa `Math.pow(base, exponente)`. Muestra el capital año a año:

```
Año 0: 1000.0€
Año 1: 1050.0€
Año 2: 1102.5€
Año 3: 1157.625€
```

> 💡 **Ojo con la precisión:** en el año 3 tu programa puede imprimir `1157.6250000000002` en vez de `1157.625` por la coma flotante binaria de los `double` (lo veremos en «Atrévete a pensar»). No es un error de tu código.

Pista: para mostrar cada año sin bucle, crea tres variables distintas (`capital1`, `capital2`, `capital3`) y usa `Math.pow(1 + TASA, i)` con `i` valiendo 1, 2 y 3.

⭐ ⭐ ⭐ Ejercicio 5: El enigma del post-incremento

Sin ejecutar, determina el valor de cada variable después de ejecutar este código. Escribe el paso a paso:

```
public class EnigmaIncremento {
    public static void main(String[] args) {
        int x = 3;
        int y = x++ + ++x;
        int z = --y + y-- + x++;
        System.out.println("x = " + x);
        System.out.println("y = " + y);
        System.out.println("z = " + z);
    }
}
```

Pista: haz una tabla con los valores de `x` e `y` después de cada operación. `x++` usa y luego incrementa; `++x` incrementa y luego usa.

⭐ ⭐ Ejercicio 6: El duelo de dados

Escribe un programa que lance un dado para el usuario (le pide su nombre con `Scanner`) y otro para la máquina (con `Math.random()`). Gana quien saque más. Muestra ambas tiradas y el ganador con un mensaje tipo:

```
Ana saca 4, la máquina saca 6.
Gana la máquina.
```

Pista: combina `nextLine()` (nombre), `(int)(Math.random() * 6) + 1` (dados) y un ternario encadenado para el veredicto. Cuidado con el empate.

⭐ ⭐ Ejercicio 7: supercalculadora con lógica

Escribe un programa que lea dos números con `Scanner` (`double`) y un carácter de operación (`+`, `-`, `*`, `/`) con `next()`.

Según el operador, muestra el resultado. Si el operador es `/` y el segundo número es 0, muestra "Error: no se puede dividir entre 0". Para cualquier otro carácter, muestra "Operador desconocido".

Pista: calcula los resultados posibles en variables y elige con ternarios encadenados o con lógica. Puedes comprobar la división entre 0 con `(op.equals("/") && b == 0)`.

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — Convert boolean values to strings 'Yes' or 'No'

Resuelve la kata **"Convert boolean values to strings 'Yes' or 'No'"** (8 kyu) en CodeWars.

Completa el método `public static String boolToWord(boolean b)` que devuelva `"Yes"` si recibe `true` y `"No"` si recibe `false`.

Pista: se puede hacer en una sola línea con el operador ternario: `b ? "Yes" : "No"`.

⭐ ⭐ ⭐ Ejercicio 9: AceptaElReto — 114 Último dígito del factorial

Resuelve el problema **114 — Último dígito del factorial** en AceptaElReto.com.

Dado un número N (0 ≤ N ≤ 1.000.000), calcula el último dígito de N! (factorial de N).

Pista: no necesitas calcular el factorial entero. Observa que 5! = 120, 6! = 720, 7! = 5040… a partir de 5, el factorial siempre termina en 0. Solo necesitas manejar los casos pequeños (0 a 4) y los grandes con una sola regla.

⭐ ⭐ Ejercicio 10: el ticket de compra con NumberFormat

Escribe un programa llamado `TicketCompra` que simule un ticket de compra con tres productos (Pan, Leche y Huevos, cada uno con su precio y su cantidad). Muestra el ticket usando `NumberFormat` con moneda y locale español para los precios:

```
=========================
    TICKET DE COMPRA
=========================
Pan    2 x 1,20 € = 2,40 €
Leche  3 x 0,95 € = 2,85 €
Huevos 1 x 3,50 € = 3,50 €
-------------------------
TOTAL               = 8,75 €
=========================
```

Requisitos: alinear los nombres a la izquierda, dos decimales en los precios y formatear todos los importes con `NumberFormat.getCurrencyInstance(new Locale("es", "ES"))`.

Pista: `NumberFormat` formatea un `double` como moneda con el separador de tu idioma (`1.234,56 €`). Para cada producto calcula el subtotal (`precio * cantidad`) y súmalo al total. El `Locale("es", "ES")` le dice "habla como en España": coma para los decimales y símbolo €.

⭐ ⭐ ⭐ Ejercicio 11: la edad a prueba de bombas

Escribe un programa llamado `EdadSegura` que pida la edad por teclado con `Scanner` y **la repita hasta que el usuario escriba un número entero**. Si el usuario escribe letras o un decimal, el programa debe avisar con "Eso no es un número entero." y volver a preguntar sin romperse (nada de `InputMismatchException`).

Cuando por fin consiga un entero, muestra con `printf`:

```
Genial, 20 años y listo para programar.
```

Pista: antes de cada `nextInt()`, pregunta con `sc.hasNextInt()`. Si devuelve `false`, descarta la basura con `sc.next()` y repite. Recuerda: `hasNextInt()` **mira** el siguiente dato sin consumirlo; si no lo descartas, se quedará ahí para siempre.

📚 Referencias

Plataforma	Problema	Dificultad
AceptaElReto	114 — Último dígito del factorial	Fácil
AceptaElReto	148 — Nochevieja	Fácil
AceptaElReto	217 — ¿Qué lado de la calle?	Fácil
CodeWars	Will you make it? (8 kyu)	Principiante
CodeWars	Convert boolean to Yes/No (8 kyu)	Principiante
CodeWars	Keep Hydrated (8 kyu)	Principiante
CodeWars	Get the Middle Character (7 kyu)	Intermedio

Boletín U03 — Avanzado Resuelto
📝 Boletín U03 — Avanzado (Resuelto)

> Las soluciones están ocultas. Inténtalo de verdad antes de destaparlas.

⭐ Ejercicio 1: calculadora de propinas

> 🔄 Solución

⭐ Ejercicio 2: conversor dólar-euro

> 🔄 Solución

⭐ ⭐ Ejercicio 3: ¿Qué imprime? — el casting traidor

> 🔄 Solución

⭐ ⭐ Ejercicio 4: interés compuesto (sin bucle)

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 5: El enigma del post-incremento

> 🔄 Solución

⭐ ⭐ Ejercicio 6: El duelo de dados

> 🔄 Solución

⭐ ⭐ Ejercicio 7: supercalculadora con lógica

> 🔁 Solución

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — Convert boolean values to strings ‘Yes’ or ‘No’

> 🔁 Solución

⭐ ⭐ ⭐ Ejercicio 9: AceptaElReto — 114 Último dígito del factorial

> 🔁 Solución

⭐ ⭐ Ejercicio 10: el ticket de compra con NumberFormat

> 🔁 Solución

⭐ ⭐ ⭐ Ejercicio 11: la edad a prueba de bombas

> 🔁 Solución

📚 Referencias

Plataforma	Problema	Dificultad
AceptaElReto	114 — Último dígito del factorial	Fácil
AceptaElReto	148 — Nochevieja	Fácil

Plataforma	Problema	Dificultad
AceptaElReto	217 — ¿Qué lado de la calle?	Fácil
CodeWars	Will you make it? (8 kyu)	Principiante
CodeWars	Convert boolean to Yes/No (8 kyu)	Principiante
CodeWars	Keep Hydrated (8 kyu)	Principiante
CodeWars	Get the Middle Character (7 kyu)	Intermedio

Boletín U03 — Extras

📝 Boletín U03 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. La solución está oculta: resístete hasta agotar tu pista.

CodeWars

1. Keep Hydrated!

Nathan bebe agua, pero no siempre suficiente. Le han dado `time` en horas y quiere saber cuántos litros de agua debe beber. La regla: bebe 0.5 litros por hora. Devuelve el número de litros **redondeado hacia abajo**.

Ejemplo: con `time = 6.7`, bebe `3` litros (3.35 → 3).

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: multiplica `time * 0.5` y trunca el resultado con `(int)`. El truncamiento y el "redondeo hacia abajo" son lo mismo aquí… por una vez.

> › 🔄 **Solución**

---

2. Opposite number

Escribe una función que reciba un número y devuelva su opuesto (el mismo número con el signo cambiado).

Ejemplos: `1` → `-1`, `14` → `-14`, `-34` → `34`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: el operador unario `-` invierte el signo de cualquier número: `-numero`. O multiplícalo por `-1`.

> › 🔄 **Solución**

---

3. MakeUpperCase

Escribe una función que convierta un String a mayúsculas.

Ejemplo: `"hello"` → `"HELLO"`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: ¿recuerdas el método de `String` del punto 8 que pone todo en mayúsculas? Es la primera herramienta de la caja.

> 🔄 **Solución**

---

4. Count of positives / sum of negatives

Dado un array de enteros, devuelve un array de dos elementos: el **número de números positivos** en el primer hueco y la **suma de los negativos** en el segundo. Si el array está vacío, devuelve un array vacío.

Ejemplo: `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10, -11, -12, -13, -14, -15]` → `[10, -65]`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

Pista: no hace falta ni bucles: declara `int[] resultado = new int[2];` y usa un bucle `for...each` (ya lo has visto en clase) para contar positivos y sumar negativos con `if`. Aquí lo importante es recordar cómo acceder a los huecos de un array.

> 🔄 **Solución**

---

5. Get the Middle Character

Dada una palabra, devuelve el carácter (o los dos caracteres) del centro.

Ejemplos: `"test"` → `"es"`, `"testing"` → `"t"`, `"middle"` → `"dd"`, `"A"` → `"A"`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

Pista: usa `length()` y `substring()` del punto 9. Si la longitud es par, devuelve los dos caracteres centrales; si es impar, solo uno.

> 🔄 Solución

AceptaElReto

6. 148 — Nochevieja

El 31 de diciembre, los más impacientes miran el reloj y cuentan cuántos segundos faltan para las 12 de la noche. Dado un instante con formato `HH:MM:SS`, calcula los segundos que faltan para medianoche.

Entrada: un número con el total de casos, y en cada línea una hora en formato `HH:MM:SS` (con dos dígitos por campo, pero los ceros a la izquierda pueden omitirse).

Ejemplo:

```
3
00:00:00
23:59:59
05:30:00
```

Salida:

```
86400
1
66600
```

- Enunciado en AceptaElReto
- Dificultad: Fácil

Pista: el día tiene 86400 segundos. Convierte la hora a segundos totales (`hora * 3600 + minuto * 60 + segundo`) y réstalo de 86400. Para separar los campos, lee con `nextInt()` y descarta los dos puntos con `next()`, o usa `Scanner.useDelimiter(":")`. El truco está en `%02d`… o en leer directamente.

> 🔄 Solución

7. 217 — ¿Qué lado de la calle?

En la calle principal de Doña Lita las casas se distribuyen en dos lados: los números impares van en un lado y los pares en el otro. Dado un número de casa, di en qué lado está.

Entrada: el número 0 indica el fin de la entrada (no hay que procesarlo). Para cada número de casa (positivo), muestra:

```
IZQUIERDA
```

o

```
DERECHA
```

según corresponda. El criterio es simple: un número de casa es `IZQUIERDA` si es par y `DERECHA` si es impar (o al revés, según la calle; lee bien el enunciado original).

- Enunciado en AceptaElReto
- Dificultad: Fácil

Pista: ¿qué operador te dice si un número es par o impar? El `%` del punto 3. El bucle termina cuando lees un 0, con un `while`.

> 🔄 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines las estructuras de control (U04) podrás volver a estos problemas con `for` y `switch` para hacer soluciones más elegantes. El material no se pierde: se reutiliza.

Boletín U04 — Inicial

📝 Boletín U04 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, enciende el semáforo y haz que tu código decida, repita y sobreviva. Nadie nace sabiendo cuándo poner un `break`.

Ejercicio 1: El portero del club

Escribe un programa llamado `Portero` que, con las variables `int edad = 17` y `boolean tieneEntrada = true`, decida con `if/else`:

- Si `edad >= 18` **Y** tiene entrada → "Adelante, que pase el mayor de edad."
- Si es mayor de edad pero **no** tiene entrada → "Faltas tú solito, sin entrada no hay paraíso."
- Si es menor → "Fuera de aquí, pequeñín."

Ejercicio 2: ¿Qué imprime? — el semáforo invertido

Sin ejecutar, escribe la salida exacta de este programa:

```
public class SemaforoInvertido {
    public static void main(String[] args) {
        int nota = 8;

        if (nota >= 5) {
            System.out.println("Aprobado");
        } else if (nota >= 7) {
            System.out.println("Notable");
        } else if (nota >= 9) {
            System.out.println("Sobresaliente");
        } else {
            System.out.println("Suspenso");
        }
    }
}
```

Pista: recuerda que gana el **primer** `if` que se cumple, aunque haya otros más "precisos" después.

Ejercicio 3: El menú del día

Escribe un programa llamado `MenuDia` que, con `int dia = 4`, muestre con `switch` el menú correspondiente:

- `1` → Lunes: Lentejas
- `2` → Martes: Paella
- `3` → Miércoles: Macarrones
- `4` → Jueves: Fabada
- `5` → Viernes: Pescado
- Cualquier otro número → `default`: Fin de semana, no hay menú

No te olvides del `break` en cada `case` ni del `default`.

Ejercicio 4: La cuenta atrás del cohete

Escribe un programa llamado `Cohete` que cuente desde 5 hasta 0 con un `while` y luego muestre "¡Despegue! 🚀". Muestra un número por línea.

Pista: `int contador = 5;` y `while (contador >= 0) { ... contador--; }`.

Ejercicio 5: La tabla del 7

Escribe un programa llamado `TablaDelSiete` que muestre la tabla de multiplicar del 7 (del 1 al 10) usando un `for`:

```
7 x 1 = 7
7 x 2 = 14
...
```

Ejercicio 6: Solo los pares

Escribe un programa llamado `SoloPares` que imprima los números pares del 2 al 20 usando un `for` y `continue`. Cada número en su línea.

Pista: recorre del 1 al 20 y usa `continue` para saltarte los impares (`if (i % 2 != 0) continue;`).

Ejercicio 7: El detective de divisores

Escribe un programa llamado `DetectiveDivisores` que compruebe con `for` y `break` si el número `int numero = 29` tiene algún divisor entre 2 y `numero - 1`. Si lo encuentra, muestra "No es primo" y sale; si no, muestra "Es primo".

Pista: recorre los divisores y, en cuanto encuentres uno (`numero % divisor == 0`), haz `break`.

Ejercicio 8: La edad blindada

Escribe un programa llamado `EdadBlindada` que pida la edad con `Scanner.nextInt()` dentro de un `try/catch`. Si el usuario escribe algo que no es un número, atrapa el `InputMismatchException` y muestra "Eso no es una edad válida.". Si lo es, muestra "Tienes X años."

Pista: necesitas `import java.util.Scanner;` e `import java.util.InputMismatchException;`. Después del `catch`, el programa debe seguir vivo.

Ejercicio 9: CodeWars — Even or Odd

Resuelve la kata **"Even or Odd"** (8 kyu) en CodeWars.

Crea el método `public static String even_or_odd(int number)` que devuelva `"Even"` si el número es par y `"Odd"` si es impar.

Pista: el operador `%` del punto 3 de la UO3 es tu mejor amigo: `number % 2 == 0`.

Boletín U04 — Inicial Resuelto

📝 Boletín U04 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: El portero del club

> 🔄 Solución

Ejercicio 2: ¿Qué imprime? — el semáforo invertido

> 🔄 Solución

Ejercicio 3: El menú del día

> 🔄 Solución

Ejercicio 4: La cuenta atrás del cohete

> 🔄 Solución

Ejercicio 5: La tabla del 7

> 🔄 Solución

Ejercicio 6: Solo los pares

> 🔄 Solución

Ejercicio 7: El detective de divisores

> Solución

Ejercicio 8: La edad blindada

> Solución

Ejercicio 9: CodeWars — Even or Odd

> Solución

Boletín U04 — Avanzado

📝 Boletín U04 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: La calculadora de notas

Escribe un programa que convierta una nota numérica (`double nota`) en su calificación textual usando `if/else if`:

- `>= 9` → "Sobresaliente"
- `>= 7` → "Notable"
- `>= 5` → "Aprobado"
- `>= 0` → "Suspenso"
- Cualquier otro valor (negativo o mayor que 10) → "Nota inválida"

Usa `double nota = 8.7;` y muestra el resultado. Fíjate en el orden: el caso inválido debe comprobarse antes que los rangos.

Pista: un `if` inicial para el caso inválido (`nota < 0 || nota > 10`) y después la cascada de rangos de arriba abajo.

⭐ Ejercicio 2: El menú que no se rinde

Escribe un programa que muestre un menú de dos opciones (1. Jugar, 2. Salir) con `do-while` y repita la pregunta hasta que el usuario escriba 1 o 2. Usa `Scanner`. Al final muestra "Has elegido la opción X.".

Pista: `do { ... } while (opcion != 1 && opcion != 2);`. El `do-while` garantiza que el menú se muestre al menos una vez.

⭐ ⭐ Ejercicio 3: ¿Qué imprime? — la pirámide

Sin ejecutar, escribe la salida exacta:

```
public class Piramide {
    public static void main(String[] args) {
        for (int fila = 1; fila <= 4; fila++) {
            for (int col = 1; col <= fila; col++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

Pista: el bucle interior depende de `fila`: la fila 1 imprime 1 asterisco, la fila 2 imprime 2… ¿Cuántos asteriscos en total?

⭐ ⭐ Ejercicio 4: La escalera de números

Escribe un programa que imprima esta escalera de números (bucles anidados):

```
1
1 2
1 2 3
1 2 3 4
```

Pista: bucle exterior del 1 al 4 (las filas) y bucle interior que imprime del 1 al número de fila, con `print` para no saltar de línea y un `println()` vacío al final de cada fila.

⭐ ⭐ ⭐ Ejercicio 5: ¿Qué imprime? — break, continue y la etiqueta

Sin ejecutar, escribe la salida exacta:

```java
public class Escapista {
    public static void main(String[] args) {
        exterior:
        for (int i = 1; i <= 4; i++) {
            for (int j = 1; j <= 3; j++) {
                if (j == 2 && i >= 3) {
                    continue exterior;
                }
                if (i * j >= 8) {
                    break exterior;
                }
                System.out.print(i + "" + j + " ");
            }
        }
    }
}
```

Pista: haz una tabla de pares `(i, j)`. El `continue exterior` salta a la siguiente `i`; el `break exterior` apaga todos los bucles. Comprueba par a par hasta dónde llega.

⭐ ⭐ Ejercicio 6: cazador de primos

Escribe un programa que imprima todos los números primos del 1 al 50, cada uno en su línea. Reutiliza la lógica del detective de divisores (`for` + `break`) dentro de otro `for`.

Pista: anida dos bucles: uno que recorra del 1 al 50 y otro interior que busque divisores. Usa un booleano `esPrimo` que se ponga a `false` con `break` si aparece un divisor.

⭐ ⭐ Ejercicio 7: La suma centinela

Escribe un programa que sume números enteros introducidos por el usuario hasta que escriba `0`. Muestra la suma final. Usa `while` y un centinela.

Pista: `while (numero != 0) { suma += numero; numero = sc.nextInt(); }`. El 0 no se suma: es la señal de parada.

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — Categorize New Member

Resuelve la kata **"Categorize New Member"** (7 kyu) en CodeWars.

Completa el método `public static String[] openOrSenior(int[][] data)` que recibe pares `{edad, handicap}` y devuelve `"Senior"` si el miembro tiene al menos 55 años Y un handicap mayor que 7; si no, `"Open"`.

Pista: recorre el array con un `for` y decide cada caso con una condición combinada (`&&`): `data[i][0] >= 55 && data[i][1] > 7`.

⭐ ⭐ ⭐ Ejercicio 9: AceptaElReto — 156 Ascensor

Resuelve el problema **156 — Ascensor** en AceptaElReto.com.

Un ascensor parte de la planta 0. Dado un número de visitas y las plantas de cada visita, calcula la distancia total recorrida. El valor `0` indica el fin de la entrada (no se procesa). Usa `Math.abs()` para las distancias.

Ejemplo:

```
5
5 1 10 4 2
0
```

Distancia: |0-5| + |5-1| + |1-10| + |10-4| + |4-2| = 5 + 4 + 9 + 6 + 2 = **26**.

Pista: un `while` que lea el número de visitas y rompa con `break` si es 0; dentro, un `for` que acumule `Math.abs(piso - pisoActual)` y actualice `pisoActual`.

📚 Referencias

Plataforma	Problema	Dificultad
AceptaElReto	156 — Ascensor	Fácil

Plataforma	Problema	Dificultad
AceptaElReto	149 — San Fermines	Fácil
AceptaElReto	340 — Siguiente con mismo número de cifras	Medio
CodeWars	Even or Odd (8 kyu)	Principiante
CodeWars	Categorize New Member (7 kyu)	Aficionado
CodeWars	Return Negative (8 kyu)	Principiante

Boletín U04 — Avanzado Resuelto

📝 Boletín U04 — Avanzado (Resuelto)

> Las soluciones están ocultas. Inténtalo de verdad antes de destaparlas.

⭐ Ejercicio 1: La calculadora de notas

> 🔄 Solución

⭐ Ejercicio 2: El menú que no se rinde

> 🔄 Solución

⭐ ⭐ Ejercicio 3: ¿Qué imprime? — la pirámide

> 🔄 Solución

⭐ ⭐ Ejercicio 4: La escalera de números

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 5: ¿Qué imprime? — break, continue y la etiqueta

> 🔄 Solución

⭐ ⭐ Ejercicio 6: cazador de primos

> 🔄 Solución

⭐ ⭐ Ejercicio 7: La suma centinela

> 🔁 Solución

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — Categorize New Member

> 🔁 Solución

⭐ ⭐ ⭐ Ejercicio 9: AceptaElReto — 156 Ascensor

> 🔁 Solución

📚 Referencias

Plataforma	Problema	Dificultad
AceptaElReto	156 — Ascensor	Fácil
AceptaElReto	149 — San Fermines	Fácil
AceptaElReto	340 — Siguiente con mismo número de cifras	Medio
CodeWars	Even or Odd (8 kyu)	Principiante
CodeWars	Categorize New Member (7 kyu)	Aficionado
CodeWars	Return Negative (8 kyu)	Principiante

Boletín U04 — Extras
📝 Boletín U04 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. La solución está oculta: resístete hasta agotar tu pista.

CodeWars

1. Return Negative

Te dan un número y tienes que devolverlo negativo. Pero ojo: si el número ya es negativo, déjalo como está.

Ejemplos: `1` → `-1`, `-14` → `-14`, `34` → `-34`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: un ternario decide: si es mayor que 0, cambia el signo (`-num`); si no, devuélvelo tal cual.

> 🔄 **Solución**

---

2. Sum of positive

Te dan un array de enteros. Devuelve la suma de todos los **positivos**.

Ejemplo: `[1, -4, 7, 12]` → `1 + 7 + 12 = 20`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: recorre el array con un `for` y, con un `if`, suma solo los números mayores que 0.

> 🔄 **Solución**

---

3. Grasshopper — Summation

Escribe un programa que calcule la suma de todos los números desde 1 hasta `n`.

Ejemplos: `summation(2)` → `3` (1 + 2), `summation(8)` → `36` (1 + 2 + … + 8).

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: un `for` que acumule del 1 al `n` en una variable `suma`. (O la fórmula matemática `n * (n + 1) / 2`, si te va el atajo.)

> 🔁 **Solución**

---

4. Sum without highest and lowest number

Suma todos los números de un array **excepto** el más alto y el más bajo. Si el array está vacío, es `null` o tiene un solo elemento, devuelve `0`.

Ejemplo: `[6, 2, 1, 8, 10]` → `2 + 6 + 8 = 16`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

Pista: recorre el array una vez para acumular la suma y, en el mismo bucle, rastrea el mínimo y el máximo con dos `if`. Al final: `suma - minimo - maximo`.

> 🔁 **Solución**

AceptaElReto

5. 149 — San Fermines

En los sanfermines, los mozos quieren saber la velocidad máxima de los toros para saber si salir corriendo o volar. Dado un número de toros y la velocidad de cada uno, di cuál es la máxima.

Entrada: varios casos de prueba hasta el final de la entrada (EOF). Cada caso: un número N y a continuación las N velocidades.

Ejemplo:

```
3
10 20 15
5
50 12 90 7 25
```

Salida:

```
20
90
```

- Enunciado en AceptaElReto
- Dificultad: Fácil

Pista: un `while (sc.hasNextInt())` lee hasta el final de la entrada. Dentro, un `for` recorre las velocidades y un `if` va guardando el máximo.

> 🔄 **Solución**

---

6. 340 — Siguiente con mismo número de cifras

Dado un número natural N, encuentra el siguiente número que tenga el **mismo número de cifras** que N. Si no existe (porque N es el último con ese número de cifras), muestra `MENSAJE`.

Entrada: varios casos separados por espacios, que termina con un `0`.

Ejemplos:

```
1 → 2
9 → MENSAJE
99 → MENSAJE
100 → 101
```

- Enunciado en AceptaElReto
- Dificultad: Medio

Pista: cuenta las cifras de N con un `while (copia > 0) { cifras++; copia /= 10; }`. Si N es `9`, `99`, `999`… (todos nueves), no hay siguiente. En el resto de casos, la respuesta es `N + 1`.

> 🔄 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines los bucles y las condiciones podrás volver a los problemas de la U03 (como Nochevieja o ¿Qué lado de la calle?) y resolverlos con `for` y `switch` para hacer soluciones más elegantes. El material no se pierde: se reutiliza.

Boletín U05 — Inicial

📝 Boletín U05 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, dale a ejecutar y deja que tus métodos hagan el trabajo pesado. Un `main` con todo dentro está bien… hasta que deja de caber.

Ejercicio 1: El saludo oficial

Escribe un programa llamado `Saludo` que tenga un método `public static void saludar()` que imprima `¡Hola, Java!`. El `main` debe llamar a `saludar()` tres veces, una por línea.

Pista: el método vive fuera del `main`; el `main` solo lo invoca por su nombre con paréntesis y punto y coma.

Ejercicio 2: ¿Qué pasa? — la variable fantasma

Este programa no compila. Míralo con calma y responde: ¿por qué falla y qué cambiarías para arreglarlo?

```
public class Fantasma {
    static int puntos = 10;

    public static void sumar() {
        int extra = 5;
        puntos += extra;
    }

    public static void main(String[] args) {
        sumar();
        System.out.println(puntos);
        System.out.println(extra);
    }
}
```

Pista: `extra` nace y muere dentro de `sumar()`. El compilador no la ve desde `main`.

Ejercicio 3: La ficha de presentación

Escribe un programa llamado `Ficha` con un método `public static void presentar(String nombre, int edad)` que imprima:

```
Me llamo Ana y tengo 20 años.
```

El `main` debe llamar al método con `("Ana", 20)` y con `("Luis", 15)`, cada llamada en su línea.

Ejercicio 4: El primer `return`

Escribe un programa llamado `Suma` con un método `public static int sumar(int a, int b)` que **devuelva** la suma en lugar de imprimirla.

En el `main`, guarda el resultado de `sumar(3, 4)` en una variable e imprime `3 + 4 = 7`. Después imprime el resultado de `sumar(10, 20)` directamente: `10 + 20 = 30`.

Pista: `return a + b;` entrega el valor a quien llamó; no lo imprime nadie todavía.

Ejercicio 5: Doble, por favor

Escribe un programa llamado `Doble` con **dos** métodos:

- `public static int calcularDoble(int n)` → devuelve el doble de `n`.
- `public static void mostrarDoble(int n)` → imprime `El doble de 6 es 12`.

El `main` debe usar los dos con el valor `6`. Fíjate en la diferencia: uno devuelve, el otro habla.

Ejercicio 6: ¿Qué imprime? — el viaje de ida y vuelta

Sin ejecutar, escribe la salida exacta, línea a línea:

```java
public class Viaje {
    static int sumar(int a, int b) {
        System.out.println("sumando...");
        return a + b;
    }

    public static void main(String[] args) {
        System.out.println("antes");
        int total = sumar(3, 4);
        System.out.println("total: " + total);
    }
}
```

Pista: el programa no salta de forma mágica: se para en la llamada, ejecuta el método y vuelve con el valor en la mano.

Ejercicio 7: ¿Par o impar?

Escribe un programa llamado `ParOImpar` con un método `public static boolean esPar(int n)` que devuelva `true` si el número es par.

El `main` debe comprobar el `7` y el `12` con un `if/else` e imprimir `7 es impar` y `12 es par`.

Pista: un solo `return` con el operador `%` de la U03.

Ejercicio 8: La puerta de la edad

Escribe un programa llamado `Puerta` con un método `public static boolean mayorDeEdad(int edad)` que devuelva `true` si la persona tiene 18 años o más.

El `main` debe comprobar `15`, `18` y `30` e imprimir, una por línea:

```
15: no entra
18: entra
30: entra
```

Pista: puedes usar un ternario en cada `println`, como en el ejercicio 6 de los extras de la U04.

Ejercicio 9: CodeWars — Century From Year

Resuelve la kata **“Century From Year”** (8 kyu) en CodeWars.

Completa el método `public static int century(int year)` que devuelve el siglo al que pertenece un año: el primer siglo va del año 1 al 100, el segundo del 101 al 200, y así sucesivamente.

Ejemplos: `century(1705)` → `18`, `century(1900)` → `19`, `century(1601)` → `17`, `century(89)` → `1`.

Pista: la división entera se come el año 1601 (`1601 / 100` es `16`); ajusta restando 1 antes de dividir o redondeando hacia arriba con `Math.ceil`.

Boletín U05 — Inicial Resuelto

📝 Boletín U05 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: El saludo oficial

> 🔄 Solución

Ejercicio 2: ¿Qué pasa? — la variable fantasma

> 🔄 Solución

Ejercicio 3: La ficha de presentación

> 🔄 Solución

Ejercicio 4: El primer `return`

> 🔄 Solución

Ejercicio 5: Doble, por favor

> 🔄 Solución

Ejercicio 6: ¿Qué imprime? — el viaje de ida y vuelta

> 🔄 Solución

Ejercicio 7: ¿Par o impar?

> Solución

Ejercicio 8: La puerta de la edad

> Solución

Ejercicio 9: CodeWars — Century From Year

> Solución

Boletín U05 — Avanzado

📝 Boletín U05 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: El clasificador de notas

Escribe un programa con un método `public static String calificar(double nota)` que devuelva la calificación textual con las mismas reglas de la U04: `>= 9` → "Sobresaliente", `>= 7` → "Notable", `>= 5` → "Aprobado", `>= 0` → "Suspenso", y fuera de rango → "Nota inválida".

El `main` debe imprimir el resultado de `calificar(8.7)`, `calificar(4.5)`, `calificar(11)` y `calificar(-2)`, uno por línea.

Pista: el método **devuelve** cadenas con `return`, no imprime nada. Comprueba primero si la nota está fuera de rango.

⭐ Ejercicio 2: Tablas a demanda

Escribe un método `public static void imprimirTabla(int n)` que imprima la tabla de multiplicar del `n` (del 1 al 10), una línea por cada multiplicación.

El `main` debe pedirle la tabla del 7, luego un separador `---`, y luego la tabla del 3.

Pista: el método no sabe nada del 7 ni del 3: solo conoce su parámetro `n`. Un `for` y a volar.

⭐ ⭐ Ejercicio 3: ¿Qué imprime? — la sombra del parámetro

Sin ejecutar, escribe la salida exacta:

```java
public class Sombra {
    static int valor = 100;

    public static void cambiar(int valor) {
        valor = valor + 1;
    }

    public static void main(String[] args) {
        cambiar(5);
        System.out.println(valor);
    }
}
```

Pista: dentro de `cambiar`, el parámetro `valor` tapa al campo `valor`. ¿Cuál de los dos cambia con `valor + 1`?

⭐ ⭐ Ejercicio 4: La suma de dígitos

Escribe un método `public static int sumaDigitos(int n)` que devuelva la suma de las cifras de `n` (por ejemplo, `1234` → `10`).

El `main` debe mostrar `sumaDigitos(1234)` y `sumaDigitos(905)`.

Pista: `n % 10` saca el último dígito y `n / 10` se lo come; repite con un `while` hasta que `n` sea 0. Si el número puede ser negativo, usa `Math.abs(n)` al principio.

⭐ ⭐ ⭐ Ejercicio 5: ¿Qué imprime? — el return que corta

Sin ejecutar, escribe la salida exacta:

```java
public class Corte {
    public static String signo(int n) {
        if (n < 0) {
            return "negativo";
        }
        if (n == 0) {
            return "cero";
        }
        return "positivo";
    }

    public static int primeroPar(int a, int b) {
        if (a % 2 == 0) {
            return a;
        }
        if (b % 2 == 0) {
            return b;
        }
        return -1;
    }

    public static void main(String[] args) {
        System.out.println(signo(-5) + " " + signo(0) + " " + signo(4));
        System.out.println(primeroPar(7, 4));
        System.out.println(primeroPar(3, 5));
    }
}
```

Pista: en cada método solo se ejecuta **un** `return`: el resto del código queda muerto. Recorre los casos uno a uno.

⭐ ⭐ ⭐ Ejercicio 6: Divide el informe

Este programa funciona, pero todo vive dentro del `main`:

```java
public class Informe {
    public static void main(String[] args) {
        double a = 7.5, b = 9.0, c = 4.5, d = 8.25;

        double media = (a + b + c + d) / 4;
        double maximo = a;
        if (b > maximo) maximo = b;
        if (c > maximo) maximo = c;
        if (d > maximo) maximo = d;
        double minimo = a;
        if (b < minimo) minimo = b;
        if (c < minimo) minimo = c;
        if (d < minimo) minimo = d;

        System.out.println("Media: " + media);
        System.out.println("Máximo: " + maximo);
        System.out.println("Mínimo: " + minimo);
    }
}
```

Trocealo en **tres métodos** que devuelven `double`: `media(a, b, c, d)`, `maximo(a, b, c, d)` y `minimo(a, b, c, d)`. El `main` debe quedarse en tres líneas de `println` que llaman a los métodos.

Pista: los tres métodos son `public static double ...` con `return` y ninguna llamada a `System.out`. Comparar `a, b, c, d` es exactamente lo mismo dentro de cada uno.

⭐ ⭐ Ejercicio 7: El contador de vocales

Escribe un método `public static int contarVocales(String texto)` que devuelva cuántas vocales hay en `texto` (sin distinguir mayúsculas de minúsculas).

El `main` debe imprimir `contarVocales("Hola mundo")` (da `4`) y `contarVocales("zzz")` (da `0`).

Pista: recorre con `for (int i = 0; i < texto.length(); i++)`, mira cada carácter con `charAt(i)` y compáralo con las cinco vocales en sus dos versiones.

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — 'Disemvowel' Trolls

Resuelve la kata **"'Disemvowel' Trolls"** (7 kyu) en CodeWars.

Crea el método `public static String disemvowel(String str)` que devuelve la cadena sin vocales.

Ejemplo: `"This website is for losers LOL!"` → `"Ths wbst s fr lsrs LL!"`.

Pista: recorre con `charAt` y concatena solo lo que no sea vocal. La solución con regex llega en la U13; aquí nos vale un `if` con muchas comparaciones.

⭐ ⭐ ⭐ Ejercicio 9: AceptaElReto — 165 Número hyperpar

Resuelve el problema **165 — Número hyperpar** en AceptaElReto.com.

Un número es *hyperpar* cuando **todos** sus dígitos son pares. La entrada contiene un número por línea hasta un número negativo (que no se procesa). Para cada caso, muestra `SI` o `NO`.

Ejemplo:

```
2460
1234
2
-1
```

```
SI
NO
SI
```

Pista: envuelve la comprobación en un método `public static boolean esHyperpar(int n)`: con un `while` saca dígitos con `n % 10` y, si alguno es impar, `return false` en cuanto lo encuentres. Si el bucle acaba sin encontrar ninguno, `return true`.

📚 Referencias

Plataforma	Problema	Dificultad
AceptaElReto	165 — Número hyperpar	Fácil
AceptaElReto	115 — Número de Kaprekar	Medio
CodeWars	Century From Year (8 kyu)	Principiante
CodeWars	‘Disemvowel’ Trolls (7 kyu)	Aficionado
CodeWars	Volume of a Cuboid (8 kyu)	Principiante
CodeWars	Drink about (8 kyu)	Principiante

Boletín U05 — Avanzado Resuelto

📝 Boletín U05 — Avanzado (Resuelto)

> Las soluciones están ocultas. Inténtalo de verdad antes de destaparlas.

⭐ Ejercicio 1: El clasificador de notas

> 🔄 Solución

⭐ Ejercicio 2: Tablas a demanda

> 🔄 Solución

⭐ ⭐ Ejercicio 3: ¿Qué imprime? — la sombra del parámetro

> 🔄 Solución

⭐ ⭐ Ejercicio 4: La suma de dígitos

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 5: ¿Qué imprime? — el return que corta

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 6: Divide el informe

> 🔄 Solución

⭐ ⭐ Ejercicio 7: El contador de vocales

> 🔁 Solución

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — ‘Disemvowel’ Trolls

> 🔁 Solución

⭐ ⭐ ⭐ Ejercicio 9: AceptaElReto — 165 Número hyperpar

> 🔁 Solución

📚 Referencias

Plataforma	Problema	Dificultad
AceptaElReto	165 — Número hyperpar	Fácil
AceptaElReto	115 — Número de Kaprekar	Medio
CodeWars	Century From Year (8 kyu)	Principiante
CodeWars	‘Disemvowel’ Trolls (7 kyu)	Aficionado
CodeWars	Volume of a Cuboid (8 kyu)	Principiante
CodeWars	Drink about (8 kyu)	Principiante

Boletín U05 — Extras

📝 Boletín U05 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. La solución está oculta: resístete hasta agotar tu pista.

CodeWars

1. L1: Set Alarm

Un booleano `employed` te dice si trabajas y otro, `vacation`, si estás de vacaciones. Devuelve `true` **solo** si trabajas y no estás de vacaciones.

Ejemplos: `setAlarm(true, false)` → `true`, `setAlarm(false, false)` → `false`, `setAlarm(true, true)` → `false`, `setAlarm(false, true)` → `false`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: una sola condición con `&&` y un `!` delante de las vacaciones.

> 🔄 Solución

---

2. Beginner Series #2 Clock

El reloj marca `h` horas, `m` minutos y `s` segundos desde la medianoche. Escribe el método `Past`, que devuelve ese tiempo en **milisegundos**.

Ejemplos: `Past(0, 1, 1)` → `61000`, `Past(1, 0, 1)` → `3601000`, `Past(0, 0, 0)` → `0`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: 1 hora = 3.600.000 ms, 1 minuto = 60.000 ms y 1 segundo = 1.000 ms. Suma las tres partes y multiplica al final.

> 🔄 Solución

---

3. Volume of a Cuboid

El método `getVolumeOfCuboid` recibe largo, ancho y alto de una caja y devuelve su volumen.

Ejemplos: `(1, 2, 3)` → `6`, `(6, 2, 1)` → `12`, `(5, 3, 2)` → `30`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: el volumen de un paralelepípedo es ancho × alto × largo. Un `return`, tres multiplicaciones, ni un `if`.

> 🔄 **Solución**

---

4. Drink about

Dada una edad, devuelve qué bebida corresponde: menos de 14 → `"drink toddy"`, menos de 18 → `"drink coke"`, menos de 21 → `"drink beer"` y a partir de 21 → `"drink whisky"`.

Ejemplos: `10` → `"drink toddy"`, `15` → `"drink coke"`, `20` → `"drink beer"`, `30` → `"drink whisky"`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: son cuatro `return` encadenados con `if`, de la edad más pequeña a la mayor, como la cascada de notas de la U04.

> 🔄 **Solución**

AceptaElReto

5. 116 — ¡Hola mundo!

En la primera clase de cualquier curso de programación se debería salir habiendo escrito un «hola mundo». Pues eso: dado un número `n` (`0 ≤ n ≤ 5`), escribe `n` líneas con la cadena `Hola mundo.`.

Ejemplo:

```
3
```

Salida:

```
Hola mundo.
Hola mundo.
Hola mundo.
```

- Enunciado en AceptaElReto
- Dificultad: Fácil

Pista: un método `public static void imprimirHolaMundo(int n)` con un `for` de `0` a `n - 1`, y un `main` que solo lee y llama.

> 🔄 **Solución**

---

6. 115 — Número de Kaprekar

Un número es de *Kaprekar* si su cuadrado puede partirse en dos números que sumados devuelvan el original, y el segundo de la partida no puede ser cero. Por ejemplo: `9² = 81` → `8 + 1 = 9` (¡es Kaprekar!) y `703² = 494209` → `494 + 209 = 703`. En cambio, `100` no lo es: `10000` podría partirse en `100 + 00`, pero `00` vale cero.

Entrada: varios números (≥ 1 y < 65536) hasta un `0` que marca el final.

Ejemplo:

```
22222
75
99
100
0
```

Salida:

```
SI
NO
SI
NO
```

- Enunciado en AceptaElReto
- Dificultad: Medio

Pista: envuelve todo en `public static boolean esKaprekar(int n)`. El cuadrado puede pasarse de `int` (usa `long`), y debes probar **todos** los cortes posibles: `p = 10, 100, 1000, ...` hasta superar el cuadrado, con `izq = cuadrado / p` y `der = cuadrado % p`. Si `der != 0` y `izq + der == n`, has ganado.

> 🔁 Solución

> 🧭 **¿Y si te quedas con ganas?** Con los métodos en la mano puedes reescribir tus programas de la U04 troceando cada bloque en una función con nombre. Y si te apetece un reto de futuro, la U08 te enseñará a hacer que un método se llame a sí mismo (recursión): el juego de la caja rusa del código.

Boletín U06 — Inicial

📝 Boletín U06 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, crea tu primer aparcamiento de datos y haz que el `for-each` deje de parecer magia. Ninguna plaza nace sabiendo tener dueño.

Ejercicio 1: ¿Qué imprime? — Array de booleanos

```
boolean[] flags = new boolean[3];
flags[1] = true;
System.out.println(flags[0] + " " + flags[1] + " " + flags[2]);
```

¿Qué imprime? ¿Cuál es el valor por defecto de un `boolean` en un array?

Ejercicio 2: Encuentra el error — NullPointerException

```
String[] nombres = new String[3];
nombres[0] = "Ana";
nombres[1] = "Bob";
System.out.println(nombres[2].toUpperCase());
```

¿Qué ocurre al ejecutar este código? ¿Por qué?

Ejercicio 3: Completa el código — for básico para buscar el mayor

Completa el siguiente programa para que encuentre e imprima el número más grande del array:

```java
int[] numeros = {12, 45, 7, 34, 89, 23};
int mayor = numeros[0];

for (int i = 1; i < ______; i++) {   // ¿hasta dónde llega el bucle?
    if (numeros[i] ______ mayor) {    // ¿qué operador?
        ______ = numeros[i];          // ¿qué asignamos?
    }
}

System.out.println("El mayor es: " + mayor);
```

Ejercicio 4: Escribe este programa — contar números pares

Crea un array de 10 enteros con valores que tú elijas. Recórrelo con un bucle `for` y cuenta cuántos de ellos son pares. Al final, imprime el total de pares y el array original con `Arrays.toString`.

Ejemplo de salida:

```
Array: [3, 8, 12, 5, 7, 10, 2, 9, 6, 1]
Pares: 5
```

Pista: un número es par si `numeros[i] % 2 == 0`.

Ejercicio 5: Encuentra el error — length vs length()

```java
int[] numeros = {10, 20, 30};
String texto = "Hola";

System.out.println(numeros.length());
System.out.println(texto.length);
```

¿Qué líneas tienen error? Explica la diferencia entre `length` (sin paréntesis) y `length()` (con paréntesis).

Ejercicio 6: ¿Qué imprime? — la suma de los impares

Sin ejecutar, escribe la salida exacta de este programa:

```
public class SumaImpares {
    public static void main(String[] args) {
        int[] datos = {3, 8, 2, 9, 5};
        int total = 0;

        for (int n : datos) {
            if (n % 2 == 1) {
                total += n;
            }
        }

        System.out.println(total);
    }
}
```

Pista: el `for-each` recorre todos los valores; solo se suman los que dejan resto 1 al dividir entre 2.

Ejercicio 7: Escribe este programa — búsqueda lineal

Crea un array de enteros llamado `edades` con 8 valores. Pide al usuario un número por teclado (con `Scanner`) y busca si ese número está en el array. Imprime «Encontrado en posición X» o «No encontrado».

```
Introduce edad a buscar: 25
Encontrado en posición 3
```

Pista: usa una variable `posicion = -1` como “no encontrado”, y `break` en cuanto lo encuentres.

Ejercicio 8: Escribe este programa — el inverso

Crea un array de 10 enteros, rellénalo con los números del 1 al 10 y luego imprímelo en **orden inverso** (del 10 al 1). Hazlo con un `for` que recorra el array hacia atrás.

Pista: el bucle va de `length - 1` hasta `0`, bajando con `i--`.

Ejercicio 9: Escribe este programa — la clase Arrays en acción

Crea el array `int[] notas = {7, 3, 9, 5, 2, 8}` y haz lo siguiente:

1. Muéstralo con `Arrays.toString`.
2. Ordena con `Arrays.sort` y muéstralo otra vez.
3. Busca el `8` con `Arrays.binarySearch` e imprime su posición.

Pista: `import java.util.Arrays;` al principio, y recuerda: `binarySearch` solo es fiable si el array ya está ordenado.

Boletín U06 — Inicial Resuelto

📝 Boletín U06 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: ¿Qué imprime? — Array de booleanos

> 🔄 Solución

Ejercicio 2: Encuentra el error — NullPointerException

> 🔄 Solución

Ejercicio 3: Completa el código — for básico para buscar el mayor

> 🔄 Solución

Ejercicio 4: Escribe este programa — contar números pares

> 🔄 Solución

Ejercicio 5: Encuentra el error — length vs length()

> 🔄 Solución

Ejercicio 6: ¿Qué imprime? — la suma de los impares

> Solución

Ejercicio 7: Escribe este programa — búsqueda lineal

> Solución

Ejercicio 8: Escribe este programa — el inverso

> Solución

Ejercicio 9: Escribe este programa — la clase Arrays en acción

> Solución

Boletín U06 — Avanzado

📝 Boletín U06 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: La fusión de arrays ordenados

Escribe un método `fusionarArrays(int[] a, int[] b)` que reciba dos arrays ordenados de menor a mayor y devuelva un **nuevo array** también ordenado con todos los elementos de ambos. No uses `Arrays.sort()`. Hazlo con el algoritmo de fusión (merge) tipo «dos punteros».

Pista: avanza con dos índices, uno por array, comparando en cada paso cuál elemento es menor. Cuando un array se acabe, copia el resto del otro.

⭐ Ejercicio 2: Rotación circular a la derecha

Implementa un método `rotarDerecha(int[] arr, int k)` que desplace cada elemento del array `k` posiciones hacia la derecha. Los elementos que «salen» por el final vuelven a entrar por el principio.

Ejemplo: `{1, 2, 3, 4, 5}` con `k = 2` → `{4, 5, 1, 2, 3}`.

Pista: usa un array temporal del tamaño de `k` con los últimos `k` elementos, desplaza el resto hacia la derecha y luego coloca los guardados al principio. (O divide los índices con `(i + k) % arr.length`.)

⭐ Ejercicio 3: Suma de diagonales (matriz cuadrada)

Crea un programa que genere una matriz cuadrada `int[N][N]` con valores aleatorios entre 1 y 100, y calcule:

1. Suma de la **diagonal principal** (de arriba-izquierda a abajo-derecha).
2. Suma de la **diagonal secundaria** (de arriba-derecha a abajo-izquierda).

3. Diferencia absoluta entre ambas sumas.

Usa `N = 5` para las pruebas y `Math.random()` para rellenarla.

Pista: en la diagonal principal, `fila == columna`, así que un solo bucle con `matriz[i][i]` basta. En la secundaria, `columna = N - 1 - i`.

⭐ ⭐ Ejercicio 4: ¿Está ordenado?

Escribe un método `public static boolean estaOrdenado(int[] arr)` que devuelva `true` si el array está ordenado **de menor a mayor** (cada elemento es menor o igual que el siguiente), y `false` en caso contrario.

Pruebas:

- `estaOrdenado({1, 2, 3, 4})` → `true`
- `estaOrdenado({1, 3, 2, 4})` → `false`
- `estaOrdenado({})` → `true`
- `estaOrdenado({7})` → `true`

Pista: recorre con un `for` de `i = 1` hasta el final y pregunta si `arr[i] < arr[i - 1]`. En cuanto un vecino rompa el orden, devuelve `false`.

⭐ ⭐ Ejercicio 5: El inverso en el sitio

Escribe un método `public static void invertir(int[] arr)` que dé la vuelta al array **sin crear otro array** (usa dos punteros y una variable temporal).

Prueba con `{1, 2, 3, 4, 5}` y muestra el resultado con `Arrays.toString` → debe salir `[5, 4, 3, 2, 1]`.

Pista: `izquierda = 0` y `derecha = arr.length - 1`; mientras `izquierda < derecha`, intercambia y mueve ambos hacia el centro.

⭐ ⭐ Ejercicio 6: ¿Qué imprime? — el doble bucle que cuenta parejas

Sin ejecutar, escribe la salida exacta de este programa:

```
public class CuentaParejas {
    public static void main(String[] args) {
        int[] datos = {2, 4, 6};
        int contador = 0;

        for (int i = 0; i < datos.length; i++) {
            for (int j = i + 1; j < datos.length; j++) {
                if (datos[i] < datos[j]) {
                    contador++;
                }
            }
        }

        System.out.println(contador);
    }
}
```

Pista: cuenta las parejas `(i, j)` con `i < j` donde el primero es menor que el segundo. Con `{2, 4, 6}` todas las parejas lo cumplen. ¿Cuántas hay?

⭐ ⭐ ⭐ Ejercicio 7: Estadísticas de clase

Pide al usuario las notas de 20 alumnos con `Scanner`, guárdalas en un `double[]` y calcula:

- La nota media.
- La nota más alta y la más baja.
- Cuántos alumnos aprobaron (nota >= 5).

Usa un bucle para rellenar, otro para sumar y el patrón del máximo/mínimo acumulado.

Pista: para el mínimo empieza con la primera nota; para el máximo, también. Guarda las notas en un array y recórrelo varias veces: cada cálculo es un bucle.

⭐ ⭐ ⭐ Ejercicio 8: El gran reto — buscaminas simplificado

Crea un array bidimensional `boolean[5][5]` que represente un campo de minas. Coloca 5 minas en posiciones **aleatorias** (con `Math.random()`). El usuario introduce coordenadas `(fila, columna)` y el programa dice si hay mina o no. Si acierta una mina, el juego termina mostrando «¡BOOM!».

Extras opcionales: di cuántas minas hay alrededor de la casilla (mira las 8 vecinas) y termina cuando hayas comprobado todas las casillas sin minas.

Pista: comprueba antes de mirar una vecina que su fila y su columna estén entre 0 y 4, o te saldrás del array con `ArrayIndexOutOfBoundsException`. El juego usa un `while` que termina al pisar una mina o al agotar las casillas.

⭐ ⭐ ⭐ Ejercicio 9: Compactar — los ceros al final

Escribe un método `public static void compactar(int[] arr)` que mueva todos los ceros al final del array, **manteniendo el orden** de los elementos que no son cero.

Ejemplos:

- `{0, 3, 0, 1, 0, 2}` → `{3, 1, 2, 0, 0, 0}`
- `{1, 2, 3}` → `{1, 2, 3}`
- `{0, 0, 0}` → `{0, 0, 0}`

Pista: usa un segundo índice `pos` que marque dónde va el siguiente valor no cero. Recorre con `i`, y cuando `arr[i]` no sea 0, cópialo a `arr[pos]` y sube `pos`. Al final, rellena de ceros desde `pos` hasta el final.

Boletín U06 — Avanzado Resuelto

📝 Boletín U06 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

⭐ Ejercicio 1: La fusión de arrays ordenados

> 🔄 Solución

⭐ Ejercicio 2: Rotación circular a la derecha

> 🔄 Solución

⭐ Ejercicio 3: Suma de diagonales (matriz cuadrada)

> 🔄 Solución

⭐ ⭐ Ejercicio 4: ¿Está ordenado?

> 🔄 Solución

⭐ ⭐ Ejercicio 5: El inverso en el sitio

> 🔄 Solución

⭐ ⭐ Ejercicio 6: ¿Qué imprime? — el doble bucle que cuenta parejas

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 7: Estadísticas de clase

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 8: El gran reto — buscaminas simplificado

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 9: Compactar — los ceros al final

> 🔄 Solución

Boletín U06 — Extras

📝 Boletín U06 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. Las soluciones están ocultas: agota tu pista antes de mirarlas.

CodeWars

1. Convert number to reversed array of digits

Te dan un número entero no negativo. Devuelve sus dígitos en un array de enteros, **en orden inverso**.

Ejemplo: `35231` → `[1, 3, 2, 5, 3]` · `0` → `[0]`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

> 💡 **Pista**

> 🔁 **Solución**

---

2. Find the smallest integer in the array

Te dan un array de enteros. Devuelve el entero **más pequeño**.

Ejemplos: `[78, 56, 232, 12, 11, 43]` → `11` · `[34, -345, -1, 100]` → `-345`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

> 💡 **Pista**

> 🔁 **Solución**

---

3. Count by X

Devuelve un array con los **primeros `n` múltiplos** de `x`.

Ejemplos: `countBy(2, 5)` → `[2, 4, 6, 8, 10]` · `countBy(1, 10)` → `[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

> 💡 **Pista**

> 🔁 **Solución**

---

4. Take a Ten Minute Walk

Vives en una ciudad en cuadrícula. Te dan un array de direcciones (`'n'`, `'s'`, `'e'`, `'w'`). Devuelve `true` si el paseo dura **exactamente 10 minutos** (una manzana por minuto) **y te deja en el punto de partida**.

Ejemplos: `['n','s','n','s','n','s','n','s','n','s']` → `true` · `['n','n','n','s','n','s','n','s','n','s']` → `false`.

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔁 **Solución**

AceptaElReto

5. 171 — Abadías de piedra

Una cordillera tiene una serie de montañas, cada una con su altura. Se puede construir una **abadía** sobre una montaña si es **más alta que todas las que tiene a su derecha** (hacia el mar). Cuenta cuántas montañas cumplen la condición.

Entrada: varios casos de prueba. Cada caso empieza con `N` (número de montañas), seguido de `N` alturas en la misma línea. Un `0` termina la entrada.

Ejemplo:

```
3
5 3 4
4
2 3 4 1
0
```

Salida:

```
2
2
```

- Enunciado en AceptaElReto
- Dificultad: Fácil

> 💡 **Pista**

> 🔁 **Solución**

---

6. 369 — Contando en la arena

Un niño escribe los números en la arena con marcas de conteo: el `1` lo escribe como `1`, el `2` como `11`, el `3` como `111`… Es decir, cada número se representa con tantos unos como valga. Dado un número, escríbelo como lo haría el niño.

Entrada: varios números, uno por línea, hasta un `0` final.

Ejemplo:

```
1
5
3
0
```

Salida:

```
1
11111
111
```

- Enunciado en AceptaElReto
- Dificultad: Fácil

> 💡 **Pista**

> 🔁 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines el aparcamiento entero —crear, recorrer, ordenar, buscar, invertir y compactar—, vuelve a estos problemas con el punto 9 como examen. Y en la U07 te esperan los algoritmos: búsqueda binaria, burbuja y Big O. Los arrays que has aprendido hoy son la materia prima de todo lo que viene.

Boletín U07 — Inicial

📝 Boletín U07 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE y deja que tus bucles busquen, ordenen y calculen. El ordenador nunca se cansa de recorrer un array: eres tú quien decide cómo.

Ejercicio 1: ¿Qué imprime? — la búsqueda de la gema

Sin ejecutar, escribe la salida exacta de este programa:

```
public class BusquedaGema {
    public static void main(String[] args) {
        int[] cofre = {12, 7, 25, 9, 31};
        int objetivo = 9;
        int posicion = -1;

        for (int i = 0; i < cofre.length; i++) {
            if (cofre[i] == objetivo) {
                posicion = i;
                break;
            }
        }

        System.out.println("Posición: " + posicion);
    }
}
```

Pista: recorre el array mentalmente elemento a elemento. ¿En qué índice aparece el 9 por primera vez?

Ejercicio 2: El buscador de tesoros

Escribe un método `public static int buscar(int[] datos, int objetivo)` que recorra el array con un `for` y devuelva el **índice** donde aparece el objetivo. Si no está, devuelve `-1`.

Prueba con `int[] tesoros = {4, 8, 15, 16, 23, 42}`:

- `buscar(tesoros, 15)` → debe devolver `2`
- `buscar(tesoros, 7)` → debe devolver `-1`

Pista: devuelve `i` en cuanto encuentres el objetivo; solo después del bucle devuelves `-1`.

Ejercicio 3: ¿Qué imprime? — la bombolla mínima

Sin ejecutar, escribe la salida exacta de este programa:

```
public class BombollaMinima {
    public static void main(String[] args) {
        int[] datos = {3, 1, 2};

        for (int i = 0; i < datos.length - 1; i++) {
            for (int j = 0; j < datos.length - 1 - i; j++) {
                if (datos[j] > datos[j + 1]) {
                    int temp = datos[j];
                    datos[j] = datos[j + 1];
                    datos[j + 1] = temp;
                }
            }
        }

        for (int num : datos) {
            System.out.print(num + " ");
        }
    }
}
```

Pista: haz la traza en un papel. Primera pasada: el 3 vs 1, luego 3 vs 2. ¿Qué queda al final?

Ejercicio 4: La caja de zapatos ordenada

Escribe un método `public static void ordenar(int[] datos)` que ordene el array **de menor a mayor** usando la ordenación por inserción.

Prueba con `int[] caja = {9, 2, 7, 1}` y muestra el array resultante con un `for...each`.

Pista: guarda `int clave = datos[i]` y desliza hacia la derecha los elementos mayores con un `while`.

Ejercicio 5: El detective de la búsqueda binaria

Escribe un método `public static int busquedaBinaria(int[] datos, int objetivo)` que use la búsqueda binaria sobre un array **ya ordenado** y devuelva el índice del objetivo (o `-1`).

Prueba con `int[] agenda = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}`:

- `busquedaBinaria(agenda, 23)` → debe devolver `5`
- `busquedaBinaria(agenda, 30)` → debe devolver `-1`

Pista: dos punteros `izquierda = 0` y `derecha = datos.length - 1`, un `while (izquierda <= derecha)` y `int medio = izquierda + (derecha - izquierda) / 2`.

Ejercicio 6: ¿Qué imprime? — la binaria que falla

Sin ejecutar, escribe la salida exacta de este programa:

```java
public class BinariaQueFalla {
    public static void main(String[] args) {
        int[] datos = {10, 20, 30, 40, 50};
        int objetivo = 35;
        int izquierda = 0;
        int derecha = datos.length - 1;
        boolean encontrado = false;

        while (izquierda <= derecha) {
            int medio = izquierda + (derecha - izquierda) / 2;
            if (datos[medio] == objetivo) {
                encontrado = true;
                break;
            } else if (datos[medio] < objetivo) {
                izquierda = medio + 1;
            } else {
                derecha = medio - 1;
            }
        }

        System.out.println(encontrado);
    }
}
```

Pista: el 35 no está en el array. ¿Qué valor tiene `encontrado` al final del bucle?

Ejercicio 7: El contador de intercambios

Escribe un programa llamado `ContadorIntercambios` que use la bombolla para ordenar `int[] datos = {5, 2, 9, 1, 5}` y **cuente cuántos intercambios** realiza en total. Al final muestra: `Intercambios: X`.

Pista: declara `int intercambios = 0;` y súmale 1 dentro del `if` de intercambio, justo después de la operación.

Ejercicio 8: La nota más alta de la clase

Escribe un método `public static int notaMaxima(int[] notas)` que recorra el array y devuelva la nota más alta.

Prueba con `int[] notas = {7, 9, 5, 10, 6, 8, 4}` → debe devolver `10`.

Pista: usa una variable `maximo` inicializada con el primer elemento (`notas[0]`) y compara con cada uno.

Ejercicio 9: CodeWars — Find the smallest integer in the array

Resuelve la kata **"Find the smallest integer in the array"** (8 kyu) en CodeWars.

Crea el método `public static int findSmallestInt(int[] args)` que devuelva el entero más pequeño del array.

Pista: es el mismo patrón del ejercicio 8: recorre y compara. Aquí el array nunca está vacío.

Boletín U07 — Inicial Resuelto

📝 Boletín U07 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: ¿Qué imprime? — la búsqueda de la gema

> 🔄 Solución

Ejercicio 2: El buscador de tesoros

> 🔄 Solución

Ejercicio 3: ¿Qué imprime? — la bombolla mínima

> 🔄 Solución

Ejercicio 4: La caja de zapatos ordenada

> 🔄 Solución

Ejercicio 5: El detective de la búsqueda binaria

> 🔄 Solución

Ejercicio 6: ¿Qué imprime? — la binaria que falla

> 🔄 Solución

Ejercicio 7: El contador de intercambios

> Solución

Ejercicio 8: La nota más alta de la clase

> Solución

Ejercicio 9: CodeWars — Find the smallest integer in the array

> Solución

Boletín U07 — Avanzado

📝 Boletín U07 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: ¿Qué imprime? — la binaria con historial

Sin ejecutar, escribe la salida exacta de este programa:

```
public class BinariaHistorial {
    public static void main(String[] args) {
        int[] datos = {2, 4, 6, 8, 10, 12, 14, 16};
        int objetivo = 10;
        int izquierda = 0;
        int derecha = datos.length - 1;

        while (izquierda <= derecha) {
            int medio = izquierda + (derecha - izquierda) / 2;
            System.out.println("Pruebo el índice " + medio);

            if (datos[medio] == objetivo) {
                System.out.println("Encontrado en " + medio);
                break;
            } else if (datos[medio] < objetivo) {
                izquierda = medio + 1;
            } else {
                derecha = medio - 1;
            }
        }
    }
}
```

Pista: escribe en una tabla cada vuelta con sus `izquierda`, `derecha`, `medio` y qué decide. Solo son 3 o 4 líneas de traza.

⭐ Ejercicio 2: El buscador binario con historial

Escribe un método `public static int busquedaBinaria(int[] datos, int objetivo)` como el de la teoría, pero que **cada vez que pruebe un índice, lo muestre por pantalla**: `Probando el índice X`. Al final devuelve el índice o `-1`.

Prueba con `int[] datos = {1, 3, 5, 7, 9, 11, 13, 15, 17, 19}` y `objetivo = 7`.

Pista: el `System.out.println` va dentro del `while`, justo después de calcular `medio` y antes de comparar.

⭐ ⭐ Ejercicio 3: La bombolla con recuento (y flag)

Escribe un programa llamado `BombollaRecuento` que ordene `int[] datos = {9, 3, 7, 1, 5}` con bombolla y **cuente los intercambios**, igual que en el inicial. Pero además, añade el flag `boolean huboIntercambio` con su `break` para no hacer pasadas inútiles.

Muestra al final el array ordenado y `Intercambios: X`.

Pista: el flag se resetea a `false` al principio de cada pasada y se pone a `true` dentro del `if`. Después de la pasada: `if (!huboIntercambio) break;`.

⭐ ⭐ Ejercicio 4: La inserción descendente

Modifica la ordenación por inserción para que ordene **de mayor a menor**. Escribe un método `public static void ordenarDescendente(int[] datos)`.

Prueba con `int[] notas = {6, 9, 3, 8, 5}` y muestra el resultado.

Pista: solo cambia un signo: en la condición del `while`, los mayores deben deslizarse hacia la derecha. Piensa cuál es ahora el "orden correcto".

⭐ ⭐ Ejercicio 5: El analista de complejidad

Di la complejidad Big O de cada método y justifica brevemente:

```java
public class Analista {

    public static int metodoA(int[] datos) {
        int total = 0;
        for (int num : datos) {
            total += num;
        }
        return total;
    }

    public static int metodoB(int[] datos) {
        int pares = 0;
        for (int i = 0; i < datos.length; i++) {
            for (int j = i + 1; j < datos.length; j++) {
                if (datos[i] == datos[j]) {
                    pares++;
                }
            }
        }
        return pares;
    }

    public static int metodoC(int[] datos, int objetivo) {
        int izquierda = 0;
        int derecha = datos.length - 1;
        while (izquierda <= derecha) {
            int medio = izquierda + (derecha - izquierda) / 2;
            if (datos[medio] == objetivo) return medio;
            if (datos[medio] < objetivo) izquierda = medio + 1;
            else derecha = medio - 1;
        }
        return -1;
    }
}
```

Pista: cuenta bucles: uno → O(n), dos anidados → O(n²). El que divide el segmento por la mitad en cada vuelta es O(log n). El `j = i + 1` no lo salva de ser O(n²): sigue siendo casi n × n.

⭐ ⭐ Ejercicio 6: El cazador de parejas

Escribe un método `public static boolean existePareja(int[] datos, int sumaObjetivo)` que devuelva `true` si existen **dos elementos distintos** del array cuya suma sea `sumaObjetivo`.

Prueba con `int[] precios = {10, 3, 7, 5, 12}`:

- `existePareja(precios, 17)` → `true` (10 + 7)
- `existePareja(precios, 25)` → `false`

Pista: dos bucles anidados con `j = i + 1` (para no probar un elemento consigo mismo). En el interior: `if (datos[i] + datos[j] == sumaObjetivo) return true;`. Al final, `return false;`.

⭐ ⭐ Ejercicio 7: El detective de inversiones

Una **inversión** es una pareja de posiciones `(i, j)` con `i < j` donde `datos[i] > datos[j]` (están desordenadas). Escribe un método `public static int contarInversiones(int[] datos)` que las cuente.

Prueba con `int[] datos = {2, 4, 1, 3}` → hay 3 inversiones: (2,1), (4,1), (4,3).

Pista: el patrón de doble bucle con `j = i + 1` otra vez, pero ahora el `if` compara `datos[i] > datos[j]` y suma 1 al contador.

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — Ones and Zeros

Resuelve la kata **"Ones and Zeros"** (7 kyu) en CodeWars.

Te dan un array de enteros (solo 0 y 1) que representa un número en binario. Devuelve su valor decimal.

Ejemplo: `[1, 0, 1, 1]` → `11` (1·8 + 0·4 + 1·2 + 1·1).

Pista: recorre el array de izquierda a derecha acumulando `valor = valor * 2 + digito`. Ese es el algoritmo para convertir de binario a decimal sin usar `Math.pow`.

⭐ ⭐ ⭐ Ejercicio 9: AceptaElReto — 100 Constante de Kaprekar

Resuelve el problema **100 — Constante de Kaprekar** en AceptaElReto.com.

El algoritmo de Kaprekar: dado un número de 4 cifras, ordena sus dígitos de mayor a menor y de menor a mayor, resta ambos, y repite con el resultado. Al final siempre se llega a **6174** (o al propio 6174) y, si el número tiene todas las cifras iguales (como 1111), la diferencia da 0.

La entrada trae varios números (pueden tener menos de 4 cifras: hay que completar con ceros a la izquierda). Para cada uno, muestra **cuántas iteraciones** se necesitan para alcanzar 6174. El 6174 necesita 0 iteraciones. **Cuidado con los repdigits** (1111, 5555…): la primera resta da 0 y el bucle nunca llegaría a 6174; el problema oficial pide **8** para ellos. El 0 de entrada termina el programa.

Pista: para ordenar los dígitos, extráelos en un `int[]` de 4 posiciones con `% 10` y `/ 10`, ordénalo con bombolla (¡reutiliza la UO7!), y reconstruye el número mayor (dígitos en orden descendente) y el menor (ascendente). Cuenta las iteraciones con un contador.

📚 Referencias

Plataforma	Problema	Dificultad
AceptaElReto	100 — Constante de Kaprekar	Fácil
AceptaElReto	185 — Potitos	Fácil
CodeWars	Find the smallest integer in the array (8 kyu)	Principiante
CodeWars	Ones and Zeros (7 kyu)	Aficionado
CodeWars	You only need one (8 kyu)	Principiante

Boletín U07 — Avanzado Resuelto
📝 Boletín U07 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: ¿Qué imprime? — la binaria con historial

> 🔄 Solución

Ejercicio 2: El buscador binario con historial

> 🔄 Solución

Ejercicio 3: La bombolla con recuento (y flag)

> 🔄 Solución

Ejercicio 4: La inserción descendente

> 🔄 Solución

Ejercicio 5: El analista de complejidad

> 🔄 Solución

Ejercicio 6: El cazador de parejas

> 🔄 Solución

Ejercicio 7: El detective de inversiones

> Solución

Ejercicio 8: CodeWars — Ones and Zeros

> Solución

Ejercicio 9: AceptaElReto — 100 Constante de Kaprekar

> Solución

Boletín U07 — Extras
📝 Boletín U07 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. La solución está oculta: resístete hasta agotar tu pista.

CodeWars

1. You only need one — Beginner

Te dan un array y un valor. Devuelve `true` si el array **contiene** el valor.

Ejemplos: `{"a", "b", "c"}, "a"` → `true` · `{"a", "b", "c"}, "z"` → `false`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: un `for` que recorra el array y un `if` que devuelva `true` si algún elemento es igual al valor. Después del bucle, `false`.

> 🔄 **Solución**

---

2. Square(n) Sum

Te dan un array de enteros. Devuelve la suma de los **cuadrados** de cada número.

Ejemplo: `[1, 2, 2]` → `1 + 4 + 4 = 9`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: un acumulador `suma` y, dentro del bucle, `suma += numero * numero;`.

> 🔄 **Solución**

---

3. Convert number to reversed array of digits

Te dan un número entero no negativo. Devuelve sus dígitos en un array de enteros, **en orden inverso**.

Ejemplo: `35231` → `[1, 3, 2, 5, 3]` · `0` → `[0]`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: para extraer dígitos, `% 10` te da el último y `/ 10` se lo quita. Con eso ya salen invertidos: el primero que sacas es el último del número.

> 🔄 **Solución**

---

4. Count the divisors of a number

Cuenta cuántos divisores tiene un número `n` (positivo).

Ejemplos: `divisors(4)` → `3` (1, 2, 4) · `divisors(12)` → `6` (1, 2, 3, 4, 6, 12).

- Enunciado en CodeWars
- Dificultad: 7 kyu

Pista: un bucle del 1 a `n` con un `if (n % i == 0)` y un contador. (Optimización opcional: solo llegas a la raíz cuadrada y cuentas de dos en dos.)

> 🔄 **Solución**

AceptaElReto

5. 185 — Potitos

Un padre le compra potitos (botes de comida para bebé) a su hijo. A veces el bebé **le da la vuelta** al bote para ver el reverso, donde se lee el ingrediente *que menos le gusta* (o "FIN" si no hay ninguno). El padre quiere saber los ingredientes que el bebé **odia**, para dejar de comprarlos.

Entrada: varios casos de prueba. Cada caso empieza con `N` (número de potitos). A continuación, `N` líneas: cada una con un ingrediente `I` que le gusta, un guion `--`, y opcionalmente un `FIN` o un ingrediente que detesta. El caso `0` termina.

Ejemplo:

```
4
pollo--FIN
pescado--FIN
verdura--atun
pollo--FIN
0
```

Salida:

```
atun
```

(El único detestado es `atun`; los demás no tenían reverso o decían `FIN`.) Los ingredientes detestados se muestran **ordenados alfabéticamente** y **sin repetir**.

- Enunciado en AceptaElReto
- Dificultad: Fácil

Pista: para cada línea, separa con `split("--")`. Si la segunda parte no es `"FIN"`, es un ingrediente detestado: guárdalo. Al final, ordénalo (la bombolla del punto 4, pero con `String.compareTo` en vez de `>`) y elimina los repetidos.

> 🔄 **Solución**

---

6. 342 — ¡No lo puedes saber!

Un jugador piensa un número oculto en un rango `[ini, fin]` y un segundo jugador hace preguntas ("¿El 500?") recibiendo respuestas **"menor"** o **"mayor o igual"**. Dado el rango, el número oculto y la lista de preguntas realizadas, di si el segundo jugador puede **saber con certeza** cuál es el número.

Entrada: varios casos. Cada caso: una línea con `ini`, `fin` y `n` (el número oculto). La siguiente línea tiene `k` (no de preguntas) y las `k` hipótesis. Termina con `0 0 0`.

Ejemplo:

```
1 10 1
2 1 2
1 1000 450
2 400 500
0 0 0
```

Salida:

```
LO SABE
NO LO SABE
```

En el primer caso, el jugador pregunta por 1 y 2: responde “mayor o igual” (1) y “menor” (2), así que solo queda el 1 → LO SABE. En el segundo, con preguntas 400 y 500 solo acota a 450, 451… 500 → aún quedan varios → NO LO SABE.

- Enunciado en AceptaElReto
- Dificultad: Media

Pista: es la **búsqueda binaria** del punto 3 al revés: cada pregunta divide el rango posible en dos según la respuesta. Lleva `min` y `max` (los límites posibles). Si la hipótesis es menor que el oculto, `min` sube a hipótesis + 1; si es mayor o igual, `max` baja a la hipótesis. Al final, LO SABE si `min == max == n`.

> 🔄 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines cerchas, bombollas y Big O, vuelve a los problemas de esta unidad con el punto 9 como examen. Y en la U08 te esperan las bestias: QuickSort, MergeSort y la recursividad. Ordenar un millón de datos en un suspiro está a un paso.

Boletín U08 — Inicial

📝 Boletín U08 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, respira hondo y recuerda: para entender la recursividad, primero tienes que entender la recursividad.

Ejercicio 1: El factorial del valiente

Escribe un programa llamado `FactorialValiente` con un método recursivo `static long fact(int n)` que calcule el factorial de un número. Pruébalo con `fact(5)`, `fact(0)` y `fact(10)`.

Recuerda: `0! = 1` y `n! = n * (n-1)!`.

Ejercicio 2: La abuela suma el array

Escribe un programa llamado `SumaRecursiva` que sume todos los elementos de un array usando un método recursivo:

```
static int sumar(int[] arr, int indice)
```

Si `indice == arr.length`, devuelve 0. Si no, `arr[indice] + sumar(arr, indice + 1)`. Pruébalo con `{3, 8, 2, 10, 5}` → debe dar 28.

Ejercicio 3: ¿Qué imprime? — la cuenta atrás recursiva

Sin ejecutar, escribe la salida exacta de este programa:

```java
public class CuentaRara {
    static void cuenta(int n) {
        if (n == 0) {
            System.out.println("¡BOOM!");
            return;
        }
        System.out.println(n);
        cuenta(n - 1);
    }

    public static void main(String[] args) {
        cuenta(3);
    }
}
```

Pista: primero baja imprimiendo números y, cuando llega al caso base, hace BOOM.

Ejercicio 4: La potencia del aprendiz

Escribe un programa llamado `PotenciaRecursiva` con un método recursivo `static double potencia(double base, int exponente)` que calcule `base^exponente`:

- Caso base: `exponente == 0` → devuelve 1.
- Caso recursivo: `base * potencia(base, exponente - 1)`.

Pruébalo con `potencia(2, 10)` → 1024.

Ejercicio 5: El detective de dígitos

Escribe un programa llamado `ContadorDigitos` con un método recursivo `static int contarDigitos(int n)` que cuente cuántos dígitos tiene un número:

- Caso base: `n < 10` → devuelve 1.
- Caso recursivo: `1 + contarDigitos(n / 10)`.

Pruébalo con `12345` → debe dar 5.

Ejercicio 6: El espejo de letras

Escribe un programa llamado `InvertirTexto` con un método recursivo `static String invertir(String s)` que devuelva el texto al revés:

- Caso base: `s.isEmpty()` → devuelve `""`.
- Caso recursivo: `invertir(s.substring(1)) + s.charAt(0)`.

Pruébalo con `"hola"` → debe devolver `"aloh"`.

Ejercicio 7: ¿Qué imprime? — el enigma del stack

Sin ejecutar, escribe la salida exacta:

```
public class EnigmaStack {
    static int misterio(int n) {
        if (n == 0) return 0;
        return n % 10 + misterio(n / 10);
    }

    public static void main(String[] args) {
        System.out.println(misterio(1234));
    }
}
```

Pista: `n % 10` extrae la última cifra y `n / 10` se la quita.

Ejercicio 8: El guardián de los palíndromos

Escribe un programa llamado `GuardianPalindromos` con un método recursivo que compruebe si una palabra es un palíndromo (se lee igual hacia delante y hacia atrás):

```
static boolean esPalindromo(String s, int inicio, int fin)
```

Pruébalo con `"reconocer"` (sí), `"salas"` (sí) y `"hola"` (no). Pista: si las puntas no coinciden, ya puedes devolver `false` sin seguir recursando.

Ejercicio 9: CodeWars — Factorial

Resuelve la kata **“Factorial”** (7 kyu) en CodeWars.

Crea el método `public static long factorial(int n)` que devuelva el factorial de `n` (recuerda: `0! = 1`). ¡Resuélvela con recursividad, que es lo que toca!

Pista: `if (n <= 1) return 1; return n * factorial(n - 1);`

Boletín U08 — Inicial (Resuelto)

📝 Boletín U08 — Inicial (Resuelto)

> Las soluciones están escondidas detrás de cada 🔄. Primero intenta resolverlos tú; mirar la respuesta antes de intentarlo es como copiar en un examen de magia: el truco se estropea.

Ejercicio 1: El factorial del valiente

> › 🔄 Solución

Ejercicio 2: La abuela suma el array

> › 🔄 Solución

Ejercicio 3: ¿Qué imprime? — la cuenta atrás recursiva

> › 🔄 Solución

Ejercicio 4: La potencia del aprendiz

> › 🔄 Solución

Ejercicio 5: El detective de dígitos

> › 🔄 Solución

Ejercicio 6: El espejo de letras

> › 🔄 Solución

Ejercicio 7: ¿Qué imprime? — el enigma del stack

> Solución

Ejercicio 8: El guardián de los palíndromos

> Solución

Ejercicio 9: CodeWars — Factorial

> Solución

Boletín U08 — Avanzado

📝 Boletín U08 — Avanzado

> La zona intermedia: aquí ya no basta con copiar el patrón, hay que pensar. Cada ejercicio trae su **Pista** para cuando lleves 10 minutos dándole vueltas.

Ejercicio 1: ⭐ Fibonacci con el contador chivato

Crea un programa `FiboContador` que calcule el enésimo Fibonacci con la versión ingenua recursiva PERO que además cuente **cuántas llamadas hace en total**.

```
static long llamadas;   // variable estática que suma 1 en cada llamada
static long fibo(int n)
```

Pruébalo con `fibo(30)`. ¿Cuántas llamadas hace? ¿Y cuántas haría si solo sumaras los índices `fibo(30) + fibo(29) + ... + 1`?

Pista: súmale 1 a `llamadas` como primera línea del método, antes de cualquier caso base.

Ejercicio 2: ⭐ ⭐ ¿Qué imprime? — el árbol de llamadas

Sin ejecutar, escribe la salida exacta y el **orden de las llamadas** (quién llama a quién):

```
public class ArbolRaro {
    static void pintar(int n) {
        if (n == 0) return;
        System.out.println("bajo " + n);
        pintar(n - 1);
        System.out.println("subo " + n);
    }

    public static void main(String[] args) {
        pintar(3);
    }
}
```

Pista: el `System.out.println` de después de la llamada recursiva no se ejecuta hasta que esa llamada termina. Dibuja la pila con lápiz y papel.

Ejercicio 3: ⭐ ⭐ El palíndromo rebelde

Amplía el palíndromo para que **ignore espacios, signos y mayúsculas**:

- `"Anita lava la tina"` → true
- `"La ruta natural"` → true
- `"No soy un palíndromo"` → false

```
static boolean esPalindromoFrase(String s, int inicio, int fin)
```

Pista: en vez de comparar directamente `s.charAt(inicio)`, saltate los caracteres que no sean letras avanzando `inicio` o retrocediendo `fin` en el propio método. Usa `Character.isLetter()` y `Character.toLowerCase()`.

Ejercicio 4: ⭐ ⭐ La potencia exprés (divide y vencerás)

Escribe un programa `PotenciaRapida` con un método recursivo `static long potenciaRapida(int base, int exponente)` que calcule `base^exponente` en **O(log n)**:

```
potenciaRapida(b, e):
  si e == 0 → 1
  mitad = potenciaRapida(b, e / 2)
  si e es par  → mitad * mitad
  si e es impar → mitad * mitad * b
```

Pruébalo con `potenciaRapida(2, 20)` → 1048576. ¿Cuántas llamadas hace comparado con `potencia(2, 20)` del boletín inicial?

Pista: ojo con el redondeo: cuando `e` es impar, `e / 2` se queda con la parte entera y por eso multiplicas por `b` una vez más.

Ejercicio 5: ⭐ ⭐ ⭐ Quicksort con mediana de tres

Mejora el Quicksort de la unidad: en vez de coger el primer elemento como pivote, elige la **mediana de tres** (primero, medio y último) para evitar el peor caso con arrays casi ordenados.

```
static void quicksort(int[] arr, int inicio, int fin)
```

Pruébalo con el array ya ordenado `{1, 2, 3, 4, 5, 6, 7, 8}`. ¿Cuántas particiones hace tu versión con mediana de tres?

Pista: `int medio = (inicio + fin) / 2;` compara `arr[inicio]`, `arr[medio]` y `arr[fin]` y coloca el del medio en `arr[inicio]` (intercambiándolos) antes de particionar con la técnica de la unidad.

Ejercicio 6: ⭐ ⭐ ⭐ Mergesort con el contador de comparaciones

Modifica el Mergesort de la unidad para que cuente **cuántas comparaciones** hace la fusión en total, y muestra el número al final.

```
static long comparaciones;   // suma 1 en cada comparación de la fusión
```

Pruébalo con `{9, 8, 7, 6, 5, 4, 3, 2, 1}` (el peor caso visual). ¿Cuántas comparaciones? ¿Y con `{1, 2, 3, 4, 5, 6, 7, 8, 9}`?

Pista: en la fusión, suma 1 en cada `while` que compara dos elementos. En los bucles donde una lista ya se ha acabado, también hay comparaciones contra el final del array: cuéntalas con la misma variable.

Ejercicio 7: ⭐ ⭐ ⭐ las torres de Hanói con contador

Implementa las Torres de Hanói recursivas de la unidad y añade un **contador de movimientos**:

```
static int movimientos;
static void hanoi(int n, char origen, char destino, char auxiliar)
```

Cada vez que se mueva un disco, imprime `"Mueve disco X de ORIGEN a DESTINO"` y suma 1 al contador. Al final, imprime el total. Pruébalo con 3, 4 y 8 discos.

Pista: con `n` discos el mínimo de movimientos es `2^n - 1`. Si tu contador con 8 discos no da 255, algo estás moviendo de más.

Ejercicio 8: ⭐ ⭐ ⭐ CodeWars — Sort Numbers

Resuelve la kata **"Sort Numbers"** (7 kyu) en CodeWars.

Crea el método `public static int[] sortArray(int[] nums)` que devuelva el array ordenado de menor a mayor. Si `nums` es `null` o está vacío, devuelve un array vacío.

Pista: haz un método `int[] copia` (para no mutar el original) y ordénalo con `Arrays.sort()`. O, si te sientes con ganas, implementa tu propio Mergesort sobre la copia.

Ejercicio 9: ⭐ ⭐ ⭐ AceptaElReto — 104 Móviles

Resuelve el problema **"Móviles" (104)** de AceptaElReto.

Un móvil cuelga de una barra con dos pesos `izq` y `der` a distancias `dizq` y `dder`. Está equilibrado si `izq * dizq == der * dder` y además las dos subbarras (que pueden contener otros móviles) también lo están. Un peso `0` significa que ahí cuelga otro móvil, que se describe a continuación. Devuelve `SI` si el móvil está equilibrado y `NO` en caso contrario.

Pista: no hay un caso base con `n`: los móviles se leen recursivamente. Lee `izq dizq der dder`; si `izq == 0`, hay que leer (y comprobar) un submóvil completo antes de continuar; si `der == 0`, otro. Recuerda que **ambos** submóviles deben estar equilibrados, no solo la barra.

Boletín U08 — Avanzado (Resuelto)

📝 Boletín U08 — Avanzado (Resuelto)

> Los ejercicios avanzados con sus soluciones. Inténtalos primero; bajar a ver la solución sin sudar es como pedir el 10 antes del examen.

Ejercicio 1: ⭐ Fibonacci con el contador chivato

> › 🔄 Solución

Ejercicio 2: ⭐ ⭐ ¿Qué imprime? — el árbol de llamadas

> › 🔄 Solución

Ejercicio 3: ⭐ ⭐ El palíndromo rebelde

> › 🔄 Solución

Ejercicio 4: ⭐ ⭐ La potencia exprés (divide y vencerás)

> › 🔄 Solución

Ejercicio 5: ⭐ ⭐ ⭐ Quicksort con mediana de tres

> › 🔄 Solución

Ejercicio 6: ⭐ ⭐ ⭐ Mergesort con el contador de comparaciones

> 🔄 Solución

Ejercicio 7: ⭐ ⭐ ⭐ las torres de Hanói con contador

> 🔄 Solución

Ejercicio 8: ⭐ ⭐ ⭐ CodeWars — Sort Numbers

> 🔄 Solución

Ejercicio 9: ⭐ ⭐ ⭐ AceptaElReto — 104 Móviles

> 🔄 Solución

Boletín U08 — Extras

📝 Boletín U08 — Extras

> ¿Te has quedado con ganas de más? Esta sección es para los que quieren llegar al examen con el stack templado: 4 katas de CodeWars y 2 problemas de AceptaElReto. Cada pista está escondida en su desplegable para no arruinar el misterio.

 CodeWars

> Regístrate en CodeWars, elige Java y resuelve las katas por orden. Empiezan asequibles y suben de nivel.

Kata 1: Factorial (7 kyu)

En CodeWars.

En matemáticas, el factorial de un entero no negativo `n`, denotado `n!`, es el producto de todos los enteros positivos menores o iguales que `n`:

```
n! = 1 * 2 * 3 * ... * n
```

Con la convención de que `0! = 1`. Implementa `public static long factorial(int n)` con recursividad.

> › 💡 **Pista**

Kata 2: Descending Order (7 kyu)

En CodeWars.

Toma un número no negativo y devuelve el **número mayor posible** reordenando sus dígitos de mayor a menor. Ejemplos:

- 42145 → 54421
- 145263 → 654321
- 123456789 → 987654321

> 💡 Pista

Kata 3: Are the numbers in order? (7 kyu)

En CodeWars.

Escribe `public static boolean isAscOrder(int[] arr)` que devuelva `true` si los números están **estrictamente en orden ascendente** (cada uno mayor que el anterior) y `false` en caso contrario.

- `[1, 2, 3, 4]` → true
- `[1, 4, 13, 97, 508, 717]` → true
- `[1, 3, 2]` → false
- `[9, 5, 5]` → false (no es estrictamente ascendente)

> 💡 Pista

Kata 4: Tribonacci Sequence (6 kyu)

En CodeWars.

Igual que Fibonacci pero con tres números de arranque: cada término es la suma de los tres anteriores. El enunciado siempre da una semilla `double[] signature` de 3 elementos y un número `n`. Devuelve los primeros `n` términos:

- `{1, 1, 1}, 10` → `{1, 1, 1, 3, 5, 9, 17, 31, 57, 105}`
- `{0, 0, 1}, 10` → `{0, 0, 1, 1, 2, 4, 7, 13, 24, 44}`
- `{1, 1, 1}, 1` → `{1}` (recuerda: si `n < 3`, solo devuelves los que pida)

> 💡 Pista

🏆 AceptaElReto

> Dos problemas del concurso ProgramaMe. El primero es un clásico absoluto de la recursividad; el segundo, un sudorífero de ordenación.

Reto 1: Móviles (104)

En AceptaElReto.

Un móvil colgante se describe con cuatro enteros: `pi di pd dd` (peso y distancia izquierdos, peso y distancia derechos). La barra está equilibrada si `pi * di == pd * dd`. Un peso `0` indica que ahí cuelga otro móvil, que se describe a continuación en la entrada. Debes decir `SI` si el móvil completo está equilibrado (barra Y submóviles) y `NO` en caso contrario. La entrada termina con `0 0 0 0`.

> › 💡 **Pista**

Reto 2: Tortitas (143)

En AceptaElReto.

Tienes una pila de tortitas con tamaños distintos. Con una espátula puedes dar la vuelta a las `k` tortitas superiores (invertir el orden de las primeras `k`). Encuentra el mínimo número de vueltas para ordenarlas de mayor (arriba) a menor (abajo). La entrada termina con `0`. Para cada caso, imprime cuántas vueltas necesitas.

> › 💡 **Pista**

Boletín U09 — Inicial

📝 Boletín U09 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, enciende la fábrica de galletas y haz que tus objetos nazcan, hablen y funcionen. Nadie nace sabiendo cuándo usar `this`.

Ejercicio 1: El perro que habla

Escribe una clase `Perro` con dos atributos: `String nombre` e `int edad`. Incluye un método `void ladrar()` que imprima `"¡Guau guau! Soy [nombre]"`. Luego, en un `main`, crea un perro llamado `Toby` de 3 años y haz que ladre.

Pista: recuerda que los atributos se inicializan en el constructor con `this`.

Ejercicio 2: El teléfono con batería

Escribe una clase `Telefono` con los atributos `String marca` e `int bateria`. Incluye un método `void llamar()` que imprima `"Llamando... (batería al X%)"` y otro `void cargar(int minutos)` que suba la batería en los minutos recibidos. Crea en el `main` un `Telefono` de marca `"Nokia"` con batería inicial `50`, llámalo y cárgalo `30` minutos.

Pista: `cargar(int minutos)` modifica el atributo `bateria`: `this.bateria += minutos;`.

Ejercicio 3: El estudiante con nota

Escribe una clase `Estudiante` con los atributos `String nombre` y `double nota`. Incluye un método `void mostrarEstado()` que imprima `"[nombre] ha aprobado"` si la nota es mayor o igual que 5, o `"[nombre] ha suspendido"` si es menor. Crea en el `main` dos estudiantes (una con nota 8.5 y otro con nota 3) y muestra su estado.

Pista: dentro de `mostrarEstado()` puedes usar `if/else` con `this.nota`.

Ejercicio 4: La película en cartelera

Escribe una clase `Pelicula` con los atributos `String titulo` y `String genero`. Incluye un método `void mostrarCartelera()` que imprima `"Ahora en cines: [titulo] ([genero])"`. Crea en el `main` una película `"El curso de Java"` de género `"Terror"` y muéstrala en cartelera.

Pista: el constructor recibe los dos valores y los asigna con `this`.

Ejercicio 5: La persona educada

Escribe una clase `Persona` con los atributos `String nombre` y `int edad`. Usa un constructor con parámetros. Añade un método `void presentarse()` que imprima `"Hola, soy [nombre] y tengo [edad] años."`. Crea en el `main` a `"Ana"` de 25 años y haz que se presente.

Pista: esta es la clase con la que vas a entender por qué `this` separa al atributo del parámetro.

Ejercicio 6: El círculo calculador

Escribe una clase `Circulo` con el atributo `double radio`. Incluye un método `double calcularArea()` que devuelva el área (`π * radio²`). Usa `Math.PI` para π. Crea en el `main` un círculo de radio `2.5` y muestra su área.

Pista: dentro del método, `return Math.PI * this.radio * this.radio;`.

Ejercicio 7: El videojuego jugable

Escribe una clase `Videojuego` con los atributos `String titulo` y `int horasJugadas`. Incluye un método `void jugar(int horas)` que sume las horas a `horasJugadas` y otro `void mostrar()` que imprima `"[titulo] - X horas"`. Crea en el `main` un juego con 10 horas, juega 5 más y muéstralo.

Pista: suma las horas al atributo: `this.horasJugadas += horas;`.

Ejercicio 8: ¿Qué imprime? — el taller de coches

Sin ejecutar, escribe la salida exacta de este programa:

```java
public class Taller {
    public static void main(String[] args) {
        Coche c1 = new Coche("Seat", 120);
        Coche c2 = new Coche("Ford", 90);
        c1.acelerar(30);
        c1.mostrar();
        c2.mostrar();
    }
}

class Coche {
    String marca;
    int velocidad;

    public Coche(String marca, int velocidad) {
        this.marca = marca;
        this.velocidad = velocidad;
    }

    void acelerar(int inc) {
        velocidad += inc;
    }

    void mostrar() {
        System.out.println(marca + " va a " + velocidad + " km/h");
    }
}
```

Pista: ¿cuántos objetos hay? Cada uno guarda su propia `velocidad`; acelerar al `c1` no toca a `c2`.

Ejercicio 9: CodeWars — Object Oriented Piracy

Resuelve la kata **"Object Oriented Piracy"** (8 kyu) en CodeWars.

Crea la clase `Ship` con un constructor que reciba `draft` (calado en pies) y `crew` (tripulación), y un método `boolean isWorthIt()` que devuelva `true` si `draft - (crew * 1.5) > 20`.

Pista: guarda `draft` y `crew` en atributos con `this` y usa el operador `-` y `*` de la UO3 dentro de `isWorthIt()`.

Boletín U09 — Inicial Resuelto

📝 Boletín U09 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: El perro que habla

> 🔄 Solución

Ejercicio 2: El teléfono con batería

> 🔄 Solución

Ejercicio 3: El estudiante con nota

> 🔄 Solución

Ejercicio 4: La película en cartelera

> 🔄 Solución

Ejercicio 5: La persona educada

> 🔄 Solución

Ejercicio 6: El círculo calculador

> 🔄 Solución

Ejercicio 7: El videojuego jugable

> Solución

Ejercicio 8: ¿Qué imprime? — el taller de coches

> Solución

Ejercicio 9: CodeWars — Object Oriented Piracy

> Solución

Boletín U09 — Avanzado

📝 Boletín U09 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: La biblioteca

Escribe una clase `Libro` con los atributos `String titulo`, `String autor` e `int paginas`. Añade un método `void mostrarInfo()` que imprima:

```
El Quijote, de Miguel de Cervantes (863 páginas)
```

Crea en el `main` dos libros distintos y muestra su información.

Pista: tres atributos, un constructor con tres parámetros y `this.` tres veces. El método usa los atributos para construir la frase.

⭐ Ejercicio 2: El rectángulo razonador

Escribe una clase `Rectangulo` con los atributos `double ancho` y `double alto`. Añade:

- `double calcularArea()` → `ancho * alto`
- `double calcularPerimetro()` → `2 * (ancho + alto)`
- `boolean esCuadrado()` → `true` si `ancho == alto`

Crea un rectángulo de 4 x 4 y otro de 5 x 8 y prueba los tres métodos en ambos.

Pista: los tres métodos devuelven valores con `return`. `esCuadrado()` devuelve el resultado de una comparación con `==`.

⭐ Ejercicio 3: La cuenta bancaria blindada

Escribe una clase `CuentaBancaria` con los atributos `String titular` y `double saldo`. Añade:

- `void ingresar(double cantidad)` → suma a saldo.
- `void retirar(double cantidad)` → resta a saldo **solo si** `cantidad <= saldo`; si no, imprime `"Saldo insuficiente"`.
- `void mostrar()` → imprime `"Titular: X | Saldo: Y €"`.

Crea una cuenta con 100 €, retira 30 €, intenta retirar 200 € y muestra el saldo final.

Pista: dentro de `retirar`, un `if (cantidad <= this.saldo)` protege el saldo de quedarse en negativo. Ese `if` es la diferencia entre una cuenta y un agujero.

⭐ ⭐ Ejercicio 4: La hora que se corrige sola

Escribe una clase `Hora` con los atributos `int hora`, `int minuto` y `int segundo`. El constructor debe validar: si los valores no son válidos (hora entre 0 y 23, minuto y segundo entre 0 y 59), se inicializan a 0. Añade el método `void incrementarSegundo()` que suma 1 segundo gestionando los acarreos: si llega a 60 segundos pasa a 0 y suma un minuto, y así con los minutos y las horas.

Crea una `Hora(23, 59, 59)`, llama a `incrementarSegundo()` y muestra `00:00:00`.

Pista: la validación es un `if` grande en el constructor. El acarreo son tres `if` encadenados, de segundo a minuto a hora. Para mostrar con dos dígitos, imprime `"0" + valor` si es menor que 10.

⭐ ⭐ Ejercicio 5: ¿Qué imprime? — el baile de referencias

Sin ejecutar, escribe la salida exacta:

```java
public class Baile {
    public static void main(String[] args) {
        Punto a = new Punto(3, 4);
        Punto b = a;
        b.x = 10;
        System.out.println("a.x = " + a.x);

        Punto c = new Punto(1, 1);
        cambiar(c);
        System.out.println("c.x = " + c.x);
    }

    static void cambiar(Punto p) {
        p.x = 99;
        p = new Punto(50, 50);
    }
}

class Punto {
    int x;
    int y;

    public Punto(int x, int y) {
        this.x = x;
        this.y = y;
    }
}
```

Pista: `b = a` no copia el objeto: copia la referencia. Cuando `cambiar(c)` recibe `c`, el parámetro `p` es una *copia* de la referencia, así que modificar `p.x` sí se nota, pero `p = new Punto(...)` solo reasigna el parámetro local.

⭐ ⭐ Ejercicio 6: El correo que se encadena

Escribe una clase `Email` con los atributos `String remitente`, `String destinatario` y `String asunto`. Crea **tres constructores sobrecargados**:

- `Email(String remitente, String destinatario, String asunto)` → el completo.
- `Email(String remitente, String destinatario)` → asunto por defecto `"(sin asunto)"`.

- `Email(String remitente)` → destinatario `"(sin destino)"` y asunto `"(sin asunto)"`.

Usa `this(...)` para encadenar y evitar repetir código. Añade `void mostrar()` que imprima los tres datos.

Pista: el constructor de un parámetro llama al de dos, y el de dos llama al de tres. Con `this(...)` escribes la asignación completa una sola vez, en el constructor de tres parámetros.

⭐ ⭐ Ejercicio 7: La fracción que se simplifica

Escribe una clase `Fraccion` con los atributos `int numerador` y `int denominador`. Añade:

- Constructor que valide: si `denominador == 0`, se usa `1`.
- `Fraccion sumar(Fraccion otra)` → devuelve una nueva fracción con `(a.num * b.den + b.num * a.den) / (a.den * b.den)`.
- `void simplificar()` → divide numerador y denominador por su máximo común divisor (MCD).

Crea `1/2` y `1/3`, súmalas y simplifica el resultado.

Pista: para el MCD usa el algoritmo de Euclides (restas o módulos) o `Math.abs` con un bucle. `simplificar()` no devuelve nada: modifica `this`.

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — Building blocks

Resuelve la kata **“Building blocks”** (7 kyu) en CodeWars.

Crea la clase `Block` con un constructor que reciba las tres dimensiones (como `int[]` de 3 o como 3 enteros) y los métodos:

- `int getWidth()`, `int getLength()`, `int getHeight()`
- `int getVolume()` → `width * length * height`
- `int getSurfaceArea()` → `2 * (w*l + w*h + l*h)`

Pista: guarda las tres dimensiones en atributos con `this` y deja que los getters simplemente las devuelvan. La superficie es la suma de las caras por dos.

⭐ ⭐ ⭐ Ejercicio 9: CodeWars — Object Oriented Piracy

Resuelve la kata **"Object Oriented Piracy"** (8 kyu) en CodeWars.

Crea la clase `Ship` que recibe un `draft` (calado en unidades) y un `crew` (tripulantes). Implementa:

- `getDraft()` y `getCrew()` → los getters de siempre.
- `isWorthIt()` → devuelve `true` si el barco merece la pena saquearlo: el calado total menos `1.5` por cada tripulante debe superar 20.

```
Ship titanic = new Ship(15, 10);
titanic.isWorthIt(); // false: 15 - 1.5*10 = 0, no merece la pena
```

Pista: un getter devuelve el atributo tal cual; `isWorthIt()` combina ambos: `return draft - 1.5 * crew > 20;`. El `1.5` es una constante con nombre, mejor que un número suelto.

📚 Referencias

Plataforma	Problema	Dificultad
AceptaElReto	148 — Nochevieja	Fácil
CodeWars	Object Oriented Piracy (8 kyu)	Principiante
CodeWars	Building blocks (7 kyu)	Aficionado
CodeWars	FIXME: Get Full Name (7 kyu)	Aficionado

Boletín U09 — Avanzado Resuelto
📝 Boletín U09 — Avanzado (Resuelto)

> Las soluciones están ocultas. Inténtalo de verdad antes de destaparlas.

⭐ Ejercicio 1: La biblioteca

> 🔄 Solución

⭐ Ejercicio 2: El rectángulo razonador

> 🔄 Solución

⭐ Ejercicio 3: La cuenta bancaria blindada

> 🔄 Solución

⭐ ⭐ Ejercicio 4: La hora que se corrige sola

> 🔄 Solución

⭐ ⭐ Ejercicio 5: ¿Qué imprime? — el baile de referencias

> 🔄 Solución

⭐ ⭐ Ejercicio 6: El correo que se encadena

> 🔄 Solución

⭐ ⭐ Ejercicio 7: La fracción que se simplifica

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 8: CodeWars — Building blocks

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 9: CodeWars — Object Oriented Piracy

> 🔄 Solución

> 📚 **¿Quieres más?** Revisa el boletín de **extras** de esta unidad: tiene cuatro katas de CodeWars (incluida una de depuración de una clase) y dos problemas de AceptaElReto.

Boletín U09 — Extras

📝 Boletín U09 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. La solución está oculta: resístete hasta agotar tu pista.

CodeWars

1. FIXME: Get Full Name

Te dan una clase `Dinglemouse` con un método `getFullName()` que debería devolver `"Nombre Apellido"`, pero el resultado está al revés o incompleto. Arréglala.

- Enunciado en CodeWars
- Dificultad: 7 kyu

Pista: fíjate en qué atributos existen (`firstName` y `lastName`) y en cómo el método los concatena. La clase está hecha para romperse: busca el error de `this` o de orden.

> 🔄 **Solución**

---

2. Geometry Basics: Distance between points in 2D

Te dan una clase `Point` con dos propiedades (`x` e `y`) y tienes que implementar el método `distanceBetweenPoints(Point a, Point b)` que devuelva la distancia entre ellos.

Ejemplo: `Point(3, 3)` y `Point(3, 3)` → `0`. `Point(1, 6)` y `Point(4, 2)` → `5`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: distancia euclidiana `Math.sqrt(Math.pow(a.x - b.x, 2) + Math.pow(a.y - b.y, 2))`. Los objetos se pasan como parámetro, como hicimos con `Fraccion.sumar`.

> 🔄 **Solución**

---

3. Grasshopper — Personalized Message

Crea el método `greet(name, owner)` que devuelva `"Hello boss"` si `name` y `owner` son iguales, o `"Hello guest"` si son distintos.

Ejemplos: `greet("Daniel", "Daniel")` → `"Hello boss"`, `greet("Greg", "Daniel")` → `"Hello guest"`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: compara `String` con `.equals()`, nunca con `==`. Un `if/else` o un ternario.

> 🔄 **Solución**

---

4. Sleigh Authentication

Tienes que completar la clase `Sleigh` con un método `authenticate(name, password)` que devuelva `true` solo si `name` es `"Santa Claus"` y `password` es `"Ho Ho Ho!"`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: devuelve `name.equals("Santa Claus") && password.equals("Ho Ho Ho!")`. Otra vez `.equals()`, y el `&&` de la UO4.

> 🔄 **Solución**

AceptaElReto

5. 148 — Nochevieja

Ramón se pasa el día de Nochevieja contando los minutos que faltan para medianoche. Para cada hora de la entrada (formato `HH:MM`), di cuántos minutos faltan para las `00:00`. La entrada termina con `00:00`, que no se procesa.

Entrada:

```
23:45
21:30
00:01
00:00
```

Salida:

```
15
150
1439
```

- Enunciado en AceptaElReto
- Dificultad: Fácil

Pista: separa la hora y el minuto con `split(":")`. Los minutos que faltan son `(23 - hora) * 60 + (60 - minuto)`. Escribe una clase `Hora` con un método `minutosHastaMedianoche()` y verás lo natural que queda.

> 🔄 **Solución**

---

6. 117 — La fiesta aburrida

Tinín odia saludar a desconocidos. La entrada empieza con un número N (cuánta gente hay) y luego N líneas con el formato `"Soy Lotario"`. Para cada una, imprime `"Hola, Lotario."`.

Entrada:

```
3
Soy Lotario
Soy Aldonza
Soy Ender
```

Salida:

```
Hola, Lotario.
Hola, Aldonza.
Hola, Ender.
```

- Enunciado en AceptaElReto

- Dificultad: Fácil

Pista: `split(" ")` sobre cada línea y coge la segunda parte (índice 1). Puedes modelar cada persona como un objeto de una clase `Persona` con el nombre y un método `saludar()`.

> › 🔄 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines las clases, el paso natural es proteger sus datos: eso es la **encapsulación** de la U10. Y si quieres ver objetos que se crean y destruyen solos, espérate a los arrays de la U06 y las colecciones de la U12, donde crearás decenas de objetos en un bucle.

Boletín U10 — Inicial

📝 Boletín U10 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, ponle cadenato a tu primera clase y haz que el `static` deje de parecer magia. Ningún atributo nace sabiendo ser `private`.

Ejercicio 1: La casa de cristal

Esta clase es una casa de cristal: todo el mundo puede entrar y tocar lo que quiera.

```
public class Persona {
    public String nombre;
    public int edad;
}
```

Convierte los atributos en `private` y añade getters y setters para `nombre` y `edad`. Escribe también un `main` que cree una `Persona`, le ponga “Ana” y 25 años usando los setters, y luego muestre los valores con los getters.

Ejercicio 2: El coche del vecindario

Parte de esta clase:

```
public class Coche {
    private String marca;
    private double velocidad;

    public Coche(String marca) {
        this.marca = marca;
        this.velocidad = 0;
    }

    public String getMarca() {
        return marca;
    }
}
```

Añade `getVelocidad()` y un `setVelocidad(double velocidad)` que **rechace** las velocidades negativas y las superiores a 200 (mostrando "Velocidad inválida."). Escribe un `main` que cree un coche "Seat" e intente ponerle -50 y luego 120.

Ejercicio 3: El termómetro con cerebro

Escribe una clase `Termometro` con:

- Atributo `private double temperatura`.
- Constructor que inicialice la temperatura a 20.0.
- Getter `getTemperatura()`.
- Setter `setTemperatura(double temperatura)` que **solo** acepte valores entre -273.15 y 100.0. Si el valor no es válido, muestra "Temperatura fuera de rango.".

En un `main`, crea un termómetro, ponle -500 (debe rechazarse) y luego 36.5, y muestra la temperatura final.

Ejercicio 4: getter sin setter

Esta clase representa la configuración de una app: el idioma se elige al crearla y **no debe poder cambiarse después**.

```
public class Configuracion {
    private String idioma;

    public Configuracion(String idioma) {
        this.idioma = idioma;
    }

    public String getIdioma() {
        return idioma;
    }
}
```

Fíjate: tiene getter pero **no** setter. Escribe un `main` que cree una configuración con "es" y muestre el idioma. Responde: ¿por qué no tiene setter? ¿Qué le pasaría a un usuario que intentase hacer `config.idioma = "va"` desde fuera?

Ejercicio 5: El contador de la clase

Escribe una clase `Contador` con:

- Atributo `public static int total = 0;`.
- Un constructor que incremente `total` en 1 cada vez que se crea un objeto.

En un `main`, crea 3 objetos `Contador` y muestra `Contador.total`. Responde: ¿cuánto vale `total` y por qué no es 1?

Ejercicio 6: La calculadora sin pilas

Escribe una clase `Utilidades` con dos métodos **estáticos**: `sumar(int a, int b)` y `restar(int a, int b)`. En un `main`, usa ambos métodos **sin crear ningún objeto** (usa el nombre de la clase) y muestra los resultados de `sumar(5, 3)` y `restar(10, 4)`.

Pista: `int resultado = Utilidades.sumar(5, 3);`

Ejercicio 7: las constantes del barrio

Escribe una clase `Constantes` que contenga:

- `public static final double IVA = 0.21;`
- `public static final int MAX_INTENTOS_LOGIN = 3;`
- `public static final String NOMBRE_APP = "GestionCurso";`

En un `main`, muestra las tres constantes. Después, intenta hacer `Constantes.IVA = 0.5;` y responde: ¿qué ocurre al compilar?

Ejercicio 8: ¿Qué imprime? — el puzle estático

Sin ejecutar, escribe la salida exacta de este programa:

```java
public class Puzle {
    static int s = 0;
    int i = 0;

    public Puzle() {
        s++;
        i = s;
    }

    public static void main(String[] args) {
        Puzle p1 = new Puzle();
        Puzle p2 = new Puzle();
        System.out.println(p1.i + " " + p2.i + " " + Puzle.s);
    }
}
```

Pista: `s` es de la clase (una sola copia compartida); `i` es de cada objeto y se copia del valor de `s` en el momento de nacer.

Ejercicio 9: CodeWars — Square(n) Sum

Resuelve la kata **"Square(n) Sum"** (8 kyu) en CodeWars.

Completa el método `public static int squareSum(int[] n)` que reciba un array de enteros y devuelva la suma de los cuadrados de cada número.

Ejemplo: `[1, 2, 2]` → `1 + 4 + 4 = 9`.

Pista: usa un `for` que acumule `n[i] * n[i]`. Los arrays se recorren con la UO6, pero ya puedes usar el índice.

Boletín U10 — Inicial Resuelto

📝 Boletín U10 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: La casa de cristal

> 🔄 Solución

Ejercicio 2: El coche del vecindario

> 🔄 Solución

Ejercicio 3: El termómetro con cerebro

> 🔄 Solución

Ejercicio 4: getter sin setter

> 🔄 Solución

Ejercicio 5: El contador de la clase

> 🔄 Solución

Ejercicio 6: La calculadora sin pilas

> 🔄 Solución

Ejercicio 7: las constantes del barrio

> Solución

Ejercicio 8: ¿Qué imprime? — el puzle estático

> Solución

Ejercicio 9: CodeWars — Square(n) Sum

> Solución

Boletín U10 — Avanzado

📝 Boletín U10 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: empleado con validación

Escribe una clase `Empleado` con:

- Atributos `private String nombre`, `private double salario`.
- Constructor que valide que el salario no sea negativo (si lo es, lo deja en 0 y muestra un aviso).
- Getters y setters. El setter de salario también debe rechazar negativos.

En un `main`, crea un empleado "Laura" con salario 1500, intenta ponerle -300 y muestra el salario final.

Pista: reutiliza la validación del setter dentro del constructor: `setSalario(salario)` en vez de asignar a pelo.

⭐ Ejercicio 2: Círculo encapsulado

Escribe una clase `Circulo` con:

- Atributo `private double radio`.
- Constructor que reciba el radio.
- Getter `getRadio()`, setter `setRadio(double)` que rechace radios negativos o cero.
- Métodos `getArea()` y `getPerimetro()` que usen `Math.PI`.

En un `main`, crea un círculo de radio 5 y muestra su área y perímetro.

Pista: `area = Math.PI * radio * radio;` y `perimetro = 2 * Math.PI * radio;`. Recuerda que `Math.PI` es una constante estática.

⭐ ⭐ Ejercicio 3: JavaBean Alumno

Escribe una clase `Alumno` siguiendo el estilo JavaBean (el estándar para clases de datos):

- Atributos `private String nombre`, `private int edad`, `private double notaMedia`.
- Constructor sin parámetros y constructor con los tres valores.
- Getter y setter para **cada** atributo.
- El setter de `edad` debe rechazar edades fuera de 0 y 120; el de `notaMedia`, fuera de 0 y 10.

En un `main`, crea un alumno con el constructor completo y luego modifica su nota con el setter.

Pista: un JavaBean es "atributos privados + getters/setters + constructores": el patrón que verás en cualquier framework. Los setters validan; los getters solo leen.

⭐ ⭐ Ejercicio 4: hora inmutable

Escribe una clase `Hora` que represente una hora del día y que sea **inmutable**: sus atributos solo se asignan en el constructor y no tienen setters.

- Atributos `private final int hora`, `private final int minuto`.
- Constructor que valide `hora` entre 0 y 23 y `minuto` entre 0 y 59.
- Getters `getHora()` y `getMinuto()`.
- Método `mostrar()` que devuelva `"HH:MM"` (con ceros: `09:05`).

En un `main`, crea una hora 9:05 y muéstrala. Responde: ¿por qué no necesita setters?

Pista: usa `String.format("%02d:%02d", hora, minuto)` o `"0" + ...` cuando el valor sea menor que 10.

⭐ ⭐ Ejercicio 5: ¿Qué imprime? — el puzle de los gatos

Sin ejecutar, escribe la salida exacta:

```java
public class Gato {
    public static int totalGatos = 0;
    private String nombre;
    private int vidas;

    public Gato(String nombre) {
        this.nombre = nombre;
        this.vidas = 9;
        totalGatos++;
    }

    public void perderVida() {
        if (vidas > 0) {
            vidas--;
        }
    }

    public String toString() {
        return nombre + " (" + vidas + " vidas)";
    }
}
```

```java
public class PuzleGatos {
    public static void main(String[] args) {
        Gato g1 = new Gato("Bigotes");
        Gato g2 = new Gato("Garfield");
        g1.perderVida();
        g1.perderVida();
        g2.perderVida();

        System.out.println(g1);
        System.out.println(g2);
        System.out.println("Total: " + Gato.totalGatos);
    }
}
```

Pista: cada gato nace con 9 vidas y las pierde una a una con `perderVida()`. Cuenta cuántas veces se llama sobre cada gato, y recuerda que `totalGatos` es `static`.

⭐ ⭐ Ejercicio 6: contador de usuarios

Escribe una clase `Usuario` que asigne a cada objeto un `id` **único y automático**:

- Atributo `private static int contador = 0;` y `private int id;`.
- Constructor que incremente `contador` y asigne `id = contador`.
- Método `public static int getTotalUsuarios()`.
- Getter `getId()`.

En un `main`, crea 5 usuarios y muestra el id del último y el total de usuarios.

Pista: el patrón del punto 5: el `static` cuenta cuántos se han creado, y cada objeto se "congela" su número al nacer. `getTotalUsuarios` es estático porque pregunta a la clase, no a un objeto.

⭐ ⭐ ⭐ Ejercicio 7: La clase utilitaria OperacionesArray

Escribe una clase `OperacionesArray` **utilitaria** (constructor privado) con estos métodos `static`:

- `suma(int[] numeros)` → suma todos los elementos.
- `media(double[] numeros)` → devuelve el promedio.
- `maximo(int[] numeros)` → devuelve el mayor.
- `estaOrdenado(int[] numeros)` → `true` si cada elemento es mayor o igual que el anterior.

En un `main`, usa la clase **sin crear ningún objeto** sobre `{3, 1, 4, 1, 5}` y `{1, 2, 3}`.

Pista: el constructor `private OperacionesArray() {}` impide instanciarla, como `Math`. Para `estaOrdenado`, recorre con un `for` y compara cada elemento con el anterior (`numeros[i] < numeros[i - 1]` → no está ordenado).

⭐ ⭐ ⭐ Ejercicio 8: validador de datos

Escribe una clase `Validador` **utilitaria** (constructor privado) con estos métodos `static`:

- `esEmailValido(String email)` → `true` si contiene exactamente una `@` y al menos un `.` después de la `@`.

- `esEdadValida(int edad)` → `true` si está entre 0 y 120.
- `esTextoNoVacio(String texto)` → `true` si no es `null` y no está en blanco.

En un `main`, prueba los tres métodos con casos válidos e inválidos (por ejemplo `"ana@mail.com"`, `"ana@"`, `"hola"`, `-5`, `200`, `null`).

Pista: usa `String.indexOf("@")` para localizar la `@`, `indexOf("@", pos + 1)` para comprobar que no hay una segunda, y `indexOf(".", pos)` para el punto después de la `@`. Un `return` temprano en cada condición fallida simplifica mucho.

⭐ ⭐ ⭐ Ejercicio 9: El gran reto — refactoriza el banco

Este código funciona… pero es una puerta abierta. Refactorízalo siguiendo los pasos del punto 8 (Be the Code):

```
public class CuentaBancaria {
    public String titular;
    public double saldo;

    public CuentaBancaria(String titular, double saldo) {
        titular = titular;
        saldo = saldo;
    }

    public void retirar(double cantidad) {
        saldo = saldo - cantidad;
    }

    public void ingresar(double cantidad) {
        saldo = saldo + cantidad;
    }
}
```

Debe quedar así:

1. Atributos `private`.
2. Constructor con `this` y que valide que el saldo inicial no sea negativo.
3. Getters para ambos; **sin setters** para el saldo.

4. `retirar(double)` que rechace cantidades negativas y que **no permita** dejar el saldo en negativo (si `cantidad > saldo`, avisa y no retira).
5. `ingresar(double)` que rechace cantidades negativas.
6. Una constante `public static final String NOMBRE_BANCO = "Banco DAM";`.

Escribe también un `main` de prueba que cree una cuenta, ingrese, intente retirar más de lo que tiene y muestre el saldo.

Pista: sin setter para el saldo, solo las operaciones `retirar` e `ingresar` pueden tocarlo: es la frontera del negocio. Recuerda el bug del constructor (`titular = titular` se asigna a sí mismo) y valida todo antes de asignar.

Boletín U10 — Avanzado Resuelto

📝 Boletín U10 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

⭐ Ejercicio 1: empleado con validación

> 🔄 Solución

⭐ Ejercicio 2: Círculo encapsulado

> 🔄 Solución

⭐ ⭐ Ejercicio 3: JavaBean Alumno

> 🔄 Solución

⭐ ⭐ Ejercicio 4: hora inmutable

> 🔄 Solución

⭐ ⭐ Ejercicio 5: ¿Qué imprime? — el puzle de los gatos

> 🔄 Solución

⭐ ⭐ Ejercicio 6: contador de usuarios

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 7: La clase utilitaria OperacionesArray

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 8: validador de datos

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 9: El gran reto — refactoriza el banco

> 🔄 Solución

Boletín U10 — Extras

📝 Boletín U10 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. La solución está oculta: resístete hasta agotar tu pista. Los `private`, los getters y el `static` son aquí los protagonistas: encapsula, protege y comparte sin miedo.

CodeWars

1. Playing with cubes II

Te dan un molde de clase `Cube` con un atributo privado `side`. Tu trabajo: construirla con getter y setter, y dos constructores (uno vacío que inicializa `side` a 0, y otro que recibe el lado).

Ejemplo de uso:

```
Cube c = new Cube(3);
c.getSide();  // 3
c.setSide(7);
c.getSide();  // 7
```

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: `side` debe ser `private` (nadie lo toca desde fuera) y los únicos portales son `getSide()` y `setSide()`. El constructor sin parámetros pone `this.side = 0;`. Exactamente lo que has visto en la unidad: encapsulación pura.

> › 🔄 **Solución**

---

2. Classy Extentions

Crea la clase `Pet` con un atributo privado `name` (String) y un método `speak()` que devuelva el nombre del animal. Después crea la clase `Cat` que **hereda** de `Pet` y sobrescribe `speak()` para que devuelva `"[name] meows."`.

Ejemplos:

```
new Cat("Milo").speak();  // "Milo meows."
new Cat("Garfield").speak();  // "Garfield meows."
```

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: `name` es privado, así que `Cat` no puede leerlo directamente: necesita un getter `getName()` en `Pet` (recuerda: los privados no se heredan, pero existen dentro del objeto). `Cat extends Pet` y usa `super(nombre)` para construir la parte del padre.

> Solución

---

3. Sleigh Authentication

Papá Noel tiene un método `authenticate(name, password)` y solo admite dos credenciales: el nombre debe ser `"Santa Claus"` y la contraseña `"Ho Ho Ho!"`. Devuelve `true` solo si ambas coinciden.

Ejemplos: `authenticate("Santa Claus", "Ho Ho Ho!")` → `true`, `authenticate("Santa", "Ho Ho Ho!")` → `false`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: los dos valores son constantes: `private static final String NOMBRE_VALIDO = "Santa Claus";`. Compara con `.equals()`, nunca con `==`. Y nota el `static`: el método no necesita objeto, es pura lógica de clase.

> Solución

---

4. Object Oriented Piracy

Crea la clase `Ship` que recibe un `draft` (calado) y un `crew` (tripulantes). Implementa `isWorthIt()`: devuelve `true` si el calado total menos `1.5` por cada tripulante supera 20.

Ejemplo: `new Ship(15, 10).isWorthIt()` → `false` (`15 - 1.5*10 = 0`).

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: guarda `draft` y `crew` en atributos `private final` (se fijan en el constructor y ya no cambian). El método combina ambos: `return draft - 1.5 * crew > 20;`. El `1.5` merece ser una constante con nombre.

> 🔄 Solución

AceptaElReto

5. 117 — La fiesta aburrida

Tinín odia las fiestas, y cada persona que se le acerca se presenta con el formato `"Soy Lotario"`. Ayúdale a responder `"Hola, [nombre]."` a cada uno. La entrada empieza con un número que indica cuántas personas hay, seguido de una línea por persona. Escribe la solución con una clase `Persona` que guarde el nombre y un método `saludar()`.

Entrada de ejemplo:

```
3
Soy Lotario
Soy Aldonza
Soy Ender
```

Salida de ejemplo:

```
Hola, Lotario.
Hola, Aldonza.
Hola, Ender.
```

- Enunciado en AceptaElReto
- Dificultad: Fácil

Pista: lee la línea, quita el `"Soy "` inicial (`linea.substring(4)` o `split(" ")[1]`), crea la `Persona` con ese nombre y llama a su `saludar()`. Es una excusa perfecta para una clase con un atributo y un método, en vez de un main que lo hace todo.

> 🔄 Solución

6. 117 bis — La fiesta aburrida con contador estático

Reto extra con `static`: usa la clase `Persona` del ejercicio 5, pero añade un atributo `private static int totalSaludos` que cuente cuántas personas ha saludado Tinín en total. Tras cada saludo, muestra el total acumulado.

Entrada de ejemplo:

```
2
Soy Lotario
Soy Ender
```

Salida de ejemplo:

```
Hola, Lotario. (saludos: 1)
Hola, Ender. (saludos: 2)
```

Pista: `static` significa "de la clase, no del objeto": todos los `Persona` comparten `totalSaludos`. Instruméntalo dentro de `saludar()` con `totalSaludos++`. Así practicas que el contador sube para todos los objetos, como el `Contador` del boletín inicial.

> 🔄 Solución

> 🧭 **¿Y si te quedas con ganas?** Cuando domines la encapsulación, vuelve a los problemas de unidades anteriores y reescríbelos con clases bien blindadas: un `Rectangulo` con su área como método, un `Numero` con su análisis como método… El `private`, los getters y el `static` transforman un script suelto en un diseño. El material no se pierde: se reutiliza.

Boletín U11 — Inicial

📝 Boletín U11 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, dale la mano a tu primera superclase y haz que `extends` deje de parecer magia. La herencia en Java es como la de verdad: a veces te llevas genial con las subclases, a veces quieres renegar de todo. Pero nadie nace sabiendo usar `super`.

Ejercicio 1: ¿Qué imprime? — La familia musical

```
class Musico {
    void tocar() { System.out.println("El músico toca un instrumento"); }
}

class Guitarrista extends Musico {
    void tocar() { System.out.println("El guitarrista toca la guitarra"); }
}

class Bajista extends Guitarrista {
    void tocar() { System.out.println("El bajista toca el bajo"); }
}

public class Banda {
    public static void main(String[] args) {
        Bajista b = new Bajista();
        b.tocar();
    }
}
```

¿Qué imprime? ¿Por qué?

Ejercicio 2: Encuentra el error — extends mal usado

```java
public class Animal {
    private String especie;

    public Animal(String especie) {
        this.especie = especie;
    }
}

public class Perro extends Animal {
    private String raza;

    public Perro(String raza) {
        this.raza = raza;
    }
}
```

Este código **no compila**. ¿Por qué? Explica el error y corrígelo.

Ejercicio 3: Completa el código — el gato que llama a su padre

```java
public class Animal {
    public void hacerSonido() {
        System.out.println("Algún sonido genérico...");
    }
}

public class Gato extends Animal {
    @Override
    public void hacerSonido() {
        ________.hacerSonido();   // primero lo del padre
        System.out.println("¡MIAU!");
    }
}
```

¿Qué palabra falta en el hueco para que `Gato` primero ejecute el sonido de `Animal` y después su "¡MIAU!"? Escribe además un `main` que cree un `Gato` y llame a `hacerSonido()`.

Ejercicio 4: Escribe este programa — la herencia de vehículos

Crea una jerarquía de 3 niveles usando `extends`:

- `Vehiculo` (atributo: `String marca`)
- `Coche` (atributo: `int numPuertas`)
- `Deportivo` (atributo: `int velocidadMaxima`)

Cada clase debe tener un constructor que reciba sus atributos y use `super`. En `main()`, crea un `Deportivo` de marca "Ferrari", 2 puertas y 340 km/h. Imprime sus atributos.

Ejercicio 5: ¿Qué imprime? — polimorfismo con referencias

```
class X {
    void mensaje() { System.out.println("X"); }
}

class Y extends X {
    void mensaje() { System.out.println("Y"); }
}

class Z extends Y { }

public class Test {
    public static void main(String[] args) {
        X ref1 = new Y();
        X ref2 = new Z();
        Y ref3 = new Z();

        ref1.mensaje();
        ref2.mensaje();
        ref3.mensaje();
    }
}
```

¿Qué imprime cada llamada? ¿Por qué el tipo de la referencia no decide nada?

Ejercicio 6: Escribe este programa — la granja polimórfica

Crea una clase `Animal` con método `hacerSonido()`. Crea `Vaca`, `Oveja` y `Gallina` que lo sobrescriban. En `main()`, crea un `ArrayList<Animal>`, mete una vaca, una oveja y una gallina, recórrelo con un for-each llamando a `hacerSonido()`.

```
// Salida esperada:
// Muuuu
// Beeee
// Cloc cloc
```

Ejercicio 7: Encuentra el error — @Override que no lo es

```
public class Animal {
    public void hacerSonido() {
        System.out.println("...");
    }
}

public class Pez extends Animal {
    @Override
    public void hacerSonido() { }   // ¿compila?

    @Override
    public void nadar() { }         // ¿compila?
}
```

Una de las dos líneas con `@Override` impide compilar. ¿Cuál y por qué? ¿Qué te avisa el compilador en el instante en que escribes esa línea?

Ejercicio 8: Escribe este programa — el perro bien heredado

Parte de esta clase base:

```java
public class Animal {
    protected String nombre;
    protected int edad;

    public Animal(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }
}
```

Escribe una clase `Perro extends Animal` con:

- Constructor que use `super(nombre, edad)`.
- Método `ladrar()` que imprima `nombre + " dice: ¡Guau!"`.
- Un `main` que cree un `Perro("Firulais", 3)` y llame a `ladrar()`.

Responde: ¿por qué `Perro` puede usar `nombre` y `edad` aunque no las declare?

Ejercicio 9: ¿Qué imprime? — la cadena de constructores

```java
class Abuelo {
    public Abuelo() { System.out.println("Abuelo"); }
}

class Padre extends Abuelo {
    public Padre() { System.out.println("Padre"); }
}

class Hijo extends Padre {
    public Hijo() { System.out.println("Hijo"); }
}

public class Test {
    public static void main(String[] args) {
        new Hijo();
    }
}
```

¿Qué imprime y por qué en ese orden?

Boletín U11 — Inicial Resuelto

📝 Boletín U11 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: ¿Qué imprime? — La familia musical

> 🔄 Solución

Ejercicio 2: Encuentra el error — extends mal usado

> 🔄 Solución

Ejercicio 3: Completa el código — el gato que llama a su padre

> 🔄 Solución

Ejercicio 4: Escribe este programa — la herencia de vehículos

> 🔄 Solución

Ejercicio 5: ¿Qué imprime? — polimorfismo con referencias

> 🔄 Solución

Ejercicio 6: Escribe este programa — la granja polimórfica

> Solución

Ejercicio 7: Encuentra el error — @Override que no lo es

> Solución

Ejercicio 8: Escribe este programa — el perro bien heredado

> Solución

Ejercicio 9: ¿Qué imprime? — la cadena de constructores

> Solución

Boletín U11 — Avanzado

📝 Boletín U11 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐⭐ para pensar, ⭐⭐⭐ para concursar. La herencia es como la familia: a veces heredas cosas buenas, a veces te toca la colección de sellos de tu tío abuelo. Pero con interfaces, al menos eliges qué implementar.

⭐ Ejercicio 1: Interfaz FiguraGeometrica

Crea una interfaz `FiguraGeometrica` con dos métodos:

- `double calcularArea()`
- `double calcularPerimetro()`

Implementa la interfaz en:

- `Circulo` (constructor con radio)
- `Rectangulo` (constructor con ancho y alto)
- `TrianguloRectangulo` (constructor con base y altura)

En el `main`, crea un `ArrayList<FiguraGeometrica>`, añade un círculo de radio 5, un rectángulo 4x3 y un triángulo rectángulo 3x4. Recórrelos imprimiendo área y perímetro de cada uno.

```
public interface FiguraGeometrica {
    double calcularArea();
    double calcularPerimetro();
}
```

Pista: para el triángulo rectángulo, `area = base * altura / 2` y el perímetro es `base + altura + hipotenusa`, con `hipotenusa = Math.sqrt(base*base + altura*altura)`.

⭐ Ejercicio 2: Jerarquía de Empleados

Crea una clase base `Empleado` con:

- `String nombre`

- `double salarioBase`
- Constructor, getters
- Método `double calcularSalario()` que devuelva el `salarioBase`

Crea dos subclases:

- `Gerente`: tiene un `double bono` extra. `calcularSalario()` devuelve `salarioBase + bono`.
- `Vendedor`: tiene un `double comision` por venta y un `int ventasRealizadas`. `calcularSalario()` devuelve `salarioBase + comision * ventasRealizadas`.

En el `main`, crea un array de `Empleado` con un gerente y un vendedor, recórrelo polimórficamente y muestra el salario de cada uno.

```
public class Empleado {
    protected String nombre;
    protected double salarioBase;

    public Empleado(String nombre, double salarioBase) {
        this.nombre = nombre;
        this.salarioBase = salarioBase;
    }

    public double calcularSalario() {
        return salarioBase;
    }
}
```

Pista: declara `nombre` y `salarioBase` como `protected` para que las subclases los usen sin getters. El `@Override` de `calcularSalario()` en cada subclase es el corazón del polimorfismo.

⭐ ⭐ Ejercicio 3: Sistema de pagos con interfaz

Crea una interfaz `Pagable` con el método:

- `boolean procesarPago(double cantidad)`

Implementa la interfaz en:

- `TarjetaCredito`: tiene `double limite` y `double saldoUsado`. Puede pagar si `cantidad + saldoUsado <= limite`.

- `PayPal`: tiene `double saldo`. Puede pagar si `cantidad <= saldo`.
- `TransferenciaBancaria`: tiene `double saldo`. Puede pagar siempre que `cantidad <= saldo`, pero tiene un coste fijo de 1 € por transferencia.

```
public interface Pagable {
    boolean procesarPago(double cantidad);
}
```

Ejemplo:

```
Pagable tarjeta = new TarjetaCredito(1000, 0);
tarjeta.procesarPago(500);   // true
tarjeta.procesarPago(600);   // false (supera el límite)
```

Pista: cada clase decide su propia lógica de aprobación; solo el contrato `procesarPago` es común. Para la transferencia, comprueba que `cantidad + 1 <= saldo`.

⭐ ⭐ Ejercicio 4: Interfaces múltiples: Volador y Nadador

Crea dos interfaces:

- `Volador`: método `void volar()`
- `Nadador`: método `void nadar()`

Crea una clase `Pato` que implemente ambas interfaces, una clase `Avion` que solo implemente `Volador` y una clase `Pez` que solo implemente `Nadador`.

En el `main`, crea un `ArrayList<Volador>` con un `Pato` y un `Avion`, y recórrelo llamando a `volar()`. Luego haz lo mismo con un `ArrayList<Nadador>`.

```
public interface Volador { void volar(); }
public interface Nadador { void nadar(); }
```

Pista: el `Pato` es la estrella: una sola clase que firma dos contratos. Los `ArrayList` de tipo interfaz aceptan cualquier clase que implemente ese contrato.

⭐ ⭐ Ejercicio 5: Downcasting seguro

Crea una jerarquía `Empleado` → `Programador`, `Diseñador`:

- `Programador` tiene `void escribirCodigo()`.
- `Diseñador` tiene `void disenar()`.
- `Empleado` tiene `String nombre` y `void mostrarInfo()`.

En el `main`, crea un `ArrayList<Empleado>` con varios empleados de ambos tipos y recórrelo usando `instanceof` para llamar a los métodos específicos.

Pista: dentro del bucle, `if (e instanceof Programador)` → `((Programador) e).escribirCodigo();`. Siempre con `instanceof` antes del cast: nunca bajes sin preguntar.

⭐ ⭐ Ejercicio 6: Calculadora de figuras con clase abstracta

Crea una clase abstracta `Figura` con:

- Atributo `protected String color`.
- Constructor que reciba el color.
- Métodos abstractos `double calcularArea()` y `double calcularPerimetro()`.
- Método concreto `void mostrarColor()` que imprima el color.

Implementa `Circulo` (radio) y `Rectangulo` (ancho, alto). En el `main`, crea un `ArrayList<Figura>` con un círculo rojo de radio 3 y un rectángulo azul de 4x2, y muestra el área de cada uno y su color.

Pista: el área total de la lista se calcula recorriéndola con `for (Figura f : figuras)`. `mostrarColor()` ya está hecho: las subclases solo implementan los dos métodos abstractos.

⭐ ⭐ ⭐ Ejercicio 7: Sistema de notificaciones polimórfico

Crea una interfaz `Notificable` con:

- `void enviar(String mensaje)`

- `String getEstado()`

Implementa:

- `EmailNotificacion`: atributos `String direccion`, `boolean enviado`. Al enviar, imprime "Enviando email a [dirección]: [mensaje]". Estado: "Enviado" o "Pendiente".
- `SMSNotificacion`: atributos `String telefono`, `boolean enviado`. Al enviar, imprime "Enviando SMS a [teléfono]: [mensaje]". Estado similar.
- `PushNotificacion`: atributos `String dispositivoId`, `boolean enviado`. Al enviar, imprime "Enviando push a [dispositivoId]: [mensaje]".

```
public interface Notificable {
    void enviar(String mensaje);
    String getEstado();
}
```

En el `main`, crea un `ArrayList<Notificable>` con los tres tipos. Añade un método estático que recorra la lista y envíe todas las notificaciones:

```
public static void enviarTodas(List<Notificable> notificaciones, String mensaje) {
    for (Notificable n : notificaciones) {
        n.enviar(mensaje);
    }
}
```

Pista: los tres tipos comparten el patrón: al enviar, ponen `enviado = true` y devuelven "Enviado"; si no, "Pendiente". El polimorfismo hace que `enviarTodas` no sepa con qué tipo trata.

⭐ ⭐ ⭐ Ejercicio 8: Template method — las bebidas

Crea una clase abstracta `Bebida` con el patrón template method:

```java
public abstract class Bebida {
    public final void preparar() {
        hervirAgua();
        prepararIngrediente();
        servirEnTaza();
        anadirExtras();
    }

    private void hervirAgua() { System.out.println("Hirviendo agua..."); }
    private void servirEnTaza() { System.out.println("Sirviendo en taza..."); }

    protected abstract void prepararIngrediente();
    protected abstract void anadirExtras();
}
```

Implementa `Te` (bolsita de té + limón) y `Cafe` (café molido + azúcar). En el `main`, prepara un `Te` y un `Cafe` con una variable de tipo `Bebida`.

Pista: las subclases solo rellenan los dos métodos `protected abstract`. El `final` en `preparar()` garantiza que nadie reordene los pasos. Para probar ambas, usa una referencia polimórfica: `Bebida b = new Te(); b.preparar();`.

⭐ ⭐ ⭐ Ejercicio 9: El gran reto — vehículos con combustible

Crea una jerarquía de vehículos:

- `Vehiculo` (abstracta): `String matricula`, `int combustible`, `abstract boolean mover()`
- `Coche`: gasta 5 de combustible por movimiento
- `Moto`: gasta 3 de combustible por movimiento
- `Camion`: gasta 10 de combustible por movimiento, pero puede llevar `int carga`

Cada vehículo tiene un `mover()` que reduce el combustible y devuelve `true` si pudo moverse. Si no hay suficiente, imprime “Sin combustible” y devuelve `false`.

En `main()`, crea un `ArrayList<Vehiculo>` con varios vehículos. Cada vehículo se mueve repetidamente mientras pueda y cuenta cuántos movimientos hizo.

Pista: da a cada subclase una constante `private static final int GASTO` con lo que consume por movimiento. En `mover()`: `if (combustible >= GASTO) { combustible -= GASTO; ...; return true; } else { System.out.println("Sin combustible"); return false; }`. El bucle del `main` usa el valor de retorno: `while (v.mover()) { movimientos++; }`. Así nunca se queda en un bucle infinito cuando el combustible no llega para moverse.

Boletín U11 — Avanzado Resuelto

📝 Boletín U11 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

⭐ Ejercicio 1: Interfaz FiguraGeometrica

> 🔄 Solución

⭐ Ejercicio 2: Jerarquía de Empleados

> 🔄 Solución

⭐ ⭐ Ejercicio 3: Sistema de pagos con interfaz

> 🔄 Solución

⭐ ⭐ Ejercicio 4: Interfaces múltiples: Volador y Nadador

> 🔄 Solución

⭐ ⭐ Ejercicio 5: Downcasting seguro

> 🔄 Solución

⭐ ⭐ Ejercicio 6: Calculadora de figuras con clase abstracta

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 7: Sistema de notificaciones polimórfico

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 8: Template method — las bebidas

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 9: El gran reto — vehículos con combustible

> 🔄 Solución

Boletín U11 — Extras

📝 Boletín U11 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. La solución está oculta: resístete hasta agotar tu pista. CodeWars y AceptaElReto son los gimnasios donde los programadores se retan a diario: aquí es donde la herencia deja de ser teoría y se convierte en reflejos.

CodeWars

1. Thinkful — Object Drills: Quarks

Crea la clase `Quark` con tres propiedades: `color` (String), `flavor` (String) y `baryon_number` (siempre `1.0`). Además:

- Constructor que recibe `color` y `flavor`.
- `interact(otro)` → intercambia los colores de los dos quarks.

Ejemplo:

```
Quark q1 = new Quark("red", "up");
Quark q2 = new Quark("blue", "strange");
q1.interact(q2);
q1.color;  // "blue"
q2.color;  // "red"
q1.baryon_number;  // 1.0
```

- Enunciado en CodeWars
- Dificultad: 7 kyu

Pista: `baryon_number` es una constante que todos los quarks comparten: `public final double baryon_number = 1.0;`. `interact()` usa una variable temporal para intercambiar: `String temp = this.color; this.color = otro.color; otro.color = temp;`. El `this` desambigua quién es quién en el intercambio.

> › 🔄 Solución

---

2. Building blocks

Crea la clase `Block` que recibe las tres dimensiones (como `int[]` de 3 o como 3 enteros) y los métodos:

- `int getWidth()`, `int getLength()`, `int getHeight()`
- `int getVolume()` → `width * length * height`
- `int getSurfaceArea()` → `2 * (w*l + w*h + l*h)`

Ejemplo: `new Block(new int[]{2, 4, 6})` → volumen 48, superficie `2*(2*4 + 2*6 + 4*6) = 88`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

Pista: guarda las tres dimensiones en atributos privados en el constructor, y deja que los getters las devuelvan. Para la superficie, la fórmula es la suma de las tres caras por dos. El polimorfismo no aparece aquí, pero el objeto con estado y comportamiento sí: la excusa perfecta para repasar la U09 mientras piensas en la herencia.

> 🔄 **Solución**

---

3. Basic subclasses — Adam and Eve

Según el mito, Adam y Eva fueron los primeros humanos. Tu trabajo es "hacer el trabajo de Dios": crear un método estático `create()` que devuelva un array de `Human` con dos objetos: el primero un `Man` y el segundo una `Woman`. Ambas clases heredan de `Human`, y cada humano tiene `name`, `sex` y la propiedad `species` con valor `"Human"`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: herencia pura: `class Man extends Human` y `class Woman extends Human`. Cada subclase llama a `super(...)` para rellenar el nombre y el sexo. El array de retorno es de tipo `Human`, así que acepta ambas subclases.

> 🔄 **Solución**

---

4. Object Oriented Piracy

Crea la clase `Ship` que recibe un `draft` (calado) y un `crew` (tripulantes). Implementa `isWorthIt()`: devuelve `true` si el calado total menos `1.5` por cada tripulante supera 20.

Ejemplo: `new Ship(15, 10).isWorthIt()` → `false` (`15 - 1.5*10 = 0`).

- Enunciado en CodeWars
- Dificultad: 8 kyu

Pista: guarda `draft` y `crew` en atributos `private final`. El método combina ambos: `return draft - 1.5 * crew > 20;`. Es la clase de objeto simple que ya manejas: una excusa para repasar que el estado vive en el objeto, no en el main.

> 🔄 **Solución**

AceptaElReto

5. 117 — La fiesta aburrida

Tinín odia las fiestas, y cada persona que se le acerca se presenta con el formato `"Soy Lotario"`. Ayúdale a responder `"Hola, [nombre]."` a cada uno. La entrada empieza con un número que indica cuántas personas hay, seguido de una línea por persona. Resuélvelo con una clase `Persona` que encapsule el nombre y un método `saludar()`.

Entrada de ejemplo:

```
3
Soy Lotario
Soy Aldonza
Soy Ender
```

Salida de ejemplo:

```
Hola, Lotario.
Hola, Aldonza.
Hola, Ender.
```

- Enunciado en AceptaElReto
- Dificultad: Fácil

Pista: lee la línea y quita el `"Soy "` inicial con `linea.substring(4)` o `split(" ")[1]`. Crea la `Persona` con ese nombre y llama a `saludar()`. Es la excusa perfecta para ver que un objeto con un método puede sustituir a un main que hace de todo.

> 🔄 Solución

6. 119 — Escudos del ejército romano

Un general divide sus legionarios en formaciones **cuadradas** (lo más grande posible), repitiendo con los que queden libres hasta agotarlos. Cada cuadrado de lado `n` necesita escudos según el perímetro más la cobertura: los legionarios del borde llevan más escudos que los del interior. Para un cuadrado de `n × n`, los escudos son `n² + 4n` (una base por soldado más el perímetro exterior). Dado el número de legionarios, calcula el mínimo de escudos necesarios.

Entrada: varios casos de prueba, cada uno con el número de legionarios. Termina con `0`.

Entrada de ejemplo:

```
35
20
10
0
```

Salida de ejemplo:

```
71
44
26
```

- Enunciado en AceptaElReto
- Dificultad: Fácil/Media

Pista: mientras queden legionarios, encuentra el mayor cuadrado `n` tal que `n² <= restantes` (prueba `n` creciente o usa `Math.sqrt`). Suma los escudos de ese cuadrado y resta `n²` de los restantes. Encapsula la lógica en una clase `Formacion` con métodos como `mayorCuadrado()` y `calcularEscudos()`.

> 🔄 Solución

> 🧭 **¿Y si te quedas con ganas?** Cuando domines la herencia, vuelve a los problemas de las unidades anteriores y reescríbelos con jerarquías: un `Lector` abstracto, un `Solucionador` polimórfico, figuras que se calculan solas. El material no se pierde: se reutiliza.

Boletín U12 — Inicial

📝 Boletín U12 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, crea tu primer `ArrayList`, tu primera `Caja<T>` y haz que el `HashMap` deje de parecer magia. Empieza suave, que las colecciones muerden poco a poco.

Ejercicio 1: ¿Qué imprime? — ArrayList remove por índice vs valor

```
import java.util.ArrayList;

public class Test {
    public static void main(String[] args) {
        ArrayList<String> lista = new ArrayList<>();
        lista.add("A");
        lista.add("B");
        lista.add("C");
        lista.add("B");
        lista.add("D");

        lista.remove(1);          // remove por índice
        lista.remove("B");        // remove por objeto

        System.out.println(lista);
    }
}
```

¿Qué imprime? ¿Por qué el segundo `remove("B")` no borra el mismo elemento que el primero?

Ejercicio 2: Encuentra el error — size() vs length vs length()

```java
ArrayList<String> nombres = new ArrayList<>();
nombres.add("Ana");

int[] edades = {20, 30};
String saludo = "Hola";

System.out.println(nombres.length);   // línea 1
System.out.println(edades.size());    // línea 2
System.out.println(saludo.length);    // línea 3
```

¿Qué líneas tienen error? Explica qué usa cada tipo para preguntar cuánto mide: `size()`, `length` o `length()`.

Ejercicio 3: Completa el código — for-each que suma una lista

Completa el siguiente programa para que sume todos los números de una `ArrayList<Integer>`:

```java
import java.util.ArrayList;

public class SumaLista {
    public static void main(String[] args) {
        ArrayList<Integer> numeros = new ArrayList<>();
        numeros.add(4);
        numeros.add(9);
        numeros.add(2);
        numeros.add(7);

        int suma = ______;
        for (______ n : numeros) {      // ¿qué tipo y qué variable?
            suma ______ n;              // ¿qué operador?
        }

        System.out.println("Suma: " + suma);
    }
}
```

¿Qué falta en cada hueco? ¿Cuánto vale la suma al final?

Ejercicio 4: Escribe este programa — la lista de la compra

Crea un programa con un `ArrayList<String>` llamado `compra` y haz lo siguiente:

1. Añade `"Leche"`, `"Pan"` y `"Huevos"`.
2. Añade `"Café"` en la posición 1 (entre Leche y Pan).
3. Muestra el tamaño de la lista.
4. Borra el elemento de la posición 2.
5. Recorre la lista con un for-each e imprime cada elemento.

Pista: usa `add(e)`, `add(i, e)`, `remove(i)`, `size()` y un for-each.

Ejercicio 5: ¿Qué imprime? — el HashSet que no deja repetir

```
import java.util.HashSet;

public class Test {
    public static void main(String[] args) {
        HashSet<String> invitados = new HashSet<>();
        invitados.add("Ana");
        invitados.add("Bob");
        invitados.add("Ana");

        System.out.println(invitados.size());
        System.out.println(invitados.contains("Bob"));
    }
}
```

¿Qué imprime? ¿Por qué el segundo `add("Ana")` no provoca ningún error ni aviso?

Ejercicio 6: Escribe este programa — sin duplicados

Crea un `ArrayList<String>` con nombres de compañeros, **con repetidos a propósito** (mínimo 8 elementos, 3 de ellos repetidos). Después:

1. Construye un `LinkedHashSet<String>` a partir de la lista.
2. Imprime la lista original y el conjunto.
3. Explica en un comentario qué diferencia ves y por qué.

Pista: el constructor de `LinkedHashSet` acepta cualquier colección: `new LinkedHashSet<>(lista)`.

Ejercicio 7: Completa el código — tu primera clase genérica

```
public class Caja<______> {        // ¿qué parámetro de tipo falta?
    private ______ contenido;

    public void guardar(______ contenido) {
        this.contenido = contenido;
    }

    public ______ sacar() {
        return contenido;
    }
}
```

Completa los huecos para que `Caja<String>` guarde Strings y `Caja<Integer>` guarde Integers. ¿Por qué `Caja<int>` no compila?

Ejercicio 8: ¿Qué imprime? — HashMap con put repetido

```java
import java.util.HashMap;

public class Test {
    public static void main(String[] args) {
        HashMap<String, Integer> mapa = new HashMap<>();
        mapa.put("Ana", 10);
        mapa.put("Bob", 20);
        mapa.put("Ana", 30);

        System.out.println(mapa.get("Ana"));
        System.out.println(mapa.size());
    }
}
```

¿Qué imprime? ¿Por qué `size()` no es 3?

Ejercicio 9: Escribe este programa — contador de palabras con HashMap

Crea un programa que tenga un array de palabras (hardcodeado) como este:

```java
String[] palabras = {"hola", "mundo", "hola", "java", "mundo", "hola", "adios"};
```

Usa un `HashMap<String, Integer>` para contar cuántas veces aparece cada palabra. Al final, recorre el mapa con un bucle for-each sobre `entrySet()` y muestra cada palabra con su cuenta.

Pista: el patrón estrella es `mapa.put(p, mapa.getOrDefault(p, 0) + 1)`.

Boletín U12 — Inicial Resuelto

📝 Boletín U12 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: ¿Qué imprime? — ArrayList remove por índice vs valor

> 🔄 Solución

Ejercicio 2: Encuentra el error — size() vs length vs length()

> 🔄 Solución

Ejercicio 3: Completa el código — for-each que suma una lista

> 🔄 Solución

Ejercicio 4: Escribe este programa — la lista de la compra

> 🔄 Solución

Ejercicio 5: ¿Qué imprime? — el HashSet que no deja repetir

> 🔄 Solución

Ejercicio 6: Escribe este programa — sin duplicados

> Solución

Ejercicio 7: Completa el código — tu primera clase genérica

> Solución

Ejercicio 8: ¿Qué imprime? — HashMap con put repetido

> Solución

Ejercicio 9: Escribe este programa — contador de palabras con HashMap

> Solución

Boletín U12 — Avanzado

📝 Boletín U12 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: La cola del supermercado con LinkedList

Simula una cola de supermercado usando `LinkedList<String>`. El programa debe mostrar un menú:

1. **Llega cliente** → Añade un nombre al final de la cola.
2. **Atender cliente** → Elimina y muestra el primero de la cola.
3. **¿Quién sigue?** → Muestra el primero sin eliminarlo.
4. **Estado de la cola** → Muestra todos los clientes en orden.
5. **Salir**

Usa los métodos `addLast()`, `removeFirst()` y `getFirst()` de `LinkedList`, con un `switch` y un `Scanner`.

Pista: el menú se repite con un `while` hasta que el usuario elija `0`. Cuida los casos en los que la cola está vacía antes de llamar a `removeFirst()` o `getFirst()`.

⭐ ⭐ Ejercicio 2: Intersección y unión de conjuntos

Crea dos `HashSet<Integer>` con números aleatorios (entre 1 y 20, 8 elementos cada uno). Calcula y muestra:

- **Intersección:** elementos que están en ambos conjuntos.
- **Unión:** todos los elementos sin repetir.
- **Diferencia simétrica:** elementos que están en uno u otro, pero no en ambos.

Pista: usa `retainAll()`, `addAll()` y `removeAll()` de la interfaz `Set`. Para la diferencia simétrica: `union.removeAll(interseccion)`.

⭐ ⭐ Ejercicio 3: Eliminar duplicados manteniendo el orden

Crea un `ArrayList<Integer>` con elementos repetidos (`[3, 1, 4, 1, 5, 9, 2, 6, 5, 3, 5]`). Escribe un método que devuelva un nuevo `ArrayList<Integer>` **sin duplicados pero manteniendo el orden de primera aparición**.

Pista: un `HashSet` no vale directamente porque pierde el orden. Usa un `LinkedHashSet` (conserva el orden de inserción) o recorre manualmente comprobando con `contains` antes de añadir.

⭐ ⭐ ⭐ Ejercicio 4: ¿Qué imprime? — el remove que rompe el baile

Sin ejecutar, responde qué ocurre con este programa:

```
import java.util.ArrayList;

public class Puzle {
    public static void main(String[] args) {
        ArrayList<String> palabras = new ArrayList<>();
        palabras.add("hola");
        palabras.add("mundo");
        palabras.add("adiós");

        for (String p : palabras) {
            if (p.equals("mundo")) {
                palabras.remove(p);
            }
        }
        System.out.println(palabras);
    }
}
```

¿Imprime algo? ¿O lanza una excepción? ¿Cuál y por qué?

Pista: piensa en lo que hace el `for-each` por debajo (usa un `Iterator`) y en lo que pasa si la colección cambia mientras se recorre.

⭐ ⭐ Ejercicio 5: Filtrar con Iterator

Crea un `ArrayList<Integer>` con los números del 1 al 10. Recórrelo con un `Iterator<Integer>` explícito y **borra todos los números pares** usando `it.remove()`. Al final, imprime la lista.

Pista: recorre con `while (it.hasNext())`, obtén cada número con `it.next()` y comprueba si es par con `% 2 == 0`. Nunca uses `lista.remove(...)` dentro del bucle.

⭐ ⭐ Ejercicio 6: El TreeSet ordenado

Crea un programa que pida al usuario palabras con `Scanner` hasta que escriba `"fin"`. Guárdalas en un `TreeSet<String>`. Al terminar, muestra:

1. Las palabras ordenadas (imprimir el TreeSet directamente).
2. La primera palabra (`first()`) y la última (`last()`).
3. Las palabras que van antes que `"m"` en orden alfabético (`headSet("m")`).

Pista: el TreeSet ordena y elimina duplicados automáticamente. Comprueba qué pasa si el usuario repite una palabra.

⭐ Pila genérica <T>: Ejercicio 7

Implementa una clase genérica `Pila<T>` que funcione como una pila (LIFO). Debe tener los métodos:

- `void push(T elemento)` — apila un elemento.
- `T pop()` — desapila y devuelve el elemento superior (lanza `EmptyStackException` si está vacía).
- `T peek()` — devuelve el elemento superior sin desapilarlo.
- `boolean isEmpty()` — indica si está vacía.
- `int size()` — número de elementos.

Internamente, usa un `ArrayList<T>` como almacenamiento. Pruébala con `Pila<Integer>`, `Pila<String>` y `Pila<Double>`.

Pista: `EmptyStackException` está en `java.util.EmptyStackException`. Recuerda comprobar `isEmpty()` antes de `pop()`/`peek()`.

⭐ ⭐ Ejercicio 8: HashMap inverso

Escribe un método genérico estático:

```
public static <K, V> HashMap<V, K> invertirMapa(HashMap<K, V> original)
```

Que devuelva un nuevo `HashMap` intercambiando claves y valores. Si hay valores duplicados en el mapa original, el último encontrado sobrescribe al anterior.

Prueba con un mapa de `String → Integer` y otro de `String → String`.

Pista: recorre `original.entrySet()` y haz `invertido.put(e.getValue(), e.getKey())`.

⭐ ⭐ Ejercicio 9: TreeMap — frecuencia de letras

Escribe un programa que lea un texto por teclado (o use uno hardcodeado) y cuente cuántas veces aparece cada **letra** (ignorando espacios, números y signos). Usa un `TreeMap<Character, Integer>` para que las letras se muestren automáticamente ordenadas alfabéticamente.

Ejemplo de salida para «Hola mundo»:

```
a: 1, d: 1, h: 1, l: 1, m: 1, n: 1, o: 2, u: 1
```

Pista: recorre el `String` con `toCharArray()` y usa `Character.isLetter(c)` para filtrar. El `getOrDefault` suma el contador; el TreeMap ordena solo.

⭐ ⭐ ⭐ Ejercicio 10: Wildcards — suma y mezcla de números

Implementa un método que sume todos los números de una lista, aceptando cualquier subtipo de `Number`:

```
public static double sumar(List<? extends Number> lista)
```

Pruébalo con `List<Integer>`, `List<Double>` y `List<Float>`. ¿Qué ocurre si intentas pasar una `List<String>`?

Crea también un segundo método que **mezcle** dos listas de números de tipos distintos en una sola `List<Double>`:

```
public static List<Double> mezclar(List<? extends Number> a, List<? extends Number>
b)
```

Pista: para `sumar`, recorre con `for (Number n : lista)` y usa `n.doubleValue()`. Para `mezclar`, usa `addAll()` y convierte cada elemento con `doubleValue()`. Y no intentes `add` en `sumar`: `? extends` es de solo lectura (PECS).

⭐ ⭐ ⭐ Ejercicio 11: Sistema de votaciones con método genérico

Crea un sistema de votaciones donde:

- Cada votante puede votar por un candidato (String).
- Usa un `HashMap<String, Integer>` para los votos.
- Usa un `TreeMap<String, Integer>` para mostrar el ranking ordenado.

Crea un método genérico:

```
public static <T> T obtenerGanador(Map<T, Integer> votos)
```

Que devuelva la clave con más votos. Pruébalo con un `HashMap<String, Integer>` y otro `HashMap<Integer, Integer>`.

Pista: recorre `votos.entrySet()` y guarda el ganador provisional comparando `getValue()` con un máximo acumulado. El método es genérico porque el tipo de la clave (`String`, `Integer`…) no importa.

⭐ ⭐ ⭐ Ejercicio 12: el type erasure al descubierto

La clase `Caja<T>` guarda un valor y lo devuelve con `getValor()`:

```
public class Caja<T> {
    private T valor;

    public Caja(T valor) {
        this.valor = valor;
    }

    public T getValor() {
        return valor;
    }
}
```

Sin ejecutar, responde:

1. ¿Son `Caja<String>` y `Caja<Integer>` clases distintas en tiempo de ejecución?
2. ¿Qué tipo tiene realmente `caja.getValor()` dentro del bytecode si lo compilas como `Caja<String>`?
3. Escribe un `main` que cree `Caja<String>` y `Caja<Integer>` y compruebe con `getClass()` que ambas son instancias de la misma clase `Caja`.

Pista: el type erasure convierte `Caja<T>` en `Caja` a pelo (con `Object` donde estaba `T`). Por eso `caja.getClass()` devuelve lo mismo para `Caja<String>` y `Caja<Integer>`: en runtime no hay dos clases, solo una `Caja`. El cast de `getValor()` lo añade el compilador, no tu código.

Boletín U12 — Avanzado Resuelto
📝 Boletín U12 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

⭐ Ejercicio 1: La cola del supermercado con LinkedList

> 🔄 Solución

⭐ ⭐ Ejercicio 2: Intersección y unión de conjuntos

> 🔄 Solución

⭐ ⭐ Ejercicio 3: Eliminar duplicados manteniendo el orden

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 4: ¿Qué imprime? — el remove que rompe el baile

> 🔄 Solución

⭐ ⭐ Ejercicio 5: Filtrar con Iterator

> 🔄 Solución

⭐ ⭐ Ejercicio 6: El TreeSet ordenado

> 🔄 Solución

⭐ Pila genérica <T>: Ejercicio 7

> 🔄 Solución

⭐ ⭐ Ejercicio 8: HashMap inverso

> 🔄 Solución

⭐ ⭐ Ejercicio 9: TreeMap — frecuencia de letras

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 10: Wildcards — suma y mezcla de números

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 11: Sistema de votaciones con método genérico

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 12: el type erasure al descubierto

> 🔄 Solución

Boletín U12 — Extras

📝 Boletín U12 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. Las soluciones están ocultas: agota tu pista antes de mirarlas.

CodeWars

1. Convert a string to an array

Te dan una cadena de texto separada por espacios. Escribe una función que la divida y devuelva un array de palabras.

Ejemplo: `"Robin Singh"` → `["Robin", "Singh"]`, y `"I love arrays they are my favorite"` → `["I", "love", "arrays", "they", "are", "my", "favorite"]`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

2. Counting Duplicates

Escribe una función que devuelva cuántos caracteres **distintos** (letras y dígitos, sin distinguir mayúsculas) aparecen más de una vez en una cadena.

Ejemplo: `"abcde"` → `0`, `"aabbcde"` → `2` (a y b), `"indivisibility"` → `1` (la i), `"aA11"` → `2` (a y 1).

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

3. Find the unique number

Tienes un array de números donde todos son iguales excepto uno. Encuentra el número único.

Ejemplo: `[ 1, 1, 1, 2, 1, 1 ]` → 2, y `[ 0, 0, 0.55, 0, 0 ]` → `0.55`.

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔁 **Solución**

AceptaElReto

4. 152 — Va de modas...

Dado un conjunto de números, la **moda** es el valor (o valores) que más se repite. Te piden calcular la moda de cada distribución.

Entrada: varios casos de prueba. Cada caso comienza con un número que indica cuántos valores tiene el conjunto (nunca mayor de 25.000). En la siguiente línea se dan los valores separados por espacios. La entrada termina cuando el primer número es 0.

Salida: para cada caso, la moda (se garantiza que solo hay una).

Ejemplo: `1 2 2 3 3 3 4 4 4 4 5` → 4, y `1 8 9 6 3 2 1 5 4 7 9 6 3 2 1 4 7` → 1.

- Enunciado en AceptaElReto
- Dificultad: ⭐ ⭐

> 💡 **Pista**

> 🔁 **Solución**

---

5. 416 — Michael J. Fox y el Pato Donald

En un grupo de personas, hay que comprobar si **dos personas cumplen años el mismo día**. Te dan las fechas de nacimiento de cada una en formato `día/mes/año`.

Entrada: varios casos de prueba en dos líneas cada uno. La primera línea tiene el número de personas del grupo; la segunda, sus fechas de nacimiento separadas por espacios. La entrada termina con un `0`.

Salida: `SI` si hay algún cumpleaños repetido (mismo día y mes) y `NO` en caso contrario.

Ejemplo: `9/6/1961 22/10/1938 31/5/1961 20/4/1964` → `NO`, y `9/6/1961 22/10/1938 31/5/1961 20/4/1964 9/6/1934` → `SI`.

- Enunciado en AceptaElReto
- Dificultad: ⭐ ⭐

> › 💡 **Pista**

> › 🔄 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines listas, conjuntos, genéricos y mapas, vuelve a los problemas de unidades anteriores y resuélvelos guardando los datos en colecciones: el contador de notas, el buscador de nombres… Todo lo que antes era un array paralelo ahora es un mapa. El material no se pierde: se reutiliza.

Boletín U13 — Inicial

📝 Boletín U13 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, escribe tu primera `->` y haz que la cinta transportadora del stream deje de parecer magia. Las funciones viajan solas, pero el que programa eres tú. Empieza suave, que la flecha no muerde.

Ejercicio 1: Completa el código — tu primera lambda

Completa las lambdas para que hagan lo que dice el comentario:

```
Predicate<Integer> esMayorDeEdad = ______;      // edad >= 18
Function<Integer, Integer> doble = ______;      // x * 2
Consumer<String> imprimir = ______;             // System.out.println(s)
Supplier<String> saludar = ______;              // "¡Hola!"
```

¿Qué tipo necesita la variable para que la lambda compile en cada caso?

Ejercicio 2: ¿Qué imprime? — el orden de la flecha

```
import java.util.function.Function;

public class Test {
    public static void main(String[] args) {
        Function<Integer, Integer> operacion = x -> x * x + 1;
        System.out.println(operacion.apply(5));
        System.out.println(operacion.apply(0));
    }
}
```

¿Qué imprime? ¿Por qué el cuerpo `x * x + 1` no necesita `return` ni llaves?

Ejercicio 3: Encuentra el error — la lambda mal vestida

```
import java.util.function.Predicate;

public class Error {
    public static void main(String[] args) {
        Predicate<Integer> esPositivo = x -> x > 0;
        System.out.println(esPositivo.test(-3));
        System.out.println(esPositivo.accept(5));   // ¿Qué ocurre aquí?
    }
}
```

Este código **no compila**. ¿Por qué? ¿Qué método deberías llamar en vez de `accept`?

Ejercicio 4: Escribe este programa — filtrar pares con streams

Crea un programa que tenga esta lista:

```
List<Integer> numeros = List.of(10, 15, 22, 33, 40, 55);
```

Usa un stream para **filtrar los pares**, recogerlos en una lista con `toList()` y mostrarla. ¿Cuántos pares hay?

Ejercicio 5: ¿Qué imprime? — el pipeline básico

```java
import java.util.*;
import java.util.stream.*;

public class Test {
    public static void main(String[] args) {
        List<String> palabras = List.of("sol", "luna", "mar", "cielo", "sol");
        long largas = palabras.stream()
            .filter(p -> p.length() >= 4)
            .distinct()
            .count();
        System.out.println(largas);
    }
}
```

¿Qué imprime? ¿Por qué `distinct()` cambia el resultado frente a no usarlo?

Ejercicio 6: Completa el código — mayúsculas con map

Completa el pipeline para que transforme cada palabra a mayúsculas y las recoja en una lista:

```java
List<String> palabras = List.of("hola", "java", "mundo");

List<String> mayusculas = palabras.stream()
    .______(String::toUpperCase)
    .______();
```

¿Qué operación intermedia y qué terminal necesitas? ¿Y si usaras `Collectors.toList()` en vez de `toList()`?

Ejercicio 7: Escribe este programa — longitud de cada palabra

Crea un programa que tenga un array de nombres y use un stream con `map` para calcular la **longitud de cada nombre**, recogiéndolo en una `List<Integer>`. Después muestra el resultado con `forEach(System.out::println)`.

Ejercicio 8: Encuentra el error — la cinta que nunca arranca

```java
import java.util.*;
import java.util.stream.*;

public class Error {
    public static void main(String[] args) {
        List<Integer> numeros = List.of(1, 2, 3, 4, 5);
        Stream<Integer> flujo = numeros.stream()
            .filter(n -> n % 2 == 1)
            .map(n -> n * 10);
        System.out.println("Preparado");
    }
}
```

Este programa **compila y ejecuta**, pero no imprime ningún número transformado. ¿Por qué? ¿Qué le falta al pipeline?

Ejercicio 9: Completa el código — un Consumer para imprimir

Tienes una lista de Strings y quieres imprimir cada uno entre paréntesis, por ejemplo `(hola)`. Completa el `forEach`:

```java
List<String> palabras = List.of("hola", "java");

palabras.stream()
    .forEach(p -> System.out._____("(" + p + ")"));
```

¿Qué método de `System.out` imprime sin salto de línea? ¿Y cuál añade el salto?

Boletín U13 — Inicial Resuelto

📝 Boletín U13 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: Completa el código — tu primera lambda

> 🔄 Solución

Ejercicio 2: ¿Qué imprime? — el orden de la flecha

> 🔄 Solución

Ejercicio 3: Encuentra el error — la lambda mal vestida

> 🔄 Solución

Ejercicio 4: Escribe este programa — filtrar pares con streams

> 🔄 Solución

Ejercicio 5: ¿Qué imprime? — el pipeline básico

> 🔄 Solución

Ejercicio 6: Completa el código — mayúsculas con map

> Solución

Ejercicio 7: Escribe este programa — longitud de cada palabra

> Solución

Ejercicio 8: Encuentra el error — la cinta que nunca arranca

> Solución

Ejercicio 9: Completa el código — un Consumer para imprimir

> Solución

Boletín U13 — Avanzado

📝 Boletín U13 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ ⭐ Ejercicio 1: Ordenar con referencias a método

Tienes una lista de nombres. Ordena la lista **por longitud** (de menor a mayor) con un stream usando `sorted()` y la referencia `String::length` combinada con `Comparator.comparing`. Después muéstrala.

Pista: `sorted(Comparator.comparing(String::length))` ordena por longitud sin tocar la lista original. Si quieres el orden inverso, usa `reversed()`.

⭐ ⭐ Ejercicio 2: Agrupar palabras por su primera letra

Tienes una lista de palabras. Usa `groupingBy` para agruparlas por su **primera letra** y muestra el mapa resultante. Después, con `groupingBy(p -> p.charAt(0), Collectors.counting())`, cuenta cuántas palabras hay en cada grupo.

Pista: `groupingBy` devuelve `Map<Character, List<String>>`. El segundo argumento `counting()` cambia el valor del mapa a `Long`.

⭐ ⭐ Ejercicio 3: Optional — el que no se deja engañar

Implementa un método que reciba una `List<Integer>` y devuelva el **máximo** usando `max(Integer::compareTo)`, gestionando el resultado con `orElse` para que devuelva `-1` si la lista está vacía. Prueba con una lista vacía y con una llena.

Pista: `max` devuelve `Optional<Integer>`. No uses `get()` a ciegas: `orElse(-1)` aterriza con seguridad.

⭐ ⭐ ⭐ Ejercicio 4: El pipeline completo

Tienes esta lista de números:

```
List<Integer> numeros = List.of(12, 5, 8, 3, 9, 5, 12, 7);
```

Construye un pipeline que: filtre los **mayores o iguales a 5**, los **eleve al cuadrado** (`n * n`), **elimine los duplicados**, los **ordene de mayor a menor** y se quede con los **3 primeros**. Recoge el resultado en una lista con `toList()`.

Pista: para ordenar de mayor a menor: `sorted(Comparator.reverseOrder())`. Recuerda que el orden de las estaciones importa: `distinct` antes de `sorted` cambia la cuenta.

⭐ ⭐ Ejercicio 5: De lista a mapa con `toMap`

Crea una clase sencilla `Alumno` con `nombre` y `nota`. Con una lista de 5 alumnos, usa `Collectors.toMap` para obtener un `Map<String, Integer>` donde la clave sea el nombre y el valor la nota. Como los nombres son únicos, usa una función de fusión por si acaso.

Pista: `Collectors.toMap(Alumno::getNombre, Alumno::getNota, (a, b) -> a)`. La fusión `(a, b) -> a` evita la `IllegalStateException` si se repite una clave.

⭐ ⭐ ⭐ Ejercicio 6: El máximo con `reduce` y comparador

Implementa el máximo de una `List<Integer>` de dos formas: con `reduce` y un acumulador que vaya guardando el mayor (sin usar `Math::max`), y con `max`. ¿Qué devuelve cada una? ¿Cuál necesitas una identidad?

Pista: `reduce(Integer.MIN_VALUE, (a, b) -> a > b ? a : b)` usa `Integer.MIN_VALUE` como identidad. `max(Integer::compareTo)` devuelve un `Optional`.

⭐ ⭐ Ejercicio 7: Frecuencias con `groupingBy`

Tienes un array de palabras con repetidas:

```
String[] palabras = {"hola", "adios", "hola", "java", "hola", "adios"};
```

Usa `Arrays.stream` y `groupingBy(p -> p, Collectors.counting())` para contar cuántas veces aparece cada palabra. Muestra el mapa y, después, la palabra que más veces aparece.

Pista: el mapa es `Map<String, Long>`. Para la palabra más repetida, recorre `entrySet()` comparando valores, o usa streams de nuevo con `max(Map.Entry.comparingByValue())`.

⭐ ⭐ ⭐ Ejercicio 8: Optional y streams, la pareja

Tienes una lista de nombres. Busca, con streams, el **primer nombre que empiece por "J"** usando `filter(...).findFirst()`. Gestiona el `Optional` resultante con `ifPresent` para imprimirlo y con `orElse` para mostrar "no hay nadie" si no existe. Prueba con una lista que tenga "J" y con otra que no.

Pista: `findFirst()` devuelve `Optional<String>`. Con `ifPresent(System.out::println)` imprimes solo si hay valor; `orElse("no hay nadie")` cubre la ausencia.

⭐ ⭐ ⭐ Ejercicio 9: el stream que se niega a morir

Observa este código y responde **sin ejecutarlo**:

```java
import java.util.*;
import java.util.stream.*;

public class Test {
    public static void main(String[] args) {
        Stream<Integer> flujo = List.of(1, 2, 3).stream();
        long a = flujo.count();
        long b = flujo.count();
        System.out.println(a + " " + b);
    }
}
```

1. ¿Compila?
2. ¿Qué ocurre al ejecutarlo?
3. ¿Cómo lo arreglarías?

Pista: un stream es de un solo uso. La primera operación terminal lo consume. Si quieres contar dos veces, crea dos streams (`List.of(1, 2, 3).stream()` dos veces).

> 🔄 Solución

Boletín U13 — Avanzado Resuelto

📝 Boletín U13 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

⭐ ⭐ Ejercicio 1: Ordenar con referencias a método

> 🔄 Solución

⭐ ⭐ Ejercicio 2: Agrupar palabras por su primera letra

> 🔄 Solución

⭐ ⭐ Ejercicio 3: Optional — el que no se deja engañar

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 4: El pipeline completo

> 🔄 Solución

⭐ ⭐ Ejercicio 5: De lista a mapa con `toMap`

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 6: El máximo con `reduce` y comparador

> 🔄 Solución

⭐ ⭐ Ejercicio 7: Frecuencias con `groupingBy`

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 8: Optional y streams, la pareja

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 9: el stream que se niega a morir

> 🔄 Solución

Boletín U13 — Extras

📝 Boletín U13 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. Las soluciones están ocultas: agota tu pista antes de mirarlas.

CodeWars

1. Enumerable Magic #1 - True for All?

Implementa la función `all`: recibe una lista y un `Predicate`, y devuelve `true` si el predicado es verdadero para **todos** los elementos. Si la lista está vacía, devuelve `true` (nada ha fallado la prueba).

Ejemplo: `all([1, 2, 3, 4, 5], x -> x < 9)` → `true`, y `all([1, 2, 3, 4, 5], x -> x > 9)` → `false`.

- Enunciado en CodeWars
- Dificultad: 8 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

2. Sorted? yes? no? how?

Recibes un array de números. Devuelve `"yes, ascending"` si está ordenado de menor a mayor, `"yes, descending"` si está de mayor a menor, y `"no"` si no está ordenado.

Ejemplo: `[1, 2]` → `"yes, ascending"`, `[15, 7, 3]` → `"yes, descending"`, `[4, 2, 30]` → `"no"`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

3. Sum of odd numbers

Dado el triángulo de números impares consecutivos:

```
             1
          3     5
       7     9    11
   13    15    17    19
21    23    25    27    29
```

Devuelve la suma de la fila `n` (empezando por 1). **Ejemplo:** `n=1` → `1`, `n=2` → `8`, `n=3` → `27`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

> 💡 **Pista**

> 🔁 **Solución**

---

4. Two Oldest Ages

Implementa `twoOldestAges`: recibe un array de edades (siempre con al menos 2 elementos) y devuelve un array con las **dos edades más altas**, en el formato `[segunda más alta, la más alta]`.

Ejemplo: `[1, 2, 10, 8]` → `[8, 10]`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

> 💡 **Pista**

> 🔁 **Solución**

AceptaElReto

5. 219 — La lotería de la peña Atlética

La peña atlética solo compra décimos con **números pares**. Te dan una lista de décimos de cada administración y debes decir cuántos puede comprar.

Entrada: el primer número indica cuántos casos de prueba hay. Cada caso son dos líneas: el número de décimos `n` y la lista de `n` números (entre 0 y 99.999).

Salida: para cada caso, cuántos décimos son pares.

Ejemplo: `10` y `1 2 3 4 5 6 7 8 9 10` → `5`.

- Enunciado en AceptaElReto
- Dificultad: ⭐

> 💡 **Pista**

> 🔄 **Solución**

---

6. 105 — Ventas

El bar de Javier abre todos los días menos los lunes. Apunta la caja de cada día de la semana (martes, miércoles, jueves, viernes, sábado y domingo). Debe decir: el **día de más ventas**, el **día de menos ventas** (o `EMPATE` si hay empate en el máximo o el mínimo), y si las **ventas del domingo superan la media semanal** (`SI` o `NO`).

Entrada: varios casos de prueba. Cada caso son 6 números (las ventas de martes a domingo). El programa termina cuando el primer número del caso es `-1`.

Salida: para cada caso, `DIA_MAX DIA_MIN SI/NO`.

Ejemplo: `185.50 250.36 163.45 535.20 950.22 450.38` → `SABADO JUEVES SI`.

- Enunciado en AceptaElReto
- Dificultad: ⭐ ⭐

> 💡 **Pista**

> 🔄 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines lambdas y streams, vuelve a los problemas de unidades anteriores y reescríbelos con pipelines: el contador de frecuencias de la U13 con `groupingBy`, los bucles de la U07 con `filter` + `reduce`, la agenda de la U13 con `Collectors.toMap`… Todo lo que antes era un bucle ahora es una declaración. El material no se pierde: se reutiliza.

Boletín U14 — Inicial

📝 Boletín U14 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, crea tu primer archivo con `FileWriter` y haz que `readLine()` deje de parecer magia. El disco es tuyo y los datos esperan. Empieza suave, que los ficheros muerden poco a poco.

Ejercicio 1: Encuentra el error — IOException sin capturar

```
import java.io.*;

public class Test {
    public static void main(String[] args) {
        FileWriter writer = new FileWriter("salida.txt");
        writer.write("Hola mundo");
        writer.close();
    }
}
```

Este código **no compila**. ¿Por qué? ¿Qué dos formas hay de solucionarlo?

Ejercicio 2: Completa el código — try-with-resources

Completa el siguiente programa para que lea un archivo y muestre su contenido:

```java
import java.io.*;
import java.nio.file.*;

public class Lector {
    public static void main(String[] args) {
        Path ruta = Paths.get("datos.txt");

        try (_______ reader = Files.newBufferedReader(ruta)) {  // ¿qué tipo?
            String linea;
            while ((linea = reader.readLine()) != null) {
                System.out.println(_______);  // ¿qué va aquí?
            }
        } catch (IOException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
```

Ejercicio 3: Escribe este programa — guardar ciudades en un archivo

Crea un array de cadenas con 5 nombres de ciudades. Escribe cada nombre en una línea de un archivo llamado `ciudades.txt`, usando `BufferedWriter` y `try-with-resources`. No olvides el salto de línea.

Pista: `BufferedWriter` tiene `write(...)` y `newLine()`.

Ejercicio 4: Encuentra el error — File.createNewFile sin comprobar

```java
File f = new File("documento.txt");
f.createNewFile();
FileWriter w = new FileWriter(f);
w.write("Contenido importante");
w.close();
```

¿Qué pasa si el archivo `documento.txt` ya existe? ¿Qué devuelve `createNewFile()`?

Ejercicio 5: ¿Qué imprime? — el contador de líneas

```java
import java.io.*;

public class Test {
    public static void main(String[] args) throws IOException {
        File f = new File("datos.txt");
        FileWriter w = new FileWriter(f);
        w.write("linea1\nlinea2\nlinea3\n");
        w.close();

        BufferedReader r = new BufferedReader(new FileReader(f));
        int contador = 0;
        while (r.readLine() != null) {
            contador++;
        }
        r.close();
        System.out.println(contador);
    }
}
```

¿Qué imprime? ¿El último `\n` cuenta como una línea más?

Ejercicio 6: Escribe este programa — el diario personal

Escribe un programa que añada una línea a `diario.txt` con la fecha de hoy y el texto que el usuario introduzca por teclado. Cada ejecución debe **añadir al final sin borrar** lo anterior.

Pista: `new FileWriter("diario.txt", true)` y para la fecha `java.time.LocalDate.now()`.

Ejercicio 7: ¿Qué imprime? — matches() vs find()

```java
import java.util.regex.*;

public class Test {
    public static void main(String[] args) {
        String texto = "abc123";

        System.out.println(texto.matches("\\d+"));
        System.out.println(texto.matches("\\w+"));

        Pattern p = Pattern.compile("\\d+");
        Matcher m = p.matcher(texto);
        while (m.find()) {
            System.out.println("Número: " + m.group());
        }
    }
}
```

¿Qué imprime cada línea? ¿Por qué `matches("\\d+")` da `false` pero `find()` sí encuentra algo?

Ejercicio 8: Escribe este programa — contar palabras con split

Pide al usuario una frase por teclado y muestra cuántas palabras tiene, ignorando los espacios dobles.

Pista: `frase.split("\\s+")` trocea por "uno o más espacios". Cuidado con el `.trim()`.

Ejercicio 9: Escribe este programa — contar dígitos con Matcher

Escribe un programa que cuente cuántos **dígitos** hay en una frase. Por ejemplo, `"En 2026 hay 12 unidades"` → 6 dígitos (2, 0, 2, 6, 1 y 2).

Pista: compila `Pattern.compile("\\d")`, usa `matcher.find()` en un bucle y lleva un contador.

Boletín U14 — Inicial Resuelto

📝 Boletín U14 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: Encuentra el error — IOException sin capturar

> 🔄 Solución

Ejercicio 2: Completa el código — try-with-resources

> 🔄 Solución

Ejercicio 3: Escribe este programa — guardar ciudades en un archivo

> 🔄 Solución

Ejercicio 4: Encuentra el error — File.createNewFile sin comprobar

> 🔄 Solución

Ejercicio 5: ¿Qué imprime? — el contador de líneas

> 🔄 Solución

Ejercicio 6: Escribe este programa — el diario personal

> Solución

Ejercicio 7: ¿Qué imprime? — matches() vs find()

> Solución

Ejercicio 8: Escribe este programa — contar palabras con split

> Solución

Ejercicio 9: Escribe este programa — contar dígitos con Matcher

> Solución

Boletín U14 — Avanzado

📝 Boletín U14 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: Buscador de archivos por extensión

Crea un programa que pida una ruta de directorio y una extensión (ej: `.txt`, `.java`) y liste **recursivamente** todos los archivos con esa extensión. Usa la clase `File` y su método `listFiles()`.

Pista: si el archivo es un directorio, llama al método de nuevo (recursión). Recuerda comprobar `isDirectory()` antes de `listFiles()`.

⭐ Ejercicio 2: Lector de CSV con Scanner

Dado un archivo `datos.csv` con el siguiente formato (sin cabecera):

```
Ana;25;DAM
Bob;22;DAW
Carlos;30;DAM
```

Usa `Scanner` con `useDelimiter()` para leer el archivo y mostrar los datos en formato de tabla alineada con `printf()`.

Pista: `useDelimiter(";|\\R")` corta por `;` o por salto de línea. Repasa los formatos de `printf` en la UO3, punto 7.

⭐ ⭐ Ejercicio 3: Filtro de líneas por palabra clave

Crea un programa que lea un archivo de texto (`origen.txt`) y escriba en `destino.txt` solo las líneas que contienen una palabra clave (pedida al usuario). Usa `BufferedReader` y `PrintWriter`.

Muestra al final cuántas líneas coincidieron y cuántas se descartaron.

Pista: la comprobación es `linea.contains(palabraClave)`. Lleva dos contadores.

⭐ ⭐ Ejercicio 4: Separador de líneas pares e impares

Crea un programa que lea un archivo `entrada.txt` y genere dos archivos:

- `pares.txt` → contiene las líneas en posición par (0, 2, 4…).
- `impares.txt` → contiene las líneas en posición impar (1, 3, 5…).

Usa `try-with-resources` con **tres** recursos (un `BufferedReader` y dos `PrintWriter`).

Pista: los tres recursos van entre los paréntesis del `try`, separados por `;`. Usa `% 2` sobre el número de línea.

⭐ ⭐ Ejercicio 5: Split con regex — analizador de frases

Escribe un programa que lea una frase del usuario y use `split()` con una expresión regular para:

1. Separar las palabras (ignorando espacios, comas, puntos y signos).
2. Mostrar cuántas palabras hay.
3. Mostrar la palabra más larga.
4. Mostrar las palabras que empiezan por vocal.

Ejemplo: `"Hola, mundo. Esto es Java: ¿mola?"` →

```
Palabras: 6
Más larga: "mundo"
Empiezan por vocal: ["Esto"]
```

Pista: el separador que ignora todo lo que no sea letra es `"[^a-zA-ZáéíóúüñÑ]+"`. Para las vocales, comprueba la primera letra con `matches("[aeiouAEIOUáéíóú]")` o con un `indexOf` sobre una cadena de vocales.

⭐ ⭐ ⭐ Ejercicio 6: Validador de datos con regex

Crea un programa que lea un archivo `datos.txt` donde cada línea contiene un dato y su tipo (separados por `;`):

```
ana@email.com;email
12345678Z;dni
+34 612345678;telefono
91 123 45 67;telefono
esto-no-es-email;email
```

Valida cada línea según el tipo usando expresiones regulares:

- **Correo:** formato básico `xxx@xxx.xxx`.
- **DNI:** 8 dígitos + letra mayúscula (la letra debe ser válida según el algoritmo módulo 23).
- **Teléfono:** opcional `+34` seguido de 9 dígitos, con o sin espacios.

Muestra un resumen: cuántos válidos, cuántos inválidos, y lista los inválidos.

Pista: para cada línea, haz `linea.split(";")`, mira el tipo con `equals` y aplica el patrón correspondiente con `matches()`.

⭐ ⭐ ⭐ Ejercicio 7: Cifrado César con archivos

Crea un programa que lea un archivo `mensaje.txt`, desplace cada carácter **3 posiciones** en el alfabeto (cifrado César) y escriba el resultado en `mensaje_cifrado.txt`. Luego, otro programa (o el mismo con una opción) que lo descifre. Usa `try-with-resources` y `BufferedReader/PrintWriter`.

Pista: por cada `char`, si es letra haz `(char) (c + 3)` y cuidado con los extremos (la `z` debe volver a la `a`: usa `% 26` sobre la posición en el alfabeto).

⭐ ⭐ ⭐ Ejercicio 8: Serialización de estudiantes

Crea una clase `Estudiante` que implemente `Serializable` con `String nombre`, `int edad` y `double notaMedia`. Crea un programa que guarde un `ArrayList<Estudiante>` en un archivo `estudiantes.dat` usando `ObjectOutputStream`. Luego, otro programa (o el mismo con una opción) que lo lea con `ObjectInputStream` y muestre los datos formateados.

Pista: acuérdate del `serialVersionUID`. El `readObject()` devuelve `Object`: haz el casting a `List<Estudiante>` con calma y comprueba que no sea `null`.

⭐ ⭐ Ejercicio 9: El contador de líneas, palabras y caracteres

Crea un programa que lea un archivo de texto y muestre cuántas líneas, palabras y caracteres tiene. Usa `BufferedReader` para leer.

Pista: cada línea suma 1 al contador de líneas y `linea.length()` al de caracteres; para las palabras, `linea.split("\\s+").length` (con cuidado con las líneas vacías).

Reto extra: resuélvelo también con NIO (`Files.readAllLines`) y compara la diferencia.

Boletín U14 — Avanzado Resuelto

📝 Boletín U14 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

⭐ Ejercicio 1: Buscador de archivos por extensión

> 🔄 Solución

⭐ Ejercicio 2: Lector de CSV con Scanner

> 🔄 Solución

⭐ ⭐ Ejercicio 3: Filtro de líneas por palabra clave

> 🔄 Solución

⭐ ⭐ Ejercicio 4: Separador de líneas pares e impares

> 🔄 Solución

⭐ ⭐ Ejercicio 5: Split con regex — analizador de frases

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 6: Validador de datos con regex

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 7: Cifrado César con archivos

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 8: Serialización de estudiantes

> 🔄 Solución

⭐ ⭐ Ejercicio 9: El contador de líneas, palabras y caracteres

> 🔄 Solución

Boletín U14 — Extras

📝 Boletín U14 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. Las soluciones están ocultas: agota tu pista antes de mirarlas.

CodeWars

1. Regex validate PIN code

Crea una función que valide un PIN: un `String` que es válido solo si tiene **4 o 6 dígitos exactos**.

Ejemplo: `"1234"` → `true`, `"12345"` → `false`, `"a234"` → `false`, `"123456"` → `true`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

2. Two to One

Te dan dos strings en minúsculas con letras de la `a` a la `z`. Devuelve un único string con las letras **distintas de ambos, ordenadas alfabéticamente**.

Ejemplo: `"xyaabbbccccdefww"` y `"xxxxyyyyabklmopq"` → `"abcdefklmopqwxy"`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

3. Categorize New Member

El club de tenis clasifica a sus nuevos socios. Un socio es **"Senior"** si tiene 55 años o más **y** un hándicap mayor que 7; si no, es **"Open"**. Te dan un array de pares `[edad, hándicap]` y debes devolver un array con la categoría de cada uno.

Ejemplo: `[[45, 12], [55, 21], [19, -2], [104, 20]]` → `["Open", "Senior", "Open", "Senior"]`.

- Enunciado en CodeWars
- Dificultad: 7 kyu

> 💡 **Pista**

> 🔁 **Solución**

---

4. Primes in numbers

Dado un número positivo `n`, devuelve su descomposición en factores primos con el formato `"(p1**exp1)(p2**exp2)"`. Si el exponente es 1, se escribe solo `"(p)"`.

Ejemplo: `86240` → `"(2**5)(5)(7**2)(11)"`.

- Enunciado en CodeWars
- Dificultad: 5 kyu

> 💡 **Pista**

> 🔁 **Solución**

AceptaElReto

5. 108 — Hormigas

Sobre una barra de longitud `L` cm hay `n` hormigas. Cada hormiga se mueve a 1 cm/s hacia el extremo que le toque (el enunciado te da las posiciones). Cuando dos hormigas se cruzan, ambas cambian de sentido. Calcula, para cada caso de prueba, el **tiempo mínimo** y el **tiempo máximo** que tardan en caerse todas de la barra.

Entrada: varios casos de prueba. Cada caso empieza con `L` y `n`, seguido de las `n` posiciones de las hormigas.

- Enunciado en AceptaElReto
- Dificultad: ⭐ ⭐

> 💡 **Pista**

> 🔁 **Solución**

---

6. 140 — Suma de dígitos

Dado un número positivo, calcula la suma de sus dígitos. La entrada es una secuencia de números que termina con `-1`.

Ejemplo: `123` → `123: 6`, `55` → `55: 10`, `1000` → `1000: 1`.

- Enunciado en AceptaElReto
- Dificultad: ⭐

> 💡 **Pista**

> 🔁 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines los ficheros y las regex, vuelve a los problemas de unidades anteriores y resuélvelos leyendo los datos desde un archivo en vez de pedirlos por teclado, o validando la entrada con un `matches()`. El material no se pierde: se reutiliza.

Boletín U15 — Inicial

📝 Boletín U15 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, añade la dependencia de SQLite y haz que tu primer `SELECT` deje de parecer magia. Conectar es el 90% de la batalla; el resto es cuestión de práctica. Empieza suave, que las bases de datos muerden poco a poco.

Ejercicio 1: ¿Qué necesitas para usar JDBC?

Responde brevemente:

1. ¿Qué dependencia Maven (grupo y artefacto) necesitas para conectar Java con SQLite?
2. ¿Qué clase de Java proporciona el método `getConnection()`?
3. ¿Qué interfaz representa la conexión abierta a la base de datos?
4. ¿Qué excepción *checked* tienes que manejar siempre al trabajar con JDBC?

Ejercicio 2: Completa el código — la conexión

Completa los tipos que faltan:

```
String url = "jdbc:sqlite:instituto.db";

String sql = "SELECT * FROM alumnos";

try (______ con = DriverManager.getConnection(url);
     ______ stmt = con.createStatement();
     ______ rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        System.out.println(rs.getString("nombre"));
    }

} catch (______ e) {  // ¿qué excepción?
    System.err.println("Error: " + e.getMessage());
}
```

¿Qué tres interfaces van en los huecos? ¿Y la excepción?

Ejercicio 3: ¿Qué imprime? — ResultSet vacío

```
try (Connection con = DriverManager.getConnection(url);
     Statement stmt = con.createStatement();
     ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos WHERE id = 9999")) {

    if (rs.next()) {
        System.out.println("Encontrado: " + rs.getString("nombre"));
    } else {
        System.out.println("No encontrado");
    }
}
```

Si no hay ningún alumno con `id = 9999`, ¿qué imprime? ¿Qué devuelve `rs.next()` la primera vez que se llama?

Ejercicio 4: Encuentra el error — SQLException sin manejo

```
public class Test {
    public static void main(String[] args) {
        Connection con = DriverManager.getConnection("jdbc:sqlite:instituto.db");
        Statement stmt = con.createStatement();
        ResultSet rs = stmt.executeQuery("SELECT * FROM alumnos");
        while (rs.next()) {
            System.out.println(rs.getString("nombre"));
        }
    }
}
```

Este código **no compila**. ¿Por qué? ¿Qué dos cosas faltan para que funcione?

Ejercicio 5: Escribe este programa — la primera conexión

Crea una clase `TestConexion` que:

1. Se conecte a una base de datos SQLite `test.db`.
2. Cree una tabla `alumnos(id INTEGER PRIMARY KEY, nombre TEXT, nota REAL)`.
3. Inserte 3 alumnos.
4. Muestre «Conexión y tabla creadas» si todo ha ido bien.

Pista: usa `try-with-resources`, un `Statement` para el `CREATE TABLE` y `executeUpdate()` para insertar. SQLite crea el archivo solo.

Ejercicio 6: ¿Qué imprime? — executeQuery en UPDATE

```
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery("UPDATE alumnos SET nota = 10 WHERE id = 1");
```

¿Qué ocurre al ejecutar esta línea? ¿Por qué no debes usar `executeQuery()` para un `UPDATE`?

Ejercicio 7: Completa el código — INSERT con PreparedStatement

Completa los huecos para insertar un alumno de forma segura:

```
String sql = "INSERT INTO alumnos (nombre, nota) VALUES (?, ?)";

try (Connection con = DriverManager.getConnection(url);
     PreparedStatement pstmt = con.prepareStatement(sql)) {

    pstmt._____(1, "Ana");
    pstmt._____(2, 8.5);

    int filas = pstmt._____();
    System.out.println("Insertadas " + filas + " fila(s)");
}
```

¿Qué métodos van en los huecos? ¿En qué número empiezan los índices de los `?`?

Ejercicio 8: Escribe este programa — listar con try-with-resources

Escribe un método `listarAlumnos()` que:

1. Se conecte a la base de datos con `try-with-resources`.
2. Ejecute `SELECT * FROM alumnos`.
3. Muestre cada alumno con `printf("%d - %s (%.2f)", id, nombre, nota)`.
4. Maneje `SQLException` mostrando `e.getMessage()`.

Pista: `Connection`, `Statement` y `ResultSet` se cierran solos dentro del `try`. Recorre con `while (rs.next())`.

Ejercicio 9: Encuentra el error — índices del PreparedStatement

```
String sql = "INSERT INTO alumnos (nombre, nota, curso) VALUES (?, ?, ?)";
PreparedStatement pstmt = con.prepareStatement(sql);
pstmt.setString(0, "Ana");    // ¿índice correcto?
pstmt.setDouble(1, 8.5);      // ¿índice correcto?
pstmt.setString(2, "DAM");    // ¿índice correcto?
pstmt.executeUpdate();
```

¿Qué índices son correctos para los `?` de un `PreparedStatement`? ¿En qué número empiezan y qué pasa si usas `0`?

Boletín U15 — Inicial Resuelto

📝 Boletín U15 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: ¿Qué necesitas para usar JDBC?

> 🔄 Solución

Ejercicio 2: Completa el código — la conexión

> 🔄 Solución

Ejercicio 3: ¿Qué imprime? — ResultSet vacío

> 🔄 Solución

Ejercicio 4: Encuentra el error — SQLException sin manejo

> 🔄 Solución

Ejercicio 5: Escribe este programa — la primera conexión

> 🔄 Solución

Ejercicio 6: ¿Qué imprime? — executeQuery en UPDATE

> Solución

Ejercicio 7: Completa el código — INSERT con PreparedStatement

> Solución

Ejercicio 8: Escribe este programa — listar con try-with-resources

> Solución

Ejercicio 9: Encuentra el error — índices del PreparedStatement

> Solución

Boletín U15 — Avanzado

📝 Boletín U15 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐ ⭐ para pensar, ⭐ ⭐ ⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: Conexión desde archivo de propiedades

Crea un archivo `db.properties` con los datos de conexión:

```
url=jdbc:sqlite:instituto.db
```

Escribe un programa que lea este archivo usando la clase `Properties` y establezca la conexión. Si el archivo no existe o falta la propiedad `url`, muestra un mensaje de error claro.

Pista: carga el archivo con `props.load(Files.newInputStream(Path.of("db.properties")))` y usa `props.getProperty("url")`. `load` lanza una `IOException` (la viste en la U14 con los ficheros) además de la `SQLException`.

⭐ Ejercicio 2: INSERT con clave autogenerada

Inserta un nuevo alumno en la tabla `alumnos` y **recupera el ID** que la base de datos le ha asignado automáticamente (es un `AUTOINCREMENT`). Usa `PreparedStatement` con `Statement.RETURN_GENERATED_KEYS` y el método `getGeneratedKeys()`.

Pista:

```
PreparedStatement pstmt = con.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS);
pstmt.executeUpdate();
ResultSet claves = pstmt.getGeneratedKeys();
if (claves.next()) {
    int id = claves.getInt(1);
}
```

El `getGeneratedKeys()` devuelve un `ResultSet` con la clave que acaba de generar la base de datos. Se lee con `next()` y `getInt(1)`.

⭐ Ejercicio 3: UPDATE condicional

Actualiza el curso de todos los alumnos que tengan una edad superior a un valor dado. Por ejemplo:

```
¿Edad mínima? 25
¿Nuevo curso? DAM2
```

Todos los alumnos mayores de 25 años pasan al curso «DAM2». Muestra cuántas filas se actualizaron.

Pista: `UPDATE alumnos SET curso = ? WHERE edad > ?` con `setInt` y `setString`. `executeUpdate()` te devuelve el número de filas afectadas: es tu mejor amigo para confirmar que algo cambió.

⭐ ⭐ Ejercicio 4: INNER JOIN con PreparedStatement

Dada una tabla `matriculas` con `id_alumno`, `asignatura`, `nota`, escribe un programa que reciba un nombre de alumno y muestre todas sus asignaturas y notas. Usa un `INNER JOIN` entre `alumnos` y `matriculas`.

Ejemplo de salida:

```
Alumno: Ana García
  Matemáticas: 8.5
  Programación: 9.0
  Bases de Datos: 7.5
```

Pista: el `?` va en la parte del nombre: `SELECT a.nombre, m.asignatura, m.nota FROM alumnos a INNER JOIN matriculas m ON m.id_alumno = a.id WHERE a.nombre = ?`. El `JOIN` relaciona las dos tablas en una sola consulta: ni un bucle, ni consultas dentro de bucles.

⭐ ⭐ Ejercicio 5: Búsqueda con LIKE

Implementa una búsqueda de alumnos por nombre usando `LIKE` y `PreparedStatement`. El usuario escribe una parte del nombre y se muestran todos los que coinciden. Si no hay resultados, muestra «Sin resultados».

Pista: `SELECT * FROM alumnos WHERE nombre LIKE ?` con `pstmt.setString(1, "%" + texto + "%")`. Los `%` son comodines y van dentro del **valor**, no en el SQL. El `%texto%` busca el texto en cualquier posición.

⭐ ⭐ Ejercicio 6: Fechas en JDBC

Añade una columna `fecha_nacimiento DATE` a la tabla `alumnos` (asume que ya existe). Crea un programa que:

1. Pida nombre, edad, curso y fecha de nacimiento (formato `YYYY-MM-DD`).
2. Inserte el alumno usando `PreparedStatement` con `java.sql.Date.valueOf()`.
3. Liste todos los alumnos mostrando también su fecha de nacimiento.

Pista: `Date.valueOf("2000-03-15")` convierte el texto en `java.sql.Date` (¡ojo: es `java.sql.Date`, no `java.util.Date`!). Para leerla, `rs.getDate("fecha_nacimiento")`. Recuerda comprobar el valor que devuelve `executeUpdate()`.

⭐ ⭐ Ejercicio 7: Batch INSERT — 100 alumnos de prueba

Crea un programa que inserte **100 alumnos de prueba** en la tabla `alumnos` usando lotes (batch). Los nombres pueden ser genéricos: `Alumno1`, `Alumno2`, etc.

Usa `addBatch()` y `executeBatch()` de `PreparedStatement`. Mide el tiempo que tarda con `System.currentTimeMillis()`.

Pista: en el bucle, haces `addBatch()` en cada vuelta y una sola `executeBatch()` al final (o cada 50). Para medir: `long inicio = System.currentTimeMillis(); … long fin = System.currentTimeMillis();` y restas. Compara mentalmente con 100 `executeUpdate()` sueltos.

⭐ ⭐ ⭐ Ejercicio 8: El patrón DAO

Implementa el patrón **Data Access Object (DAO)** para la tabla `alumnos`. Crea las siguientes clases:

1. `Alumno` — clase modelo con `id`, `nombre`, `edad`, `curso`.
2. `AlumnoDAO` — interfaz con métodos: `listar()`, `buscarPorId(int id)`, `buscarPorNombre(String nombre)`, `insertar(Alumno a)`, `actualizar(Alumno a)`, `eliminar(int id)`.
3. `AlumnoDAOImpl` — implementación concreta con JDBC y SQLite.
4. `Main` — programa con menú que use el DAO.

Pista: la URL (`jdbc:sqlite:instituto.db`) es una constante privada de la implementación. Cada método abre su propia conexión con `try-with-resources`. El `Main` solo habla con la interfaz `AlumnoDAO`; el SQL no le importa.

⭐ ⭐ ⭐ Ejercicio 9: Transacción bancaria atómica

Simula una transferencia entre dos cuentas en una tabla `cuentas(id, titular, saldo)`. La transferencia debe ser **atómica**: quita 100 € de una cuenta, ponlos en la otra, y si falla cualquier paso, haz `rollback()` para que no quede el sistema a medias.

Ejemplo de salida:

```
Saldo antes: Ana 500, Luis 300
Transferencia OK
Saldo después: Ana 400, Luis 400
```

Pista: `con.setAutoCommit(false)`, después las dos operaciones con `PreparedStatement`, y al final `con.commit()`. El `rollback()` va en el `catch (SQLException e)` interno. Prueba a forzar el fallo (por ejemplo, una cuenta inexistente) y comprueba que el saldo de Ana no cambia.

Boletín U15 — Avanzado Resuelto

📝 Boletín U15 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

⭐ Ejercicio 1: Conexión desde archivo de propiedades

> 🔄 Solución

⭐ Ejercicio 2: INSERT con clave autogenerada

> 🔄 Solución

⭐ Ejercicio 3: UPDATE condicional

> 🔄 Solución

⭐ ⭐ Ejercicio 4: INNER JOIN con PreparedStatement

> 🔄 Solución

⭐ ⭐ Ejercicio 5: Búsqueda con LIKE

> 🔄 Solución

⭐ ⭐ Ejercicio 6: Fechas en JDBC

> 🔄 Solución

⭐ ⭐ Ejercicio 7: Batch INSERT — 100 alumnos de prueba

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 8: El patrón DAO

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 9: Transacción bancaria atómica

> 🔄 Solución

Boletín U15 — Extras

📝 Boletín U15 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas. Las soluciones están ocultas: agota tu pista antes de mirarlas.

CodeWars

1. SQL Basics: Simple JOIN

Crea una consulta `SELECT` que devuelva todos los productos junto con la empresa que los fabrica. Tendrás que relacionar las tablas `products` y `companies` mediante la clave foránea `company_id`.

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

2. SQL with Street Fighter: Total Wins

Es hora de decidir qué luchadores pasan a las semifinales del campeonato mundial de Street Fighter. Cada combate registra si el luchador ganó (1) o perdió (0), y el movimiento con el que terminó. Como los ataques ki han sido prohibidos, **no se cuentan** los combates terminados con Hadoken, Shouoken o Kikoken. Devuelve `name`, `won` y `lost` sumando las victorias y derrotas, ordena de más a menos victorias y devuelve los 6 mejores.

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

3. SQL Basics: Simple HAVING

Tienes una tabla `people` con `id`, `name` y `age`. Cuenta cuántas personas tienen la misma edad y devuelve **solo los grupos de edad con 10 o más personas**.

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

4. SQL Basics: Group By Day

Tienes una tabla `orders` con `id`, `datetime` y `amount`. Cuenta cuántos pedidos hay **por día**, extrayendo la fecha del campo `datetime`.

- Enunciado en CodeWars
- Dificultad: 5 kyu

> 💡 **Pista**

> 🔄 **Solución**

AceptaElReto

5. 245 — ¿Quién gana la partida?

Un grupo de jugadores participa en un juego por turnos con números. Cada ronda, el jugador que acierta se descarta y el que falla pasa al **final de la cola** para volver a intentarlo. Simula las rondas y determina quién gana la partida.

- Enunciado en AceptaElReto
- Dificultad: ⭐ ⭐ ⭐

> 💡 **Pista**

> 🔄 **Solución**

---

6. 424 — Billetes de autobús

Hay varias rutas de autobús entre dos ciudades, cada una con su hora de salida y de llegada. Quieres coger **el máximo número de autobuses posible** sin que se solapen (coger uno, bajarte, y poder subir a otro que salga después de llegar).

- Enunciado en AceptaElReto
- Dificultad: ⭐ ⭐

> › 💡 **Pista**

> › 🔄 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines el JDBC, vuelve a los problemas de AceptaElReto de unidades anteriores y reescríbelos guardando los datos de entrada en una tabla SQLite con `PreparedStatement`: te sorprenderá lo natural que resulta que tus algoritmos hablen con una base de datos. El material no se pierde: se reutiliza.

Boletín U15 — Inicial

📝 Boletín U15 — Inicial

> Sin soluciones. Sin prisas. Abre el IDE, crea tu primer `HttpServer` y haz que el navegador te hable de tú a tú. Prometo que ningún bit saldrá herido. Empieza suave, que la web muerde poco a poco.

Ejercicio 1: Escribe este programa — Hola, mundo web

Crea un servidor HTTP en el puerto 8080 con una ruta `/` que devuelva el texto `Hola, mundo web!`. Abre `http://localhost:8080` en tu navegador y comprueba que lo ves.

Pista: `HttpServer.create(new InetSocketAddress(8080), 0)`, después `createContext("/", ...)` y `sendResponseHeaders(200, ...)`. No olvides cerrar el cuerpo de la respuesta.

Ejercicio 2: Escribe este programa — servidor de la hora

Añade a tu servidor una ruta `/hora` que devuelva la hora actual en texto plano con formato `HH:mm:ss`.

Cada vez que recargues el navegador, la hora cambia. Magia negra con `LocalTime.now()`.

```
// Pista:
String hora = java.time.LocalTime.now().format(
    java.time.format.DateTimeFormatter.ofPattern("HH:mm:ss")
);
```

Ejercicio 3: Escribe este programa — página que dice tu nombre

Añade una ruta `/saludo?nombre=Pepe` que devuelva una página HTML con `<h1>¡Hola, Pepe!</h1>`.

Si no se pasa nombre, que diga `¡Hola, desconocido!`.

Pista: `e.getRequestURI().getQuery()` te da `nombre=Pepe`. Divide por `=` y listo.

Ejercicio 4: Escribe este programa — contador de visitas global

Usa una variable `static int visitas = 0`. Cada vez que alguien visita `/`, incrementa el contador y devuelve:

```
Eres el visitante número 47
```

¿Qué pasa si dos personas recargan a la vez? (spoiler: problemas — pero de momento no te preocupes).

Ejercicio 5: Escribe este programa — generador de excusas para entregas tarde

Crea una ruta `/excusa` que devuelva una excusa generada aleatoriamente combinando elementos de tres arrays:

```
String[] sujetos = {"Mi perro", "GitHub", "El plugin de IntelliJ", "La conexión"};
String[] verbos  = {"se comió", "borró", "corrompió", "perdió"};
String[] objetos = {"el examen", "la práctica", "los apuntes", "mi paciencia"};
```

Ejemplo: *“GitHub borró la práctica”*. Devuélvelo en HTML con buena letra.

Ejercicio 6: Escribe este programa — tabla de multiplicar personalizada

Ruta `/tabla?num=7` que genere una tabla HTML completa con la tabla de multiplicar del 7 (del 7×1 al 7×10).

Si no se pasa número, usa el 5 por defecto. Formatea la tabla con `border=1` y colores alternos en las filas.

Ejercicio 7: Escribe este programa — conversor de euros a pesetas (sí, pesetas)

Ruta `/conversor?euros=50` que devuelva HTML con el resultado: "50 euros son 8319.3 pesetas".

1 € = 166.386 pts. Muestra dos decimales.

Pista: `String.format("%.2f", valor)` para los decimales. Sí, pesetas. Si eres joven, pregúntale a un viejo qué era eso.

Ejercicio 8: Escribe este programa — página de estado del servidor

Ruta `/estado` que devuelva un JSON con esta pinta:

```
{
    "servidor": "ok",
    "hora": "14:30:01",
    "visitas": 47,
    "version": "1.0",
    "autor": "Tu nombre aquí"
}
```

No olvides el `Content-Type: application/json` o el navegador se pondrá tonto.

Ejercicio 9: Encuentra el error — el Content-Type olvidado

Un compañero te enseña este handler y jura que devuelve JSON:

```
server.createContext("/api/frase", intercambio -> {
    String json = """
        {"frase": "El código limpio es como un buen chiste"}
        """;
    intercambio.sendResponseHeaders(200, json.getBytes().length);
    intercambio.getResponseBody().write(json.getBytes());
    intercambio.getResponseBody().close();
});
```

El navegador muestra el texto, pero el `fetch` que lo consume se queja: `r.json()` peta. ¿Qué falta y por qué es tan importante?

Boletín U15 — Inicial Resuelto

📝 Boletín U15 — Inicial (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

Ejercicio 1: Escribe este programa — Hola, mundo web

> 🔄 Solución

Ejercicio 2: Escribe este programa — servidor de la hora

> 🔄 Solución

Ejercicio 3: Escribe este programa — página que dice tu nombre

> 🔄 Solución

Ejercicio 4: Escribe este programa — contador de visitas global

> 🔄 Solución

Ejercicio 5: Escribe este programa — generador de excusas para entregas tarde

> 🔄 Solución

Ejercicio 6: Escribe este programa — tabla de multiplicar personalizada

> Solución

Ejercicio 7: Escribe este programa — conversor de euros a pesetas

> Solución

Ejercicio 8: Escribe este programa — página de estado del servidor

> Solución

Ejercicio 9: Encuentra el error — el Content-Type olvidado

> Solución

Boletín U15 — Avanzado

📝 Boletín U15 — Avanzado

> Dificultad progresiva. ⭐ para calentar, ⭐⭐ para pensar, ⭐⭐⭐ para concursar. Cada ejercicio incluye una pista (resiste a mirarla).

⭐ Ejercicio 1: API de frases motivacionales

Crea un endpoint `GET /api/frase` que devuelva un JSON con una frase aleatoria de un array precargado y su autor.

```
{"frase": "El código limpio es como un buen chiste: si tienes que explicarlo, es malo", "autor": "Alguien que sabe"}
```

El frontend es un HTML con un botón "Nueva frase" que al hacer clic hace `fetch('/api/frase')` y muestra la frase en pantalla.

Pista: usa `Math.random()` para elegir un índice aleatorio del array, o `Random.nextInt(longitud)`.

⭐ Ejercicio 2: Formulario de contacto con POST

Crea una ruta `/contacto` que sirva un formulario HTML (GET) con campos `nombre` y `mensaje`, y una ruta `/enviar` que reciba los datos por POST y los muestre en una página de confirmación.

Pista: en el handler de `/enviar` comprueba `"POST".equals(e.getRequestMethod())` y lee el cuerpo con `e.getRequestBody().readAllBytes()`. El formato del cuerpo es `nombre=Ana&mensaje=Hola`.

⭐ Ejercicio 3: Piedra, papel, tijera online

Endpoint `POST /api/jugar` que recibe:

```
{"jugada": "piedra"}
```

Y devuelve:

```
{"jugadaPC": "tijera", "resultado": "ganaste"}
```

Reglas clásicas: piedra > tijera, tijera > papel, papel > piedra.

Frontend: tres botones con emojis 🪨 📄 ✂️. Al hacer clic, envía la jugada y muestra el resultado.

Pista: la jugada del PC se elige con `Random`. Las reglas se pueden implementar con un `Map<String, String>` donde la clave vence al valor: `{"piedra": "tijera", "tijera": "papel", "papel": "piedra"}`.

⭐ ⭐ Ejercicio 4: El tiempo que NO hace

Crea `GET /api/clima?ciudad=Madrid` que devuelva un JSON con datos meteorológicos **aleatorios** (generados cada vez):

```
{"ciudad": "Madrid", "temperatura": 28, "humedad": 45, "estado": "soleado"}
```

Estados posibles: `"soleado"`, `"nublado"`, `"lluvia"`, `"tormenta"`. Frontend con emojis y temperaturas de colores.

Pista: usa `String[] estados = {...}` y elige aleatoriamente. La temperatura puede ser `random.nextInt(40) - 5`.

⭐ ⭐ Ejercicio 5: Traductor chungo (pero funcional)

Implementa un endpoint `POST /api/traducir` que reciba:

```
{"texto": "hola", "idioma": "en"}
```

Y devuelva:

```
{"traduccion": "hello"}
```

Usa un `HashMap<String, HashMap<String, String>>` como diccionario. Mete al menos 10 palabras en español traducidas a inglés y francés.

Pista: inicializa el diccionario con bloques `static`. `diccionario.get("hola").get("en")` te da `"hello"`. Los mapas los viste en la U13.

⭐ ⭐ Ejercicio 6: API REST de tareas con prioridad

Implementa un CRUD completo de tareas donde cada tarea tiene: `id`, `titulo`, `prioridad` (`"ALTA"`, `"MEDIA"`, `"BAJA"`).

Método	Ruta	Descripción
GET	`/api/tareas`	Lista todas
POST	`/api/tareas`	Crea una (JSON: `{"titulo": "...", "prioridad": "ALTA"}`)
PUT	`/api/tareas/{id}`	Cambia prioridad (JSON: `{"prioridad": "BAJA"}`)
DELETE	`/api/tareas/{id}`	Borra una

Frontend: tabla con colores de fondo según prioridad (rojo ALTA, amarillo MEDIA, verde BAJA). Botones para crear, cambiar prioridad y borrar.

Pista: guarda las tareas en un `ConcurrentHashMap<Integer, Tarea>` con un `AtomicInteger` para los IDs. Para el path param, parsea la ruta con `substring`.

⭐ ⭐ Ejercicio 7: Cliente GET — los repos de GitHub

Usa `HttpClient` para consultar la API de GitHub (`https://api.github.com/users/{usuario}/repos`) y mostrar solo el **nombre** y el **lenguaje** de cada repositorio de un usuario (que se pide por teclado). Luego guarda los resultados en un archivo `repos.txt`.

Pista: parsea la respuesta con Gson (`JsonArray`), recórrela, y escribe con `Files.writeString` (los ficheros los viste en la U14). Recuerda la cabecera `User-Agent`, que GitHub exige.

⭐ ⭐ Ejercicio 8: Cliente POST — crear una publicación en jsonplaceholder

Usa `HttpClient` para hacer un `POST` a `https://jsonplaceholder.typicode.com/posts` con un cuerpo JSON:

```
{"title": "Mi primera API", "body": "Consumida desde Java", "userId": 1}
```

Muestra el código de estado y el cuerpo de la respuesta. Comprueba que la cabecera `Content-Type: application/json` está puesta.

Pista: `HttpRequest.newBuilder().uri(...).header("Content-Type", "application/json").POST(HttpRequest.BodyPublishers.ofString(json)).build()`.

⭐ ⭐ ⭐ Ejercicio 9: Middleware de logging

Crea una clase `LoggerMiddleware` que envuelva cualquier `HttpHandler` y registre en consola:

```
[2026-06-21 14:30:01] GET /api/peliculas → 200 (15ms)
[2026-06-21 14:30:05] POST /api/tareas → 201 (3ms)
```

Debe poder aplicarse a cualquier handler así:

```
server.createContext("/api", new LoggerMiddleware(new TareasHandler()));
```

Pista: guarda `System.currentTimeMillis()` antes y después de llamar al handler original. Usa `e.getRequestMethod()`, `e.getRequestURI()` y `e.getResponseCode()` (tras enviar cabeceras)

⭐ ⭐ ⭐ Ejercicio 10: la petición asíncrona con sendAsync

Usa `HttpClient` para pedir datos a la API pública de GitHub **sin bloquear el hilo principal** con `sendAsync()`. Descarga los repos de un usuario y, cuando la respuesta llegue, imprime el número de repos y el código de estado.

```
// Esqueleto para completar
HttpClient cliente = HttpClient.newHttpClient();
HttpRequest peticion = HttpRequest.newBuilder()
        .uri(URI.create("https://api.github.com/users/google/repos"))
        .GET()
        .build();
```

Completa el programa para que:

- Use `sendAsync(peticion, HttpResponse.BodyHandlers.ofString())`.
- Encadene `.thenAccept(...)` para procesar la respuesta cuando llegue (imprime `statusCode()` y, si es 200, cuenta las apariciones de `"full_name"` en el JSON con `split`).
- Añada un `System.out.println("Petición lanzada, seguimos trabajando...");` ANTES de que llegue la respuesta, para demostrar que el hilo no se bloqueó.

Pista: `sendAsync` devuelve un `CompletableFuture<HttpResponse<String>>`. El `thenAccept` recibe la respuesta cuando esté lista, pero el `main` sigue corriendo mientras tanto. Para que el programa no termine antes de que llegue la respuesta, espera al final con `.join()` sobre el `CompletableFuture`. Sin el `join()`, el `main` se acaba y la petición se pierde.

> 🔄 **Solución**

.

Boletín U15 — Avanzado Resuelto

📝 Boletín U15 — Avanzado (Resuelto)

> Las soluciones están ocultas en cada ejercicio. No hagas trampa: primero inténtalo de verdad.

⭐ Ejercicio 1: API de frases motivacionales

> 🔄 Solución

⭐ Ejercicio 2: Formulario de contacto con POST

> 🔄 Solución

⭐ Ejercicio 3: Piedra, papel, tijera online

> 🔄 Solución

⭐ ⭐ Ejercicio 4: El tiempo que NO hace

> 🔄 Solución

⭐ ⭐ Ejercicio 5: Traductor chungo (pero funcional)

> 🔄 Solución

⭐ ⭐ Ejercicio 6: API REST de tareas con prioridad

> 🔄 Solución

⭐ ⭐ Ejercicio 7: Cliente GET — los repos de GitHub

> 🔄 Solución

⭐ ⭐ Ejercicio 8: Cliente POST — crear una publicación en jsonplaceholder

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 9: Middleware de logging

> 🔄 Solución

⭐ ⭐ ⭐ Ejercicio 10: la petición asíncrona con sendAsync

> 🔄 Solución

Boletín U15 — Extras

📝 Boletín U15 — Extras

> Ejercicios de CodeWars y AceptaElReto con pistas y soluciones. Las soluciones están ocultas: agota tu pista antes de mirarlas.

CodeWars

1. IP Validation

Te dan una cadena que dice ser una dirección IPv4. Escribe una función que devuelva `true` solo si es una IP válida: cuatro partes, cada una un número entre 0 y 255, sin ceros a la izquierda y sin caracteres extraños.

Ejemplos: `"1.2.3.4"` → `true`, `"123.045.067.089"` → `false` (ceros a la izquierda), `"12.34.56"` → `false` (faltan partes), `"a.b.c.d"` → `false`.

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

2. Simple URL parser

Te dan una URL como `"http://www.codewars.com/kata/56f8fe6a2e6c0dc83b0008a7?page=1"`. Escribe una función que la divida en **protocolo**, **dominio** y **ruta**.

Ejemplos: `"http://www.codewars.com/path"` → protocolo `http`, dominio `www.codewars.com`, ruta `/path`. `"https://example.com"` → protocolo `https`, dominio `example.com`, ruta vacía.

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔄 **Solución**

---

3. Extract the domain name from a URL

Te dan una URL completa y tienes que devolver solo el **nombre de dominio** (sin protocolo, sin `www.`, sin extensión).

Ejemplos: `"http://github.com/carbonfive/raygun"` → `"github"`, `"http://www.zombie-bites.com"` → `"zombie-bites"`, `"https://www.cnet.com"` → `"cnet"`.

- Enunciado en CodeWars
- Dificultad: 5 kyu

> 💡 **Pista**

> 🔁 **Solución**

---

4. Decode the Morse code

Te dan un mensaje en código Morse (letras separadas por un espacio, palabras por tres espacios). Escríbelo en texto legible.

Ejemplo: `"... --- ..."` → `"SOS"`, `".... . -.-- .--- ..- -.. ."` → `"HEY JUDE"`.

- Enunciado en CodeWars
- Dificultad: 6 kyu

> 💡 **Pista**

> 🔁 **Solución**

AceptaElReto

5. 396 — ¿Cuántos días faltan?

Se dan dos fechas y hay que decir **cuántos días hay entre ellas** (los días que faltan para la segunda desde la primera).

Entrada: varios casos de prueba. Cada caso trae dos fechas con el formato día, mes y año. La entrada termina cuando no quedan datos.

- Enunciado en AceptaElReto

- Dificultad: ⭐ ⭐

> 💡 **Pista**

> 🔁 **Solución**

---

6. 462 — Día de la semana

Te dan una fecha (día, mes y año) y tienes que decir **qué día de la semana es**.

Entrada: varios casos. Cada caso: una fecha en una línea con día, mes y año. La entrada termina con `0 0 0`.

- Enunciado en AceptaElReto
- Dificultad: ⭐ ⭐

> 💡 **Pista**

> 🔁 **Solución**

> 🧭 **¿Y si te quedas con ganas?** Cuando domines servidores y clientes HTTP, vuelve a los problemas de unidades anteriores y plantéalos como APIs: un problema que devolvía texto por consola puede devolver JSON por HTTP. El material no se pierde: se reutiliza.